

Паттерны проектирования API

Джей Джей Гивакс

Предисловие Джона Скита



API Design Patterns

JJ GEEWAX

FOREWORD BY JON SKEET



MANNING

SHELTER ISLAND

Паттерны проектирования API

Джей Джей Гивакс
Предисловие Джона Скита



Санкт-Петербург • Москва • Минск

2023

ББК 32.988.02-018
УДК 004.438.5
Г46

Гивакс Джей Джей

Г46 Паттерны проектирования API. — СПб.: Питер, 2023. — 512 с.: ил. — (Серия «Библиотека программиста»).

ISBN 978-5-4461-1984-4

API представляют собой контракты, которые определяют принципы взаимодействия приложений, сервисов и компонентов. Паттерны проектирования API — это набор лучших практик, спецификаций и стандартов, обеспечивающих простоту и надежность их использования для разработчиков. Книга объединяет и объясняет наиболее важные паттерны, используемые сообществом разработчиков API и экспертами Google.

Паттерны проектирования API определяют набор принципов для разработки внутренних и публичных API. Джей Джей Гивакс, будучи специалистом из Google, рассказывает о паттернах, которые обеспечат вашим API согласованность, масштабируемость и гибкость. Вы узнаете, как улучшить дизайн самых распространенных API и как действовать в сложных пограничных случаях. Понятные иллюстрации, актуальные примеры и подробные сценарии позволят тщательно разобраться в каждом паттерне.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.988.02-018
УДК 004.438.5

Права на издание получены по соглашению с Manning Publications. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 978-1617295850 англ.
ISBN 978-5-4461-1984-4

© 2021 by Manning Publications Co. All rights reserved
© Перевод на русский язык ООО «Прогресс книга», 2023
© Издание на русском языке, оформление ООО «Прогресс книга», 2023
© Серия «Библиотека программиста», 2023

Краткое содержание

Предисловие	20
Вступление	23
Благодарности	25
О книге.	27
Об авторе	34
Иллюстрация на обложке	35
От издательства	36

Часть I. Введение

Глава 1. Знакомство с API.	38
Глава 2. Знакомство с паттернами проектирования API.	51

Часть II. Принципы проектирования

Глава 3. Именованние	70
Глава 4. Состав и иерархия ресурсов	86
Глава 5. Типы данных и значения по умолчанию	106

Часть III. Основы

Глава 6. Идентификация ресурсов	130
Глава 7. Стандартные методы	150
Глава 8. Частичное обновление и извлечение	170

6 Краткое содержание

Глава 9. Пользовательские методы	191
Глава 10. Длительные операции	205
Глава 11. Перезапускаемые задачи	229

Часть IV. Связи ресурсов

Глава 12. Подресурсы-одиночки.	244
Глава 13. Перекрестные ссылки	256
Глава 14. Ассоциирующие ресурсы	264
Глава 15. Пользовательские методы Add и Remove.	276
Глава 16. Полиморфизм	284

Часть V. Коллективные операции

Глава 17. Копирование и перемещение	298
Глава 18. Пакетные операции	315
Глава 19. Удаление на основе критерия	333
Глава 20. Анонимная запись	343
Глава 21. Пагинация	351
Глава 22. Фильтрация	367
Глава 23. Импорт и экспорт.	382

Часть VI. Безопасность

Глава 24. Версионирование и совместимость.	404
Глава 25. Мягкое удаление	429
Глава 26. Повтор запросов	444
Глава 27. Валидация запросов	457
Глава 28. Ревизии ресурсов	465
Глава 29. Повтор запросов	482
Глава 30. Аутентификация запросов	494

Оглавление

Предисловие	20
Вступление	23
Благодарности	25
О книге.	27
Для кого эта книга	27
Структура издания	27
О коде	31
Онлайн-ресурсы.	33
Об авторе	34
Иллюстрация на обложке	35
От издательства	36

Часть I. Введение

Глава 1. Знакомство с API.	38
1.1. Что такое веб-API	38
1.2. Почему API важны.	40
1.3. Что значит ресурсно-ориентированные API.	41
1.4. Что делает API «хорошим»	44
1.4.1. Функциональность	45
1.4.2. Выразительность.	45
1.4.3. Простота	46
1.4.4. Предсказуемость	48
Резюме.	49

8 Оглавление

Глава 2. Знакомство с паттернами проектирования API.	51
2.1. Что такое паттерны проектирования API	51
2.2. Чем важны паттерны проектирования API	54
2.3. Анатомия паттерна проектирования API.	57
2.3.1. Имя и аннотация	57
2.3.2. Зачем он нужен.	57
2.3.3. Обзор	58
2.3.4. Реализация	59
2.3.5. Компромиссы	60
2.4. Практическое исследование: Twapi, Twitter-подобный API.	60
2.4.1. Обзор	61
2.4.2. Создание списка сообщений.	61
2.4.3. Экспорт данных	64
Резюме.	68

Часть II. Принципы проектирования

Глава 3. Именованное	70
3.1. Почему важны имена	71
3.2. Что делает имя «хорошим»	72
3.2.1. Выразительность.	72
3.2.2. Простота	72
3.2.3. Предсказуемость	73
3.3. Язык, грамматика и синтаксис	74
3.3.1. Язык.	75
3.3.2. Грамматика	75
3.3.3. Синтаксис	78
3.4. Контекст	79
3.5. Типы данных и единицы измерения.	80
3.6. Практическое исследование: что происходит при выборе плохих имен.	82
3.7. Упражнения	84
Резюме.	85
Глава 4. Состав и иерархия ресурсов	86
4.1. Что такое схема ресурсов.	87
4.1.1. Типы связей	88
4.1.2. Блок-схемы связей между сущностями.	91
4.2. Выбор правильных связей	93
4.2.1. Нужна ли вообще связь.	93
4.2.2. Ссылки или встроенные данные	95
4.2.3. Иерархия.	97

4.3. Антипаттерны	99
4.3.1. Ресурсы для всего	99
4.3.2. Глубокие иерархии	102
4.3.3. Встраивание всего	104
4.4. Упражнения	105
Резюме	105
Глава 5. Типы данных и значения по умолчанию	106
5.1. Знакомство с типами данных	107
5.1.1. Отсутствующее значение и значение null	108
5.2. Логический тип	109
5.3. Числа	111
5.3.1. Границы	112
5.3.2. Значения по умолчанию	113
5.3.3. Сериализация	114
5.4. Строки	115
5.4.1. Границы	116
5.4.2. Значения по умолчанию	117
5.4.3. Сериализация	118
5.5. Перечисления	119
5.6. Списки	120
5.6.1. Атомарность	121
5.6.2. Границы	122
5.6.3. Значения по умолчанию	123
5.7. Карты	123
5.7.1. Границы	126
5.7.2. Предустановленные значения	127
5.8. Упражнения	127
Резюме	127

Часть III. Основы

Глава 6. Идентификация ресурсов	130
6.1. Что такое идентификатор	130
6.2. Что делает идентификатор хорошим	131
6.2.1. Простота использования	131
6.2.2. Уникальность	132
6.2.3. Постоянство	132
6.2.4. Быстрая и легкая генерация	133
6.2.5. Непредсказуемость	133
6.2.6. Читательность, передача и возможность проверки	134
6.2.7. Информационная плотность	134

6.3. Как выглядит хороший идентификатор	135
6.3.1. Тип данных	135
6.3.2. Набор символов	135
6.3.3. Формат идентификатора.	136
6.3.4. Контрольные суммы.	137
6.3.5. Тип ресурса	138
6.3.6. Иерархия и область уникальности.	138
6.4. Реализация.	140
6.4.1. Размер	140
6.4.2. Генерация	141
6.4.3. Отметка об удалении (tombstoning).	143
6.4.4. Контрольная сумма	144
6.4.5. Хранилище базы данных.	145
6.5. UUID	147
6.6. Упражнения	148
Резюме.	149
Глава 7. Стандартные методы	150
7.1. Зачем это нужно	150
7.2. Обзор	151
7.3. Реализация.	152
7.3.1. Какие методы должны поддерживаться	153
7.3.2. Идемпотентность и побочные эффекты	154
7.3.3. Метод Get	155
7.3.4. Метод List	156
7.3.5. Метод Create	159
7.3.6. Метод Update	161
7.3.7. Метод Delete	162
7.3.8. Метод Replace.	164
7.3.9. Итоговое определение API.	166
7.4. Компромиссы	168
7.5. Упражнения	168
Резюме.	169
Глава 8. Частичное обновление и извлечение	170
8.1. Зачем это нужно	171
8.1.1. Частичное извлечение	171
8.1.2. Частичное обновление	171
8.2. Обзор	174
8.3. Реализация.	176
8.3.1. Передача	176

8.3.2. Карты и вложенные интерфейсы	178
8.3.3. Повторяющиеся поля	180
8.3.4. Значения по умолчанию	183
8.3.5. Неявные маски полей	184
8.3.6. Обновление динамических структур данных	185
8.3.7. Недействительные поля	187
8.3.8. Итоговое определение API	187
8.4. Компромиссы	188
8.4.1. Универсальная поддержка	188
8.4.2. Альтернативные реализации	189
8.5. Упражнения	190
Резюме	190
Глава 9. Пользовательские методы	191
9.1. Зачем это нужно	192
9.1.1. Почему бы просто не задействовать стандартные методы	192
9.2. Обзор	195
9.3. Реализация	196
9.3.1. Побочные эффекты	197
9.3.2. Ресурсы или коллекции	199
9.3.3. Пользовательские методы без сохранения состояния	200
9.3.4. Итоговое определение API	202
9.4. Компромиссы	203
9.5. Упражнения	204
Резюме	204
Глава 10. Длительные операции	205
10.1. Зачем это нужно	206
10.2. Обзор	207
10.3. Реализация	210
10.3.1. Как выглядит длительная операция	210
10.3.2. Иерархия ресурсов	212
10.3.3. Разрешение	212
10.3.4. Обработка ошибок	216
10.3.5. Отслеживание прогресса	218
10.3.6. Отмена операций	219
10.3.7. Приостановка и возобновление операций	221
10.3.8. Инспектирование операций	222
10.3.9. Продолжительность хранения	224
10.3.10. Итоговое определение API	225

12 Оглавление

10.4. Компромиссы	227
10.5. Упражнения	228
Резюме	228
Глава 11. Перезапускаемые задачи	229
11.1. Зачем это нужно	230
11.2. Обзор	231
11.3. Реализация	232
11.3.1. Ресурсы Job	232
11.3.2. Пользовательский метод Run	235
11.3.3. Ресурсы выполнения задачи	236
11.3.4. Итоговое определение API	239
11.4. Компромиссы	241
11.5. Упражнения	241
Резюме	242
 Часть IV. Связи ресурсов	
Глава 12. Подресурсы-одиночки.	244
12.1. Зачем это нужно	244
12.1.1. Почему нужно использовать подресурс-одиночку?	245
12.2. Обзор	246
12.3. Реализация	247
12.3.1. Стандартные методы	248
12.3.2. Сброс	251
12.3.3. Иерархия	251
12.3.4. Итоговое определение API	252
12.4. Компромиссы	253
12.4.1. Атомарность выполнения	254
12.4.2. Ровно один подресурс	254
12.5. Упражнения	254
Резюме	255
Глава 13. Перекрестные ссылки	256
13.1. Зачем это нужно	256
13.2. Обзор	257
13.3. Реализация	258
13.3.1. Имя ссылочного поля	258
13.3.2. Целостность данных	259
13.3.3. Значение или ссылка	260
13.3.4. Итоговое определение API	262
13.4. Компромиссы	263
13.5. Упражнения	263
Резюме	263

Глава 14. Ассоциирующие ресурсы	264
14.1. Зачем это нужно	264
14.2. Обзор	265
14.2.1. Ассоциирующие методы-псевдонимы.	267
14.3. Реализация.	267
14.3.1. Выбор имени для ассоциирующего ресурса	267
14.3.2. Поведение стандартных методов	268
14.3.3. Уникальность	268
14.3.4. Поля только для чтения.	269
14.3.5. Ассоциирующие методы-псевдонимы.	270
14.3.6. Ссылочная целостность.	271
14.3.7. Итоговое определение API.	272
14.4. Компромиссы	274
14.4.1. Сложность	274
14.4.2. Отделенность ассоциирующих ресурсов	274
14.5. Упражнения	275
Резюме.	275
Глава 15. Пользовательские методы Add и Remove.	276
15.1. Зачем это нужно	276
15.2. Обзор	277
15.3. Реализация.	278
15.3.1. Вывод списка связанных ресурсов	279
15.3.2. Целостность данных.	279
15.3.3. Итоговое определение API.	280
15.4. Компромиссы	282
15.4.1. Подчинительная связь	282
15.4.2. Метаданные о связи	282
15.5. Упражнения	282
Резюме.	283
Глава 16. Полиморфизм	284
16.1. Зачем это нужно	284
16.2. Обзор	285
16.3. Реализация.	286
16.3.1. Когда использовать полиморфные ресурсы	286
16.3.2. Полиморфная структура	288
16.3.3. Полиморфное поведение	291
16.3.4. Почему не стоит использовать полиморфные методы.	292
16.3.5. Итоговое определение API.	294
16.4. Компромиссы	295
16.5. Упражнения	295
Резюме.	296

Часть V. Коллективные операции

Глава 17. Копирование и перемещение	298
17.1. Зачем это нужно	299
17.2. Обзор	299
17.3. Реализация	300
17.3.1. Идентификаторы	300
17.3.2. Дочерние ресурсы	303
17.3.3. Связанные ресурсы	304
17.3.4. Внешние данные	307
17.3.5. Наследуемые метаданные	309
17.3.6. Атомарность	310
17.3.7. Итоговое определение API	312
17.4. Компромиссы	313
17.5. Упражнения	314
Резюме	314
Глава 18. Пакетные операции	315
18.1. Зачем это нужно	315
18.2. Обзор	316
18.3. Реализация	317
18.3.1. Атомарность	318
18.3.2. Операции над коллекцией	318
18.3.3. Упорядочивание результатов	319
18.3.4. Общие поля	320
18.3.5. Оперирование с разными родителями	321
18.3.6. Пакетный метод Get	322
18.3.7. Пакетный метод Delete	324
18.3.8. Пакетный метод Create	326
18.3.9. Пакетный метод Update	327
18.3.10. Итоговое определение API	329
18.4. Компромиссы	331
18.5. Упражнения	332
Резюме	332
Глава 19. Удаление на основе критерия	333
19.1. Зачем это нужно	334
19.2. Обзор	335
19.3. Реализация	335
19.3.1. Фильтрация результатов	337
19.3.2. По умолчанию только проверка	337
19.3.3. Подсчет результатов	338

19.3.4. Выборка результатов	339
19.3.5. Согласованность	340
19.3.6. Итоговое определение API	341
19.4. Компромиссы	341
19.5. Упражнения	341
Резюме	342
Глава 20. Анонимная запись	343
20.1. Зачем это нужно	343
20.2. Обзор	344
20.3. Реализация	346
20.3.1. Согласованность	347
20.3.2. Итоговое определение API	348
20.4. Компромиссы	349
20.5. Упражнения	350
Резюме	350
Глава 21. Пагинация	351
21.1. Зачем это нужно	352
21.2. Обзор	352
21.3. Реализация	353
21.3.1. Размер страницы	354
21.3.2. Маркеры страниц	357
21.3.3. Общее количество	360
21.3.4. Пагинация внутри ресурсов	361
21.3.5. Итоговое определение API	363
21.4. Компромиссы	363
21.4.1. Двусторонняя пагинация	363
21.4.2. Произвольный выбор окна	364
21.5. Антипаттерн: смещения и границы	364
21.6. Упражнения	366
Резюме	366
Глава 22. Фильтрация	367
22.1. Зачем это нужно	367
22.2. Обзор	368
22.3. Реализация	369
22.3.1. Структура	370
22.3.2. Синтаксис фильтра и поведение	373
22.3.3. Итоговое определение API	380
22.4. Компромиссы	380
22.5. Упражнения	381
Резюме	381

Глава 23. Импорт и экспорт	382
23.1. Зачем это нужно	383
23.2. Обзор	384
23.3. Реализация	386
23.3.1. Методы Import и Export	386
23.3.2. Взаимодействие с системами хранения данных	387
23.3.3. Преобразование между ресурсами и байтами	389
23.3.4. Согласованность	390
23.3.5. Идентификаторы и коллизии	392
23.3.6. Обработка связанных ресурсов	393
23.3.7. Сбои и повторы	394
23.3.8. Фильтрация и маски полей	397
23.3.9. Итоговое определение API	399
23.4. Компромиссы	401
23.5. Упражнения	402
Резюме	402

Часть VI. Безопасность

Глава 24. Версионирование и совместимость	404
24.1. Зачем это нужно	404
24.2. Обзор	405
24.2.1. Что такое совместимость	406
24.2.2. Определение обратной совместимости	407
24.3. Реализация	415
24.3.1. Вечная стабильность	415
24.3.2. Подвижная нестабильность	417
24.3.3. Семантическое версионирование	420
24.4. Компромиссы	423
24.4.1. Детализация или простота?	423
24.4.2. Стабильность или новая функциональность?	424
24.4.3. Удовлетворенность или универсальность?	425
24.5. Упражнения	427
Резюме	428
Глава 25. Мягкое удаление	429
25.1. Зачем это нужно	430
25.2. Обзор	430
25.3. Реализация	431
25.3.1. Обозначение удаления	432
25.3.2. Изменение стандартных методов	434
25.3.3. Метод Undelete	436

25.3.4. Метод Exrpunge	437
25.3.5. Сроки хранения	438
25.3.6. Ссылочная целостность	439
25.3.7. Влияние на другие методы	440
25.3.8. Добавление мягкого удаления для разных версий	441
25.3.9. Итоговое определение API	441
25.4. Компромиссы	442
25.5. Упражнения	443
Резюме	443
Глава 26. Повтор запросов	444
26.1. Зачем это нужно	444
26.2. Обзор	446
26.3. Реализация	447
26.3.1. Идентификатор запроса	448
26.3.2. Кэширование ответа	449
26.3.3. Согласованность	450
26.3.4. Коллизии ID запросов	452
26.3.5. Срок действия кэша	454
26.3.6. Итоговое определение API	454
26.4. Компромиссы	456
26.5. Упражнения	456
Резюме	456
Глава 27. Валидация запросов	457
27.1. Зачем это нужно	457
27.2. Обзор	459
27.3. Реализация	459
27.3.1. Внешние зависимости	461
27.3.2. Особые побочные эффекты	462
27.3.3. Итоговое определение API	463
27.4. Компромиссы	464
27.5. Упражнения	464
Резюме	464
Глава 28. Ревизии ресурсов	465
28.1. Зачем это нужно	466
28.2. Обзор	466
28.3. Реализация	467
28.3.1. Идентификаторы ревизий	468
28.3.2. Создание ревизий	469
28.3.3. Извлечение конкретных ревизий	473
28.3.4. Перечисление ревизий	474

28.3.5. Восстановление прежней ревизии	475
28.3.6. Удаление ревизий	476
28.3.7. Обработка дочерних ресурсов	478
28.3.8. Итоговое определение API	479
28.4. Компромиссы	480
28.5. Упражнения	480
Резюме	481
Глава 29. Повтор запросов	482
29.1. Зачем это нужно	482
29.2. Обзор	483
29.2.1. Интервал повторов на стороне клиента	484
29.2.2. Интервал повторов, определяемый сервером	484
29.3. Реализация	485
29.3.1. Допустимость повтора	485
29.3.2. Экспоненциальная выдержка	487
29.3.3. Retry-After	490
29.3.4. Итоговое определение API	492
29.4. Компромиссы	493
29.5. Упражнения	493
Резюме	493
Глава 30. Аутентификация запросов	494
30.1. Зачем это нужно	495
30.1.1. Источник	495
30.1.2. Целостность	495
30.1.3. Невозможность отказа	496
30.2. Обзор	497
30.3. Реализация	498
30.3.1. Генерация учетных данных	498
30.3.2. Регистрация и обмен учетными данными	499
30.3.3. Генерация и верификация сырых подписей	500
30.3.4. Цифровой отпечаток для запроса	502
30.3.5. Добавление подписи	504
30.3.6. Аутентификация запросов	507
30.3.7. Итоговое определение API	508
30.4. Компромиссы	508
30.5. Упражнения	509
Резюме	510

Посвящается Ка-эль и Луке. Вы потрясающие.

Предисловие

Все началось с электронной ударной установки. Летом 2019 года один мой друг увлек меня игрой на ней, и я погрузился в это занятие с головой. Иногда я действительно играл на барабанах, но большую часть времени все же проводил за написанием кода, который позволял управлять конфигурацией установки с помощью команд MIDI SysEx.

Когда нагрянула пандемия COVID-19, у меня внезапно появились иные приоритеты, направленные на настройку аудиовизуальной связи в нашей местной церкви, где мы перешли в формат удаленного проведения служб и рассматривали возможности возобновления личных встреч. Для этого мне пришлось изучать протоколы VISCA, NDI и OSC (для камер и аудиомикшеров), а также осваивать программно-ориентированную интеграцию с Zoom, VLC, PowerPoint, Stream Deck и др.

Эти проекты не несут в себе больших объемов бизнес-логики. Практически весь код относится к интеграции, что одновременно и раздражает, и дает большие возможности. Раздражение вызывали протоколы, которые либо были плохо задокументированы, либо не предназначались для тех задач, которые я пытался реализовать с их помощью, либо просто не согласовывались друг с другом. Положительная сторона всего этого заключается в том, что, как только вам удастся разобраться с интеграцией, вы сможете очень легко писать полезные приложения.

Несмотря на то что моя работа в последние несколько лет заключалась в основном в локальной интеграции, точно такие же раздражение и ощущение больших

возможностей возникают и при работе с веб-API. Каждый опыт выбора нового веб-API представляет собой кривую, состоящую из эмоциональных реакций в виде волнения, замешательства, раздражения, принятия и достигаемого в итоге умиротворения. Как только ты досконально осваиваешь мощный API, возникает чувство, что ты являешься дирижером величественного оркестра, готового играть любую предложенную тобой музыку, — даже если ноты скрипачей согласуются лишь под конец, а для группы медных духовых без каких-либо очевидных причин приходится использовать палочку другого цвета.

Данная книга сама по себе не изменит положение дел. Это всего лишь книга. Но если после прочтения вы будете следовать указаниям, изложенным в ней, то она поможет вам обеспечить пользователям более качественный опыт. Если же эту книгу прочтет множество людей, то вместе мы сможем добиться сдвига в направлении более согласованного и менее раздражающего опыта работы с API.

Важно понять, что сама эта книга в целом гораздо более ценна, чем сумма ее фрагментов. Относительно любого из разбираемых Джей Джейем паттернов любая команда может сделать свой обдуманный выбор (хотя кто-то может упустить некоторые из указанных здесь пограничных случаев). Для конкретной ситуации этот выбор может оказаться даже более удачным, чем предложенные в книге рекомендации, ввиду ограниченных требований контекста. Такой подход позволяет получить множество локальных оптимальных решений, но дает очень фрагментированную общую картину, поскольку в этом случае даже в API одной компании потенциально применяется несколько разрозненных техник.

Помимо согласованности для любой отдельно взятой задачи, книга также предлагает единый подход, охватывающий множество сфер проектирования API. Разработчикам программных интерфейсов редко предоставляется пространство для углубленного анализа этих аспектов, и я считаю, что мне очень повезло работать с Джей Джейем и другими (в частности, с Люком Спирингером (Luke Spieringer) над обсуждением многих тем книги. Я счастлив, что вклад, внесенный компанией Google в проектирование API, может быть весьма полезным многим разработчикам благодаря этой книге и системе AIP, расположенной по адресу <https://aip.dev>.

Я абсолютно уверен в ценности этой книги, но она не упростит разработку API. Однако она позволит вам избавиться от ненужных сложностей, которые возникают в ходе проектирования интерфейсов, давая возможность сосредоточиться только на том, что действительно уникально для ваших API. Будьте готовы думать, и думать усиленно, но также знайте, что результатом этих размышлений может стать API, работа с которым доставит лишь удовольствие. Ваши пользователи могут никогда не поблагодарить вас за это открыто. Ведь

22 Предисловие

хорошо спроектированный API зачастую воспринимается как само собой разумеющийся, хоть и является результатом огромных усилий. Но зато вы сможете спать спокойно, зная, что ваши пользователи не будут злиться при работе с API, который пусть и функционирует, но сделан на троечку.

Используйте эту книгу как основу, чтобы построить такой API, который сможет стать фундаментом для других.

*Джон Скит (Jon Skeet),
Staff Developer Relations Engineer, Google*

Вступление

Многие из нас изучали информатику. Мы анализировали среду выполнения и пространственную сложность алгоритмов с помощью нотации «О большое», знакомились со всевозможными алгоритмами сортировки и изучали различные способы обхода бинарных деревьев. Поэтому, окончив колледж, я планировал, что моя работа будет в первую очередь связана с математикой и точными науками. Но каким же было мое удивление, когда в действительности все оказалось совсем не так.

На деле бóльшая часть работы была связана с проектированием, структурированием и дизайном, а не математикой и алгоритмами. Мне никогда не приходилось думать о том, какой алгоритм сортировки использовать, поскольку для этого всегда имелась библиотека (обычно что-то вроде `array.sort()`). Однако мне все же доводилось долго и упорно размышлять над создаваемыми классами, над функциями для них и параметрами, которые должна принимать каждая функция. Все это оказалось намного более сложным, чем я ожидал.

В реальном мире я узнал, что идеально оптимизированный код далеко не столь же ценен, как хорошо спроектированный. И это оказалось вдвойне верным в отношении веб-API, поскольку они, как правило, имеют гораздо более обширную аудиторию и намного больший спектр случаев применения.

Но здесь напрашиваются вопросы: что значит «хорошо спроектированное ПО»? Что такое «хорошо спроектированный API»? Чтобы ответить на эти вопросы, довольно долго мне приходилось полагаться в основном на случайный набор ресурсов. Для одних тем я отыскивал интересные посты, которые раскрывали популярные современные альтернативы. Для других находил отдельные полезные ответы на Stack Overflow, которые подсказывали нужное направление.

Однако во многих случаях было довольно мало материалов по рассматриваемой теме, и мне приходилось самостоятельно придумывать ответы, надеясь, что мои коллеги не слишком их осудят.

Спустя многие годы подобных изысканий (и таскания повсюду блокнота с надписью «Пугающие проблемы API») я наконец-то решил, что пришло время записать всю информацию, которую я собрал и использовал в работе. Поначалу все это выглядело как набор правил для Google, которые мы с Люком Снинрингером систематизировали и которые в конечном итоге стали `AIP.dev`. Но эти правила выглядели как свод законов. В них говорилось, что нужно делать, но не пояснялось, почему нужно делать именно так. После долгого анализа и исследований, сопровождавшихся непрерывным повторением в уме этого вопроса, я готов представить вам эти правила, а также пояснить их причины.

Несомненно, было бы замечательно, если бы эта книга стала абсолютным решением для всех проблем в мире проектирования API, но, к сожалению, этого не случится. И причина тому проста: как и в архитектуре, выбор любой модели проектирования обычно опирается на личное мнение. Это означает, что одни из вас могут посчитать мои указания удачными и элегантными и будут использовать их для всех последующих проектов. Другие же могут решить, что эта книга демонстрирует отвратительные и чрезмерно ограниченные модели дизайна, и в связи с этим, наоборот, воспринять их в качестве примера того, как не нужно проектировать API. Поскольку я не могу угодить всем, моей единственной целью было предоставить набор проверенных в деле рекомендаций, подкрепив их логическими обоснованиями, объясняющими, почему они выглядят именно так.

Будете ли вы использовать их в качестве примеров для подражания или избегать — дело ваше. По меньшей мере надеюсь, что рассмотренные в книге темы вызовут много дискуссий и инициируют дальнейшую масштабную работу в этом удивительном, сложном и запутанном мире проектирования API.

Благодарности

Как и бо́льшая часть моей работы, эта книга стала результатом усилий множества людей. В первую очередь хочу поблагодарить свою жену, Ка-эль, за то, что выслушивала мои разглагольствования и жалобы в трудный период окончательной доработки рукописи. Велика вероятность, что не будь постоянной поддержки Ка-эль, я мог бы забросить работу над книгой. Кроме того, аналогичную роль сыграли и многие другие люди, включая Кристен Раньери, Бекки Сусел, Джанетт Кларк, Норриса Кларка, Тадж Кларка, Шерин Чан, Асфию Фазал и Адаму Диалло (Kristen Ranieri, Becky Susel, Janette Clarke, Norris Clarke, Tahj Clarke, Sheryn Chan, Asfia Fazal, Adama Diallo), которым я также очень благодарен.

Команда приверженцев API сыграла важную роль в рецензировании и обсуждении тем книги, а также предоставила ценные рекомендации. В частности, хочу сказать спасибо Эрику Брюэру, Хонгу Чжану, Люку Снирингеру, Джону Скиту, Альфреду Фуллеру, Энджи Лин, Тибо Хоттелье, Гаррету Джонсу, Тиму Берксу, Маку Ахмаду, Карлосу О'Райану, Маршу Гардинеру, Майку Кистлеру, Эрику Уилеру, Максу Россу, Марку Якобсу, Джейсон Вудард, Майклу Рубину, Майло Мартину, Брэду Майерсу, Сэму Маквити, Робу Клевенджеру, Майку Шварцу, Льюису Дейли, Майклу Ричардсу и Брайану Гранту (Eric Brewer, Hong Zhang, Luke Sneeringer, Jon Skeet, Alfred Fuller, Angie Lin, Thibaud Hottelier, Garrett Jones, Tim Burks, Mak Ahmad, Carlos O'Ryan, Marsh Gardiner, Mike Kistler, Eric Wheeler, Max Ross, Marc Jacobs, Jason Woodard, Michael Rubin, Milo Martin, Brad Meyers, Sam McVeety, Rob Clevenger, Mike Schwartz, Lewis Daly, Michael Richards, Brian Grant) за всю их помощь в течение нескольких лет.

Кроме того, многие участвовали в создании книги косвенно, выполняя возложенную на них часть работы, за что я выражаю признательность Рою Филдингу,

Банде четырех (Эрих Гамма, Ричард Хелм, Ральф Джонсон и Джон Влиссидес), Санджаю Гемаватту, Урсу Хельцле, Эндрю Файксу, Шону Куинлану и Ларри Гринфилду (Roy Fielding, Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, Sanjay Ghemawatt, Urs Hoelzle, Andrew Fikes, Sean Quinlan, and Larry Greenfield). Хочу также поблагодарить Стю Фельдмана, Ари Балого, Рича Санци, Йорга Хайлига, Эяль Манора, Юрия Израилевского, Уолта Драммонда, Цезаря Сенгупту и Патрика Тео (Stu Feldman, Ari Balogh, Rich Sanzi, Joerg Heilig, Eyal Manor, Yury Izrailevsky, Walt Drummond, Caesar Sengupta, Patrick Teo) за их поддержку и советы при изучении рабочих тем в Google.

Отдельная благодарность Дэйву Наглу (Dave Nagle) за поддержку в области рекламы, облачных сервисов, API и прочих аспектов, а также за то, что помог мне выйти из зоны комфорта. Выражаю признательность и Марку Чедвику (Mark Chadwick), который более десяти лет назад помог мне преодолеть синдром самозванца в проектировании API. Конструктивная обратная связь Марка и его добрые слова во многом подтолкнули меня погрузиться в изучение этой интересной области компьютерной науки. Отдельное спасибо говорю Марку Хаммонду (Mark Hammond), который научил меня ставить все под сомнение.

Данный проект оказался бы невозможным без команды Manning, в частности Майка Стивенса (Mike Stephens) и Маржаны Байс (Marjan Bace), которые одобрили саму идею книги, а также Кристины Тейлор (Christina Taylor), посвятившей себя моему очередному долгосрочному проекту. Выражаю признательность также Элу Кринкеру (Al Krinker) за детальный обзор всех глав, редактору проекта Дейдре Хиаму (Deirdre Hiam), литературному редактору Мишель Митчелл (Michele Mitchell), корректору Кери Хэйлз (Keri Hales) и техническому редактору Ивану Мартиновичу (Ivan Martinovic). Спасибо всем сотрудникам Manning, кто помог воплотить этот проект в жизнь.

Благодарю всех рецензентов: Акшата Пола, Энтони Крэппа, Брайана Дейли, Криса Хенегана, Даниэль Бретой, Дэвида Дж. Бисака, Дениза Вехби, Херардо Лекароса, Жана Лазару, Джона К. Гунвальдсона, Хорхе Эсекьель Бо, Йорта Роденбурга, Люка Купку, Марка Ненадова, Рахул Райа, Ричарда Янга, Роджера Доуэлла, Рубена Вандегинсте, Сатей Кумар Саху, Ставена Смита, Юла Уильямса, Юрия Бодарева и Зохеба Айнапора (Akshat Paul, Anthony Cramp, Brian Daley, Chris Heneghan, Daniel Bretoi, David J. Biesack, Deniz Vehbi, Gerardo Lecaros, Jean Lazarou, John C. Gunvaldson, Jorge Ezequiel Bo, Jort Rodenburg, Luke Kupka, Mark Nenadov, Rahul Rai, Richard Young, Roger Dowell, Ruben Vandeginste, Satej Kumar Sahu, Steven Smith, Yul Williams, Yurii Bodarev, Zoheb Ainapore). Ваши рекомендации помогли улучшить эту книгу.

О книге

Я написал эту книгу, стремясь предложить разработчикам безопасные, гибкие и многократно используемые паттерны создания API. Она начинается с рассмотрения ряда общих принципов проектирования, и далее описывается набор паттернов проектирования, которые могут предоставить простые решения для типичных сценариев создания API.

ДЛЯ КОГО ЭТА КНИГА

Эта книга для всех, кто уже разрабатывает веб-API или планирует заняться его разработкой, в особенности собираясь сделать его публичным. Будет хорошо, если читатели будут знать о некоторых форматах сериализации (например, JSON, Google Protocol Buffers или Apache Thrift) и распространенных парадигмах хранения данных (например, схемах реляционных баз данных), но эти знания вовсе не обязательны. Будет кстати и наличие опыта работы с HTTP и его методами (например, GET и POST), поскольку именно этот протокол был выбран для использования в примерах книги. Если в процессе разработки API вы столкнетесь со сложностями и подумаете: «С этим уже точно кто-то разобрался», — то эта книга для вас.

СТРУКТУРА ИЗДАНИЯ

Книга разделена на шесть частей. Первые две посвящены более общим темам разработки API, а последующие четыре — самим паттернам проектирования. В части I закладывается фундамент для всего последующего материала,

предоставляется ряд определений и оценочных схем для веб-API и паттернов проектирования, которые мы будем применять к этим веб-API в дальнейшем.

- Глава 1 начинается с точного определения понятия API и разъяснения важности этого программного продукта. Здесь же предоставляется своеобразная схема для оценки качества API.
- В главе 2 продолжает рассматриваться тема главы 1, изучаются способы применения паттернов проектирования к API и дается объяснение того, в чем заключается их польза для разработчиков. Здесь также разбирается анатомия паттерна проектирования API и приводится краткий пример того, как использование одного из таких паттернов может помочь получить более качественный API.

В части II продолжается раскрытие материала на основе заложенного фундамента, выделяются несколько общих принципов дизайна, которые нужно учитывать при создании любого API.

- В главе 3 разбираются все компоненты, которым может потребоваться присвоение имен в API, и поясняется, на что необходимо обращать внимание при выборе этих имен. Здесь также демонстрируется, почему именование является критически важным, хотя и кажется чем-то незначительным.
- В главе 4 описываются более крупные API, где могут использоваться несколько взаимосвязанных ресурсов. Рассматривается ряд вопросов, актуальных при выборе ресурсов и их связей. Завершается глава разбором нескольких примеров того, чего следует избегать.
- В главе 5 описано, как правильно использовать в API различные типы данных и предустановленные для них значения. В ней охвачены наиболее распространенные типы, такие как строки и числа, а также более сложные наподобие карт и списков.

В части III мы переходим к началу каталога паттернов проектирования и первыми рассмотрим фундаментальные паттерны, которые должны применяться практически ко всем API.

- В главе 6 представлен подробный разбор способов идентификации ресурсов пользователями API, с погружением в детали идентификаторов, такие как tombstoning (отметка об удалении), набор символов и кодировки, а также использование контрольных сумм для различения между отсутствующими и недействительными ID.
- В главе 7 подробно описывается, как должны работать различные стандартные методы веб-API (**Get**, **List**, **Create**, **Update** и **Delete**). В ней также объясняется, почему важно, чтобы каждый стандартный метод вел себя совершенно одинаково во всех ресурсах, а не менялся в зависимости от особенностей каждого из них.

- В главе 8 особое внимание уделяется двум стандартным методам (**Get** и **Update**) и показывается, как пользователи могут взаимодействовать не со всем ресурсом сразу, а с отдельными его частями. Кроме того, в ней разъясняется, почему это необходимо и полезно (и для пользователей, и для API), а также то, как сделать поддержку этой функциональности максимально ненавязчивой.
- В главе 9 мы выйдем за рамки стандартных методов и откроем путь к реализации в API любых нужных нам действий с помощью пользовательских методов. Особое внимание уделяется объяснению того, когда такие методы имеют смысл (и когда нет), а также тому, как принимать это решение в собственных API.
- В главе 10 рассматривается уникальный сценарий, в котором методы API могут не быть мгновенными, и поясняется, как поддерживать это удобным для пользователей образом с помощью длительных операций (*longrunning operations*, LRO). В ней разбираются принцип работы LRO и все методы, которые могут поддерживаться этими операциями, включая паузу, возобновление и отмену длительно выполняющейся задачи.
- В главе 11 читатель познакомится с принципом выполнения повторяющейся работы, в некотором роде напоминающей планирование задач для веб-API. Здесь объясняется, как использовать ресурсы **Execution** и выполнять их либо по расписанию, либо по требованию.

Часть IV посвящена ресурсам и принципам их взаимосвязи и в некотором роде содержит более обширный разбор материала главы 4.

- В главе 12 показывается, как небольшие изолированные части связанных данных можно отделять в подресурсы-одиночки. В ней детально рассматриваются обстоятельства, при которых это оказывается хорошей идеей (или нет).
- В главе 13 описывается, как ресурсы в веб-API должны хранить ссылки на другие ресурсы с помощью либо ссылочных указателей, либо встроенных значений. В ней также объясняется, как обрабатывать пограничные случаи, такие как каскадирование удалений или обновлений по мере изменения ссылочных данных во времени.
- В главе 14 более широко раскрывается тема связей между ресурсами по типу «один к одному» и объясняется, как использовать ассоциирующие ресурсы для представления связей по схеме «многие ко многим». В ней также описываются способы хранения метаданных об этих связях.
- В главе 15 рассматривается использование сокращенных методов **Add** и **Remove** в качестве альтернативы ассоциирующим ресурсам при обработке связей формата «многие ко многим». Помимо этого, в ней разбираются некоторые компромиссы при использовании этих методов и поясняется, почему они не всегда оказываются подходящими.

- Глава 16 посвящена изучению сложного принципа полиморфизма, при котором переменные могут получать целый спектр различных типов. В ней описывается обработка полиморфных полей в ресурсах API, а также объясняется, почему полиморфных методов стоит избегать.

В части V мы выйдем за границы взаимодействия с одним ресурсом API за раз и начнем рассматривать паттерны проектирования, предназначенные для взаимодействия с целыми коллекциями ресурсов.

- В главе 17 объясняется, как ресурсы можно копировать или перемещать в API. В ней разбираются тонкие моменты, такие как обработка внешних данных, метаданные, наследуемые от другого родителя, и то, как правильно рассматривать дочерние ресурсы.
- В главе 18 рассказывается о способах адаптирования стандартных методов (*Get*, *Create*, *Update* и *Delete*) под одновременное оперирование не с одним ресурсом, а с их коллекцией. В ней также разбирается ряд сложных тем, например, как должны возвращаться результаты и как обрабатывать частичные сбои.
- В главе 19 более широко раскрывается идея пакетного метода удаления, приведенная в главе 17, с примером того, как можно удалять ресурсы, соответствующие заданному фильтру, а не конкретному идентификатору. Помимо этого, в ней рассматриваются способы налаживания согласованности и даются практические рекомендации, помогающие избежать случайного уничтожения данных.
- В главе 20 подробно рассматривается поглощение не относящихся к ресурсам данных, которые сами по себе не допускают прямой адресации. В ней изучается использование анонимных записей, а также темы согласованности и компромиссов в случаях, когда подобный тип поглощения анонимных данных оказывается удачным решением для API.
- В главе 21 объясняется, как обрабатывать просмотр больших наборов данных с помощью разбивки на страницы, используя для перебора этих данных непрозрачные токены. В ней также показывается, как применять пагинацию внутри одиночных крупных ресурсов.
- В главе 22 рассказывается, как применять критерий фильтрации к листингам ресурсов, а также демонстрируется лучший способ представления этих фильтров в API. Все это применяется непосредственно к темам, разобранным в главе 19.
- В главе 23 рассматривается обработка импорта и экспорта ресурсов в API. Кроме того, здесь подробно описаны различия между операциями импорта и экспорта в сравнении с резервным копированием и восстановлением.

В части VI мы сосредоточимся на несколько менее интересной теме реализации безопасности в API. Узнаем, как обеспечивать их защиту от злоумышленников

и обезопасить предоставляемые в API методы от собственных ошибок пользователей.

- В главе 24 исследуется тема версионирования и объясняется, что означает совместимость разных версий. Идея совместимости рассматривается через призму разнообразия, и поясняется важность определения политики совместимости, согласующейся по всему API.
- В главе 25 начинается работа по защите пользователей от собственных ошибок, предоставляется паттерн (мягкое удаление), позволяющий удалять ресурсы из представления, не удаляя их полностью из системы.
- В главе 26 описывается защита системы от повторяющихся действий с помощью идентификаторов запросов. В ней разбираются тонкости использования ID запросов, а также алгоритм обеспечения правильной обработки этих ID в крупномасштабных системах.
- В главе 27 рассказывается о проверочных запросах, которые позволяют пользователям получить превью действия в API без фактического выполнения внутренней операции. В ней также демонстрируется обработка более продвинутых моментов, таких как побочные эффекты, возникающие во время запросов реального времени и проверочных запросов.
- В главе 28 вводится понятие ревизии ресурсов, используемой для отслеживания изменений во времени. Помимо этого, в ней разбираются связанные базовые операции, такие как откат к предыдущим ревизиям, и более продвинутые темы наподобие применения ревизий к дочерним ресурсам в иерархии.
- В главе 29 представляется паттерн для информирования пользователей о том, когда запросы API нужно повторить. Кроме того, здесь приводятся руководства по различным кодам HTTP-ответов и инструкции, поясняющие, когда их можно безопасно повторять.
- В главе 30 рассматриваются тема аутентификации отдельных запросов и различные критерии безопасности, которые необходимо учитывать при аутентификации пользователей в API. В ней приводится спецификация для цифровой подписи запросов API, которая соответствует практическим рекомендациям по безопасности, позволяющим убеждаться в достоверности источника и целостности запросов API, а также в том, что они не будут отвергнуты в дальнейшем.

О КОДЕ

В книге приводится множество примеров исходного кода, как в пронумерованных листингах, так и во встроенных в текст. В обоих случаях код оформлен моноширинным шрифтом, позволяющим отделить его от обычного текста. Иногда элементы кода дополнительно выделяются **жирным шрифтом**, чтобы обозначить

те фрагменты, которые изменились в течение сделанных шагов, например, когда в существующую строку добавляется новый функционал.

Во многих случаях начальный исходный код подвергался реформатированию. То есть мы добавляли разрывы строк и перерабатывали отступы, чтобы оптимально задействовать все доступное пространство страниц книги. Кроме того, во многих случаях, когда код листинга описывался в тексте, комментарии из этого листинга удалялись. Многие листинги сопровождаются аннотациями, которые поясняют наиболее важные детали.

После долгих обсуждений с первыми читателями и командой рецензентов я по ряду причин решил использовать в качестве стандартного языка TypeScript (TS). Он будет понятен как тем, кто знаком с динамическими языками (такими как JavaScript или Python), так и тем, кто работает со статическими (например, Java или C++). Это может кому-то не понравиться, и не все читатели смогут с ходу начать писать собственный TS-код, однако фрагменты, приводимые на этом языке, можно рассматривать просто как псевдокод, который должен быть понятен большинству разработчиков.

Когда дело доходит до определения API с помощью TS, нужно реализовывать два вида компонентов: ресурсы и методы. Что касается первых, то примитивы TS (например, интерфейсы) оказываются довольно выразительными при определении схем ресурсов API, позволяя почти всегда вписывать определения API буквально в несколько строк. В связи с этим все ресурсы API определяются в виде интерфейсов TS, что дает дополнительный бонус в виде более наглядных представлений JSON.

Что же касается методов API, то с ними все немного сложнее. В этом случае я предпочел задействовать для представления целого API абстрактные классы TS, следуя соглашению, обычно используемому при реализации RPC с помощью Google Protocol Buffers. Это дает возможность определять только методы API, не беспокоясь о внутренних реализациях.

При рассмотрении входов и выходов методов API я решил снова ориентироваться на Protocol Buffers, размышляя в контексте интерфейсов запросов и ответов. Это означает, что в большинстве случаев будут встречаться интерфейсы, представляющие эти входы и выходы и названные по имени метода API с добавлением суффикса `-Request` или `-Response` (например, `CreateChatRoomRequest` для метода `CreateChatRoom`).

Наконец, поскольку эта книга во многом опирается на принципы RESTful, необходимо было обеспечить способ отображения этих RPC в URL (и HTTP-метод). Для этого я предпочел использовать декораторы TS в качестве аннотаций различных методов API, выделив по одному для каждого отдельного HTTP-метода (например, `@get`, `@post`, `@delete`). Чтобы указать путь URL, в который должен отображаться метод API, каждый декоратор принимает шаблонную строку,

которая также поддерживает подстановку переменных в интерфейсе запроса. К примеру, `@get("/{id=chatRooms/*}")` будет заполнять поле ID запроса. В этом случае звездочка обозначает заполнитель (placeholder) для любого значения, за исключением символа слеша.

Как бы ни было замечательно иметь возможность опираться на спецификации OpenAPI для всех этих паттернов проектирования, есть ряд проблем, которые могут создать сложности для читателей. Во-первых, спецификации OpenAPI предназначены для использования в первую очередь компьютерами (например, генераторами кода, рендерами документации и т. д.). А поскольку цель этой книги — донести сложные темы проектирования API до других разработчиков API, то OpenAPI оказывается не лучшим способом из доступных.

Во-вторых, несмотря на все волшебство проекта OpenAPI, его использование будет довольно громоздким независимо от того, в каком формате мы станем представлять API: в YAML или JSON. К сожалению, хоть эти сложные темы и можно выразить с помощью OpenAPI, такой подход окажется не самым лаконичным решением и приведет к большому количеству лишнего содержимого, не имеющего особой ценности.

Ну и последнее: при оценке использования OpenAPI, Protocol Buffers и TypeScript первые читатели и рецензенты дали четкую обратную связь, что вариант с TS для данного конкретного случая подходит лучше всего. Имейте в виду, что я во все не навязываю никому определять API с помощью TS. Просто для данного проекта этот язык подошел наилучшим образом.

ОНЛАЙН-РЕСУРСЫ

Дополнительную информацию по проектированию API можно найти на ресурсе <https://aip.dev>, где детально разбирается множество сопредельных тем.

Об авторе

Джей Джей Гивакс трудится инженером ПО в Google, специализируясь на платежных системах, облачной инфраструктуре и проектировании API. Он также является автором книги *Google Cloud Platform in Action* и сооснователем AIP.dev, запущенного в Google общепромышленного ресурса, который посвящен сотрудничеству в сфере разработки стандартов построения API. Живет в Сингапуре со своей женой Ка-эль и сыном Лукой.

Иллюстрация на обложке

На обложке представлена иллюстрация под названием *Marchand d'Estampes à Vienne*, или «Торговец полиграфией в Вене». Она взята из коллекции костюмов различных стран, составленной Жаком Грассе де Сен-Совером (1757–1810 годы) и опубликованной им в 1797 году под названием *Costumes de Différents Pays*. Каждая иллюстрация была старательно отрисована и раскрашена от руки. Богатое разнообразие коллекции Грассе де Сен-Совера напоминает нам, насколько культурно разнообразными были города и регионы планеты буквально 200 лет назад. Будучи изолированными друг от друга, люди говорили на разных диалектах и языках. В поселениях по одеянию человека можно было легко определить, где он живет, чем занимается и какое положение в обществе занимает.

С тех пор манеры одеваться очень изменились. Теперь сложно различить даже жителей разных континентов, не говоря уже о гражданах разных городов, регионов и стран. Возможно, мы променяли культурное разнообразие на более разнообразную личную жизнь — и уж точно на более разнообразную и динамичную технологическую.

Во времена, когда сложно отличить одну компьютерную книгу от другой, издательство Mapping чувствует изобретательность и новаторство компьютерного бизнеса с помощью обложек, отражающих богатое разнообразие региональной жизни двухсотлетней давности, о котором нам напоминают картины Грассе де Сен-Совера.

От издательства

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

Часть I

Введение

Проектирование API — непростая задача. Будь она простой, потребности в этой книге не возникло бы. Однако прежде чем переходить к изучению инструментов и паттернов, способствующих более гибкой и удобной разработке API, нам нужно утвердить основополагающую терминологию и сформировать представление о том, чего ожидать от книги. В следующих двух главах мы разберем кое-какой ознакомительный материал, который послужит платформой, облегчающей изучение материала всей оставшейся книги.

Главу 1 начнем с подробного определения значения API. Более того, мы разберемся, как должен выглядеть хороший API и как отличить его от плохого. Затем в главе 2 мы более внимательно рассмотрим понятие паттерна проектирования и анатомию паттернов, представленных на страницах книги, чтобы научиться использовать их для построения стабильно хороших API.



Знакомство с API

В этой главе

- ✓ Что такое интерфейсы.
- ✓ Что такое API.
- ✓ Что такое ориентация на ресурсы.
- ✓ Что делает API «хорошим».

Раз вы выбрали эту книгу, то, скорее всего, уже в целом знакомы с API. Кроме того, вы наверняка уже знаете, что API означает «программный интерфейс приложения», так что в первой главе мы сосредоточимся на тщательном разъяснении значений этих базовых понятий и их значимости. Начнем с более подробного разбора идеи API.

1.1. ЧТО ТАКОЕ ВЕБ-API

API определяет способ взаимодействия компьютерных систем. А поскольку в изолированном режиме функционирует очень небольшое количество систем, то можно не удивляться тому, что API буквально повсюду. Их можно найти в библиотеках, которые мы используем из языковых диспетчеров пакетов (например, в библиотеке шифрования, предоставляющей метод наподобие `function encrypt(input: string): string`), а также в коде, который пишем сами, даже если

он и не предназначен для использования другими. Но существует особый тип API, который создается для раскрытия по сети и используется удаленно многими людьми. Именно такие программные интерфейсы, зачастую называемые веб-API, и рассматриваются в этой книге.

Веб-API привлекают внимание благодаря нескольким аспектам, но самым интересным из них, вероятно, является то, что создатели таких API имеют практически полный контроль над ними, при том, что конечные пользователи, наоборот, в контроле ограничены. Используя библиотеку, мы работаем с ее локальными копиями, в связи с чем создатели API могут в любое время делать все, что захотят, без риска навредить пользователям. Веб-API отличаются тем, что никаких копий не используется. В результате при внесении в веб-API изменений последние сказываются на пользователях вне зависимости от того, просили они о них или нет.

Представьте, к примеру, вызов веб-API, который позволяет вам зашифровать данные. Если работающая над ним команда решит начать использовать для шифрования другой алгоритм, то в этом случае у вас не будет выбора. При вызове соответствующего метода шифрования ваши данные будут зашифрованы последним заданным алгоритмом. В более экстремальном примере авторы API могут решить полностью закрыть его и игнорировать ваши запросы. В этот момент ваше приложение внезапно перестанет работать, и вы ничего не сможете с этим поделать. Оба описанных сценария показаны на рис. 1.1.

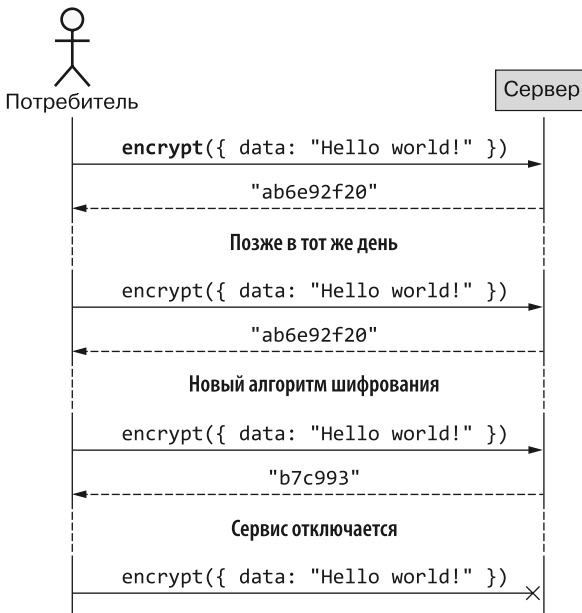


Рис. 1.1. Возможные сценарии при работе с веб-API со стороны потребителя

Тем не менее эти особенности, оказывающиеся для потребителей недостатками, для разработчиков API обычно выступают главными преимуществами, так как позволяют сохранять полный контроль над своим детищем. К примеру, если в API шифрования используется новый суперсекретный алгоритм, то создатели этого интерфейса вряд ли захотят просто так раскрыть его код миру в форме библиотеки. Вместо этого они скорее предпочтут использовать веб-API, который позволит им предоставить пользователям *функциональность* этого суперсекретного алгоритма, не выдавая свою ценную интеллектуальную собственность. В иных случаях системе может потребоваться невероятная вычислительная мощность, что приведет к очень длительному выполнению, если развернуть ее в качестве библиотеки и запускать на домашнем ПК или ноутбуке. В похожих случаях, например при работе со многими API машинного обучения, создание веб-API позволяет предоставить потребителям мощную функциональность, скрыв вычислительные требования. Схематично это представлено на рис. 1.2.

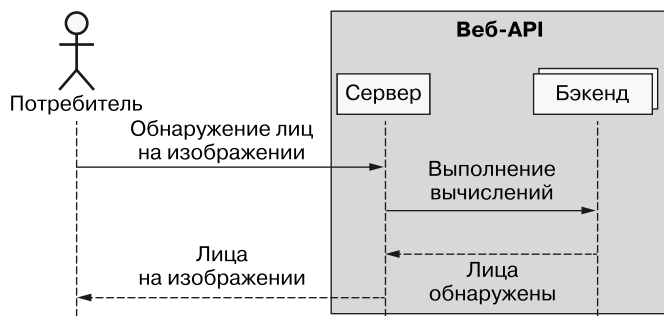


Рис. 1.2. Пример веб-API, скрывающего необходимую вычислительную мощность

Теперь, когда мы понимаем, что такое API (и, в частности, веб-API), возникает вопрос: в чем их значимость?

1.2. ПОЧЕМУ API ВАЖНЫ

Программное обеспечение нередко создается исключительно для использования человеком, и в этом нет ничего плохого. Однако за последние несколько лет мы видим все больший уклон в сторону автоматизации, когда компьютерные программы создаются для того, чтобы делать то же, что и человек, но быстрее и эффективнее. К сожалению, именно на этом рубеже ПО «только для людей» становится проблемой.

Разрабатывая нечто, предназначенное исключительно для использования человеком, и подразумевая взаимодействие с системой с помощью мыши и клавиатуры, мы склонны смешивать строение системы и визуальные аспекты

с сырыми данными и функциональными аспектами. Проблема в том, что может быть сложно объяснить компьютеру, как взаимодействовать с графическим интерфейсом. Причем она усугубляется тем, что изменение визуальных аспектов программы также может потребовать переобучения компьютера взаимодействию с этим *новым* графическим интерфейсом. Те изменения, которые кажутся чисто косметическими для нас, оказываются совершенно нераспознаваемыми для компьютера. Иными словами, для компьютера нет такого понятия, как «чисто косметически».

API — это интерфейсы, которые предназначены специально для компьютеров и имеют важные свойства, позволяющие компьютерам легко их использовать. К примеру, у этих интерфейсов отсутствуют визуальные составляющие, значит, не нужно беспокоиться о внешних изменениях. К тому же эти интерфейсы обычно развиваются только «совместимым» образом (подробнее см. в главе 24), значит, нет необходимости переучивать компьютер чему-либо в связи с изменениями. Если кратко, то API предоставляют способ говорить на языке, который нужен компьютерам, чтобы осуществлять безопасное и стабильное взаимодействие.

Но все это не ограничивается простой автоматизацией. API также открывают возможности для композиции, которая позволяет нам рассматривать функциональность как кубики лего, соединяя детали все новыми и новыми способами, чтобы получить нечто, в разы превосходящее сумму своих частей. Более того, эти получаемые композиции API также можно рассматривать как строительные блоки, формируя из них еще более сложные и невероятные проекты.

Но здесь возникает важный вопрос: как сделать так, чтобы создаваемые нами API сочетались подобно кубикам лего? Начнем с рассмотрения используемой для этого *ресурсно-ориентированной* стратегии.

1.3. ЧТО ЗНАЧИТ РЕСУРСНО-ОРИЕНТИРОВАННЫЕ API

Многие из существующих сегодня веб-API действуют подобно слугам: вы просите их выполнить что-либо, и они это делают. К примеру, если мы хотим узнать прогноз погоды в своем городе, то можем приказать веб-API `predictWeather(postalCode=10011)`. Такой вид передачи приказа другому удаленному компьютеру с помощью вызова предварительно настроенной подпрограммы или метода обычно называется выполнением «удаленного вызова процедуры» (remote procedure call, RPC), так как, по существу, мы *вызываем* библиотечную функцию (или *процедуру*) для выполнения на другом компьютере, находящемся в другом (*удаленном*) месте. Важнейший аспект подобных API — акцент на выполняемых действиях. То есть мы думаем о вычислении погоды (`predictWeather(postalCode=...)`), либо

шифровании данных (`encrypt(data=...)`), либо отправке электронной почты (`sendEmail(to=...)`), во всех случаях делая акцент на «выполнении» чего-либо.

Тогда почему не все API ориентированы на RPC? Одна из основных причин связана с идеей «сохранения состояния» (statefulness), когда вызовы API могут быть либо «с сохранением состояния» (stateful), либо «без сохранения состояния» (stateless). Вызов API рассматривается как stateless, когда его можно совершить независимо от всех других запросов API без дополнительного контекста или данных. Например, вызов веб-API для прогнозирования погоды задействует только один независимый ввод данных (почтовый код), в связи с чем считается stateless. С другой стороны, веб-API, который сохраняет выбранные пользователем города и предоставляет для них прогнозы погоды, не имеет входных данных в среде выполнения, но требует, чтобы у пользователя уже были сохранены интересующие его города. В результате такой вид запроса API, подразумевающего предварительные запросы или использование ранее сохраненных данных, будет считаться stateful. На деле оказывается, что RPC-ориентированные API отлично подходят для stateless-функциональности, но при этом гораздо менее эффективны, когда мы вводим stateful-методы API.

ПРИМЕЧАНИЕ

Если вы знакомы с концепцией REST, то сейчас будет кстати сказать, что этот раздел не посвящен конкретно REST и RESTful API, а является более общим для всех API, которые выделяют «ресурсы» (как большинство RESTful API). Другими словами, хоть здесь и будет много пересечений с темой REST, данный раздел все же охватывает более общий материал.

Чтобы понять, почему это так, мы рассмотрим пример stateful-API для бронирования билетов на самолет. В табл. 1.1 отражен список RPC для взаимодействия с планированием авиаперелетов, включая такие действия, как планирование новых броней, просмотр существующих и отмена полета.

Таблица 1.1. Перечень методов для примера API бронирования авиаперелетов

Метод	Описание
<code>ScheduleFlight()</code>	Планирует новый перелет
<code>GetFlightDetails()</code>	Показывает информацию о конкретном перелете
<code>ShowAllBookings()</code>	Показывает все забронированные на данный момент перелеты
<code>CancelReservation()</code>	Отменяет резервирование перелета
<code>RescheduleFlight()</code>	Переназначает существующий перелет на другую дату или время
<code>UpgradeTrip()</code>	Переводит из эконом-класса в первый класс

Каждый из этих RPC отлично говорит сам за себя, но здесь никак не обойтись без запоминания этих методов API, многие из которых очень похожи между собой. Так, иногда метод говорит о «перелете» (тот же `RescheduleFlight()`), а в других случаях оперирует словом «резервирование» (например, `CancelReservation()`). Кроме того, нужно помнить, какие из множества синонимичных форм этих действий использовались. К примеру, следует фиксировать, с помощью какого из методов мы просматриваем все свои брони: `ShowFlights()`, `ShowAllFlights()`, `ListFlights()` или `ListAllFlights()` (в данном случае это `ShowAllFlights()`). Но как можно решить такую проблему? С помощью стандартизации.

Ориентированность на ресурсы призвана помочь решить эту проблему в двух направлениях, предоставляя стандартный набор строительных блоков для использования при проектировании API. Во-первых, ресурсно-ориентированные API опираются на идею «ресурсов», являющихся ключевыми единицами, которые мы сохраняем и с которыми взаимодействуем, стандартизируя все, что реализует API. Во-вторых, вместо того чтобы использовать для любого нужного действия произвольных имен RPC, ресурсно-ориентированные API ограничивают действия до небольшого стандартного набора (описанного в табл. 1.2), который применяется к каждому ресурсу, формируя полезные действия в API. Если посмотреть немного под другим углом, то ресурсно-ориентированные API — это просто особый тип API, основанных на RPC, в которых каждый RPC следует понятному и стандартизированному паттерну: `<StandardMethod><Resource>()`.

Таблица 1.2. Перечень стандартных методов и их значения

RPC	Описание
<code>Create<Resource>()</code>	Создает новый <code><Resource></code>
<code>Get<Resource>()</code>	Показывает информацию о конкретном <code><Resource></code>
<code>List<Resources>()</code>	Показывает список всех существующих <code><Resources></code>
<code>Delete<Resource>()</code>	Удаляет существующий <code><Resource></code>
<code>Update<Resource>()</code>	Обновляет существующий <code><Resource></code> на месте

Если пойти дальше по этому пути особых, ограниченных RPC, то получится, что вместо разнообразия всевозможных методов RPC, показанных в табл. 1.1, мы можем придумать один ресурс (например, `FlightReservation`) и получить равнозначную функциональность с набором стандартных методов, показанных в табл. 1.3.

Таблица 1.3. Перечень стандартных методов, применяемых к ресурсу перелетов

Метод		Ресурс		Методы
Create				CreateFlightReservation()
Get				GetFlightReservation()
List	×	FlightReservation	=	ListFlightReservations()
Delete				DeleteFlightReservation()
Update				UpdateFlightReservation()

Очевидно, что стандартизация оказывается более организованной, но значит ли это, что все ресурсно-ориентированные API лучше, чем RPC-ориентированные? Вообще-то нет. В некоторых сценариях RPC-ориентированные API окажутся более эффективными (особенно в случае stateless-методов API). Однако во многих других случаях пользователям будет намного проще понять, изучить и запомнить именно ресурсно-ориентированные API. Причина в том, что предоставляемая ими стандартизация упрощает совмещение того, что вы уже знаете (например, набора стандартных методов), с тем, что вы можете легко усвоить (например, названием нового ресурса), позволяя сразу начать взаимодействовать с API. Говоря в цифрах, если вы знаете, скажем, пять стандартных методов, то благодаря надежному паттерну освоение одного нового ресурса в действительности будет приравнено к освоению пяти новых RPC.

Конечно же, не все API одинаковы, и было бы несколько опрометчиво определять их сложность в формате размера списка того, что «нужно изучить». С другой стороны, здесь работает один важный принцип: сила паттернов. На деле изучать совмещаемые части и объединять их в более сложные компоненты, которые следуют установленному образцу, оказывается легче, чем изучать предварительно собранные компоненты, которые каждый раз соответствуют особой структуре. Поскольку ресурсно-ориентированные API зиждутся на проверенных временем паттернах проектирования, зачастую их проще выучить, а значит, они оказываются «лучше», чем их RPC-ориентированные аналоги. Но вслед за этим возникает важный вопрос: что в данном случае значит «лучше»? Как нам понять, является ли API «хорошим»? Что здесь вообще значит «хороший»?

1.4. ЧТО ДЕЛАЕТ API «ХОРОШИМ»

Прежде чем рассматривать различные аспекты, делающие API «хорошим», нужно разобраться, зачем мы вообще используем программный интерфейс. Иначе говоря, какова цель его создания? Обычно она сводится к двум простым причинам:

- 1) у нас есть востребованная определенными пользователями функциональность;
- 2) эти пользователи хотят использовать данную функциональность программно.

К примеру, у нас может быть система, которая прекрасно справляется с переводом текстов. В мире наверняка много людей, заинтересованных в такой возможности, но одного этого недостаточно. В конце концов, мы можем запустить вместо API мобильное приложение, предоставляющее эту прекрасную систему перевода. Чтобы получить API, люди, заинтересованные в этой функциональности, должны также захотеть написать программу, позволяющую использовать ее. Учитывая два этих критерия, какие качества будут желательными для нашего API?

1.4.1. Функциональность

Начнем с самого важного: вне зависимости от того, как будет выглядеть окончательный интерфейс, система в целом должна быть *функциональной*. Иначе говоря, она должна делать то, что пользователям действительно нужно. Если она предназначена для перевода текстов с одного языка на другой, то должна уметь это делать. Кроме того, к большинству систем может предъявляться множество *нефункциональных* требований. Например, если наша система переводит текст, то к ней могут предъявляться нефункциональные требования, связанные с такими аспектами, как задержка (например, процесс перевода должен занимать несколько миллисекунд, а не дней) или точность (например, переводы не должны оказываться ложными и вводить в заблуждение). Вот эти два аспекта могут составлять функциональную сторону системы.

1.4.2. Выразительность

Помимо функциональной эффективности, система также должна иметь интерфейс, который бы позволял пользователям выражать свои намерения ясно и просто. Другими словами, если система переводит текст, то ее API должен быть создан так, чтобы делать это можно было простым и понятным способом. В данном случае это может быть RPC под названием `TranslateText()`. Конечно, это может звучать вполне очевидно, но в действительности нередко оказывается сложнее, чем кажется.

К примерам подобной скрытой сложности относится случай, когда API уже поддерживает некую функциональность, но по недосмотру мы не замечаем, что она нужна пользователям, и не реализуем выразительного способа доступа к ней. В результате люди вынуждены использовать ее *обходными путями*, совершая

ряд непривычных и не всегда очевидных действий. К примеру, если API предоставляет возможность переводить текст, то пользователь может косвенно заставить его исключительно для определения языка, даже если переводить ничего не хочет. Нетрудно представить, что прямой RPC `DetectLanguage()` оказался бы для пользователей куда удобнее, чем совершение множества вызовов API в попытке угадать язык.

Листинг 1.1. Функциональность, позволяющая определять язык с помощью только метода API `TranslateText`

```
function detectLanguage(inputText: string): string {
  const supportedLanguages: string[] = ['en', 'es', ... ];
  for (let language of supportedLanguages) {
    let translatedText = TranslateApi.TranslateText({
      text: inputText,
      targetLanguage: language
    });
    if (translatedText == inputText) {
      return language;
    }
  }
  return null;
}
```

Здесь подразумевается, что API определяет метод `TranslateText`, который получает вводный текст и целевой язык, в который нужно его перевести

Если переведенный текст совпадает с исходным, то мы понимаем, что перед нами один и тот же язык

Если мы не находим переведенный текст, совпадающий с исходным, то возвращаем `null`, указывая, что язык исходного текста определить невозможно

Как показывает этот пример, API, которые поддерживают определенную функциональность, но не упрощают пользователям доступ к ней, оказываются не самыми удачными. При этом выразительные API предоставляют пользователям возможность ясно выражать не только то, что они хотят сделать (например, перевести текст), но и каким именно образом (например, в течение 150 миллисекунд, с точностью 95 %).

1.4.3. Простота

Одним из важнейших факторов, связанных с удобством использования любой системы, является ее *простота*. И хотя логично предположить, что достичь простоты можно за счет сокращения *компонентов* (RPC, ресурсов и т. п.) в API, к сожалению, это не так. К примеру, API может опираться на один метод `ExecuteAction()`, обрабатывающий всю функциональность. Тем не менее это ничего не упрощает, а только перемещает сложность из одного места (множества разных RPC) в другое (много настроек в одном RPC). Как же тогда выглядит простой API?

Вместо того чтобы пытаться излишне сокращать количество RPC, API должен стремиться раскрывать нужную пользователям функциональность наиболее доступным способом, делая API максимально простым, но не более. Представьте, что в API перевода нужно добавить возможность определять язык входного

текста. Можно сделать это путем возвращения определенного исходного текста в ответ на перевод. Однако это все равно запутывает функциональность, скрывая ее внутри метода, предназначенного для другой цели. Вместо этого будет разумнее создать новый метод специально для поставленной задачи, например `DetectLanguage()`. (Обратите внимание, что мы также можем включать распознавание языка при переводе содержимого, но это служит уже совсем для другой цели.)

Еще один универсальный подход к обеспечению простоты основан на старом правиле «стандартного случая» (делать обработку стандартных случаев быстрой), но вместо этого ориентируется на простоту использования, оставляя пространство для пограничных случаев. Такая измененная формулировка подразумевает, что нужно «реализовать типичный случай превосходно, а продвинутый сделать возможным». Это означает, что при каждом добавлении чего-либо, усложняющего API в угоду продвинутому пользователю, лучше все-таки скрывать это усложнение от типичного пользователя, который заинтересован лишь в стандартном случае. При таком подходе более частые сценарии получатся простыми и легкими в использовании, но при этом сохранятся и продвинутые возможности для тех, кому они нужны.

В качестве примера представим, что наш API перевода включает принцип использования модели машинного обучения, которая понадобится при переводе текста, когда вместо того, чтобы указать целевой язык, мы выбираем основанную на нем модель и используем ее как «механизм перевода». Несмотря на то что такая функциональность предоставляет гораздо больше гибкости и контроля для пользователей, она окажется и намного сложнее. Этот новый типичный случай показан на рис. 1.3.

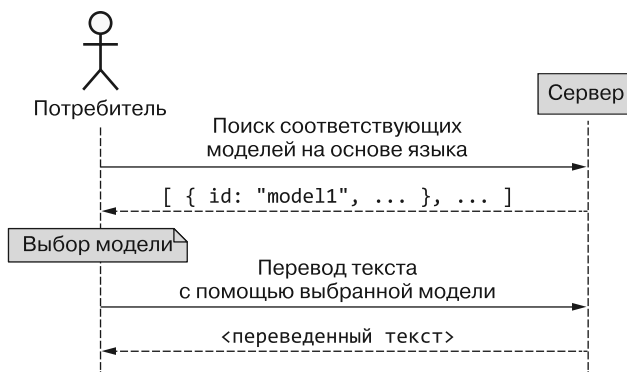


Рис. 1.3. Перевод текста после выбора модели

Как видите, в итоге в обмен на поддержку более продвинутой функциональности мы существенно усложнили процесс перевода. Чтобы увидеть это

более наглядно, сравните код из листинга 1.2 с простотой вызова `TranslateText("Hello world", "es")`.

Листинг 1.2. Перевод текста после выбора модели

```
function translateText(inputText: string,
                      targetLanguage: string): string {
  let sourceLanguage =
    TranslateAPI.DetectLanguage(inputText);
  let model = TranslateApi.ListModels({
    filter: `sourceLanguage:${sourceLanguage}
           targetLanguage:${targetLanguage}`,
  })[0];
  return TranslateApi.TranslateText({
    text: inputText,
    modelId: model.id
  });
}
```

Поскольку нам нужно выбирать модель, сначала необходимо узнать язык входного текста. Чтобы определить его, мы можем опереться на гипотетический метод `DetectLanguage()`, предоставленный API

Теперь, когда у нас наконец есть все необходимые входные данные, можно вернуться к переводу текста на целевой язык

Когда нам будут известны и исходный, и целевой языки, можно будет выбрать любую подходящую модель из тех, что предлагает API

Как сделать такой API максимально простым, одновременно сделав типичный случай превосходным, а продвинутый — возможным? Поскольку типичный случай подразумевает пользователей, которых не интересует конкретная модель, можно спроектировать API так, чтобы он принимал либо `targetLanguage`, либо `modelId`. Тогда продвинутый случай будет по-прежнему рабочим (код из листинга 1.2 продолжит функционировать), но типичный случай будет выглядеть намного проще, опираясь просто на параметр `targetLanguage` (и ожидая, что параметр `modelId` останется неопределенным) (листинг 1.3).

Листинг 1.3. Перевод текста на целевой язык (типичный случай)

```
function translateText(inputText: string,
                      targetLanguage: string,
                      modelId?: string): string {
  return TranslateApi.TranslateText({
    text: inputText,
    targetLanguage: targetLanguage,
    modelId: modelId,
  });
}
```

Теперь, когда у нас есть представление о том, насколько для «хорошего» API важна простота, рассмотрим заключительный критерий: предсказуемость.

1.4.4. Предсказуемость

Если в жизни сюрпризы иногда оказываются забавными, то в API им точно места нет, хоть в определении интерфейса, хоть во внутреннем поведении. Это утверждение можно сравнить с поговоркой об инвестировании: «Если вы очень им увлечены, значит, действуете неправильно». Так что же мы подразумеваем под API «без сюрпризов»?

API без сюрпризов опираются на повторяющиеся паттерны, применяемые и к внешнему определению API, и к его поведению. Например, если API перевода текста содержит метод `TranslateText()`, который в качестве параметра получает входное содержимое в поле `text`, то при добавлении метода `DetectLanguage()` входное содержимое должно также называться `text` (а не `inputText`, `content` или `textContent`). Хотя это и выглядит очевидным сейчас, помните, что многие API построены несколькими командами и выбор имен для полей при наличии множества параметров зачастую оказывается произвольным. В итоге, когда два отдельных человека отвечают за два таких отдельных поля, возникает большая вероятность, что они сделают разный выбор. И когда это происходит, мы получаем несогласованный API, то есть API с сюрпризом.

Несмотря на то что такая несогласованность может казаться незначительной, в действительности подобные проблемы оказываются намного более важными, чем выглядят на первый взгляд. Причина в том, что пользователи API довольно редко изучают каждую деталь, тщательно вчитываясь в документацию. Вместо этого они просматривают ровно столько, сколько нужно, чтобы добиться требуемого результата. То есть если человек узнает, что поле называется `text` в одном текстовом сообщении, он наверняка предположит, что оно так же называется и в другом. При этом человек будет строить аналогичные догадки, основанные на известных ему фактах, и в отношении остального. Если этот процесс потерпит неудачу (например, потому что в другом сообщении поле было названо `inputText`), то работа встанет и пользователю придется выяснять, почему его предположение не оправдалось.

Очевидный вывод здесь в том, что API, которые опираются на повторяющиеся, предсказуемые паттерны (например, согласованное именование полей), проще и быстрее освоить, а значит, они лучше. Аналогичные преимущества дают и более сложные паттерны, такие как стандартные действия, которые мы видели при изучении ресурсно-ориентированных API. Это дает нам представление об основной цели книги: научить использовать известные, четко определенные, понятные и (надеюсь) простые паттерны для создания предсказуемых, доступных, следовательно, и более качественных API. Теперь, когда мы разобрались с понятием API и выяснили, что делает их хорошими, пора начать рассматривать высокоуровневые паттерны, которым мы можем следовать при разработке наших API.

РЕЗЮМЕ

- Интерфейсы представляют собой контракты, определяющие, как две системы должны взаимодействовать друг с другом.
- API — это особые типы интерфейсов, которые определяют правила взаимодействия двух компьютерных систем. Они могут выражаться в разных формах, таких как скачиваемые библиотеки и веб-API.

- Веб-API отличаются тем, что предоставляют функциональность по сети, скрывая конкретную реализацию или вычислительные требования, необходимые для ее выполнения.
- Ресурсно-ориентированные API — это способ проектирования программных интерфейсов, позволяющий снизить сложность за счет использования стандартного набора действий, называемых *методами*, среди ограниченного набора компонентов, называемых *ресурсами*.
- Однозначно определить, что значит «хороший» API, сложно, но обычно они отличаются функциональностью, выразительностью, простотой и предсказуемостью.

Знакомство с паттернами проектирования API

В этой главе

- ✓ Что такое паттерн проектирования API.
- ✓ Почему паттерны проектирования API важны.
- ✓ Анатомия и структура паттерна проектирования API.
- ✓ Проектирование API с помощью паттерна и без него.

Получив представление о том, что делает API «хорошими», мы можем начать изучать способы использования различных паттернов при их создании. Сначала мы выясним, что собой представляют эти паттерны, почему они важны и как будут описываться в последующих главах. В завершение мы рассмотрим пример API и разберемся, как использование предварительно созданных паттернов может сэкономить много времени и избавить нас от проблем в будущем.

2.1. ЧТО ТАКОЕ ПАТТЕРНЫ ПРОЕКТИРОВАНИЯ API

Прежде чем заняться изучением этих паттернов, нам нужно заложить кое-какую основу, начав с простого вопроса: что такое паттерн проектирования? Если мы утвердим, что проектирование ПО относится к структуре или схеме некоего кода, написанного для решения задачи, то *паттерн* проектирования ПО будет означать возможность его повторяющегося применения к множеству схожих задач по созданию ПО с внесением лишь минимальных правок,

подстраивающих его под частные сценарии. Это означает, что данный паттерн не является какой-то заранее собранной библиотекой, с помощью которой мы решаем отдельные задачи, а выступает скорее в роли схемы для решения схожих по структуре задач.

Если это звучит слишком абстрактно, то в качестве более осязаемой аналогии представим, что хотим построить на заднем дворе сарай. Для этого можно выбрать один из множества технологических подходов, начиная от техник строительства, которые использовали несколько сотен лет назад, и заканчивая современными решениями, доступными благодаря таким компаниям, как Lowe и Home Depot. Вариантов оказывается и впрямь очень много, но вот четыре основных.

- Купить готовый сарай и привезти его во двор.
- Купить комплект для сборки сарая (чертежи и материалы) и собрать его самостоятельно.
- Купить набор чертежей сараев, скорректировать проект под свои нужды, потом также построить его самостоятельно.
- Спроектировать и построить сарай с нуля.

Если представить эти варианты через призму ПО, то они будут охватывать диапазон от использования готового программного пакета до создания системы, полностью настраиваемой под нашу задачу. В табл. 2.1 мы видим, что по ходу списка варианты становятся все сложнее, но при этом от варианта к варианту растет гибкость. Другими словами, самый простой вариант является наименее гибким, а самый сложный, наоборот, обеспечивает максимальную подвижность в решениях.

Таблица 2.1. Сравнение способов постройки сарая со способами создания ПО

Вариант	Сложность	Гибкость	Программный эквивалент
Купить готовый	Проще некуда	Отсутствует	Использовать готовый программный пакет
Собрать из набора	Легко	Очень низкая	Создать путем кастомизации существующего ПО
Построить по чертежам	Нужно постараться	Средняя	Создать на основе проектной документации
Построить с нуля	Трудно	Максимальная	Написать полностью заказное ПО

Инженеры ПО чаще всего выбирают вариант «построить с нуля». Иногда это необходимо, особенно в случаях, когда стоящие перед нами задачи новы. В прочих ситуациях этот выбор выигрывает в соотношении «затраты — выгоды»,

поскольку задача различается ровно в той степени, чтобы не позволить нам опереться на более простые варианты. И все же бывают случаи, когда мы знаем библиотеку, которая способна решить нашу задачу полноценно (или достаточно близко к этому), и мы предпочитаем опереться на кого-то, кто уже реализовал это решение. Оказывается, промежуточные варианты (индивидуальная настройка существующего ПО или его создание на основе проектной документации) выбираются гораздо реже, хотя, вероятно, могут использоваться чаще, давая отличные результаты. Вот здесь и вступают в игру паттерны проектирования.

На высоком уровне они предназначены для варианта «постройки на основе чертежей», но в отношении ПО. Аналогично тому, как в чертежах сарая указываются размеры, обозначается расположение окон и дверей, а также перечисляются материалы для крыши, паттерны проектирования тоже несут в себе набор спецификаций и деталей для кода, который мы будем писать. В сфере ПО это обычно означает формирование высокоуровневой схемы кода, а также описание нюансов ее использования для решения конкретной задачи. Тем не менее редко бывает так, что паттерн проектирования создается для автономного использования. Чаще всего они фокусируются на конкретных компонентах, а не целых системах. Другими словами, чертежи дают информацию об одном аспекте (например, форме крыши) или компоненте (например, дизайне окна), а не обо всем сарае. На первый взгляд это может показаться недостатком, но так бывает, только если целью является реально постройка сарая. Если же вы создаете лишь нечто подобное ему, то наличие чертежей для каждого отдельного компонента будет означать возможность сопоставлять их все в нужных вам соотношениях, выбирая форму крыши А и дизайн окна В. Это относится и к нашему обсуждению паттернов проектирования, поскольку каждый из них фокусируется на одном компоненте или типе задачи системы, помогая вам создавать ровно то, что вы хотите, собирая множество готовых частей воедино.

К примеру, если вы хотите добавить в систему логирование отладки, то наверняка предпочтете только один способ логирования сообщений. Реализовать это можно по-разному (например, используя одну общую глобальную переменную), но существует специальный паттерн проектирования, предназначенный для решения именно этой программной задачи. Данный паттерн описан в фундаментальной работе «Паттерны проектирования» (*Design Patterns*) (Гамма и др., 1994 год) и называется «Одиночка» (или «Синглтон»). Он гарантирует создание только одного экземпляра класса. Этот «чертеж» вызывает класс с закрытым конструктором и одним статическим методом `getInstance()`, который всегда возвращает один экземпляр этого класса (он создает этот один экземпляр, только если тот еще не существует). Причем данный паттерн вовсе не является завершенным (какой смысл иметь класс-одиночку, который ничего не делает?). Тем не менее

это четко определенный и протестированный паттерн, которому можно следовать, когда требуется решить эту небольшую отдельную задачу — всегда иметь только один экземпляр класса.

Теперь, когда мы знаем в целом, чем являются паттерны проектирования ПО, нужно задать вопрос: что такое паттерны проектирования API? Если опереться на определение API, приведенное в главе 1, то они представляют собой просто паттерны проектирования программного обеспечения, примененные к API, а не ко всему ПО. Это означает, что паттерны разработки API, подобно стандартным паттернам, — просто чертежи, используемые для создания и структурирования API. Поскольку в данном случае фокус на интерфейсе, а не реализации, то в большинстве случаев паттерн проектирования API будет сосредоточен исключительно на интерфейсе без обязательного построения реализации. Несмотря на то что большинство паттернов создания API зачастую будут пассивны в отношении внутренней реализации этих интерфейсов, иногда они диктуют конкретные аспекты поведения API. К примеру, такой паттерн может указывать, что определенный RPC *согласуется под конец*, подразумевая, что возвращаемые от этого RPC данные могут быть слегка устаревшими (например, он может производить считывание из кэша, а не из общей системы хранения).

Чуть позже мы более детально разберем дальнейшее документирование паттернов API, но сначала кратко рассмотрим, почему нам вообще нужно беспокоиться о паттернах проектирования API.

2.2. ЧЕМ ВАЖНЫ ПАТТЕРНЫ ПРОЕКТИРОВАНИЯ API

Как мы уже узнали, паттерны разработки API оказываются полезны при построении этих интерфейсов точно так же, как чертежи при построении сарая: они выступают в качестве готовых строительных блоков, которые можно использовать в наших проектах. При этом мы не разобрались, зачем они вообще нам нужны. Неужели не настолько умны, чтобы самостоятельно создавать хорошие API? Разве нам не лучше известна сфера реализации проекта и технические задачи? Конечно, зачастую это так, но в действительности некоторые из техник, с помощью которых мы создаем хорошо спроектированное ПО, при построении API не слишком эффективны. Если говорить конкретнее, то итеративный подход, рекомендуемый в процессе гибкой разработки, сложно применить при проектировании API. Чтобы понять, почему это так, нужно взглянуть на два аспекта программных систем. Сначала нам необходимо изучить гибкость (или ригидность) различных интерфейсов в общем и затем уже понять, как аудитория этого интерфейса влияет на нашу возможность вносить изменения и экспериментировать со всем дизайном. Начнем с рассмотрения гибкости.

Как мы видели в главе 1, API — это особый вид интерфейса, который создается в первую очередь для взаимодействия компьютерных систем. Несмотря на то что наличие программного доступа к системе очень ценно, он существенно повышает хрупкость в том плане, что изменения интерфейса могут легко вызвать сбой у его пользователей. К примеру, изменение названия поля в API вызовет сбой во всем коде, написанном пользователями до этого изменения. С позиции сервера API этот старый код будет обращаться к чему-то, используя имя, которого уже не существует. Данный сценарий не столь актуален для других видов интерфейсов, таких как *графические интерфейсы пользователя* (graphical user-interfaces, GUI), которые используются в основном людьми, а не компьютерами, в результате чего оказываются намного более устойчивыми к изменениям. Это означает, что, даже если изменение может оказаться раздражающим или эстетически некрасивым, оно вряд ли приведет к катастрофическому сбою, который бы сделал использование интерфейса в принципе невозможным. Например, изменение цвета области или кнопки на веб-странице может сделать макет некрасивым и неудобным, но мы все равно сможем разобраться, как работает интерфейс и как с его помощью достичь нужного нам результата.

Нередко это свойство интерфейсов называют *гибкостью*, подразумевая, что те из них, где пользователи могут свободно приспосабливаться к изменениям, являются *гибкими*, а те, где даже небольшие корректировки (например, переименование поля) приводят к полному сбою — *ригидными*. Это различие важно, поскольку возможность вносить множество изменений в значительной степени определяется гибкостью интерфейса. Более того, мы можем видеть, что ригидные интерфейсы, как это бывает и в других программных проектах, существенно усложняют итеративное продвижение к идеальному дизайну. Это означает, что мы зачастую оказываемся в тупике со всеми решениями дизайна, как хорошими, так и плохими. Здесь можно подумать, что тогда из-за ригидности API никогда не удастся использовать итеративный процесс разработки. Но благодаря еще одному важному свойству интерфейсов — видимости — это не всегда оказывается так.

Как правило, большинство интерфейсов можно разделить на две категории: те, которые пользователи могут просматривать и с которыми могут взаимодействовать (в ПО обычно зовутся *фронтендом*), и недоступные им (*бэкенд*). К примеру, открыв браузер, мы можем легко наблюдать графический интерфейс Facebook. Однако у нас нет возможности видеть, как Facebook хранит наш профиль в этой соцсети и другие данные. Если использовать для этого свойства видимости более формальные выражения, можно говорить, что фронтенд (часть, доступная пользователям для просмотра и взаимодействия) обычно считается *публичным*, а бэкенд (видимый только небольшой внутренней группе ресурса) — *закрытым*. Это различие важно тем, что отчасти определяет

нашу возможность вносить изменения в разные виды интерфейсов, в частности в ригидные, такие как API.

Если мы вносим изменение в публичный интерфейс, то оно станет видимо для всего мира и потенциально окажет на него влияние. Ввиду столь обширной аудитории бездумное изменение может вызвать злость и раздражение у пользователей. При том, что этот факт однозначно касается ригидных интерфейсов, он относится и к гибким. К примеру, во времена зарождения Facebook большинство значительных функциональных или дизайн-изменений вызывали у студентов колледжей гнев, длящийся до нескольких недель. Но что, если интерфейс не публичный? Насколько сложно вносить изменения в бэкенд-интерфейсы, видимые только членам внутренней группы людей? В этом сценарии количество подверженных изменениям людей существенно меньше и, возможно, даже ограничено участниками одной команды или офиса, так что здесь мы более свободны. И это прекрасно, поскольку означает, что у нас должна быть возможность быстро продвигаться к идеальному дизайну, попутно применяя принципы гибкой разработки.

Так в чем же особенность API? Оказывается, проектируя множество API (которые ригидны по определению) и делясь ими с миром, мы получаем худший сценарий для обоих вариантов. Это означает, что вносить изменения будет даже сложнее, чем в случае любой другой комбинации этих двух свойств (обобщенно представлено в табл. 2.2).

Таблица 2.2. Сложность изменения различных интерфейсов

Гибкость	Аудитория	Пример интерфейса	Сложность изменения
Гибкий	Закрытый	Консоль внутреннего мониторинга	Очень просто
Гибкий	Публичный	Facebook.com	Нужно постараться
Ригидный	Закрытый	API внутреннего хранилища фотографий	Трудно
Ригидный	Публичный	Публичный Facebook API	Очень трудно

Проще говоря, этот «вдвойне худший» сценарий (и ригидный, и сложный для изменения) делает многократно используемые и проверенные паттерны проектирования еще более важными при построении API, чем при создании других видов ПО. В то время как в большинстве программных проектов код чаще всего оказывается закрыт и недоступен для просмотра, дизайн-решения при создании API всегда оказываются видны всем пользователям сервиса. И поскольку это серьезно ограничивает нашу возможность вносить постепенные доработки, то использование существующих, прошедших проверку временем паттернов оказывается очень ценным фактором, позволяющим наладить все с первого раза, а не в конечном итоге, как это бывает в большинстве ПО.

Теперь, когда мы изучили ряд причин для использования паттернов проектирования, пора углубиться в их суть, разобрав все их компоненты.

2.3. АНАТОМИЯ ПАТТЕРНА ПРОЕКТИРОВАНИЯ API

Как и многое в разработке ПО, паттерны проектирования API состоят из нескольких компонентов, каждый из которых отвечает за отдельный аспект использования самого паттерна. Очевидно, что главный компонент отвечает за принцип его действия, но есть и другие компоненты, которые нацелены на не технические стороны. К ним относятся, например, определение, существует ли паттерн для интересующего нас набора задач, понимание, подойдет ли он для решаемой задачи, а также выяснение, почему он делает все именно таким образом, а не использует (возможно, более простую) альтернативу.

Разобраться во всех тонкостях может быть довольно трудно, поэтому представим, что создаем сервис хранения данных и его клиентам нужен API, с помощью которого они смогут получать из него свои данные. Данный сценарий мы возьмем за основу при изучении каждого компонента паттерна, начав с самого первого: его имени.

2.3.1. Имя и аннотация

Каждый паттерн проектирования в каталоге имеет уникальное имя. Оно должно быть достаточно описательным, чтобы раскрывать значение паттерна, но при этом не слишком длинным. К примеру, при описании паттерна, используемого в нашем сценарии экспорта данных, мы можем назвать его «Паттерн импорта, экспорта, резервирования, восстановления, снимков системы и отката», но все же лучшим вариантом будет «Паттерн ввода/вывода» либо «Паттерн IO».

Несмотря на то что для понимания и определения паттерна обычно достаточно его имени, иногда оно не полностью объясняет решаемую им задачу. Поэтому для того чтобы обеспечить паттерну сжатое и простое представление, к нему также прилагается краткое описание задачи, которую он решает. К примеру, можно сказать, что паттерн ввода/вывода «предлагает структурированный способ перемещения данных к различным ресурсам хранения и адресатам, а также от них». Если коротко, то общая цель этого раздела паттерна — научить вас быстро определять, стоит ли продолжать изучать его на предмет того, подходит ли он для решения поставленной задачи.

2.3.2. Зачем он нужен

Поскольку цель паттерна проектирования API — предоставить решение для определенной категории задач, то начать стоит с определения области задачи, которую мы собираемся решать. Раздел мотивации подробно рассматривает поставленную задачу, наглядно показывая, почему для нее вообще требуется паттерн. Это означает, что сначала нам нужна ее подробная формулировка, которая обычно выражает цели пользователя. В случае нашего примера с экспортом

данных может сложиться сценарий, в котором пользователь «хочет экспортировать какие-то данные из сервиса во внешнюю систему хранения».

Далее нам нужно более точно разобраться в том, что может потребоваться пользователям. Например, мы можем выяснить, что им нужно экспортировать данные в несколько систем хранения, а не только в Amazon S3. Они также могут захотеть выполнять с экспортируемыми данными определенные действия, например сжатие или шифрование. Эти требования окажут непосредственное влияние на паттерн проектирования, поэтому важно сформулировать детали решаемой с его помощью задачи.

Затем, когда мы более полно поймем потенциальные требования пользователей, нужно будет изучить пограничные случаи, которые могут возникнуть в ходе фактической реализации. К примеру, нам нужно понять, как система должна вести себя, когда данные оказываются слишком большими (и какой объем считать слишком большим). Кроме того, необходимо изучить, как система должна реагировать в случае сбоя. Например, когда процедура экспорта дает сбой, необходимо описать, нужно ли ее повторять. Эти нетипичные сценарии в действительности окажутся гораздо более распространенными, чем можно ожидать, и несмотря на то, что мы можем не сразу решить, как обрабатывать каждый из них, очень важно, чтобы паттерн учитывал эти пробелы, позволяя заполнить их позднее.

2.3.3. Обзор

Теперь мы приближаемся к интересной части: попробуем объяснить решение, рекомендуемое паттерном проектирования для данной области задачи. Здесь мы фокусируемся уже не на определении задачи, а на предложении описания ее решения. Это означает, что мы начинаем изучать сам подход и используемые в нем методы. В нашем сценарии этот раздел может пояснять различные компоненты и их ответственность. К примеру, один компонент позволяет описать подлежащие экспорту данные, второй — систему хранения, в которую эти данные экспортируются, а третий — настройки шифрования и сжатия, применяемые перед отправкой.

Во многих случаях определение задачи и список требований к решению будут определять его общую структуру. В таких ситуациях цель обзора заключается в том, чтобы независимо от степени очевидности решения выделять данную структуру явно, а не формировать ее позднее на основе описания задачи. К примеру, если мы определяем паттерн для поиска по списку ресурсов, то будет весьма очевидным наличие параметра запроса. Тем не менее прочие аспекты (такие как формат этого параметра или гарантии согласованности поиска) могут не быть очевидными и требовать дополнительного пояснения. Ведь даже очевидные решения могут иметь тонкие нюансы, заслуживающие внимания.

В других случаях даже при четко определенной задаче может не быть одного очевидного решения. Вместо этого иногда наблюдается несколько разных вариантов, каждый из которых может нести собственные компромиссы. К примеру, есть много способов, позволяющих моделировать в API связи в формате «многие ко многим», и каждый из них имеет как достоинства, так и недостатки. Однако важно учитывать, что API выбирает один вариант и применяет его последовательно. В подобных случаях обзор позволяет разобрать каждый вариант и используемую рекомендованным паттерном стратегию. Этот раздел может содержать краткое рассмотрение плюсов и минусов других возможных вариантов, но большая часть анализа будет отведена перечислению компромиссов в конце описания паттерна.

2.3.4. Реализация

Мы подобрались к важнейшему элементу каждого паттерна проектирования: реализации. Здесь нам нужно глубоко понять область задачи и сформировать представление общей тактики и стратегии. Наиболее важной составляющей этого раздела будут представленные в виде кода определения интерфейсов, объясняющие, как будет выглядеть API, использующий данный паттерн. Эти определения API фокусируются на структуре ресурсов и конкретных способах взаимодействия с ними. Сюда будет входить ряд компонентов, таких как присутствующие в ресурсах или запросах поля, формат данных, которые могут использоваться в этих полях (например, строки в кодировке Base64), а также способ, которым эти ресурсы связаны (например, иерархическая связь).

Во многих случаях объяснить работу API только с помощью его представления и самих определений полей не получится. Иначе говоря, в то время как структура и список полей могут казаться понятными, поведение этих структур и взаимодействие разных полей могут быть куда более сложными. В таких ситуациях нам потребуется более подробный разбор этих неочевидных аспектов дизайна. К примеру, при экспорте данных можно указать способ их сжатия перед помещением в сервис хранения, обозначив алгоритм шифрования в специальном строковом поле. В такой ситуации паттерн может разъяснять возможные значения для данного поля (он может использовать тот же формат, что и в HTTP-заголовке Ассерпт-Encoding), действия при передаче недействительного варианта (например, возвращение ошибки), а также последствия в случае оставления поля пустым (например, использование предустановленного сжатия `gzip`).

Наконец, этот раздел будет включать пример определения API с комментариями, объясняющими, как должен выглядеть API, правильно реализующий данный паттерн. Все это будет определено в коде с комментариями, разъясняющими поведение различных полей, и будет опираться на конкретный пример сценария, который иллюстрирует решаемую паттерном задачу. Этот раздел чаще всего будет длиннее остальных и будет содержать больше подробностей.

2.3.5. Компромиссы

К этому моменту мы уже понимаем, что нам дает этот паттерн проектирования, но еще необходимо прояснить, чего он не дает сделать, так как это тоже очень важно. Честно говоря, бывают нюансы, которые при реализации паттерна задуманным для него способом оказываются невозможными. В подобных случаях очень важно понимать, на какие жертвы необходимо пойти, чтобы получить выгоду от использования данного паттерна. Возможности здесь оказываются довольно обширными, простираясь от функциональных ограничений (например, нельзя экспортировать данные напрямую через скачивание в браузере) до повышенной сложности (например, приходится набирать намного больше текста, чтобы описать, куда вы хотите отправить данные), а также более технических аспектов наподобие согласованности данных (например, вы можете видеть данные, которые уже немного устарели, не будучи при этом уверенными, действительно ли это так). Поэтому описание компромиссов может быть представлено как простыми объяснениями, так и подробным разбором тонкостей ограничений при использовании конкретного паттерна проектирования.

Кроме того, даже когда определенный паттерн будет идеально подходить под область вашей задачи, наверняка будут существовать сценарии, в которых он окажется не самым лучшим решением. В таких случаях важно понимать последствия его использования именно в данном контексте, когда это вроде бы и подходящий паттерн, но не идеальный. Этот раздел паттерна будет прояснять последствия подобных легких отклонений.

Теперь, когда мы разобрались, как структурируются и объясняются паттерны проектирования API, пора выяснить, какой вклад они могут внести при создании предположительно простого программного интерфейса.

2.4. ПРАКТИЧЕСКОЕ ИССЛЕДОВАНИЕ: TWAPI, TWITTER-ПОДОБНЫЙ API

Twitter можно условно представить как виртуальное место, в котором вы можете обмениваться с другими людьми короткими сообщениями. Немного пугает мысль о том, что весь бизнес построен исключительно на обмене сообщениями, но, оказывается, этого достаточно для развития технологической компании с многомиллиардными доходами. О чем здесь не упоминается, так это о том, что даже при использовании чрезвычайно простого принципа «под капотом» прячется довольно запутанная и сложная схема. И чтобы лучше это понять, мы начнем с изучения принципа работы Twitter-подобного API, который назовем Twapi.

2.4.1. Обзор

При создании Twarі наша главная ответственность состоит в том, чтобы предоставить людям возможность размещать собственные сообщения и просматривать сообщения, размещенные другими участниками. Снаружи все это выглядит довольно просто, но есть ряд скрытых подвохов, которые нужно учитывать. Сначала предположим, что у нас есть простой вызов API, позволяющий создать сообщение Twarі. Затем мы рассмотрим два дополнительных действия, которые могут пригодиться этому API: перечисление множества сообщений и экспорт всех сообщений в другую систему хранения.

Прежде чем продолжать, нужно обратить внимание на пару важных нюансов. Во-первых, это будет лишь пример API. То есть основной упор будет делаться на рассмотрение способа определения интерфейса, а не на разбор реализации. На языке программирования это подразумевает, что мы будем говорить только об определении функции, а заполнением ее тела займемся позже. Во-вторых, это будет наш первый опыт знакомства с определением API. Если вы не читали раздел «О книге» (см. выше), то сейчас самое время сделать это, чтобы формат подачи с помощью TypeScript не стал для вас слишком неожиданным.

Ну а теперь перейдем непосредственно к созданию списков сообщений в Twarі.

2.4.2. Создание списка сообщений

Имея возможность создавать сообщения, мы наверняка захотим объединять их в списки. Кроме того, мы также заинтересованы видеть сообщения наших друзей, и если заглянуть чуть дальше, то нам может потребоваться возможность видеть длинный список избранных популярных сообщений наших друзей (своего рода лента новостей). Начнем с определения простого метода API, предназначенного для реализации этого, пока без каких-либо паттернов.

Без паттернов проектирования

Изначально нам нужно отправить запрос на создание списка из нескольких сообщений. Для этого необходимо знать, чьи сообщения нас интересуют. Назовем его `parent`. В ответе нам нужно, чтобы API присылал простой список сообщений. Все это взаимодействие отражено на рис. 2.1.

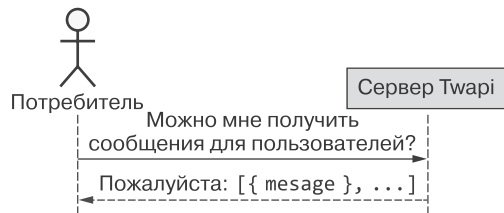


Рис. 2.1. Простой поток запроса сообщений Twarі

Разобравшись с потоком действий, позволяющим перечислить эти сообщения, нужно оформить его в реальное определение API (листинг 2.1).

Листинг 2.1. Пример API, перечисляющего сообщения Twapi

```

Сначала мы определяем сервис API
как абстрактный класс. Это просто коллекция
методов API, определенных как функции TS
abstract class Twapi {
  static version = "v1";
  static title = "Twapi API";

  @get("/{parent=users/*/}/messages")
  listMessages(req: ListMessagesRequest): ListMessagesResponse;
}

Для хранения метаданных об API можно
использовать статические переменные
TypeScript, такие как name или version

Здесь с помощью специальных оберточных
функций мы определяем HTTP-метод (GET)
и URL-паттерн (/users/<user-id>/messages),
который должен отображаться в эту функцию

Здесь функция ListMessages принимает
ListMessagesRequest и возвращает
ListMessagesResponse

ListMessagesRequest получает
один параметр: parent. Это владелец
сообщений, которые мы хотим перечислить

ListMessagesResponse возвращает простой
список сообщений, принадлежащих
указанному пользователю
interface ListMessagesRequest {
  parent: string;
}

interface ListMessagesResponse {
  results: Message[];
}
  
```

Как видите, это определение API достаточно простое. Оно получает один параметр и возвращает список соответствующих сообщений. Но теперь представим, что развертываем его в качестве нашего API, и учтем одну из серьезнейших проблем, которая наверняка возникнет с течением времени: огромные объемы данных.

По мере подключения к сервису все большего количества людей данный список сообщений станет очень длинным. Это может не вызывать сложностей, пока ответы будут нести десятки или сотни сообщений, но что, если их количество начнет достигать тысяч, сотен тысяч и даже миллионов? Единичный HTTP-ответ с 500 000 сообщений, каждое из которых содержит до 140 символов, подразумевает вплоть до 70 Мбайт данных. Для типичного пользователя API это может оказаться чересчур много, не говоря уже о том, что один HTTP-запрос будет приводить к пересылке сервером БД Twapi 70 Мбайт данных.

Что же можно сделать? Очевидным ответом будет поручить API разбивать потенциально крупные ответы на мелкие части и позволить пользователям запрашивать эти части поочередно. Для этого можно использовать паттерн пагинации (см. главу 26).

Паттерн пагинации

Как мы узнаем в главе 26, этот паттерн предоставляет способ извлечения длинного списка элементов в виде небольших, более удобных для обработки фрагментов данных, а не целиком за раз. Паттерн опирается на использование

дополнительных полей в запросе и ответе. При этом выглядеть они будут довольно просто. Общий поток действий показан на рис. 2.2.

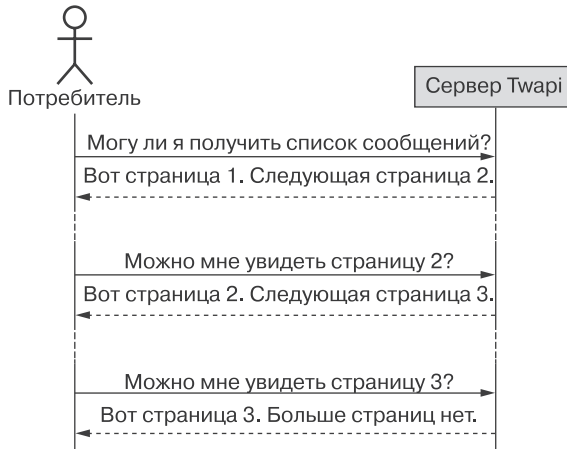


Рис. 2.2. Пример потока действий паттерна пагинации для извлечения сообщений Twapi

Вот как это выглядит в самом определении API (листинг 2.2).

Листинг 2.2. Пример API, перечисляющего сообщения Twapi с помощью пагинации

```

abstract class Twapi {
    static version = "v1";
    static title = "Twapi API";

    @get("/{parent=users/*/}/messages")
    ListMessages(req: ListMessagesRequest): ListMessagesResponse;
}

interface ListMessagesRequest {
    parent: string;
    pageToken: string;
    maxPageSize: number?;
}

interface ListMessagesResponse {
    results: Message[];
    nextPageToken: string;
}
  
```

Заметьте, что определение метода остается прежним; здесь изменения не требуются

Чтобы прояснить, какая часть (страница) данных нам нужна, мы добавляем в запрос параметр `pageToken`

Мы также добавляем способ, позволяющий указать максимальное количество сообщений, которые сервер Twapi должен возвращать в одной части

Ответ сервера Twapi будет включать способ получения очередной части сообщений

Что произойдет, если не начать с этого паттерна

Благодаря этим скромным изменениям в сервисе API мы фактически создали метод API, способный выдержать резкий рост количества сообщений Twapi, но оставили без ответа очевидный вопрос: зачем озадачиваться реализацией данного

паттерна в самом начале? Почему бы не добавить эти поля позже, когда возникнет проблема? Иначе говоря, почему нам нужно беспокоиться об исправлении чего-либо, что еще не сломалось? Как вы узнаете позже при изучении обратной совместимости, причина в том, чтобы просто избежать сбоя в существующем ПО.

В данном случае переход от нашего более простого исходного дизайна (отправки всех данных в одном ответе) к паттерну пагинации (разделению данных на небольшие куски) может казаться безобидным действием, но в реальности это приведет к нарушению работы уже имеющейся функциональности. При таком сценарии существующий код будет предполагать, что все запрошенные данные содержатся в одном ответе, а не разбиваются на части. В свою очередь, это приведет к двум серьезным проблемам.

Во-первых, поскольку ранее написанное ПО ожидает, что все данные вернутся в одном ответе, в нем нет способа найти те данные, которые показываются на последующих страницах. В результате код, написанный до изменения, по сути, оказывается отрезан от всего, кроме первого куска данных, что ведет ко второй проблеме.

Во-вторых, поскольку существующие потребители понятия не имеют, как получить дополнительные куски данных, они считают, что получили все данные, но на деле имеют лишь небольшую часть. Это недопонимание может привести к сложно обнаруживаемым ошибкам. К примеру, потребители, пытающиеся вычислить среднее значение некоторых данных, могут получить величину, которая выглядит точной, но на деле является средней лишь по первой части возвращенных данных. Это наверняка приведет к принятию ошибочного значения, но очевидной ошибки не вызовет. В результате такой баг может довольно долго оставаться незамеченным.

Теперь, разобравшись с примером перечисления сообщений, пора рассмотреть, что может подтолкнуть нас к использованию паттерна при экспорте данных.

2.4.3. Экспорт данных

В определенный момент пользователи сервиса Twarі могут захотеть экспортировать свои сообщения. Аналогично случаю с их перечислением, сначала нам нужно учесть сценарий, в котором количество экспортируемых данных становится очень большим (возможно, сотни мегабайт). Далее, в отличие от вывода списка сообщений, нам следует учесть, что в качестве получателей данных здесь могут выступать многие сервисы хранения и в идеале нам нужен способ, позволяющий регулярно интегрировать новые, когда они набирают популярность. Кроме того, мы можем захотеть применять к данным различные трансформации перед отправкой, например, шифровать их, сжимать или анонимизировать некоторые части. Наконец, все это вряд ли будет работать хорошо в синхронном режиме, то есть нам потребуется способ, указывающий на наличие незавершенной работы,

выполняющейся в фоновом режиме (фактического экспорта данных), чтобы потребитель мог мониторить ее прогресс.

Начнем с создания простой реализации для этого API и рассмотрим проблемы, которые могут возникнуть в дальнейшем.

Без паттерна проектирования

Как уже говорилось, нам нужно решить ряд основных вопросов: огромные объемы данных, конечная точка их отправки, различные трансформации или конфигурации данных (например, сжатие или шифрование) и, наконец, асинхронная природа API. Поскольку мы создаем лишь базовый API для экспорта сообщений Twpapi (листинг 2.3), то самым простым вариантом, охватывающим большинство из перечисленных аспектов, будет реализация сжатия файла для последующего скачивания. Если кратко, то суть в том, что при выполнении пользователем запроса к API ответ не будет содержать сами данные. Вместо этого он будет содержать указатель на место, откуда их можно будет скачать через какое-то время.

Листинг 2.3. Простой API для экспорта сообщений

```
abstract class Twpapi {
    static version = "v1";
    static title = "Twpapi API";

    @post("/{parent=users/*/messages:export}")
    ExportMessages(req: ExportMessagesRequest):
        ExportMessagesResponse;
}

interface ExportMessagesRequest {
    // Родительский пользователь экспортируемых сообщений
    parent: string;
}

interface ExportMessagesResponse {
    // Расположение сжатого файла
    // с запрошенными сообщениями
    exportDownloadUri: string;
}
```

В URI, с которым мы делаем сопоставление, используется HTTP-глагол POST и особый синтаксис, сигнализирующий о том, что выполняется особое действие, а не одно из стандартных действий REST

Так же как и в предыдущих примерах, мы опираемся на функцию ExportMessages, которая получает запрос и возвращает ответ

Мы хотим разрешить экспорт данных только для одного пользователя одновременно, поэтому область экспортирующего метода ограничивается одним родителем (пользователем)

В ответе сервера Twpapi указывается расположение сжатого файла для дальнейшего скачивания с простого файлового сервера, а не из сервиса API

Этот API реализует главную задачу (экспорт данных) и несколько вторичных (асинхронное извлечение), но упускает ряд важных аспектов. Начнем с того, что у нас нет способа определения дополнительной конфигурации использованных данных. К примеру, у нас нет возможности выбирать формат сжатия или ключи и алгоритм для их шифрования. Кроме того, у нас также не было возможности выбрать адрес отправки данных. Вместо этого нам просто сказали, где их можно будет получить позже. Наконец, если мы посмотрим внимательнее, то станет очевидно, что асинхронная природа интерфейса играет нам на руку лишь частично: при том, что сервис действительно асинхронно возвращает расположение, откуда можно будет скачать данные, у нас нет способа мониторить

прогресс операции экспорта, а также возможности отменить ее в том случае, если мы вдруг передумаем.

Посмотрим, можно ли улучшить имеющийся дизайн, используя несколько паттернов, которые будут определены позднее в нашем каталоге. Основное внимание мы уделим паттерну импорта/экспорта.

Паттерн импорта/экспорта

Как мы узнаем в главе 28, этот паттерн предназначен для задач, аналогичных текущей: у нас есть некий объем данных в сервисе API, а потребителям нужен способ их получить (или добавить). Тем не менее, в отличие от паттерна пагинации, этот паттерн в реализации своей задачи будет опираться на другие, такие как паттерн длительных операций (рассматриваемый в главе 13). Начнем с определения API (листинг 2.4), после чего более детально рассмотрим, как работают все его части. Как и прежде, имейте в виду, что мы не будем углубляться во все подробности каждого аспекта паттерна, а просто сделаем общий обзор его актуальных частей.

Листинг 2.4. API для экспорта сообщений с помощью паттерна проектирования

```
abstract class Twapi {
    static version = "v1";
    static title = "Twapi API";

    @post("/{parent=users/*/}/messages:export")
    ExportMessages(req: ExportMessagesRequest):
        Operation<ExportMessagesResponse,
            ExportMessagesMetadata>;
}

interface ExportMessagesRequest {
    // родительский пользователь экспортируемых сообщений
    parent: string;
    outputConfig: MessageOutputConfig;
}

interface MessageOutputConfig {
    destination: Destination;
    compressionConfig?: CompressionConfig;
    encryptionConfig?: EncryptionConfig;
}

interface ExportMessagesResponse {
    outputConfig: MessageOutputConfig;
}

interface ExportMessagesMetadata {
    // целое между 0 и 100,
    // отражающее прогресс операции
    progressPercent: number;
}
```

В отличие от предыдущих примеров, возвращаемый тип нашего метода `ExportMessages` является длительно выполняющейся операцией, которая по завершении возвращает `ExportMessagesResponse` и сообщает метаданные об операции, используя интерфейс `ExportMessagesMetadata`.

Помимо родителя (пользователя), `ExportMessagesRequest` также дополнительно получает конфигурацию выходных данных.

Здесь мы определяем «место назначения», где окажутся данные после завершения операции.

Кроме того, можно настроить способ сжатия или шифрования данных, используя отдельные объекты конфигурации.

При выводе данных в назначенное место результат будет отражать использованную конфигурацию.

`ExportMessagesMetadata` будет содержать информацию об операции, например прогресс (в процентах).

Чем же хорош этот паттерн? Сначала за счет применения инкапсулированного интерфейса выходной конфигурации мы можем получать различные параметры во время запроса, после чего повторно использовать это же содержимое в ответе в качестве подтверждения для потребителя. Далее внутри этой конфигурации можно определить несколько разных параметров, которые мы более детально разберем в листинге 2.5. Наконец, у нас есть возможность отслеживать прогресс операции экспорта, используя метаданные длительно выполняющегося процесса, которые хранят прогресс в процентах (0 % = «не начался», а 100 % = «завершено»).

При всем при этом вы могли заметить, что здесь не были определены несколько строительных блоков, которые мы использовали в предыдущем определении API. Разберемся, как они выглядят, предложив несколько вариантов конфигураций (листинг 2.5).

Листинг 2.5. Интерфейсы для настройки мест назначения и конфигураций

```
interface Destination {
    typeId: string;
}
interface FileDestination extends Destination {
    // Загруженный файл будет размещен на наших серверах.
    fileName: string;
}

interface AmazonS3Destination extends Destination {
    // Потребуется доступ к записи в корзину
    // и префикс объекта, выданный аккаунту
    // AWS с ID 1234-5678-1234.
    uriPrefix: string;
}

interface CompressionConfig {
    typeId: string;
}

interface GzipCompressionConfig {
    // целое между 1 и 9
    compressionLevel: number;
}

interface EncryptionConfig {
    // Здесь можно разместить всевозможные
    // настройки шифрования либо расширить
    // интерфейс аналогично CompressionConfig
}
```

Так же как в исходном примере, здесь можно определить место назначения на файловом сервере, откуда файл можно будет скачать позднее

Помимо примера скачивания файла, можно также запросить, чтобы данные были сохранены в Amazon Simple Storage Service (S3)

Здесь определяются различные параметры сжатия и настройки для каждого из них

Можно либо определить все параметры настройки шифрования в одном интерфейсе, либо прописать их с помощью такой же структуры с подклассом, как в случае сжатия

Здесь мы видим различные способы определения настройки точки отправки данных, их сжатия и шифрования. Теперь осталось лишь понять, как именно работает этот функционал длительной операции. Более подробно этот паттерн мы разберем в главе 28, а сейчас лишь взглянем на простое определение API для

этих интерфейсов, чтобы получить хотя бы общее представление об их работе (листинг 2.6).

Листинг 2.6. Определение интерфейсов типичной ошибки и длительной операции

```
interface OperationError {
    code: string;
    message: string;
    details?: any;
}

interface Operation<ResultT, MetadataT> {
    id: string;
    done: boolean;
    result?: ResultT | OperationError;
    metadata?: MetadataT;
}
```

В качестве одного из основных компонентов нам понадобится интерфейс ошибки, который (по меньшей мере) будет представлять код ошибки и сообщение. Сюда также можно включить поле со множеством прочих подробностей о возникшей ошибке

Длительная операция — это структура по типу промиса, которая параметризуется на основе результата и типов метаданных (аналогично универсальным шаблонам в Java)

В отличие от других интерфейсов, специфичных для экспорта сообщений, эти (*Error* и *Operation*) являются более общими и могут распространяться по всему API. Другими словами, можно смело рассматривать их как более универсальные строительные блоки, а не такие, которые служат конкретной функции.

Что будет, если начать без паттерна

В отличие от предыдущего примера, где варианты на основе паттерна и без него выглядели довольно похожими, в этом сценарии подобные взаимные альтернативы в итоговой поверхности API различаются уже значительно. В результате ответ на этот вопрос будет прост: если вы позднее вдруг ощутите необходимость в предоставленной функциональности (различных точках экспорта, отдельных конфигурациях и т. д.), то заложенный в основу подход без паттерна приведет к нарушающим работу потребителей изменениям. Если же начать решение этой задачи с подхода на основе паттернов, то по мере возникновения новой функциональности API будет развиваться гармонично.

РЕЗЮМЕ

- Паттерны проектирования API представляют собой в некотором роде адаптируемые чертежи, предназначенные для разработки и структуризации API.
- Важность паттернов проектирования заключается в их способности минимизировать обширные структурные изменения в API, которые по своей природе являются ригидными и изменяются с трудом.
- Используемые в течение книги паттерны проектирования API будут иметь несколько разделов, включая имя и аннотацию, предлагаемые правила, мотивацию, обзор, реализацию и компромиссы использования того или иного паттерна в сравнении с пользовательскими альтернативами.

Часть II

Принципы проектирования

В нашем наборе инструментов проектирования будут находиться не только паттерны. Следующие несколько глав будут посвящены ряду тем, которые не слишком хорошо вписываются в понятие паттернов проектирования (согласно определению главы 2), но все равно оказываются важны и полезны в деле создания хороших API (согласно определению главы 1). В главе 3 мы рассмотрим руководства по именованию различных компонентов API. Затем в главе 4 научимся логически упорядочивать эти компоненты и определять их взаимные связи. Завершит этот отрезок пути глава 5, в которой мы узнаем, как выбирать между множеством типов данных, доступных для разных полей, которые могут понадобиться в наших API.

3

Именование

В этой главе

- ✓ Почему важно уделять внимание именам.
- ✓ Чем одни имена лучше других.
- ✓ Как выбирать язык, грамматику и синтаксис.
- ✓ Как контекст влияет на значение имени.
- ✓ Практическое исследование последствий неудачного выбора имен.

Нравится нам это или нет, но имена повсюду. В каждой системе ПО, которую мы создаем, и каждом API, который проектируем или используем, образуется множество разных имен, которые будут жить намного дольше, чем мы планируем. В связи с этим становится очевидной важность правильного выбора этих имен (даже если мы не всегда придаем ему достаточно внимания). В этой главе мы изучим различные именуемые компоненты API, ряд стратегий, позволяющих выбрать хорошие имена, и несколько общих принципов, которые помогут в случае трудных решений — а такие неизбежно будут возникать.

3.1. ПОЧЕМУ ВАЖНЫ ИМЕНА

В целом в мире разработки ПО практически невозможно избежать выбора имен для всевозможных компонентов. Если бы это было возможно, то нам бы потребовался способ писать фрагменты кода, используя исключительно ключевые слова языка (например, `class`, `for` или `if`), что привело бы в лучшем случае к нечитаемым программам. С учетом этого скомпилированное ПО является особым случаем. Дело в том, что в традиционном скомпилированном коде имена функций и переменных важны лишь для тех, кто имеет доступ к исходному коду, поскольку само имя в процессе компиляции (или минификации) и развертывания обычно исчезает.

А вот в случае проектирования и создания API выбираемые имена оказываются намного важнее, так как с ними будут взаимодействовать пользователи. Иначе говоря, эти имена не исчезнут вследствие компиляции и не окажутся скрытыми от внешнего мира. Это означает, что нам при выборе имен для API необходимо уделять этому особое внимание.

Здесь возникает резонный вопрос: нельзя ли просто изменить имя, если оно окажется неудачным? Как мы узнаем в главе 24, изменение имен в API может оказаться весьма проблематичным. Представьте, что часто меняете имя активно используемой функции в исходном коде, после чего понимаете, что теперь нужно проделать огромную работу по поиску и замене, чтобы обновить все ссылки на данное имя функции. Хотя это и неудобно, но все-таки возможно (а в некоторых IDE даже просто). Однако подумайте о случае, когда этот исходный код доступен публично для разработки на его основе отдельных проектов. Даже если вы каким-то образом сможете обновить все ссылки на весь доступный публичный исходный код, всегда будет оставаться закрытый, к которому у вас не окажется доступа.

Если пояснить несколько иначе, то изменение публичных имен в API в некотором смысле сродни изменению вашего адреса или номера телефона. Чтобы успешно изменить этот номер везде, вам придется связаться с каждым, у кого когда-либо он был, включая вашу бабушку (которая может использовать бумажную адресную книгу) и каждую торговую компанию, которая когда-либо имела к нему доступ. Даже если у вас есть способ, позволяющий связаться с каждым, у кого есть ваш номер, вам еще надо добиться от них изменения контактной информации о вас, на что не у всех может быть время.

Выяснив, почему важно выбирать хорошие имена (и избегать их изменения), логично ответить на важный вопрос: что делает имя «хорошим»?

3.2. ЧТО ДЕЛАЕТ ИМЯ «ХОРОШИМ»

Как мы узнали в главе 1, API считаются «хорошими», когда им присущи функциональность, выразительность, простота и предсказуемость. Имена же в этом довольно похожи, за исключением того факта, что не обязательно являются функциональными (иначе говоря, имя ничего не делает). Далее мы рассмотрим подмножество атрибутов и несколько примеров выбора имен, начав с выразительности.

3.2.1. Выразительность

Самое важное в имени — его способность четко передавать значение элемента, которому оно присвоено. Этот элемент может быть функцией или RPC (например, `CreateAccount`), ресурсом или сообщением (например, `WeatherReading`), полем или свойством (например, `postal_address`) либо чем-то совсем другим, тем же значением перечисления (например, `Color.BLUE`), но имя должно четко доносить до читателя значение того, что представляет. Это может показаться простым, но часто бывает сложно взглянуть на имя свежим взглядом, забыв о контексте, который мы сформировали за время работы в конкретной области. Этот контекст обычно является солидным ресурсом, но в данном случае он выступает скорее недостатком: из-за него мы выбираем неудачные имена.

К примеру, слово «тема» зачастую используется в контексте асинхронного обмена сообщениями (в Apache Kafka или RabbitMQ). Однако оно также применяется в специфичной сфере машинного обучения и обработки естественного языка, где звучит как *моделирование темы*. Если бы вы применили термин *тема* в API машинного обучения, то его пользователи вполне могли бы впасть в замешательство, пытаясь понять, о какой теме идет речь. Если же это реально возможно (вероятно, ваш API использует и асинхронный обмен сообщениями, и моделирование темы), то вы можете выбрать имя, более выразительное, чем `topic`, например `model_topic` или `messaging_topic`, чтобы не вводить пользователей в заблуждение.

3.2.2. Простота

Выразительность для имени определенно важна, но оно может оказаться проблемным еще и из-за лишней длины, не добавляющей ему ясности. Обратимся к предыдущему примеру, где термин `topic` относился к разным областям компьютерной науки. Если окажется, что API использует только асинхронный обмен сообщениями (например, API в стиле Apache Kafka) и никак не связан с машинным обучением, то имя `topic` будет достаточно ясным и простым, а вот

`messaging_topic` дополнительной ценности уже не добавит. Если кратко, то имена должны быть выразительными, но только до той степени, в какой каждая часть имени оправдывает свое присутствие.

С другой стороны, слишком упрощать имена тоже не стоит. Представьте, что у нас есть API, который должен сохранять заданные пользователем настройки. Его ресурс может называться `UserSpecifiedPreferences`. Однако `Specified` не приносит в это имя особой пользы. И наоборот, если назвать ресурс просто `Preferences`, то станет непонятно, чьи это настройки, и в дальнейшем возникнет путаница, когда потребуется обрабатывать и сохранять системные или администраторские настройки. В этом случае `UserPreferences` выглядит золотой серединой, являясь одновременно и выразительным, и простым именем. Все это пояснение обобщенно представлено в табл. 3.1.

Таблица 3.1. Выбор между простыми и выразительными именами

Имя	Примечание
<code>UserSpecifiedPreferences</code>	Выразительное, но недостаточно простое
<code>UserPreferences</code>	Достаточно простое и выразительное
<code>Preferences</code>	Слишком простое

3.2.3. Предсказуемость

Теперь, когда мы нашли баланс между выразительностью и простотой, остается еще один важный аспект: предсказуемость. Представьте себе API, в котором имя `topic` используется для группировки схожих асинхронных сообщений (аналогично Apache Kafka). Затем представьте, что этот API в других местах использует имя `messaging_topic`, не имея особых причин для выбора того или другого. Это ведет к довольно неприятным и необычным обстоятельствам (листинг 3.1).

Листинг 3.1. Пример затрудненного кода с несогласованным именованием

```
function handleMessage(message: Message) {
  if (message.topic == "budget.purge") {
    client.PurgeTopic({
      messagingTopic: "budget.update"
    });
  }
}
```

Здесь мы используем имя `topic` для считывания темы сообщения

Здесь для выражения того же принципа мы используем имя `messagingTopic`

Если даже этот случай не покажется вам раздражающим, обратите внимание на нарушение важного принципа. По общему правилу мы должны использовать

для представления одной вещи одно имя, а для представления других — другие. Если рассмотреть этот принцип как аксиому, то возникает важный вопрос: чем `topic` отличается от `messagingTopic`? В конце концов мы использовали разные имена, значит, они должны представлять разные вещи, ведь так?

Основная цель в том, чтобы пользователи могли узнать имя и продолжить разработку, исходя из этого понимания, имея возможность прогнозировать, как будут выглядеть имена в дальнейшем (например, если они решат представить тот же концепт). Мы можем согласованно использовать `topic` во всех точках API, где имеется в виду «тема данного сообщения» (и других имен в других случаях). Так мы позволяем пользователям отталкиваться от уже известного им и не прибегать к проверке каждого имени ради того, чтобы убедиться, что его значение совпадает с предполагаемым.

Сформировав представление о характеристиках хороших имен, пора изучить общие рекомендации, которые помогут избежать проблем при именовании элементов в API. Начнем с фундаментальных аспектов языка, грамматики и синтаксиса.

3.3. ЯЗЫК, ГРАММАТИКА И СИНТАКСИС

В то время как код — это нули и единицы, фундаментально хранящиеся в виде чисел, именование является субъективной конструкцией, которую мы выражаем с помощью языка. В отличие от языков программирования, в которых имеются твердые правила о том, что валидно, а что нет, стандартный язык ориентирован в первую очередь на людей, а не компьютеры, в связи с чем имеет не столь жесткие правила. В результате процесс выбора имени может оказаться более гибким и неоднозначным, что может стать как преимуществом, так и недостатком.

С другой стороны, неоднозначность позволяет давать элементам более общие имена, расширяя их рабочий спектр. К примеру, если назвать поле не `jpeg_url`, а `image_url`, то мы не окажемся ограничены одним форматом (JPEG). Опять же, когда есть несколько способов выражения одного и того же, мы зачастую склонны использовать их попеременно, что делает именование непредсказуемым (см. подраздел 3.2.3), приводя к раздражающим и неудобным API. Чтобы в некоторой степени этого избежать, даже вопреки высокой подвижности языка мы можем с помощью установки собственных правил сохранить столь ценную для хорошего API предсказуемость. В текущем разделе мы изучим ряд простых правил языка — они помогут минимизировать количество произвольных выборов, с которыми мы сталкиваемся при именовании.

3.3.1. Язык

Несмотря на все языковое разнообразие нашего мира, наиболее распространенным в сфере разработки ПО является американский английский. Это совсем не значит, что он лучше или хуже других языков. Однако если наша цель — получить ПО, с которым сможет работать максимальное количество людей по всему миру, то любой другой язык станет скорее помехой, чем подспорьем.

Это означает, что должны использоваться понятия английского языка (например, *BookStore* вместо *Liberia*) и предпочтения должны отдаваться американскому стилю написания (например, *color* вместо *colour*). Это дает дополнительное преимущество в виде высокой согласованности с набором символов ASCII, за исключением буквально нескольких слов, которые американский английский позаимствовал из других языков (например, *café*).

Это не означает, что комментарии API должны быть именно на этом языке. Если аудитория API является исключительно французской, то будет разумным предоставить документацию на французском (которая может автоматически генерироваться из комментариев спецификации API). Однако команда инженеров, использующих этот API, скорее всего, будет использовать другие API, которые вряд ли нацелены исключительно на французских потребителей. В результате сохраняется принцип, что, даже если для аудитории API американский английский не основной язык, сам API все равно должен использовать именно его в качестве общего языка для всех участников, совместно использующих разные API.

3.3.2. Грамматика

Учитывая, что в качестве стандартного языка в API будет использоваться американский английский, возникает довольно много нюансов, поскольку это достаточно сложный язык со множеством времен и наклонений. К счастью, произношение здесь не станет проблемой, так как исходный код составляется в письменной форме, а не разговорной. Но это исключает не все потенциальные проблемы.

Вместо того чтобы пытаться диктовать, как применять каждый аспект грамматики американского английского к именованию элементов API, в этом подразделе мы затронем несколько наиболее распространенных проблем. Начнем с рассмотрения действий (например, методов RPC или глаголов RESTful).

Императивные действия

В любом API есть эквиваленты используемым в языках программирования «функциям», которые выполняют фактическую работу. Это может быть чисто RESTful API, опирающийся только на конкретный список предустановленных настроек

(Get, Create, Delete и т. д.). В таком случае делать здесь особо ничего не придется, поскольку все действия будут принимать форму `<StandardVerb><Noun>` (например, `CreateBook`). В случае не RESTful или ресурсно-ориентированных API, которые допускают нестандартные глаголы, у нас возникает уже больше вариантов для именования действий.

У стандартных глаголов REST есть один важный общий аспект: они все представлены в повелительном наклонении. Иначе говоря, они все являются командами или приказами. Если это не до конца понятно, то представьте себе армейского сержанта, который кричит вам что-то вроде: «Удали эту информацию о погоде!», «Заархивируй эту запись журнала!» Как бы нелепо эти команды ни звучали для армии, вы точно понимаете, чего от вас требовали.

С другой стороны, иногда имена функций, которые мы пишем, могут иметь изъявительное наклонение. Типичный пример — это когда функция что-то проверяет, например `String.IsNullOrEmpty()` в C#. В этом случае глагол «быть» принимает изъявительное наклонение (задавая вопрос о ресурсе), а не повелительное (приказывая сервису выполнить что-либо).

Несмотря на то что нет ничего плохого в принятии функциями такого наклонения, при использовании в веб-API это оставляет без ответа пару важных вопросов. Первый касается действий, которые выглядят так, будто могут быть обработаны без обращения к удаленному сервису: «`A isValid()` приводит к удаленному вызову или обрабатывается локально?» По нашему мнению, пользователи предполагают, что все вызовы методов происходят по сети, и наличие выполняющегося таким образом stateless-вызова может запутать.

Второй вопрос звучит следующим образом: «Как должен выглядеть ответ?» Возьмем случай с `RPC isValid()`. Должен ли он возвращать простое логическое поле, указывающее на валидность входных данных? А может, в случае недействительности этих данных он должен возвращать список ошибок? С другой стороны, `GetValidationErrors()` оказывается более понятным: он возвращает либо пустой список при полностью валидных входных данных, либо список ошибок. Здесь в отношении формы ожидаемого ответа сомнений не возникает.

Предлоги

Еще один запутывающий момент при выборе имен связан с предлогами, такими как *with*, *to* или *for*. Хотя эти слова очень распространены в повседневном английском общении, при использовании в контексте веб-API, в частности в именах ресурсов, они могут указывать на более сложные внутренние проблемы этого API.

К примеру, API библиотеки может иметь способ перечисления ресурсов `Book`. Если этому API потребуется способ, позволяющий перечислять ресурсы `Book`

и добавлять связанные с этой книгой ресурсы `Author`, то может возникнуть соблазн создать для этой комбинации новый ресурс: `BookWithAuthor` (который будет выводиться списком с помощью вызова `ListBooksWithAuthors` или чего-то подобного). На первый взгляд это может показаться нормальным, но как быть в случае, когда нам нужно перечислить ресурсы `Book` с вложенными ресурсами `Publisher`? Или и с `Author`, и с `Publisher`? Здесь нам сначала придется создать 30 разных RPC для вызова в зависимости от требуемых нам ресурсов.

В этом случае нужный нам предлог (`with`) указывает на более фундаментальную проблему: мы ищем способ, позволяющий выводить список ресурсов и включать в ответ различные атрибуты. При этом можно решить данную задачу с помощью маски или представления (см. главу 8) и одновременно избежать этого ресурса со странным именем. В данном случае предлог стал указателем на что-то не всегда верное. Поэтому, хоть предлоги и не стоит запрещать полностью (например, поле может иметь имя `bits_per_second`), эти каверзные маленькие слова оказываются своеобразным *кодом «с душком»*, намекая на присутствие некой недоработки, требующей дополнительного внимания.

Множественное число

Чаще всего мы будем выбирать имена в единственном числе, такие как `Book`, `Publisher` и `Author` (а не `Books`, `Publishers` или `Authors`). В дальнейшем выбранные имена будут обретать в API новые значения и цели. Например, на ресурс `Book` где-то может быть создана ссылка из поля `Author.favoriteBook` (см. главу 13). Тем не менее, когда речь идет о множестве этих ресурсов, порой возникает дополнительная путаница. Если API использует RESTful URL, то имя коллекции из нескольких ресурсов почти всегда прописывается во множественном числе. К примеру, когда мы запрашиваем один ресурс `Book`, имя коллекции в URL почти однозначно будет выглядеть примерно так: `/books/1234`.

В случае имен, использованных нами в качестве примеров (например, `Book`), это особой проблемы не создает, так как упоминание нескольких ресурсов `Book` требует лишь добавления `s`, превращающего имя во множественное `Books`. Однако некоторые имена оказываются более сложными. Представьте, что мы создаем API для кабинета ортопеда. Когда у нас есть ресурс `Foot`, нам приходится нарушить этот паттерн простого добавления `s`, получив коллекцию `feet`.

Данный пример, конечно, нарушает паттерн, но он хотя бы ясен и однозначен. А что, если наш API относится к работе с людьми и имеет ресурс `Person`? Как тогда будет называться коллекция: `persons` или `people`? Говоря иначе, `Person(id=1234)` должен извлекаться при посещении URL `/persons/1234` или `/people/1234`? К счастью, правила выбранного нами американского английского предписывают правильный ответ: использовать `people`.

Но все же некоторые случаи будут довольно озадачивающими. Представьте, что работаете над API для аквариума. Какой будет коллекция для ресурса `Octopus`?

Главное в таких случаях — выбрать паттерн и придерживаться его, для чего нередко приходится поискать ответ в Сети среди знатоков грамматики (в данном случае отлично подойдет *octopuses*). Это также означает, что при создании множественного числа мы не должны опираться исключительно на простое добавление *s* — чем обычно грешат программисты при поиске паттернов.

3.3.3. Синтаксис

Мы подошли к более техническому аспекту именования. Как и в случае с предыдущими нюансами, когда дело доходит до синтаксиса, применяются те же правила. Во-первых, нужно выбрать что-то одно и придерживаться этого выбора. Во-вторых, если существует стандарт (например, правила написания в американском английском), то использовать его. Что же все это значит на практике? Перейдем к регистрам.

Регистры

В процессе определения API нам нужно присваивать имена различным компонентам, которые представлены в виде ресурсов, RPC и полей. Для каждого из них мы будем использовать тот или иной регистр, то есть своеобразный формат, в котором отображается имя. Чаще всего это отображение заметно, когда несколько слов объединяются для создания одной лексической единицы. К примеру, если бы у нас было поле, представляющее имя человека, то нам могло бы понадобиться назвать его *first name*. Но практически во всех языках программирования пробелы являются символом лексического разделения, значит, нам нужно совместить *first name* в одну единицу, что раскрывает целый ряд возможных вариантов, таких как верблюжий, змеиный или кебаб-регистр.

При объединении слов с использованием верблюжьего регистра первые буквы всех объединяемых слов, кроме первого, прописываются заглавными. Значит, *first name* будет отображаться как **firstName** (в таких случаях заглавные буквы напоминают горбы верблюда). При объединении слов с помощью змеиного регистра они разделяются нижним подчеркиванием, то есть мы получаем **first_name** (подразумевается, что это похоже на змею). Объединяя слова с помощью кебаб-регистра, мы разделяем их символами дефиса, то есть в данном случае получаем **first-name** (по аналогии с нанизанным на шампур шашлыком). В зависимости от языка, используемого для представления спецификации API, разные компоненты отображаются в разных регистрах. К примеру, в языке Google Protocol Buffer стандартом является использование для сообщений (интерфейсы TypeScript) верхнего верблюжьего регистра, как в **UserSettings** (обратите внимание на заглавную U), а для имен полей — змеиного регистра, как в **first_name**. С другой стороны, в стандартах спецификации OpenAPI имена полей оформляются с помощью верблюжьего регистра, как **firstName**.

Как уже отмечалось, конкретный выбор не столь важен при условии, что он используется согласованно во всей программе. Например, если вы используете `user_settings` для сообщения буфера протоколов (<https://developers.google.com/protocol-buffers>), пользователь может подумать, что перед ним имя поля, а не сообщение. Это только запутает любого, кто использует API. И коль речь зашла о типах, коротко обсудим зарезервированные слова.

Зарезервированные ключевые слова

В большинстве языков для определения API есть способ, позволяющий указать тип данных, хранящихся в конкретном атрибуте. К примеру, в TS можно написать `firstName: string`, указав, что поле `firstName` содержит примитивное строковое значение. Это также подразумевает, что термин «строка» имеет особую значимость, даже если используется в другом месте кода. В результате может быть опасно использовать для имен в API специальные ключевые слова, и этого нужно стараться избегать.

Если это выглядит сложным, есть смысл поразмышлять о том, что на самом деле выражают поля или сообщения, а не о том, что будет самым простым вариантом. К примеру, вместо *to* и *from* (относятся к специальным зарезервированным словам в языках наподобие Python) вы можете попробовать использовать более специфичную терминологию, такую как *sender* и *recipient* (если API связан с обработкой сообщений) или *payer* и *payee* (если связан с платежами).

Кроме того, важно учитывать целевую аудиторию вашего API. К примеру, если интерфейс будет использоваться только в JavaScript (возможно, он предназначен для работы исключительно в браузере), то о ключевых словах из других языков (например, из Python или Ruby) можно не беспокоиться. Тем не менее, если это не чревато большим количеством лишней работы, будет лучше избегать ключевых слов из других языков. В конце концов, вы никогда не знаете, будет ли ваш API использоваться одним из этих языков.

Теперь, когда мы обсудили все технические моменты, пора переходить к рассмотрению того, как на выбор имен может повлиять контекст, в котором живет и работает наш API.

3.4. КОНТЕКСТ

Несмотря на то что имена сами по себе порой могут донести всю необходимую информацию, чаще всего их смысл и предназначение мы выясняем на основе контекста, в котором они используются. К примеру, когда мы применяем в API слово *book*, то можем относить его к ресурсу, находящемуся в API библиотеки. Но при этом мы также можем обозначать с его помощью действие, совершаемое

в API бронирования перелетов. То есть одни и те же слова и терминология в зависимости от контекста могут означать совершенно разные вещи. Поэтому при выборе имен нам нужно иметь в виду контекст применения нашего API.

Важно помнить, что это работает в обе стороны. Контекст может придать имени дополнительное значение, которого в ином случае будет не хватать. И он же может запутать нас при использовании имен, которые несут очень специфичное значение, но в конкретном контексте смысла не имеют. Например, имя *record* без окружающего контекста может оказаться малоинформативным, а вот при использовании в API звукозаписи оно уже обретает дополнительный смысл, вытекающий из данного контекста.

Если коротко, то, несмотря на отсутствие строгих правил именования элементов в различных контекстах, важно помнить, что все выбираемые в API имена неразрывно связаны с его контекстом. То есть нам необходимо осознавать данный контекст и то значение, которое он может придавать выбираемым именам.

Теперь немного сменим тему и поговорим о типах данных и единицах измерения, в частности о том, как они должны использоваться в выбираемых нами именах.

3.5. ТИПЫ ДАННЫХ И ЕДИНИЦЫ ИЗМЕРЕНИЯ

Несмотря на то что многие имена полей оказываются достаточно описательными и без добавления единиц измерения (например, `firstName: string`), другие, не имеющие их, могут вводить в заблуждение. Представьте поле `size`. В зависимости от контекста (см. раздел 3.4) оно может иметь совершенно разные значения и единицы измерения. В листинге 3.2 показан пример одного и того же поля (`size`), имеющего неоднозначное значение и неопределенные единицы измерения.

Листинг 3.2. Использование поля `size` в контексте аудиоклипа и изображения

```

interface AudioClip {
  content: string;
  size: number;
}

interface Image {
  content: string;
  size: number;
}

```

Здесь может находиться двоичное аудиосообщение в кодировке Base64

Непонятно, какие единицы измерения используются в этих полях. Имеется ли в виду размер в байтах? Или же продолжительность аудио в секундах? Или размеры изображения? А может, что-то другое?

В этом примере поле `size` может означать разные вещи, но и сами эти разные значения также ведут к совершенно разным вариантам единиц измерения (байтам, секундам, пикселям и т. д.). К счастью, эта связь двухсторонняя, и указание единиц измерения все прояснит. Иначе говоря, `sizeBytes` и `sizeMegapixels` будут выглядеть намного понятнее, чем просто `size` (листинг 3.3).

Листинг 3.3. Аудиоклип и изображение с более понятными полями `size`, указывающими единицы измерения

```
interface AudioClip {
    content: string;
    sizeBytes: number;
}

interface Image {
    content: string;
    sizeMegapixels: number;
}
```

Теперь значения этих полей гораздо понятнее, так как в них указаны единицы измерения

Значит ли это, что в любой ситуации нужно просто добавлять в поля единицы измерения или формат? Ведь это определенно уменьшит путаницу в случаях, подобных показанным. Представьте, что нам нужно сохранять разрешение изображения в пикселях и его размер в байтах. Для этого можно задействовать два поля: `sizeBytes` и `dimensionsPixels`. Но разрешение содержит больше одного числа, так как подразумевает высоту и ширину. Как вариант, можно использовать строковое поле и указать разрешение в хорошо известном формате (листинг 3.4).

Листинг 3.4. Изображение, разрешение которого хранится в строковом поле в пикселях

```
interface Image {
    content: string;
    sizeBytes: number;

    // Разрешение (в пикселях). Например, "1024x768".
    dimensionsPixels: string;
}
```

Формат поля описывается в комментарии перед полем

Единицы измерения понятны (пиксели), но использование примитивного типа данных может смущать

Несмотря на то что такой вариант технически допустим и понятен, он отражает склонность всегда использовать примитивные типы данных, даже когда они могут не иметь смысла. Иначе говоря, аналогично тому, как имена иногда становятся понятнее при указании единиц измерения, в других случаях они могут становиться более понятными за счет использования более емкого типа данных. В таком случае вместо применения строкового типа, совмещающего два числа, можно использовать интерфейс `Dimensions` с числовыми полями ширины и высоты, в именах которых также указаны единицы измерения (пиксели) (листинг 3.5).

Листинг 3.5. Изображение, в котором разрешение выражается более емким типом данных

```
interface Image {
    content: string;
    sizeBytes: number;
    dimensions: Dimensions;
}

interface Dimensions {
    lengthPixels: number;
    widthPixels: number;
}
```

В этом случае имя поля `dimensions` не нуждается в указании единицы измерения, поскольку это значение выражается с помощью более емкого типа данных

Единицы измерения полей (пиксели) понятны без какого-либо строкового форматирования

Здесь значение поля `dimensions` вполне очевидно. Более того, нам не нужно раскрывать некие особые структурные подробности самого поля, поскольку интерфейс `Dimensions` уже это сделал. Завершим эту тему несколькими практическими исследованиями возможных проблем, возникающих при неосторожном выборе имен в API.

3.6. ПРАКТИЧЕСКОЕ ИССЛЕДОВАНИЕ: ЧТО ПРОИСХОДИТ ПРИ ВЫБОРЕ ПЛОХИХ ИМЕН

Эти рекомендации по выбору подходящих имен, учитывающие различные значимые аспекты, очень полезны, но будет нелишним разобрать пару реальных примеров использования не самых удачных имен. Это также позволит нам увидеть возможные последствия и проблемы, которые могут возникнуть в подобных ситуациях. Начнем со случая, в котором имени недостает весьма тонкого, но важного элемента.

Неочевидное значение

Если бы вы зашли в кондитерскую *Krispy Kreme* и попросили десять пончиков, то ожидали бы получить именно столько, ведь так? И вы бы удивились, если бы получили всего восемь? В таком случае можно предположить, что в кондитерской они просто закончились. Конечно, это неправильно, если сначала вы получаете лишь восемь пончиков и вынуждены просить еще два, чтобы забрать желанные десять.

А что, если бы вместо этого у вас была возможность попросить только *N* пончиков? Иначе говоря, вы могли бы обратиться к продавцу с просьбой: «Дайте мне до десяти пончиков», на что он бы выдал любое их количество, но не больше десяти. (Нужно также учитывать, что вы можете получить и нуль пончиков.) Внезапно это странное поведение в первом примере с кондитерской обретает смысл. Это по-прежнему неудобно (я еще не видел кондитерской с такой системой заказов), но уже не вызывает недоумения.

В главе 21 мы познакомимся с паттерном проектирования, который покажет, как можно постранично пролистывать множество ресурсов безопасно, понятно и легко. И оказывается, эта эксклюзивная возможность запрашивать только максимум (а не конкретное количество) в точности отражает принцип работы этого паттерна пагинации (при использовании поля `maxPageSize`).

Исторически сложилось, что разработчики в Google следуют этому паттерну описанным образом за одним существенным исключением: вместо того, чтобы указать `maxPageSize` в качестве просьбы «дайте мне максимум N элементов», запросы устанавливают `pageSize`. Отсутствие этих трех начальных символов (`max`) ведет к невероятному количеству путаницы, как в случае с заказом пончиков: человек думает, что просит конкретное количество, но по факту можно заказать лишь максимальное.

Самый распространенный сценарий — это когда кто-то просит десять элементов, получает восемь и думает, что элементов больше нет (так же как мы можем предположить, что в кондитерской закончились пончики). В действительности же это не так: одно лишь получение восьми пончиков еще не означает, что они кончились. Это лишь говорит о том, что продавцу нужно пойти в цех и принести еще. В итоге пользователи API упускают множество элементов поиска, так как перестают перебирать страницы результатов до фактического завершения списка.

И хотя это может раздражать и создавать неудобства, предлагаю взглянуть на еще более серьезную ошибку, вызванную путаницей единиц измерения для полей.

Единицы измерения

В 1999 году NASA планировали совершить маневр марсианским спутником Mars Climate Orbiter (МКО), направив его вращаться по орбите этой планеты на высоте 140 миль. Они провели множество вычислений, чтобы выяснить, какие именно тяговые импульсы нужно приложить к спутнику, чтобы он оказался в нужном положении и смог выполнить маневр. К сожалению, вскоре после проведения расчетов команда заметила, что спутник оказался не в 140 милях над поверхностью, а гораздо ниже. Проведенные впоследствии вычисления показали, что он находился всего в 35 милях от планеты. При этом минимальная высота, на которой он мог продолжать функционировать, составляла 50 миль, то есть в итоге зонд просто разрушился в марсианской атмосфере.

В ходе расследования происшествия выяснилось, что команда Lockheed Martin произвела вывод в стандартных единицах США (в частности, в фунтах-силы в секунду), а команда NASA работала с единицами СИ (в Н-с, или ньютон-секундах). Быстрые вычисления показали, что 1 фунт-сила эквивалентна 4,45 Н-с, в результате чего спутник получил в четыре раза больший импульс, чем требовалось, и оказался ниже минимальной допустимой высоты.

Листинг 3.6. (Сильно упрощенный) пример API для вычислений тяговых характеристик МСО

```
abstract class MarsClimateOrbiter {
    CalculateImpulse(CalculateImpulseRequest):
        CalculateImpulseResponse;
    CalculateManeuver(CalculateManeuverRequest):
        CalculateManeuverResponse;
}

interface CalculateImpulseResponse {
    impulse: number;
}

interface CalculateManeuverRequest {
    impulse: number;
}
```

Интерфейсы `CalculateImpulseRequest` и `CalculateManeuverResponse` опущены для краткости

Здесь вычисляется импульс, но отсутствуют единицы измерения. Это подразумевает возможность передачи предыдущего вывода в качестве следующего ввода

С другой стороны, если бы имена полей в точке интеграции включали единицы измерения, ошибка была бы намного более очевидной (листинг 3.7).

Листинг 3.7. Альтернативы примерам интерфейсов с указанием единиц измерения

```
interface CalculateImpulseResponse {
    impulsePoundForceSeconds: number;
}

interface CalculateManeuverRequest {
    impulseNewtonSeconds: number;
}
```

Здесь уже становится очевидно, что нельзя просто взять вывод одного метода API и передать его следующему методу, поскольку в них используются разные единицы измерения

Естественно, Mars Climate Orbiter как программно, так и технически был устроен намного сложнее, и вряд ли представленного на сайте https://en.wikipedia.org/wiki/Mars_Climate_Orbiter#Cause_of_failure сценария можно было избежать просто за счет использования более описательных имен. Однако этот пример все равно удачно показывает, в чем заключается ценность таких имен и как они помогают подчеркнуть различия в принятых допущениях, особенно при координировании работы разных команд.

3.7. УПРАЖНЕНИЯ

1. Представьте, что вам нужно создать API для управления повторяющимися графиками («Это событие происходит один раз в месяц»). Старший инженер утверждает, что для всех случаев будет достаточно сохранять значение в секундах между событиями. Другой инженер считает, что этот API должен предоставлять для разных временных единиц (например, секунд, минут, часов, дней, недель, месяцев, лет) разные поля. Какой из предложенных дизайнов правильно обрисует значение задуманной функциональности и окажется наиболее эффективным?

2. В системах хранения данных вашей компании в качестве единиц измерения используются гигабайты (10^9 байт). Например, при создании общего каталога вы можете установить размер 10 Гбайт с помощью инструкции `sizeGB = 10`. Инженеры подготовили к запуску новый API, в котором пропускная способность сети будет измеряться в гигабитах (2^{30} бит), и планируют устанавливать лимиты пропускной способности именно в этих единицах (например, `bandwidthLimitGib = 1`). Не окажется ли это слишком тонким различием, которое будет запутывать пользователей? Объясните, почему это будет так (или нет).

РЕЗЮМЕ

- Хорошие имена, как и хорошие API, просты, выразительны и предсказуемы.
- Когда дело доходит до языка, грамматики и синтаксиса (и других произвольных выборов), зачастую правильным решением будет выбрать что-то одно и придерживаться этого в дальнейшем.
- Предлоги в именах обычно считаются в API признаком, который указывает на наличие более серьезных проблем дизайна, требующих исправления.
- Помните, что контекст использования имени может как давать дополнительную информацию, так и запутывать. Обязательно учитывайте его при выборе имен.
- Добавляйте единицы измерения для примитивов и опирайтесь на более емкие типы данных, чтобы раскрыть информацию, не представленную в самом имени.

4

Состав и иерархия ресурсов

В этой главе

- ✓ Что такое схема ресурсов.
- ✓ Различные типы связей ресурсов (ссылочные, «многие ко многим» и т. д.).
- ✓ Как блок-схемы связей между сущностями описывают схему ресурсов.
- ✓ Как выбирать правильную связь между ресурсами.
- ✓ Антипаттерны схем ресурсов, которых нужно избегать.

Как мы узнали в главе 1, переключение внимания с действий на ресурсы позволяет проще и быстрее знакомиться с API с помощью простых паттернов. Например, REST предоставляет набор стандартных глаголов, которые можно применять к различным ресурсам. То есть для каждого ресурса, о котором мы узнаём, мы также получаем пять разных действий, которые можно к нему применить (create, get, list, delete и update).

И хотя это довольно серьезная особенность, намного важнее здесь внимательно продумывать ресурсы, которые мы определяем как часть API. Ключевой момент при выборе правильных ресурсов — это понимание того, как они будут сочетаться друг с другом в будущем. В текущей главе мы узнаем, как организовывать различные ресурсы в API, а также разберем подходящие для этого варианты и познакомимся с общими принципами выбора правильного способа ассоциирования

ресурсов между собой. Кроме того, мы изучим два антипаттерна (как *не* надо делать) при выборе способа организации набора ресурсов API. Сначала разберемся, что конкретно имеется в виду под схемой ресурсов.

4.1. ЧТО ТАКОЕ СХЕМА РЕСУРСОВ

Когда мы говорим о схеме ресурсов, то обычно подразумеваем их расположение в API, поля, определяющие эти ресурсы, и то, как они связаны с помощью этих полей. Другими словами, схема ресурсов — это модель связей сущностей для конкретной структуры API. К примеру, если мы создаем API для чата, то схема ресурсов получится в результате ряда наших выборов, которые могут привести к созданию ресурсов `ChatRoom` и `User`, определенным образом связанных с комнатой. При этом нас интересует тот самый способ, с помощью которого ресурсы `User` и `ChatRoom` ассоциированы друг с другом. Если вам доводилось проектировать реляционную базу данных с различными таблицами, то это должно быть вам знакомо: схема базы данных по своей сути зачастую очень похожа на представление API.

И хотя можно подумать, что связь — это единственное, что имеет значение, на деле все немного сложнее. Несмотря на то что непосредственное влияние на итоговую схему ресурсов имеет действительно только сама связь, есть много других факторов, которые влияют на нее косвенно. Очевидный пример — это выбор самих ресурсов: если мы решим не использовать ресурс `User`, а придерживаться простого списка пользователей по имени (`members: string[]`), то задействовать другие ресурсы не придется и вопрос отпадет сам собой.

Чтобы понять схему API, рассмотрим ее в виде соединенных линиями блоков. Например, рис. 4.1 показывает, как можно представить вариант чата со множеством пользователей и комнат, все из которых связаны между собой.



Рис. 4.1. В чате может быть множество пользователей

А если бы мы создавали API онлайн-магазина (например, нечто вроде Amazon), то могли бы сохранять ресурсы `User` вместе с их различными адресами (для отправки купленных товаров) и способами оплаты. Более того, способы оплаты могут сами содержать ссылку на адрес для выставления счета. Подобная схема показана на рис. 4.2.

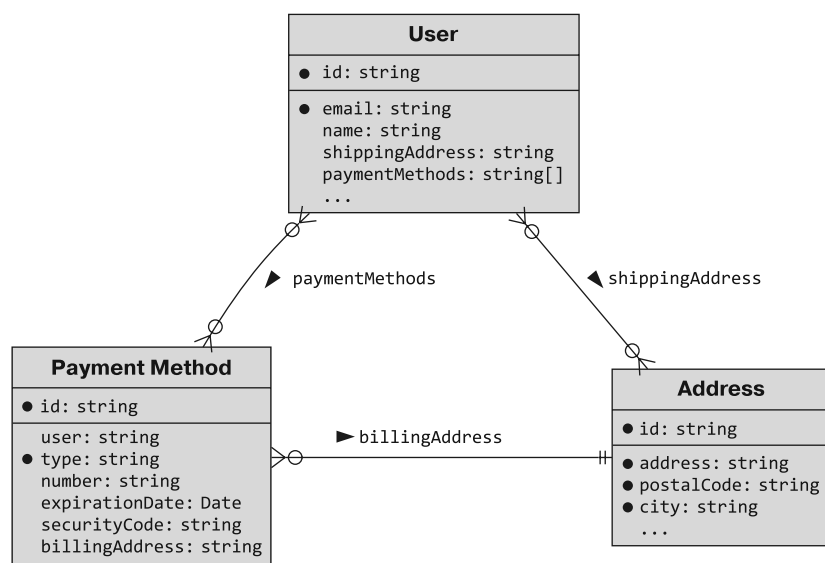


Рис. 4.2. Пользователи, способы оплаты и адреса связаны по-разному

Если коротко, то важно помнить, что схема ресурсов — это обширное понятие, включающее ресурсы, которые мы выбираем для API, а также описание связей и взаимодействия этих ресурсов. В следующем подразделе мы кратко разберем типы связей, а также виды взаимодействия (и ограничения), которые они могут обеспечить.

4.1.1. Типы связей

Продумывая схему ресурсов, приходится учитывать множество способов, с помощью которых могут быть связаны эти ресурсы. Важно понимать, что все связи по своей природе являются двунаправленными, то есть даже если какая-то связь кажется односторонней (например, сообщение указывает на одного пользователя в качестве своего автора), обратная связь все равно существует, даже если не определена явно (например, пользователь может быть автором множества разных сообщений). Углубимся в наиболее распространенную форму связей: ссылки.

Ссылочные связи

Простейший способ связи ресурсов — ссылка. Под этим мы подразумеваем, что один ресурс ссылается или указывает на другой. К примеру, в API чата у нас могут быть ресурсы `Message`, составляющие содержимое этого чата. Очевидно, что в данном сценарии каждое сообщение будет написано одним пользователем,

ссылка на которого будет храниться в поле **author** этого сообщения. Это приведет к простой ссылочной связи между пользователем и сообщением: оно будет содержать поле **author**, указывающее на конкретного пользователя, как показано на рис. 4.3.



Рис. 4.3. Ресурс сообщения содержит ссылку на конкретного пользователя, являющегося его автором

Эта ссылочная связь иногда называется *отношением по внешнему ключу*, поскольку каждый ресурс **Message** будет указывать ровно на один ресурс **User** в качестве автора. Хотя, как уже говорилось, с ресурсом **User** может быть связано множество сообщений. В результате это можно также рассматривать как связь «многие к одному», при которой пользователь может написать много сообщений, но в качестве автора любого сообщения всегда будет указан лишь один пользователь.

Связи «многие ко многим»

Будучи более сложной версией ссылок, этот тип связи представляет сценарий, в котором ресурсы объединяются так, что каждый указывает на несколько других. Например, если у нас есть ресурс **ChatRoom** для групповой беседы, то он будет содержать множество отдельных пользователей в качестве участников. Однако каждый пользователь также может быть участником нескольких комнат чата. В этом сценарии ресурс **ChatRoom** имеет связь с пользователями в формате «многие ко многим». Этот ресурс указывает на множество ресурсов **User** в качестве участников данной комнаты, причем каждый **User** может указывать на множество комнат чата, участником которых он является.

Механизм работы этого типа связей мы рассмотрим позже в части III, но сами связи «многие ко многим» очень распространены в API и могут быть выражены несколькими способами, каждый из которых имеет свои достоинства и недостатки.

Самоадресация

На данном этапе мы разберем то, что может прозвучать странно, но в действительности является просто особой версией ссылки: самоадресацию. При такой связи ресурс указывает на другой ресурс в точности того же типа, поэтому «само» означает тип, а не сам ресурс. Фактически она подобна обычной ссылочной

связи. Однако называть ее нужно иначе в связи с типичным визуальным представлением, показанным на рис. 4.4, где стрелка указывает на сам ресурс.

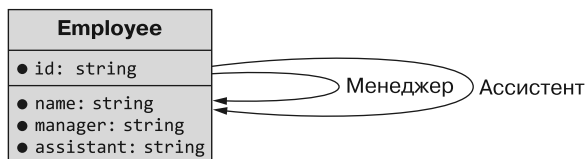


Рис. 4.4. Ресурс сотрудника указывает на другие ресурсы сотрудников в качестве менеджеров и ассистентов

Здесь можно задаться вопросом, зачем вообще ресурсу указывать на другой ресурс того же типа. Это разумный вопрос, но подобный тип связи в действительности возникает намного чаще, чем кажется на первый взгляд. Самоадресация в большинстве случаев наблюдается в иерархических связях, где ресурс является узлом в дереве или API сетевого типа, в котором данные могут быть представлены в виде направленного графа (подобно социальной сети).

Представьте API для хранения справочной информации о штате компании. В этом сценарии сотрудники указывают друг на друга, чтобы отслеживать вектор подчинения (например, сотрудник 1 отчетывается перед сотрудником 2). Этот API также может иметь дополнительную самоадресацию для особых связей (например, ассистент сотрудника 1 является сотрудником 2). В каждом из этих случаев можно смоделировать схему ресурсов с помощью самоадресации.

Иерархические связи

В завершение нужно разобрать очень специфичный вид связи, который выступает еще одной вариацией стандартной ссылочной связи: иерархию. Иерархическими являются связи, в которых один ресурс содержит указатель на другой, но этот указатель обычно направлен вверх и подразумевает более обширную связь, нежели один ресурс, указывающий на другой. В отличие от обычных ссылочных связей иерархии также отражают принцип *содержания* или *владения* между ресурсами, что проще всего понять с помощью аналогии папок на вашем компьютере. Папки (или каталоги) содержат множество файлов, которыми они как бы *владеют*. При этом папки также могут содержать другие папки, которые содержат другие папки, и т. д., порой без ограничения.

Несмотря на то что подобная связь может казаться незначительной, она подразумевает кое-какие важные атрибуты и поведение. Например, при удалении папки на компьютере обычно удаляются и все файлы (а также другие папки), содержащиеся внутри нее. Либо, если вы получаете доступ к конкретной папке, это зачастую подразумевает и доступ к ее файлам (а также другим папкам) внутри. Аналогичное поведение присуще и ресурсам.

Прежде чем идти дальше, предлагаю посмотреть, как выглядят иерархические связи. Мы уже использовали примеры таких связей в предыдущих главах, так что они не должны вас удивить. К примеру, мы говорили о ресурсах `ChatRoom`, которые состоят из множества ресурсов `Message`. В этом случае подразумевается иерархия ресурсов `ChatRoom`, *содержащих* или *владеющих* `Messages`, как показано на рис. 4.5.



Рис. 4.5. Ресурсы `ChatRoom` выступают владельцами ресурсов `Message` через иерархическую связь

Как вы можете догадаться, обычно подразумевается, что наличие доступа к ресурсу `ChatRoom` также дает доступ к ресурсам `Message`, которые составляют содержимое комнаты чата. Кроме того, при удалении ресурса `ChatRoom` обычно подразумевается, что это действие каскадируется вниз на ресурсы `Message` на основе иерархической связи «родитель — потомок». В некоторых случаях этот каскадный эффект оказывается весьма кстати (например, хорошо иметь возможность удалить всю папку на компьютере, обходясь без необходимости удалять все содержащиеся в ней файлы по отдельности). В прочих же случаях каскадное поведение может оказываться довольно проблематичным (например, мы можем решить, что даем доступ к папке, когда в действительности даем его ко всем ее файлам, включая вложенные папки).

Как правило, иерархические связи сложны и вызывают множество каверзных вопросов. Например, могут ли ресурсы менять своих родителей? Иначе говоря, можно ли переместить ресурс `Message` из одного ресурса `ChatRoom` в другой? (Обычно это оказывается неудачной идеей.) Мы еще поговорим об иерархии, а также о ее достоинствах и недостатках в разделе 4.2.

4.1.2. Блок-схемы связей между сущностями

В схемах этой главы вы могли заметить интересные символы в конце линий, связывающих разные ресурсы. И хотя у нас нет возможности углубиться в UML (Unified Modeling Language, унифицированный язык моделирования; <https://ru.wikipedia.org/wiki/UML>), можно по крайней мере взглянуть на некоторые из этих стрелок и разобрать принцип их действия.

Если коротко, то вместо того чтобы просто проводить произвольную стрелку от одного ресурса к другому, можно с ее помощью выражать важную информацию

об описываемой ею связи. То есть каждый конец стрелки может сообщить нам, *сколько* ресурсов находится на другом конце. К примеру, рис. 4.6 показывает, что в школе есть множество учащихся, но каждый из них принадлежит лишь к одной школе. Кроме того, каждый учащийся посещает множество классов, каждый из которых включает множество учащихся.

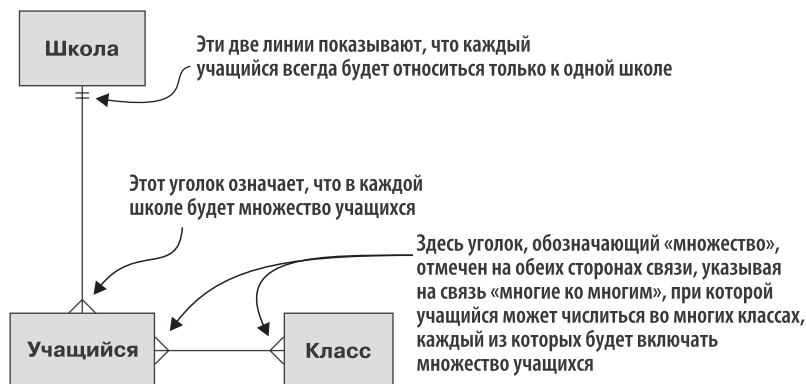


Рис. 4.6. Пример блок-схемы связей сущностей, показывающей школы, учащихся и классы

Два этих символа являются самыми распространенными, но иногда вы можете встретить и другие. Они относятся к случаям, в которых любые связанные ресурсы являются необязательными. Например, технически класс может состоять из нуля учащихся (и наоборот). В результате, опять же технически, блок-схема может выглядеть как на рис. 4.7.

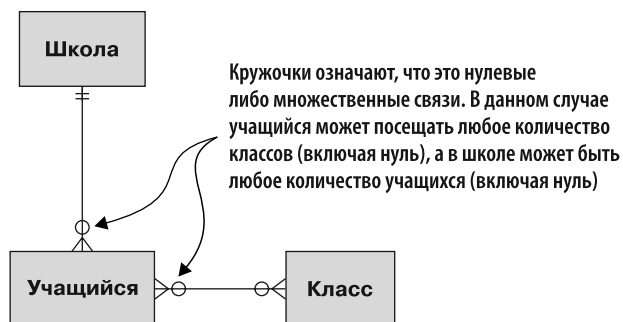


Рис. 4.7. Обновленная блок-схема, показывающая, что учащиеся могут не присутствовать в классах (а классы могут не включать учащихся)

Иногда бывает трудно освоить чтение подобной нотации, поэтому будет излишним прояснить направление, в котором нужно читать каждую связь. Лучше

всего начинать чтение подобных блок-схем с одного ресурса, затем следовать по линии, обращать внимание на *конец* этой линии, завершая чтение у другого ресурса. При этом важно помнить, что символ соединения у ресурса, с которого мы начинаем чтение, пропускается. Чтобы было более понятно, рис. 4.8 показывает связь между ресурсами *School* и *Student*, разбитую на две отдельные блок-схемы, где направление чтения связей указано рядом с линией.

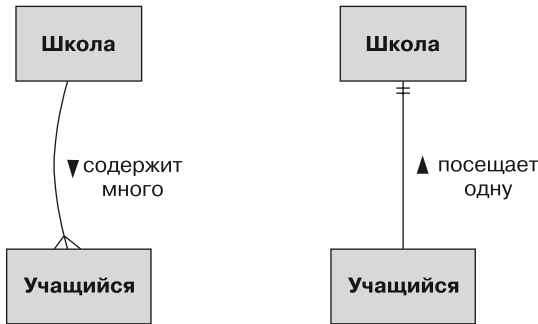


Рис. 4.8. Как читать связи между сущностями на примере ресурсов *School* и *Student*

Как видите, в школе есть много учащихся, относящихся только к ней. Ради упрощения мы объединяем эти две связи в единую блок-схему, используя одну связывающую линию, как было в предыдущих примерах.

Теперь, когда мы научились читать блок-схемы, пора перейти к более важной задаче моделирования наших API с помощью выбора правильных связей между ресурсами.

4.2. ВЫБОР ПРАВИЛЬНЫХ СВЯЗЕЙ

Как мы узнали в разделе 4.1, выбор правильного типа связи зачастую зависит от выбора самих ресурсов, поэтому в данном разделе мы будем рассматривать эти действия как связанные. Начнем с ответа на самый важный вопрос: нужна ли вообще нам связь?

4.2.1. Нужна ли вообще связь

При создании API после выбора списка элементов или ресурсов очередным шагом будет выбор взаимосвязи этих ресурсов. Подобно нормализации схемы базы данных, зачастую возникает соблазн связать все, что может когда-либо потребовать связи, чтобы создать богатую сеть ресурсов, по которой можно легко перемещаться с помощью разнообразных указателей и ссылок. Несмотря на то

что подобный подход определенно приведет к очень богатому и описательному API, он также может стать несостоятельным и привести к серьезному падению производительности по мере получения API все большего объема трафика и сохранения все большего объема данных.

Чтобы наглядно увидеть, о чем идет речь, мы представим себе простой API, в котором пользователи могут подписываться друг на друга (подобно Instagram или Twitter). В этом случае у пользователей работает самоадресация по принципу «многие ко многим», в которой пользователь подписан на многих других пользователей, каждый из которых может иметь множество подписчиков. Схематично это отражено на рис. 4.9.

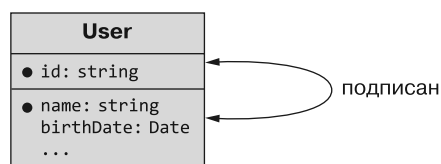


Рис. 4.9. Ресурс User использует самоадресацию, отражая, что пользователи подписываются друг на друга

На первый взгляд это кажется вполне безобидным, но по факту может создать серьезную путаницу, когда у вас будут пользователи с миллионом подписчиков (и пользователи, подписанные на миллионы других пользователей), поскольку такие связи ведут к сценариям, в которых одно изменение в одном ресурсе может повлиять на миллион других связанных ресурсов. К примеру, если кто-то известный удалит свой аккаунт в Instagram, то потребуются удалить или обновить миллионы записей (в зависимости от внутренней схемы базы данных).

Это не значит, что нужно всеми силами избегать каких-либо связей. Наоборот, следует заранее взвешивать долгосрочные затраты любой связи. И подобно тому, как важно понимать конечную стоимость ипотеки еще до подписания контракта, необходимо понять и реальные затраты на конкретный проект API еще на стадии его разработки, а не в конце процесса. В сценариях, в которых поддержание связей является поистине критическим (как в предыдущем примере, где сохранение связей подписчиков очень важно), существуют способы, позволяющие избежать потери быстродействия, но это не избавляет от необходимости обдуманно выбирать ссылочные связи в любом API.

Иначе говоря, ссылочные связи должны быть содержательными и фундаментальными для требуемого поведения. Можно еще сказать, что эти связи никогда не должны быть случайными, оказываясь приятным дополнением или чем-то, что может потребоваться в дальнейшем. Вместо этого любая ссылочная связь должна иметь значение для реализации основной задачи API.

Например, такие сервисы, как Twitter, Facebook и Instagram, построены на основе связей между подписанными друг на друга пользователями. Самореферентная связь между пользователями действительно важна для этих сервисов — в конце концов без нее Instagram станет похож на простое приложение для хранения фотографий. Сравните это с чем-то вроде сервиса прямого обмена сообщениями. Очевидно, что хоть этот вид API и подразумевает связи между пользователями в форме чатов, они определенно не оказываются для приложения настолько же критически значимыми. К примеру, важно иметь связь между двумя пользователями чата, а вот наличие полной коллекции всех связей между потенциальными пользователями имеет не столь большое значение.

4.2.2. Ссылки или встроенные данные

Предположив, что некоторые связи критически важны для поведения и функциональности вашего API, следующим шагом нужно ответить на некоторые важные вопросы. В первую очередь необходимо выяснить, что будет разумнее: встроить данные в API (то есть сохранить копию внутри ресурса) либо использовать ссылки (то есть сохранить только указатель на официальные данные). Рассмотрим эти ситуации на простом примере уже знакомого нам сценария с чатом.

Представьте, что в каждой комнате чата должен присутствовать один администратор. У нас есть два варианта: можно либо указать на пользователя-администратора с помощью ссылочного поля (например, `adminUserId`), либо встроить данные для этого пользователя и представить их как часть ресурса `ChatRoom`. Наглядно оба этих случая показаны на рис. 4.10 и 4.11.

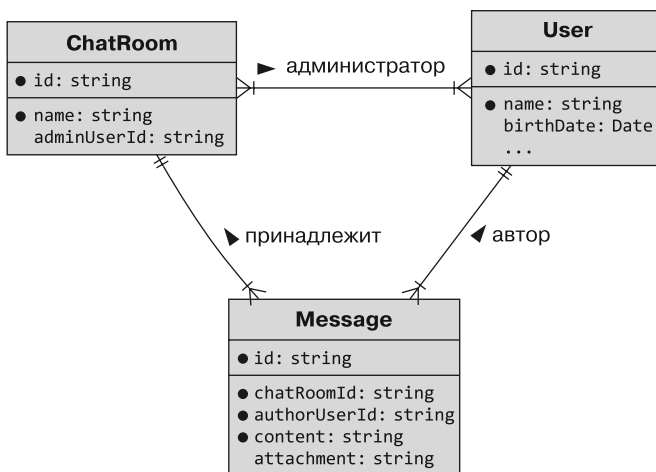


Рис. 4.10. Ресурс ChatRoom со ссылкой на ресурс администратора User

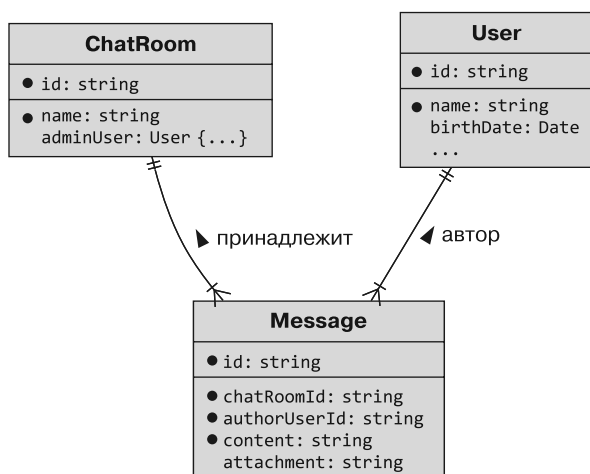


Рис. 4.11. Ресурс ChatRoom, в который встроен администратор

В первой блок-схеме (см. рис. 4.10) мы видим, что ресурс ChatRoom при определении администратора указывает на ресурс User. Во второй же блок-схеме (см. рис. 4.11) мы видим, что информация об администраторе *содержится* в поле `adminUser`. Здесь возникает очевидный вопрос: какой вариант лучше? Оказывается, ответ зависит от поставленных вопросов или выполняемых действий.

Если вы хотите увидеть имя администратора, то будет намного проще использовать встраивание данных. Чтобы понять, почему это так, рассмотрим необходимые для этого действия в каждом из двух сценариев (листинг 4.1).

Листинг 4.1. Получение информации об администраторе для примеров со ссылкой и встроенными данными

```

function getAdminUserNameInline(chatRoomId: string): string {
    let chatRoomInline = GetChatRoomInline(chatRoomId);
    return chatRoomInline.adminUser.name;
}

function getAdminUserNameReference(chatRoomId: string): string {
    let chatRoomRef = GetChatRoomReference(chatRoomId);
    let adminUser = GetAdminUser(chatRoomRef.adminUserId);
    return adminUser.name;
}
  
```

В обоих случаях мы начинаем с извлечения ресурса ChatRoom

В случае со ссылкой для получения дополнительной информации об администраторе необходимо выполнить второй поиск

Самым очевидным различием в этих двух функциях является количество фактических вызовов API, требующих ответа сети. В первой, где данные возвращаются встроенными, для извлечения всей релевантной информации нужен всего один вызов API. Во второй необходимо сначала извлечь ресурс ChatRoom,

и только потом удастся узнать, какой пользователь нас интересует. После этого для выяснения его имени потребуется извлечь ресурс `User`.

Значит ли это, что встраивание данных всегда оказывается лучшим решением? Не совсем. Это лишь один пример, в котором для получения интересующей информации необходимо сделать два вызова. Но что, если нас эта информация интересует нечасто? Если это так, то при каждом запросе кем-то ресурса `ChatRoom` мы будем отправлять множество байтов, которые будут просто игнорироваться. Иначе говоря, когда кто-то запрашивает ресурс `ChatRoom`, мы также сообщаем в ответ все о ресурсе `User`, являющемся администратором этой комнаты.

Это может показаться пустяком, но что, если каждый пользователь также хранит информацию обо всех своих друзьях (другие ресурсы `User`)? В этом случае возвращение вместе с комнатой чата информации о ее администраторе может привести к большому объему лишних данных. Более того, в подобных случаях это зачастую подразумевает дополнительную вычислительную нагрузку (обычно связанную с запросами к БД, которые внутренне требуют объединения данных). Причем эта нагрузка может довольно быстро увеличиваться непредсказуемым образом. Что же тогда делать?

К сожалению, это будет зависеть от того, что делает ваш API. То есть решение нужно принимать на основе анализа. В качестве универсального правила здесь можно применять оптимизацию под типичный случай, не нарушая возможности выполнения продвинутого. Это означает, что сначала необходимо оценить суть связанного ресурса, его величину и то, насколько он может разрастись, после чего уже решить, целесообразно ли включать информацию об администраторе в ответ. В этом сценарии типичный пользователь наверняка чаще будет заинтересован в обмене сообщениями с друзьями, а не выяснении того, кто же является администратором комнаты чата. В результате встраивание этих данных вряд ли окажется столь уж важным. С другой стороны, ресурс `User` может быть довольно маленьким, а значит, и встраивание данных в случае их частого использования в API не станет большой проблемой.

По этой теме сказано достаточно, и нам осталось разобраться с еще одним важным видом связей: иерархией. Рассмотрим, когда есть смысл предпочесть именно ее.

4.2.3. Иерархия

Как мы уже узнали, иерархические связи представляют особый тип ссылочных связей между родительским и дочерним ресурсом, а не двумя обычно равнозначными ресурсами. Основные отличия этого вида связи состоят в каскадировании эффекта, вызванного совершаемыми действиями, а также наследовании поведения и свойств родителя потомком. К примеру, удаление родительского ресурса обычно подразумевает удаление дочернего. Аналогичным образом доступ

(или его отсутствие) к родителю обычно подразумевает такой же уровень доступа к дочернему ресурсу. Эти уникальные и потенциально ценные особенности данной связи наделяют ее огромными возможностями как в положительном, так и отрицательном плане.

Как же решить, когда есть смысл выстраивать ресурсы с помощью иерархической связи, а не простой? Предположив, что мы уже определили эту специфичную связь как фундаментальную для нашего API (см. подраздел 4.2.1). Тогда можно использовать эти варианты поведения как важный индикатор того, является ли данная связь удачным выбором.

К примеру, когда мы удаляем комнату чата, то, скорее всего, хотим удалить и принадлежащие ей ресурсы `Message`. Более того, если у кого-то есть доступ к ресурсу `ChatRoom`, то по логике у него также должен быть доступ и к просмотру связанных с этой комнатой ресурсов `Message`. Эти два признака означают, что нам, скорее всего, нужно моделировать связь в виде иерархии.

Кроме того, бывают признаки, указывающие и на *неуместность* иерархической связи. Поскольку дочерний ресурс должен всегда иметь лишь одного родителя, становится очевидно, что ресурс, у которого он уже есть, второго родителя получить не сможет. То есть если вы задумали реализовать связь «один ко многим», то иерархия точно не подойдет. К примеру, ресурсы `Message` должны принадлежать только одной комнате чата. Если мы захотим связать один ресурс `Message` со множеством ресурсов `ChatRoom`, то иерархия вряд ли будет эффективна.

Это не значит, что ресурсы могут указывать на один (и только один) ресурс. Все же большинство из них будут являться потомками и при этом иметь возможность также выступать родителями для других ресурсов. Представьте, что ресурсы `ChatRoom` принадлежат компаниям (новый тип ресурса для этого примера). Компании могут использовать политики хранения (представленные ресурсами `RetentionPolicy`), указывающие, как долго хранятся сообщения до своего удаления. Эти политики хранения будут потомками одной компании, но на них могут ссылаться любые ресурсы `ChatRoom` в компании. Если это вас запутало, то взгляните на схему ресурсов на рис. 4.12.

Как видите, ресурсы `User`, `ChatRoom` и `RetentionPolicy` являются потомками `Company`. Аналогичным образом ресурсы `Message` — потомки `ChatRoom`. Однако `RetentionPolicy` может использоваться многократно и применяться ко множеству разных комнат.

Надеюсь, к этому моменту вы хорошо поняли, когда лучше опираться на ссылки, а не на иерархию (и когда лучше встраивать информацию, а не использовать в качестве указателя ссылки). Однако после изучения этих тем обычно возникает пара рефлекторных действий, которые мы склонны совершать при проектировании API. В связи с этим далее мы познакомимся с антипаттернами построения схем ресурсов и узнаем, как их избегать.

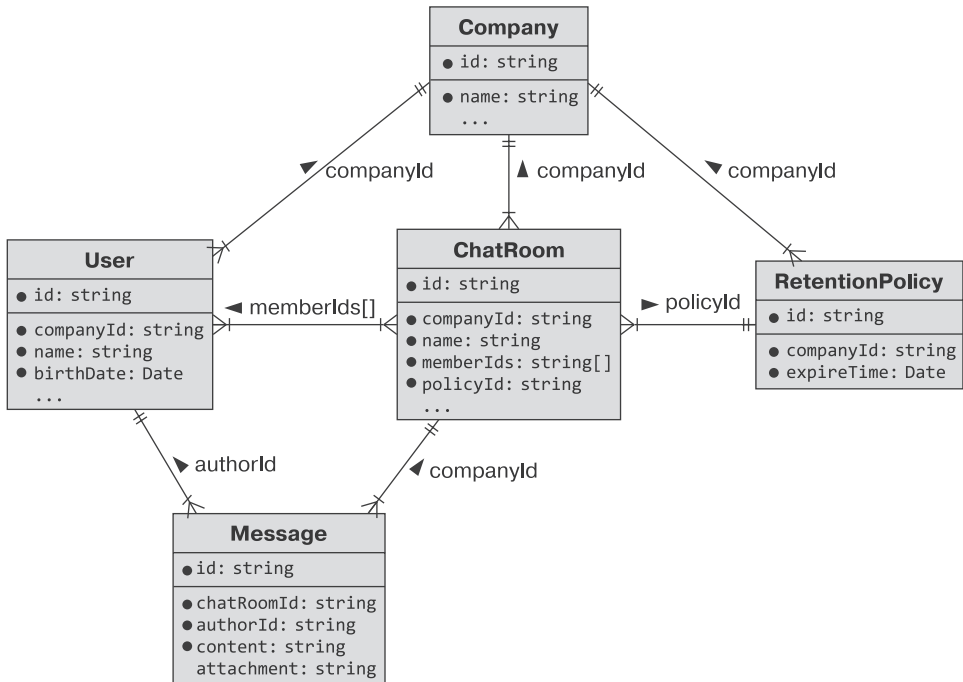


Рис. 4.12. Схема ресурсов, отражающая политики хранения компании, касающиеся комнат чата

4.3. АНТИПАТТЕРНЫ

Как и в большинстве тем, существуют шаблонные модели поведения, которым легко следовать при любых обстоятельствах. Но эти модели нередко приводят нас к ошибкам. В конце концов гораздо проще слепо следовать правилу, чем глубоко продумывать дизайн API и выбирать наилучший способ размещения ресурсов и организации связи между ними.

4.3.1. Ресурсы для всего

Идея создать ресурсы даже для малейшего компонента, который вы хотите смоделировать в API, зачастую выглядит соблазнительной. Как правило, это происходит во время проектирования, когда кто-то решает, что каждый компонент, который становится типом данных, должен быть ресурсом. Представьте, что у нас есть API, в котором можно сохранять подписи к изображениям. Для каждой подписи мы можем решить сохранять саму ее область (возможно, в виде ограничительной рамки), а также список заметок, выступающих ее содержимым.

На данном этапе у нас есть четыре отдельных компонента: изображения, подписи, ограничительные рамки и заметки. Здесь возникает вопрос: какие из них должны стать ресурсами, а какие — завершить свой путь в роли простых типов данных? Одним из ошибочных действий здесь будет представление всего в виде ресурсов, независимо от обстоятельств (листинг 4.2). В таком случае у нас может получиться схема ресурсов, подобная изображенной на рис. 4.13.

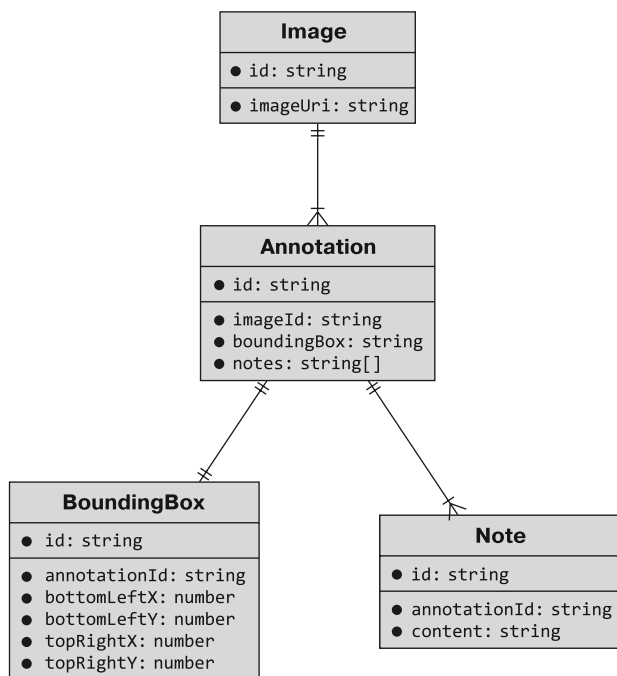


Рис. 4.13. Схема ресурсов, показывающая все компоненты, представленные в виде ресурсов

Теперь у нас есть четыре полных ресурса, имеющих полноценный жизненный цикл. Иначе говоря, для каждого из этих ресурсов нам нужно реализовать пять разных методов, получив в общей сложности 20 разных вызовов API. Но тут возникают вопросы: а правильный ли это выбор? Действительно ли нам нужно давать каждой ограничительной рамке собственный идентификатор? И действительно ли нам нужны отдельные ресурсы для каждой отдельной заметки?

Если разобрать эту схему чуть подробнее, то можно понять две важные вещи. Во-первых, ресурс **BoundingBox** связан с подписью отношением «один к одному», значит, мы практически не получим пользы от помещения его в отдельный ресурс. Во-вторых, маловероятно, что список ресурсов **Note** станет исключительно большим, следовательно, представление их в качестве ресурсов не слишком нам поможет. Если все это учесть, то у нас получится намного более простая схема.

Листинг 4.2. Упрощенные интерфейсы, опирающиеся на встроенные представления

```

interface Image {
  id: string;
}

interface Annotation {
  id: string;
  boundingBox: BoundingBox;
  notes: Note[];
}

interface BoundingBox {
  bottomLeftPoint: Point;
  topRightPoint: Point;
}

interface Point {
  x: number;
  y: number;
}

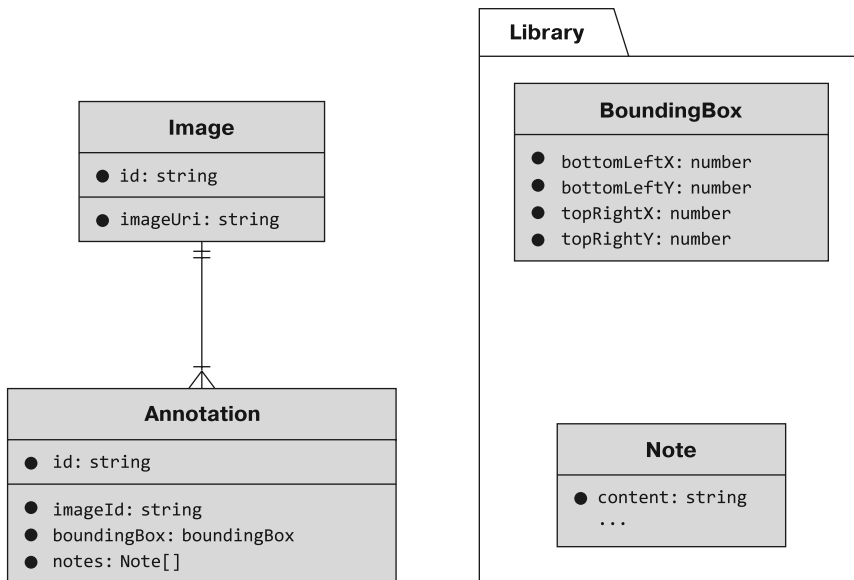
interface Note {
  content: string;
  createTime: Date;
}

```

Эти два интерфейса являются фактическими ресурсами (обратите внимание на поле идентификатора)

Все это не ресурсы, а просто типы данных, которые отображаются в API внутри ресурса Annotation

Это означает, что на деле нам нужно продумывать всего два ресурса, а не четыре, что существенно упрощает работу. На рис. 4.14 показана упрощенная схема ресурсов.

**Рис. 4.14.** Схема для двух ресурсов, а не четырех

В качестве общего правила здесь можно порекомендовать избегать двух проблем. Во-первых, если вам не нужно взаимодействовать с одним из предполагаемых ресурсов, независимым от связанного с ним другого ресурса, то вы вполне можете оставить его просто типом данных. В этом примере маловероятно, что кому-то потребуется манипулировать ограничительной рамкой вне контекста самого изображения, а значит, ей будет достаточно принять роль типа данных.

Во-вторых, если вы заинтересованы в прямом взаимодействии с рассматриваемым компонентом (например, вы можете решить — удалять одну заметку или отдельно создавать новые), то при наличии возможности встроить его (см. подраздел 4.2.2) это может оказаться весьма кстати. В данном случае ресурсы **Note** были достаточно малы и не предполагали разрастания до большой коллекции для каждой подписи, следовательно, их наверняка можно спокойно встроить вместо того, чтобы выражать в качестве независимых.

4.3.2. Глубокие иерархии

Следующий распространенный антипаттерн связан конкретно с иерархиями. Нередко иерархические связи могут оказываться столь мощными и полезными, что мы стремимся использовать их везде, где только можно. Тем не менее излишне углубленные иерархии могут сбивать с толку и создавать сложности в работе.

Представим, что создаем систему для управления каталогом курсов в университетах. Нам нужно отслеживать различные университеты, факультеты или учебную программу, в которой существуют конкретный курс (например, медицинский факультет или инженерный), доступные курсы, семестр, на котором они преподаются, а также различные связанные с ними документы (например, расписание занятий). Один из способов, позволяющих отслеживать это, — собрать все в одну иерархию, как показано на рис. 4.15.

Несмотря на то что технически это вполне рабочее решение, его довольно сложно анализировать и удерживать все множество родителей в уме. Это также означает следующее: теперь, чтобы найти один документ, необходимо знать довольно много информации (подробнее об этом в подразделе 6.3.6), которую будет сложно сохранить и запомнить. Однако самое важное здесь в том, что получить

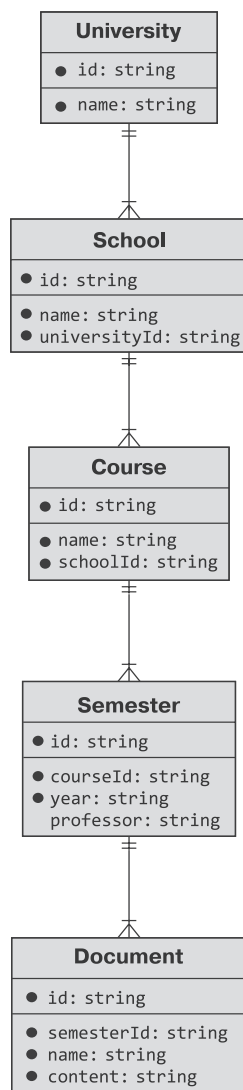


Рис. 4.15. Каталог курсов с глубокой иерархией

нужный результат можно, обойдясь без выгод, предлагаемых иерархией. Иначе говоря, наверняка можно создать не менее качественный API без такого количества иерархических уровней.

Что, если мы решим, что ключевыми элементами являются университеты, курсы и документы? В этом случае факультет и семестр становятся полями курса. Прежде чем это реализовать, мы спрашиваем, являются ли уровни факультета и семестра в иерархии действительно столь важными.

Например, для чего мы планируем использовать иерархическое поведение? Собираемся ли мы часто удалять факультеты? (Маловероятно.) А семестры? (Тоже вряд ли.) Скорее всего, нам просто нужно видеть все семестры, на которых преподается представленный курс. Но этого можно добиться, выполнив фильтрацию на основе идентификатора курса и аналогичным образом создав список курсов по ID факультета.

Помните, что это не означает, что нам нужно полностью избавиться от этих ресурсов. К примеру, нам вряд ли потребуется перечисление доступных семестров, а вот факультеты будут интересны. Предполагаемое здесь изменение — смена связей между факультетами и курсами с иерархической на ссылочную. Эта менее углубленная иерархическая структура показана на рис. 4.16.

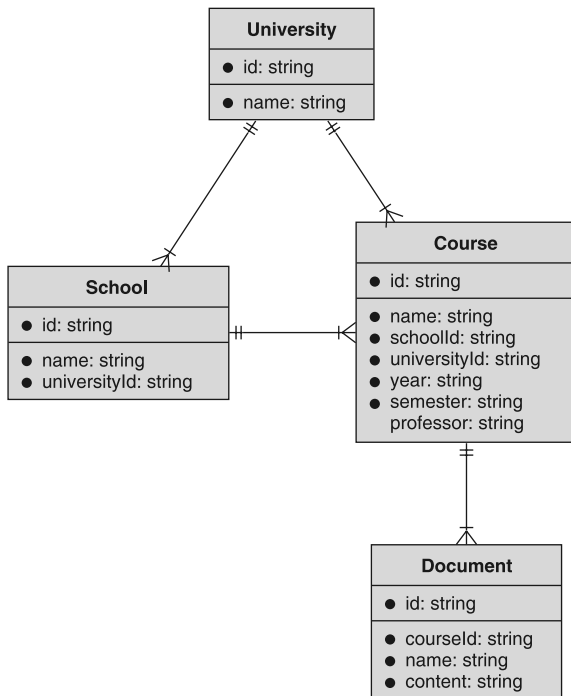


Рис. 4.16. Каталог курсов с менее углубленной иерархией

Упорядочив ресурсы таким образом, мы все равно можем совершать действия наподобие предоставления доступа ко всем документам для заданного курса или составления списка всех курсов для заданного семестра, но при этом мы сократили иерархию до таких частей, для которых все это действительно полезно.

4.3.3. Встраивание всего

С появлением нереляционных баз данных, в частности широкомасштабных систем хранения в формате «ключ — значение», зачастую возникает склонность денормализовать (или встроить) все данные, включая те, которые в прошлом были бы четко разбиты по отдельным таблицам для объединения с помощью модных SQL-запросов. И хотя иногда имеет смысл свернуть информацию в один общий ресурс вместо того, чтобы создавать для каждого ее элемента собственный (как мы узнали в подразделе 4.3.2), при свертывании слишком большого количества информации это может оказаться столь же вредоносным, как и выделение каждого ее элемента в отдельный ресурс.

Одна из основных причин этого связана с целостностью данных, которая также является распространенной проблемой в денормализованных схемах нереляционных баз данных. Представим, что создаем API библиотеки, который сохраняет книги, написанные разными авторами. Один из вариантов — использовать два ресурса, **Book** и **Author**. При этом ресурсы **Book** будут указывать на ресурсы **Author**, как показано на рис. 4.17. Если бы мы решили, наоборот, встроить эту информацию, то могли бы сохранить имена авторов (и прочую информацию) прямо в ресурсе **Book**, как показано на рис. 4.18.

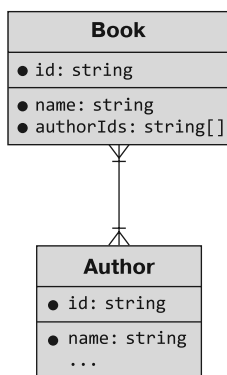


Рис. 4.17. **Book** и **Author** представлены как отдельные ресурсы

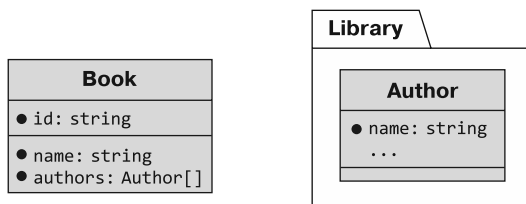


Рис. 4.18. Ресурс **Book** смоделирован с информацией **Author** внутри

Как мы узнали ранее в подразделе 4.2.2, подобное встраивание данных может как оказаться очень ценным, так и привести к реальным проблемам. Например,

при обновлении имени одного из авторов книги обновится ли оно для всех книг, которые он написал? Ответ будет зависеть, скорее всего, от реализации, но сам факт, что нам необходимо ставить этот вопрос, подразумевает возможную путаницу. Иначе говоря, встраивая данные, которые будут использоваться другими ресурсами (например, авторами, которые написали несколько книг), мы создаем сложную ситуацию, в которой нам нужно решать, как пользователи API должны обновлять эти общие данные (например, имена авторов). И хотя решить подобные проблемы можно с помощью рабочих способов, по-хорошему этот вопрос вообще не должен возникать.

4.4. УПРАЖНЕНИЯ

1. Представьте, что создаете API закладок, в котором у вас будут храниться закладки для различных URL, упорядоченные по разным каталогам. Смоделируете ли вы эту схему как два отдельных ресурса (**Bookmark** и **Folder**) или же как один (например, **Entity**) со встроенным типом, указывающим на то, выступает ли этот ресурс в качестве каталога либо закладки?
2. Придумайте пример и нарисуйте блок-схему связей сущностей, подразумевающую связи «многие ко многим», «один ко многим», «один к одному» и по желанию «один ко многим». Постарайтесь использовать правильные символы для точек соединения.

РЕЗЮМЕ

- Схема ресурсов означает расположение ресурсов в API и связи между ними.
- Противьтесь соблазну связать все ресурсы в API и сохраняйте связи, только если они предоставляют для него важную функциональность.
- Иногда имеет смысл сохранять для компонента отдельный ресурс. В прочих же случаях будет лучше встроить эти данные и оставить компонент в виде типа данных. Принимаемое решение будет зависеть от того, требуется ли вам возможность точечного взаимодействия с этим компонентом.
- Избегайте излишне углубленных иерархических связей, поскольку их может быть сложно понять, как и управлять ими.

5

Типы данных и значения по умолчанию

В этой главе

- ✓ Что мы подразумеваем под типами данных.
- ✓ Чем значение `null` отличается от полностью отсутствующего.
- ✓ Знакомство с примитивными типами и их коллекциями.
- ✓ Обработка сериализации различных типов данных.

При проектировании любого API всегда нужно думать о том, какие типы данных мы хотим получать, понимать и потенциально сохранять. Иногда это может звучать довольно просто: поле `"name"` может быть просто строкой символов. Однако в этом вопросе кроется целый комплекс сложностей. К примеру, как должна быть представлена эта строка символов в байтах (оказывается, для этого есть много вариантов)? Что произойдет, если имя в вызове API будет опущено? Будет ли это отличаться от случая передачи пустого имени (например, `{ "name": "" }`)? В этой главе мы изучим различные типы данных, с которыми вы будете встречаться при проектировании или использовании API, а также узнаем, как понимать внутреннее представление данных и обрабатывать предустановленные значения.

5.1. ЗНАКОМСТВО С ТИПАМИ ДАННЫХ

Типы данных — важная часть практически каждого языка программирования. С их помощью разработчик сообщает компьютеру, как обрабатывать фрагмент данных и как эти данные должны взаимодействовать с различными операторами самого языка или типа. И даже в языках с динамической типизацией (где одна переменная может выступать в качестве разных типов данных) информация типа в конкретный момент времени все равно остается очень важной, указывая, как язык программирования должен рассматривать ту или иную переменную.

Тем не менее при проектировании API приходится выходить за рамки мышления в контексте одного языка, главным образом из-за того, что основная цель API — позволить взаимодействовать с сервисом любому человеку, программирующему на любом языке. Как правило, для этого мы опираемся на выбранный протокол сериализации, который получает структурированное представление данных на используемом нами языке и преобразует эти данные в универсальное представление сериализованных байтов, которое отправляет запрашивающему клиенту. В свою очередь, клиент десериализует эти байты, преобразуя их обратно в представление в памяти, с которым сможет взаимодействовать из используемого им языка, возможно, отличного от нашего. На рис. 5.1 этот процесс показан схематично.

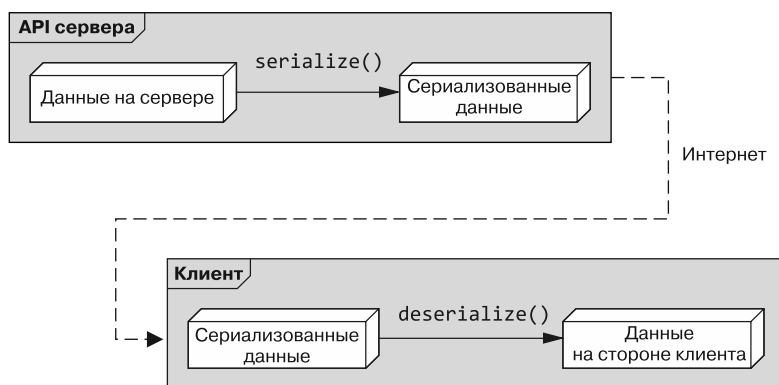


Рис. 5.1. Перемещение данных от сервера API к клиенту

Несмотря на то что процесс сериализации предоставляет огромные удобства (по сути, позволяя использовать API из любого языка программирования), он не лишен недостатков. Самый большой из них — это потеря некоторой части информации в процессе передачи, что обуславливается отличиями в принципе работы разных языков. Иначе говоря, из-за различий в специфике и функциональности языков использование любых протоколов сериализации будет так или иначе вести к потерям.

Это возвращает нас к вопросу о важности типов данных и их применения в API. Проще говоря, при учете всех будущих пользователей API определенно будет недостаточно опираться только на типы, предоставляемые именно нашим языком программирования. Вместо этого необходимо рассуждать в контексте типов данных, предлагаемых выбранным нами форматом сериализации (чаще всего это JSON), и продумывать, как можно его расширить в случаях, где он не отвечает нашим потребностям. Это означает, что нам нужно определиться с типами данных, которые мы планируем отправлять и получать от клиентов, а также сопроводить их грамотной документацией, чтобы клиенты не удивлялись тому или иному поведению.

Это не значит, что веб-API должен иметь строгую схему в рамках системы реляционной базы данных. Все же одной из наиболее сильных особенностей современных инструментов разработки является гибкость, предоставляемая динамическими бессхемными структурами и хранилищами данных. Тем не менее важно учитывать типы данных, с которыми мы взаимодействуем, при необходимости проясняя их дополнительными аннотациями. Не имея дополнительного контекста о представленных данных, мы можем в итоге застрять, пытаясь понять, что же на самом деле подразумевал клиент. Скажем, $a + b$ может работать одним образом для численных значений (например, $2 + 4$ будет давать 6), но действовать совершенно иначе для текста (например, `"2" + "4"` будет давать `"24"`). Без понимания этого контекста нам бы пришлось угадывать намерение клиента при использовании оператора `+`: подразумевает он сложение или конкатенацию? А что, если одно значение численное, а второе строковое? Это может привести к еще большему количеству догадок. Но что будет, если значение вообще опущено?

5.1.1. Отсутствующее значение и значение `null`

Это может показаться странным, но сложнее всего бывает в тех случаях, когда данные отсутствуют. Начнем с того, что во многих форматах сериализации есть значение `null`, которое работает как флаг, указывающий на не-значение (например, строка может быть `value` или буквально `null`). Как же должен действовать API при попытке сложить, например, `null` и 2? Любой API, использующий формат сериализации с поддержкой подобного значения `null`, должен будет решать, как лучше всего обрабатывать подобный тип ввода. Должен ли он представить, что `null` математически равен 0? А как быть со складыванием `null` и `"value"`? Нужно ли в этом сценарии рассматривать `null` как пустую строку (`""`) и пробовать его конкатенировать?

Все усложняется и тем, что динамические структуры данных (например, объект JSON) имеют еще одну проблему: что, если значение просто отсутствует? Иначе говоря, что, если вместо его передачи в явной форме как `null` ключ, где оно должно

храниться, будет полностью отсутствовать? Будет ли это рассматриваться так же, как явное значение `null`? Чтобы разобраться, представьте, что у вас есть ресурс `Fruit`, который должен содержать имя и цвет, представленные строковыми типами. Взгляните на следующий пример объектов JSON и значений, которые API может распознать в качестве цвета ресурса:

```
fruit1 = { name: "Apple", color: "red" };
fruit2 = { name: "Apple", color: "" };
fruit3 = { name: "Apple", color: null };
fruit4 = { name: "Apple" };
```

Как видите, первое значение цвета вполне очевидно (`fruit1.color == "red"`). Но вот что делать с другими? Должен ли пустой цвет (`""`) рассматриваться так же, как явный цвет `null` (отличается ли `fruit2.color` от `fruit3.color`)? А как быть с фруктом, у которого *отсутствует* значение цвета? Будет ли `fruit3.color` рассматриваться иначе, чем `fruit4.color`? Это все может показаться лишь нюансами JSON, но они существуют и во многих других форматах сериализации (например, в Google Protocol Buffers (<https://opensource.google/projects/protobuf>) тоже есть аналогичное поведение) и представляют сценарии, с которыми любой API должен либо разобраться, либо пойти на риск оказаться несовместимым и запутанным для пользователей. Говоря иначе, нельзя просто предположить, что библиотека сериализации будет функционировать правильно, поскольку наверняка все будут использовать разные библиотеки сериализации.

В этой главе мы пройдемся по всем распространенным типам данных, начав с простых (иногда называемых *примитивными*) и продвигаясь в направлении более сложных (таких как карты и коллекции). Вы уже должны быть знакомы с большинством этих понятий, так что мы уделим меньше внимания основам каждого типа и сосредоточимся на важных моментах их грамотного использования в веб-API. При этом мы разберем различные характерные для них сложные моменты и оптимальные способы их решения, подходящие для любого API. В процессе мы будем придерживаться формата JSON, но большинство рекомендаций окажутся применимы к любому другому формату, поддерживающему динамические структуры с помощью как выводимых, так и явных определений типов. Начнем с самого простого: значений `true` и `false`.

5.2. ЛОГИЧЕСКИЙ ТИП

В большинстве языков программирования логический тип, или булев тип, является простейшим из доступных, выражая одно из двух значений: `true` либо `false`. Ввиду столь ограниченного набора значений мы обычно опираемся на этот тип при хранении флагов или просто вопросов с ответами «да/нет». К примеру, можно сохранить в комнате чата флаги, указывающие, находится ли эта комната в архиве и допустимы ли в ней боты (листинг 5.1).

Листинг 5.1. Ресурс ChatRoom с логическими флагами

```
interface ChatRoom {
  id: string;
  // ...
  archived: boolean;
  allowChatbots: boolean;
}
```

Здесь будет много других полей для комнаты

Этот флаг отмечает, была ли комната отправлена в архив

А этот флаг указывает, допустимы ли в комнате боты

Как бы то ни было, но логические флаги могут стать довольно ограничивающими в будущем, когда вопросы формата «да/нет» сменятся более общими и потребуют уже подробного ответа, который логическое поле предоставить не сможет. К примеру, в дальнейшем может возникнуть ситуация, когда комнаты чатов допускают множество разных типов участников, а не только обычных пользователей и ботов. В этом сценарии наш текущий дизайн приведет к длинному списку из всех допустимых (или недопустимых) типов участников, например `allowChatbots`, `allowModerators`, `allowChildren`, `allowAnonymousUsers` и т. д. Если есть такая возможность, то будет разумнее избежать использования коллекции логических флагов и задействовать вместо них другой тип данных или структуру для определения категорий участников, разрешенных или запрещенных в конкретной комнате чата.

Если же предположить, что логическое поле подходит для переменной, то нужно учесть еще кое-какие важные нюансы. Имена булевых полей отражают свой положительный или отрицательный смысл. В примере с `allowChatbots` значение `true` указывает, что боты в данной комнате разрешены. Но название поля могло звучать и как `disallowChatbots`, что при значении `true` уже означало бы запрет ботов. На каком же основании выбирать ту или иную формулировку?

Как правило, положительные логические поля для большинства из нас оказываются наиболее понятными по одной простой причине: осознание двойного отрицания требует от человека больших мыслительных усилий. А значение `false` для отрицательного поля будет означать именно это. Представьте поле `disallowChatbots`. Как вы проверите, разрешены ли в данной комнате боты (листинг 5.2)?

Листинг 5.2. Функция для добавления бота в комнату в случае его допустимости

```
function addChatbotToRoom(chatbot: Chatbot,
                          roomId: number): void {
  let room = getChatRoomById(roomId);
  if (room.disallowChatbots === false) {
    chatbot.join(room.id);
  }
}
```

Здесь мы можем проверить, не установлен ли запрет на присутствие в комнате ботов. Если указано значение `false`, то к ней можно присоединиться

Сложно ли в этом разобраться? Не то чтобы очень. Но не создаст ли это бóльшую мыслительную нагрузку, чем более простая версия (`if (room.allowChatbots) { ... }`)? Для большинства из нас наверняка. На основе одного только этого факта практически всегда будет лучше устанавливать положительные логические флаги. Однако возможны сценарии, когда разумнее использовать именно отрицательные, обычно в случае предустановленного или сброшенного значения поля.

Например, в некоторых языках или форматах сериализации (до недавних пор в Google Protocol Buffers v3) многие примитивы, такие как булевы поля, могли быть установлены только на значение `zero`, а не `null` или `missing`. Здесь можно догадаться, что для логических значений `zero` практически всегда приравнивается к `false`. То есть когда мы рассматриваем значение поля `allowChatbots` при создании комнаты чата, то без дополнительного вмешательства эта комната участие ботов (`defaultChatRoom.allowChatbots == false`) *не разрешит*. Но тогда мы не можем отличить пользователя, говорящего: «Я не устанавливал это значение, поэтому делайте, что считаете нужным» (то есть значение `null` или `missing`), от того, который утверждает: «Я не хочу разрешать ботов в чате» (то есть явное значение `false`). В обоих случаях значение будет просто `false`. Что же тут можно сделать?

И хотя решить эту проблему можно множеством способов, один из типичных вариантов — опереться на положительный или отрицательный характер имени логического поля, чтобы получить «правильное» значение по умолчанию. Иначе говоря, можно выбрать имя для поля так, чтобы значение `zero` обеспечивало искомое нами предустановленное состояние. Например, если нам нужно, чтобы по умолчанию анонимные пользователи были разрешены в чате, то можно назвать поле `disallowAnonymousUsers`, чтобы значение `zero (false)` приводило к нужному нам состоянию по умолчанию (допуску анонимных пользователей). К сожалению, этот выбор закрепляет это состояние в имени поля, лишая нас дальнейшей гибкости (то есть в будущем не получится изменить этот предустановленный вариант), но, как говорится, отчаянные времена требуют отчаянных мер.

5.3. ЧИСЛА

Численные значения являются уже чуть более сложными, нежели простые вопросы в формате «да/нет», и позволяют сохранять всевозможную ценную информацию, такую как подсчет (например, `viewCount`), размер и расстояние (например, `itemWeight`) или значения валюты (например, `priceUsd`). Как правило, везде, где нам могут понадобиться арифметические операции (или даже если эти операции будут иметь лишь логический смысл), числовые поля идеально подойдут на роль типа данных.

Существует один важный вид значений, который не слишком подойдет под этот тип данных: числовые идентификаторы. Это может удивить, поскольку многие базы данных (и почти все реляционные базы данных) в качестве ID первичных ключей для строк используют автоматическое инкрементирование целочисленных значений. Так почему бы нам не поступить так же с идентификаторами в API?

Несмотря на то что из разных соображений (например, производительности или ограничения пространства) мы по-прежнему можем внутренне использовать числовое поле, на поверхности API числовые типы лучше всего задействовать для тех значений, которые благодаря выражению числами принесут некую арифметическую пользу. Иными словами, если API выражает элементы, которые содержат значение веса в граммах, то теоретически можно сложить все эти значения для определения общего веса набора этих элементов в граммах. С другой стороны, обычно нет смысла складывать набор числовых идентификаторов и продельвать что-либо с этим значением. Наоборот, сложение множества числовых ID скорее окажется бесполезным. В связи с этим важно полагаться на числовые типы данных именно из-за приносимой ими численной или арифметической пользы, а не из-за того, что они пишутся только в численных символах (возможно, с десятичными запятыми). Обычно значения, которые только выглядят как числа, но ведут себя подобно токенам или символам, скорее должны быть строковыми (см. раздел 5.4).

Учитывая все сказанное, всякий раз при использовании численных значений в API нужно принимать во внимание ряд нюансов. Сперва рассмотрим определение границ, или диапазонов, приемлемых значений для числовых полей.

5.3.1. Границы

При определении границ числового поля, будь то целое или десятичное число (либо иная математическая репрезентация, например, дроби, мнимые числа и т. д.), важно учитывать верхнюю границу (максимальное значение), нижнюю границу (минимальное значение) и нулевое значение. Поскольку все они влияют друг на друга, рассмотрим границы абсолютного значения, а затем решим, стоит ли воспользоваться и положительной, и отрицательной стороной этого диапазона.

Когда речь идет об определении границ, основное внимание уделяется размеру. Иначе говоря, все числа в конечном итоге необходимо где-то хранить. То есть нам нужно знать, сколько пространства в битах под них выделять. Для малых целых чисел может быть достаточно и 8 бит, обеспечивающих 256 возможных значений (от -127 до 127). Для большинства распространенных числовых значений обычно хватает 32 бит, дающих около четырех миллиардов вариаций

(от $-2\,000\,000\,000$ до $2\,000\,000\,000$). Для большинства элементов, которые нас могут интересовать, удовлетворительным размером будет 64 бита, обеспечивающих около 18 квинтильонов возможных значений (от -9 квинтильонов до 9 квинтильонов). Этот диапазон становится более запутанным, когда мы вводим числа с плавающей запятой, поскольку они существуют во множестве различных вариаций, занимающих вплоть до 256 бит в хранилище. Но главная суть в том, что обычно есть представление, которое подойдет для диапазона, предполагаемого вами в API.

Несмотря на то что все эти возможности прекрасны, вы можете поинтересоваться, почему нас вообще волнует размер чисел, которые будут находиться где-то в базе данных. Разве смысл API не в том, чтобы как раз абстрагировать подобные вещи? Все верно. Однако это все же важно, поскольку способы, с помощью которых разные компьютеры и разные языки программирования обрабатывают числа, не универсальны. Например, в JavaScript даже не существует полноценного целочисленного значения, а есть лишь численный тип, который обрабатывает все числа этого языка. Более того, многие языки обрабатывают невероятно огромные числа очень разными способами. К примеру, в Python 2 тип `int` способен хранить 32-битные целые, в то время как тип `long` может обрабатывать произвольно большие числа, сохраняя не менее 36 бит. Проще говоря, если в ответ API должен будет отправить потребителю очень большое число, то существует возможность, что язык на стороне получателя окажется неспособен правильно его считать и понять. Кроме того, получатели этих потенциально огромных чисел должны понимать, сколько пространства выделять под их сохранение. В общем, границы здесь оказываются очень важны.

В результате внутри системы разработчики обычно опираются на 64-битные целочисленные типы для всех целых чисел, если только нет объективных причин этого не делать. Как правило, даже когда вам не требуется 64-битный диапазон в данный момент, это вовсе не значит, что он не потребуется в дальнейшем. Так что лучше установить нижние и верхние границы с запасом.

5.3.2. Значения по умолчанию

Как мы узнали в подразделе 5.1.1, при использовании большинства типов данных необходимо учитывать и ряд дополнительных сценариев, в частности значения `null` и `missing`, когда поля просто опущены, например, в логических типах (см. раздел 5.2). Однако в форматах сериализации, которые не предоставляют примитивам механизмов для хранения значения `null`, у нас нет способа, позволяющего различать истинное значение `0` (или `0.0`) и предустановленное, с помощью которого пользователь говорит: «Здесь у меня нет предпочтений, поэтому можете сами выбрать то, что для меня лучше».

Листинг 5.4. Сложение чисел приводит к арифметическим проблемам с плавающей запятой

```
const jsonAddition(): number {
  const a = JSON.parse('{ "value": 0.1 }');
  const b = JSON.parse('{ "value": 0.2 }');
  return a['value'] + b['value'];
}
```

К сожалению, вернется 0.30000000000000004, а не 0.3

Что же делать? Сценарий в листинге 5.4 должен в достаточной степени показать, что использование численных значений с предположением, что они будут одинаковы в разных языках, может оказаться довольно опрометчивым. При этом короткий ответ на поставленный вопрос некоторым пуристам может не понравиться, хотя работает очень хорошо. Речь идет об использовании строк символов.

Эта простая стратегия, в которой численные значения при сериализации выражаются в виде строковых, — просто механизм, позволяющий избежать интерпретации этих примитивных численных значений как фактических чисел. Вместо этого сами значения могут интерпретироваться библиотекой, которая лучше справится с обработкой таких сценариев. Иначе говоря, вместо того чтобы парсить библиотекой JSON значение 0.2 как JS-тип Number, можно использовать библиотеку наподобие Decimal.js, чтобы спарсить это значение в десятичном типе произвольной точности (листинг 5.5).

Листинг 5.5. Правильное сложение чисел в целях избежания проблем с плавающей запятой

```
const Decimal = require('decimal.js');

const jsonDecimalAddition(): number {
  const a = JSON.parse('{ "value": "0.1" }');
  const b = JSON.parse('{ "value": "0.2" }');
  return Decimal(a['value']).add(Decimal(b['value']));
}
```

Обратите внимание, что эти численные значения являются строками, а не фактическими числами

При их сложении с помощью библиотеки произвольной точности наподобие Decimal.js получается правильное значение (0.3)

Ну а поскольку эта стратегия опирается на строки, то нужно познакомиться и со строковыми полями.

5.4. СТРОКИ

Практически в каждом языке программирования мы рассматриваем строки как само собой разумеющееся, не вникая в принцип их работы. Даже те из нас, кто уделяет время изучению обработки строк символов в C, склонны избегать погружения в кодировки символов и великий мир Юникода. И хотя становиться экспертом по Юникоду, кодированию символов или прочим связанным со строками темам не обязательно, есть ряд вещей, которые играют значимую

роль при выборе подходящей обработки строк в API. А поскольку большинство полей в API обычно являются именно строками, то это становится еще более важным.

В западном мире можно вполне безопасно рассматривать строки как коллекции отдельных символов, представляющих текстовое содержимое. Это довольно сильное обобщение, но все же наша книга не о Юникоде, так что придется придерживаться его. Поскольку большая часть содержимого, которое нам потребуется вносить в API, по своей природе является текстовой, строки здесь оказываются, пожалуй, наиболее полезным из всех типов данных.

При этом их также можно считать и наиболее универсальным типом из доступных при построении API. Они могут обрабатывать простые поля наподобие имен, адресов и общих сведений; могут отвечать за большие блоки текста; а в случаях, когда формат сериализации не поддерживает отправку сырых байтов по сети, могут представлять закодированный двоичный формат (например, байты в кодировке Base64). Строки также позволяют наиболее эффективно сохранять уникальные идентификаторы, даже если те выглядят как числа, а не текст. Внутренне их можно сохранить в виде байтов (дополнительную информацию см. в главе 6), но представление в API практически однозначно лучше делать с помощью строк символов.

Однако прежде чем учиться спасать мир с помощью строк, коротко рассмотрим некоторые потенциальные проблемы при использовании строковых полей и научимся применять их наиболее эффективно, начиная с граничных условий.

5.4.1. Границы

Как мы узнали в подразделе 5.3.1, важность граничных условий обуславливается необходимостью учитывать, сколько пространства требуется выделять для хранения данных. Строковых значений этот аспект касается в той же степени, что и численных. Если вам доводилось определять схему реляционной базы данных, получая в итоге `VARCHAR(128)`, где значение 128 оказывалось полностью произвольным выбором, то вам должна быть знакома эта порой нежелательная необходимость.

Точно так же, как в рассмотренном нами случае с числами, эти лимиты размеров играют важную роль, поскольку получатели данных на другой стороне должны понимать, сколько пространства выделять под сохранение этих значений. При этом здесь необходимо заранее закладывать в API еще и запас на увеличение размеров, иначе в дальнейшем можно оказаться в очень неприятной ситуации. Поэтому при выборе максимальных длин строковых полей, как правило, лучше ошибиться в большую сторону.

Далее нужно разобраться, как именно определять максимальную длину. Оказывается, наше изначальное определение строки как коллекции символов работает только в ограниченном количестве случаев, поскольку многие языки не следуют этому принципу в нужной нам степени. Как бы то ни было, но еще бо́льшая проблема возникает из-за того, что единицы измерения для хранилища (байты на диске) не сохраняют связь «один к одному» с единицами измерения длин строк (которые мы называли *символами*). Что же делать?

Чтобы не писать целую главу по Юникоду, проще всего решить этот вопрос, продолжив размышлять в контексте символов (несмотря на то что в Юникоде они называются *кодowymi точками*) и затем предположить наиболее масштабный формат сериализации для задач хранения: UTF-32. Это означает, что при сохранении строковых данных мы будем выделять не привычный один байт на символ, как это делается при использовании ASCII, а целых четыре байта.

Помимо пространства хранилища, есть еще один аспект, который нужно учитывать в отношении каждого строкового поля: обработка излишнего ввода. В случае чисел значение, выходящее за допустимый диапазон, можно спокойно отклонить в API, сопроводив это дружелюбным сообщением: «Пожалуйста, выберите значение между 0 и 10». В случае же строковых значений у нас есть два варианта. В первом можно всегда отвергать значение, подобно выходящему из диапазона числу, но в зависимости от обстоятельств и контекста это может оказаться необязательным. Во втором варианте можно отсекал текст, выходящий за установленную границу.

И хотя отсечение выглядит неплохим решением, оно может запутать и оказаться неожиданным, что явно не впишется в характеристики хорошего API, о чем мы говорили еще в главе 1. Кроме того, здесь возникает ряд новых вариаций для API (это поле должно отвергать текст или отсекал его?), что может привести к еще большей непредсказуемости, поскольку разные поля будут иметь разное поведение. В результате так же, как и с числами, обычно разумнее будет отвергать любой ввод, превышающий установленный предел поля.

5.4.2. Значения по умолчанию

Аналогично числам и булевым полям многие форматы сериализации могут не допускать использования нулевого значения (`null`), что отличается от полного отсутствия значения (`""`). В результате может быть сложно определить разницу между пользователем, указывающим, что строка должна быть пустой, и пользователем, который сознательно просит, чтобы сам API поступил с полем наилучшим образом с учетом остального контекста.

К счастью, есть несколько решений. Во многих случаях пустая строка просто является недопустимым значением для поля. В итоге такую строку можно

использовать в качестве флага, указывающего на то, что нужно подставить и сохранить значение по умолчанию. В прочих случаях строковое значение может предполагать конкретный набор подходящих вариантов, одним из которых является пустая строка. В таком сценарии разумнее всего позволить варианту "default" действовать как флаг, указывающий на то, что необходимо сохранить именно значение по умолчанию.

5.4.3. Сериализация

Благодаря повсеместности стандарта Юникода практически все фреймворки сериализации и языки обрабатывают строки в основном одинаково. Это значит, в отличие от численных значений, изученных нами в разделе 5.3, мы уделим меньше внимания точности или тонким ошибкам переполнения и больше — безопасной обработке строк, которые могут охватывать весь спектр человеческого языка, а не только символы, используемые в западном мире.

Внутренне строки представляют собой не что иное, как наборы байтов. Однако то, как мы интерпретируем эти байты (кодирование), говорит нам, как преобразовывать их в то, что будет выглядеть уже как реальный текст — в каком бы языке мы ни работали. Проще говоря, когда дело доходит до сериализации используемой нами на сервере API строки, мы не можем просто отправить ее обратно в любой используемой на тот момент кодировке. Вместо этого нужно выполнять стандартизацию по единому формату кодирования и использовать его согласованно во всех запросах и ответах.

Несмотря на наличие большого количества форматов кодирования (например, ASCII, UTF-8 [<https://tools.ietf.org/html/rfc3629>], UTF-16 [<https://tools.ietf.org/html/rfc2781>] и т. д.), мир склонился к использованию именно UTF-8, поскольку он довольно компактен в отношении большинства символов и при этом достаточно гибок для кодирования всех возможных кодовых точек Юникода. Большинство ориентированных на строки форматов сериализации (например, JSON и XML; <https://www.w3.org/TR/xml/>) остановились на использовании именно UTF-8, в то время как другие (например, YAML) явно не обозначают, какое кодирование нужно использовать. Проще говоря, если нет весомой причины этого не делать, то для кодировки строкового содержимого API должен использовать формат UTF-8.

Если вы думаете, что всю работу можно поручить библиотеке, то *почти* правы, но не совсем. Оказывается, даже после того, как мы укажем тип кодировки, благодаря *формам нормализации* Юникода у нас остается еще много способов репрезентации одного и того же строкового содержимого (https://unicode.org/reports/tr15/#Norm_Forms). В некотором смысле это можно сравнить с вариациями представления числа 4: 4, 1 + 3, 2 + 2, 2 × 2, 8 / 2 и т. д. В строке UTF-8 можно проделать то же самое с помощью двоичных представлений. Например, символ è можно выразить как один особый символ (è или 0x00e9) либо как комбинацию

базового символа (е или 0x0065) и символа ударения (´ или 0x0301). Визуально и семантически эти два представления идентичны, но на диске выражаются разными байтами, а значит, для компьютера, который работает по принципу точного сопоставления, являются совершенно разными значениями.

Для обхождения этого нюанса в Юникоде (<http://www.unicode.org/versions/Unicode13.0.0/>) поддерживаются различные формы нормализации, и некоторые форматы сериализации (например, XML) во избежание смешивания и сопоставления этих семантически идентичных представлений текста стандартизованы по конкретной форме (в случае XML это Normalization Form C). И если для строки, представляющей развернутое описание ресурса API, это может не быть столь важным, то для строки, представляющей ID, данный нюанс имеет большое значение. Если идентификаторы могут иметь различные байтовые представления, то может получиться так, что один и тот же семантический ID будет ссылаться на разные ресурсы. В результате в API желательно отвергать входящие строки, не закодированные в UTF-8 с помощью Normalization Form C. А если речь идет о строках, представляющих идентификаторы ресурсов, то это даже обязательно. Подробнее тема ID и их форматов рассматривается в главе 6.

5.5. ПЕРЕЧИСЛЕНИЯ

Перечисления, служащие для программистов своего рода выпадающими селекторами, являются основным элементом в мире строго типизированных языков. И если в языках наподобие Java они могут быть весьма полезны, то их перенос в мир API зачастую становится ошибкой.

Обычно перечисления оказываются очень ценны, так как применяются и для проверки, когда допустимыми признаются только указанные значения, и для сжатия, когда каждое значение выражается числом, а не текстовой формой, на которую мы ссылаемся в коде. Однако когда дело доходит до веб-API, эти две предоставляемых ими положительных возможности обычно приводят к снижению гибкости и ясности (листинг 5.6).

Листинг 5.6. Перечисления в роли типов в API

```
enum Color {
    Brown = 1,
    Blue,
    Green,
}

interface Person {
    id: string;
    name: string;
    eyeColor: Color;
}
```

Здесь можно определить много
возможных цветов

Этот список в дальнейшем можно продолжить,
включая дополнительные варианты

Это перечисление можно
использовать для обработки
цвета глаз человека

Рассмотрим в качестве примера перечисление в листинге 5.6. Если мы используем его с целочисленными значениями, то в итоге `person.eyeColor` может оказаться установлен, например, на 2. Очевидно, что такой вид будет менее понятным, чем когда этот параметр установлен на `"blue"`, поскольку в первом варианте придется дополнительно выяснять, что конкретно означает это число. В коде это обычно не вызывает проблем, но все же, когда дело доходит до просмотра логов запросов, такой подход может доставить достаточно хлопот.

Более того, после добавления на сервер новых значений перечисления клиенту нужно будет обновить свою копию локального сопоставления (что зачастую требует обновления клиентской библиотеки), а не просто отправлять другое значение. Что еще хуже, если API решит добавить новое значение перечисления и клиенту не сообщат его смысл, то клиентский код просто не поймет, что ему нужно делать.

Рассмотрим пример сценария, в котором API добавляет новое значение `Color`, например `Hazel` (#4). Если в клиентский код не будет внесено это новое значение, то мы рискуем оказаться в непонятной ситуации. С другой стороны, если мы используем другой тип поля (например, строку), то можем получить аналогичную ошибку из-за неизвестного значения, зато уже не будем озадачены его смыслом (`"hazel"` намного понятнее, чем 4).

Говоря проще, если есть возможность использовать другой тип (например, строку), то перечислений обычно лучше избегать. Это особенно актуально в ситуациях, когда ожидается добавление новых значений, тем более если для рассматриваемого значения есть некий стандарт. К примеру, вместо того чтобы задействовать перечисления для допустимых типов файлов (PDF, документ Word и т. д.), будет намного безопаснее использовать строковое поле, которое допускает конкретные типы медиа (раньше назывались MIME-типы), такие как `"application/pdf"` или `"application/msword"`.

5.6. СПИСКИ

Теперь, когда мы разобрались с примитивными типами данных, можно начать знакомиться с их коллекциями, простейшими из которых являются список или массив. Если коротко, то списки — это не что иное, как группа неких других типов данных, таких как строки, числа, карты (см. раздел 5.7) или даже более короткие списки. Эти коллекции обычно адресуются с помощью *индекса*, то есть позиции элемента в списке, указываемой в квадратных скобках (например, `items[2]`), и подерживаются практически всеми форматами сериализации (например, JSON).

При том, что все системы хранилищ поддерживают списки элементов нативно, важно помнить, что цель API в том, чтобы предоставить максимально удобный интерфейс удаленному пользователю, а не раскрыть конкретные данные в том

виде, в котором они хранятся в базе данных. В связи с этой особенностью списки в современных веб-API зачастую используются неверно. Когда же их стоит использовать?

Как правило, списки отлично подходят для простых коллекций примитивов, которые выражают нечто, присущее ресурсу API. Например, если у вас есть ресурс **Book**, то с помощью списка строк вы можете выразить те же теги книги или категории (листинг 5.7).

Листинг 5.7. Сохранение списка категорий в ресурсе **Book**

```
interface Book {
    id: string;
    title: string;
    categories: string[];
}
```

Здесь у книг хранится список
прикрепленных строковых категорий



5.6.1. Атомарность

В этом примере есть один важный нюанс, который нужно запомнить: несмотря на наличие в полях списка множества элементов, эти элементы желательно рассматривать и использовать как атомарную часть данных, которую необходимо изменять и заменять целиком, а не по частям. Иначе говоря, если мы хотим обновить категории в ресурсе **Book**, то должны делать это путем полной замены в нем списка элементов. Другими словами, не должно быть возможности изменить второй элемент в поле списка. И на это есть много веских причин. Например, то, что порядок становится невероятно важен, когда мы обращаемся к элементам по их позиции в списке. При этом мы также не можем быть уверены, что в список между делом не будет добавлен элемент, который сместит другие элементы на новые позиции.

Кроме того, как и для любого другого поля ресурса, нежелательно, чтобы данные могли изменяться из разных мест. В связи с этим лучше не допускать возможность изменения содержимого поля списка несколькими методами. Это становится особенно заманчивым из-за принципов нормализации, на которые мы обучены полагаться в случае реляционных баз данных. К примеру, если для хранения списка категорий в ресурсе **Book** мы используем нечто вроде **MySQL**, то по факту можем задействовать отдельную таблицу **BookCategories** с уникальным идентификатором, внешний ключ для книги и строку категории с ограничением уникальности. Есть соблазн разрешить такое редактирование категорий, раскрыв для этого API с помощью предоставления ID книги и категории, которую нужно отредактировать (поскольку технически для уникального идентифицирования нужной строки этого достаточно).

Разрешение подобной модификации создает очень большую опасность, связанную с согласованностью: есть вероятность, что кто-нибудь может установить

список категорий. В то время как кто-то другой будет редактировать одну из них, используя другую точку входа. В этом случае даже использование других механизмов для изоляции транзакций (например, ETags; <https://tools.ietf.org/html/rfc7232#section-2.3>) не слишком поможет, и у нас получится API, полный сюрпризов.

Есть хорошее негласное правило, которое утверждает, что значения списка лучше всего рассматривать, как если бы этот список был строкой элементов в JSON-кодировке. Иначе говоря, вы устанавливаете не `book.categories = ["history", "science"]`, а что-то ближе к `book.categories = ["\history\", \"science\"]`. Вы вряд ли станете ожидать, что API позволит вам изменить даже один символ в строке, значит, не стоит ожидать и возможности изменить запись в поле списка.

Более подробно тематика ресурсов и их взаимосвязей освещается в части IV, а именно в главах 13–15, в которых описывается идея использования полей списков для выражения реляционных данных в веб-API.

Далее нужно рассмотреть, должно ли допускаться использование в списке разных типов данных. Иначе говоря, будет ли удачной идеей смешивать и сопоставлять строковые значения с числовыми в одном списке? Несмотря на то что подобный подход не приведет к катастрофе, он может внести путаницу, особенно учитывая рекомендации из подраздела 5.3.3 относительно обработки больших и десятичных чисел. Эти значения можно представить в виде строк, поэтому может оказаться сложным определить, является данная запись строкой или же числовым значением, выраженным с помощью строки. В связи с этим обычно лучше придерживаться использования одного типа данных и хранить значения списка в однородной форме.

5.6.2. Границы

Последний весьма распространенный вопрос по спискам касается их размера: что происходит, когда список становится очень большим и теряет управляемость? Чтобы избежать подобных проблем, все списки должны иметь определенные рамки, указывающие максимально допустимое количество элементов и максимальный размер одного элемента (обработка границ численных и строковых типов данных разбирается в подразделах 5.3.1 и 5.4.1). Кроме того, входные данные, превышающие установленные границы, должны отвергаться, а не обрезаться (по тем же причинам, которые описывались в подразделе 5.4.1). Это поможет избежать сюрпризов и путаницы, возникающих ввиду излишне громоздких списков значений, которые могут разрастаться до трудноуправляемых размеров.

Если потенциальный размер списка сложно оценить заранее либо есть веская причина ожидать его роста без жестко установленных границ, то лучше всего будет не встраивать список в ресурс, а задействовать подколлекцию фактических ресурсов. Несмотря на то что этот подход может показаться хлопотным, такой

API окажется намного более управляемым в будущем, поскольку ресурсы уже не станут невероятно огромными из-за неограниченных полей списков. Однако если в этом случае у вас все же возникнут проблемы, то рекомендую обратиться к паттерну пагинации (глава 21), который поможет разбить огромные неупорядоченные ресурсы на более мелкие управляемые части.

5.6.3. Значения по умолчанию

Списки аналогичны строкам тем, что в некоторых форматах сериализации и библиотеках тоже сложно различать значение `zero` (для списков это `[]`) и значение `null`. Хотя в отличие от строк, здесь практически всегда разумно использовать пустой список как значение, чтобы API не мог задействовать это пустое значение в качестве запроса на использование вместо него предустановленного. В итоге перед нами встает сложный вопрос: как указать в этом сценарии, что мы хотим, чтобы некое значение было значением по умолчанию, а не буквально пустым списком элементов?

К сожалению, в подобных ситуациях нет простого и элегантного ответа. Получается, либо можно использовать предустановленное значение при создании (с условием, что для создаваемого ресурса пустой список не будет допустимым), либо значение списка просто является неподходящим типом данных для этой информации. Если первый случай по той или иной причине не работает, то единственной безопасной альтернативой будет полностью пропустить это значение списка и начать работать с этими данными как с подобающей подколлекцией ресурсов, управляемых отдельно от родительского ресурса. Очевидно, что это не идеальное решение, но, учитывая ограничения, оно определенно является наиболее безопасным вариантом из доступных.

5.7. КАРТЫ

Наконец, можно обсудить наиболее универсальный и интересный тип данных из доступных: карты. В этом разделе мы разберем два похожих, но различающихся вида данных, основанных на паре «ключ — значение»: пользовательские типы данных (которые мы называли *ресурсами* или *интерфейсами*) и динамические карты «ключей — значений» (в JSON зачастую зовутся *объектами* или *картами*). Эти типы данных не идентичны, но разница между ними не столь велика и в основном проявляется в наличии предопределенной схемы. Иначе говоря, карты — это произвольные коллекции пар «ключ — значение», и пользовательские типы данных, или *подресурсы*, могут выглядеть точно так же. Тем не менее у них есть предопределенный набор ключей и конкретные типы для соответствующих значений. Начнем с рассмотрения тех, в которых используется предопределенная схема.

По мере развития ресурсов и предоставления ими все большего количества информации обычно схожие элементы этой информации мы начинаем группировать. К примеру, по ходу постепенного расширения конфигурации ресурса ChatRoom в определенный момент мы можем решить сгруппировать эти поля конфигурации (листинг 5.8).

Листинг 5.8. Сохранение данных в ресурсе или в отдельной структуре

```
interface ChatRoomFlat {
    id: string;
    name: string;
    password: string;
    requirePassword: boolean;
    allowAnonymousUsers: boolean;
    allowChatBots: boolean;
}

interface ChatRoomReGrouped {
    id: string;
    name: string;
    securityConfig: SecurityConfig;
}

interface SecurityConfig {
    password: string;
    requirePassword: boolean;
    allowAnonymousUsers: boolean;
    allowChatBots: boolean;
}
```

В базовом варианте без группировки все поля хранятся непосредственно в ресурсе

А здесь вся информация, относящаяся к доступу в ChatRoom, помещается в отдельную структуру

В этом случае мы просто сгруппировали несколько полей на основе общей темы управления безопасностью и доступом в комнату чата. Этот уровень абстракции имеет смысл только потому, что мы действительно мыслим в контексте группировки схожих полей одного ресурса. Если же поля относятся к ресурсу, но при этом фундаментально различаются и имеют значение вне контекста ресурса, то здесь может пригодиться подресурс-одиночка, о котором рассказывается в главе 12.

Теперь, разобравшись с пользовательскими типами данных и их предопределенными схемами, можно переключиться на динамические карты «ключ — значение». Несмотря на то что в конечном итоге они могут отображаться точно так же, эти две структуры зачастую используются совершенно по-разному. Если пользовательский тип данных мы использовали в качестве способа совмещения похожих полей и изолирования их внутри одного поля, динамические карты «ключ — значение» лучше подходят для хранения произвольных, динамических данных, не имеющих ожидаемой структуры. Иначе говоря, пользовательские типы данных просто представляют способ перестановки полей, которые мы знаем и хотим организовать для более удобного использования, а карты лучше

подходят для динамических пар «ключ — значение», ключи которых на момент определения API нам неизвестны. Более того, несмотря на возможное пересечение ключей среди ресурсов, здесь отсутствует требование для всех ресурсов иметь одинаковые ключи, как это должно быть в случае с полем пользовательского типа данных.

Подобный вид произвольной структуры пар «ключ — значение» отлично подходит для таких аспектов, как динамические настройки или конфигурации, зависящие от деталей конкретного экземпляра ресурса. Чтобы понять, о чем я, представьте, что у нас есть API для управления информацией о товарах в продуктовом магазине. Очевидно, что нам понадобится способ, позволяющий сообщить, какие ингредиенты и в каком количестве содержатся в каждом продукте, например 3 грамма сахара, 5 граммов белка, 7 миллиграммов кальция и т. д. Будет довольно сложно сделать это с помощью предопределенной схемы, перечисляющей все возможные ингредиенты. И даже если бы мы смогли сделать это таким образом, многие значения для этих элементов оказались бы нулевыми, поскольку в каждом продукте свой набор ингредиентов. В этом случае карта окажется более эффективной (листинг 5.9).

Листинг 5.9. Отслеживание ингредиентов и их количества с помощью поля карты

```
interface GroceryItem {
    id: string;
    name: string;
    calories: number;
    ingredientAmounts: Map<string, string>;
}
```

Можно использовать карту, сопоставляющую ингредиенты с их количеством

В листинге 5.10 показано, как может выглядеть JSON-представление `GroceryItem` при использовании определенной нами выше схемы. В этом случае ингредиенты динамические и могут свободно изменяться для каждого отдельного вида продукта.

Листинг 5.10. Пример JSON-представления карты ингредиентов

```
{
  "id": "514a0119-bc3f-4e3f-9a64-8ad48600c5d8",
  "name": "Pringles",
  "calories": "150",
  "ingredientAmounts": {
    "fat": "9 g",
    "sodium": "150 mg",
    "carbohydrate": "15 g",
    "sugar": "1 g",
    "fiber": "1 g",
    "protein": "1 g"
  }
}
```

Важно отметить, что ввиду возможности выбирать тип данных для ключей карты технически мы вольны выбирать что угодно. При этом некоторые варианты окажутся проблемными (например, какой-нибудь объемный тип, который сложно сериализовать). Иные варианты на первый взгляд могут выглядеть подходящими, но на деле оказываться нежелательными. Например, технически может иметь смысл позволить числовому значению выступать типом для ключа карты. Однако, как мы узнали в подразделе 5.3.3, числа могут вызывать проблемы, в частности, когда становятся огромными, а иногда и маленькими десятичными значениями. В результате на роль типа для ключей карты лучше всего подойдут, пожалуй, строки.

Еще один значительный нюанс. Поскольку ключи карт технически являются уникальными идентификаторами, также важно, чтобы эти строковые значения кодировались в формате UTF-8 и, как мы узнали в подразделе 5.4.3 (и узнаем еще лучше в главе 6), были представлены в Normalization Form C во избежание повторов значений ключей из-за особенностей их байтовой репрезентации.

5.7.1. Границы

В то время как схемы пользовательских типов данных позволяют избежать проблем с границами (схема определяет собственные условия ограничения), карты напоминают списки тем, что могут так же легко разрастаться до неуправляемого размера. В результате для них тоже нужно определять верхнюю границу, устанавливающую максимальное количество ключей и значений в поле. Этот момент является очень важным, поскольку позволяет понимать, насколько сильно может разрастись содержимое поля. Кроме того, важно устанавливать лимиты, определяющие, насколько большим может стать ключ или значение в виде строки (более подробную информацию об установке границ строковых значений см. в подразделе 5.4.1). Как правило, это подразумевает установку лимита примерно 100 символов для ключа и около 500 символов для значения.

В редких случаях размеры значений могут устанавливаться неравными. В итоге разработчики некоторых API предпочитают определять верхнюю границу для общего количества символов, сохраняемых в поле карты, позволяя пользователям распределять это количество между всеми значениями (к примеру, на несколько мелких значений или на одно-два, но очень больших). И хотя при необходимости такая стратегия является приемлемой, ее лучше избегать, поскольку она может стать опасной затеей, когда все больше и больше данных сохраняется в карте вместо того, чтобы размещаться в более подходящем месте.

5.7.2. Предустановленные значения

В отличие от списков, практически во всех форматах сериализации и языках программирования довольно легко провести различие между значением пустой карты, или нулем, (`{}`) и нулевым значением (`null`). В этом случае можно безопасно позволить значению `null` указывать, что API должен поступать с заданным полем на свое усмотрение, исходя из остального контекста запроса API. Пустая же карта, в свою очередь, предоставляет пользователю способ, позволяющий указать на то, что карта не должна содержать никаких данных.

5.8. УПРАЖНЕНИЯ

1. Компания в Японии, нацеленная исключительно на японоговорящих пользователей, хочет использовать для строковых полей API кодировку UTF-16 вместо UTF-8. Что ей нужно учесть, прежде чем принимать решение?
2. Представьте, что ресурс `ChatRoom` в данный момент архивирует все сообщения старше 24 часов. Если вы хотите добавить возможность отключения этого поведения, то как нужно назвать поле, которое будет за это отвечать? Окажется ли такой дизайн наилучшим для подобной настройки?
3. Если ваш API написан на языке, нативно поддерживающем произвольно огромные числа (например, Python 3), будет ли при этом лучшим вариантом сериализовать их в JSON в качестве строк? Или в данном случае можно опереться на стандартную численную сериализацию JSON?
4. Если вы хотите выражать нативный язык комнаты чата, то будет ли приемлемым использование для поддерживаемых языков значения перечисления? Если да, то как будет выглядеть это перечисление? Если же нет, то какой тип данных подойдет лучше?
5. Если в конкретной карте наблюдается совершенно неравномерное распределение размеров значений, то как лучше всего установить подходящие границы размеров для различных ключей и значений?

РЕЗЮМЕ

- Для каждого значения необходимо учитывать и `null`, и `undefined`, или отсутствующее значение, которые могут как иметь, так и не иметь общий смысл.
- Логические типы лучше всего подходят для флагов и должны именоваться так, чтобы истинное значение означало положительный аспект (например, `enableFeature` вместо `disableFeature`).

- Численные значения должны нести смысл, а не просто состоять из цифр.
- Чтобы избежать проблем с огромными числами (больше 32 или 64 бит) либо сложностей в арифметике с плавающей точкой в языках, не имеющих подходящих нативных представлений, численные значения нужно сериализовать в виде строк.
- Строки нужно кодировать в UTF-8, а те, которые используются в качестве идентификатора, необходимо нормализовать в форму Normalization Form C.
- Перечислений обычно желательно избегать, используя вместо них строковые значения и выполняя проверку на стороне сервера, а не клиента.
- Списки необходимо рассматривать как атомарные элементы-коллекции, индивидуально адресуемые только на стороне клиента.
- Как для списков, так и для карт нужно ограничивать общее количество допустимых в коллекции элементов. Однако карты должны дополнительно ограничивать размеры ключей и значений.

Часть III

Основы

В описываемых далее паттернах проектирования вместо того, чтобы рассматривать узкую, специфичную проблему, мы займемся изучением более обширных тем, применимых практически ко всем API. Эти ключевые паттерны не только определяют контекст для проектирования хороших API, но и послужат строительными блоками для паттернов, с которыми мы познакомимся позднее.

В главе 6 мы начнем с рассмотрения уникальной идентификации ресурсов в API. Затем в главах с 7-й по 9-ю определим основной набор паттернов взаимодействия со стандартными и пользовательскими методами (а также узнаем, как они применяются при работе с подмножествами ресурса). В главе 10 мы научимся обрабатывать продолжительно выполняющиеся методы API с помощью LRO (длительных операций) и, наконец, в главе 11 разберем, как организовывать выполнение повторяющихся процессов в API с помощью перезапускаемых задач.

Идентификация ресурсов

В этой главе

- ✓ Что такое идентификатор.
- ✓ Атрибуты хороших идентификаторов.
- ✓ Как выглядят хорошие идентификаторы.
- ✓ Реализация системы для создания идентификаторов.
- ✓ Как на основе этого руководства работать с UUID.

В этой главе мы подробно изучим идентификаторы ресурсов, разобрав, чем они являются, что определяет хороший ID и как их можно использовать в API. Кроме того, мы познакомимся с некоторыми распространенными идентификаторами (например, универсальным уникальным идентификатором (universally unique identifiers, UUID); <https://tools.ietf.org/html/rfc4122>), а также с новыми пользовательскими форматами, предназначенными специально для использования в веб-API.

6.1. ЧТО ТАКОЕ ИДЕНТИФИКАТОР

Что же конкретно мы имеем в виду под *идентификаторами ресурсов*? Если коротко, то ID дает возможность уникально обращаться к отдельным ресурсам API и говорить о них. В более техническом смысле эти идентификаторы представляют набор байтов (обычно строковое значение или целое число), которые можно

использовать как способ, позволяющий указать на конкретный ресурс в коллекции. Практически всегда ID дают возможность пользователям API выполнять определенный тип поиска. Иначе говоря, эти идентификаторы применяются для обращения к одному ресурсу, находящемуся в обширной коллекции ресурсов.

На деле подобные идентификаторы мы довольно часто используем в повседневной жизни. Однако не все они одинаково полезны (или даже одинаково уникальны). К примеру, во многих странах в качестве ID гражданина используются его имя и дата рождения, но известно множество случаев, когда людей путали из-за совпадения этих данных. Более удачный вариант, по крайней мере в США, — это девятизначный страховой номер (social security number, SSN), который используется для уникальной идентификации человека в большинстве финансовых транзакций (оплата налогов, получение кредита и т. д.). Однако и у этого номера есть недостатки. К примеру, на данный момент такие SSN могут охватить в общей сложности один миллиард человек (девять цифр), значит, по мере роста населения США эти номера могут просто закончиться.

Очевидно, что у ресурсов в API должны быть идентификаторы. Однако далеко не так очевидно, как именно они должны выглядеть. Иными словами, мы не проговорили, какие атрибуты делают ID хорошими или плохими. Так что в следующем разделе мы подробно рассмотрим именно этот вопрос.

6.2. ЧТО ДЕЛАЕТ ИДЕНТИФИКАТОР ХОРОШИМ

Практически для всех ресурсов, с которыми нам потребуется взаимодействовать в API, нужны идентификаторы, поэтому важность их наличия очевидна. И поскольку именно нам как разработчикам API нужно выбирать их представление, то возникает большой вопрос: что же делает ID хорошим? Чтобы ответить на него, потребуется изучить много разных атрибутов и параметров, между которыми нужно будет выбирать. Пока что мы сосредоточимся исключительно на самих атрибутах, а более конкретные рекомендации рассмотрим позднее в разделе 6.3.

6.2.1. Простота использования

Начнем с самого простого и очевидного: идентификаторы должны быть удобны в использовании в большинстве типичных сценариев. Возможно, самым распространенным случаем здесь будет поиск отдельных ресурсов в веб-API (стандартный метод GET, о котором мы узнали в разделе 1.4). Это означает, что отправка на сервер запроса в формате: «Пожалуйста, выдайте мне ресурс <ID>» должна осуществляться легко и удобно, минимизируя вероятность ошибок. В частности, здесь необходимо учитывать, что идентификаторы зачастую будут показываться

в URI. Это означает, что использование, к примеру, прямого слеша (/) или другого зарезервированного символа в ID вызовет сложности, поскольку они имеют особое значение в URI и в этом контексте их нужно избегать.

6.2.2. Уникальность

Есть еще одно очевидное требование к хорошему идентификатору: он должен быть поистине уникальным. Иначе говоря, ID должен по определению быть единственным в своем роде, иначе он просто не будет способен правильно выполнить свою задачу по уникальному идентифицированию одного ресурса в API. На первый взгляд это может показаться очень простым, но есть ряд тонкостей относительно того, что значит быть уникальным. Одна из них гласит, что уникальность редко является абсолютной и обычно зависит от контекста или диапазона, в котором рассматривается идентификатор. К примеру, известное всем имя компьютерной компании Apple на самом деле не является уникальным (существуют еще Apple Records, Apple Bank и т. д.). В этом смысле нам необходимо решить, должен ли ID быть уникальным во всех ресурсах одного типа, во всех ресурсах в одном API или во всех ресурсах в мире. Надо признать, что истинная глобальная уникальность теоретически невозможна, но на практике должна быть достижима при условии достаточно большого пространства ключей (возможных вариантов идентификаторов) и отсутствии намеренных злоумышленников.

6.2.3. Постоянство

Далее идет более тонкая идея, что идентификаторы после присваивания, скорее всего, изменяться не должны. В первую очередь это требование обусловлено возможным появлением проблем (а с ними и дополнительных решений) в сценариях, когда ID все же изменяются. Представьте случай, в котором `Book(id=1234, name="Becoming")` меняет свой идентификатор на `Book(id=5678, name="Becoming")`. Само по себе это не станет большой проблемой, если не рассматривать вариант создания новой книги: `Book(id=1234, name="Design Patterns")`. В этом случае для новой книги будет повторно использован исходный идентификатор (1234). В результате в ответ на запрос `Book 1234` вы будете получать разные результаты, в зависимости от того, *когда* будет сделан этот запрос.

В большинстве сценариев это не вызовет особых сложностей, но представьте, что пользователи могут указывать свою любимую книгу и кто-то выбирает в ее качестве `Book 1234 (User (favoriteBook=1234, ...))`. Здесь возникает разумный вопрос: какая именно книга для этого пользователя является любимой: *Becoming* или *Design Patterns*? В этом мире непостоянных идентификаторов, чтобы получить ответ на подобный вопрос, нам также нужно знать, когда книга была выбрана в качестве любимой или когда именно извлекается `Book 1234`. Без этой

информации будет неясно, о какой конкретно книге идет речь. Таким образом, лучше всего рассматривать ID даже не как постоянные, а как одноразовые. То есть как только возникает Book 1234, вы больше никогда не должны использовать этот идентификатор, даже после удаления Book 1234.

6.2.4. Быстрая и легкая генерация

До сих пор в качестве примеров идентификаторов мы использовали простые числа наподобие 1234, но в реальности они вряд ли будут именно такими. Вместо этого ID наверняка будут включать определенный элемент случайности, а не опираться на возрастающие числа. Поскольку обычно это означает, что при выборе идентификаторов происходит больше вычислений, важно убедиться, что этот выбор не займет слишком много времени.

К примеру, раз мы сказали, что все идентификаторы должны быть постоянными и использоваться единожды, значит, нам нужно убедиться, что любой выбираемый ID прежде не использовался. Для этого можно организовать список ресурсов и всякий раз случайно выбирать из него ID, проверяя, не использовался ли он прежде. Но это будет не самым эффективным решением. Со временем такой подход будет работать все медленнее и медленнее, становясь все менее пригодным для нашей задачи. Какой бы вариант мы в итоге ни предпочли, он должен быть быстрым, простым и, что самое главное, иметь прогнозируемые характеристики производительности.

6.2.5. Непредсказуемость

Помимо учета базовой скорости и простоты, а также их прогнозируемых показателей, важно иметь в виду такой критерий, как сложность предсказания очередного ID. В конечном итоге это упирается в безопасность и достигается с помощью достаточно большого пространства ключей. Но если метод выбора следующего доступного номера окажется так же прост, как выбор самих идентификаторов, то это упростит обнаружение и эксплуатацию потенциальных уязвимостей, например плохо настроенных правил безопасности.

К примеру, если атакующий будет даже просто проводить разведку, произвольно перебирая ресурсы на предмет их незащищенности в пространстве ключей из 256-битных целых чисел, выделенном случайным образом, то у него будет очень мало шансов наткнуться хотя бы на существующий ресурс, не говоря уже о плохо настроенном. С другой стороны, если мы используем существенно более прогнозируемый метод, например отсчет целых чисел в одном пространстве ключей (1, 2, 3...), то атакующий гарантированно найдет существующий ресурс и вполне может обнаружить некорректно настроенный, который, вместо того чтобы быть закрытым от посторонних глаз, окажется раскрытым.

6.2.6. Читабельность, передача и возможность проверки

Хоть нам как инженерам не хотелось бы это признавать, иногда используемые в API идентификаторы требуется передавать по телефону, вставлять в текстовые сообщения или передавать через другие не вычислительные устройства. Это значит, нам не нужна, к примеру, путаница между цифрой 1, символом l в нижнем регистре, I в верхнем регистре и вертикальной чертой (|), поскольку запись этих знаков на бумаге может оказаться довольно коварной (попробуйте передать 1111|11111111111111 по телефону, что сложно даже при использовании шрифта кода!). Если кратко, то мы должны учитывать, что идентификаторы будут интерпретироваться и передаваться людьми, поэтому важно, чтобы ID в этом отношении не оказывались излишне сложными.

Помимо простоты передачи, хороший идентификатор позволяет легко определить, не был ли он скопирован с ошибкой. Иначе говоря, у нас должна быть возможность отличать валидный ID, указывающий на ничто, от полностью недействительного, который никогда не будет ни на что указывать. Простой способ, позволяющий обеспечить это, — использовать в идентификаторе сегмент его контрольной суммы. Это вовсе не безумная идея, и такой прием применяется довольно часто. К примеру, ID, используемые для книг, называются ISBN (https://ru.wikipedia.org/wiki/Международный_стандартный_книжный_номер) и будут работать только таким образом. В этом случае последняя цифра ISBN выступает в роли контрольной, позволяющей проверить, были ли верно введены все предыдущие.

6.2.7. Информационная плотность

Наконец, поскольку идентификаторы будут использоваться все время, то важную роль будут играть размер и пространственная эффективность. То есть мы должны стараться вложить максимум информации в минимальное значение. Это может подразумевать выбор более плотного набора символов, такого как Base64 (<https://tools.ietf.org/html/rfc4648>) вместо менее плотного, такого как простые числительные. Здесь важно учесть, что при использовании численных ID мы можем сохранять до десяти вариантов каждого символа, а при использовании текста в кодировке Base64 — до 64.

Оптимизация уплотнения информации будет выступать балансиrom вместе с другими атрибутами, определяющими хорошие идентификаторы. К примеру, в Base64 доступно много символов, но они включают и i, и I, и цифру 1, что, как мы недавно узнали, при записывании или чтении может вызвать затруднения. Чтобы понять, как все это объединяется, мы рассмотрим различные вопросы проектирования, касающиеся выбора формата ID.

6.3. КАК ВЫГЛЯДИТ ХОРОШИЙ ИДЕНТИФИКАТОР

Теперь, когда мы хорошо понимаем, какими должны быть атрибуты хорошего идентификатора, можно реализовать схему для определения ID ресурсов. Сначала мы пройдемся по предполагаемой реализации и затем сравним ее с различными критериями, описанными в разделе 6.2.

6.3.1. Тип данных

Прежде чем мы сможем переходить к чему-либо еще, нужно разобраться с типом данных для хранения идентификаторов. Несмотря на наличие множества разумных вариантов, строки пока что являются наиболее гибким, а значит, и рекомендуемым решением. Они обеспечивают наибольшую плотность информации (7 бит энтропии на символ, используемый по HTTP), привычны в использовании и предоставляют свободу действий при принятии решений о том, как сделать их удобными для передачи, чтения и копирования.

Несмотря на то что есть и несколько других вариантов, начиная от стандартных (например, положительные целые числа) и заканчивая менее привычными (сырые неинтерпретированные байты), они создают искусственные ограничения, накладываемые их типом (например, целые числа ограничены десятью вариантами на символ ASCII или ~3,3 бита энтропии на символ ASCII при передаче по HTTP), и имеют прочие недостатки (например, с сырыми байтами неудобно работать во многих языках программирования).

Поскольку мы решили остановиться на строках, то перед нами возникает множество вопросов. Для начала, какой у этих строк будет набор символов? Можно ли использовать любую валидную строку Юникода? Есть ли в этом плане какие-то ограничения? После этого нужно решить, все ли символы будут допустимыми. К примеру, в HTTP прямой слеш ("/") обычно используется в качестве делителя, значит, в идентификаторе не нужен. Кроме того, как мы узнали ранее, одни символы (например, вертикальную черту "|") легко спутать с другими (например, "I", "1" или "l"). Что делать с ними? Начнем с рассмотрения набора символов.

6.3.2. Набор символов

Я бы с удовольствием вспомнил и объяснил Юникод со всем его внутренним устройством, но на это ушло бы слишком много времени, да и мои знания по этому стандарту наверняка имеют пробелы. В результате я сосредоточусь

на некоторых простых примерах с наборами символов, чтобы прояснить, почему эта реализация будет опираться на ASCII (American Standard Code for Information Interchange, американский стандартный код для обмена информацией).

ASCII довольно прост: семь бит данных для выражения 128 различных символов. Эти данные обычно можно втиснуть в восьмибитный байт, используемый в HTTP-запросах при взаимодействии с большинством современных веб-API. Нетрудно догадаться, что существует куда больше 128 символов, и задача Юникода состоит именно в том, чтобы решить этот вопрос. Это замечательно. Тем не менее в стремлении обеспечить максимальную обратную совместимость Юникод фактически позволяет использовать несколько способов для выражения одного и того же фрагмента текста. Например, *á* можно представить как одну кодовую точку (U+00E1) или как сочетание *a* и кодовых точек символа ударения (U+0061 и U+0301). На наших экранах или на бумаге оба варианта будут выглядеть как *á*, однако в базе данных будут сохранены по-разному — что и является проблемой, так как компьютеры видят не визуальную форму символа *á*, а лежащие в его основе байты.

И хотя эту проблему можно решить разными способами (например, использовать Normalization Form C или другие приемы Юникода), эта неоднозначность приносит намного больше вреда, чем пользы (например, теперь нам придется определять, как обрабатывать данные, отправленные не в Normalization Form C, и т. д.). В итоге наиболее безопасным вариантом для современных веб-API будет использование уникальных идентификаторов ASCII.

Теперь, когда мы остановились на выборе ASCII, нужно решить, будем ли мы в силу других причин дополнительно сокращать его до подмножества, выбранного из 128 символов.

6.3.3. Формат идентификатора

Несмотря на возможность использовать все 128 символов ASCII (или даже все строки, которые способен представить Юникод), есть один важный и желательный атрибут идентификаторов, который мы обсуждали в подразделе 6.2.6: простота чтения, передачи и копирования. Это означает, что нам просто нужно избегать символов ASCII, которые легко спутать с другими или которые могут спутывать URL в HTTP-запросе. Это приводит нас к очень полезному формату сериализации: разработанному Дугласом Крокфордом Base32 (<https://www.crockford.com/base32.html>).

Base32 опирается на использование 32 из 128 доступных символов ASCII. Сюда входят буквы от A до Z и цифры от 0 до 9, но отбрасываются многие специальные символы (например, прямой слеш), а также I, L, O и U. И если первые три отброшены во избежание путаницы (I и 1 можно перепутать с 1, а O с 0), то четвертый

был исключен по другой интересной причине: сквернословие. Оказывается, при исключении символа U снижается вероятность появления в идентификаторах англоязычных нецензурных слов.

Учитывая все вышесказанное, вы могли подметить и еще один неясный момент: почему мы решили отбросить L? Ведь в верхнем регистре эту букву ни с чем не спутаешь; однако ее можно спутать в нижнем регистре (с 1). Причина в том, что этот формат имеет единую каноническую форму, состоящую из букв верхнего регистра (за вычетом упомянутых) и цифр от 0 до 9, но гораздо больше подвержен ошибкам при декодировании из различных входных источников. В этом случае вместо того чтобы просто сказать, что L является недопустимым символом, данный алгоритм будет предполагать, что с ней (с ее нижним регистром) перепутали цифру 1, и станет рассматривать ее именно так. Эта нечувствительность к регистру работает более обобщенно (например, а рассматривается как A) и относится в том числе к другим потенциально путаемым символам, таким как O и o (которые будут трактоваться как 0). Иначе говоря, данный формат чрезвычайно восприимчив к ошибкам и спроектирован так, чтобы выполнять правильные действия на основе предположения, что эти ошибки будут возникать.

Наконец, Base32 рассматривает дефисы как необязательные, то есть у нас есть гибкая возможность вводить их ради повышения читабельности. Это означает, что у нас может быть ID `abcde-12345-ghjkm-67890`, который в Base32 традиционно будет представлен как `ABCDE12345GNJMK67890`.

Если коротко, то коды Крокфорда отвечают довольно многим критериям, подчеркнутым в разделе 6.2: их легко использовать, делать уникальными, непредсказуемыми, они обеспечивают неплохую плотность информации (32 варианта или пять бит на символ ASCII) и, что самое важное, легко читаются, копируются и передаются благодаря удобному и быстрому процессу декодирования. Далее нам осталось разобрать еще несколько вопросов.

6.3.4. Контрольные суммы

Одно из ключевых требований, указанных в подразделе 6.2.6, — возможность отличать отсутствующий идентификатор (например, который не создавался или был удален) от того, который никогда не мог существовать, что обозначало бы очевидную ошибку в ID. Решить это можно включением в идентификатор определенной проверки целостности. Эта проверка обычно принимает форму фиксированного количества символов контрольной суммы, идущих в конце ID и генерируемых из остального его содержимого. Благодаря этому при парсинге идентификатора можно просто повторно вычислять символы контрольной суммы и проверять их совпадение. Если они не совпадут, значит, определенная часть ID была повреждена и его можно считать недействительным.

В Base32 есть простой алгоритм, основанный на делении с остатком. Если коротко, то мы рассматриваем значение как целое, делим его на 37 (наименьшее простое число больше 32) и вычисляем остаток от этой операции. Затем мы используем одну цифру Base37 (спецификация включает пять дополнительных символов для контрольных сумм), чтобы обозначить остаток.

6.3.5. Тип ресурса

В первую очередь мы уделяли внимание уникальной части ID, но при этом нам очень пригодится возможность узнавать по идентификатору тип обозначаемого им ресурса. Иначе говоря, ID ресурса наподобие `abcde-12345-ghjkm-67890` окажется не слишком полезен, если попутно нам не сообщат, какой у этого ресурса *тип* (например, чтобы решить, какой RPC-вызов сделать или какой URL посетить для получения этого ресурса). Одним из типичных решений будет добавить в начало ID имя соответствующей коллекции, например `books/abcde-12345-ghjkm-67890`. В итоге мы сможем получить такой ID, понять, на какой ресурс он указывает, и проделать с ним полезные действия. Используя прямой слеш в качестве делителя, мы получаем полноценный идентификатор, который впишется в HTTP-запрос (например, `GET/books/abcde-12345-ghjkm-67890`). Доступны и другие варианты (например, использование двоеточия, что будет давать ID наподобие `book:abcde-12345-ghjkm-67890`). Тем не менее, учитывая тесную связь с HTTP и активное использование в стандартных URL, лучшим вариантом для имени коллекции будет именно прямой слеш.

6.3.6. Иерархия и область уникальности

Теперь, когда мы решили, что будем использовать в ID имя коллекции (например, писать `books/1234` вместо `1234`), появляется еще один вопрос: как в идентификаторе работает иерархия? Например, если у нас есть и ресурсы `Book`, и ресурсы `Page` (при этом страницы принадлежат книгам), то будем ли мы выражать эту иерархическую связь в ID или же оставим каждый ресурс самостоятельным? Иначе говоря, будут ли ресурсы `Page` иметь идентификаторы наподобие `pages/5678` (ресурсы верхнего уровня) или же `books/1234/pages/5678` (представляя иерархическую связь в идентификаторе)?

Ответ в том, что иерархические связи очень полезны и должны использоваться в подходящих местах, но при этом имеют смысл лишь в определенном наборе сценариев. Слишком часто разработчики API опираются на иерархические связи как на выражение *ассоциации* между двумя ресурсами, а не подчиненности, что может вести к проблемам. Например, мы можем сказать: «Книга 1 сейчас находится на полке 1» и получить идентификаторы ресурсов наподобие `shelves/1/books/1`. Поскольку книги со временем перемещаются с полки на

полку, нам нужно будет изменять ID, чтобы отразить эти перемещения. И, как вы можете догадаться, это идет вразрез с требованиями постоянства, указанными в подразделе 6.2.3.

Установить правило «никогда не перемещать книги между полками» может оказаться разумным (тогда выбранный идентификатор будет полностью приемлем), однако подобное ограничение не является необходимым. Вместо этого мы должны просто заметить, что текущая позиция книги на полке является изменяемым атрибутом самой книги, и сохранить `shelfId` как свойство ресурса `Book`.

Когда именно будет иметь смысл использовать иерархические идентификаторы? Как правило, API должны опираться на иерархические отношения, только когда представляют *владение* одного ресурса другим, в частности, фокусируясь на каскадируемых характеристиках этой связи. Это значит, мы должны располагать ресурсы друг под другом в случаях, когда нас интересует каскадирование между ними эффектов, например удаления (при удалении родителя удаляется его потомок), или каскадирование правил безопасности (устанавливаем правила безопасности в родителе, и они наследуются его потомками). При этом, как мы уже отмечали, ресурсы не должны иметь возможности перемещаться от одного родителя к другому. Например, если `Books` может перемещаться между `Shelves`, то `Pages` вряд ли смогут перемещаться между `Books`. Это означает, что нам нужно идентифицировать ресурс `Page` как `books/1/pages/2`, а не `pages/2`.

Кроме того, в случаях, когда принадлежность одного ресурса другому мы выражаем с помощью иерархических идентификаторов, важно помнить, что дочерний ресурс (страница) существует только внутри контекста своего родителя (книги). Это означает, что мы никогда не можем рассматривать страницу 2 отдельно, поскольку это ведет к очевидному вопросу: страница 2 какой книги? На практике это означает, что ID страницы всегда включает ID родительского ресурса.

И здесь возникает более тонкий, но при этом очень важный вопрос: может ли у нас быть несколько страниц с ID 2? Например, может у нас быть и `books/1/pages/2`, и `books/9/pages/2`? Думаю, этот пример должен предполагать очевидный ответ «да». В конце концов, в большинстве книг есть вторая страница. Поскольку родительский ресурс отличается, а идентификатор является комбинацией двух ресурсов в иерархической связи, то, несмотря на *похожесть* этих двух ID (оба заканчиваются на `pages/2`), сами их строки будут полностью различаться, а эти две страницы принадлежать к разным книгам (в данном примере — к книгам 1 и 9).

Получив представление о том, как должен выглядеть хороший API, мы перейдем к рассмотрению низкоуровневых деталей создания и работы с идентификаторами, которые будут отвечать этим ожиданиям.

6.4. РЕАЛИЗАЦИЯ

До этого момента мы изучали характеристики идентификаторов и их специфику, но есть и другие важные технические детали, о которых мы поговорим в текущем разделе. Начнем с рассмотрения простой и основополагающей характеристики ID: его размера.

6.4.1. Размер

Какую длину должен иметь конкретный идентификатор? Конечно, можно просто сказать, что со временем он изменяется (то есть по мере создания новых ID их длина может расти), но это все усложнит для пользователей API, которые могут хранить эти ID в своих базах данных. Поэтому обычно будет лучше обеспечить некоторую предсказуемость в требованиях к этим идентификаторам и просто установить фиксированный размер.

В этом случае в зависимости от стоящих задач у нас есть несколько вариантов. Начнем с самого простого. Нам могут потребоваться идентификаторы, которые будут уникальны только для одного типа ресурсов. Иначе говоря, у нас вполне могут быть `book 1` и `author 1`, поскольку различать 1 мы будем на основе разных типов ресурсов. В этом сценарии нам, скорее всего, хватит 64 бит пространства хранилища (аналогично одному 64-битному числу в реляционной базе данных наподобие MySQL). При использовании Base32 у нас есть 32 варианта (или пять бит) на один символ ASCII, то есть для ID нам может хватить всего 12 символов (60 бит). При сериализации (и десериализации) нам потребуется еще один символ для контрольной суммы, в результате чего мы получим уже 13 символов, создавая идентификаторы наподобие `books/AHM6A83HENMP~` (или `books/ahm6a83henmp~`, или `books/ahm6-a83h-enmp~`, где нижний регистр и дефисы улучшают читабельность).

В некоторых других случаях нам могут потребоваться идентификаторы, которые будут уникальными глобально, а не просто среди одного типа ресурсов или даже одного API. Для этого желательно будет сделать так, чтобы количество возможных ID было таким же, как в стандартных 128-битных UUID (универсальных уникальных идентификаторах), которые выражаются 32 шестнадцатеричными цифрами. Поскольку шесть из этих битов зарезервированы под метаданные ID, для значения самого идентификатора остается 122 бита. При использовании Base32 нам потребуется около 24 символов, которые дадут 120-битный ID. Прибавляем дополнительную цифру контрольной суммы и получаем всего 25 символов, из которых будут получаться идентификаторы наподобие `books/64s36d-1n6rv-kge9g-c5h66~` (помните, что дефисы необязательны и технически могут размещаться где угодно, а буквы не чувствительны к регистру).

Как мы узнаем позже, нам также может потребоваться дополнительный 0 в начале, позволяющий различить идентификаторы и псевдонимы, что увеличит наши стандартные ID еще на один символ. Ну и теперь, когда мы все это изучили, пора поговорить о важном, но довольно тонком нюансе: создании ID.

6.4.2. Генерация

Мы довольно много говорили о формате и типах данных, которые может иметь ID. Однако при этом предполагались просто случайные идентификаторы, и мы ничего не говорили о том, как их вообще создавать. Поэтому далее мы перейдем к рассмотрению данного процесса и начнем с его основополагающего аспекта: источника.

Источник

Прежде чем мы начнем разбирать генерацию идентификаторов, нам нужно понять, кто вообще их будет создавать. Здесь важно отметить, что позволять пользователям API самостоятельно выбирать ID — очень опасная затея, и лучше так не делать.

Во-первых, это может вызвать путаницу, когда имя уже где-то использовалось в прошлом. Поскольку наше требование к быстродействию (подраздел 6.2.3) означает, что идентификаторы никогда нельзя использовать повторно (даже после удаления), то пользователь, который захочет воссоздать ресурс, может озадачиться при получении сообщения о коллизии ID, ведь он считает, что ресурса с таким ID не существует (он может не существовать сейчас, но существовал однажды в прошлом). Конечно, это можно исправить с помощью достаточно большого набора доступных ID, но люди не слишком хорошо справляются с выбором из наборов случайных значений.

Во-вторых, при выборе собственных идентификаторов пользователи API склонны принимать неудачные решения. Как вариант, таковым может быть добавление в сам идентификатор личной информации. Иначе говоря, кто-то может создать книгу с ID `pad(base32-Encode("harry-potter-8"))` (равнозначен `D1GQ4WKS5NR6YX3MCNS2TE05==`), которую автор, естественно, захочет сохранить в тайне от мира. Тем не менее это может быть сложно, поскольку большинство систем не рассматривают ID (даже в кодировке Base32) как секретную информацию. Вместо этого идентификаторы обычно трактуются как прозрачные, случайные элементы данных, предназначенные для использования любым наиболее удобным путем. В результате хранить их в файлах журнала и обмениваться ими с другими пользователями считается вполне безопасным. Сюда же входит показ этих ID на больших информационных щитах в офисе, выделяющих

потенциальные ошибки или статистики отдельных ресурсов. Если API допускает пользовательские идентификаторы, то однажды может произойти случай, когда какой-нибудь крупный клиент (или правительство) потребует сохранить часть ID в секрете, что может оказаться чрезвычайно сложной или даже невозможной задачей.

Наконец, одно из последних требований к идентификаторам гласит, что они должны быть непредсказуемыми (см. подраздел 6.2.5). Несмотря на то что пользователи вашего API вполне могут обладать знаниями в области криптографии или генераторов псевдослучайных чисел, типичным такой случай назвать нельзя. В результате обращение к пользователям с просьбой выбрать собственный идентификатор может обернуться проблемой. В лучшем случае это будет просто раздражать («Почему вы не можете сами предоставить мне ID?»), в худшем же такой вариант станет просто опасным, даже если эта возможность окажется добровольной. В частности, такой подход создает для пользователей возможность генерировать прогнозируемые идентификаторы, делая их легкой мишенью для атак из-за ошибок в системе безопасности.

Генераторы псевдослучайных чисел

Как и в большинстве задач программирования, генерировать случайные идентификаторы мы можем разными способами. Мы рассмотрим два наиболее актуальных. Первый вариант — использовать криптографически безопасного генератора случайных байтов (например, функции `crypto.randomBytes()` из Node.js) и преобразование его результата в Base32 (с помощью, допустим, Node.js пакета `base32-encode`). Второй вариант — использовать алгоритм случайного выбора, чтобы выбрать установленное количество символов из алфавита Base32. Ни один из этих вариантов не является строго предпочтительным, они просто разные.

Первый ориентирован на количество байтов, использованных в случайно выбранном идентификаторе (выбирает ID в том виде, в котором он будет храниться в базе данных), а второй, наоборот, генерирует идентификатор в той форме, которую увидит пользователь. Если вы все просчитаете заранее, зная, сколько символов должны представлять ваши байты или сколько байтов нужно будет зарезервировать в базе данных под выбранные вами символы, то подойдет любой вариант.

Математика здесь довольно простая, поскольку мы просто выполняем преобразование между восьмибитными байтами и пятибитными символами. Это означает, что восемь бит каждого байта ID должны быть представлены символами, способными содержать пять бит. Если кратко, то это означает `base32Length == Math.ceil(bytes * 8 / 5)`.

В качестве примера рассмотрим генерацию случайного идентификатора, ориентированного на байтовое представление (листинг 6.1).

Листинг 6.1. Генерация случайного идентификатора в кодировке Base32

```
const crypto = require('crypto');
const base32Encode = require('base32-encode');

function generateId(sizeBytes: number): string {
  const bytes = crypto.randomBytes(sizeBytes);
  return base32Encode(bytes, 'Crockford');
}
```

Теперь, когда мы научились генерировать ID, нам также нужно убедиться, что он не используется и не использовался в прошлом. Рассмотрим, как это можно сделать.

6.4.3. Отметка об удалении (tombstoning)

Как мы узнали выше о постоянстве идентификаторов, важно, чтобы они использовались единожды за все время существования API. Это значит, даже после удаления ресурса нельзя повторно использовать освободившийся ID. Реализовать это можно многими способами, одни из которых требуют больше пространства хранилища, чем другие.

Один из простых вариантов — опереться на принцип *мягкого удаления* (разбираемый в главе 25). В этом паттерне проектирования вместо фактического полного удаления ресурса и его данных (жесткого удаления) мы просто отмечаем его как удаленный. Затем с точки зрения нашего кода генерации идентификатора мы просто продолжаем создавать случайные значения в поиске свободного, попутно проверяя базу данных, чтобы убедиться в реальной доступности этого ID. И хотя этот паттерн может быть полезен независимо от потребности в идентификаторах с отметкой об удалении, он подвержен известной проблеме непредсказуемого времени выполнения, которое статистически при добавлении новых ID и сокращении набора доступных ID будет лишь ухудшаться. Тем не менее при наличии 120-битного идентификатора размер пространства ключей будет примерно равен количеству бактериальных клеток на планете. То есть выбор одного и того же идентификатора дважды по вероятности близок к шансу дважды выбрать одну и ту же бактериальную клетку из всех живущих на планете. Иначе говоря, коллизия в таком случае становится очень маловероятной.

В случаях, когда использование паттерна мягкого удаления не имеет смысла (возможно, подразумеваемые данные очень значимы и по какой-то причине вам

необходимо реально от них избавиться), у вас есть и много других вариантов. Например, можно сохранять использованные идентификаторы где-нибудь в хеш-карте либо опереться на фильтр Блума и добавлять в этот фильтр ID при начале их использования. Как бы то ни было, коллизий все равно лучше избегать за счет достаточно большого размера идентификаторов относительно количества создаваемых ресурсов и использования криптографически безопасного источника случайности для генерации этих ID.

6.4.4. Контрольная сумма

Как отмечалось в подразделе 6.3.4, чтобы вычислить контрольную сумму, достаточно взять байтовое значение как целое число и использовать остаток от его деления на 37. Однако иногда код понятнее слов. Обратите внимание, что для обработки больших целых чисел, которые могут получаться из байтового буфера произвольной длины, мы будем опираться на тип `BigInt` (листинг 6.2).

Листинг 6.2. Вычисление значения хеш-суммы в формате Base32

```
function calculateChecksum(bytes: Buffer): number {
  const intValue = BigInt(`0x${bytes.toString('hex')}`);
  return Number(intValue % BigInt(37));
}
```

Начинаем с преобразования буфера байтов в значение `BigInt`

Вычисление контрольной суммы путем определения остатка от деления на 37

Кроме того, нам потребуется способ кодирования этой контрольной суммы в строку (а не просто численное значение). Для этого можно использовать алфавит Base32 с пятью дополнительными символами: *, ~, #, = и U (ввиду нечувствительности Base32 к регистру при парсинге значений сюда также относится u) (листинг 6.3). Таким образом, любое значение, которое получается в результате вычисления контрольной суммы (в диапазоне от 0 до 36), будет иметь символьное представление для использования в идентификаторе.

Листинг 6.3. Выбор символа для контрольной суммы с помощью алфавита Base32

```
function getChecksumCharacter(checksumValue: number): string {
  const alphabet = '0123456789ABCDEFGF' +
    'HJKMNPQRSTVWXYZ*~$=U';
  return alphabet[Math.abs(checksumValue)];
}
```

Определяем алфавит (Base32 плюс пять дополнительных символов)

Возвращаем символ в позицию `checksumValue`

Здесь может возникнуть логичный вопрос: хорошо, у нас есть символ контрольной суммы, и что с ним делать? Все довольно просто: мы присоединяем этот символ в конец нашего ID при отображении его пользователям, а при получении идентификаторов можем ориентироваться на этот символ, чтобы проверить их правильность. Начнем с обновления функции `generateId` для включения символа контрольной суммы (листинг 6.4).

Листинг 6.4. Генерация идентификатора в кодировке Base32 с символом контрольной суммы

```
const crypto = require('crypto');
const base32Encode = require('base32-encode');

function generateIdWithChecksum(sizeBytes: number): string {
  const bytes = crypto.randomBytes(sizeBytes);
  const checksum = calculateChecksum(bytes);
  const checksumChar = getChecksumCharacter(checksum);
  const encoded = base32Encode(bytes, 'Crockford');
  return encoded + checksumChar;
}
```

Вычисляем контрольную сумму и получаем соответствующий символ

Возвращаем сериализованный в Base32 идентификатор с присоединенным символом контрольной суммы

С другой стороны, при парсинге всех идентификаторов нам необходимо разделять их на две части: сам ID и символ контрольной суммы. Если при ее проверке окажется, что она неверна, то это не означает, что она была вычислена неправильно. Это говорит о том, что в идентификаторе была допущена опечатка или ошибка, и такой ID нужно отвергнуть как недействительный. Листинг 6.5 показывает этот принцип работы на практике, реализуя функцию `verifyIdentifier`, которая возвращает логическое значение, указывающее, можно ли рассматривать заданный идентификатор как действительный.

Листинг 6.5. Проверка идентификатора в формате Base32 на основе его контрольной суммы

```
function verifyId(identifier: string): boolean {
  const value = identifier.substring(
    0, identifier.length-1);
  const checksumChar = identifier[identifier.length-1];
  const buffer = Buffer.from(
    base32Decode(value, 'Crockford'));
  return (getChecksumCharacter(calculateChecksum(buffer))
    == checksumChar);
}
```

Разделяем идентификатор на две части: значение и контрольная сумма

Декодируем значение Base32 в сырые байты

Возвращаем ответ, указывающий, соответствует ли вычисленная контрольная сумма переданной

Теперь, когда мы рассмотрели, как вычислять и проверять значения контрольных сумм, пора переходить к вопросу о том, как все это работает с позиции хранилища данных.

6.4.5. Хранилище базы данных

Несмотря на все наши разговоры о нежелательности использования автоинкрементируемых целых чисел, в большинстве баз данных такой подход рекомендуется и они даже в некоторой степени оптимизированы под этот процесс. Здесь возникает очевидный вопрос: если мы используем этот новоиспеченный случайный идентификатор в кодировке Base32, то как нужно его сохранить, чтобы наши базы данных не падали из-за истощения или путаницы?

Сначала нужно решить, что конкретно нам нужно сохранять в базе данных. Очевидно, что сохранять надо будет сам идентификатор, но как быть с контрольной суммой? Это значение (и любые другие метаданные, которые у нас могут быть закодированы в ID при его передаче пользователям) *не должно* сохраняться в БД. Вместо этого нужно всегда вычислять его динамически в качестве этапа верификации для входящего ID (и динамически присоединять к исходящим ID). Если нам когда-нибудь придется вносить изменения в алгоритм вычисления контрольной суммы, то будет большим облегчением узнать, что при этом не нужно перебирать все записи в базе данных, чтобы они включали новый символ контрольной суммы.

Теперь, когда мы решили, что сохранять нам требуется лишь фактический идентификатор, нужно разобрать, как конкретно это делать. Для сохранения данных существует множество вариантов, но мы рассмотрим лишь три, начав с самого простого: строк.

Многие базы данных (особенно хранилища в формате «ключ — значение») очень удачно используют строковые значения в качестве идентификаторов. Если вам доведется работать с одной из таких БД, то сохранение ID станет для вас невероятно простой задачей: нужно будет просто сохранять строковое представление идентификатора вашего ресурса в том виде, в каком его видят клиенты (конечно, после удаления символа контрольной суммы). Это очень просто, поэтому используйте данный вариант, если его будет достаточно. Не забывайте и о том, что кодировка Base32 сохраняет порядок сортировки, то есть не должно возникать странностей, когда упорядочивание в базе данных работает не так, как на клиентской стороне.

К сожалению, не все базы данных хорошо справляются с сохранением (и индексированием) строковых значений. Более того, при сохранении строкового представления идентификатора вы по факту тратите лишнее пространство (поскольку каждый восьмибитный символ представляет лишь пять бит фактических данных). Учитывая все это, мы всегда можем пойти самым нижним уровнем и сохранять сырые байты. Многие базы данных неплохо справляются с использованием полей сырых байтов в качестве ID. При работе с такими БД для первичного ключа можно задействовать именно тип сырых байтовых данных (например, в MySQL это называется **BINARY**), обеспечивая ограничения уникальности и добавляя индексы, которые сделают запросы поиска по одному ключу быстрыми и простыми.

Наконец, если мы правильно выберем размер ID, то можем сделать его таким, что он прекрасно впишется в стандартный целочисленный тип (например, 32- или 64-битный). Если такое возможно, то это окажется, пожалуй, лучшим вариантом, поскольку практически все базы данных очень хорошо справляются с хранением и индексированием численных значений. Для этого мы просто берем байты, представленные нашим идентификатором, и рассматриваем их как представляющие

целое число, что мы уже научились делать при изучении вычисления контрольных сумм для ID (например, `byteBuffer.readBigInt64BEInt(0)`). Использование целого числа в качестве внутреннего формата хранения наверняка окажется самым широко поддерживаемым вариантом, поскольку согласуется почти со всеми системами баз данных.

6.5. UUID

Если вы не знаете, что такое UUID, то лучше пропустите этот раздел. Если коротко, то UUID — это общепринятые 128-битные форматы идентификаторов, имеющие относительно длинную историю (их спецификация, RFC-4122, датируется июлем 2005 года). По сути, это стандартный формат ID с большим количеством удобных возможностей, таких как пространства имен, множество доступных ID, а значит, и минимальный шанс коллизий. Мы не будем углубляться в подробности UUID, но вы можете распознать их по разделенной дефисами форме, которая выглядит примерно так: `123e4567-e89b-12d3-a456-426655440000`.

Если же вы знакомы с этим форматом и, дочитав до этого момента, задаетесь вопросом: «Почему мы просто не используем UUID?», — то вы не одиноки (ведь тогда, например, можно было бы просто пропустить данную главу). Если ответить коротко, то UUID отлично подходят, когда они нужны, но есть три важных причины, по которым не стоит заменять всю эту главу фразой «просто используйте UUID».

Во-первых, UUID огромны (122 полных бита пространства идентификаторов, 128 бит данных в общем), что оказывается чрезмерным для большинства типичных сценариев. В подобных случаях, если вы не планируете создавать триллионы ресурсов, то можете склониться к более короткому и читабельному формату ID.

Во-вторых, UUID в своем строковом формате не представляют особой информационной плотности, поскольку для передачи данных опираются только на шестнадцатеричную (Base16) запись. Это означает, что мы передаем в каждом символе ASCII четыре бита информации, а не пять, как происходит при использовании кодировки Base32. И если это может не иметь значения для компьютеров (все же современная компрессия довольно хороша), то когда нам нужно скопировать и поделиться идентификаторами нетехническими средствами (например, передать по телефону), высокая плотность информации может оказаться приятным преимуществом.

Наконец, UUID по определению не содержит в себе контрольной суммы. Это означает, что у нас нет удачного способа, позволяющего с высокой уверенностью отличить отсутствующий UUID от UUID, который никогда не мог существовать (а значит, мы имеем типографическую ошибку). В подобных случаях для того, чтобы различать эти два сценария, есть смысл внести в идентификатор

дополнительное значение контрольной суммы, возможно, используя метод ее вычисления с помощью Base32.

Если принять во внимание все вышесказанное, то может иметь смысл использовать UUID внутренне, а для клиента представлять идентификатор в виде строки Base32. Это позволит воспользоваться всеми преимуществами UUID, такими как пространства имен (UUID v3 и v5) или значения на основе временных меток (UUID v1 и v2), и в то же время набором возможностей Base32, например, контрольной суммой, читабельностью и высокой плотностью информации.

Вы можете создавать UUID v4 в кодировке Base32 (случайно генерируемый) с символом контрольной суммы (листинг 6.6).

Листинг 6.6. Генерация случайного UUID v4 в формате Base32

```

Генерируем UUID (в этом случае
случайный UUID v4) и сохраняем его в буфере
const base32Encode = require('base32-encode');
const uuid4 = require('uuid/v4');

function generateBase32EncodedUuid(): string {
  const b = Buffer.alloc(16);
  uuid4(null, b);
  const checksum = calculateChecksum(b);
  const checksumChar = getChecksumCharacter(checksum);
  return base32Encode(b, 'Crockford') + checksumChar;
}

```

В этом примере мы используем пакет `uuid` в NPM

Начинаем с выделения 16-байтового буфера для хранения UUID

Вычисляем контрольную сумму и получаем правильный символ контрольной суммы в формате Base 32

Возвращаем значение в кодировке Base32 (и контрольную сумму)

Этот способ также может быть актуален, если у нас есть база данных, которая лучше справляется с сохранением и индексированием UUID, а не строковыми, байтовыми или целочисленными представлениями идентификаторов, о чем мы говорили в подразделе 6.4.5.

6.6. УПРАЖНЕНИЯ

1. Спроектируйте новую схему кодирования/декодирования с большим размером контрольной суммы, выбранным относительно размера идентификатора.
2. Вычислите вероятность коллизии для случайно выбранного двухбайтового (16-битного) идентификатора. (Подсказка: загляните в задачу про дни рождения¹.)
3. Спроектируйте алгоритм, позволяющий избежать коллизий при использовании двухбайтового (16-битного) идентификатора, который не опирается на один счетчик. (Подсказка: подумайте о распределенном выделении памяти.)

¹ https://ru.wikipedia.org/wiki/Парадокс_дней_рождения.

РЕЗЮМЕ

- Идентификаторы — это значения, с помощью которых можно уникально указать на конкретные ресурсы в API.
- Хорошие идентификаторы просты в использовании, уникальны, постоянны, непредсказуемы, легко читаются, копируются и передаются. При этом они также быстро и легко генерируются и имеют высокую информационную плотность.
- С точки зрения клиента идентификаторы должны быть строками, которые используют набор символов ASCII, в идеале опираясь на формат сериализации Base32.
- В идентификаторах должен присутствовать символ контрольной суммы для того, чтобы отличать несуществующий ресурс от ID, который никогда не мог указывать на ресурс (и скорее всего, является результатом ошибки).



Стандартные методы

В этой главе

- ✓ Что такое идемпотентность и каковы ее побочные эффекты.
- ✓ Разбор всех стандартных методов (Get, List, Create, Update, Delete).
- ✓ Дополнительный полустандартный метод: Replace.
- ✓ Плюсы и минусы использования стандартных методов.

Из главы 1 мы узнали, что к характеристикам хорошего API в том числе относится прогнозируемость. Один из лучших способов получения прогнозируемого API — это варьирование только доступных ресурсов с сохранением согласованного набора конкретных действий (зачастую называемых методами), которые любой может выполнять с этими ресурсами. Это означает, что все подобные методы должны детально согласовываться как внешне, так и в поведении. В противном случае, когда одно и то же действие при выполнении для разных ресурсов не является одинаковым, прогнозируемость полностью теряется. Этот паттерн описывает конкретные правила, которым нужно следовать при реализации стандартных методов в ресурсах API.

7.1. ЗАЧЕМ ЭТО НУЖНО

Одним из наиболее ценных аспектов грамотно спроектированного API является то, что пользователи могут применять уже известные им знания, чтобы быстро освоить работу этого API. Иначе говоря, чем меньше сведений нужно изучать

о новом API пользователю, тем быстрее он сможет начать применять его для достижения своих целей. Один из способов реализации этого, зачастую выражаемый в RESTful API, заключается в использовании набора стандартных действий (методов) в различных определяемых API ресурсах. Пользователи по-прежнему должны будут узнать о ресурсах в этих API, но, освоив их, уже окажутся знакомы с набором стандартных методов, которые можно выполнять в этих ресурсах.

К сожалению, это работает, только если каждый стандартный метод по-настоящему стандартизирован, как во всем API, так и более обобщенно среди большинства веб-API. К счастью, уже давно существует стандарт REST, который заложил основу для такой стандартизации веб-API. На основе этого, во многом ориентируясь на спецификацию REST, данный паттерн выделяет все стандартные методы и различные правила, которые нужно применять к каждому из них.

7.2. ОБЗОР

Поскольку наша цель — добиться большей согласованности и получить более предсказуемый API, начнем с рассмотрения списка предлагаемых стандартных методов и их общих целей (табл. 7.1).

Таблица 7.1. Обзор стандартных методов

Имя	Поведение	Пример
Get	Извлекает существующий ресурс	GetChatRoom()
List	Выводит список коллекции ресурсов	ListChatRooms()
Create	Создает новый ресурс	CreateMessage()
Update	Обновляет существующий ресурс	UpdateUserProfile()
Delete	Удаляет существующий ресурс	DeleteChatRoom()
Replace	Заменяет весь ресурс	ReplaceChatRoom()

Несмотря на то что эти методы и их описания могут казаться простыми, здесь не хватает большого количества информации, что создает неопределенность. В конечном итоге она ведет к различным интерпретациям того, как должен действовать каждый метод. В результате один и тот же «стандартный» метод в других API или даже в других ресурсах одного и того же API действует по-разному.

В качестве примера рассмотрим один метод: **Delete**. Согласно описанному в табл. 7.1 поведению, он удаляет существующий ресурс. Иначе говоря, можно

ожидать, что если ресурс был создан и пользователь его удаляет, то мы должны получить эквивалент кода ответа HTTP 200 OK и двигаться дальше. Но что, если ресурс еще не существует? Должен ли тогда ответ быть OK? Или мы должны получить ошибку 404 Not Found?

Одни могут утверждать, что смысл этого метода в удалении ресурса, и раз ресурс больше не существует, то метод сделал свою работу и должен вернуть OK. Другие будут указывать на то, что важно отличать имевшийся результат (при выполнении метода ресурс не существовал) от результата поведения (ресурс был конкретно удален выполнением метода), а значит, попытка удалить несуществующий ресурс должна возвращать ошибку Not Found, указывая, что сейчас этот ресурс не существует, но при этом он не существовал и при выполнении метода.

Далее, что, если ресурс существует, но пользователь, пытающийся выполнить методы, не имеет к нему доступа? Должен ли тогда результат быть ошибкой 404 Not Found? Или ошибкой 403 Forbidden? Должны ли эти коды ошибок отличаться в зависимости от фактического существования ресурса? В конечном итоге это тонкое решение в процессе разработки должно предотвратить существенную проблему безопасности. В этом случае если неавторизованный пользователь попытается извлечь ресурс, то сможет определить, действительно ли тот не существует (получив ответ 404 Not Found) или же существует, но у запрашивающего нет к нему доступа (возвращается ответ 403 Forbidden). Благодаря этому пользователь, не имеющий прав доступа, может исследовать систему на предмет существующих ресурсов и в дальнейшем потенциально использовать их для атаки.

Однако разработчикам API не обязательно каждый раз снова отвечать на эти вопросы. Вместо этого руководство данной главы дает ряд стандартных ответов, которые гарантируют согласованность (и предотвращают утечки информации или прочие проблемы) в стандартных методах таким образом, который обеспечит гармоничное развитие по мере расширения API. Тем не менее здесь мы не только рассмотрим набор правил, но изучим и сами эти типы утонченных проблем.

7.3. РЕАЛИЗАЦИЯ

Прежде чем перейти к специфике каждого стандартного метода, мы начнем с изучения межкультурных аспектов, которые касаются всех этих методов. Первым вопросом будет такой: обязательно ли каждый ресурс должен поддерживать все стандартные методы? Иначе говоря, если один из моих ресурсов является неизменяемым, должен ли он все равно поддерживать стандартный метод Update? Или, если он постоянный, должен ли все равно поддерживать стандартный метод Delete?

7.3.1. Какие методы должны поддерживаться

Поскольку цель стандартизации набора методов состоит в поддержании согласованности, может показаться странным, что этот вопрос вообще обсуждается. Иными словами, зачем отводить целую главу под изучение реализации набора стандартных моделей поведения одним и тем же способом, только чтобы затем позволить некоторым ресурсам вообще отказаться от реализации этих методов. И это правильное замечание, но в действительности практичность все же является критическим компонентом дизайна API, и нам не избежать реальных сценариев, в которых определенным ресурсам не потребуется поддерживать все стандартные методы. Проще говоря, не все такие методы необходимы каждому типу ресурса.

Тем не менее сценарии бывают разными, и будет важно отличать случаи, когда метод вообще не должен поддерживаться (например, возвращать эквивалент HTTP-ошибки **405 Method Not Allowed**), от тех, когда метод просто не разрешен в конкретном экземпляре (возвращается эквивалент HTTP-ошибки **403 Forbidden**). При этом не должен возникать сценарий, в котором API просто опускает определенный маршрут (например, метод **Update**) и при попытке использовать этот метод для ресурса возвращает ошибку **404 Not Found**. Если API будет так делать, то данное действие будет ошибочно указывать на то, что *ресурс* не существует: на самом деле виноват метод, который для этого типа ресурса отсутствует технически.

Каковы правила, позволяющие определить необходимость поддержки всех стандартных методов? Каких-то жестких требований здесь нет, но есть общее руководство, согласно которому каждый стандартный метод должен существовать в каждом ресурсе, если только нет веской причины для обратного (эта причина должна быть задокументирована и объяснена). Например, если ресурс в действительности является подресурсом-одиночкой (см. главу 12), то существует только благодаря существованию своего родителя. В результате смысл для него будут иметь лишь стандартные методы **Get** и **Update**; остальные можно полностью проигнорировать. Это также является примером случая, когда метод в корне не имеет смысла, в результате чего должна возвращаться ошибка, равнозначная **405 Method Not Allowed**.

В иных случаях определенные методы могут оказываться бессмысленными для конкретного экземпляра ресурса, но при этом быть нужными для самого типа этого ресурса. Например, если в хранилище для определенного каталога установлен флаг **write protection**, который запрещает изменение находящихся в нем ресурсов, API все равно должен поддерживать стандартные методы (например, **Update** и **Delete**), но возвращать ошибку при попытке выполнения этих действий для ресурса, если в данный момент они не разрешены. Сюда также включаются типы ресурсов, которые могут рассматриваться как постоянные

(то есть не подлежать удалению) или неизменяемые (то есть не должны меняться). Но то, что они постоянны или неизменяемы сейчас, еще не означает, что их статус не станет другим в будущем. В связи с этим будет разумнее реализовать полный набор стандартных методов и просто возвращать код ошибки 403 `Forbidden` при чьей-либо попытке изменить или удалить эти ресурсы.

Ну а теперь, разобравшись, когда и какие методы нужно определять, пора рассмотреть одно из основных требований к стандартным методам: отсутствие побочных эффектов.

7.3.2. Идемпотентность и побочные эффекты

Поскольку критической частью стандартных методов является их предсказуемость, то, естественно, они должны делать только то, что заявлено, и ничего более. Проще говоря, стандартные методы не должны иметь побочных эффектов или неожиданного поведения. Более того, некоторые методы обладают свойством, называемым *идемпотентность*, которое указывает, будет ли многократное повторение одного и того же запроса (с одинаковыми параметрами) иметь тот же эффект, что и один запрос. Нарушение общепринятых требований относительно этого свойства в определенных методах может привести к катастрофическим последствиям, таким как потеря или повреждение данных.

Определить, что является допустимым и не вносит побочных эффектов, может быть непросто, поскольку во многих API есть сценарии, когда требуется дополнительное поведение, помимо того, которое указано простым стандартным методом. При этом зачастую стандартный метод может выглядеть подходящим местом для размещения такого поведения. Это может вызывать еще большую путаницу, когда дополнительное поведение оказывается не столь явным или просто не изменяет какое-либо значимое состояние, в результате чего мы все еще можем считать такой метод идемпотентным.

Что представляет собой побочный эффект? Некоторые из них являются очевидными. К примеру, в API электронной почты создание письма и сохранение его информации в базе данных не будет рассматриваться как побочный эффект. Но если этот стандартный метод `Create` также связался по SMTP с сервером для отправки этого письма, то подобное поведение уже считается побочным эффектом (а значит, его использования в стандартном методе `Create` нужно избегать).

А как быть со счетчиком запросов, когда при каждом извлечении ресурса значение счетчика увеличивается с целью отслеживания количества извлечений с помощью стандартного метода `Get`? Технически это означает, что данный метод больше не идемпотентен, поскольку внутреннее состояние изменяется, но будет ли это такой уж большой проблемой? Сложность в том, что все зависит от обстоятельств. Будет ли такое обновление счетчика влиять на производительность,

когда запрос окажется значительно более медленным, или станет варьироваться больше, чем в противном случае? Что происходит, если обновить счетчик не удастся по любой причине? Можно ли получить ответ с ошибкой, когда извлечение ресурса завершается отказом просто потому, что в счетчике случился технический сбой, помешавший ему обновиться?

В конечном счете, как мы узнаем в главе 24 при определении политики версионирования, решение о том, что является побочным эффектом, в некоторой степени зависит от трактовки. Если ярких побочных эффектов (таких как подключение к сторонним или внешним сервисам либо активация дополнительной работы, которая может вызвать сбой стандартного метода или привести к частично выполненному запросу) нужно избегать любой ценой, то некоторые более тонкие случаи с учетом ожиданий клиента могут оказаться уместными для API.

Теперь, разобравшись с общими аспектами стандартных методов, мы перейдем к более подробному изучению важных нюансов каждого и начнем с методов только для чтения.

7.3.3. Метод Get

Задача метода `Get` довольно проста: возвращать данные из записи ресурса, хранящейся в сервисе (листинг 7.1). Для этого метод получает всего один элемент входных данных: идентификатор нужного ресурса. Иначе говоря, это поиск данных строго по принципу «ключ — значение».

Листинг 7.1. Пример стандартного метода `Get`

```
abstract class ChatRoomApi {
    @get("/{id=chatRooms/*}")
    GetChatRoom(req: GetChatRoomRequest): ChatRoom;
}

interface GetChatRoomRequest {
    id: string;
}
```

Стандартный метод `Get` всегда извлекает ресурс по его уникальному идентификатору

Результатом выполнения метода `Get` всегда должен быть сам ресурс

Как и большинство методов, которые не изменяют активно внутренние данные, `Get` тоже должен быть идемпотентным. То есть при многократном выполнении он всегда должен давать один и тот же результат, если только параллельно не вносятся какие-то иные изменения. Это также означает, что данный метод, как и все стандартные методы, не должен иметь заметных побочных эффектов.

И хотя о нем можно сказать еще многое, например, об извлечении конкретных ревизий ресурса, эти темы мы обсудим далее, когда будем изучать будущие паттерны проектирования. Сейчас же перейдем к следующему стандартному методу только для чтения: `List`.

7.3.4. Метод List

Поскольку метод `Get` представляет поиск исключительно по принципу «ключ — значение», то для просмотра всех доступных ресурсов нам, естественно, нужен другой механизм, которым и является метод `List`. В нем мы указываем конкретную коллекцию, которую хотим просмотреть, и в результате получаем список всех ресурсов, принадлежащих этой коллекции.

Важно отметить, что стандартный метод `List` отличается от `Get` своей целью: вместо конкретного ресурса мы запрашиваем список ресурсов, принадлежащих конкретной коллекции. Она сама может принадлежать другому конкретному ресурсу, но так бывает не всегда. Например, рассмотрим случай, в котором у нас может быть набор ресурсов `ChatRoom`, каждый из которых содержит коллекцию ресурсов `Messages` (листинг 7.2). Для вывода списка ресурсов `ChatRoom` потребуется обратиться к коллекции `chatRooms`, в то время как вывод ресурсов `Message` из `ChatRoom` будет направлен на коллекцию `messages`, принадлежащую конкретной `ChatRoom`.

Листинг 7.2. Пример стандартного метода `List`

```
abstract class ChatRoomApi {
    @get("/chatRooms")
    ListChatRooms(req: ListChatRoomsRequest): ListChatRoomsResponse;

    @get("/{parent=chatRooms/*/}/messages")
    ListMessages(req: ListMessagesRequest): ListMessagesResponse;
}

interface ListChatRoomsRequest {
    filter: string;
}

interface ListChatRoomsResponse {
    results: ChatRoom[];
}

interface ListMessagesRequest {
    parent: string;
    filter: string;
}

interface ListMessagesResponse {
    results: Message[];
}
```

При выводе ресурсов, принадлежащих коллекции верхнего уровня, цели нет

При выводе ресурсов, принадлежащих подколлекции ресурса, целью будет родительский ресурс (в данном случае ресурс `ChatRoom`)

Несмотря на то что этот принцип вывода набора ресурсов может показаться таким же простым, как метод `Get`, на деле в нем кроется намного больше нюансов. И хотя мы не станем их рассматривать (например, некоторые аспекты будут описаны в будущих паттернах проектирования, в частности в главе 21), несколько тем все же стоит затронуть именно здесь. Начнем с контроля доступа.

Контроль доступа

Вдаваться в подробности здесь особого смысла нет, поэтому мы осветим всего один важный аспект: как должен метод `List` работать, когда разные запрашивающие стороны имеют разные права доступа к ресурсам? Иными словами, что, если нужно вывести все доступные ресурсы `ChatRoom` в API, но доступ есть только к некоторым из них? Как здесь должен действовать API?

Очевидно, что методы API должны быть согласованными в своем поведении, но значит ли это, что каждый ответ должен оказываться одинаковым для любого запрашивающего? Нет. Ответ должен включать только те ресурсы, к которым у запрашивающего есть доступ. Это значит, что разные люди при выводе одних и тех же ресурсов будут получать разные представления данных.

По возможности ресурсы желательно расположить так, чтобы минимизировать шанс возникновения подобных сценариев (например, обеспечить, чтобы ресурсы для одного пользователя имели отдельного родителя). В результате при выводе недоступных запрашивающему ресурсов будет возвращаться простая ошибка `403 Forbidden`. Тем не менее здесь не удастся избежать случаев, когда пользователь имеет доступ лишь к части элементов коллекции, но не ко всем.

Подсчет результатов

Зачастую возникает соблазн сопроводить вывод списка подсчетом элементов. Несмотря на то что показ общего количества совпадающих результатов окажется для клиентов приятным бонусом, такой прием зачастую доставляет хлопоты в дальнейшем, когда количество элементов в списке выходит за изначально предполагаемое. Ситуация особенно усложняется в случае распределенных систем хранения данных, которые не предназначены для предоставления быстрого доступа к подсчету элементов, соответствующих конкретному запросу. Если коротко, то обычно нежелательно включать в ответы на стандартный метод `List` подсчет.

Если же определенный подсчет результатов действительно необходим или участвующие в этом данные коллекции гарантированно будут оставаться достаточно небольшими, чтобы результаты подсчета обрабатывались, не создавая излишней вычислительной нагрузки, то обычно эффективнее всего показывать общее количество результатов с помощью приблизительной оценки, а не фактического подсчета. И если общее количество будет приблизительным, а не точным, также важно, чтобы отведенное для этого поле называлось соответственно (например, `resultCountEstimate`). Это поможет избежать путаницы относительно точности значения. Если вы решите, что подсчет результатов важен и целесообразен, то, учитывая возможные непредвиденные моменты, все равно лучше назвать поле приблизительным (даже если значение будет представлять точное количество). Это связано с тем, что, если в будущем вам для сокращения нагрузки на

хранилище потребуется переключиться на приблизительный подсчет, можно будет спокойно сделать это, не создавая путаницы или сложностей. В конце концов, точный подсчет технически является очень точной оценкой.

Сортировка

По тем же причинам применять к списку элементов сортировку тоже обычно не рекомендуется. Аналогично показу количества результатов возможность их упорядочивания является приятным бонусом, в частности для потребителей, которые отрисовывают пользовательские интерфейсы. К сожалению, разрешение сортировки результатов в стандартном методе `List` может вызвать еще больше трудностей, чем отображение их подсчета.

И хотя на ранней стадии жизни API добавление сортировки может оказаться простой задачей при относительно небольшом объеме данных, со временем этот процесс, скорее всего, будет все больше и больше усложняться. Возвращение в списке большого количества элементов требует более длительной сортировки сервером, но, что самое важное, в этом случае будет очень сложно применять глобальный порядок сортировки к списку, который собирается из нескольких хранилищ (как это бывает в случае распределенных систем хранения). Рассмотрим в качестве примера попытку объединить и упорядочить один миллиард элементов, разбросанных по ста разным хранилищам. Эту задачу будет сложно решить даже для статических данных при автономной обработке (даже если каждый источник данных сначала будет упорядочивать небольшую их часть) и еще сложнее — для меняющегося набора данных наподобие интерактивного API. Более того, выполнение такого вида сортировки приведет к перегрузке серверов API.

В целом эта крохотная функциональная возможность имеет тенденцию значительно увеличивать сложность в будущем при относительно небольшой ценности для потребителя API. Поэтому, как и в случае с подсчетом результатов, ее использование обычно окажется плохой идеей.

Фильтрация

Если упорядочивание и подсчет элементов, извлекаемых стандартным методом `List`, являются нежелательными, то фильтрация результатов обычно оказывается полезной, а значит, приветствуемой возможностью. Причина в том, что альтернативы применению фильтрации (например, запрос всех элементов и отфильтровывание результатов) исключительно затратны как по времени обработки, так и по нагрузке на сеть.

Когда дело доходит до реализации фильтрации, нужно учитывать важный нюанс. Несмотря на то что идея создать строго типизированную фильтрующую структуру (например, со схемой для разных ситуаций, а также оценочными условиями «и» и «или») может показаться соблазнительной, подобный дизайн обычно не оправдывает себя. В конце концов, есть весомая причина, по которой

SQL все еще принимает запросы как простые строки. Аналогично перечислениям, как показано в главе 5, по мере расширения функциональности или различных доступных для фильтрации типов клиентам требуется обновлять локальные схемы, чтобы воспользоваться этими изменениями. В результате, несмотря на то что фильтрация приветствуется, рекомендуется в качестве типа данных для выражения самого фильтра использовать строку. Ее затем можно парсить сервером и применять перед возвращением результатов, либо путем передачи считанного фильтра в систему хранения при наличии такой возможности, либо непосредственно с помощью сервиса API. Эта тема подробнее разбирается в главе 22.

До сих пор мы фокусировались на считывании данных из API. Теперь же пора сменить направление и рассмотреть их размещение в API с помощью метода `Create`.

7.3.5. Метод `Create`

Ни метод `Get` (подраздел 7.3.3), ни метод `List` (подраздел 7.3.4) не имеют никакой пользы, если в API отсутствуют данные. А основным механизмом для их размещения в API выступает стандартный метод `Create`. Его цель довольно проста: на основе предоставленной информации о ресурсе создать этот ресурс в API так, чтобы его можно было извлекать по идентификатору или находить во время вывода списка ресурсов. Для наглядности в листинге 7.3 показан пример создания ресурсов `ChatRoom` и `Message`. Как видите, запрос (отправленный с помощью HTTP-метода `POST`) содержит информацию о ресурсе, и итоговый ответ всегда будет являться этим созданным ресурсом. Кроме того, целью выступает либо родительский ресурс, если таковой доступен, либо просто коллекция верхнего уровня (например, `"/chatRooms"`).

Листинг 7.3. Пример стандартного метода `Create`

В обоих случаях метод `Create` использует HTTP-глагол `POST`

```

abstract class ChatRoomApi {
    → @post("/chatRooms")
      CreateChatRoom(req: CreateChatRoomRequest): ChatRoom; ←
    → @post("/{parent=chatRooms/*/}/messages")
      CreateMessage(req: CreateMessageRequest): Message; ←
    }

    interface CreateChatRoomRequest {
        resource: ChatRoom;
    }

    interface CreateMessageRequest {
        parent: string;
        resource: Message;
    }
}

```

Метод `Create` всегда возвращает созданный ресурс

И хотя принцип, стоящий за созданием ресурсов, довольно прост, нам нужно более детально изучить кое-какие не слишком заметные моменты. Начнем с краткого рассмотрения идентификации этих создаваемых ресурсов.

Идентификаторы

Как мы узнали в главе 6, обычно для создаваемых ресурсов лучше всего использовать идентификаторы, генерируемые сервером. Иначе говоря, мы должны позволить API выбирать ID для ресурса, а не делать это самим. Однако существует много случаев, когда разумнее оставить выбор потребителю API (например, если API используется мобильным устройством, синхронизирующим локальный и удаленный набор ресурсов). В подобных сценариях абсолютно приемлемо разрешить выбирать ID потребителям. Тем не менее они должны следовать рекомендациям, описанным в главе 6.

Если же вам необходимо предоставить идентификатор ресурса в момент его создания, то это нужно делать путем установки поля ID в интерфейсе самого ресурса. Например, чтобы создать ресурс `ChatRoom` с конкретным идентификатором, мы пропишем примерно такой HTTP-запрос: `POST /chatRooms { id: 1234, ... }`. Как станет ясно позднее, для этого есть альтернативный способ, подразумевающий использование полустандартного метода, но, поскольку не все API будут поддерживать этот метод, лучше устанавливать ID вместе с методом `Create`.

Согласованность

Еще несколько лет назад можно было не сомневаться, что каждый создаваемый нами элемент данных будет сохранен в одной базе данных. Обычно речь шла о реляционной БД наподобие MySQL или PostgreSQL. Сегодня же существует гораздо больше вариантов хранения, которые масштабируются горизонтально и при увеличении объема данных не подвергаются сбоям. Иначе говоря, когда у вас оказывается слишком много данных для текущей конфигурации БД, вы можете просто активировать дополнительные узлы хранилища, и система станет лучше справляться с большим объемом данных.

Такие системы стали чудодейственным лекарством против эпидемии огромных наборов данных и невероятно большого количества запросов, но зачастую имеют недостатки. Один из самых распространенных связан с согласованностью, когда *конечная согласованность* является типичным побочным эффектом, казалось бы, бесконечной масштабируемости. В хранилищах с конечной согласованностью данные постепенно реплицируются по системе, но обычно достижение всех доступных узлов занимает какое-то время, что ведет к ситуации, когда данные были обновлены, но реплицировались еще не везде. Это означает, что после создания ресурса какое-то время он может не отображаться в запросе `list`. Что еще хуже, в зависимости от настройки маршрутизации есть вероятность,

что вы создадите ресурс, но при попытке его извлечь с помощью метода `Get` получите ошибку `404 Not Found`.

И хотя в зависимости от используемой технологии это может оказаться неизбежным, такую ситуацию необходимо стараться по возможности исключать. Одна из важнейших особенностей API состоит в его поддержке транзакционной семантики, и к ключевым аспектам этого поведения относится *строгая согласованность*. Это означает, что вы должны всегда иметь возможность сразу же считать записанные данные. Другими словами, это значит, что когда API сообщил о создании ресурса, тот должен быть создан в полном смысле и доступен для всех других стандартных методов. То есть у вас должна быть возможность увидеть его в выводе стандартного метода `List`, извлечь его с помощью метода `Get`, изменить с помощью `Update` и даже удалить из системы с помощью `Delete`.

Если вы окажетесь в позиции, когда у вас есть данные, которыми невозможно управлять таким образом, то вам следует всерьез задуматься о применении пользовательского метода для загрузки нужных данных (см. главу 9). Причина очень проста: API ожидает согласованности стандартных методов, но, как мы увидим в главе 9, пользовательские методы этому ожиданию не соответствуют. Поэтому вместо того чтобы задействовать согласующийся в конечном итоге метод `CreateLogEntry`, подумайте о применении пользовательского метода импорта наподобие `ImportLogEntries`, который наглядно пояснит потенциальным пользователям, что на согласование результатов по всей системе уйдет некоторое время. Если же вы точно знаете, когда закончится репликация данных по системе, то еще одним вариантом будет использовать длительные операции, о которых мы подробно поговорим в главе 10.

Теперь, когда у нас есть представление о тонкостях создания новых ресурсов, можно перейти к изменению существующих с помощью стандартного метода `Update`.

7.3.6. Метод Update

После того как новый ресурс будет загружен в API, если он сам не является неизменяемым, нам потребуется простой способ его изменения, в качестве которого и выступает метод `Update`. Задача этого метода — изменять существующую информацию одного ресурса, избегая при этом внесения побочных эффектов, о чем говорилось в подразделе 7.3.2.

Этот рекомендуемый способ обновления ресурсов опирается на HTTP-метод `PATCH`, указывающий на конкретный ресурс с помощью его уникального идентификатора и возвращающий этот ресурс в измененном виде. Одним из ключевых аспектов метода `PATCH` является то, что он частично изменяет ресурс, а не заменяет его полностью. Позднее мы разберем это более подробно, поскольку по

данной теме нужно очень многое обсудить (например, как отличать намерение пользователя «не обновлять это поле» от намерения «установить это поле пустым»), но пока главное — понять, что метод `Update` должен обновлять только те аспекты ресурса, которые явно просит обновить потребитель API (листинг 7.4).

Листинг 7.4. Пример стандартного метода `Update`

```
abstract class ChatRoomApi {
    @patch("/{resource.id=chatRooms/*}")
    UpdateChatRoom(req: UpdateChatRoomRequest): ChatRoom;
}

interface UpdateChatRoomRequest {
    resource: ChatRoom;
    // ...
}
```

Метод `Update` изменяет только определенные элементы ресурса с помощью HTTP-глагола `PATCH`

В главе 8 мы узнаем, как безопасно обрабатывать частичное обновление ресурсов

Метод `Update` идеально подходит для изменения существующего ресурса, но все еще есть некоторые сценарии, когда обновлять лучше по-другому. Речь идет о сценариях наподобие перехода из одного состояния в другое, которые лучше реализовывать с помощью альтернативного действия. Например, вместо того чтобы устанавливать для ресурса `ChatRoom` статус `archived`, с большей вероятностью для этого будет использован пользовательский метод `ArchiveChatRoom()` (см. главу 9). Если кратко, то, несмотря на то что метод `Update` является стандартным механизмом изменения существующего ресурса, это далеко не единственный способ реализации данной задачи.

7.3.7. Метод `Delete`

Когда ресурс исполнил свою роль в API, его нужно удалить. Именно для этого служит стандартный метод `Delete`. В листинге 7.5 показано, как этот метод может выглядеть при удалении из API ресурса `ChatRoom`. Вполне ожидаемо он опирается на HTTP-метод `DELETE` и нацеливается на нужный ресурс по его уникальному идентификатору. Более того, в отличие от большинства других стандартных методов, здесь типом возвращаемого значения будет сообщение, в определении API выражаемое как `void`. Причина в том, что успешный результат выполнения `Delete` — это полное исчезновение ресурса.

Листинг 7.5. Стандартный метод `Delete`

```
abstract class ChatRoomApi {
    @delete("/{id=chatRooms/*}")
    DeleteChatRoom(req: DeleteChatRoomRequest): void;
}

interface DeleteChatRoomRequest {
    id: string;
}
```

Для удаления ресурса метод `Delete` использует HTTP-глагол `DELETE`

Результатом является пустое сообщение ответа, а не фактический интерфейс ответа

Идемпотентность

Несмотря на то что назначение этого метода тоже очень простое, здесь возможна некоторая путаница в том, нужно ли считать его идемпотентным и как обрабатывать запросы на удаление уже удаленных ресурсов. В конечном итоге это сводится к вопросу о том, метод **Delete** ориентируется больше на результат, будучи декларативным, или же на действия, являясь императивным. С другой стороны, можно представить удаление ресурса в виде простого запроса, с помощью которого вы объявляете, что указанный ресурс больше не должен существовать. В этом случае удаление уже удаленного ресурса должно рассматриваться успешным ввиду того простого факта, что по завершении обработки запроса ресурс отсутствует. Иначе говоря, выполнение стандартного метода **Delete** дважды подряд с одинаковыми параметрами даст один и тот же успешный результат, а значит, метод является идемпотентным, как показано на рис. 7.1.

Если же взглянуть с императивной стороны, то можно представить метод **Delete** как запрос на выполнение действия, которое нацелено на удаление ресурса. Следовательно, если на момент получения такого запроса ресурс не существует, значит, выполнить запрос невозможно и он должен завершаться отказом. Иначе говоря, попытка удалить один ресурс два раза подряд может быть успешной в первый раз, но неудачной во второй, как показано на рис. 7.2. В итоге работающий таким образом метод **Delete** *не будет* считаться идемпотентным. Но какой из этих вариантов верный?

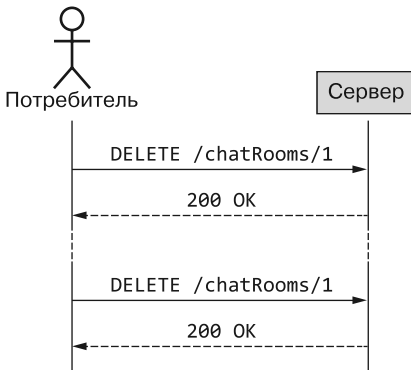


Рис. 7.1. Декларативное представление стандартного метода **Delete** демонстрирует его идемпотентное поведение

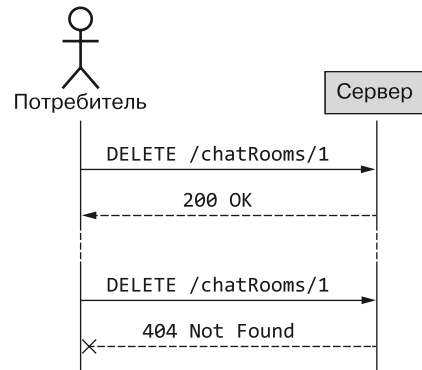


Рис. 7.2. Императивное представление метода **Delete** демонстрирует неидемпотентное поведение

Несмотря на то что в мире существует множество декларативных API (например, Kubernetes), ресурсно-ориентированные API обычно по своей природе императивны. В результате метод **Delete** должен следовать неидемпотентной

модели поведения. Иными словами, попытка удалить несуществующий ресурс должна завершаться отказом. Это может привести к различным осложнениям, связанным с возможным обрывом сетевых соединений и потерей ответов, но мы еще поговорим о повторяемости запросов API в главе 26.

Последним осталось рассмотреть полустандартный метод **Replace**, связанный с нашей реализацией стандартного метода **Update**.

7.3.8. Метод Replace

Как мы узнали в подразделе 7.3.6, метод **Update** отвечает за изменение данных существующих ресурсов. Там же говорилось о том, что для обновления только конкретных элементов ресурса он опирается исключительно на HTTP-метод **PATCH** (больше информации о котором мы узнаем в главе 8). Но что, если требуется обновить весь ресурс? Метод **Update** предоставляет удобный способ для установки в ресурсе конкретных полей и управления имеющимися. Но это означает, что если вам неизвестно о каком-то поле (например, оно было добавлено в последующей мини-версии, о чем мы поговорим в главе 24), то может получиться так, что в ресурсе будет присутствовать значение, существование которого вы не планировали.

Например, на рис. 7.3 мы видим потребителя, обновляющего ресурс **ChatRoom** с намерением установить поле **description** с помощью HTTP-метода **PATCH**. В этом случае если потребитель захочет стереть все теги в этом ресурсе, а клиент не будет знать о поле **tags**, то клиент никак не сможет выполнить такой запрос. Как же обработать подобный случай?

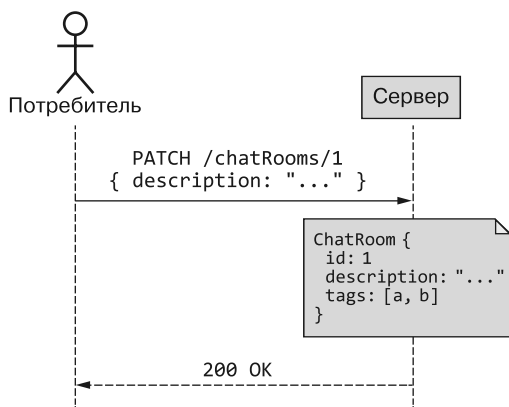


Рис. 7.3. Метод **Update** изменяет удаленный ресурс, но не гарантирует его итоговое содержимое

Именно для этой цели и предназначен полустандартный метод **Replace**: заменить весь ресурс информацией, предоставленной в данном запросе. Это значит, что даже если у сервиса есть дополнительные поля, о которых клиенту еще не известно, эти поля, при условии их отсутствия в запросе на замену ресурса, будут удалены, как показано на рис. 7.4.

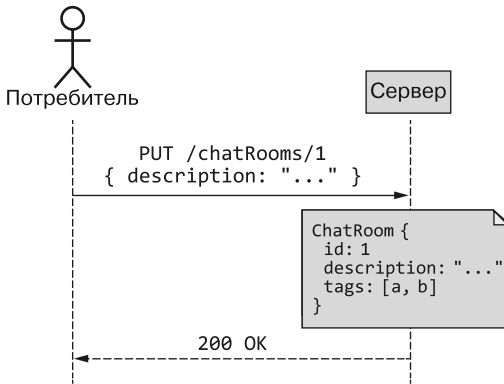


Рис. 7.4. Стандартный метод **Replace** гарантирует идентичность удаленного (remote) ресурса содержимому запроса

Чтобы этого достичь, метод **Replace** вместо HTTP-глагола **PATCH** использует HTTP-глагол **PUT**, нацеленный на сам ресурс, и предоставляет точную исчерпывающую информацию, которая должна храниться в этом ресурсе (листинг 7.6). По сравнению с выборочными изменениями, производимыми стандартным методом **Update**, такая полная замена гарантирует, что удаленная копия ресурса будет выглядеть так, как заявлено в запросе. То есть клиентам не придется беспокоиться о возможном присутствии неизвестных им дополнительных полей.

Листинг 7.6. Определение стандартного метода **Replace**

```

abstract class ChatRoomApi {
    @put("/{resource.id=chatRooms/*}")
    ReplaceChatRoom(req: ReplaceChatRoomRequest):
        ChatRoom;
}

interface ReplaceChatRoomRequest {
    resource: ChatRoom;
}
  
```

В отличие от стандартного метода **Update**, метод **Replace** опирается на HTTP-глагол **PUT**

Аналогично методу **Update**, метод **Replace** возвращает обновленный (или созданный) ресурс

Это ведет к потенциально озадачивающему вопросу: если мы можем заменить содержимое ресурса нужной нам информацией, нельзя ли использовать этот же

механизм для замены несуществующего ресурса? Иначе говоря, может ли этот магический метод `Replace` стать инструментом, позволяющим создавать новые ресурсы?

Если отвечать коротко, то да: этот механизм замены можно использовать для создания новых ресурсов, но он не должен становиться заменой стандартному методу `Create`, поскольку возможны случаи, в которых потребители API могут захотеть создать ресурс, если (и только если) этот ресурс еще не существует. Аналогичным образом они могут захотеть обновить ресурс, если (и только если) этот ресурс *уже существует*. При использовании метода `Replace`, несмотря на конечный успех, у вас нет возможности узнать, заменяете (обновляете) вы существующий ресурс или же создаете новый. Вместо этого вы знаете только то, что информация, однажды хранившаяся на месте конкретного ресурса, теперь заменена содержимым, переданным в запросе на замену.

Кроме того, как вы могли догадаться, создание ресурсов с помощью метода `Replace` означает, что API должен поддерживать пользовательские идентификаторы. А как мы узнали в главе 6, хоть это поведение и является возможным и допустимым в некоторых случаях, обычно по ряду причин его лучше избегать (см. подраздел 6.4.2).

На этом мы закончили изучение основных стандартных методов, с помощью которых будет строиться львиная доля взаимодействия в большинстве ресурсно-ориентированных API. В следующем подразделе мы коротко рассмотрим, как это может выглядеть при общем объединении.

7.3.9. Итоговое определение API

В листинге 7.7 показана коллекция всех стандартных методов, которые мы разбирали: `Create`, `Get`, `List`, `Update`, `Delete` и даже `Replace`. Этот пример продолжает предыдущие, используя ресурсы `ChatRoom` и их дочерние ресурсы `Message`.

Листинг 7.7. Заключительное определение API

```
abstract class ChatRoomApi {
    @post("/chatRooms")
    CreateChatRoom(req: CreateChatRoomRequest): ChatRoom;

    @get("/{id=chatRooms/*}")
    GetChatRoom(req: GetChatRoomRequest): ChatRoom;

    @get("/chatRooms")
    ListChatRooms(req: ListChatRoomsRequest): ListChatRoomsResponse;

    @patch("/{resource.id=chatRooms/*}")
    UpdateChatRoom(req: UpdateChatRoomRequest): ChatRoom;

    @put("/{resource.id=chatRooms/*}")
    ReplaceChatRoom(req: ReplaceChatRoomRequest): ChatRoom;
```

```

    @delete("/{id=chatRooms/*}")
    DeleteChatRoom(req: DeleteChatRoomRequest): void;

    @post("/{parent=chatRooms/*}/messages")
    CreateMessage(req: CreateMessageRequest): Message;

    @get("/{parent=chatRooms/*}/messages")
    ListMessages(req: ListMessagesRequest): ListMessagesResponse;
}

interface CreateChatRoomRequest {
    resource: ChatRoom;
}

interface GetChatRoomRequest {
    id: string;
}

interface ListChatRoomsRequest {
    parent: string;
    filter: string;
}

interface ListChatRoomsResponse {
    results: ChatRoom[];
}

interface UpdateChatRoomRequest {
    resource: ChatRoom;
}

interface ReplaceChatRoomRequest {
    resource: ChatRoom;
}

interface DeleteChatRoomRequest {
    id: string;
}

interface CreateMessageRequest {
    parent: string;
    resource: Message;
}

interface ListMessagesRequest {
    parent: string;
    filter: string;
}

interface ListMessagesResponse {
    results: Message[];
}

```

7.4. КОМПРОМИССЫ

Всякий раз, опираясь на стандарты, а не на специально разработанные инструменты, вы теряете часть гибкости. И к сожалению, если ваш сценарий не будет в точности соответствовать форме, для которой был разработан стандарт, этот компромисс может иногда оказаться довольно болезненным. Однако взамен вы практически всегда получаете немало пользы.

В некотором смысле это можно сравнить с покупкой одежды. Новая рубашка размера М может оказаться слегка великоватой, но размер S определенно будет мал. В итоге перед вами выбор: либо худеть, либо носить рубашку чуть большего размера. Однако преимущество в том, что эта рубашка стоит около 10 долларов, а не 100, поскольку производится массово. В похожем сценарии мы оказываемся при работе с ресурсно-ориентированными API и стандартными методами.

Эти стандартные методы подходят не для всех ситуаций. Время от времени в них будут встречаться несоответствия, например, когда нечто будет не создаваться, а скорее появляться. И вы в полной мере можете спроектировать для подобных уникальных случаев собственные методы (см. главу 9). Но в обмен на использование стандартных методов вместо создания полностью настраиваемых API на основе RPC вы получаете преимущество: ваши пользователи без особых усилий смогут быстро изучить различные методы. Что еще лучше, освоив эти методы (если они не сделали этого ранее при работе с RESTful API), пользователи будут знать, как эти методы функционируют во всех ресурсах вашего API, что мгновенно приумножит их знания.

Если кратко, то стандартные методы должны (и наверняка будут) на 90 % обеспечивать реализацию API. А для остальных сценариев у нас есть пользовательские методы, которые мы изучим в следующей главе. Тем не менее стандартизация использования набора общепринятых строительных блоков настолько полезна, что практически всегда лучшим способом будет создание API с помощью стандартных методов, а пользовательские варианты применяются, только когда этого требует некий непредвиденный сценарий.

7.5. УПРАЖНЕНИЯ

1. Допускается ли замена использования методов `Create` и `Update` методом `Replace`?
2. Что, если ваша система хранения не предполагает строгой согласованности создаваемых данных? Какие есть варианты для создания данных без нарушения принципов стандартного метода `Create`?
3. Почему стандартный метод `Update` опирается на HTTP-глагол `PATCH`, а не `PUT`?

4. Представьте стандартный метод `Get`, который также обновляет счетчик посещений. Является ли он идемпотентным? А идемпотентен ли стандартный метод `Delete`?
5. Почему в стандартном методе `List` нужно избегать включения результатов подсчета или поддержки пользовательской сортировки?

РЕЗЮМЕ

- Стандартные методы — инструмент, позволяющий повышать согласованность и предсказуемость.
- Критически важно, чтобы все стандартные методы следовали одним и тем же принципам поведения (например, все методы `Create` должны действовать одинаково).
- Идемпотентность — характеристика, определяющая возможность повторяющегося вызова метода с получением идентичных результатов.
- Не все стандартные методы должны быть идемпотентны, но они не должны иметь побочных эффектов, при которых вызов метода вносит в систему API дополнительные изменения.
- Несмотря на то что это может показаться нелогичным, стандартный метод `Delete` не должен быть идемпотентным.
- Благодаря стандартным методам нам доступен узкий диапазон моделей поведения и характеристик, но взамен мы получаем намного более простые в освоении API, которые позволяют пользователям опереться на уже имеющиеся у них знания о ресурсно-ориентированных API.

8

Частичное обновление и извлечение

В этой главе

- ✓ Почему иногда может потребоваться обновить или извлечь лишь часть ресурса.
- ✓ Как лучше всего передать сервису API интересующие его поля.
- ✓ Как обрабатывать отдельные элементы в сложных полях, таких как карты и интерфейсы.
- ✓ Нужно ли поддерживать адресацию отдельных элементов в повторяющихся полях (например, массивах).
- ✓ Как определить значения по умолчанию и обработать неявные маски полей.
- ✓ Как работать с недопустимыми спецификациями полей.

В главе 7 мы узнали, что нам важно иметь возможность обновлять ресурсы частично, чтобы не опираться постоянно на полную замену. В этом паттерне мы изучим использование *масок полей* в качестве инструмента обновления только нужных полей в конкретном ресурсе. Кроме того, мы разберем, как применять маски в той же задаче, но в обратном направлении, то есть как извлекать отдельные поля в ресурсе. Несмотря на то что возможность получать не весь ресурс, а лишь интересующие его части нужна реже, она особенно важна в чувствительных к памяти приложениях, таких как IoT-устройства, использующие вывод API.

8.1. ЗАЧЕМ ЭТО НУЖНО

В данном паттерне проектирования мы будем разбирать две стороны одной задачи, связанные с разными представлениями ресурса. Во всех API, встреченных нами к текущему моменту, мы рассматривали ресурсы только как самодостаточные, атомарные единицы. Иными словами, мы никогда не рассматривали возможность того, что ресурс можно разбить на составляющие его атрибуты и управлять им на более детальном уровне. И хотя это существенно упрощает некоторые парадигмы программирования (например, не нужно беспокоиться об управлении частью ресурса), в случаях, когда нам нужно быть более конкретными и выразительными во взаимодействии с API, могут возникать проблемы. Далее мы разберем два варианта реализации новой функциональности, которые это ограничение делает невозможными.

8.1.1. Частичное извлечение

В большинстве API при извлечении ресурса мы получаем либо его целиком, либо ошибку — все или ничего. Для многих из нас это не создает особых проблем. В конце концов, чего еще мы можем хотеть? Но что, если ресурс в этом API вырастет до размера в сотни полей и субполей? А если доступные запрашивающей стороне ресурсы сильно ограничены, как в случае с миниатюрным IoT-устройством? Или вдруг у нас ограниченная скорость сети или очень дорогая связь? В подобных сценариях возможность управления объемом информации, которая возвращается от API, внезапно становится очень важной.

Может показаться, что это актуально лишь для огромных ресурсов либо очень ограниченных условий. Однако нужно учитывать, что даже если одно новое поле и занимает очень мало места (а вместе с этим возникают и дополнительные вычислительные затраты, и расходуется пропускная способность), при выведении списка ресурсов этот минимальный объем занимаемого пространства умножается на количество элементов. И можно себе представить, как внезапно то, что было небольшим, в действительности разрастается в нечто гораздо большее, поскольку со временем количество ресурсов увеличивается. В конечном итоге предоставление пользователям API возможности извлекать только часть ресурса (или списка ресурсов) несет в себе огромную пользу.

8.1.2. Частичное обновление

За поддержкой частичного обновления стоит чуть более сложная логика, которая менее сосредоточена на факторах производительности и проблемах наподобие аппаратных ограничений. Вместо того чтобы беспокоиться о считывании конкретных частей ресурса, нас больше волнует возможность точно нацеливаться

и изменять конкретные поля. Так почему нас должен волновать подобный вид точечного обновления? Разве нам не важно, чтобы ресурс выглядел ровно так, как мы этого хотим, а не представлял коллекцию малых изменений?

Здесь важна согласованность данных. Как пользователи API, при наличии лишь стандартного метода **Replace** (см. главу 7) мы вынуждены обновлять все поля ресурса, а не только те, в которых заинтересованы. Переписывание полей, которые в данный момент не имеют для нас значения, без должной проверки согласованности может привести к потере данных, что явно нас не порадует. Чтобы понять, как такое может произойти, представьте следующий фрагмент кода (табл. 8.1), выполняемый двумя отдельными клиентами в отношении одного ресурса.

Таблица 8.1. Пример последовательности кода, выполняемого двумя пользователями, который может вызвать проблемы

Вызов API пользователем 1	Вызов API пользователем 2
<pre>let chatRoom = GetChatRoom({ id: '1' }); chatRoom.title = "My Chat!"; chatRoom = ReplaceChatRoom({ chatRoom: chatRoom, });</pre>	<pre>let chatRoom = GetChatRoom({ id: '1' }); let desc = fetchRandomQuote(); chatRoom.description = desc; chatRoom = ReplaceChatRoom({ chatRoom: chatRoom, });</pre>

Если эти две последовательности выполнить без какой-либо проверки согласованности или блокировки, то в итоге значение будет иметь только последний метод **Replace**. Получается, что вызов первого, также выполняемого пользователем 1, никогда и не происходил! Причем на первый взгляд ничто в этом коде не предполагает возникновение конфликта. Пользователь 1 хотел обновить заголовок ресурса **ChatRoom**, в то время как пользователь 2 хотел лишь изменить поле описания, что вообще с этим не связано. И даже если добавить сюда проверку согласованности, второй метод **Replace** будет завершаться отказом и потребует выполнить повтор, чтобы убедиться в фактической отprawке изменения.

Но это еще не вся проблема, и есть другие варианты, позволяющие спровоцировать потерю данных. Как мы более подробно узнаем в главе 24, API редко остаются неизменными. Вместо этого они склонны постепенно развиваться в целях включения новых возможностей и исправления багов в существующих. И один из наиболее частых сценариев, на который мы в итоге опираемся,

заключается в возможности добавлять в ресурсы новые поля, сохраняя обратную совместимость новой версии. Но это может создать серьезную проблему, если единственным механизмом обновления ресурсов выступает их полная замена и ваши локальные интерфейсы не согласуются с полным (новым) списком полей ресурса.

К примеру, представим, что хотим обновить заголовок ресурса `ChatRoom`, но за время, прошедшее с последнего обновления нашей клиентской библиотеки (когда в ресурсе `ChatRoom` мы сохраняли лишь поле заголовка и администратора), сервис добавил новое поле описания. В подобном контексте, если кто-то ранее установил описание (`description`) для ресурса, есть шанс, что наш стандартный метод `Replace` сотрет эти данные и мы вновь столкнемся с их потерей. Листинг 8.1 демонстрирует этот сценарий, показывая двух разных клиентов, обновляющих ресурс `ChatRoom`. В первом код соответствует всем последним изменениям и знает о новом поле описания. Во втором же код какое-то время не обновлялся и никогда не слышал об этом поле.

Листинг 8.1. Два клиента обновляют один ресурс

```
> let chatRoom = latestClient.GetChatRoom({ id: '1' });
> chatRoom.description = 'Description!';
> ReplaceChatRoom({ chatRoom: chatRoom });
> console.log(chatRoom);
{
  id: '1',
  title: 'Old title',
  description: 'description'
}

> chatRoom = oldClient.GetChatRoom({ id: '1' });
> console.log(chatRoom);
{
  id: '1',
  title: 'Old title',
}
> chatRoom.title = 'New title';
> ReplaceChatRoom({ chatRoom: chatRoom });

> chatRoom = latestClient.GetChatRoom({ id: '1' });
> console.log(chatRoom);
{
  id: '1',
  title: 'New title',
  description: null
}
```

Сначала мы используем «последний клиент», чтобы обновить описание комнаты

Если мы получаем ресурс с помощью старого клиента, то поле `description` будет отсутствовать

Затем мы без проблем обновляем заголовок с помощью старого клиента

Однако теперь при повторном использовании нового клиента поле `description` оказывается стерто

И хотя может показаться, будто здесь что-то не так, правда в том, что метод `Replace` делает ровно то, что от него ожидается. Напомню: он предназначен конкретно для того, чтобы сделать удаленный ресурс *в точности* таким, какой

передается в запросе. Это подразумевает удаление данных из всех отсутствующих полей, независимо от того, было ли это нашим намерением.

Если кратко, то цель частичного обновления состоит в том, чтобы позволить потребителю API выражать свои намерения более конкретно. Если он захочет заменить весь ресурс, то может воспользоваться стандартным методом **Replace**. Если же он захочет обновить всего одно поле, то должен иметь в своем распоряжении более точный механизм, который позволит выразить это намерение. В таком случае частичное обновление окажется отличным решением.

8.2. ОБЗОР

Чтобы достичь обеих целей (частичного извлечения и частичного обновления), нам достаточно опереться на один инструмент: маски полей. По сути они представляют просто набор строк, но эти строки выражают список полей, которые нас интересуют в конкретном ресурсе. Когда дело доходит до извлечения ресурса и мы хотим более точно выразить, какие поля нас интересуют, можно просто передать маску, указывающую эти поля, как показано на рис. 8.1.

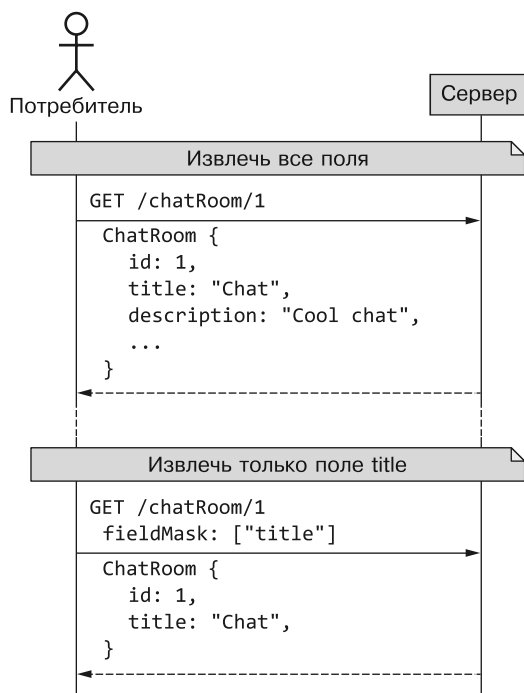


Рис. 8.1. Использование маски полей при извлечении ресурса

Этот инструмент можно использовать и для того, чтобы указывать, какие поля должен обновить сервис в ресурсе. В этом случае при обновлении ресурса можно передать список полей, которые мы хотим изменить, как показано на рис. 8.2.

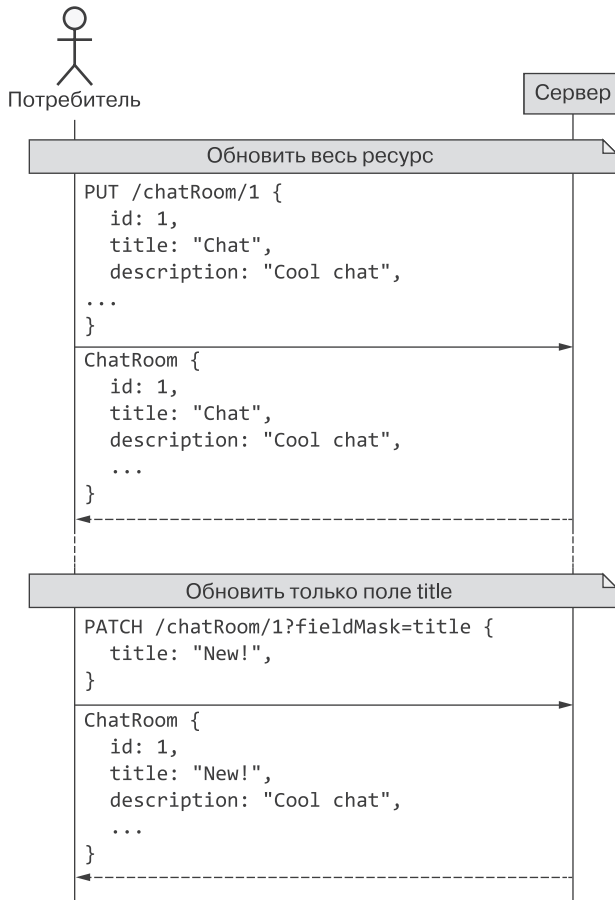


Рис. 8.2. Замена всего ресурса в сравнении с обновлением одного поля

Более того, поскольку JSON является динамической структурой данных, если в запросе **PATCH** будет отсутствовать сама маска полей, то ее можно вывести из атрибутов, присутствующих в объекте JSON (показан на рис. 8.3). И хотя этот вывод маски может сильно усложниться при работе в API с динамическими структурами данных, в большинстве случаев он дает наиболее ожидаемые результаты.

То, что мы увидели, может казаться простым, но есть немало пограничных случаев и коварных сценариев, которые оказываются куда более сложными, чем кажутся. В следующем разделе мы узнаем, как реализовывать поддержку масок полей в API.

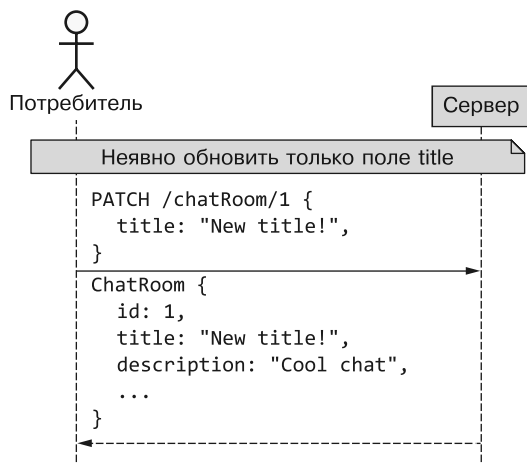


Рис. 8.3. Обновление одного поля с помощью неявной маски полей, выведенной из предоставленных данных

8.3. РЕАЛИЗАЦИЯ

Теперь, когда мы имеем общее представление о масках полей, нужно разобрать их работу более подробно. Иначе говоря, мы знаем, что у нас должна быть возможность отправлять этот список произвольных полей в запросах `GET` и `PATCH`, получая в результате более конкретное обновление или извлечение. Но как нам реализовать отправку этих полей? В конце концов, запрос `GET` не принимает тело, а запрос `PATCH` должен содержать в качестве тела весь ресурс. Разберем, как передавать маски полей, не вызывая существенных нарушений в стандартном запросе, который мы определили в главе 7.

8.3.1. Передача

Несмотря на то что маски полей несут в себе огромный потенциал, возникает важный вопрос: как передавать их на сервер API в запросе? И если учесть нашу тесную связь с HTTP, то как нам это сделать, не нарушая правил HTTP и дизайна ресурсно-ориентированных API? Это становится особенно сложным ввиду двух важных ограничений запросов `GET` и `PATCH`.

Начнем с того, что в запросах GET не допускается использование тела (и многие HTTP-серверы в случае передачи будут отбрасывать его). Это значит, мы никак не можем с помощью тела HTTP-запроса указать поля, которые хотим извлечь. А вот с запросами PATCH ситуация в некотором смысле обратная. Тело в них использовать можно (именно с его помощью мы обновляем сам ресурс), однако ресурсно-ориентированный дизайн требует, чтобы оно являлось представлением обновляемого ресурса. Иными словами, несмотря на то, что технически мы *можем* передать поле и ресурс вместе в одном представлении JSON, это нарушит несколько фундаментальных положений метода Update и в дальнейшем приведет к всевозможной рассогласованности.

Листинг 8.2. Путаница с частичным обновлением, вызванная нарушением правил HTTP-метода PATCH

```
PATCH /chatRooms/1 HTTP/1.1
Content-Type: application/json
```

```
{
  "chatRoom": {
    "id": 1,
    "title": "Cool Chat"
  },
  "fieldMask": ["title"]
}
```

Ресурс ChatRoom в этом случае не является телом запроса, нарушая требования стандартного метода Update

Сама FieldMask находится рядом с ресурсом ChatRoom

Что же делать? Помимо тела HTTP-запроса, наши маски полей могут располагаться в двух потенциальных местах: заголовках и строках запроса. И хотя технически сработают и те и другие, параметры запросов оказываются на порядок доступнее, в частности, с учетом того, что их можно изменять даже в запросе браузера при том, что заголовки погружены глубоко в структуру HTTP и являются труднодоступными. Так что параметры запроса, пожалуй, подойдут лучше всего.

К сожалению, для повторяющихся параметров запросов нет никакой спецификации, то есть интерпретация этих параметров будет зависеть от используемого HTTP-сервера. Возьмем в качестве примера табл. 8.2. В ней видно, что разные серверы обрабатывают эти входящие данные по-разному, поэтому важно опираться на библиотеку, которая будет согласовываться с наиболее распространенным стандартом многократного использования одного и того же поля, например `?fieldMask=title&fieldMask=description`. И хотя одна из альтернатив — `fieldMask=a,b` — выглядит более лаконичной, это может создать проблемы при просмотре карт или вложенных интерфейсов (см. подраздел 8.3.5).

Это означает, что нам нужно дополнить сообщение запроса как для метода Update, так и для метода Get. Как видите, для этого достаточно добавить в сообщения запроса новый атрибут `fieldMask` (листинг 8.3).

Таблица 8.2. Примеры обработки многозначных параметров запросов разными системами

Система	Ожидаемый формат	Примечания
PHP	?fieldMask[]=a&fieldMask[]=b	
Flask (Python)	?fieldMask=a&fieldMask=b	request.args.to_dict(flat=False)
Spring (Java)	?fieldMask=a&fieldMask=b	
	?fieldMask=a,b	
Express (Node.js)	?fieldMask=a&fieldMask=b	

Листинг 8.3. Пример стандартных запросов Get и Update после добавления масок полей

```
type FieldMask = string[];
interface GetChatRoomRequest {
  id: string;
  fieldMask: FieldMask;
}
interface UpdateChatRoomRequest {
  resource: ChatRoom;
  fieldMask: FieldMask;
}
```

← Тип FieldMask является не чем иным, как массивом путей

← Можно просто добавить свойство fieldMask в интерфейсы запроса update и get

Теперь, когда мы умеем передавать эти значения масок полей в API (соблюдая стандарты HTTP и ресурсно-ориентированных API), можно более пристально рассмотреть значения, которые содержит каждая запись маски. Начнем с карт и вложенных интерфейсов.

8.3.2. Карты и вложенные интерфейсы

До сих пор мы говорили лишь о плоских структурах данных ресурсов — то есть таких, в которых отсутствуют какие-либо вложенные типы значений. И как бы нам ни нравился такой простой мир, он не всегда отражает реальность, где в ресурсах у нас бывают значения, которые сами являются другими типами интерфейсов. Кроме того, возможен сценарий, когда ресурс содержит поле, имеющее тип карты, являющейся просто коллекцией пар «ключ — значение». Это ведет нас к интересному вопросу: действует ли эта идея частичного обновления и извлечения во вложенных структурах? Или работает ли она только на верхнем уровне, в действительности рассматривая ресурсы как полностью плоские структуры?

Хорошая новость в том, что определенно есть путь, позволяющий выделять вложенные поля с помощью масок (как во вложенных статических интерфейсах, так и в динамических значениях в стиле карт). Но есть и плохая новость: для этого в полях масок потребуется соблюдать свою специфику и использовать особые символы перехода. Чтобы понять, как это работает, мы начнем с простого списка правил, которые можно собрать в эффективный инструментарий, предназначенный для решения большинства сценариев. Сопутствующие примеры будут приведены чуть ниже.

1. В отдельных частях спецификации поля необходимо использовать точку (.) в качестве разделителя.
2. Ко всем полям вложенного сообщения можно обращаться с помощью звездочки (*).
3. Ключи карт всегда должны быть строками.
4. Все части спецификации поля (имена полей или ключи карт), которые нельзя представить как строковый литерал в кавычках, необходимо обособить обратными кавычками (` `).
5. Вместо символьного литерала обратной галочки можно использовать двойные обратные кавычки (` `).

Если эти правила вас насторожили, то не переживайте, примеры все прояснят. Практический разбор этой уникальной области задачи мы начнем с листинга 8.4. В данном случае нужно представить, что наш ресурс `ChatRoom` содержит и поле `LoggingConfig`, и поле карты `settings`, которое может хранить произвольные данные в формате «ключ — значение». Посмотрим, как можно реализовать адресацию каждого из этих полей.

Листинг 8.4. Добавление вложенного поля интерфейса и динамического поля карты

```
interface ChatRoom {
  id: string;
  title: string;
  description: string;
  loggingConfig: LoggingConfig;
  settings: Object;
}

interface LoggingConfig {
  maxSizeMb: number;
  maxMessageCount: number;
}
```

Здесь у нас вложенное поле, хотя оно имеет статическую структуру с правильно определенными полями

Поле `settings` является произвольной картой «ключ — значение», содержащей значения разных типов

Чтобы помочь вам представить, как можно применить каждое из приведенных правил, в табл. 8.3 я привожу представление данных в ресурсе `ChatRoom` с примером того, как адресовать одно поле в качестве записи в маске.

Таблица 8.3. Представление ресурса с соответствующими записями маски полей для каждого поля

Ресурс ChatRoom	Запись маски поля
<pre>{ "id": 1, "title": "Chat Room", "description": "Cool chat!" "loggingConfig": { "maxSizeMb": 10, "maxMessageCount": 1000 }, "settings": { "1234": "abcd", "`hello`": null, "*": "star", "test.value": true "test": { "value": false } } }</pre>	<pre>"*" "id" "title" "description" "loggingConfig.*" "loggingConfig.maxSizeMb" "loggingConfig.maxMessageCount" "settings.*" "settings.`1234`" "settings.``hello``" "settings.``*`" "settings.`test.value`" "settings.test.*" "settings.test.value"</pre>

Как видите, есть способ, позволяющий адресовать каждое поле ресурса, используя перечисленные правила. Несмотря на то что большинство примеров достаточно просты (скажем, "description" и "settings.test.value"), остальные довольно невзрачны и на первый взгляд могут запутывать. Например, в случаях, когда ключи карты можно интерпретировать как численные значения, их нужно заключать в обратные кавычки. То же касается ситуаций, в которых точка может ошибочно приниматься за разделитель (как в случае с "settings.test.value" и "settings.`test.value`", которые действительно легко спутать). Наконец, если в качестве символьного литерала необходимо прописывать обратные кавычки, то их можно удвоить (просто используйте `` там, где должна стоять `).

Следуя этим пяти правилам определения формата маски, мы должны иметь возможность адресовать любое вложенное поле в структуре данных, даже если оно содержит странные символьные литералы (наподобие точек, звездочек или обратных кавычек).

Тем не менее здесь все же остается одна важная категория типов полей ресурсов: повторяющиеся поля, или массивы. В следующем подразделе мы рассмотрим, как можно обращаться к ним в ресурсах с помощью масок полей.

8.3.3. Повторяющиеся поля

У нас есть понятный и простой способ адресации полей во вложенных интерфейсах и полях карт, но что делать с повторяющимися полями, такими как списки строк? Что, если такое поле само является вложенным интерфейсом?

Как нам обратиться к фамилиям всех авторов заданной книги? К счастью, для этого есть несложный способ. Но прежде чем мы к нему перейдем, необходимо разобрать одно важное ограничение — адресацию элементов повторяющихся полей по их индексам.

Мы все умеем обращаться к штучным элементам массива в знакомых нам языках программирования. Практически всегда обращение для получения первого элемента в массиве с названием `item` будет выглядеть как `item[0]`. И хотя это определенно важнейшая часть функциональности любого языка программирования, имеет ли такая возможность смысл в веб-API?

Если мы окажемся единственным пользователем этого API или ресурс, к которому мы обращаемся, полностью неизменяем, а индекс является уникальным идентификатором или чем-то подобным, то возможно. Но почти всегда эта функциональность в веб-API во многих смыслах вносит путаницу.

Для начала, доступ к элементу по его индексу подразумевает, что этот индекс является уникальным идентификатором, когда в большинстве случаев это не так. Индекс является ID *сейчас*, но это не стабильный идентификатор в том смысле, что со временем может легко изменяться, как вследствие добавления элемента перед нужным нам элементом, так и ввиду замены значения всего массива. Далее, использование индекса в качестве ID явно подразумевает стабильную последовательность этого конкретного списка. То есть порядок элементов в списке с течением времени должен будет оставаться согласованным, даже по мере прибавления в его конец новых элементов. И хотя все это может выглядеть незначительным следствием, поддержание порядка создаст немало трудностей, оказавшись в итоге не слишком полезным.

Ввиду всех этих сложностей в веб-API реально нет смысла в индивидуальной адресации элементов списка по их индексу. Если подобная функциональность действительно окажется необходима, то будет намного эффективнее использовать поля карты (в которых есть стабильные, истинные, локальные уникальные идентификаторы для всех их элементов) или коллекцию подресурсов, каждый из которых имеет собственный глобальный уникальный идентификатор. И в результате важно, чтобы API *не* поддерживали возможность извлекать или обновлять отдельный элемент в повторяющемся поле на основе его индекса. Иначе говоря, потребитель *не должен* иметь возможности дать API команду обновить или извлечь «элемент по индексу 0» в повторяющемся поле ресурса. Эта идея первого элемента абсолютно бессмысленна, если API не будет гарантировать множество дополнительной функциональности, поддержка которой в итоге станет очень трудоемкой.

К счастью, это не означает полное отсутствие возможности взаимодействовать с элементами в повторяющихся полях. Наоборот, есть очень полезное действие, которое мы можем к ним применить, и для его реализации нам потребуется способ, позволяющий выразить это намерение.

Представим, что наш ресурс `ChatRoom` включает поле администратора с типом `User`. Мы уже знаем, как получить только имя администратора (например, с помощью `fieldMask=admin.name`), но что, если существует несколько администраторов (листинг 8.5)? Как можно получить только имя каждого из них?

Листинг 8.5. Представление `ChatRoom` с множеством администраторов

```
interface ChatRoom {
    id: string;
    title: string;
    description: string;
    administrators: User[];
}

interface User {
    name: string;
    email: string;
    // ...
}
```

В этом случае `administrators` — это повторяющееся поле интерфейса `User`

В данном случае можно снова использовать звездочку для выражения своего рода цикла *for-each*. Иными словами, можно рассмотреть префикс в `"administrators.*"` как способ сказать: «Для каждого администратора предоставить только перечисленные поля». Тогда для извлечения только имени мы задействуем значение маски поля `"administrators*.name"`. Однако нужно помнить, что эта дополнительная функция не мешает нам запросить всех администраторов, используя (простую) маску этого поля.

К сожалению, хоть эта возможность и позволяет нам выбирать отдельные поля среди элементов повторяющегося поля, обновлять мы их все равно не можем (листинг 8.6). И объясняется эта невозможность теми же причинами, по которым нельзя адресовать `"administrators[0]"` — стабильность повторяющегося поля не гарантируется, в связи с чем нельзя знать, какой элемент в списке и каким значением нужно обновить.

Листинг 8.6. (Плохой) пример попытки обновить элементы в повторяющемся поле по их индексу

```
PATCH /chatRooms/1?fieldMask=administrators*.name HTTP/1.1
Content-Type: application/json
```

```
{
  "administrators": [
    { "name": "New name for Index 0" },
    { "name": "New name for Index 1" }
  ]
}
```

Без строгой согласованности ничто не гарантирует, что этот код заменит нужного администратора

Как видите, если порядок ничем не гарантируется или добавление нового администратора привело к смещению индексов, то уже нельзя быть уверенным

в получении желаемого результата. Вместо обновления отдельных элементов в повторяющемся поле нужно выполнять полную замену во всех повторяющихся полях.

8.3.4. Значения по умолчанию

Как мы узнали в главе 5, цель предустановленного значения в том, чтобы «делать нужное дело» для пользователя. Суть в том, что оставить что-то пустым — это способ для пользователя API сказать, что у него нет конкретной идеи об этом аспекте API и в этом случае он доверяет выбор наиболее подходящего поведения самому API. В сценарии с масками полей предустановленные значения для стандартных методов (`Get` или `Update`) различаются незначительно.

В методе `Get` значение по умолчанию почти всегда представляет полный список доступных в ресурсе полей. То есть каждое поле, присутствующее в интерфейсе для ресурса, должно быть возвращено, если только не установлена маска. Это гарантирует, что метод `Get`, который не поддерживает частичное извлечение, будет вести себя так же, как если бы частичное извлечение не поддерживалось совсем (более подробно мы разберем это в подразделе 8.4.1).

Но в этом руководстве имеется исключение. В случаях, когда у ресурса есть поля, которые по какой-либо причине могут в корне испортить потребителям опыт работы с API, такие поля необходимо удалять из предустановленных конфигураций масок, оставляя неустановленными. Например, ресурс содержит поля, которые невероятно велики или потребуют настолько долгих вычислений, что метод `Get` сможет вернуть результат лишь через несколько минут. В подобном случае может оказаться более разумным исключить такое проблемное поле из возвращаемого набора предустановленных. Это будет означать, что пользователям, заинтересованным в данных полях, потребуется явно выразить свой интерес, установив маску.

Такой сценарий довольно редок, но важно то, что если поле попадет под это исключение (то есть не будет включено по умолчанию и потребует явного запроса на добавление), то этот факт должен быть отмечен в его документации. Если его в ней не отразить, это может привести к весьма запутанному поведению, разобраться в котором потребители API смогут только путем проб и ошибок, что явно окажется неприятным.

Это подводит нас к последнему важному сценарию: что, если нам нужно извлечь все поля, но мы не хотим перечислять их все в маске? Имеется в виду, что мы не хотим следовать предустановленному поведению, позволяя API решать, какие поля мы должны предположить. Вместо этого мы хотим конкретно запросить каждое доступное поле, включая те, которые могут оказаться очень большими или трудновычислимыми. Сюда также могут относиться те, которые были

добавлены с момента последнего обновления клиентского кода (см. главу 24). Как это сделать?

В действительности ответ здесь довольно прост: нужно использовать специальное ограничительное значение, указывающее «все»; в нашем случае это будет звездочка ("*"). Если это значение присутствует в маске поля, то вернуть нужно все поля, независимо от того, что среди них могут присутствовать и какие-то другие.

Теперь, разобравшись с предустановленными значениями для частичного *извлечения* данных, нужно разобраться, какой вариант по умолчанию должен быть установлен для частичного *обновления* данных с помощью стандартного метода `Update`. К сожалению, когда дело доходит до обновления ресурсов, предустановка всех полей, рекомендуемая для извлечения, не имеет смысла. В конце концов, если мы будем по умолчанию указывать все поля, то у нас получится уже метод `Replace`, а не `Update`. Как же нужно поступить?

Удобным вариантом будет попытка вывести маску поля на основе предоставленных данных. Иначе говоря, можно пройтись по входящим данным и обновить поля, если в них указано значение. В следующем подразделе мы разберем эту идею более подробно.

8.3.5. Неявные маски полей

Если для извлечения данных можно использовать предустановленное значение, то их обновление требует больше внимания и усилий. Причина в том, что если следовать тому же правилу (установить в качестве предустановленного варианта все поля), то мы получим стандартный метод `Update`, который будет действовать подобно методу `Replace` (поскольку будет заменять все поля). Это наводит на важный вопрос: что означает неустановленная маска полей в методе `Update` (при использовании HTTP-метода `PATCH`)?

Чаще всего `PATCH` демонстрирует намерение обновить ресурс с помощью только тех данных, которые переданы в теле запроса. Это подсказывает нам интересную идею о том, как получить предустановленное значение маски в методе `Update`, а именно за счет ее вывода из предоставленных данных. И хотя эта стратегия выглядит простой и очевидной, сам процесс вывода имеет широкие возможности. В действительности за счет своего потенциала он может стать основным способом, с помощью которого пользователи API будут работать с масками полей. Как же его реализовать?

Этот вывод работает путем перебора всех переданных полей и отслеживания пути к каждому из них. Если в поле установлено значение (в JSON или TypeScript это значит, что значение не `undefined`), то мы включаем путь этого поля в выводимую маску (листинг 8.7).

Листинг 8.7. Функция вывода маски полей из объекта ресурса

```

type FieldMask = string[];
function inferFieldMask(resource: Object): FieldMask {
  const fieldMask: FieldMask = [];

  for (const [key, value] of Object.entries(resource)) {
    if (value instanceof Object) {
      for (const field of inferFieldMask(value)) {
        fieldMask.push(`${key}.${field}`);
      }
    } else if (value !== undefined) {
      fieldMask.push(key);
    }
  }

  return fieldMask;
}

```

Начинаем с определения типа FieldMask в виде массива строковых имен полей

Затем перебираем все пары «ключ — значение» переданного объекта ресурса

Используем рекурсию для поиска во вложенном объекте полей и просто добавляем перед исходным полем символ «.» в качестве разделителя

Если хоть какое-то значение установлено (включая null), то мы рассматриваем его как неявную часть маски

Используя код листинга 8.7, можно сделать вывод о намерениях пользователя, просто анализируя переданные данные. Иными словами, если пользователь хочет обновить только одно поле, то может указать на это, просто предоставив интересное поле в теле запроса (например, `PATCH /chatRooms/1 { "description": "New description" }`), на основе чего можно сделать вывод, что он хотел использовать это поле (`["description"]`) в качестве маски.

Здесь важно отметить, что значение `null` не равнозначно исключению поля из тела запроса. То есть если значение `undefined` в JavaScript или отсутствующее поле в JSON не попадут в выводимую маску, то значение `null` окажется в ней. В итоге установка поля на `null` (например, `PATCH /chatRooms/1 { "description": null }`) приведет к получению той же маски, что и предыдущий пример (`["description"]`), и аналогичным образом обусловит установку в поле `description` указанного значения, которым в данном случае окажется явный `null`.

Обычно такая модель поведения полностью приемлема, но может вызывать проблемы, когда мы начинаем работать с динамическими структурами данных, где доступные в ресурсе поля могут со временем изменяться. Далее мы познакомимся с оптимальным способом обработки динамических структур данных и привносимого ими уникального поведения.

8.3.6. Обновление динамических структур данных

Как мы узнали в главе 5, работать со статическими структурами данных практически всегда проще. И основная причина в том, что в них для значений полей есть всего две категории: фактическое значение (например, `ChatRoom({ title: "Cool chat!" })`) и значение `null` (например, `ChatRoom({ title: null })`). С другой стороны, динамические структуры данных, такие как карты, имеют третью

категорию, обусловленную тем, что сами поля могут как присутствовать, так и нет: `undefined`. Например, как мы видели в подразделе 8.3.2, поле настроек являлось картой, попадающей как раз в эту категорию. Ключ (например, `"test"`) мог либо быть значением или явным `null`, либо отсутствовать в карте полностью (что в JavaScript рассматривается равнозначным `undefined`).

И это создает каверзную ситуацию. Представьте, что поле `settings` установлено на `{ "test": "value" }`. Чтобы изменить это значение, можно использовать неявную маску полей: `PATCH /chatRooms/1 { "settings": { "test": "new value" } }`. Если мы захотим установить значение на неявный `null`, это тоже сработает: `PATCH /chat-Rooms/1 { "settings": { "test": null } }`. Но как нам реализовать полное отсутствие ключа? Иначе говоря, как можно установить значение на `undefined`?

К сожалению, `undefined` не является частью спецификации JSON, поэтому его явное использование, как это возможно в JS-коде, просто не сработает. Кроме того, в большинстве языков программирования нет контрольного значения для отсутствующего в карте ключа. Поэтому для удаления ключей в динамических структурах данных нам потребуется иной механизм.

И хоть указать `undefined` в API можно с помощью специального символа, такой подход чреват ошибками и может вызвать дополнительные сложности в дальнейшем. Он также будет либо конфликтовать с диапазоном JSON-значений, либо нарушать правила спецификации JSON. Если коротко, то использование этого особого флага для выражения `undefined` окажется не самой надежной стратегией.

Еще один вариант — опереться на полную замену поля. То есть можно извлечь ресурс, удалить ненужный ключ и обновить этот ресурс, заменив все поле карты. И хотя это определенно сработает (и можно будет избежать проблем с согласованностью данных с помощью проверок новизны), подойдет такой вариант только при обновлении одного поля. Если же ресурс весь динамичен (как в случае многих веб-API для систем хранения данных), то это будет намного сложнее, поскольку мы, по сути, задействуем здесь стандартный метод `Replace`.

Вместо этого более удачным решением будет использовать явную маску полей с отсутствующим в теле запроса значением. Иначе говоря, мы явно утверждаем, что хотим обновить конкретное поле, но при этом полностью его опускаем. К примеру, запрос в листинге 8.8 обеспечит удаление ключа `"test"` из поля `settings` ресурса `ChatRoom`. Заметьте, что здесь мы не устанавливаем значение на `null`, а полностью его удаляем.

Листинг 8.8. Пример метода для удаления поля из динамической структуры данных

```
PATCH /chatRooms/1?fieldMask=settings.test HTTP/1.1
Content-Type: application/json
```

```
{}
```

← Поскольку важно лишь, чтобы `settings.test` приравнивалось к `undefined`, никаких данных предоставлять не нужно

В случае этого запроса, когда придет время определять, что нужно обновить, мы присвоим значению ресурса входное значение (`resource["settings"]["test"] = input ["settings"]["test"]`), которое в данном случае будет равнозначно `undefined`, и получим желаемый результат удаления ключа из словаря. В других языках будет важно определить, содержал ли ввод указанное поле, и если нет, то удалить ключ явно. К примеру, нельзя будет использовать в Python `input.get('settings', {}).get('test')`, поскольку тогда мы получим значение `None`. А установка в Python значения `None` не равнозначна удалению ключа из словаря Python.

И это подводит нас к заключительной теме: что происходит при предоставлении несуществующего поля?

8.3.7. Недействительные поля

Как мы узнали в подразделе 8.3.6, определение полей в маске, отсутствующей в теле запроса, является тонким трюком, позволяющим удалять данные из динамической структуры данных. Вот только как быть со случаями, когда внутренняя структура данных не является динамической?

И хотя здесь может возникнуть соблазн применить защитное программирование и выбрасывать определенную ошибку при передаче поля, которое не может существовать, такой подход вряд ли хорошо работает. И, как мы увидим в главе 24, основная причина в том, что добавление новых полей и их периодическое удаление довольно распространены. В результате нельзя считать невозможным добавление запрашивающей стороной поля, которое существовало в одной версии, но уже не существует в текущей. В этих случаях будет безопаснее просто рассматривать все структуры как динамические и никогда не выбрасывать ошибку, если указанное в маске поле не обнаружено в запросе при обновлении или в ресурсе при извлечении.

Иначе говоря, когда у нас есть поле, которое не может быть найдено, мы рассматриваем его значение как `undefined` и либо обновляем данные, либо возвращаем результат.

8.3.8. Итоговое определение API

Как мы узнали в подразделе 8.3.1, для определения новых полей в стандартных запросах `get` и `request` мы будем использовать новый тип `FieldMask` (листинг 8.9).

Листинг 8.9. Заключительное определение API

```
abstract class ChatRoomApi {
  @get("/{id=chatRooms/*}")
  GetChatRoom(req: GetChatRoomRequest): ChatRoom;
```

```

    @patch("/{resource.id=chatRooms/*}")
    UpdateChatRoom(req: UpdateChatRoomRequest): ChatRoom;
}

type FieldMask = string[];

interface GetChatRoomRequest {
    id: string;
    fieldMask: FieldMask;
}

interface UpdateChatRoomRequest {
    resource: ChatRoom;
    fieldMask: FieldMask;
}

```

8.4. КОМПРОМИССЫ

Частичное обновление и извлечение с помощью масок полей предоставляет огромные возможности, но при этом важно помнить, что цель все же остается довольно уникальной: минимизировать передачу необязательных данных и позволить точно изменять ресурсы API. Зачастую возникает соблазн рассматривать маски полей и конкретно частичное извлечение как SQL-подобные инструменты запроса для получения конкретных данных от ресурса, на котором их содержится больше, чем нужно. И хотя маски полей можно использовать для реализации этой цели, делать так определенно не стоит.

В прочих случаях, когда в API есть множество связанных данных и нужен механизм, с помощью которого пользователи могли бы извлекать конкретные данные на основе связей между разными ресурсами, возможностей масок полей уже оказывается недостаточно. Вместо этого гораздо лучше подойдет нечто наподобие GraphQL, предоставляющего мощную систему, которая позволяет объединить связанные данные в API, а также возможность ограничивать получающиеся данные с помощью форматов запросов, которые напоминают маски полей. В этой книге мы не будем знакомиться с GraphQL, но если вам потребуется подобная функциональность, то стоит обратить внимание на этот инструмент.

Теперь, разобравшись с частичным извлечением и обновлением полей с помощью масок, мы затронем еще кое-какие важные вопросы.

8.4.1. Универсальная поддержка

Для метода `Update` поддержка частичного обновления обязательна (иначе он будет работать просто как метод `Replace`), но это не значит, что она должна поддерживаться во всех API. Дело в том, что не в каждом API есть ресурсы, настолько большие или сложные, чтобы заслужить поддержку частичного извлечения.

С другой стороны, даже небольшим ресурсам может быть полезен точечный контроль обновляемых в них полей, поскольку проблема здесь в первую очередь в параллельности, а не в размере ресурса и сложности.

Хотя важно отметить, что если API решит поддерживать частичное извлечение, то это должно делаться для всей системы, а не индивидуально для отдельных ресурсов. Иными словами, если в API будет всего один ресурс, требующий поддержки частичного обновления, такой API должен реализовать эту функциональность для всех ресурсов. И причина тому — согласованность. Задача стандартного метода `Get` — быть согласованным среди всех ресурсов, значит, внесение любой вариативности, зависящей от того, с каким ресурсом происходит взаимодействие, будет вести к неожиданностям, в конечном итоге снижая качество API с позиции потребителей.

8.4.2. Альтернативные реализации

Важно также отметить, что есть и другие варианты реализации поддержки частичного обновления ресурсов с помощью HTTP-метода `PATCH`. Например, JSON Patch (RFC-6902; <https://tools.ietf.org/html/rfc6902>) представляет фантастический вариант на случай, когда требуется более явный и детальный контроль над изменяемым ресурсом. JSON Patch опирается на серии операций (подобно операционным преобразованиям), которые применяются последовательно для изменения документа JSON. Вместо того чтобы решать, какое значение установлено, JSON Patch предоставляет продвинутую функциональность, такую как возможность копировать значения между полями (без его первичного извлечения) или управлять значениями массива, используя индексы и операцию `text` для утверждения того, что элемент по этому индексу соответствует задуманному.

При этом JSON Patch — не единственный вариант. Есть еще JSON Merge Patch (RFC-7396; <https://tools.ietf.org/html/rfc7396>), который немного ближе к изученной нами реализации маски полей, но имеет собственные особенности и сам основывается на выводимой, или неявной, маске полей. Как вы можете представить, это порождает проблему с динамическими структурами данных, когда дело доходит до различия между явной установкой значений на `null` и их полным удалением из структуры. Более того, этот способ рекурсивен для вложенных объектов JSON (например, карт и вложенных интерфейсов), но не для повторяющихся полей (например, массивов), что ведет к возможной путанице в случае работы с этими расширенными структурами данных.

Многие перечисленные выше варианты являются отличными решениями. Однако идея с помощью масок указывать конкретные поля, которые нужно извлечь или обновить, довольно проста для большинства и в то же время достаточно эффективна, чтобы предоставить необходимую функциональность, позволяющую поддерживать высокую точность при извлечении и обновлении ресурсов.

8.5. УПРАЖНЕНИЯ

1. Представьте, что ресурс `ChatRoom` может содержать коллекцию администраторов, а не одного. Что мы теряем в случае использования для хранения этих ресурсов массива? Какие альтернативы у нас есть?
2. Как реализовать удаление одного атрибута из динамической структуры данных?
3. Как при частичном извлечении сообщить, какие поля нас интересуют?
4. Как указать, что нужно извлечь (обновить) все поля, не перечисляя их все в маске поля?
5. Какую маску полей нужно использовать для обновления ключа `"`hello.world`"` в карте `settings`?

РЕЗЮМЕ

- Частичное извлечение особенно важно в тех случаях, когда ресурсы велики или вычислительные возможности потребляющих API клиентов ограничены.
- Частичное обновление необходимо, поскольку тогда у нас будет возможность вносить точечные изменения, не вызывая конфликтов.
- Указывать поля, которые нужно извлечь или обновить, следует с помощью масок, поддерживающих возможности адресации полей, вложенных полей в интерфейсах и ключей карт.
- Маски полей не должны предоставлять механизм для адресации элементов в полях массива по их позиции или индексу.
- В случае частичного извлечения маски полей в качестве предустановленного значения должны предполагать *все*. При этом для частичного обновления должны использоваться неявные маски (когда указываемые поля выводятся на основе их присутствия).
- Если поля недействительны, то их нужно рассматривать, как будто они существуют, но содержат значение `undefined`.

9 Пользовательские методы

В этой главе

- ✓ Почему стандартные методы не могут охватить все сценарии.
- ✓ Как применять пользовательские методы для случаев, когда необходимы побочные эффекты.
- ✓ Как применять пользовательские методы без сохранения состояния для методов API, ориентированных на вычисления.
- ✓ Как определить цель для пользовательского метода (ресурс или коллекция).

Нередко в отношении API нам потребуется выполнять действия, которые не впишутся должным образом в один из стандартных методов. И хотя подобные случаи технически можно обработать с помощью метода ресурса `update`, требования к проведению многих аналогичных операций окажутся неуместными для стандартного метода, в результате чего мы получим непредсказуемый, запутанный и излишне сложный интерфейс. Чтобы можно было решить эту проблему, текущий паттерн описывает возможность безопасной поддержки подобных действий в ресурсах веб-API с сохранением простоты, предсказуемости и функциональности, используя для этого так называемые *пользовательские методы*.

9.1. ЗАЧЕМ ЭТО НУЖНО

При использовании большинства API рано или поздно возникает момент, когда требуется возможность выразить конкретное действие, которое не вписывается должным образом в один из стандартных методов. Например, каким будет правильный API для отправки электронного письма или синхронного перевода текста? Что, если установлен запрет на сохранение каких-либо данных (возможно, вы переводите текст для ЦРУ)? Несмотря на то что подобные действия можно объединить в стандартный метод (скорее всего, в действие `create` или `update`), таким образом вы в некотором смысле изгибаете структуру этого метода, чтобы реализовать то, для чего он не предназначен. И здесь возникает очевидный вопрос: надо ли стремиться насильно вставлять нужное поведение в существующие методы, нарушая для этого их структуру? Или стоит, наоборот, перестроить поведение в такую форму, которая лучше впишется в родную структуру метода? А может, нужно изменить саму структуру, чтобы она соответствовала нашему новому сценарию?

Ответ будет зависеть от ситуации, но в текущей главе в качестве одного из возможных решений подобной проблемы мы рассмотрим применение пользовательских методов. По сути, изменение структуры в целях соответствия новому сценарию мы используем в качестве третьего возможного решения, тем самым обеспечивая отсутствие трансформаций каких-либо методов под конкретные нужды.

Несложно догадаться, что за применением пользовательских методов стоит простой принцип — в конце концов, в большинстве RPC-ориентированных API эта идея используется постоянно. Подвох здесь заключается в деталях. Когда безопасно применять пользовательские методы? А стандартный метод точно не подходит? Можно ли параметризовать вызов пользовательского метода? Список можно продолжать.

В текущей главе мы изучим пользовательские методы и все связанные с ними детали. Но прежде чем перейти к рассмотрению принципа их работы, следует ответить на самый главный вопрос: действительно ли нам нужно применять пользовательские методы?

9.1.1. Почему бы просто не задействовать стандартные методы

Методов, изученных нами в главе 7, зачастую хватает для выполнения практически любых задач в API, однако возникают случаи, когда эти методы ощущаются как неподходящие. В такие моменты зачастую есть *чувство*, будто мы слепо следуем букве закона, но не его духу: присутствие стандартных методов, действующих в достаточной степени отличным от ожиданий образом, ведет к неожиданностям, а значит, и не самому удачному API. Если кратко, то наличие

возможности выполнить действие с помощью стандартного метода еще не означает, что мы *должны* так поступить. Но на что здесь ориентироваться? Можно ли более описательно выразить то самое ощущение, что метод «неподходящий»? Взглянем на конкретный пример: изменение состояния.

Зачастую ресурсы API могут существовать в одном из нескольких состояний. Например, электронное письмо может начинаться с состояния «Черновик», после чего переходить в состояние «Отправлено» и, возможно (при использовании модного сервиса), «Неотправленное». Очевидно, это один из тех сценариев, когда мы *можем* использовать стандартный метод `Update` (листинг 9.1), но все же кажется, что это неправильно.

Почему именно изменение состояния не вписывается в метод `Update`? Первая причина проста: большинство изменений являются теми или иными переходами, и в результате переход в новое состояние (которое хранится в поле состояния) принципиально отличается от установки значения скалярного поля (например, установка темы электронного письма). В результате установка состояния, вызывающего на этот переход, может вводить в заблуждение, что явно не станет признаком хорошего API.

Листинг 9.1. Отметка письма как отправленного с помощью метода `Update`

```
const email = GetEmail({id: 'email id here'});
UpdateEmail({id: email.id, state: 'sent'});
```

В начале состояние письма «Черновик»

В этом примере мы отправляем письмо путем обновления его состояния на «Отправлено»

Как видите, отправка почты путем обновления состояния выглядит неуместной. *Чувствуется*, что это поле должно было управляться самим сервисом API, а не обновляться клиентом. То есть рассмотрение поля в качестве механизма для перехода из одного состояния в другое может довольно сильно запутывать. Более того, такой подход раздваивает истинное назначение метода `Update`, поскольку теперь он одновременно позволяет как обновлять сохраненные данные, так и выполнять переход в новое состояние — два принципиально разных действия.

Вдобавок к путанице, вызываемой обновлением поля для перехода в новое состояние, есть вторая потенциальная проблема, связанная с побочными эффектами. Как мы узнали в главе 7, стандартные методы не должны делать что-либо помимо конкретного действия, которое предполагается вследствие их имени. Имеется в виду, что метод `Update` должен обновлять ресурс и ничего более. Но зачастую изменение состояния подразумевает необходимость выполнения ряда других действий. Например, если у нас есть ресурс электронного сообщения, которое мы хотим изменить из «Черновика» в «Отправлено», то нам также нужно отправить это письмо получателям (иначе нет смысла отмечать его как отправленное). И простое обновление поля состояния на «Отправлено» выступит в роли слабозаметного способа намекнуть, что вы собираетесь установить связь с SMTP-сервером для отправки письма определенному получателю.

И с этого момента проблема начинает расти. Когда мы используем стандартный метод `Update` для выполнения действий с подобными побочными эффектами, то должны учесть тот факт, что теперь у нас потенциально две разные системы, с которыми нужно общаться, чтобы завершить операцию обновления (рис. 9.1). Во-первых, для каждого обновления в любом случае необходимо связаться с системой хранения, чтобы обновить базу данных с помощью новой информации. Во-вторых, если речь идет об изменении состояния ресурса письма с «Черновик» на «Отправлено», то нам также нужно связаться с системой отправки электронной почты (которая затем может дополнительно связаться с почтовым сервисом получателя), чтобы выполнить задачу. То, что должно быть простой процедурой «обновления базы данных», теперь стало мешаниной зависимостей и потенциально длительных фоновых операций, в которых на каждом шагу может случиться сбой.

Кроме всего прочего, это также означает, что в зависимости от обновляемого содержимого стандартный метод `Update` может вернуть результат либо сразу, либо с задержкой до момента отправки письма. Вследствие этого возникает различная неразбериха, когда пользователь просто пытается обновить содержимое своего письма.

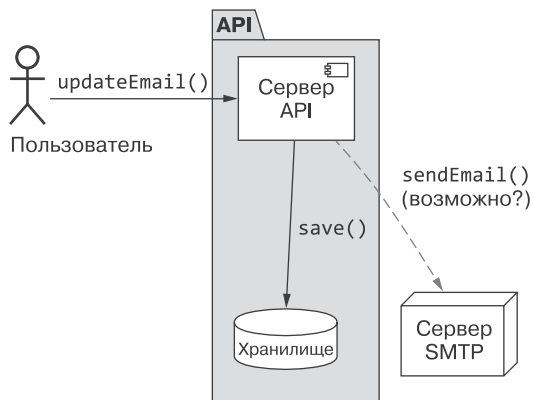


Рис. 9.1. Даже для всего одного обновления API может потребоваться связаться с несколькими системами

При этом важно сказать, что *технически* с приведенной схемой все в порядке. Многие вызовы API могут связываться со множеством разных сервисов, и это не обязательно плохо. Проблема в том, что эта схема нарушает определенные ранее правила того, что стандартные методы должны и не должны делать. В результате проблема оказывается не в API, который общается с несколькими разными системами, а в стандартном методе (например, `Update`), который связывается с разными системами, вызывая побочные эффекты и впоследствии обрастая зависимостями. Это существенно ухудшает API, лишая нас возможности опираться на определенные аспекты стандартных методов, которые вдобавок становятся непредсказуемыми и запутанными.

Прежде чем подвести итог, нужно ответить еще на один важный вопрос: почему бы не переупорядочить ресурсы так, чтобы нужные нам действия больше соответствовали функционалу стандартных методов? В этом примере можно использовать ресурсы `EmailDraft` и `Email`, в которых мы не хотим переходить между состояниями, вместо чего создаем новый ресурс `Email`, основанный на `EmailDraft` (листинг 9.2).

Листинг 9.2. Выстраивание ресурсов под соответствие стандартным методам

```

abstract class EmailApi {
    @post("/{parent=users/*/emailDrafts}")
    CreateEmailDraft(CreateEmailDraftRequest req): EmailDraft;

    @post("/{parent=users/*/emails}")
    CreateEmail(CreateEmailRequest req): Email;
}

interface EmailDraft {
    id: string;
    subject: string;
    // ...
}

interface Email {
    id: string;
    content: EmailDraft;
}

```

Мы всегда начинаем с мутабельных ресурсов EmailDraft

Создание ресурса Email на деле также привело бы к отправке письма. Ресурсы Email будут неизменяемыми

Ресурс EmailDraft содержит ключевую информацию, которая будет отправлена

Фактический Email ссылается на содержимое EmailDraft

И хотя это полностью приемлемо, подобные сдвиги подразумевают, что использование только стандартных методов является священным принципом проектирования API. В действительности стандартные методы служат посредниками на пути к цели. А в данном случае целью выступает операционный, выразительный, простой и прогнозируемый API. И когда дизайн уводит от этого, практически всегда будет лучше настроить инструменты под соответствие потребностям API, а не наоборот.

Что же делать вместо этого? Очевидный ответ — опереться на отдельный вызов API, который не является стандартным методом (а значит, может иметь побочные эффекты), чтобы изолировать функциональность в одном месте, а не перегружать существующие стандартные методы. И несмотря на кажущуюся простоту такого приема, как это часто бывает при проектировании API, дьявол здесь кроется в деталях. Теперь, когда мы узнали, зачем нужны пользовательские методы, кратко разберем, чем они являются, и рассмотрим несколько типичных случаев применения.

9.2. ОБЗОР

Пользовательские методы — это не что иное, как вызовы API, которые выходят за рамки стандартного метода, в связи с чем не подпадают под строгие требования, которые предъявляются к стандартным методам. В следующем разделе мы увидим, что они могут выглядеть несколько иначе, но с чисто технической точки зрения в них нет ничего особенного.

Стандартные методы предлагают нам невероятно полезные компоненты для построения API, но ценой за столь широкий спектр является обширный набор

руководств, правил и ограничений. С другой стороны, пользовательские методы практически ничем не ограничены, что позволяет им делать для сценария все необходимое, не вынуждая его вписываться в структуру и правила стандартных методов.

В этом есть и свои недостатки, в частности тот факт, что пользователи API не могут делать столько же уверенных предположений, сколько сделали бы в случае со списком стандартных методов. Это не значит, что пользовательские методы в API все должны быть противоречивыми или непоследовательными. Наоборот, они должны являться согласованными по всему API. Тем не менее суть в том, что по умолчанию для них обычно нет каких-либо жестких правил. То есть эти правила и соответствующие примеры устанавливает разработчик API, после чего они должны оставаться согласованными по всей его структуре.

Несмотря на то что принцип работы этого паттерна прост (пример пользовательского метода приведен в листинге 9.3), он быстро усложняется, так как нам приходится прорабатывать различные сценарии и стоящие за ними нюансы. К примеру, если изменение состояния требует дополнительного содержимого или параметризации, как можно предоставить эту информацию? Что, если метод оперирует не в одном ресурсе, а в коллекции? А как быть со случаем, когда состояние вообще отсутствует?

Листинг 9.3. Пример пользовательского метода запуска ракет

```
abstract class RocketApi {
    @post("/{id=rockets/*}:launch")
    LaunchRocket(LaunchRocketRequest req): Rocket;
}

interface Rocket {
    id: string;
    // ...
}

interface LaunchRocketRequest {
    id: string;
}
```

Пользовательские методы используют HTTP-глагол POST и специальный сепаратор «:» для объявления самого действия

Именуются пользовательские методы аналогично стандартным (<Verb><Noun>)

Теперь, когда мы видели, как выглядит пользовательский метод, и разобрали обширную картину обоснований, по которым нам нужны такие методы, пора перейти к изучению самого принципа их работы.

9.3. РЕАЛИЗАЦИЯ

Пользовательские методы во многом похожи на стандартные. Что же касается различий, то одно относится к формату HTTP-запроса. В то время как стандартные методы выражают свое поведение с помощью сочетания пути запроса

и HTTP-глагола (например, `PATCH` в ресурсе всегда указывает на метод `Update`), пользовательские не могут применять тот же механизм. Вместо этого у них есть собственный формат, который помещает релевантную информацию в путь запроса. Разберем этот формат по частям на примере с рис. 9.2.

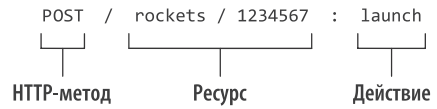


Рис. 9.2. Компоненты HTTP-запроса для пользовательского метода запуска ракет

Начнем с того, что HTTP-глаголом в пользовательских методах практически всегда является `POST`. Иногда может иметь смысл применить в пользовательском методе глагол `GET`, но намного вероятнее, что подресурс-одиночка (см. главу 12) подойдет лучше. Кроме того, может оказаться разумным использовать глагол `DELETE`, но такие случаи относительно редки.

И хотя путь ресурса здесь будет выглядеть так же, как и в стандартных методах, есть одно различие, а именно расположение действия в самом конце. Чтобы не путаться в том, где заканчивается ресурс (или коллекция) и начинается действие, важно не разделять эти два ключевых компонента прямым слешем (например, `POST /rockets/1234567/launch`). Вместо этого указать место окончания ресурса и начала действия можно с помощью двоеточия (:). Это может выглядеть несколько странно, но здесь важно избежать двусмысленности, особенно ввиду того, что при использовании прямого слеша путь получится технически верным.

Наконец, как было показано в листинге 9.3, имя RPC должно соответствовать тому же общему правилу, когда глагол сопровождается существительным, например `LaunchRocket` или `ArchiveDocument`, как в случае любого стандартного метода. Это очень важно учитывать, особенно рекомендации избегать использования предлогов, таких как *with* или *for* (например, не создавать пользовательских методов наподобие `CreateRocketForMars`), поскольку они должны быть сформулированы так же, как стандартные. Иными словами, пользовательские методы не являются механизмом для параметризации стандартных.

9.3.1. Побочные эффекты

Возможно, самое серьезное различие между пользовательскими и стандартными методами — допустимость побочных эффектов. Как мы узнали в главе 7, цель стандартного метода — служить механизмом доступа к данным ресурса и управления ими. И одним из ключевых принципов стандартного метода является то, что он делает ровно то, что должен, не вызывая дополнительно никаких фоновых действий, которые не относятся в своей основе к изначальному назначению метода. То есть если метод говорит, что создает ресурс, то он должен создавать ресурс и *не делать* больше ничего.

Пользовательские же методы не имеют подобного ограничения. Наоборот, они отлично подходят для действий с побочными эффектами. И причина здесь проста: в отношении стандартных методов у нас есть определенные ожидания относительно того, что они могут, а что нет. Пользовательские же по своей природе свободны от этих ограничений. Поэтому побочные эффекты, такие как отправка электронных писем, активация фоновых операций, обновление нескольких ресурсов и любые другие на ваше усмотрение, доступны.

Например, API электронной почты, естественно, потребует механизм для отправки писем. Тем не менее использование `CreateEmail` как для сохранения письма в системе, так и для его отправки по SMTP будет недопустимым вариантом. Вместо этого `CreateEmail` можно использовать для создания письма в состоянии «Черновика», после чего задействовать метод `SendMail` для реализации связи с удаленным сервером по SMTP и (только после успешной отправки) перевода письма в состояние «Отправлено». Использование такого потока, показанного на рис. 9.3, означает, что стандартный метод (в этом случае `CreateMail`) остается чист и прост, выполняя лишь одну задачу: сохранение записи в базе данных. Затем, чтобы совершать более сложные действия (такие как связь с удаленными SMTP-серверами), мы применяем пользовательский метод, который может выполнять любую работу, позволяющую достичь цели.

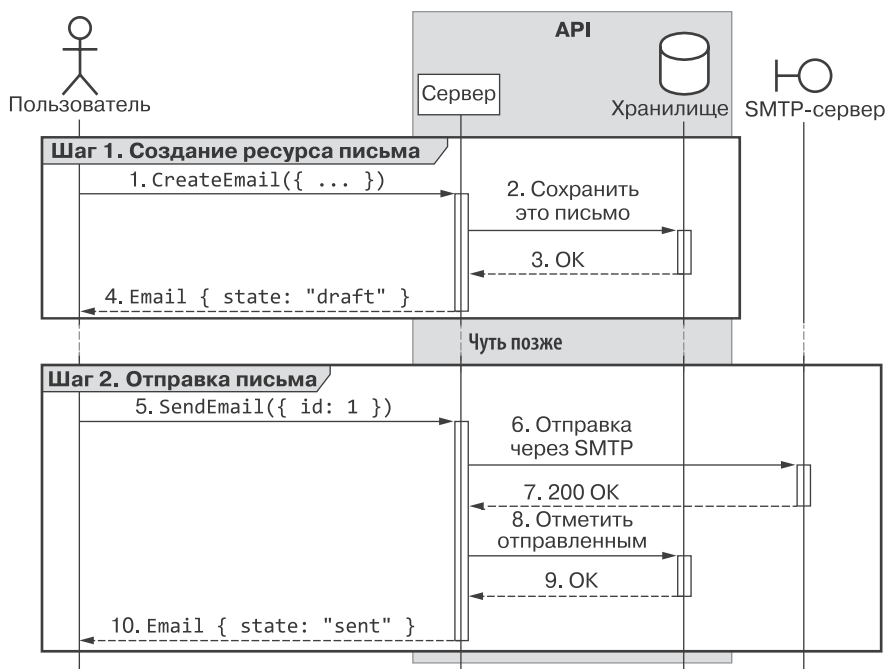


Рис. 9.3. Схема последовательности, показывающая от отправку письма с помощью пользовательских методов

До сих пор мы подразумевали, что пользовательские методы, подобно стандартным, можно применять и к ресурсам, и к коллекциям, но тему нужно изучить более глубоко. Далее мы рассмотрим, когда есть смысл использовать ресурсно-ориентированный пользовательский метод (скажем, пример `SendMail`), а когда он должен быть ориентирован на коллекцию.

9.3.2. Ресурсы или коллекции

В нашем списке стандартных методов одни работают с одним ресурсом (например, обновляя его), а другие — с родительской коллекцией (например, выводя список ресурсов), что обобщенно показано в табл. 9.1. Тем не менее, поскольку пользовательские методы по определению настраиваются под обстоятельства, это может создавать дилемму, требующую выбирать, с чем должен работать такой метод: с одним ресурсом или с родительской коллекцией.

Таблица 9.1. Различные аспекты импорта и экспорта данных

Метод	Цель	HTTP-эквивалент
CreateEmail	Родительская коллекция	POST /users/1/emails
ListEmails	Родительская коллекция	GET /users/1/emails
GetEmail	Ресурс	GET /users/1/emails/2
UpdateEmail	Ресурс	PATCH /users/1/emails/2
DeleteEmail	Ресурс	DELETE /users/1/emails/2

В качестве примера представьте, что нам нужно экспортировать все письма, принадлежащие одному пользователю (подробнее об этом паттерне в главе 23), или выполнить одну стандартную операцию для набора элементов (например, удалить множество писем с помощью одного вызова API). Должны ли эти операции нацеливаться на сам ресурс (например, `POST /users/1:exportEmails`) или же на коллекцию ресурсов (например, `POST /users/1/emails:export`)? Несмотря на то что технически разницы нет, при участии множества ресурсов из одной коллекции практически всегда будет лучше работать именно с коллекцией, оставляя ресурсно-ориентированные пользовательские методы для операций, связанных лишь с одним ресурсом.

А как быть с чем-то наподобие экспорта всей пользовательской информации, включая набор писем? В этом случае внимание пользовательского метода смещается обратно на родительский ресурс прочь от коллекции писем (они по-прежнему задействованы, просто не в центре внимания). В результате этот настраиваемый метод экспорта всей пользовательской информации будет лучше представить как `POST /users/1:export`.

Наконец, что делать в случае, когда вы работаете с набором ресурсов среди нескольких родителей? Например, нам нужно заархивировать набор электронных писем, принадлежащих множеству пользователей (скажем, `users/1/emails/2` и `users/2/emails/4`). В этом сценарии будет применяться тот же формат, но нюанс в том, что идентификатор родителя будет указываться с помощью символа подстановки. Иначе говоря, эта операция в качестве HTTP-запроса будет выглядеть как `POST /users/-/emails:archive`, где дефис играет роль подстановочного знака, вместо которого будет подставляться тело запроса, чтобы указать, какие ID писем нужно заархивировать.

9.3.3. Пользовательские методы без сохранения состояния

До этого момента все примеры пользовательских методов были привязаны к конкретной цели, в качестве которой выступал либо сам ресурс, либо коллекция и родительский ресурс. Однако, поскольку технически пользовательские методы могут делать, что пожелают, перед нами возникает еще одна нерассмотренная возможность: «Что, если у пользовательского метода нет состояния для обработки и его не нужно привязывать к ресурсу или коллекции?»

Такой метод, называемый *методом без сохранения состояния* (stateless-метод), относительно распространен и по мере роста ограниченности хранилища данных может стать критическим элементом функциональности. К примеру, разные правила конфиденциальности, такие как Общий регламент ЕС по защите данных (General Data Protection Regulation, GDPR), могут накладывать некоторые специфичные требования о том, как данные должны сохраняться и где храниться. Подобные требования означают, что stateless-метод, динамически обрабатывающий данные и возвращающий результат — в частности не сохраняя какие-либо передаваемые данные, — является ценным инструментом для добавления в коллекцию. При этом для обработки таких требований идеально подходят пользовательские методы. Пример пользовательского stateless-метода приведен в листинге 9.4.

Листинг 9.4. Пользовательский stateless-метод для перевода текста

```
abstract class TranslationApi {
  @post("/text:translate")
  TranslateText(req: TranslateTextRequest): TranslateTextResponse;
}

interface TranslateTextRequest {
  sourceLanguageCode: string;
  targetLanguageCode: string;
  text: string;
}

interface TranslateTextResponse {
  text: string;
}
```

В этом случае `text` родни подресурсу-одиночке, в котором есть пользовательский stateless-метод `translate`

Запрос на перевод текста никогда не сохраняет данных

Аналогичным образом результат содержит лишь переведенный текст и ничего не сохраняет

Чистые stateless-методы довольно редки. Все же многим API потребуется знать хотя бы платежные реквизиты, чтобы выставить счета за свой вызов. В результате довольно часто присутствует некий родительский ресурс, действующий как разрешение, или платежный контейнер (например, проект, счет плательщика или организация), на который нацеливается пользовательский метод, который в противном случае был бы stateless. Например, перевод текста может выполняться не бесплатно, тогда API потребуется, чтобы пользователь создавал ресурс, отслеживающий платежные данные для всех выполняемых переводов (листинг 9.5).

Листинг 9.5. Пользовательский метод с сохранением состояния, прикрепленный к родителю

В этом случае пользовательский метод прикрепляется к ресурсу проекта

```
abstract class TranslationApi {
  → @post("/{parent=projects/*/text:translate}")
    TranslateText(req: TranslateTextRequest): TranslateTextResponse;
}

interface TranslateTextRequest {
  → parent: string;
    sourceLanguageCode: string;
    targetLanguageCode: string;
    text: string;
}

interface TranslateTextResponse {
  text: string;
}
```

Но тут нужно учитывать, что хоть пользовательские stateless-методы и могут показаться лучшим вариантом в данный момент, важно предвидеть будущее и любую дальнейшую функциональность, введение которой вы предполагаете. Например, сейчас есть один и только один способ перевода текста. Однако в связи с появлением новых технологий машинного обучения (МО) существует вероятность, что в будущем будет больше всевозможных моделей, выполняющих разные виды перевода. Это особенно актуально для таких сфер, как медицина, где перевод текста подразумевает использование гораздо большего объема терминологии и синтаксиса, чем в речевом обиходе. Более того, возможно, что пользователи API захотят развернуть собственные модели машинного обучения или пользовательские глоссарии для перевода. Но ни один из этих сценариев не будет в достаточной степени поддерживаться исключительно stateless-методом.

В этом случае будет эффективнее предоставить пользователям возможность создавать модели МО в качестве ресурсов, после чего прикреплять к такому ресурсу пользовательский метод (листинг 9.6). В результате сам метод по-прежнему будет без сохранения состояния в том смысле, что никакие из передаваемых данных сохраняться не будут.

Листинг 9.6. Пользовательский stateless-метод, прикрепленный к ресурсу TranslationModel

```

abstract class TranslationApi {
    @post("/translationModels")
    CreateTranslationModel(req: CreateTranslationModelRequest):
        TranslationModel;

    // ...

    @post("/{id=translationModels/*/text:translate}")
    TranslateText(req: TranslateTextRequest): TranslateTextResponse;
}

interface TranslateTextRequest {
    id: string;
    sourceLanguageCode: string;
    targetLanguageCode: string;
    text: string;
}

interface TranslateTextResponse {
    text: string;
}

```

Все ресурсы TranslationModel содержат все стандартные методы обычных ресурсов

Сюда пойдут другие стандартные методы для управления ресурсами TranslationModel

Метод TranslateText нацеливается на конкретный ресурс TranslationModel

Ну а теперь, разобравшись с тем, как это работает, пора собрать все воедино в заключительном определении API.

9.3.4. Итоговое определение API

В последнем примере мы задействуем образец API отправки писем (листинг 9.7). В этом случае у нас есть несколько стандартных методов для управления ресурсами Email, а также несколько пользовательских методов для обработки изменений состояния (например, архивирования писем), отправки почты, экспорта всей коллекции писем пользователя, а также stateless-метод, способный определять допустимость почтового адреса.

Листинг 9.7. Итоговое определение API

```

abstract class EmailApi {
    static version = "v1";
    static title = "Email API";

    // ...

    @post("/{id=users/*emails/*}:send")
    SendEmail(req: SendEmailRequest): Email;

    @post("/{id=users/*emails/*}:unsend")
    UnsendEmail(req: UnsendEmailRequest): Email;
}

```

Здесь размещаются все обычные стандартные методы (например, CreateEmail, DeleteEmail и т. д.)

Пользовательские методы для отправки почты будут переводить письмо в состояние отправки, выдерживать паузу несколько секунд, связываться с SMTP-сервисом и возвращать результат

Метод Unsend позволит операции отправки отменяться в процессе задержки, вызванной методом send

```

@post("/{id=users/*/emails/*}:undelete")
UndeleteEmail(req: UndeleteEmailRequest): Email;

@post("/{parent=users/*/emails}:export")
ExportEmails(req: ExportEmailsRequest): ExportEmailsResponse;

@post("/emailAddress:validate")
ValidateEmailAddress(req: ValidateEmailAddressRequest):
    ValidateEmailAddressResponse;
}

interface Email {
    id: string;
    subject: string;
    content: string;
    state: string;
    deleted: boolean;
    // ...
}

```

Метод undelete будет обновлять свойство Email.deleted, инверсируя стандартный метод Delete (подробнее о мягком удалении читайте в главе 25)

Метод export будет получать все данные письма и передавать их в удаленное хранилище (подробнее об импорте и экспорте информации читайте в главе 23)

Очевидно, что этот stateless-метод проверки адреса почты свободен (поскольку не привязан ни к какому родителю)

9.4. КОМПРОМИССЫ

Как правило, пользовательские методы — базовый элемент функциональности, который позволяет дополнить ту, которую стандартные методы и ресурсы предоставляют для любого API. Тем не менее, как указывалось в подразделе 9.1.1, само их существование склонно вызывать противоречия с REST и принципами, определяющими дизайн RESTful API.

Хоть мы и рассмотрели это подробно ранее, есть смысл напомнить, что любые действия, выполняемые с помощью пользовательского метода, должны быть возможны и с помощью стандартных методов, исключительно при наличии должного набора ресурсов. Иными словами, пользовательские методы — это промежуточное звено, которое гарантирует, что простейшая и знакомая иерархия ресурсов также допускает нестандартные взаимодействия.

Помимо этого, важно помнить, что пользовательские методы довольно часто используются неверно или избыточно в качестве костыля для оправдания не лучшего расположения ресурсов. Довольно часто встречаются плохо спроектированные API, в которых пользовательские методы используются для всего, поскольку выбранные ресурсы и их связи просто не подходят под задачу API. API, в которых пользовательские методы задействуются для типичных действий, которые наверняка должны обрабатываться стандартными методами, — это, как правило, плохой знак. И, к сожалению, с помощью пользовательских методов можно скрывать этот неудачный дизайн гораздо дольше, чем хотелось бы многим из нас, и это мешает вносить ценные доработки в API. В результате важно быть полностью уверенным, что любой новый пользовательский метод не действует как костыль, маскирующий неудачное расположение ресурсов.

9.5. УПРАЖНЕНИЯ

1. Что, если создание ресурса требует определенного побочного эффекта? Должен ли в таком случае использоваться пользовательский метод? Почему?
2. Когда пользовательский метод должен нацеливаться на коллекцию, а когда — на родительский ресурс?
3. Почему опасно опираться исключительно на stateless-методы?

РЕЗЮМЕ

- Пользовательские методы практически всегда должны задействовать HTTP-глагол `POST` и никогда `PATCH`. Они также могут использовать глагол `GET`, если являются идиempотентными и безопасными.
- В пользовательских методах ресурс-цель отделяется от выполняемого действия с помощью символа `(:)` (например, `/missiles/1234:launch`).
- Несмотря на то что в стандартных методах побочные эффекты запрещены, в пользовательских они разрешены. При этом применять эти методы нужно нечасто и следует тщательно документировать, чтобы не ввергать пользователей в сомнения.
- Как правило, когда в процессе задействовано несколько ресурсов из одной коллекции, пользовательские методы должны нацеливаться на нее.
- Иногда, особенно в случае вычислительных задач, stateless-методы могут использоваться в API для выполнения большого объема работы. Этой стратегией нужно пользоваться осторожно, поскольку может легко возникнуть потребность в сохранении состояния, которое позднее будет сложно добавить.

10

Длительные операции

В этой главе

- ✓ Как использовать длительные операции для продолжительных вызовов API.
- ✓ Как сохранять метаданные о прогрессе операции.
- ✓ Когда операции должны находиться в иерархии ресурсов.
- ✓ Как определить статус операции (включая полученные ошибки) с помощью опроса и блокировки.
- ✓ Как взаимодействовать с выполняющимися операциями (пауза, возобновление и отмена).

В большинстве случаев входящие запросы можно обработать быстро, сгенерировав ответ в течение нескольких сот миллисекунд после получения запроса. В случаях же, когда ответы занимают намного больше времени, базовый вариант использования той же структуры API и обращение к пользователям с просьбой «подождите» не станет хорошим решением. В текущей главе мы узнаем, как с помощью длительных операций наладить асинхронную работу методов API и гарантировать, что более медленные методы (включая стандартные, которые мы видели в главе 7) смогут предоставлять быструю и согласованную обратную связь, выполняя при этом фактическую работу в фоновом режиме.

10.1. ЗАЧЕМ ЭТО НУЖНО

До сих пор все рассмотренные нами методы API являлись мгновенными и синхронными. Сервис получает запрос, сразу же выполняет определенную работу и отправляет обратно ответ. И такая модель поведения фактически жестко прописывается в проекте API, поскольку возвращаемый тип уже является результатом, а не обещанием вернуть этот результат позже.

Все это работало до сих пор, так как большая часть нужного нам поведения была относительно проста, например поиск строки в базе данных и возвращение результата. Но что происходит, когда работа становится более углубленной, сложной и ресурсоемкой? Это может быть как нечто относительно простое (например, подключение к внешнему сервису), так и подразумевающее серьезную нагрузку (например, обработка многих гигабайтов или терабайтов данных). Будь то небольшой или существенный объем работы, очевидно одно: использование одинакового дизайна API и для последовательно быстрого, и для потенциально очень медленного поведения вряд ли станет удачным решением.

Если в API ничего не менять и метод просто будет выполняться дольше, то мы получим довольно пугающую ситуацию. Как правило, когда мы пишем код, отправляющий запросы к API, мы следуем привычному циклу «написание, компиляция, тестирование». Если же запросы API занимают много времени, то мы можем подумать, что с нашим кодом что-то не так, завершить процесс, добавить инструкции вывода (или запустить отладчик) и запустить код еще раз (листинг 10.1). Если мы увидим, что он определенно зависает на вызове API, то можем повторить цикл, на этот раз подправив параметры, чтобы исключить возможные глупые ошибки. И только когда в очередной раз обнаруживается, что код продолжает зависать на вызове API, мы замечаем то самое предложение в документации, которое гласит, что запрос может занять какое-то время и нужно лишь подождать возвращения результата.

Листинг 10.1. Добавление инструкций вывода для отслеживания прогресса кода

```
function main() {
  let client = ChatRoomApiClient();
  let chatRoomName = generateChatRoomName();
  console.log(`Generated name ${chatRoomName}`);
  let chatRoom = client.CreateChatRoom({
    name: chatRoomName
  });
  console.log(`Created ChatRoom ${chatRoom.id}`);
}

main();
```

Добавление инструкций `console.log()` с целью убедиться, что проблема в длительно выполняющемся методе `CreateChatRoom`

Возможно, создание ресурсов ChatRoom по какой-то причине занимает много времени

На данном этапе можно быть более уверенным в том, где находится проблема (в конце концов, теперь мы знаем, что дело не в коде), но как долго нужно ожидать ответа? Какую продолжительность можно считать достаточно долгой? Как понять, что времени проходит слишком много и нужно изучать проблему глубже? Как увидеть прогресс выполняемой работы? А что, если мы захотим ее отменить? Наша текущая стратегия завершения выполняющегося процесса локально никак не информирует сервер, что вся выполняемая им сейчас работа делается впустую. Здесь бы очень пригодился способ, позволяющий сообщать API, когда он должен просто перестать выполнять всю эту работу, поскольку ее результат нас больше не интересует.

К сожалению, рассмотренные нами дизайны API не годятся для этой задачи. И ожидать, что существующие паттерны смогут обработать эти длительные вызовы API, несколько неразумно. Но тогда возникает очевидный вопрос: что же делать? И цель данного паттерна проектирования — ответить на этот вопрос.

10.2. ОБЗОР

Эта проблема касается не только веб-API. На самом деле она намного чаще встречается при локальном выполнении работы программой. Нам нужно сделать что-то, занимающее некоторое время, но при этом мы также хотим, чтобы наша программа в процессе ожидания завершения этой медленной функции по возможности продолжила выполнять и другую работу. Это настолько распространенный случай, что во многих современных языках программирования были созданы конструкторы для обработки подобного асинхронного поведения и упрощения управления нашими программами. В разных языках такие конструкторы называются по-разному (в Python это фьючерсы, в JS — промисы), хотя сама их цель проста: начать некую работу, но не обязательно блокироваться на время ее выполнения. Вместо этого возвращается объект, который представляет плейсхолдер для этой работы. Затем по ее завершении плейсхолдер разрешается (успешно с результатом) или отвергается (выбрасывается ошибка). Для обработки подобных сценариев пользователи могут либо регистрировать в плейсхолдере обратные вызовы, которые будут обрабатывать результат асинхронно, либо ожидать возвращения результата, блокируя выполнение кода до момента разрешения или отклонения этого плейсхолдера (по сути, делая асинхронную работу синхронной) (листинг 10.2).

Листинг 10.2. Ожидание и прикрепление обратных вызовов к промисам

```
async function factor(value: number): Promise<number[]> {
  // ...
}

async function waitOnPromise() {
```

```

let factors = await factor(...);
let promise = factor(...);
let otherFactors = await promise;
}

function promiseCallback() {
  let promiseForFactors = factor(...);
  promiseForFactors.then( (factors: number[]) => {
    // ...
  }).catch( (err) => {
    // ...
  });
}

```

Здесь можно ожидать результат. В итоге либо вернется он, либо возникнет ошибка

Здесь можно прикрепить к промису обратные вызовы, обработав и разрешаемый результат, и ошибку в случае отклонения

Цель этого паттерна — спроектировать эквивалент промисов или фьючерсов для веб-API, который мы назовем *длительные операции* (long-running operations, LRO). Эти промисы API в первую очередь сосредоточены на предоставлении инструмента для отслеживания протекания API фоновой работы и должны поддерживать аналогичные паттерны взаимодействия, такие как ожидание (блокировка) результатов, проверка статуса или асинхронное уведомление о результатах. Во многих случаях эти LRO также предоставляют возможность приостанавливать, возобновлять или отменять выполняющуюся операцию, но, как мы увидим далее, это будет зависеть от обстоятельств. Пример длительной операции и ее прохождения через систему показан на рис. 10.1.

LRO схожи с промисами, но при этом весьма серьезно отличаются. Начнем с того, что, поскольку LRO отвечают за отслеживание работы, выполняемой другим методом веб-API, они представляют просто новый возвращаемый тип для этих различных методов (листинг 10.3). В отличие от промисов в языках программирования, которые можно создавать явно для фонового выполнения произвольного кода, причины для явного создания LRO возникают редко.

Еще одно важное различие заключается в постоянстве этих длительных операций. Если промисы являются эфемерными и существуют только в контексте выполняющегося процесса создавшего их кода, то LRO по определению представляют удаленный принцип, который существует и оперирует в удаленной среде выполнения сервера API. В результате их нужно рассматривать как подблюдающие ресурсы API, присутствующие в API с собственными идентификаторами. Иными словами, эти LRO будут существовать даже после завершения создавшего их процесса.

Кроме того, в отличие от промисов, которые просто возвращают результат определенного типа, LRO используются для отслеживания прогресса и прочих метаданных о самой операции. Получается, что они должны предоставлять еще и определенную схему, позволяющую сохранять такие метаданные. Это может быть что-то наподобие прогресса операции, времени ее начала или завершения либо выполняемого в данный момент действия.

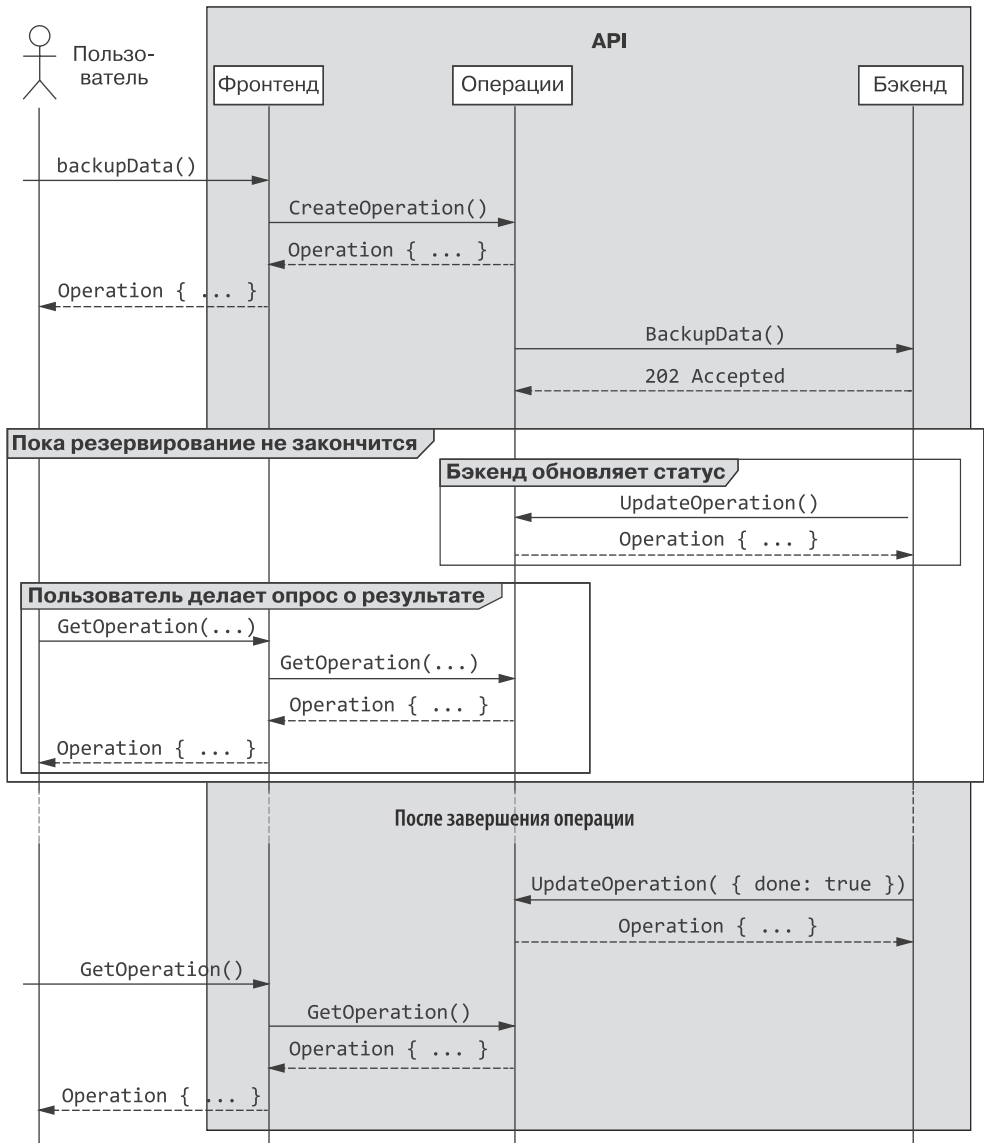


Рис. 10.1. Жизненный цикл длительной операции по бэкапу данных

Листинг 10.3. Стандартный метод Create, возвращающий LRO

```

abstract class ChatRoomApi {
    @post("/chatRooms")
    CreateChatRoom(req: CreateChatRoomRequest):
        Operation<ChatRoom, CreateChatRoomMetadata>;
}

```

Вместо возвращения непосредственно ресурса ChatRoom метод Create возвращает LRO, которая в конечном итоге приведет к появлению ресурса ChatRoom

Наконец, поскольку LRO могут существовать произвольно долго, не устраняясь вместе с породившим их процессом, это чревато возникновением проблем с управлением ресурсом `Operation`. Мы не можем быть уверены, что у нас всегда будет устойчивая запись идентификатора каждой LRO, поэтому нам также нужен способ обнаружения длительных операций, о которых знает API. Для этого мы используем сервис API со стандартным методом поиска LRO, таким как `ListOperations()`.

Теперь, когда у нас есть представление о принципе работы длительных операций, можно более подробно рассмотреть, как они выглядят и как интегрировать этот принцип в веб-API.

10.3. РЕАЛИЗАЦИЯ

Чтобы добавить поддержку асинхронных вызовов API, опирающихся на LRO, нам понадобится проделать две вещи. Во-первых, необходимо определить, как выглядят эти ресурсы `Operation`. Сюда будет входить допущение некой формы общей параметризации как для типа результата LRO (например, возвращаемого в конечном итоге ресурса), так и для интерфейса, хранящего любые нужные нам метаданные об операции (например, индикаторы прогресса и прочую информацию о самой LRO).

Во-вторых, поскольку длительные операции существуют не в той среде, в которой создаются (сервис API и клиентский код), нам нужен способ, позволяющий найти и управлять этими LRO в API. То есть нам потребуется определить несколько методов API для взаимодействия с LRO, в первую очередь стандартный метод `List`, затем стандартный метод `Get` и потенциально пользовательские методы для приостановки, возобновления и отмены операций. Начнем с определения самой LRO.

10.3.1. Как выглядит длительная операция

Чтобы понять, как должен выглядеть интерфейс LRO, нужно определиться с тем, что мы хотим делать с этим ресурсом. Для начала, поскольку это ресурс, нам нужна возможность взаимодействовать с ним, значит, потребуется обязательное поле идентификатора. Далее, основная цель этих LRO — возвращение итогового результата, то есть нам определенно нужен способ выражения этого результата. Тем не менее нам также необходимо рассмотреть случаи, когда возникает ошибка и операция не завершается успехом. Кроме того, следует учесть тот факт, что бывают операции, которые вообще не имеют результата, и успех выражается просто отсутствием ошибки. Для обработки всех этих случаев можно использовать необязательное поле результата, которое может

быть либо интерфейсом `OperationError` (с номером кода, текстом сообщения и дополнительными деталями), либо параметризованным типом результата. И в последнем варианте можно задействовать дженерики `TypeScript`, принимая входящий обобщенный тип `ResultT`.

Кроме того, нам также нужно организовать получение статуса LRO. Во многих языках промисы имеют три потенциальных состояния: ожидание, разрешен (успешно завершен) или отвергнут (завершился с ошибкой). И хотя у нас может быть возможность вывести статус LRO из отсутствующего поля результата, такой подход чреват ошибками в случаях, когда конечный результат будет просто пуст. Чтобы разобраться с этим нюансом, ресурс `Operation` должен содержать отдельный логический флаг, указывающий, завершилась ли LRO. А чтобы это указание не говорило об успехе или провале операции (например, `complete` может сообщать, что ресурс завершил выполнение успешно), мы просто назовем его `done`.

Наконец, необходимо учесть, что от этих LRO может потребоваться предоставить метаданные о выполняемой работе до того, как результат окажется доступен. Например, мы хотим показывать пользователям индикатор прогресса или приблизительное время до завершения операции. Вместо того чтобы просто раскрывать лишнюю схему значения этих метаданных, можно в качестве обобщенного параметра для этого значения принять интерфейс, называемый `MetadataT` (листинг 10.4).

Листинг 10.4. Определение интерфейсов `Operation` и `OperationError`

```

Поле done указывает, продолжает ли
Operation выполнять работу

interface Operation<ResultT, MetadataT> {
  id: string;
  done: boolean;
  result?: ResultT | OperationError;
  metadata?: MetadataT;
}

interface OperationError {
  code: string;
  message: string;
  details?: any;
}

```

Дженерики `ResultT` и `MetadataT` служат для определения интерфейса типа результата, а также для сохраняемых метаданных об LRO

Стоит также сказать, что `ResultT` не обязательно должен являться ресурсом API. В некоторых случаях это может быть пустое значение, например, когда работе не о чем сообщить, кроме как о своем завершении. Он также может выступать неким эфемерным значением, таким как интерфейс `TranslationResult`, который может содержать результаты запроса на перевод текста с одного языка на другой. Более того, при отсутствии метаданных об LRO, которые бы стоило

сохранить, этот тип, очевидно, можно устанавливать на `null`, полностью исключая данное поле.

Теперь, когда у нас есть этот тип LRO, его нужно использовать. Для этого мы просто возвращаем его как результат метода API. Например, как было показано в листинге 10.3, можно сделать стандартный метод `Create` асинхронным, возвращая тип `Operation<ChatRoom, CreateChatRoomMetadata>`, а не просто `ChatRoom`, как мы научились делать в главе 7. Однако внутренне нам нужно создать и сохранить этот ресурс `Operation`, что ведет к интересному вопросу о месте LRO в иерархии ресурсов.

10.3.2. Иерархия ресурсов

Поскольку LRO, созданные асинхронными методами, нужно где-то разместить, возникает важный и очевидный вопрос: где именно в иерархии ресурсов они должны находиться? И у нас есть два варианта. Первый — сохранять коллекцию операций под конкретным ресурсом, с которым работает LRO. Второй — сохранять операции в виде коллекции верхнего уровня, отдельно и в стороне от ресурсов, с которыми они в итоге смогут работать.

В случаях, когда LRO используются, чтобы сделать стандартные методы асинхронными (как было показано в листинге 10.3), первый вариант может быть абсолютно разумным решением. Но все же, каким бы соблазнительным ни казался такой подход, он порождает ряд проблем. Во-первых, что, если операция не обязательно фокусируется на ресурсе? Что, если вместо этого она будет более скоротечной, например, являясь транскрибированием аудио в текст? Во-вторых, если мы разделим эти коллекции по разным ресурсам (например, `chatRooms/1/operations/2` и `chatRooms/2/operations/3`), то лишимся удобной возможности запрашивать весь список операций в системе.

Ввиду всего этого куда лучшей идеей будет использовать централизованную коллекцию верхнего уровня ресурсов `Operation` (например, `/operations/1`). Располагая такой централизованной схемой ресурсов, как мы увидим в последующих разделах, мы можем найти в системе и другие LRO, а не просто адресовывать по одной операции за раз.

Теперь, когда у нас есть понимание о том, где должны находиться ресурсы длительных операций, пора разобраться с их фактическим использованием.

10.3.3. Разрешение

До сих пор мы отмечали, что методы API можно легко сделать асинхронными, просто поменяв возвращаемый тип. Этот переход от мгновенного результата к LRO, которая в конечном итоге также вернет результат, определенно является

быстрым и удобным способом обработки подобных сложностей. Однако мы ничего не сказали о том, как работать с возвращенным ресурсом `Operation`. Иначе говоря, что именно нужно делать с этим ресурсом LRO после его получения? Как добиться от него конечного результата?

В большинстве языков программирования промисы могут завершаться одной из двух стадий: отвергнут или разрешен. Это разрешение, когда промис завершается успешно и возвращает итоговое значение, пожалуй, важнейшая его часть (в нашем случае часть операции). Как же получить такой результат? Оказывается, как мы видели в листинге 10.2, для этого есть немало способов. Однако адаптация этих стилей локального разрешения промиса под мир веб-API потребует внесения некоторых корректив.

Далее мы разберем два способа, позволяющих определить конечный результат LRO. Начнем с опроса результата.

Опрос

Чтобы определить, продолжает LRO выполняться или же завершилась, достаточно просто спросить об этом сервис. То есть можно непрерывно отправлять запросы к ресурсу LRO, пока поле `done` не окажется установлено на `true` (листинг 10.5). В действительности многие клиентские библиотеки, которые опираются на различные подбоя LRO, внутренне выполняют именно такую проверку, либо со стандартным временем ожидания между запросами, либо с некой формой экспоненциального замедления, то есть постепенного увеличения промежутка между их отправками.

Листинг 10.5. Опрос на предмет обновления состояния LRO, пока оно не окажется `done`

```

async function createChatRoomAndWait(): Promise<ChatRoom> {
  let operation: Operation<ChatRoom, CreateChatRoomMetadata>;
  operation = CreateChatRoom({
    resource: { title: "Cool Chat" }
  });

  while (!operation.done) {
    operation = GetOperation<ChatRoom,
      CreateChatRoomMetadata>({
        id: operation.id
      });
    await new Promise( (resolve) => {
      setTimeout(resolve, 1000)
    });
  }

  return operation.result as ChatRoom;
}

```

Вместо возвращения `ChatRoom` метод `CreateChatRoom` возвращает `Operation`, являющуюся промисом, который в конечном итоге разрешится в `ChatRoom` (или в `OperationError`)

Цикл продолжается до тех пор, пока поле операции `done` остается `false`

В каждой итерации мы запрашиваем информацию об операции

Ожидаем некоторое время, позволяя серверу API выполнить работу

Наконец, возвращаем результат операции

Здесь нам требуется лишь метод, возвращающий в ответе ресурс `Operation` (в данном случае `CreateChatRoom`), и параметризованный метод `GetOperation` для получения статуса LRO (листинг 10.6).

Листинг 10.6. Определение стандартного метода `get` для LRO

```
abstract class ChatRoomApi {
    @post("/chatRooms")
    CreateChatRoom(req: CreateChatRoomRequest):
        Operation<ChatRoom, CreateChatRoomMetadata>;

    @get("/{id=operations/*}")
    GetOperation<ResultT, MetadataT>(req: GetOperationRequest):
        Operation<ResultT, MetadataT>;
}

interface GetOperationRequest {
    id: string;
}
```

Метод параметризуется, чтобы обеспечить
верный возвращаемый тип. Этот метод
будет использован для всех типов операций,
а не только для метода `CreateChatRoom`

Очевидный недостаток такого поведения в том, что здесь наверняка получится длинный список запросов, которые приводят лишь к повтору, что можно расценить как пустую трату ресурсов (сетевого трафика и вычислительного времени клиента и сервера API). И хотя это не очень удобно, такой способ оказывается наиболее понятным, обеспечивает своевременное получение информации о результатах (согласно тому, что мы понимаем под «своевременно») и оставляет бремя запрашивания результатов на клиенте, который лучше знает, когда именно эти результаты станут ему интересны.

Теперь можно переходить к следующему варианту поиска результата LRO: ожиданию.

Ожидание

Опрос на предмет обновления состояния LRO передает контроль клиенту. Иными словами, теперь уже клиент выбирает, как часто проверять операцию, и может останавливать проверку по любой причине, например ввиду потери интереса к результату. Один из недостатков опроса заключается в том, что он не позволяет узнать о завершении операции сразу. Конечно, можно уменьшать промежутки между запросами, но это сильно увеличит количество бесполезных запросов, не ведущих к получению обновления. По сути, система опросов несет в себе неизбежный компромисс между максимальной задержкой перед получением обновления и количеством потраченных впустую вычислительных ресурсов и времени.

Обойти это можно при использовании долгосрочного подключения к сервису API, которое закрывается только по завершении операции. То есть мы делаем

запрос к сервису API с просьбой оставлять подключение открытым и ответить, только когда поле `done` операции окажется `true`. Такое блокирование запроса гарантирует, что мы будем проинформированы о результатах операции в момент, когда сервис API узнает о ее завершении. И хотя возможно, что на сервере эта реализация все равно будет опираться на опрос, такой внутренний опрос выполняется намного проще, чем при опросе сервера удаленным клиентом.

Недостаток такого подхода заключается в том, что теперь за управление всеми этими подключениями и обеспечение отправки ответов на запросы после завершения операции (что может потребовать немало времени) отвечает сервер API. Этот процесс может стать довольно сложным, но в конечном итоге *пользователи* API получают простой паттерн взаимодействия.

Чтобы это сработало, нужно добавить пользовательский метод `wait`. Он выглядит практически идентично с параметризованным методом `GetOperation`, но вместо мгновенного возвращения статуса ожидает момента разрешения или отклонения операции (листинг 10.7).

Листинг 10.7. Определение пользовательского метода для LRO

```
abstract class ChatRoomApi {
    @get("/{id=operations/*}:wait")
    WaitOperation<ResultT, MetadataT>(req: WaitOperationRequest):
        Operation<ResultT, MetadataT>;
}

interface WaitOperationRequest {
    id: string;
}
```

Здесь мы используем HTTP-глагол GET, поскольку он должен быть идиempотентным

Стоит заметить, что входящий и возвращаемый типы в `WaitOperation` и `GetOperation` идентичны, поэтому может быть проще добавить в `GetOperation` простой флаг, который указывает, нужно ли ожидать. И хотя таким образом мы определенно используем пространство более разумно, это означает, что один логический флаг может значительно изменить внутреннее поведение метода API, что обычно окажется нежелательным. Более того, используя отдельный метод, мы упрощаем отслеживание и внедрение ориентиров на уровне сервиса, таких как ожидаемое время ответа. Если же мы используем флаг, то нам будет трудно отделить вызовы API, медленные из-за обработки, от мгновенных методов `GetOperation`, медленных из-за проблем инфраструктуры.

Как это работает в коде клиента? Если коротко, то клиент может просто ожидать ответа на вызов API. Иначе говоря, этот метод, по сути, заставляет любой асинхронный вызов API действовать как синхронный. Метод `WaitOperation` значительно упрощает нашу функцию `createChatRoomAndWait()` (листинг 10.8).

Листинг 10.8. Клиентский код, использующий пользовательский метод `WaitOperation`

```
function createChatRoomAndWait(): Promise<ChatRoom> {
  let operation: Operation<ChatRoom, CreateChatRoomMetadata>;
  operation = CreateChatRoom({
    resource: { title: "Cool Chat" }
  });

  operation = WaitOperation({ id: operation.id });
  return operation.result as ChatRoom;
}
```

Можно получить промис для создания ресурса ChatRoom с помощью стандартного метода Create

Вместо использования цикла можно просто подождать результата операции

При условии отсутствия ошибок результат будет доступен сразу по возвращении методом WaitOperation

И хотя это определенно ведет к простому синхронному клиентскому коду, ожидание ответов подразумевает множество потенциальных проблем. Если подключение по какой-то причине будет разорвано (например, клиентом является мобильное устройство, которое может потерять сигнал), мы вернемся к тому, с чего начали, — к необходимости опроса (другого типа опроса, который управляется внешними обстоятельствами, а не ориентированными на время событиями).

Теперь, разобравшись с извлечением результата LRO, пора взглянуть на менее привычный сценарий, когда результат оказывается безуспешным и нужно обработать ошибку.

10.3.4. Обработка ошибок

В большинстве веб-API ошибки проявляются в форме HTTP-кодов и сообщений. Например, мы можем получить ошибку `404 Not Found`, указывающую, что ресурс не был найден, или ошибку `403 Forbidden`, сообщающую, что у нас нет прав доступа к ресурсу. Все это отлично функционирует, когда ответ является мгновенным и синхронным, но как обработать такое поведение в асинхронной среде LRO?

И хотя здесь кажется удобным просто передавать ответ с ошибкой при сбое LRO, это может вызвать серьезные проблемы. К примеру, если ответ на `GetOperation` даст ошибку внутренней операции, то как отличить `500 Internal Server Error`, ставшую результатом операции, от той же ошибки, возникшей просто по вине кода, обрабатывающего метод `GetOperation`? Очевидно, что здесь нужна альтернатива.

Как мы уже видели, способ, с помощью которого мы будем это обрабатывать, подразумевает допущение либо дополнительного результата (указанного типа `ResultT`), либо типа `OperationError`, куда входит машиночитаемый код, понятное человеку описание и произвольная структура для сохранения дополнительной

информации об ошибке. Это означает, что метод `GetOperation` должен выбрасывать исключение (возвращая HTTP-ошибку), только если при извлечении ресурса `Operation` произошла реальная ошибка. Если же этот ресурс был извлечен успешно, то ответ с ошибкой просто будет частью результата LRO. Далее уже клиентский код будет решать, нужно выбрасывать на его стороне исключение или же обрабатывать результат с ошибкой иным способом. Например, можно настроить функцию `createChatRoomAndWait()` для обработки возможного результата с ошибкой (листинг 10.9).

Прежде чем продолжать, нужно подчеркнуть важность того, что значение кода ошибки должно быть уникальным, специфичным и предназначенным для восприятия машиной, а не человеком. Когда коды ошибок не сообщаются (или пересекаются, или предназначаются для людей, а не компьютеров), мы оказываемся в пугающей ситуации, где пользователи API могут начать выяснять причины неполадки, исходя из содержимого данного сообщения. Само по себе это не является проблемой (ведь сообщения об ошибках для того и нужны, чтобы прояснять причины проблем). Тем не менее, когда пользователь API чувствует потребность написать код, который опирается на содержимое такого сообщения, оно внезапно становится частью API. В результате, когда мы изменяем это сообщение (скажем, исправляя опечатку), то можем ненамеренно внести ошибки в рабочий код. Как мы узнаем в главе 24, этого необходимо избегать любой ценой и лучше всего использовать коды ошибок, которые будут для кода куда менее полезны, чем сами сообщения об ошибках.

Листинг 10.9. Проверка на ошибку или возвращение результата

```
function isOperationError(result: any):
    result is OperationError {
    const err = result as OperationError;
    return err.code !== undefined && err.message !== undefined;
}

function createChatRoomAndWait(): Promise<ChatRoom> {
    let operation: Operation<ChatRoom, CreateChatRoomMetadata>;
    operation = CreateChatRoom( { resource: { title: "Cool Chat" } });
    operation = WaitOperation({ id: operation.id });
    if (isOperationError(operation.result)) {
        // ...
    } else {
        return operation.result as ChatRoom;
    }
}
```

Сначала мы определяем проверяющую тип функцию, выясняя, является ли результат ошибкой

Далее можно проверить результат и в зависимости от успешности операции выполнить различные действия

В случаях, когда пользователям нужно не только знать тип ошибки, но и какую-либо дополнительную информацию о ней, идеальным местом для размещения этой структурированной машиночитаемой информации окажется поле подробностей. Несмотря на то что строгого запрета на внесение этой дополнительной информации в сообщение об ошибке нет, она окажется намного более полезной

именно в поле подробностей и определенно является необходимой. Как правило, если сообщение об ошибке отсутствовало, то мы не должны терять информацию, которую нельзя найти в документации API.

Теперь пора перейти к очередному интересному, но уже необязательному аспекту LRO: мониторингу прогресса.

10.3.5. Отслеживание прогресса

До этого момента мы рассматривали операции как выполненные или нет, не обращая внимания на промежуточные этапы между этими состояниями. Как же можно отслеживать прогресс LRO? Именно здесь в игру вступает поле метаданных ресурса `Operation`.

Как мы видели в листинге 10.4, ресурс `Operation` получает два типа для двух полей. Первый был типом результата, а второй — `MetadataT`, присваиваемый полю метаданных. Это и есть особый интерфейс, который можно использовать для сохранения информации не о результате, а о самой LRO. Ею могут выступать простые данные, такие как временная метка начала работы или, в данном случае, степень прогресса ее выполнения, оценочное время до завершения или количество обработанных данных.

И хотя прогресс зачастую означает «выполнение в процентах», здесь нет какого-то строгого требования. Во многих сценариях процентное отображение может оказаться полезным, но при этом удобнее будет показывать примерное время или даже измерять прогресс в реальных величинах, таких как количество обработанных записей. Более того, нет никакой гарантии, что будет отображаться действительное значение в процентах — и, думаю, всем хорошо известно, как это может раздражать, когда отсчет прогресса в процентах начинает идти вспять. К счастью, для поля метаданных можно использовать любой желаемый интерфейс. В листинге 10.10 показан пример кода, анализирующего историю беседы в ресурсе `ChatRoom` для сбора языковой статистики. Прогресс в нем измеряется в количестве проанализированных и подсчитанных сообщений.

Листинг 10.10. Определение пользовательского метода `AnalyzeMessages` с типом метаданных

```
abstract class ChatRoomApi {
    @post("/{parent=chatRooms/*/}/messages:analyze")
    AnalyzeMessages(req: AnalyzeMessagesRequest):
        Operation<MessageAnalysis, AnalyzeMessagesMetadata>;
}

interface AnalyzeMessagesRequest {
    parent: string;
}
```

```
interface MessageAnalysis {
  chatRoom: string;
  messageCount: number;
  participantCount: number;
  userGradeLevels: map<string, number>;
}

interface AnalyzeMessagesMetadata {
  chatRoom: string;
  messagesProcessed: number;
  messagesCounted: number;
}
```

← Результатом этого метода будет не ресурс, а интерфейс анализа, показывающий, помимо прочего, уровни качества сообщений каждого пользователя

← Интерфейс метаданных определяет прогресс на основе обработанных и подсчитанных сообщений

Как же это использовать? В подразделе 10.3.3 мы видели, как использовать опрос для непрерывной проверки длительной операции в ожидании ее завершения. Аналогичным образом можно использовать метод `GetOperation` для извлечения информации о прогрессе, хранящейся в интерфейсе `AnalyzeMessagesMetadata` (листинг 10.11).

Листинг 10.11. Отображение прогресса с помощью интерфейса метаданных

```
async function analyzeMessagesWithProgress(): Promise<MessageAnalysis> {
  let operation: Operation<MessageAnalysis, AnalyzeMessagesMetadata>;
  operation = AnalyzeMessages({ parent: 'chatRooms/1' });

  while (!operation.done) {
    operation = GetOperation<MessageAnalysis, AnalyzeMessagesMetadata>({
      id: operation.id
    });
    let metadata = result.metadata as
      AnalyzeMessagesMetadata;
    console.log(
      `Processed ${metadata.messagesProcessed} of ` +
      `${metadata.messagesCounted} messages counted...`);
    await new Promise( (resolve) => setTimeout(resolve, 1000));
  }

  return operation.result as MessageAnalysis;
}
```

← Сначала извлекаем метаданные из операции в ее текущем состоянии

← Далее выводим в консоль строку, обновляя прогресс по мере обработки и подсчета сообщений

Теперь, когда мы знаем, как проверять прогресс операции, можно перейти к изучению других способов взаимодействия с операциями. Например, что, если прогресс операции продвигается намного медленнее, чем ожидалось? Возможно, мы допустили ошибку? Как тогда отменить операцию?

10.3.6. Отмена операций

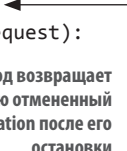
До сих пор все наши взаимодействия с длительными операциями предполагали, что, начавшись, они продолжают выполнять работу, пока либо не разрешатся успешно, либо не будут отвергнуты с ошибкой. Но что, если мы не хотим ожидать завершения операции?

Это может быть связано с разными причинами. Опечатки и ошибки в мысленных расчетах происходят намного чаще, чем мы готовы это признать. В результате бывает, что мы пишем код, который создает LRO для неверных данных, с ошибочным адресом вывода, а иногда даже просто с полностью ошибочным методом. Если бы все эти случаи были локально выполняющимися промисами, то мы бы просто завершали процесс и прекращали выполнение. Однако мы имеем дело с удаленной средой выполнения, поэтому нам нужен способ, позволяющий запросить какой-то эквивалент данного действия. С этой целью можно прибегнуть к пользовательскому методу `CancelOperation` (листинг 10.12).

Листинг 10.12. Определение пользовательского метода отмены операции

```
abstract class ChatRoomApi {
    @post("/{id=operations/*}:cancel")
    CancelOperation<ResultT, MetadataT>(req: CancelOperationRequest):
        Operation<ResultT, MetadataT>;
}

interface CancelOperationRequest {
    id: string;
}
```



Этот метод возвращает полностью отмененный ресурс `Operation` после его остановки

Так же как и метод `GetOperation`, метод `CancelOperation` будет возвращать параметризованный ресурс `Operation`, поле `done` которого будет установлено на `true` (напомню, что имя `done` мы выбрали, поскольку оно не подразумевает завершение или успех операции). Чтобы этот результат всегда рассматривался как завершенный, данный метод должен блокироваться до момента полной отмены операции и только потом возвращать результат. Бывают случаи, когда это может занять какое-то время (возможно, необходимо связаться с несколькими подсистемами). Тем не менее по-прежнему имеет смысл подождать ответа, как и в случае с методом `WaitOperation`, упомянутым в подразделе 10.3.3.

Кроме того, важно, чтобы любой промежуточный итог работы или прочий вывод, ставший результатом частичного выполнения LRO, по возможности удалялся. В конце концов, цель отмены длительной операции — получить систему, которая выглядит так, будто операция никогда и не начиналась. В случаях, когда нет возможности провести очистку при отмене операции, необходимо обеспечить, чтобы вся информация, необходимая пользователю для самостоятельного выполнения этой очистки, присутствовала в поле метаданных. К примеру, если операция создала в хранилище несколько файлов, которые по какой-либо причине невозможно удалить перед отменой операции, метаданные должны включать ссылки на эти промежуточные данные, чтобы пользователь мог выполнить их удаление самостоятельно.

Последним стоит отметить, что возможность отмены операций не должна являться строгим требованием для всего API, хотя это может казаться удобным. Дело в том, что операции отличаются по своему назначению, сложности

и наличию или отсутствию возможности отмены. Зачастую есть смысл отменять некоторые операции, но бывают и такие, которые просто не вписываются в этот паттерн. При этом никогда не нужно добавлять поддержку удаления LRO, которое не принесет выгоды пользователю.

На этом можно перейти к рассмотрению следующего типа взаимодействия, которое по своей природе схоже с отменой, но не является столь безвозвратным. Поговорим о приостановке и возобновлении.

10.3.7. Приостановка и возобновление операций

Теперь, когда мы начали взаимодействовать с операциями способами, которые значительно меняют их поведение (например, отменяя их работу), можно познакомиться с достаточно важным аспектом LRO: возможностью приостанавливать выполняющуюся работу и возобновлять ее позже. И хотя здесь можно опереться на пользовательский метод, аналогичный методу `CancelOperation`, этот сценарий требует учитывать кое-какие дополнительные нюансы.

Самое главное: если операция приостановлена, то это определенно нужно как-то выразить. К сожалению, поле `done` в ресурсе `Operation` не сработает. Приостановленная операция не завершена, поэтому нельзя просто переназначить это поле под наши цели. В итоге у нас есть два варианта: добавить в интерфейс `Operation` новое поле (`paused` или аналогичное) либо использовать для этой операции метаданные.

Как бы красиво и просто ни выглядело добавление нового поля в интерфейс `Operation`, это создает проблему: нет никакой гарантии, что каждую операцию можно будет приостановить и возобновить. В одних случаях это может иметь смысл, но в других эти LRO представляют задачи по типу «все или ничего», когда приостановка операции не сулит пользователям никаких преимуществ. К примеру, попробуйте приостановить API, запускающий ракету, когда ракета уже оторвалась от земли. Просто нет смысла.

В общем, перед нами остается менее элегантное, но более удачное решение: использовать метаданные. Это подразумевает важное правило: чтобы операцию можно было приостановить и возобновить, тип метаданных (`MetadataT`) должен содержать логическое поле `paused` (листинг 10.13).

Листинг 10.13. Добавление поля `paused` в интерфейс метаданных

```
interface AnalyzeMessagesMetadata {
    chatRoom: string;
    paused: boolean;
    messagesProcessed: number;
    messagesCounted: number;
}
```

← Чтобы поддерживать приостановку и возобновление операции, интерфейс метаданных должен содержать логическое поле `paused`

Методы паузы и возобновления (`PauseOperation` и `ResumeOperation`) по существу идентичны другим изученным нами ресурсам. Они получают идентификатор ресурса `Operation`, а также параметры для типов (`ResultT` и `MetadataT`), которые будут возвращаться при выполнении операции. И точно так же, как метод `CancelOperation`, эти методы должны возвращать результат только при успешной приостановке или возобновлении операции (листинг 10.14).

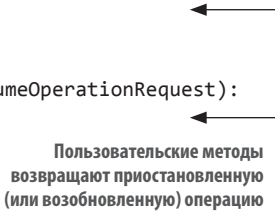
Листинг 10.14. Определение пользовательских методов приостановки и возобновления

```
abstract class ChatRoomApi {
    @post("/{id=operations/*}:pause")
    PauseOperation<ResultT, MetadataT>(req: PauseOperationRequest):
        Operation<ResultT, MetadataT>;

    @post("/{id=operations/*}:resume")
    ResumeOperation<ResultT, MetadataT>(req: ResumeOperationRequest):
        Operation<ResultT, MetadataT>;
}

interface PauseOperationRequest {
    id: string;
}

interface ResumeOperationRequest {
    id: string;
}
```



Пользовательские методы
возвращают приостановленную
(или возобновленную) операцию

Фактический процесс приостановки или возобновления внутренней работы остается в качестве детали реализации для API. Как бы ни было интересно исследовать всевозможные механизмы приостановки и возобновления работы в различных API, самое главное — сказать о том, что мы должны поддерживать такие методы в тех сценариях, где это разумно и возможно. То есть если мы захотим поддерживать приостановку метода API, который, к примеру, копирует данные из одного места в другое, то нам потребуется сохранить указатель на процесс, выполняющий копирование, и обеспечить возможность его завершения, а также поправить все то, что он успеет за время копирования натворить. Остальные же детали остаются фактическому разработчику API.

10.3.8. Инспектирование операций

Одна из уникальностей длительных операций заключается в том, что код, который создает промис API, отделен от среды, выполняющей этот промис и всю работу. То есть мы можем оказаться в довольно затруднительной ситуации, если инициировавший работу код по какой-то причине дал сбой до момента завершения промиса. Если при этом не сохранить ID операции, чтобы можно было возобновить наш цикл опросов позднее, то указатель на операцию, выполняющую

запрошенную нами работу, будет утерян со всем остальным состоянием, когда процесс завершится в нашей локальной среде выполнения.

Этот сценарий демонстрирует серьезную оплошность: мы можем извлечь информацию об одном ресурсе `Operation` по его идентификатору, но у нас нет способа, позволяющего просмотреть выполняющуюся работу или инспектировать состояние и историю системы API. К счастью, эту проблему легко решить. Чтобы восполнить пробел, нужно лишь определить стандартный метод `List` для коллекции ресурсов `Operation` в нашем API (листинг 10.15).

Этот метод должен действовать подобно другим стандартным методам `List`, но при этом критически важно реализовать поддержку хоть какой-то фильтрации. В противном случае у нас могут возникнуть сложности при постановке вопросов наподобие этого: «Какие ресурсы `Operation` на данный момент еще не завершены?» Более того, фильтрация также должна поддерживать запрос метаданных в ресурсах. Иначе мы просто не сможем ответить на вопросы типа «Какие операции приостановлены?».

Листинг 10.15. Определение стандартного метода `List` для LRO

```
abstract class ChatRoomApi {
    @get("/operations")
    ListOperations<ResultT, MetadataT>(req:
    └─ ListOperationsRequest):
        ListOperationsResponse<ResultT, MetadataT>;
}

interface ListOperationsRequest {
    filter: string;
    └─ Позже мы рассмотрим,
        как применять пагинацию
}

interface ListOperationsResponse<ResultT, MetadataT> {
    results: Operation<ResultT, MetadataT>[];
}
```

Этот метод соответствует стандартному методу `List`, но допускает параметризацию

Этот простой метод API позволяет ответить на все вопросы об операциях, которые у нас могут возникнуть, особенно когда идентификаторы неизвестны заранее. Обратите внимание: операция принимает параметры для типов результата и метаданных, но это не должно препятствовать возможности запросить все операции независимо от задействованных типов. Это можно сделать с помощью ключевого слова `TS any`. Однако инспектирование результата с целью определить конкретный тип будет лежать на плечах вызывающего (в большинстве случаев идентификатор результата будет включать тип ресурса).

Вполне очевидно, что фильтрация по всем этим ресурсам — довольно эффективный прием, но когда операции завершаются, нам нужно решить, как долго хранить полученные записи. В следующем подразделе мы разберем критерий продолжительности хранения этих записей LRO.

10.3.9. Продолжительность хранения

Несмотря на то что длительные операции являются ресурсами (если конкретно, то ресурсами `Operation`), они несколько отличаются от ресурсов, которые мы определяем сами. Начнем с того, что они создаются не явно, а в результате другого поведения в системе (например, запрос анализа данных может привести к созданию ресурса `Operation` для отслеживания выполняемой работы). Такое подспудное возникновение фундаментально отличается от типичного способа создания ресурсов, обычно основанного на стандартном методе `Create`.

Тем не менее самое важное заключается в том, что эти ресурсы очень быстро переходят из разряда жизненно необходимых в категорию устаревших. Например, любая LRO, являющаяся результатом стандартного метода (например, `CreateChatRoom`), очень важна при выполнении обработки, но становится практически бесполезной по ее завершении. Причина в том, что сам результат является перманентно записанным выводом операции, следовательно, процесс, который отслеживал создание этого ресурса, далее теряет свою ценность.

Это значит, нам нужно решить сложную проблему: рассматриваем ли мы LRO как любой другой ресурс, сохраняя их навсегда? Или они должны иметь несколько иную политику хранения, нежели более традиционные ресурсы? Если они будут следовать другой политике, то какой она должна быть?

Оказывается, справиться с этой проблемой позволяют немало различных решений, каждое со своими достоинствами и недостатками. Самое простое в управлении и самое быстрое в реализации — рассматривать ресурсы `Operation` как любые другие и сохранять их навсегда. Как правило, большинство API должны опираться на этот метод, если только он не окажется излишне обременительным для системы (обычно это случаи, когда у вас ежедневно создаются миллионы ресурсов `Operation`).

Если паттерны взаимодействия API делают этот метод пожизненного хранения затратным, то следующим по эффективности решением будет использование скользящего окна (`rolling window`), когда ресурсы `Operation` очищаются на основе временной метки их завершения (не создания), определенной в поле `expireTime`, которое устанавливается в ресурсе `Operation`. Проще говоря, если ресурс просуществовал долго, скажем, 30 дней, то должен быть безвозвратно удален (листинг 10.16).

Листинг 10.16. Добавление в интерфейс `Operation` поля срока действия

```
interface Operation<ResultT, MetadataT> {
    id: string;
    done: boolean;
    expireTime: Date;
    result?: ResultT | OperationError;
    metadata?: MetadataT;
}
```

← Чтобы определить время удаления ресурса, можно использовать поле, указывающее срок его жизни

Существует и много других сложных и богатых на возможности вариантов (например, удаление операций при удалении ресурсов, к которым они привязаны, или архивирование операций спустя определенный промежуток времени с последующим их удалением по истечении очередного временного отрезка), но их лучше избегать. В конечном счете они вызывают больше сложностей ввиду путаницы и сложных алгоритмов удаления. Вместо этого стоит опираться на простой показатель срока годности, который окажется и понятен, и однозначен в плане конечных результатов.

Самое важное — не забывать, что срок годности ресурсов не должен зависеть от внутренних результатов операции. То есть длительность хранения операции, отвечавшей за работу по созданию ресурса, и операции, отвечавшей за анализ данных, должна быть одинаковой. Если для разных типов операций использовать разные сроки хранения, то это вызовет много путаницы, при которой результаты будут исчезать непредсказуемым образом.

10.3.10. Итоговое определение API

Соберем все воедино: в листинге 10.17 показано целостное определение API для всех методов и интерфейсов, связанных с длительными операциями.

Листинг 10.17. Заключительное определение API

```
abstract class ChatRoomApi {
    @post("/{parent=chatRooms/*/}/messages:analyze")
    AnalyzeMessages(req: AnalyzeMessagesRequest):
        Operation<MessageAnalysis, AnalyzeMessagesMetadata>;

    @get("/{id=operations/*}")
    GetOperation<ResultT, MetadataT>(req: GetOperationRequest):
        Operation<ResultT, MetadataT>;

    @get("/operations")
    ListOperations<ResultT, MetadataT>(req: ListOperationsRequest):
        ListOperationsResponse<ResultT, MetadataT>;

    @get("/{id=operations/*}:wait")
    WaitOperation<ResultT, MetadataT>(req: WaitOperationRequest):
        Operation<ResultT, MetadataT>;

    @post("/{id=operations/*}:cancel")
    CancelOperation<ResultT, MetadataT>(req: CancelOperationRequest):
        Operation<ResultT, MetadataT>;

    @post("/{id=operations/*}:pause")
    PauseOperation<ResultT, MetadataT>(req: PauseOperationRequest):
        Operation<ResultT, MetadataT>;
```

```

    @post("/{id=operations/*}:resume")
    ResumeOperation<ResultT, MetadataT>(req: ResumeOperationRequest):
        Operation<ResultT, MetadataT>;
}

interface Operation<ResultT, MetadataT> {
    id: string;
    done: boolean;
    expireTime: Date;
    result?: ResultT | OperationError;
    metadata?: MetadataT;
}

interface OperationError {
    code: string;
    message: string;
    details?: any;
}

interface GetOperationRequest {
    id: string;
}

interface ListOperationsRequest {
    filter: string;
}

interface ListOperationsResponse<ResultT, MetadataT> {
    results: Operation<ResultT, MetadataT>[];
}

interface WaitOperationRequest {
    id: string;
}

interface CancelOperationRequest {
    id: string;
}

interface PauseOperationRequest {
    id: string;
}

interface ResumeOperationRequest {
    id: string;
}

interface AnalyzeMessagesRequest {
    parent: string;
}

```

```
interface MessageAnalysis {  
    chatRoom: string;  
    messageCount: number;  
    participantCount: number;  
    userGradeLevels: map<string, number>;  
}  
  
interface AnalyzeMessagesMetadata {  
    chatRoom: string;  
    paused: boolean;  
    messagesProcessed: number;  
    messagesCounted: number;  
}
```

10.4. КОМПРОМИССЫ

Очевидно, что обрабатывать длительные вызовы API можно множеством способов. К примеру, один из самых элементарных, который к тому же окажется наиболее удобен для клиентов, выглядит так: нужно просто позволить запросу продолжаться столько, сколько нужно. Компромиссом при таких упрощенных вариантах является то, что они не слишком подходят распределенным архитектурам (например, когда один микросервис иницирует работу, а другой мониторит ее прогресс).

Еще один компромисс заключается в том, что фактически отслеживать ход выполнения вызова API, который зависит до завершения, может быть непросто. И хотя API может периодически отправлять данные, при утере соединения по любой причине (сбой процесса, разрыв связи и т. д.) мы теряем возможность возобновить мониторинг прогресса. Более того, если мы решим реализовать что-то для его восстановления после подобного сбоя сети, то велика вероятность, что наше решение будет напоминать принцип длительных операций.

LRO также могут поначалу оказаться непростыми для понимания. Это сложные, обобщенные, параметризованные ресурсы, которые создаются не явно, а вследствие запроса выполнения другой работы. Это определенно разрушает тот формат создания ресурсов, который мы изучали до сих пор. Кроме того, зачастую возникает путаница, когда дело доходит до идеи «перезапускаемых задач» и LRO, о чем мы более подробно поговорим в главе 11.

Если кратко, то хоть этот паттерн и может вызывать сложности в освоении, со временем принцип промисов API станет для пользователей API вполне понятным. При этом сам по себе этот паттерн практически не накладывает каких-либо ограничений — ведь ничто не мешает нам сохранить синхронную версию различных методов API, если пользователи ожидают именно этого.

10.5. УПРАЖНЕНИЯ

1. Почему ресурсы операций необходимо держать на верхнем уровне, а не вложенными под ресурсы, с которыми они работают?
2. В чем смысл установки срока действия LRO?
3. Если пользователь ожидает разрешения операции, которая в итоге отменяется с помощью пользовательского метода `Cancel`, то каким должен быть результат?
4. Есть ли смысл при отслеживании прогресса LRO использовать одно поле для хранения информации о завершении в процентах? Почему?

РЕЗЮМЕ

- LRO — это промисы или фьючерсы для веб-API, которые действуют как инструмент, позволяющий отслеживать работу, выполняемую сервисом API в фоновом режиме.
- LRO — это параметризованные интерфейсы, возвращающие конкретный тип результата (например, ресурс, получающийся в итоге операции) и тип метаданных для хранения информации о прогрессе самой операции.
- LRO разрешаются после определенного количества времени в результате либо ошибке, о чем пользователи узнают с помощью периодического опроса обновлений статуса или уведомления о завершении в процессе ожидания.
- LRO могут приостанавливаться, возобновляться или отменяться на усмотрение API, что реализуется с помощью пользовательских методов.
- LRO должны сохраняться в хранилище, но обычно рекомендуется также реализовать их удаление через фиксированный промежуток времени, например спустя 30 дней.

11

Перезапускаемые задачи

В этой главе

- ✓ Что такое перезапускаемые задачи.
- ✓ Чем они отличаются от длительных операций.
- ✓ Как выражать перезапускаемые задачи в виде ресурсов.
- ✓ Каковы преимущества поддержки перезапускаемых задач с помощью пользовательских методов, возвращающих LRO.
- ✓ Как представлять результаты задач с помощью ресурсов для их выполнения.

Существует много случаев, когда в API нужно выразить некую пользовательскую функциональность, выполняющуюся повторно. Тем не менее нам неинтересно предоставлять для нее все подробности каждый раз, когда мы хотим ее запустить. Более того, нам может потребоваться возможность выполнять эту часть настраиваемой работы по расписанию, принадлежащему и управляемому сервером API, а не вызываемому клиентом. Благодаря этому паттерну мы изучим стандарт для определения конкретных единиц работы, которые, будучи настраиваемыми и перезапускаемыми, потенциально будут доступны для пользователей с различными уровнями доступа в API. Помимо этого, мы предоставим паттерн для сохранения вывода каждого такого выполнения.

11.1. ЗАЧЕМ ЭТО НУЖНО

Как мы узнали в главе 10, в API иногда встречаются методы, которые не могут или не должны возвращать результат сразу. Возможность обрабатывать асинхронное выполнение определенно является очень ценной. Тем не менее по-прежнему нужно, чтобы клиент где-то активировал такое выполнение, вызывая метод API. Это отлично работает для методов, которые предназначены только для выполнения по требованию, но существуют немаловажные сценарии, которые не решаются простым наличием метода, возвращающего LRO.

Во-первых, в случае типичного пользовательского метода, который выполняется асинхронно, каждый вызов требует от вызывающего предоставления всей необходимой методу конфигурации. Это может оказаться простым и незатейливым, когда конфигурации требуется немного, но по мере роста количества ее параметров с каждым вызовом потребуется предоставлять все больше информации. Это существенно повышает важность контроля исходного кода, поскольку отыскивание ошибки в параметрах конфигурации, которые хранятся где-то на ноутбуке Джима, может создать немало сложностей. Наличие возможности сохранять эти параметры в самом API может оказаться очень полезным элементом функциональности.

Во-вторых, при использовании модели вызова метода по требованию мы сводим воедино два набора разрешений: возможности выполнять метод и выбирать параметры для его вызова. Зачастую в этом нет ничего особенного, но опять же, по мере расширения параметров конфигурации нам может потребоваться разделить их ответственность на две группы: те, которые могут настраивать вызов метода, и те, которые могут вызывать его фактически. Это становится особенно важно в момент, когда пользователи API начинают различать разработчиков и операции как две команды с раздельной ответственностью и разрешениями. Несложно догадаться, что в таком случае будет сложно внедрять разрешения, гласящие нечто наподобие этого: «Пользователь А может вызывать этот метод только с конкретно этими параметрами конфигурации».

Наконец, если мы уже достаточно углубились в выполнение методов по требованию, то велика вероятность, что в конечном итоге нам потребуется возможность вызывать их автоматически, согласно повторяющемуся расписанию. И хотя вполне можно настроить клиентское устройство на выполнение этих вызовов API по конкретному графику (например, с помощью сервера, который запускает сценарий на основе определений в `crontab`), это чревато новой потенциально ошибочной подсистемой, отвечающей за успех данной работы. Вместо этого будет намного проще иметь возможность настраивать расписание в самом API, чтобы сервис мог вызывать за нас метод без внешнего участия.

Возможно, нам следует определить стандарт для настраиваемых задач, которые могут хранить различные параметры конфигурации и при необходимости перезапускаться (потенциально по расписанию).

11.2. ОБЗОР

Чтобы реализовать эти три сценария, можно опереться на простой, но мощный принцип: *задача*. Это особый тип ресурса, который разбивает на две части версию метода, выполняющуюся по требованию. Вызывая типичный метод API, мы предоставляем конфигурацию для работы, на основе которой эта работа сразу выполняется. В случае же задачи все это делится на две отдельные части функциональности. Сначала мы создаем задачу с конфигурацией для требуемой работы. Затем позднее можно будет выполнить фактическую работу, вызвав пользовательский метод `Run` в ресурсе `Job`. Пример последовательности вызовов API, обрисовывающий вариант использования задач, показан на рис. 11.1.

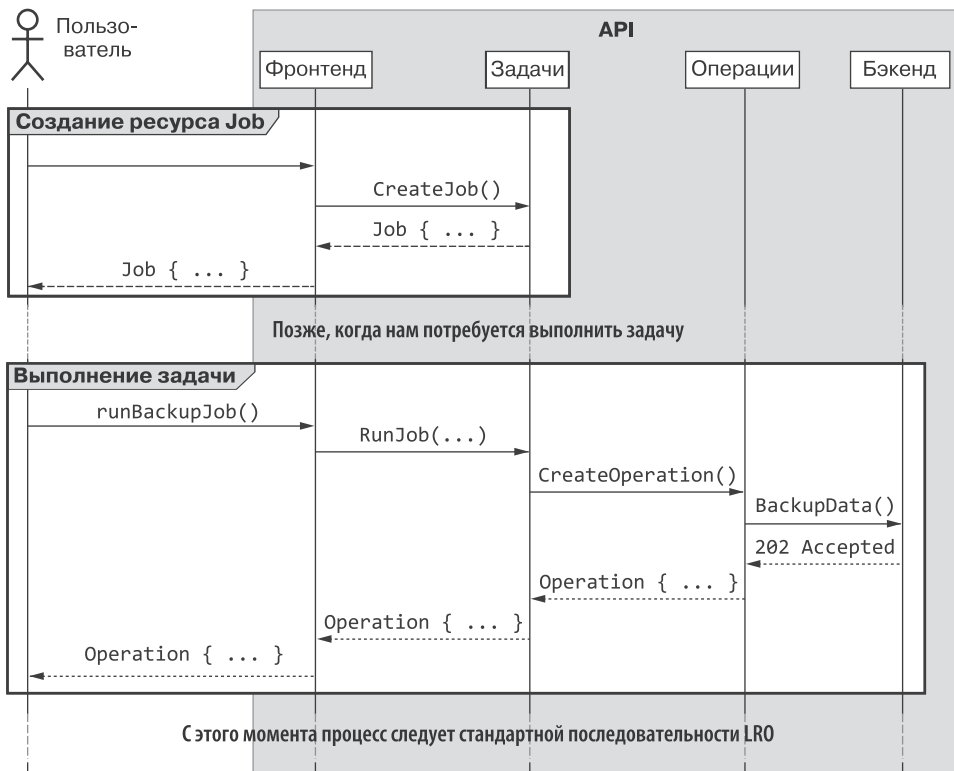


Рис. 11.1. Взаимодействие с ресурсом Job

Разделяя работу на два компонента, мы закладываем фундамент для реализации всех трех случаев использования, описанных в разделе 11.1. Во-первых, не придется беспокоиться об увеличении количества параметров, поскольку разобраться с ними нужно будет лишь раз: при создании ресурса задачи. С этого момента мы будем вызывать метод `Run` без каких-либо параметров. Во-вторых, так как у нас здесь два отдельных метода API, мы можем контролировать, кто и к какому из них имеет доступ. Можно спокойно сообщить, что конкретный пользователь имеет доступ к выполнению преднастроенной задачи, но не может создавать или изменять их. Наконец, мы получаем простой способ реализации в рамках API выполнения по графику. Вместо сложной системы расписания, получающей кучу параметров, можно просто запросить вызов единственного метода API по нужному графику вообще без параметров.

Но гораздо проще описать, чего мы хотим от наших задач, чем разобраться с тем, как эти магические задачи работают. В следующем разделе мы перейдем к подробностям функционирования задач, а также познакомимся с различными стандартными и пользовательскими методами, позволяющими реализовать их работу в должном режиме.

11.3. РЕАЛИЗАЦИЯ

Чтобы обеспечить поддержку этих перезапускаемых задач, нам потребуются реализовать два ключевых компонента: ресурс `Job` (со всеми связанными стандартными методами) и пользовательский метод `Run`, который будет фактически выполнять ожидаемую работу. Начнем с определения самого ресурса `Job`.

11.3.1. Ресурсы `Job`

Фундаментально задачи похожи на любой другой ресурс, с которым мы работаем в API. И аналогично другим ресурсам единственным поистине необходимым полем в них является уникальный идентификатор, который, как мы узнали в главе 6, в идеале должен выбираться сервисом API. Тем не менее основная цель этого типа ресурсов — сохранять множество параметров конфигурации для чего-то, что в противном случае примет форму сообщения запроса для метода, возвращающего LRO.

В качестве примера перенесем идею метода создания бэкапа в наш API чата. Для выполнения этой функциональности по требованию мы бы определили

пользовательский метод, активирующий резервное копирование ресурса `ChatRoom` и всех его сообщений. При этом все параметры, относящиеся к созданию бэкапа (например, куда поместить данные, как их сжать или зашифровать), будут находиться в запросе (листинг 11.1).

Листинг 11.1. Пользовательский метод создания бэкапа по требованию

```
abstract class ChatRoomApi {
    @post("/{id=chatRooms/*}:backup")
    BackupChatRoom(req: BackupChatRoomRequest):
        ➔ Operation<BackupChatRoomResponse,
            BackupChatRoomMetadata>;
}

interface BackupChatRoomRequest {
    id: string;
    destination: string;
    compressionFormat: string;
    encryptionKey: string;
}

interface BackupChatRoomResponse {
    destination: string;
}

interface BackupChatRoomMetadata {
    messagesCounted: number;
    messagesProcessed: number;
    bytesWritten: number;
}
```

Согласно принципу работы асинхронного метода, мы возвращаем LRO со стандартным типом результата интерфейса ответа

Этот идентификатор относится к копируемому ресурсу ChatRoom

Здесь представлены различные параметры конфигурации для операции резервного копирования

Ответ просто указывает, где на внешнем хранилище находится резервная копия

Как мы узнаём в главе 22, данный метод определенно сработает и в результате вернет итоговый адрес резервной копии. Однако, как говорилось в разделе 11.1, это может создать различные проблемы. Возможно, мы захотим, чтобы администраторы могли запускать резервное копирование, но не имели возможности выбрать ключ шифрования или настраивать какие-либо детали операции, помимо ее тайминга. А может, нам понадобится сервис для запуска резервного копирования по расписанию, но с возможностью отдельной настройки его поведения. То есть он будет только запускать сам процесс, оставляя установку конфигурации кому-то другому.

В таком случае можно довольно легко преобразовать наш метод API в перезапускаемый ресурс задачи: просто переместить все, что должно предоставляться в сообщении запроса, и рассматривать эти поля как настраиваемые уже в ресурсе `BackupChatRoomJob` (листинг 11.2).

Листинг 11.2. Ресурс задачи для резервного копирования данных чата

```
interface BackupChatRoomJob {
    id: string;
    chatRoom: string;
    destination: string;
    compressionFormat: string;
}
```

Заметьте, что у этого нового ресурса есть идентификатор и мы меняем имя поля ID, выступающего как ссылка на ресурс, для которого будет создаваться бэкап запроса, на chatRoom

← Все остальные поля сообщения запроса должны также находиться в ресурсе BackupChatRoomJob

Поскольку теперь мы работаем с ресурсом, а не методом API, возвращающим LRO, то нам нужна поддержка типичной коллекции стандартных методов. Подробно мы их разберем в заключительном определении API (см. подраздел 11.3.4), но в табл. 11.1 приведены стандартные методы, которые нужно реализовать, а также их обоснование.

Таблица 11.1. Стандартные методы и обоснование для их использования

Стандартный метод	Обоснование
GetBackupChatRoomJob	Просмотр деталей конфигурации задачи резервного копирования
CreateBackupChatRoomJob	Создание новой перезапускаемой задачи резервного копирования
UpdateBackupChatRoomJob	Обновление существующей перезапускаемой задачи резервного копирования
DeleteBackupChatRoomJob	Удаление перезапускаемой задачи резервного копирования
ListBackupChatRoomJobs	Просмотр списка доступных перезапускаемых задач резервного копирования

Как мы видели в главе 7, бывают сценарии, когда определенные методы можно опустить в сервисе API. В таком случае есть вероятность, что эти ресурсы BackupChatRoomJob можно будет рассматривать как неизменяемые. Иными словами, будет вполне нормально, если после создания они больше никогда не изменятся. Вместо этого пользователи могут удалять существующий ресурс и создавать новый, избегая тем самым потенциальных проблем с конкурентностью (например, когда задача выполняется в то время, как мы ее обновляем). В подобных случаях будет вполне допустимо опускать стандартный метод Update.

Кроме того, если метод по требованию возвращал длительную операцию (поскольку потенциально у него было много работы), эти стандартные методы должны быть мгновенными и синхронными. Дело в том, что мы отобрали у них всю фактическую работу, оставив на их ответственности лишь конфигурацию. В следующем подразделе мы рассмотрим, как активизировать сам ресурс задачи для выполнения работы, настраиваемой с помощью этих стандартных методов.

11.3.2. Пользовательский метод Run

Предположим, что мы создали и настроили ресурс `Job`. На следующем этапе нам предстоит фактически выполнить задачу, чтобы реализовать настроенную работу. Для этого каждый ресурс `Job` должен содержать пользовательский метод `Run`, отвечающий за выполнение внутренней работы (листинг 11.3). Он не должен получать никакие другие параметры (в конце концов, мы хотим обеспечить, чтобы вся соответствующая конфигурация сохранялась как часть ресурса `Job`) и должен возвращать `LRO`, которая в конечном итоге разрешается в нечто наподобие сообщения ответа нашего пользовательского метода, выполняющегося по требованию.

Листинг 11.3. Пользовательский метод `Run` для перезапускаемых задач бэкапа

```
abstract class ChatRoomApi {
  @post("/{id=backupChatRoomJobs/*}:run")
  RunBackupChatRoomJob(req: RunBackupChatRoomJobRequest):
    Operation<RunBackupChatRoomJobResponse,
      RunBackupChatRoomJobMetadata>;
}
```

Пользовательский метод `Run` соответствует стандартам, описанным в главе 9

Поскольку работы предстоит немало, то можно сразу же вернуть `LRO`

```
interface RunBackupChatRoomJobRequest {
  id: string;
}
```

Критически важно, чтобы во время выполнения никакая дополнительная конфигурация не передавалась

```
interface RunBackupChatRoomJobResponse {
  destination: string;
}
```

Эти методы переименованы, но используют те же поля, что и пользовательский метод по требованию

```
interface RunBackupChatRoomJobMetadata {
  messagesCounted: number;
  messagesProcessed: number;
  bytesWritten: number;
}
```

Как видите, при том, что большая часть полей остаются такими же, несколько имен сообщений изменяются, чтобы соответствовать новой структуре. Например, вместо сообщения `BackupChatRoomResponse` мы имеем `RunBackupChatRoomJobResponse`. Важно сказать и то, что `RunBackupChatRoomJobResponse` получает лишь один входной элемент: идентификатор ресурса `Job`, который нужно запустить. Любая другая информация, относящаяся к выполнению самой задачи, должна храниться в ресурсе и никогда не предоставляться во время выполнения.

Кроме того, важно помнить, что хоть в этом примере мы и видим очевидный вывод в форме ответа, сообщающего адрес резервной копии данных (возможно, `s3:///backup2020-01-01.bz2`), многие другие перезапускаемые задачи

не будут следовать этому паттерну. Некоторые могут выполнять работу, изменяя другие ранее существовавшие ресурсы (например, `BatchArchiveChatRoomsJob`) или создавая новые (например, `ImportChatRoomsJob`, что мы увидим в главе 23). Прочие по-прежнему могут давать просто мимолетный результат, предоставляя некий анализ, сохранение которого не предусмотрено. К сожалению, подобные случаи могут создавать проблему в связи с одной важной причиной: непостоянство сохранения ресурсов LRO (см. подраздел 13.3.10). Что же можно сделать?

11.3.3. Ресурсы выполнения задачи

В некоторых случаях результаты перезапускаемой задачи не будут иметь постоянного влияния на API. То есть вызов пользовательского метода `Run` задачи не записывает какие-либо фактические данные в API, за исключением создания ресурсов LRO; никаких обновлений существующих ресурсов, никакого создания новых, никакой записи данных во внешние хранилища, ничего!

Несмотря на то что в этом нет ничего плохого, такое поведение может вызвать существенную зависимость. Неожиданным образом постоянный характер работы, которую выполняет данная задача, оказывается во власти установленной в API политики хранения LRO. А эта политика, как отмечается в подразделе 13.3.10, довольно обширна. И хотя она подталкивает к вечному хранению и удалению каких-либо записей, ничто не мешает API иметь ресурсы `Operation` с установленным сроком действия, которые будут исчезать через определенные промежутки времени.

Поскольку нам может потребоваться такая политика хранения результатов этих задач, которая будет отличаться от политики хранения всех LRO в системе, то у нас возникают два варианта. Первый — изменить политику хранения ресурсов LRO, чтобы она соответствовала ожиданиям относительно этого конкретного типа задачи. Второй — использовать различные сроки действия для разных операций (например, большинство будут действовать в течение 30 дней, а операции для конкретного типа сохранятся навсегда). Первый вариант оказывается разумным, но будет потенциально избыточен. Второй же и вовсе ведет к несогласованности и непредсказуемости, в результате чего наш API станет запутанным и менее удобным. Что же еще можно сделать?

Третий вариант — использовать подколлекцию ресурсов с названием *executions*, которая похожа на ресурсы `Operation`. Каждый ресурс `Execution` будет представлять выход пользовательского метода `Run`, который можно вызывать для этих ресурсов задач. И в отличие от LRO, они могут иметь собственную политику хранения (обычно бесконечного).

Рассмотрим в качестве примера метод анализа, просматривающий комнату чата и формирующий рейтинг, отражающие сложность предложений, общее настроение сообщений и присутствие ругательств. Поскольку нам нужно выполнять его регулярно, эта функциональность должна быть доступна в качестве перезапускаемого экземпляра задачи, но результаты этого анализа, очевидно, не будут являться существующими ресурсами или внешними ресурсами данных. Вместо этого нужно будет определить ресурс `Execution`, представляющий результат одного выполнения задачи анализа (листинг 11.4).

Листинг 11.4. Образец ресурса `Execution` для анализа данных чата

```
interface AnalyzeChatRoomJobExecution {
    id: string;
    job: AnalyzeChatRoomJob;
    sentenceComplexity: number;
    sentiment: number;
    abuseScore: number;
}
```

Поскольку это полноценный ресурс, ему необходим уникальный идентификатор

Вся итоговая информация анализа сохраняется в ресурсе `Execution`

Чтобы точно знать, какая конфигурация использовалась для этого выполнения, мы сохраняем снимок состояния ресурса `AnalyzeChatRoomJob`

Теперь вызов пользовательского метода `Run` будет по-прежнему создавать ресурс `Operation`, чтобы отслеживать работу, выполняемую задачей (листинг 11.5). Но когда она завершится, вместо того, чтобы возвращать временный выходной интерфейс для непосредственного потребления (например, чего-то с полем `sentiment`), API будет создавать ресурс `AnalyzeChatRoomJobExecution`, а LRO — возвращать ссылку на этот ресурс. Это несколько похоже на то, как асинхронный стандартный метод `Create` в конечном счете вернул бы ресурс, созданный при разрешении LRO.

Листинг 11.5. Определение пользовательского метода `Run` с ресурсами `Execution`

```
abstract class ChatRoomApi {
    @post("/analyzeChatRoomJobs")
    CreateAnalyzeChatRoomJob(req: CreateAnalyzeChatRoomJob):
        AnalyzeChatRoomJob;

    @post("/{id=analyzeChatRoomJobs/*}:run")
    RunAnalyzeChatRoomJob(req: RunAnalyzeChatRoomJobRequest):
        Operation<AnalyzeChatRoomJobExecution, RunAnalyzeChatRoomJobMetadata>;
}
```

Стандартный метод `Create` для задач анализа

Заметьте, что мы возвращаем ресурс `Operation` с типом результата ресурса `Execution`

Как видите, пользовательский метод `Run` практически идентичен нашему предыдущему примеру с бэкапом, но результатом оказывается уже не мимолетный интерфейс ответа, а ресурс `Execution`. Чтобы показать это более наглядно, на рис. 11.2 изображен процесс выполнения данной задачи и расписано, что при этом происходит внутри API.

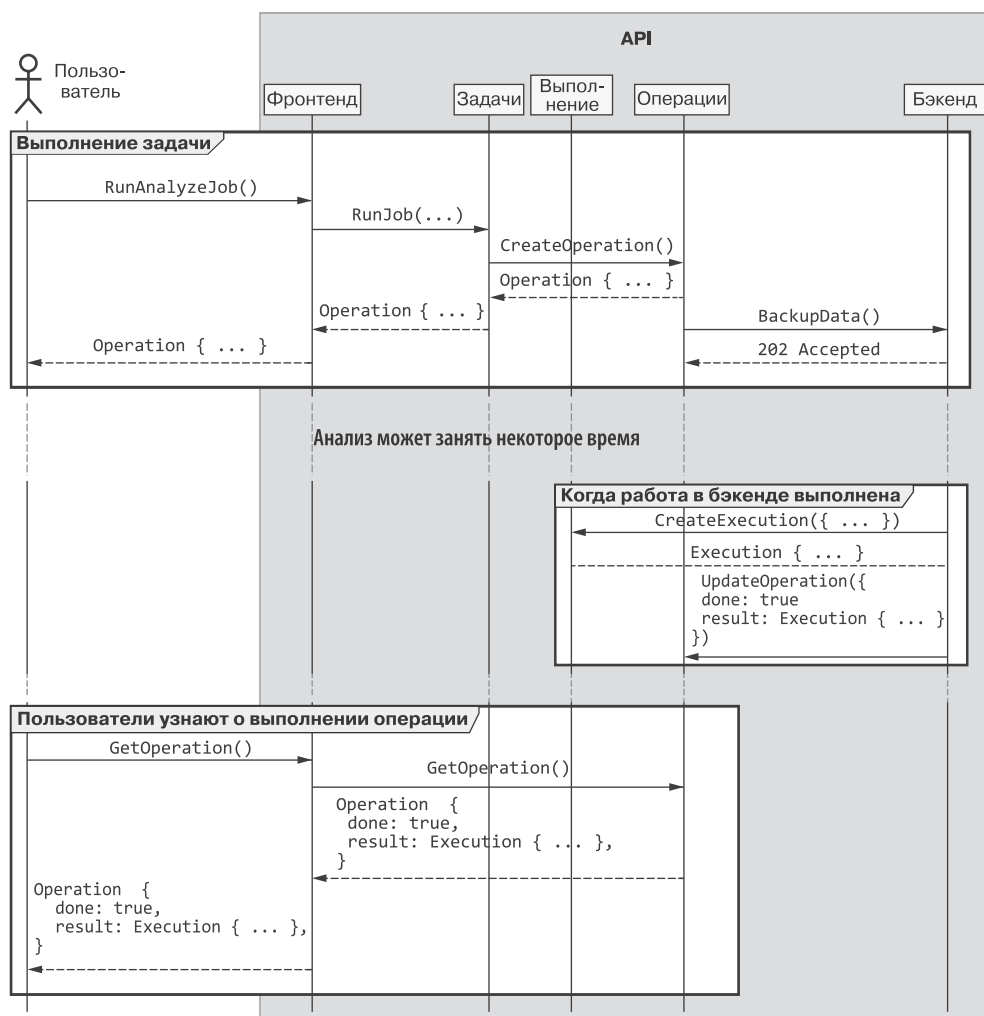


Рис. 11.2. Взаимодействие с ресурсом Job, ведущее к результатам Execution

Наконец, поскольку `executions` являются истинными ресурсами, необходимо реализовать несколько стандартных методов, чтобы сделать эти ресурсы полезными. В данном случае они неизменяемы, поэтому стандартный метод `Update` реализовывать не нужно. При этом создаются они только внутренними процессами (никогда конечными пользователями), в связи с чем реализация метода `Create` тоже не требуется. В табл. 11.2 приведены все стандартные методы, которые потребуются для `execution`, а также обоснование их использования.

Таблица 11.2. Стандартные методы и их обоснование

Стандартный метод	Обоснование
GetAnalyzeChatRoomJobExecution	Просмотр подробностей о конкретном execution
ListAnalyzeChatRoomJobExecutions	Вывод списка произошедших execution
DeleteAnalyzeChatRoomJobExecution	(Необязателен) Удаление конкретного execution

Последний вопрос касается размещения ресурсов: где в иерархии должны располагаться эти `executions`? В отличие от LRO, которые работают с ресурсами со всего API, `execution` создаются под единственный тип задачи и им ограничиваются. Кроме того, поскольку одним из наиболее частых вопросов у нас будет «Какие `execution` были выполнены для этой конкретной задачи?», то имеет смысл ограничить их контекстом этой задачи. То есть поведение, которого мы хотим добиться от `execution`, указывает, что лучше всего эти ресурсы расположить в качестве потомков самих ресурсов задачи.

11.3.4. Итоговое определение API

В качестве итогового определения API в листинге 11.6 приводятся все методы API и интерфейсы для анализа сообщений комнаты чата. В этом случае выходные результаты сохраняются навсегда с помощью ресурса `execution`.

Листинг 11.6. Итоговое определение API

```
abstract class ChatRoomApi {
    @post("/analyzeChatRoomJobs")
    CreateAnalyzeChatRoomJob(req: CreateAnalyzeChatRoomJobRequest):
        AnalyzeChatRoomJob;

    @get("/analyzeChatRoomJobs")
    ListAnalyzeChatRoomJobs(req: ListAnalyzeChatRoomJobsRequest):
        ListAnalyzeChatRoomJobsResponse;

    @get("/{id=analyzeChatRoomJobs/*}")
    GetAnalyzeChatRoomJob(req: GetAnalyzeChatRoomJobRequest):
        AnalyzeChatRoomJob;

    @patch("/{resource.id=analyzeChatRoomJobs/*}")
    UpdateAnalyzeChatRoomJob(req: UpdateAnalyzeChatRoomJobRequest):
        AnalyzeChatRoomJob;

    @post("/{id=analyzeChatRoomJobs/*}:run")
    RunAnalyzeChatRoomJob(req: RunAnalyzeChatRoomJobRequest):
        mOperation<AnalyzeChatRoomJobExecution, RunAnalyzeChatRoomJobMetadata>;
}
```

```

    @get("/{parent=analyzeChatRoomJobs/*/executions}")
    ListAnalyzeChatRoomJobExecutions(req
        ListAnalyzeChatRoomJobExecutionsRequest):
        ListAnalyzeChatRoomJobExecutionsResponse;

    @get("/{id=analyzeChatRoomJobs/*/executions/*}")
    GetAnalyzeChatRoomExecution(req: GetAnalyzeChatRoomExecutionRequest):
        AnalyzeChatRoomExecution;
}

interface AnalyzeChatRoomJob {
    id: string;
    chatRoom: string;
    destination: string;
    compressionFormat: string;
}

interface AnalyzeChatRoomJobExecution {
    id: string;
    job: AnalyzeChatRoomJob;
    sentenceComplexity: number;
    sentiment: number;
    abuseScore: number;
}

interface CreateAnalyzeChatRoomJobRequest {
    resource: AnalyzeChatRoomJob;
}

interface ListAnalyzeChatRoomJobsRequest {
    filter: string;
}

interface ListAnalyzeChatRoomJobsResponse {
    results: AnalyzeChatRoomJob[];
}

interface GetAnalyzeChatRoomJobRequest {
    id: string;
}

interface UpdateAnalyzeChatRoomJobRequest {
    resource: AnalyzeChatRoomJob;
    fieldMask: string;
}

interface RunAnalyzeChatRoomJobRequest {
    id: string;
}

interface RunAnalyzeChatRoomJobMetadata {
    messagesProcessed: number;
}

```



```

    messagesCounted: number;
}

interface ListAnalyzeChatRoomJobExecutionsRequest {
    parent: string;
    filter: string;
}

interface ListAnalyzeChatRoomJobExecutionsResponse {
    results: AnalyzeChatRoomJobExecution[];
}

interface GetAnalyzeChatRoomJobRequest {
    id: string;
}

```

11.4. КОМПРОМИССЫ

Как мы видели, перезапускаемые задачи оказываются очень полезным принципом, но при этом являются лишь одним из возможных решений для набора задач. К примеру, если бы нас интересовали разрешения на настройку, а не выполнение пользовательского метода, то мы бы реализовали более продвинутую систему разрешений, которая бы инспектировала запрос, проверяя, может ли заданный пользователь не только вызывать конкретный метод API, но и делать это с определенным набором параметров. Такой вариант определенно будет подразумевать больше работы для всех участвующих сторон, но при этом окажется более детализированным, чем разделение метода на две части (настройка и выполнение).

Когда дело доходит до ресурсов `execution`, у нас определенно есть альтернативный способ — просто навсегда сохранять ресурсы `Operation` в качестве ссылки на вывод. Единственным недостатком можно назвать необходимость фильтровать список ресурсов `Operation` для извлечения тех из них, которые связаны с интересующим нас ресурсом `Job`, но это однозначно хорошая альтернатива.

11.5. УПРАЖНЕНИЯ

1. Как перезапускаемые задачи позволяют провести различия между разрешением на выполнение действия и разрешением на его настройку?
2. Если выполняющаяся задача ведет к созданию нового ресурса, то что должно в результате получиться: `execution` или новый ресурс?
3. Почему ресурсы `execution` никогда не должны создаваться явно?

РЕЗЮМЕ

- Перезапускаемые задачи — это отличный способ, позволяющий изолировать пользователей, имеющих возможность настраивать задачу, от тех, которые могут выполнять ее.
- Задачи — это ресурсы, которые создаются и настраиваются изначально. Позднее эти задачи можно выполнять с помощью пользовательского метода `Run`.
- Когда задача не воздействует на существующий ресурс и не создает новый, результатом выполнения ресурса `Job` обычно оказывается ресурс `Execution`.
- Ресурсы `execution` подобны ресурсам только для чтения, которые можно выводить списком и извлекать, но не обновлять или создавать явно.

Часть IV

Связи ресурсов

В любом проекте API разные компоненты нужно так или иначе связывать друг с другом. В нескольких следующих главах мы освоим ряд способов связывания ресурсов и научимся создавать правильные методы API, позволяющие пользователям управлять этими связями.

Сначала в главе 12 мы разберем особый тип ресурсов, называемых подресурсами-одиночками, а также узнаем, как с их помощью сохранять изолированные данные с уникальными паттернами доступа. В главе 13 мы изучим, как ресурсы могут ссылаться друг на друга, а также научимся поддерживать ссылочную целостность. В главах 14 и 15 детально рассмотрим два разных способа управления связями между ресурсами по типу «многие ко многим» (используя ассоциирующие ресурсы или пользовательские методы `Add` и `Remove`). Наконец, глава 16 будет посвящена свойству полиморфизма, который дает ресурсам уникальную возможность принимать на себя несколько ролей в зависимости от определенных параметров.

12

Подресурсы-одиночки

В этой главе

- ✓ Что такое подресурс-одиночка.
- ✓ Почему и когда нужно отделять данные в подресурс-одиночку.
- ✓ Как стандартные методы взаимодействуют с подресурсами.
- ✓ Какое место подресурсы-одиночки занимают в иерархии.

Этот паттерн поможет нам изучить способ структурирования взаимосвязанных, но при этом независимых данных, перемещая их из набора свойств ресурса в одиночного потомка этого ресурса. Паттерн обрабатывает сценарии, в которых некая коллекция данных может изменяться независимо от своего родителя, имеет различные требования безопасности или просто может быть слишком большой, чтобы храниться непосредственно в составе ресурса.

12.1. ЗАЧЕМ ЭТО НУЖНО

При построении API иногда возникают ситуации, когда в ресурсе есть компоненты, которые вроде бы принадлежат этому ресурсу как свойства, но по той или иной причине с практической точки зрения не подходят на роль стандартных свойств. Иными словами, согласно практическим рекомендациям по проектированию API, такой компонент определенно должен быть свойством, но следовать

этим рекомендациям в данном случае будет нецелесообразно, так как это лишь затруднит работу пользователей. Например, если есть ресурс, представляющий общий документ, то ему также может потребоваться хранить список доступа (access control list, ACL), определяющий, кто имеет доступ к документу и в каком диапазоне возможностей. Несложно представить, что этот список может стать огромным (сотни или даже тысячи правил доступа) и извлечение всего ACL при каждом просмотре списка документов (например, с помощью метода `ListDocuments`) с точки зрения вычислительных ресурсов и пропускной способности зачастую окажется неоправданно затратным. Более того, в большинстве сценариев подробности ACL нас даже не интересуют, и важным этот список оказывается лишь в конкретных случаях.

Очевидный ответ — разделить подобные компоненты, но тогда возникают два неизбежных вопроса. Во-первых, как определить, что компонент заслуживает отделения? Во-вторых, когда мы решили, что его нужно отделить от основного ресурса, как это правильно реализовать?

Начнем с первого вопроса и рассмотрим некоторые возможные причины, по которым нам может потребоваться выделять компоненты из их родительских ресурсов в отдельные подресурсы. После этого можно будет перейти к реализации такого разделения с помощью паттерна подресурса-одиночки.

12.1.1. Почему нужно использовать подресурс-одиночку?

Есть много возможных причин, обуславливающих отделение компонента от ресурса, которому в обычных условиях он должен принадлежать. Некоторые из этих причин очевидны (например, компонент в несколько раз больше самого ресурса), остальные же более деликатны. Ниже мы разберем несколько распространенных поводов для отделения компонентов от родительских ресурсов.

Размер или сложность

Нередко бывает, что один компонент становится особенно большим в сравнении с самим ресурсом. Если он потенциально станет больше, чем все остальные части ресурса, вместе взятые, то вполне может оказаться разумным отделить этот компонент. Например, если у вас есть API, сохраняющий большие двоичные объекты (например, Amazon S3), то будет странным сохранять сами двоичные данные вместе с метаданными объекта. В этом случае вы их разделите: `GetObject` или `GetObjectMetadata` может возвращать метаданные об объекте, которые хранятся в базе данных, но *не включать* сами двоичные данные явно. Вместо этого они будут извлекаться отдельно с помощью другого RPC, такого как `GetObjectData`.

Безопасность

Зачастую в ресурсе есть элементы, имеющие совсем иные ограничения доступа, нежели сам ресурс. В подобных случаях отдельное хранение такой информации может стать строгим требованием. К примеру, у вас есть API, представляющий данные сотрудников компании, но информация об оплате труда сотрудника, прикрепленная к каждому из них, может являться более закрытой, нежели такие стандартные характеристики, как псевдоним, город проживания и номер телефона. В результате наверняка будет разумнее отделить эту информацию в качестве либо полностью самостоятельного ресурса, либо с помощью описанного здесь паттерна подресурса-одиночки.

Непостоянство

Помимо вопросов размера и безопасности, зачастую возникают сценарии, когда конкретные компоненты ресурса имеют необычные паттерны доступа. В частности, если есть компонент, который обновляется намного чаще других компонентов ресурса, то сохранение их вместе может привести к конфликтам записи, а в самых крайних случаях — даже к потере данных. Представим в качестве примера API совместных поездок, в котором есть ресурс *Driver*. Когда машина движется, нам нужно обновлять информацию о расположении ресурса максимально часто, чтобы отразить последнее местоположение и перемещение водителя. С другой стороны, более общие метаданные, такие как регистрационный знак, могут меняться редко или вообще оставаться постоянными. В результате есть смысл отделить активно меняющуюся информацию о местоположении от других метаданных, чтобы она могла обновляться независимо.

Очевидно, что это не весь список, поскольку есть и много других причин отделять компоненты от ресурсов, но именно эти категории оказываются наиболее частыми сценариями. Теперь, когда мы имеем представление о типичных причинах, пора более внимательно рассмотреть, как можно смоделировать это разделение в API с помощью паттерна подресурса-одиночки.

12.2. ОБЗОР

Поскольку наша цель — переместить компонент ресурса из одного свойства в отдельную сущность, один из вариантов сделать это — спроектировать интерфейс, чтобы компонент стал в некотором смысле гибридом полноценного ресурса и простого атрибута ресурса. По сути, можно создать нечто с характеристиками, соответствующими отчасти полноценному ресурсу, а отчасти — свойству такого ресурса.

Для этого нужно определить принцип подресурса-одиночки, обладающего некоторыми уникальными свойствами и методами взаимодействия (рис. 12.1).

Этот подресурс будет выступать как некий гибрид простого свойства и ресурса, то есть, с одной стороны, мы ограничим предполагаемые возможности ресурса, а с другой — несколько расширим характеристики, обычно ожидаемые от свойств ресурса.

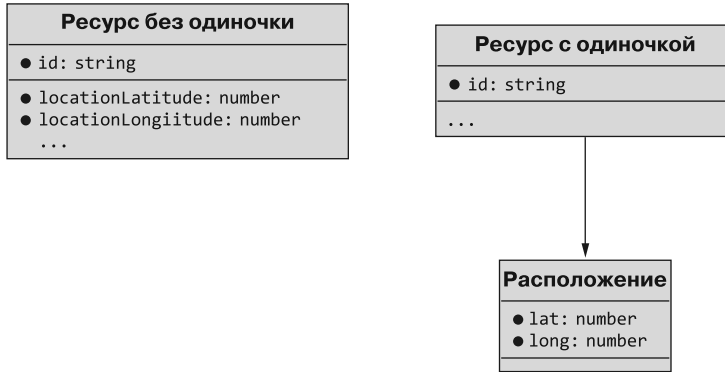


Рис. 12.1. Ресурс, перемещающий свойство местоположения в подресурс-одиночку Location

Теперь, имея общее представление о работе этого паттерна, мы более подробно разберем принцип его действия.

12.3. РЕАЛИЗАЦИЯ

После того как мы решили, что такой-то компонент нужно вынести из простого свойства ресурса в отдельный подресурс-одиночку, остается понять, как именно выглядит интерфейс для взаимодействия с этими подресурсами. Если рассматривать работу с подресурсом через призму стандартных методов, то становится ясно, что одни методы принимают поведение стандартного ресурса, в то время как другие действуют подобно его свойству (табл. 12.1).

Таблица 12.1. Описание поведения подресурса-одиночки

Стандартный метод	Действует как ...
Get<Singleton>	Отдельный ресурс
Update<Singleton>	Отдельный ресурс
Create<Singleton>	Свойство
Delete<Singleton>	Свойство
List<Singletons>	n/a

12.3.1. Стандартные методы

Методы Get и Update

Согласно табл. 12.1, и метод извлечения (**Get**), и метод изменения (**Update**) действуют точно так же, как действовали бы в любом другом ресурсе. Это означает, что подресурсы-одиночки, аналогично стандартным ресурсам, уникально адресуются по идентификаторам и могут изменяться традиционным способом с помощью частичной замены свойств (например, с помощью HTTP-глагола **PATCH**). Для полной ясности в приведенной ниже диаграмме последовательности (рис. 12.2) показано, как можно извлекать и изменять подресурс-одиночку.

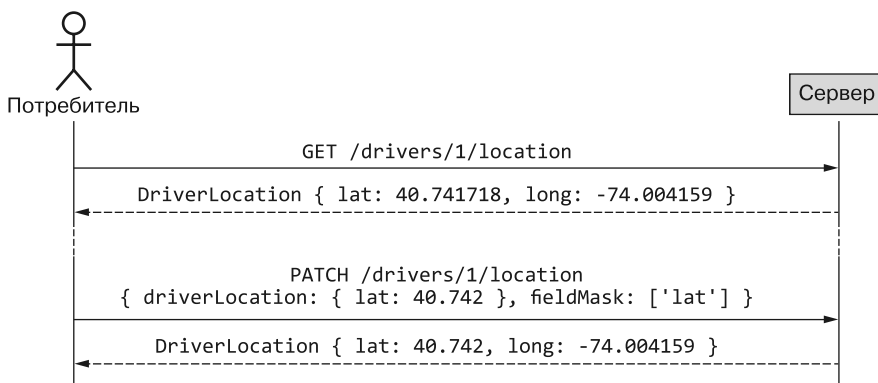


Рис. 12.2. Общая картина взаимодействия с подресурсом-одиночкой

В конечном итоге это означает, что определенные для подресурса методы **Get** и **Update** будут идентичны своим аналогам в обычном ресурсе. Важно помнить, что адресация подресурса выполняется по уникальному идентификатору, а не с помощью ID родительского ресурса. Иными словами, **GetDriverLocation**, как и любой другой метод извлечения, получает уникальный ID (например, **drivers/1/location**), а не родительский ID, как мы видели в методах **List**.

Метод Create

Продолжая рассматривать табл. 12.1, мы видим, что действие метода **Create** больше напоминает свойство. Это значит, что подресурсы-одиночки существуют просто как следствие существования их родителя. То есть так же, как нет необходимости в явном создании свойства ресурса (например, свойства **Driver.licensePlate**), нет нужды и в явном создании подресурса **DriverLocation**. Вместо этого он должен рождаться одновременно с родительским ресурсом.

Из этого также следует, что взаимодействие с подресурсом-одиночкой не требует явной инициализации. Это не значит, что никогда не возникнет потребность

обновлять свойства подресурса после того, как будет создан его родитель. Имеется в виду, что не должно быть специального метода инициализации, выполнения которого требовало бы начало обновления самого подресурса. Образец взаимодействия с родительским ресурсом, создающим подресурс-одиночку, показан на рис. 12.3.

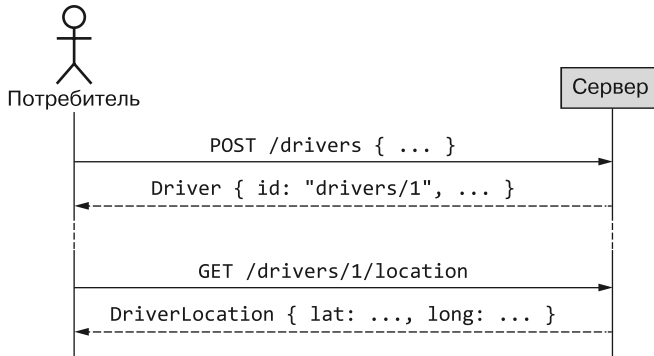


Рис. 12.3. Создание родительского ресурса ведет к созданию подресурса-одиночки

И здесь возникает очевидный вопрос: как инициализировать данные, хранящиеся в подресурсе-одиночке, одновременно с созданием родительского ресурса? Иными словами, есть ли способ, позволяющий создавать ресурс `Driver` и одновременно с этим инициализировать информацию `DriverLocation`? Ответ будет простым: нет.

Весь смысл отделения чего-либо в подресурс состоит в том, чтобы изолировать этот компонент от родительского ресурса. Это может делаться в ответ на любой из описанных в разделе 12.1 случаев (например, из-за волатильности), но последствием такого разделения становится отделение подресурса от родителя. Однако, несмотря на то что появление подресурса привязано к созданию родителя, любые изменения хранящейся в этом подресурсе информации нужно будет выполнять отдельно, поскольку эти ресурсы являются полностью отдельными. То есть создать `Driver` и установить информацию в подресурсе `DriverLocation` с помощью одного метода `CreateDriver` возможно не более, чем создать два отдельных ресурса `Driver` с помощью одного вызова `CreateDriver`. Это говорит о том, как важно выбирать разумные предустановки для подресурсов-одиночек, чтобы они могли быть полезны даже при неявной инициализации.

Метод Delete

Аналогично тому как создание родительского ресурса вызывает создание подресурса-одиночки, удаление этого родительского ресурса должно приводить к удалению подресурса. То есть подобно тому как реляционные базы данных

позволяют каскадировать удаления на основе ограничений внешнего ключа, удаление родительских ресурсов каскадируется вниз на все прикрепленные к ним подресурсы-одиночки. По сути, это означает, что в случае удаления подресурсы-одиночки действуют больше как свойства, нежели как отдельные ресурсы. Более наглядно это демонстрируется в примере каскадного удаления, показанного на рис. 12.4.

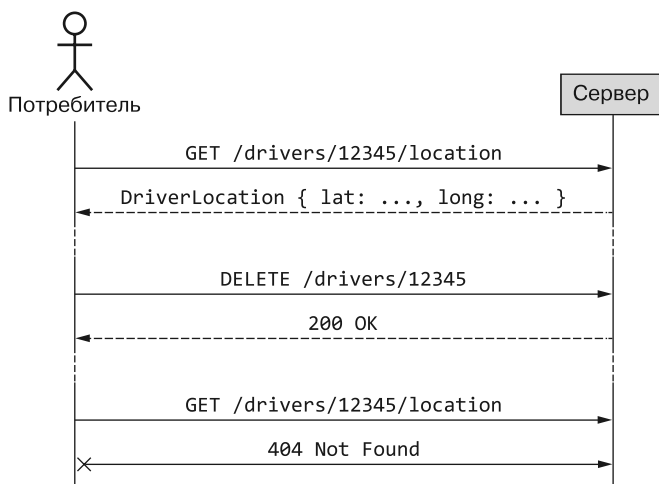


Рис. 12.4. Подресурс-одиночка удаляется вместе с родительским ресурсом

ПРЕДУПРЕЖДЕНИЕ

Если каскадирование удаления окажется неожиданным или не будет соответствовать ожиданиям типичного потребителя, то зачастую это признак того, что применение паттерна подресурса-одиночки неуместно.

Это объяснение охватывает стандартную операцию удаления, но порождает и важные вопросы относительно нестандартного удаления, такие как: «Что делать, когда удаление родителя не является мгновенным и задействует LRO (см. главу 10) или опирается на паттерн мягкого удаления (см. раздел 6.1)?» Поскольку удаление заставляет подресурс действовать в первую очередь как свойство, а не отдельный ресурс, то можно передать это с помощью нестандартного поведения. То есть родительский ресурс, отмеченный для удаления, будет продолжать существовать, пока не перестанет существовать процесс удаления этого родителя. Аналогичным образом, если родительский ресурс может удаляться какое-то время, в связи с чем использует LRO для отслеживания своего прогресса, то подресурс должен удаляться только по завершении операции удаления родителя.

12.3.2. Сброс

В некоторых случаях возникает необходимость вернуть сам подресурс-одиночку к набору подходящих предустановленных значений, в частности таких, которые были установлены при изначальном создании родительского ресурса. В этом случае API должен поддерживать метод сброса, который будет автоматически устанавливать эти значения в предустановленное состояние так, будто подресурс-одиночка только что появился в результате создания родителя. Пример взаимодействия с целью сброса подресурса показан на рис. 12.5.

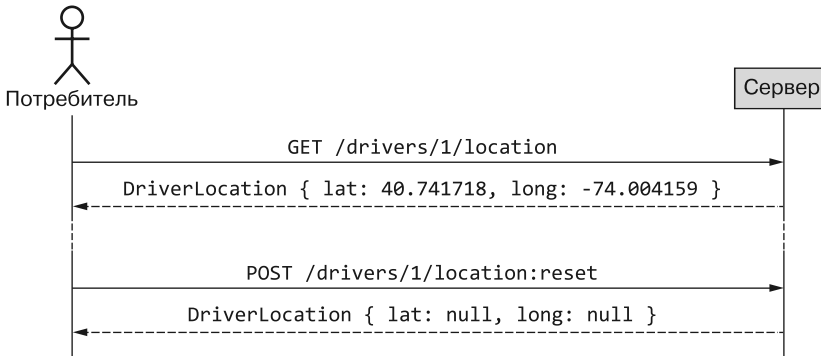


Рис. 12.5. В случае реализации сброс подресурса-одиночки должен восстанавливать подходящие значения по умолчанию

12.3.3. Иерархия

Теперь, когда мы понимаем, как взаимодействовать с подресурсами-одиночками, важно прояснить несколько деталей об иерархических связях между этими одиночками и их родителями.

Необходимые родители

Как и предполагает его имя, подресурс должен подчиняться некоему родительскому ресурсу. Это означает, что подресурс-одиночка должен всегда иметь родительский ресурс и никогда не прикрепляться на корневом уровне иерархии API. Причина в том, что глобальные одиночки функционально равнозначны общей глобальной блокировке, представляя один экземпляр ресурса, совместно используемый всеми потенциальными потребителями API. Несложно представить, что совместное использование глобального состояния всеми потенциальными потребителями API приводит к серьезным конфликтам при записи, сосредотачивая весь соответствующий трафик на одном ресурсе.

Полноценные ресурсы-родители

Еще один типичный пограничный случай проявляется в том, могут ли у под-ресурсов-одиночек быть собственные подресурсы-одиночки. Коротко говоря, если одиночка выступает родителем для других одиночек, то их вместо этого нужно делать его братьями. То есть в том, что один подресурс-одиночка будет выступать родителем другого, нет никакой пользы.

12.3.4. Итоговое определение API

Учитывая все детали этой реализации, можно определить API для нашего образца сервиса, в котором есть ресурсы `Driver`, содержащие подресурсы-одиночки `DriverLocation`. Это заключительное определение API показано в листинге 12.1.

Листинг 12.1. Итоговое определение API с помощью паттерна подресурса-одиночки

```
abstract class RideSharingApi {
    static version = "v1";
    static title = "Ride Sharing API";

    @get("/drivers")
    ListDrivers(req: ListDriversRequest): ListDriversResponse;

    @post("/drivers")
    CreateDriver(req: CreateDriverRequest): Driver;

    @get("/{id=drivers/*}")
    GetDriver(req: GetDriverRequest): Driver;

    @patch("/{resource.id=drivers/*}")
    UpdateDriver(req: UpdateDriverRequest): Driver;

    @delete("/{id=drivers/*}")
    DeleteDriver(req: DeleteDriverRequest): void;

    @get("/{id=drivers/*/location}")
    GetDriverLocation(req: GetDriverLocationRequest): DriverLocation;

    @patch("/{resource.id=drivers/*/location}")
    UpdateDriverLocation(req: UpdateDriverLocationRequest): DriverLocation;
}

interface Driver {
    id: string;
    name: string;
    licensePlate: string;
}
```

Обратите внимание: местоположение водителя не устанавливается как свойство, а отделяется в подресурс

```

interface DriverLocation {
    id: string;
    lat: number;
    long: number;
    updateTime: Date;
}

interface GetDriverRequest {
    id: string;
}

interface UpdateDriverRequest {
    resource: Driver;
    fieldMask: FieldMask;
}

interface ListDriversRequest {
    maxPageSize: number;
    pageToken: string;
}

interface ListDriversResponse {
    results: Driver[];
    nextPageToken: string;
}

interface CreateDriver {
    driver: Driver;
}

interface DeleteDriverRequest {
    id: string;
}

interface GetDriverLocationRequest {
    id: string;
}

interface UpdateDriverLocationRequest {
    resource: DriverLocation;
    fieldMask: FieldMask;
}

```

← Помните, что уникальный идентификатор ресурса `DriverLocation` полностью зависит от идентификатора ресурса `Driver`. Иными словами, `Driver` с названием `drivers/1` будет содержать `DriverLocation` с названием `drivers/1/location`

12.4. КОМПРОМИССЫ

Как мы узнали в ходе реализации этого паттерна, при его использовании неизбежны несколько компромиссов, в частности потеря некоторых возможностей. Далее мы более подробно разберем эту тему, начав с атомарности родительского ресурса и подресурса.

12.4.1. Атомарность выполнения

При использовании этого паттерна мы видели, что подресурс обладал некоторыми характеристиками свойства и ресурса одновременно. В частности, создание действовало как свойство (мы никогда не создаем подресурс-одиночку; он просто существует), а другие его черты больше напоминали фактический ресурс (мы обновляем ресурс непосредственно, а не через его родителя). Один из побочных эффектов всего этого заключается в отсутствии возможности взаимодействовать и с ресурсом, и с подресурсом. Иными словами, у нас нет способа, позволяющего атомарно оперировать и с родительским ресурсом, и с подресурсом одновременно.

И хотя это определенно оказывается ограничением (в конце концов, когда информация `DriverLocation` была свойством ресурса `Driver`, можно было атомарно создавать этот ресурс с конкретным местоположением), вызвано оно самим дизайном. Основная цель отделения подресурса от его родителя — изолирование конкретного набора информации, возможно, из-за его большого размера или ввиду специфичных требований безопасности. Иными словами, эти компромиссы возникают скорее как особенность, несмотря на то, что в результате некоторые типичные действия, легко доступные до этого момента, оказываются невозможными.

12.4.2. Ровно один подресурс

Еще один важный компромисс при использовании подресурсов-одиночек заключается в том, что применять можно лишь один такой подресурс. То есть в отличие от традиционной подколлекции, которая может одновременно содержать множество подресурсов, при использовании этого паттерна у нас может быть лишь один экземпляр конкретного подресурса.

12.5. УПРАЖНЕНИЯ

1. Насколько должны разрастись данные атрибута, чтобы возник смысл отделить его в отдельный подресурс-одиночку? Что влияет на принятие этого решения?
2. Почему подресурсы-одиночки поддерживают только два стандартных метода, `Get` и `Update`?
3. Почему подресурсы-одиночки поддерживают пользовательский метод `Reset`, а не переделывают под ту же задачу стандартный метод `Delete`?
4. Почему подресурсы-одиночки не могут быть потомками других подресурсов-одиночек?

РЕЗЮМЕ

- Подресурсы-одиночки представляют гибрид свойств и ресурсов, сохраняя данные, присущие ресурсу, но в отдельном изолированном месте.
- Данные можно отделять от ресурса в подресурс-одиночку по разным причинам, таким как размер, сложность, специфичные требования к безопасности или различие в паттернах доступа и итоговая волатильность.
- Подресурсы-одиночки поддерживают стандартные методы `Get` и `Update`, но при этом никогда не должны создаваться или удаляться, а значит, и поддерживать стандартные методы `Create` и `Delete`.
- Подресурсы-одиночки обычно также поддерживают пользовательский метод `Reset`, который возвращает атрибуты ресурса в их предустановленный вид.
- Подресурсы-одиночки необходимо крепить к родительскому ресурсу, который при этом не должен являться другим подресурсом-одиночкой.

13

Перекрестные ссылки

В этой главе

- ✓ Как именовать поля, указывая ссылки на другие ресурсы.
- ✓ Как ссылочные поля должны обрабатывать проблемы ссылочной целостности.
- ✓ Когда и почему поля должны хранить копию данных ресурса.

В любом API со множеством типов ресурсов наверняка возникнет потребность организовать между ними перекрестные ссылки. И хотя подобная задача может показаться тривиальной, многие детали такого поведения интерпретируются по-разному, в результате чего может возникнуть рассогласованность. Цель текущего паттерна — прояснить, как нужно определять такие ссылки и, что самое главное, как они должны действовать.

13.1. ЗАЧЕМ ЭТО НУЖНО

Ресурсы редко существуют в замкнутом пространстве, в связи с чем им нужен способ, позволяющий ссылаться друг на друга. Эти ссылки могут быть локальными (например, между ресурсами в одном API), глобальными (например, между ресурсами в разных частях интернета), а также чем-то средним (например, между ресурсами в разных API, предлагаемых одним провайдером) (рис. 13.1).

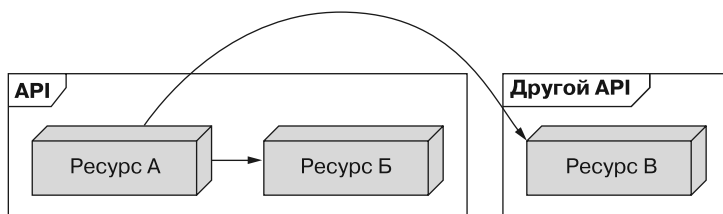


Рис. 13.1. Ресурсы могут указывать на другие ресурсы внутри одного API или во внешних API

И хотя, казалось бы, можно просто ссылаться на эти ресурсы по уникальному идентификатору, реализация должна учитывать еще и поведенческие аспекты. В связи с этим нам нужно определить руководства по созданию ссылок, а также прояснить лежащие в их основе паттерны поведения. Например, должны ли вы иметь возможность удалять ресурс, на который ведет ссылка, или такое действие необходимо запретить? Если же оно допускается, то должна ли выполняться для соответствующего значения указателя постобработка (например, сброс на `null`), учитывая тот факт, что ресурс, на который оно указывало, исчез? Или такой недействительный указатель можно оставить?

Если коротко, то, несмотря на простоту общего принципа организации ссылок между ресурсами, в деталях этот процесс может оказываться довольно сложным.

13.2. ОБЗОР

Думаю, вас не удивит, что паттерн перекрестных ссылок опирается на ссылочное свойство в одном ресурсе, указывающее на другой, отражая при этом тип ресурса с помощью названия поля. Эта ссылка выражается с использованием уникального идентификатора в виде строкового значения (см. главу 6), поскольку это позволяет с помощью стандартного URI ссылаться и на ресурсы одного API, и на ресурсы разных API одного провайдера, и на полностью отдельные ресурсы, находящиеся где-то в интернете.

Кроме того, такая ссылка полностью отделена от ресурса, на который указывает. Это обеспечивает максимальную гибкость при управлении ресурсами, предотвращает блокировку ресурсов циклическими ссылками и делает возможным масштабирование в случаях, когда на один ресурс ссылаются тысячи других. Тем не менее это также означает, что потребители должны быть готовы к возможному наличию устаревших и потенциально недействительных указателей, если ресурс, на который они ведут, перемещен или удален.

13.3. РЕАЛИЗАЦИЯ

У этого паттерна есть несколько важных аспектов. В текущем разделе мы подробно разберем ряд важных вопросов относительно того, как один ресурс должен ссылаться на другой. Начнем с рассмотрения примера ссылки и выбора правильного имени для поля, указывающего на другой ресурс.

13.3.1. Имя ссылочного поля

Несмотря на то что уникальный идентификатор может выражать тип и значение целевого ресурса, куда безопаснее будет использовать для этого имя поля. Например, если у нас есть ресурс `Book`, ссылающийся на ресурс `Author`, то ссылочное поле нужно назвать `authorId`, тем самым показав, что это уникальный ID ресурса `Author` (листинг 13.1).

Листинг 13.1. Определение ресурсов `Author` и `Book`

```
interface Book {
    id: string;
    authorId: string;
    title: string;
    // ...
}
```

Мы сохраняем ссылку на ресурс `Author`, используя строковое поле, содержащее уникальный идентификатор автора

Лишние поля опущены в целях сокращения

```
interface Author {
    id: string;
    name: string;
}
```

Уникальный идентификатор ресурса `Author`. Это значение отражается в поле `authorId`

В большинстве случаев целевой ресурс будет иметь статичный тип, то есть мы можем указывать на разные ресурсы `Author`, но типом любого такого ресурса всегда будет автор. Хотя в некоторых случаях тип все же может варьироваться, в связи с чем мы будем говорить, что ссылаемся на *динамический тип ресурса*.

Например, нам может потребоваться сохранять историю изменений авторов и книг. Это означает, что нам нужна возможность указывать не только на множество разных ресурсов, но и на их разные типы. Для этого мы используем дополнительное поле `type`, указывающее тип целевого ресурса (листинг 13.2).

Листинг 13.2. Образец ресурса `ChangeLogEntry` с динамическими типами ресурсов

```
interface ChangeLogEntry {
    id: string;
    targetId: string;
    targetType: string;
    // ...
}
```

Уникальный идентификатор целевого ресурса (например, ID автора)

Тип целевого ресурса (например, `api.mycompany.com/Author`)

Здесь сохраняются дополнительные детали о данном изменении в целевом ресурсе

Теперь, научившись определять ссылочное поле, мы более глубоко разберем важный аспект поведения — *целостность данных*.

13.3.2. Целостность данных

Поскольку в качестве указателей между ресурсами мы используем простые строковые значения, нам необходимо задуматься о свободе, предоставляемой этим типом данных, а именно об отсутствии проверки на уровне типов. Представьте, к примеру, следующий порядок событий, связанных с ресурсами книги и автора (листинг 13.3).

Листинг 13.3. Удаление автора, на которого ссылается книга

```
author = CreateAuthor({ name: "Michelle Obama" });
book = CreateBook({
  title: "Becoming",
  authorId: author.id
});
DeleteAuthor({ id: author.id });
```

Здесь мы создаем автора, а затем книгу, ссылающуюся на него

После этого мы удаляем ресурс Author из API

В этот момент у нас образуется *висячий указатель*, когда ресурс книги указывает на уже не существующего автора. Что должно происходить в таком сценарии? Ниже описаны несколько вариантов:

- 1) можно запретить удаление этого автора и выбрасывать ошибку;
- 2) можно разрешить удаление автора, но устанавливать поле `authorId` на нулевое значение;
- 3) можно разрешить удаление автора и обрабатывать недействительный указатель в среде выполнения.

Если мы предпочтем запретить удаление этого автора (или, раз уж на то пошло, любого автора, имеющего зарегистрированные в системе книги), то рискуем усложнить работу потребителей API, которым в таком случае, чтобы удалить одного автора, может понадобиться удалить сотни или тысячи связанных с ним книг. Более того, если у нас будут два ресурса, указывающих друг на друга, мы никогда не сможем удалить ни один из них. Например, представим, что каждый ресурс `Author` содержит дополнительное поле для его любимой книги и самолюбивый автор указал в ее качестве собственную, как это показано на рис. 13.2. В таком случае мы никогда не сможем удалить эту книгу, поскольку автор указал ее в качестве любимой. Мы никогда не удалим и самого этого автора, так как будут оставаться книги, указывающие на него.

Если же разрешить удаление путем сброса указателя на автора в нулевое значение, то мы избежим этой проблемы циклической ссылки. Однако в результате одного вызова нам может потребоваться обновлять потенциально огромное

количество ресурсов. Представьте одного особенно продуктивного автора, написавшего сотни или даже тысячи книг. В этом случае удаление одного ресурса автора будет подразумевать обновление множества тысяч записей. И данный процесс не только займет много времени, но и может оказаться таким, который система не способна выполнить атомарно. А если не сделать это атомарно, то возникает риск появления висячих указателей, хотя основной целью изначально является избавление от любых недействительных ссылок.

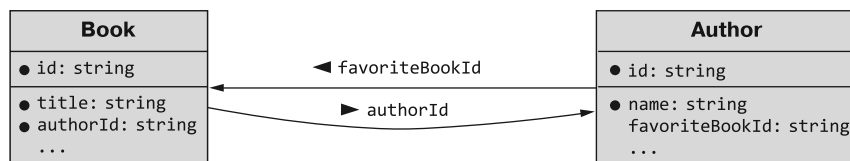


Рис. 13.2. Между двумя ресурсами могут быть циклические ссылки

Ввиду этих проблем нам остается третий вариант: просто сообщить потребителям API о том, что некоторые поля могут оказаться недействительными или указывать на удаленные ресурсы. Конечно, такой подход может оказаться неудобным, но зато предоставит наиболее последовательное поведение с прозрачными и понятными ожиданиями: ссылки нужно проверять. При этом он не нарушает какие-либо ограничения, накладываемые стандартными методами, такие как атомарность операции удаления и отсутствие побочных эффектов.

Теперь, когда мы разобрались с тем, как должны действовать ссылки в контексте изменений внутренних данных, можно перейти к распространенному вопросу о том, что же лучше использовать: ссылки или кэшированные значения.

13.3.3. Значение или ссылка

До сих пор мы работали, исходя из предположения, что в качестве ссылок между ресурсами API должны использоваться строковые идентификаторы, но это не единственный вариант. Наоборот, когда мы сталкиваемся с необходимостью сослаться на различные ресурсы в разных частях API, первым встает вопрос о том, что нужно сохранять: простой указатель на другой ресурс или же кэшированную копию самого ресурса. Во многих языках программирования это различие проявляется в виде *передачи по ссылке* или *передачи по значению*, где в первом случае мы передаем указатель на что-то в памяти, а во втором — копию рассматриваемых данных.

Отличительным фактором здесь является то, что в случае со ссылками всегда можно быть уверенным в свежести и актуальности имеющихся данных. Если же в процесс вступают копии, то возникает вопрос: не изменялись ли имеющиеся

у нас данные с момента их последнего извлечения? С другой стороны, сохранение ссылки на ресурс означает следующее: чтобы извлечь данные по этому адресу, любой пользователь API должен выполнить второй запрос. Когда мы сравниваем такой вариант с непосредственным наличием копии данных, одно только это удобство уже выглядит заманчиво. Например, чтобы извлечь имя заданного автора книги, нам потребуется выполнить два отдельных вызова API (листинг 13.4).

Листинг 13.4. Извлечение имени автора книги с помощью двух вызовов API

```
book = GetBook({
    "id: "books/1234"
});
authorName = GetAuthor({ id: book.authorId }).name;
```

Здесь мы извлекаем книгу по (случайному) уникальному идентификатору

Чтобы увидеть имя автора, нам необходим отдельный вызов API

Эти два вызова, будучи простыми, все равно удваивают количество работы, которую требуется проделать в альтернативном варианте, где при извлечении книги в ответе мы возвращаем весь ресурс `Author` (листинг 13.5).

Листинг 13.5. Переопределение ресурсов `Book` и `Author` для использования значений

```
interface Book {
    id: string;
    title: string;
    author: Author;
}

interface Author {
    id: string;
    name: string;
}
```

Здесь у нас хранится значение всего ресурса `Author`, а не просто ссылка на автора

И хотя теперь мы имеем непосредственный доступ к информации об авторе книги, не требующий выполнения второго вызова API, нам нужно решить, как заполнить эти данные на стороне сервера. Можно либо извлекать информацию об авторе вместе с информацией о книге из базы данных во время каждого запроса `GetBook` (например, использовать SQL-инструкцию `JOIN`), либо сохранить кэшированную копию информации об авторе внутри ресурса книги. Первый вариант увеличит нагрузку на базу данных, но гарантирует свежесть информации на момент запроса. Второй исключит проблему с нагрузкой на базу данных, но внесет сложности с согласованностью данных, вынудив нас придумывать стратегию, позволяющую поддерживать эту кэшированную информацию об авторе в актуальном виде (листинг 13.6).

Более того, теперь нам нужно разобраться с тем фактом, что при каждом увеличении ресурса `Author` будет увеличиваться и ответ на запрос `GetBook`. Если следовать этой стратегии в отношении других аспектов книги (например, если

мы начнем сохранять информацию об издателе книги), то размер может продолжить расти, потенциально выходя из-под контроля.

Листинг 13.6. Пример фрагмента кода с аннотацией

```
author = CreateAuthor({ name: "Michelle Robinson" });
book = CreateBook({
  title: "Becoming",
  author: { id: author.id }
});
UpdateBook({
  id: book.id,
  author: { name: "Michelle Obama" }
});
```

← Для создания книги требуется необычный синтаксис, когда мы устанавливаем только поле ID автора, а остальные разрешаем заполнять автоматически

← Это допустимо? Можно ли обновить имя автора путем обновления книги?

Наконец, такая схема может внести путаницу, когда от потребителей ожидается внесение изменений в ресурс **Book**. Могут ли они обновлять имя автора, обновляя сам ресурс **Book**? Как они реализуют обновление автора?

В результате всех этих потенциальных сложностей и каверзных вопросов обычно лучше использовать одни только ссылки, после чего задействовать нечто вроде GraphQL для их совмещения. Это позволит потребителям API выполнять единственный запрос и получать всю (именно всю) желаемую информацию о заданном ресурсе и тех ресурсах, на которые он ссылается, избегая раздувания и устраняя необходимость в кэшировании информации.

13.3.4. Итоговое определение API

Итак, мы видим заключительный набор интерфейсов, демонстрирующий, как правильно организовывать ссылки между ресурсами (листинг 13.7).

Листинг 13.7. Итоговое определение API

```
interface Book {
  id: string;
  authorId: string;
  title: string;
}

interface Author {
  id: string;
  name: string;
}

interface ChangeLogEntry {
  id: string;
  targetId: string;
  targetType: string;
  description: string;
}
```

13.4. КОМПРОМИССЫ

Как мы уже знаем, основной компромисс в случае использования ссылок таков: мы либо совершаем несколько вызовов API для получения связанной информации, либо задействуем GraphQL для получения всех интересующих нас данных.

13.5. УПРАЖНЕНИЯ

1. Когда есть смысл сохранять копию данных стороннего ресурса вместо ссылки?
2. Почему поддержание ссылочной целостности в системе API является неоправданным?
3. API гарантирует обеспечение актуальности ссылок по всей системе. Позднее по мере ее роста интерфейс решает отказаться от этого правила. Почему это решение станет опасным?

РЕЗЮМЕ

- Поля, сохраняющие ссылку, обычно должны быть строковыми (независимо от типа идентификатора) и заканчиваться суффиксом ID (например, `authorID`). При этом поля, содержащие копию данных стороннего ресурса, должны опускать этот суффикс.
- Как правило, ссылки не должны поддерживаться дольше срока жизни самого ресурса. В случае его удаления они могут стать недействительными.
- Данные целевого ресурса можно сохранять встроенными в ссылающемся ресурсе. Однако нужно учитывать, что они могут устареть, поскольку со временем целевые данные изменяются.

14

Ассоциирующие ресурсы

В этой главе

- ✓ Как обрабатывать связи в формате «многие ко многим» с помощью ассоциирующих ресурсов.
- ✓ Как выбирать имя для ассоциирующих ресурсов.
- ✓ Как поддерживать стандартные методы для ассоциирующих ресурсов.
- ✓ Каковы стратегии обработки ссылочной целостности.

В этой главе мы изучим паттерн моделирования связей «многие ко многим» между двумя ресурсами с помощью отдельного ассоциирующего ресурса. Данный паттерн позволяет потребителям явно обращаться к отдельной связи между двумя ресурсами, а также сохранять и управлять ее дополнительными метаданными.

14.1. ЗАЧЕМ ЭТО НУЖНО

Чаще всего определяемые нами связи между ресурсами API будут просты и очевидны, поскольку по своей природе они склонны к однонаправленности, выступая как указатели или ссылки, исходящие из одного ресурса к другому (например, база данных 1 принадлежит проекту 1), как показано на рис. 14.1. Когда

один тип ресурса ссылается на другой таким образом, мы говорим, что между ними имеется связь «*один ко многим*». И хотя подобный тип связи обычно при разработке API не является проблемой (зачастую требуя лишь правильной ссылки на другой ресурс), бывают случаи, когда необходимые связи оказываются более сложными, вызывая трудности при выражении в простом API.

Один из наиболее распространенных типов таких усложненных связей — «*многие ко многим*», когда между типами ресурсов возникает двусторонняя связь. Представим в качестве примера сценарий, в котором нужно отслеживать людей (пользователей) и различные группы, к которым они принадлежат. Эти пользователи могут являться членами множества разных групп, которые, в свою очередь, сами состоят из различных пользователей. Кроме того, здесь может присутствовать некая особая информация о самой связи, которую нам нужно отслеживать. Например, мы хотим сохранять точное время вступления пользователя в группу или конкретную роль, которую он может играть в этой группе.

Несмотря на то что такие связи «многие ко многим» в большинстве реляционных баз данных имеют стандартную каноническую репрезентацию (опираясь на *промежуточную таблицу*, или *таблицу ссылок*, которая содержит строки, отслеживающие связь между экземплярами двух типов), не всегда понятно, как этот принцип нужно выразить в API. Цель данного паттерна — подчеркнуть конкретный способ, с помощью которого можно представить строки в этих *промежуточных таблицах* как отдельные ресурсы, выражающие связь между двумя ресурсами в API.

14.2. ОБЗОР

Как уже говорилось, наиболее типичным способом сохранения связей «многие ко многим», в частности в реляционных базах данных, является использование *промежуточных таблиц* со строками, представляющими связи между двумя ресурсами. Например, у нас может быть таблица ресурсов *User*, еще одна таблица ресурсов *Group* и третья, сопоставляющая два этих ресурса (например, ресурсы *UserGroup*), как показано на рис. 14.2. В результате один из очевидных способов, позволяющих выразить эти ресурсы и их связи через API, — просто представить все таблицы как ресурсы. В этом сценарии схема базы данных по факту в точности совпадает с поверхностью API, поэтому мы определим API,

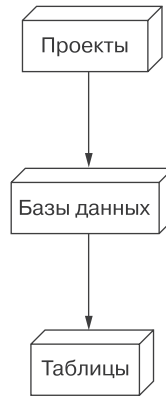


Рис. 14.1. Однонаправленные, или иерархические, связи между ресурсами

состоящий из трех ресурсов: User, Group и UserGroup (сопоставляющий или ассоциирующий ресурс).

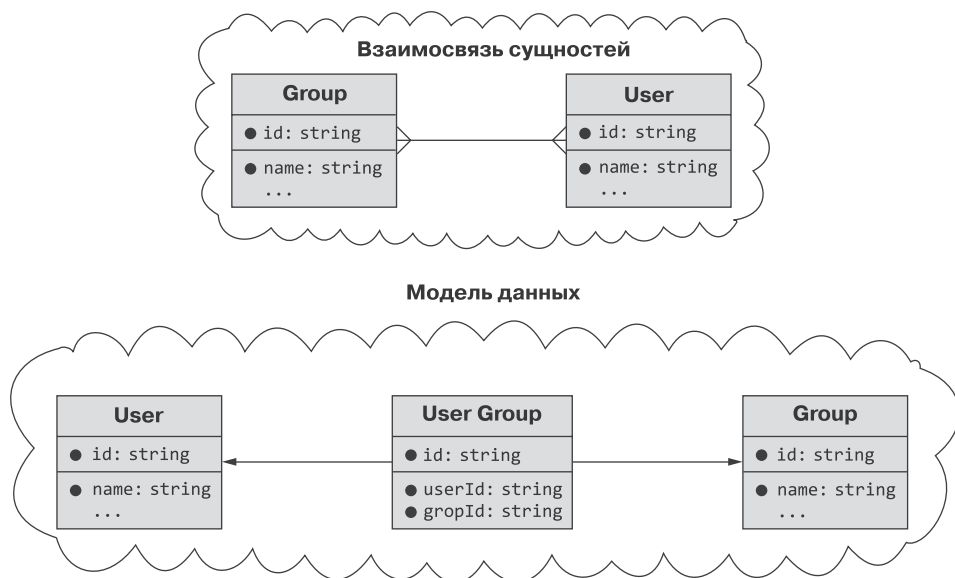


Рис. 14.2. Использование промежуточной таблицы для сохранения связей «многие ко многим»

Возникает логичный вопрос: как все это работает? На верхнем уровне можно переложить всю основную нагрузку на ассоциирующий ресурс. Поскольку связь выражается с помощью фактического ресурса, он создается и удаляется подобно любому другому. В этом примере создание ресурса **UserGroup** представляет вступающего в группу пользователя, а его удаление — покидающего. Кроме того, всегда можно просмотреть конкретную связь, если извлечь по идентификатору (например, `getUserGroup`). При этом также можно выводить список всех связей, извлекая коллекцию на основе ее родителя (например, `listUserGroups`). Если же мы хотим увидеть лишь конкретные связи (например, принадлежащие одной группе или одному пользователю), то можно применить фильтр.

Наконец, наиболее значимым аспектом использования ассоциирующего ресурса является возможность сохранять метаданные о самой связи. Например, вместо простого сохранения информации о присоединении пользователя к такой-то группе можно сохранить дополнительную информацию о времени его присоединения или роли, которую он будет играть в этой группе (скажем, администратора). Это основная причина, по которой есть смысл извлекать ресурс **UserGroup**: нам может потребоваться узнать больше подробностей о самой связи.

14.2.1. Ассоциирующие методы-псевдонимы

Иногда ассоциирующий ресурс ощущается принадлежащим одному или обоим связанным ресурсам. Иными словами, вполне естественно рассматривать группу как содержащую список пользователей. В том же смысле кажется логичным воспринимать пользователя как участника множества групп. Более того, в обоих случаях будет вполне естественным задавать оба этих вопроса относительно нашего API, например: «Участником каких групп является этот пользователь?» и «Какие пользователи состоят в этой группе?»

И хотя мы сказали, что можем легко делать запрос на вывод списка связей с помощью подходящего фильтра, зачастую в ресурсно-ориентированных API потребителям нужна возможность рассматривать связи более непосредственно. Поэтому иногда имеет смысл предоставить для таких стандартных вопросов удобные псевдонимы. Чтобы выразить это естественное сопоставление ассоциирующих ресурсов, можно под каждым из них создать подколлекцию с псевдонимом. Это позволит задавать подобные вопросы, используя один нефилтруемый запрос. Более подробно мы разберем этот нюанс позже, а пока будет достаточно сказать, что здесь в качестве методики задействуются псевдонимы, предоставляющие удобный способ задавать типичные вопросы о связях формата «многие ко многим».

14.3. РЕАЛИЗАЦИЯ

Теперь, поверхностно ознакомившись с этим паттерном, мы более подробно рассмотрим, как именно он работает и как его реализовать. Начнем с самых основ — именования.

14.3.1. Выбор имени для ассоциирующего ресурса

Прежде чем мы сможем что-либо делать, нужно выбрать имя для ассоциирующего ресурса. В некоторых случаях оно может очевидным образом происходить из контекста самого приложения. Например, в API, который позволяет студентам записываться на курсы, ассоциирующий ресурс может называться `CourseRegistration` или `CourseEnrollment`.

В иных случаях подобрать имя бывает сложнее. Например, в том же API сохранение связи между списком преподавателей (ассистентов, лаборантов и т. д.) технически не является `Registration` (регистрацией) или `Enrollment` (зачислением), а значит, нужно искать альтернативу. Типичным вариантом обработки такого случая будет использование чего-то наподобие `Membership` (принадлежности) или `Association` (ассоциации) с модификаторами, проясняющими по меньшей мере один из связываемых ресурсов (например, `CourseMembership`). В некоторых

случаях, например при вступлении пользователей в группы или клубы, связь можно назвать просто `Membership` без дополнительных модификаторов, но это всегда должно выглядеть осмысленно в рамках контекста.

Выбрав имя для ресурса, можно переходить к изучению реализации всевозможных способов взаимодействия с ним.

14.3.2. Поведение стандартных методов

Для каждого ассоциирующего ресурса нужно реализовать по меньшей мере несколько стандартных методов. Как видно из табл. 14.1, нам определенно потребуются методы `Create` и `Delete`, а в типичном сценарии, где ассоциирующий ресурс содержит метаданные (то есть когда нас интересует дополнительная информация о связи), нам также потребуются стандартные методы `Get` и `Update`. Помимо этого, просмотреть `membership` (принадлежности) к различным группам поможет метод `List`. Все они должны в точности соответствовать принципу стандартных методов, и это будет легко, учитывая, что принадлежность выражается подобно любому другому ресурсу.

Таблица 14.1. Общие сведения о стандартных методах в ассоциирующем ресурсе

Стандартный метод	Поведение	Необходим?
Create	Объединяет два ресурса	Да
Get	Показывает метаданные связи	Только при наличии метаданных
List	Выводит список связей между ресурсами	Да
Update	Обновляет метаданные связи	Только при наличии метаданных
Delete	Удаляет связь между ресурсами	Да

Прежде чем продолжать, нужно отметить еще одно важное требование, которого может не быть в реализациях стандартных методов других ресурсов: ограничения уникальности.

14.3.3. Уникальность

В отличие от большинства ресурсов, ассоциирующие обычно имеют очень важное ограничение уникальности, которое необходимо применять: два связываемых ресурса должен представлять строго один ассоциирующий. Иными словами, нам нужно быть уверенными, что пользователь не станет участником одной группы дважды, не покидая ее. Для этого в API при создании ассоциирующего ресурса присутствует новое требование проверки.

Как правило, мы возвращаем ошибку о конфликте ресурсов (HTTP-код 409 Conflict), только когда ресурс создается с уже существующим идентификатором. Хотя в этом случае один ассоциирующий ресурс, по существу, отличается от другого, только если отличаются два ассоциируемых ими ресурса. Проще говоря, пример из листинга 14.1, в котором мы пытаемся дважды добавить пользователя в одну и ту же группу, должен давать сбой, поскольку в нем создается конфликтующий ресурс.

Листинг 14.1. Ассоциирование одних и тех же данных вызывает конфликт

```
let membershipData = { userId: "Jimmy", groupId: "2" };
let membership1 = CreateMembership(membershipData);
let membership2 = CreateMembership(membershipData);
```

Сначала мы добавляем Джимми в группу 2. Здесь проблем быть не должно

Джимми уже является участником группы 2, значит, этот запрос должен выдать отказ с HTTP-кодом 409 Conflict

Далее мы рассмотрим, чем обновление ассоциирующих ресурсов может несколько отличаться от уже знакомого нам.

14.3.4. Поля только для чтения

Несмотря на то что метаданные о связи могут изменяться со временем, связанные ресурсы в течение всего своего существования должны оставаться неизменными. То есть когда есть ресурс, который представляет вступающего в группу пользователя, у нас не должно быть возможности изменить это, чтобы тот же ресурс представлял другого вступающего в группу пользователя (или того же пользователя, вступающего в другую группу). Вместо этого в данном сценарии мы должны просто удалять старый ассоциирующий ресурс и создавать отдельный, который представит новую связь «пользователь — группа».

Это ведет к вопросу о том, должна ли попытка потребителя внести подобное изменение вести к ошибке или к негласному сбою. В данном случае два перекрестно ссылающихся поля (пользователя и группы) должны рассматриваться как только для чтения, а значит, игнорироваться при попытке указать их в запросе на обновление (листинг 14.2).

Листинг 14.2. Игнорирование полей только для чтения при обновлении

```
let membership1 = CreateMembership({
  userId: "Jimmy",
  groupId: "2",
  role: "admin" });

UpdateMembership({
  id: membership1.id,
  userId: "Sally",
  role: "user"
});
```

Джимми добавляется в группу 2 в качестве администратора

При попытке изменить пользователя на Салли, а его роль — на стандартного пользователя изменяется только роль. Этот запрос просто делает Джимми стандартным пользователем группы 2, не совершая никаких действий в отношении Салли

Разобравшись со стандартными методами и их поведением в отношении ассоциирующего ресурса, перейдем к рассмотрению необязательных вспомогательных методов, которые можно реализовать для охвата типичных потребительских сценариев.

14.3.5. Ассоциирующие методы-псевдонимы

Как уже говорилось, один из наиболее часто задаваемых вопросов касается того, какие ресурсы и с какими ресурсами связаны. Например, нам нужно узнать ответы на следующие вопросы: «Участником каких групп является Джимми?» или «Какие участники состоят в группе 2?» Несмотря на то что это вполне реально сделать, применив к `ListMembershipsRequest` определенный фильтр, можно упростить этот процесс, предоставив методы-псевдонимы, отвечающие конкретно на эти вопросы (листинг 14.3).

Листинг 14.3. Вывод списка связей с помощью фильтра

```
memberships = ListMemberships({ filter: "groupId: 2" });
usersInGroup2 = memberships.map(
  (membership) => membership.user );

memberships = ListMemberships({
  filter: "userId: Jimmy" });
jimmysGroups = memberships.map(
  (membership) => membership.groupId );
```

Здесь мы создаем `ListMembershipsRequest`, который возвращает только `membership` группы 2

Чтобы получить пользователей, которые являются участниками группы 2, мы просто просматриваем поле `user` в каждом `membership`

Здесь мы создаем `ListMembershipsRequest`, возвращающий только те группы, в которых Джимми является пользователем

Чтобы увидеть, участником каких групп является Джимми, мы просто просматриваем в каждом `membership` поле `group`

И хотя такой вариант определенно сработает, может быть удобнее создать методы, сосредоточенные конкретно на этих случаях. Это можно сделать с помощью методов-псевдонимов, которые содержат подразумеваемый (и необходимый) параметр фильтрации для нужного нам запроса. В примере ресурсов `User` и `Group`, связанных ресурсом `Membership`, можно создать два псевдонима, как показано в табл. 14.2.

Таблица 14.2. Псевдонимы для частых запросов к ассоциирующим ресурсам

Вопрос	List с фильтром	Пользовательский метод
В каких группах состоит Джимми?	ListMemberships({ filter: "userId: Jimmy" })	ListUserGroups({ userId: "Jimmy" })
Какие пользователи состоят в группе 2?	ListMemberships({ filter: "groupId: 2" })	ListGroupUsers({ groupId: 2 })

Формат имен может немного запутывать, поэтому запомните, что первая (в единственном числе) часть имени — это владеющий ресурс, а вторая (множественная) часть — то, что выводится. Вот почему при выводе списка `UserGroups` мы ищем группы конкретного пользователя, а при выводе `GroupsUsers` нас интересуют пользователи конкретной группы.

14.3.6. Ссылочная целостность

Как мы видели в главе 13, ссылочная целостность в API подразумевает, что, указывая ссылку на другой ресурс, нам необходимо учитывать, действительна ли она. Например, что произойдет, если удалить ресурс, на который продолжает указывать другой ресурс? Говоря конкретнее: если у нас есть ассоциирующий ресурс, который связывает два ресурса, то как обрабатывать запросы, удаляющие один из них?

В подобных сценариях у нас есть несколько вариантов, предоставляемых и стандартизируемых большинством современных реляционных баз данных. Они представлены в табл. 14.3.

Таблица 14.3. Общее описание поведения для сохранения ссылочной целостности

Вариант	Поведение
Каскадирование	Удаление всех ресурсов, указывающих на удаляемый ресурс
Ограничение	Запретить удаление ресурсов, на которые есть ссылки из других ресурсов
Установка <code>null</code>	Любые указатели на удаляемый ресурс устанавливаются на <code>null</code>
Оставить все как есть	Удаление ресурсов никак не влияет на указатели, оставляя их недействительными

И хотя выбор каждого из этих вариантов обоснован своими причинами, в случае ассоциирующих ресурсов API, как правило, должен либо задействовать ограничение удаления, либо просто ничего не делать, позволяя ссылкам становиться недействительными. Если же вместо этого выбрать каскадирование или установку указателя на `null`, то появляется возможность спровоцировать массовый поток записей по установке значений на `null` (например, если на удаляемый ресурс ссылается 10 миллиардов других ресурсов). Если коротко, то в случае удаления ресурса, на который ссылался ассоциирующий ресурс, мы должны либо возвращать ошибку «предусловие ложно» (HTTP-код 412), либо просто удалять ресурс и оставлять ассоциирующий ресурс в виде висячего указателя, с которым впоследствии будет разбираться уже потребитель.

14.3.7. Итоговое определение API

Наконец, пришло время взглянуть на итоговый пример определения API, в котором ассоциирование ресурсов User и Group реализовано с помощью ресурса Membership (листинг 14.4).

Листинг 14.4. Итоговое определение API с помощью ассоциирования ресурсов

```
abstract class GroupApi {
    static version = "v1";
    static title = "Group API";

    // ... Другие методы опущены в целях сокращения.

    @post("/memberships")
    CreateMembership(req: CreateMembershipRequest): Membership;

    @get("/{id=memberships/*}")
    GetMembership(req: GetMembershipRequest): Membership;

    @patch("/{resource.id=memberships/*}")
    UpdateMembership(req: UpdateMembershipRequest): Membership;

    @delete("/{id=memberships/*}")
    DeleteMembership(req: DeleteMembershipRequest): void;

    @get("/memberships")
    ListMemberships(req: ListMembershipsRequest): ListMembershipsResponse;

    @get("/{groupId=groups/*}/users")
    ListGroupUsers(req: ListGroupUsersRequest): ListGroupUsersResponse;

    @get("/{userId=users/*}/groups")
    ListUserGroups(req: ListUserGroupsRequest): ListUserGroupsResponse;
}

interface Group {
    id: string;
    userCount: number;
    // ...
}

interface User {
    id: string;
    emailAddress: string;
    // ...
}

interface Membership {
    id: string;
    groupId: string;
    userId: string;
}
```

← Необязательные псевдонимы для вывода списка пользователей группы

← Необязательные псевдонимы для вывода групп, в которых состоит пользователь

← Обратите внимание: мы не встраиваем здесь список пользователей. Чтобы просмотреть участников группы, мы просто выводим принадлежность (membership) с помощью фильтра для одной группы или задействуем подколлекцию с псевдонимом (ListGroupUsers)

← Обратите внимание: мы не встраиваем здесь группы. Чтобы просмотреть группы, в которых состоит пользователь, мы просто выводим принадлежность с фильтром для одного пользователя или задействуем подколлекцию с псевдонимом (ListUserGroups)

← Обратите внимание: для ресурса, ассоциирующего пользователей с группами, выбрано имя Membership

← Здесь мы сохраняем ссылки на ассоциируемых пользователей и группы, а не на всю копию ресурсов


```

    role: string;
    expireTime: DateTime;
}

```

← Остальные поля представляют метаданные связи

```

interface ListMembershipsRequest {
    parent: string;
    maxPageSize: number;
    pageToken: string;
}

interface ListMembershipsResponse {
    results: Membership[];
    nextPageToken: string;
}

interface CreateMembershipRequest {
    resource: Membership;
}

```

← Обратите внимание: здесь нет родителя, поскольку ресурсы Membership находятся на верхнем уровне

```

interface GetMembershipRequest {
    id: string;
}

interface UpdateMembershipRequest {
    resource: Membership;
    fieldMask: FieldMask;
}

interface DeleteMembershipRequest {
    id: string;
}
// Далее следует необязательная часть

interface ListUserGroupsRequest {
    userId: string;
    maxPageSize: number;
    pageToken: string;
}

interface ListUserGroupsResponse {
    results: Group[];
    nextPageToken: string;
}

interface ListGroupUsersRequest {
    groupId: string;
    maxPageSize: number;
    pageToken: string;
}

interface ListGroupUsersResponse {
    results: User[];
    nextPageToken: string;
}

```

Теперь, когда мы рассмотрели работу каждого из вариантов, приостановимся и разберем возможности, которые теряются при использовании этого паттерна.

14.4. КОМПРОМИССЫ

Используя ассоциирующий ресурс для выражения связи «многие ко многим», вы получаете максимальную свободу в проектировании всех функциональных деталей этой связи, однако это имеет свою цену.

14.4.1. Сложность

За невероятную гибкость мы платим дополнительной сложностью интерфейса, который иногда может показаться не интуитивным. Например, когда нам нужно подключить пользователя в группу, мы создаем новую принадлежность с помощью стандартного метода `Create`, а не вызываем `JoinGroup`.

Кроме того, нам приходится учитывать дополнительный ассоциирующий ресурс, что означает увеличение поверхности API, поскольку теперь в нем есть по одному лишнему ресурсу на связь и вплоть до семи дополнительных методов (пять стандартных и два необязательных псевдонима). Зачастую эти методы легко осваиваются и запоминаются, что требует дополнительной мыслительной нагрузки. Тем не менее стоит добавить, что здесь просто нужно еще многое изучить.

14.4.2. Отделенность ассоциирующих ресурсов

При использовании ассоциирующих ресурсов даже несмотря на возможность предоставить псевдонимы, упрощающие постановку вопросов о связях ресурсов, мы все равно рассматриваем связь между двумя ресурсами отдельно от информации о каждом из них. Например, описание группы и список участников группы считаются информацией о группе. Однако извлекаем мы эти данные по-разному. Описание группы мы получаем с помощью метода `GetGroup()`, а поиск всех ее участников может подразумевать метод-псевдоним `ListGroupUsers()`.

И, как мы поняли, на то есть веская причина (список пользователей может стать очень большим), но это означает, что два этих элемента данных являются разделенными, несмотря на тесную связь обоих с конкретной группой.

14.5. УПРАЖНЕНИЯ

1. Спроектируйте API для связывания пользователей с комнатами чата, к которым они могут присоединяться. Сделайте так, чтобы при этом также сохранялось время вступления в комнату и роль пользователя в ней.
2. В приложении чата пользователи могут многократно присоединяться к комнатам и покидать их. Как бы вы смоделировали API, чтобы сохранять историю и при этом обеспечить невозможность дублирования одновременного присутствия пользователя в комнате?

РЕЗЮМЕ

- Связь «многие ко многим» подразумевает наличие двух ресурсов, каждый из которых во множественном числе присутствует в другом. Например, пользователь может состоять во множестве групп, а группа содержать множество пользователей.
- Мы можем использовать ассоциирующие ресурсы в качестве способа моделирования связей «многие ко многим» между ресурсами API.
- Ассоциирующие ресурсы — наиболее гибкий способ представления связи между двумя ресурсами.
- Эти ассоциирующие ресурсы можно использовать для хранения дополнительных метаданных о связи между двумя ресурсами.

15

Пользовательские методы Add и Remove

В этой главе

- ✓ Как управлять связями «многие ко многим» неявно.
- ✓ Преимущества и недостатки неявного ассоциирования в сравнении с явным.
- ✓ Как связывать ресурсы с помощью пользовательских методов.
- ✓ Как обрабатывать проблемы целостности данных связанных ресурсов.

В этой главе мы познакомимся с альтернативой, позволяющей моделировать связи «многие ко многим», научившись ассоциировать (и диссоциировать) ресурсы с помощью пользовательских методов `Add` и `Remove`. Этот паттерн позволяет потребителям управлять связями «многие ко многим», не вводя третий, ассоциирующий ресурс.

15.1. ЗАЧЕМ ЭТО НУЖНО

Как мы узнали в главе 14, время от времени возникает потребность отслеживать связи между ресурсами, которые иногда бывают достаточно сложными. В частности, нам нередко приходится обрабатывать ситуации, когда два ресурса могут содержать много экземпляров друг друга, что называется связью «многие ко многим».

И несмотря на то что паттерн ассоциирующего ресурса предоставляет гибкий и эффективный способ моделирования подобной связи, возникает вопрос: нет ли более простого способа делать это, если мы готовы смириться с некоторыми ограничениями? Иными словами, допуская определенные ограничения, можно ли сделать API более простым и интуитивным? И если да, то на какие именно ограничения придется пойти? Текущий паттерн представляет более простую альтернативу паттерну ассоциирующего ресурса.

15.2. ОБЗОР

Как вы могли догадаться, определенно существуют более удобные способы выражения и управления в API связями «многие ко многим», но им сопутствует ряд ограничений. В описываемом ниже паттерне мы рассмотрим способ сокрытия отдельных ресурсов, которые представляют такие связи, реализовав удаление этих связей с помощью пользовательских методов. Мы начнем с обобщения методов, после чего перейдем к изучению конкретных ограничений, возникающих при использовании этого паттерна проектирования.

По существу, мы упрощаем API, полностью скрывая ассоциирующий ресурс от потребителей и вместо него управляя связями с помощью пользовательских методов `Add` и `Remove`. Они выступают легкими способами создания и удаления связи между двумя ресурсами, пряча все детали об этой связи, за исключением самого факта ее существования (или отсутствия). В классическом примере пользователей, которые могут быть участниками нескольких групп (и групп, которые содержат множество пользователей), это означает, что мы можем просто использовать данные методы для представления вступающих (`Add`) в группу пользователей и покидающих (`Remove`) ее.

Применение этого паттерна потребует от нас учесть ограничения, на которые придется пойти. Для начала, поскольку мы будем сохранять лишь сам факт связанности двух ресурсов, у нас не будет возможности хранить какие-либо метаданные об этой связи. То есть, к примеру, мы не сможем сохранить детали о принадлежности, такие как дата вступления пользователя в группу или конкретная роль, которую он занимает в ней.

Кроме того, поскольку для добавления и удаления связи между ресурсами будут применяться пользовательские методы, нам нужно рассматривать один из них как *управляющий ресурс* — в некотором смысле так, будто он является родителем другого. Точнее говоря, нам нужно выбрать: мы добавляем пользователей в группы или группы к пользователям. В первом случае передаваться будут ресурсы пользователей, а управлять связью — ресурсы групп. Если представить это в виде фактического кода (в данном случае в ООП-стиле), то управляющий ресурс — это тот, который будет содержать методы `Add` и `Remove` (листинг 15.1).

Листинг 15.1. Фрагменты кода для двух альтернатив при выборе управляющего ресурса

```
group.addUser(userId);
user.addGroup(groupId);
```

← Когда связью управляет группа, мы добавляем пользователей в нужную группу

← Когда связью управляет пользователь, мы добавляем к заданному пользователю группы

Иногда выбор управляющего ресурса будет очевиден, но в других случаях над ним придется подумать, так как разумными могут выглядеть оба варианта. В некоторых же сценариях вообще ни один из них не будет казаться логичным выбором. Как бы то ни было, но в итоге паттерн все-таки требует назначения управляющего ресурса. Предполагая, что мы вполне сможем принять оба ограничения, рассмотрим специфику работы этого паттерна.

15.3. РЕАЛИЗАЦИЯ

Выбрав управляющий ресурс, можно определить пользовательские методы `Add` и `Remove`. Полное имя метода должно соответствовать виду `Add<ManagingResource><Associated-Resource>` (аналогично для `Remove`). Например, если у нас есть пользователи, которые могут быть добавлены в группы и удалены из них, то пользователь будет выступать связываемым ресурсом, а группа — управляющим. То есть имена методов примут вид `AddGroupUser` и `RemoveGroupUser`.

Эти методы должны принимать запрос, содержащий и родительский ресурс (в данном случае управляющий), и идентификатор добавляемого/удаляемого ресурса. Заметьте, что мы используем только ID, а не весь ресурс. Причина в том, что остальная информация окажется лишней и может ввести в заблуждение, если потребитель поверит, что у него есть возможность связывать два ресурса и одновременно обновлять один из них. Краткое описание методов `Add` и `Remove` вместе с их HTTP-эквивалентами представлено в табл. 15.1.

Таблица 15.1. Общие сведения по методам `Add` и `Remove`

Действие	Метод	HTTP-эквивалент
Добавляет пользователя 1 в группу 1	<pre>AddGroupUser({ parent: "groups/1", userId: "users/1" });</pre>	<pre>POST /groups/1/users:add { userId: "users/1" }</pre>
Удаляет пользователя 1 из группы 1	<pre>RemoveGroupUser({ parent: "groups/1", userId: "users/1" });</pre>	<pre>POST /groups/1/users:remove { userId: "users/1" }</pre>

15.3.1. Вывод списка связанных ресурсов

Список различных связей мы выводим с помощью настраиваемых стандартных методов `List`, которые выглядят так же, как методы-псевдонимы, рассмотренные нами в разделе, посвященном ассоциирующим ресурсам. Эти методы просто выводят различные связанные ресурсы, предоставляя возможность пагинации и фильтрации результатов. Поскольку мы можем рассматривать связь двумя способами (например, какие пользователи состоят в данной группе или членом каких групп является данный пользователь), то для каждого сценария нам потребуются два разных метода.

Эти методы следуют тому же соглашению об именовании, что и методы `Add` и `Remove`, включая в свои имена оба ресурса. Используя в качестве примера пользователей и группы, мы предоставим два разных метода для вывода списка различных пользователей на основе конкретного условия: `ListGroupUsers` выводит список пользователей, принадлежащих заданной группе, а `ListUserGroups` выводит список групп, членом которых является заданный пользователь. Как и любые другие пользовательские методы, наши следуют аналогичному соглашению об именовании в соответствии с HTTP-эквивалентом, но опираются на неявную подколлекцию. В табл. 15.2 это показано наглядно.

Таблица 15.2. Общие сведения по методу List

Действие	Метод	HTTP-эквивалент
Какие пользователи состоят в группе 1?	<code>ListGroupUsers({ parent: "groups/1" })</code>	GET /groups/1/users
Членом каких групп является пользователь 1?	<code>ListUserGroups({ parent: "users/1" });</code>	GET /users/1/groups

15.3.2. Целостность данных

Один из частых вопросов возникает, когда мы сталкиваемся с проблемой дублирования данных. Например, что, если попробовать добавить одного и того же пользователя в группу дважды? И наоборот, что, если попробовать удалить его из группы, членом которой он не является?

Как мы видели в главе 7, в подобных случаях поведение будет очень похоже на удаление несуществующего ресурса и создание дублирующего (а значит, конфликтующего) ресурса. Это означает, что при попытке дважды добавить пользователя в одну группу наш API должен отвечать ошибкой конфликта (например, `409 Conflict`), а при попытке удалить пользователя из несуществующей

группы мы должны возвращать код, выражающий ошибочное предположение (например, 412 *Precondition Failed*), сигнализируя о невозможности выполнения запрошенной операции.

Иначе говоря, если потребителю лишь требуется уверенность, что пользователь является (или нет) участником заданной группы, он может просто рассматривать эти условия ошибок как уже выполненную работу. Иными словами, если нам просто нужно убедиться, что Джимми состоит в группе 2, то и успешный результат, и ошибка конфликта окажутся валидны, поскольку оба указывают, что Джимми в данный момент является членом группы, как мы и хотели. Разница в коде ответа просто покажет, мы ли стали причиной добавления, но в обоих случаях результатом будет то, что пользователь теперь является участником группы.

15.3.3. Итоговое определение API

Полный образец реализации этого паттерна с помощью того же примера с пользователями и группами показан в листинге 15.2. Как видите, он намного короче и проще, чем в случае с ассоциирующим ресурсом. Тем не менее здесь отсутствует возможность хранения метаданных о связи между пользователями и группами.

Листинг 15.2. Итоговое определение API с помощью паттерна add/remove

```
abstract class GroupApi {
    static version = "v1";
    static title = "Group API";

    @get("{id=users/*}")
    GetUser(req: GetUserRequest): User;

    // ...

    @get("{id=groups/*}")
    GetGroup(req: GetGroupRequest): Group;

    // ...

    @post("{parent=groups/*/users:add}")
    AddGroupUser(req: AddGroupUserRequest): void;

    @post("{parent=group/*/users:remove}")
    RemoveGroupUser(req: RemoveGroupUserRequest): void;

    @get("{parent=groups/*/users}")
    ListGroupUsers(req: ListGroupUsersRequest): ListGroupUsersResponse;
```

← Ради сокращения все остальные методы для ресурсов User и Group опущены

← Для управления связями между пользователями и группами используются AddGroupUser и RemoveGroupUser

← Чтобы увидеть, какие пользователи находятся в заданной группе, мы используем неявное отображение подколлекций в метод ListGroupUsers


```

@get("{parent=users/*/groups}")
ListUserGroups(req: ListUserGroupsRequest):
    ListUserGroupsResponse;
}

interface Group {
    id: string;
    userCount: number;
    // ...
}

interface User {
    id: string;
    emailAddress: string;
}

interface ListUserGroupsRequest {
    parent: string;
    maxPageSize?: number;
    pageToken?: string;
    filter?: string;
}

interface ListUserGroupsResponse {
    results: Group[];
    nextPageToken: string;
}

interface ListGroupUsersRequest {
    parent: string;
    maxPageSize?: number;
    pageToken?: string;
    filter?: string;
}

interface ListGroupUsersResponse {
    results: User[];
    nextPageToken: string;
}

interface AddGroupUserRequest {
    parent: string;
    userId: string;
}

interface RemoveGroupUserRequest {
    parent: string;
    userId: string;
}

```

Чтобы увидеть, участником каких групп является пользователь, мы используем тот же принцип и создаем метод ListUserGroups

Обратите внимание: мы не встраиваем здесь пользователей, поскольку список может оказаться очень большим. Вместо этого мы выводим его с помощью ListGroupUsers

15.4. КОМПРОМИССЫ

Как мы говорили в начале главы, основная цель этого паттерна — предоставить возможность управлять связями «многие ко многим», исключив сложность полномасштабного ассоциирующего ресурса. В обмен на это упрощение мы жертвуем некоторыми функциональными возможностями.

15.4.1. Подчинительная связь

В отличие от ассоциирующего ресурса, применение пользовательских методов `Add` и `Remove` требует от нас назначения одного из ресурсов управляющим, а второго — управляемым. Иными словами, нам нужно решить, какой ресурс будет добавляться в другой и удаляться из него. Во многих случаях такой подчинительный характер связи оказывается удобным и очевидным, но иногда она может выглядеть нелогичной.

15.4.2. Метаданные о связи

Применяя вместо ассоциирующего ресурса пользовательские методы `Add` и `Remove`, мы теряем возможность сохранять метаданные о самой связи. Это означает, что мы не сможем хранить никакую информацию, помимо самого факта существования (или отсутствия) связи. Иначе говоря, мы не сможем отслеживать время создания этой связи или любые другие ее характеристики, будучи вынуждены размещать соответствующую информацию в другом месте.

15.5. УПРАЖНЕНИЯ

1. В каких случаях для моделирования связи между двумя ресурсами в формате «многие ко многим» стоит предпочесть пользовательские методы `Add` и `Remove`, а не ассоциирующий ресурс?
2. Какой ресурс окажется управляющим, а какой — ассоциируемым при связывании ресурсов `Recipe` и `Ingredient`?
3. Как должен называться метод для вывода списка ресурсов `Ingredients`, составляющих конкретный рецепт?
4. Каким должен быть результат при добавлении дублирующегося ресурса с помощью пользовательского метода `Add`?

РЕЗЮМЕ

- В сценариях, где ассоциирующие ресурсы оказываются излишне тяжеловесными (например, нет нужды в дополнительных метаданных о связи), применение пользовательских методов **Add** и **Remove** может оказаться наиболее простым способом управления связями «многие ко многим».
- Пользовательские методы **Add** и **Remove** позволяют добавлять или удалять ассоциируемый ресурс (то есть подчиненный) в отношении управляющего ресурса.
- Вывод списка связанных ресурсов можно делать с помощью стандартных методов **List**, применяемых к метаресурсам.

16

Полиморфизм

В этой главе

- ✓ Что такое полиморфизм.
- ✓ Каковы преимущества полиморфизма и когда он уместен в API.
- ✓ Как лучше структурировать полиморфные ресурсы.
- ✓ Почему полиморфных методов в ресурсно-ориентированных API лучше избегать.

При разработке систем ПО полиморфизм дает объектам возможность принимать множество разных форм, обычно опираясь на явное наследование объектов. В данном паттерне мы узнаем, как перенести этот эффективный инструмент из мира объектно-ориентированного программирования в мир ресурсно-ориентированной разработки API. Кроме того, мы изучим рекомендации о том, когда нужно использовать полиморфные ресурсы вместо полностью независимых.

16.1. ЗАЧЕМ ЭТО НУЖНО

В объектно-ориентированном программировании (ООП) полиморфизм подразумевает использование единичных общих интерфейсов среди разных конкретных типов, минимизируя количество деталей реализации, которые нам требуется понимать, чтобы взаимодействовать с определенным типом. То есть

если у нас есть `class Triangle` и `class Square`, то они оба могут реализовывать общий метод `countSides()`, указывая на это путем расширения общего `interface Shape` (листинг 16.1).

Листинг 16.1. Пример полиморфизма в TypeScript

```
interface Shape {
  countSides(): number;
}

class Triangle implements Shape {
  countSides() { return 3; }
}

class Square implements Shape {
  countSides() { return 4; }
}
```

Интерфейс объявляет одну функцию, которая должна быть реализована

Интерфейс реализуют два разных класса, а значит, они реализуют и объявленный метод

Поскольку у нас есть этот общий интерфейс, можно написать код, работающий не только с интерфейсами `Shape` (листинг 16.2). То есть нам не придется задумываться о том, является ли конкретная фигура `Triangle` или `Square` (или чем-то еще) при условии, что она реализует требования, указанные интерфейсом `Shape`.

Листинг 16.2. Полиморфный метод для подсчета сторон любой фигуры

```
function countSides(shape: Shape): number {
  return shape.countSides();
}
```

Нам не нужно указывать в качестве ввода `Triangle` или `Square`. Можно просто потребовать, чтобы класс реализовывал интерфейс `Shape`

Это очень эффективная идея ООП, которая позволяет писать намного более чистый и модульный код, вследствие чего появляется желание использовать ту же возможность в веб-API. Проблема же возникает во время попытки найти способ точного переноса этого принципа из мира ООП-языков в мир JSON и HTTP. В конце концов, что окажется эквивалентом `countSides()` в ресурсно-ориентированном веб-API? Какой ресурс сможет выразить то, что нам нужно? Цель текущего паттерна — проиллюстрировать безопасный и надежный способ привнесения наиболее полезных возможностей ООП в мир ресурсно-ориентированных API.

16.2. ОБЗОР

Этот паттерн поможет нам изучить принцип полиморфных ресурсов, то есть таких, которые могут принимать форму обобщенных интерфейсов (например, `Shape`) с явным полем, указывающим более подробный тип ресурса (например, `Triangle` или `Square`). И хотя у этой стратегии есть много преимуществ, самое

важное заключается в том, что нам не придется дублировать стандартные методы для каждой конкретной реализации. Иными словами, вместо `ListTriangles` и `ListSquares` можно будет использовать один метод `ListShapes`.

При этом нужно учитывать, что хоть эта стратегия и упрощает полиморфное поведение (например, методы, которые взаимодействуют с обобщенным ресурсом, а не конкретной реализацией), в ней ничего не говорится об особенностях хранения в этом случае данных. Иными словами, если у квадрата (`Square`) есть длина стороны, то у треугольника (`Triangle`) есть основание и высота. В результате для истинного представления деталей этих фигур нам потребуется хранить в ресурсе разные поля. Чтобы сделать это, нам придется использовать проверку на стороне сервера. То есть наш обобщенный интерфейс должен будет являться надмножеством необходимых полей, имея возможность хранить все их виды, соответствующие разным подтипам. Иными словами, ресурс `Shape` будет иметь возможность хранить радиус (для кругов), основание и высоту (для треугольников) и длину (для квадратов), проверяя эти значения в зависимости от того, какой тип указан.

Тем не менее сложности, как обычно, кроются в деталях. Например, что нужно делать, если запрос API включает поля, относящиеся к неверному типу ресурса (скажем, определение радиуса `Shape` с типом квадрата)? Должна ли здесь возникать ошибка или ситуацию нужно игнорировать? В следующем разделе мы подробно разберем, как все это работает.

16.3. РЕАЛИЗАЦИЯ

Очевидно, что полиморфизм является очень эффективным инструментом во многих языках программирования. Значит, логично предположить, что полиморфные ресурсы окажутся столь же ценными и в веб-API. Чтобы получить из этого принципа максимум возможностей, следует разобраться, как эти ресурсы работают на поведенческом и структурном уровнях. Но прежде чем приступить, нужно понять, когда вообще есть смысл обращаться к принципу полиморфизма.

16.3.1. Когда использовать полиморфные ресурсы

Зачастую при разработке API мы находим общие черты у двух предполагаемых ресурсов, что заставляет дважды задумываться об их разделении. Например, в API чата мы можем рассматривать использование ресурса `TextMessage`, а также ресурсов `PhotoMessage`, `VideoMessage` и `AudioMessage`. Все они имеют нечто общее (например, представляют сообщение, отправляемое через API чата, но при этом между ними есть и различия (в данном случае содержимое сообщения)).

Если бы мы оказались в таком сценарии при локальной разработке, то могли бы рассмотреть использование интерфейса `Message` и ряда других классов, реализующих его. Это бы дало нам возможность писать код, который работает только со значениями обобщенного интерфейса `Message`, а не с его конкретными реализациями. Но переносится ли эта логика на веб-API? Иными словами, нужно ли нам ориентироваться на тот же принцип обобщенности, принимая решение об использовании одного ресурса `Message` вместо нескольких независимых ресурсов для каждого типа сообщения?

И хотя подобный довод выглядит верным, сама логика перекладывается не столь гладко, как хотелось бы. Если говорить конкретнее, то наша задача при работе в локальном языке программирования состоит в обобщении и написании функций, поддерживающих дальнейшее расширение и повторное использование кода. А в веб-API нас больше волнует работа различных стандартных методов и то, будут ли они работать одинаково по всем ресурсам или же иметь поведенческие различия для их разных типов.

Например, отличный способ проверить уместность применения полиморфного ресурса (скажем, `Message`) вместо независимых ресурсов (например, `VideoMessage`) — проанализировать разумность сведения их всех в одном месте. То есть понять, хотим ли мы вызывать один метод и получать все типы ресурсов в одном ответе или же нам нужно выводить ресурсы только по их отдельным подтипам. В случае с API чата ответ будет очевиден: нам наверняка нужно выводить все сообщения ресурса `ChatRoom` независимо от их типа. В противном случае нам бы пришлось потрудиться над извлечением конкретного списка сообщений, для чего потребовалось бы сделать несколько вызовов API (а затем упорядочить результаты по времени создания), чтобы отобразить содержимое чата должным образом.

Рассмотрим другой сценарий, в котором API может содержать ресурсы `ChatRoom`, а также «вещательные» группы, в которых центральный узел может отправлять однонаправленные обновления всем пользователям. Несмотря на то что такой «вещательный» принцип похож на ресурс `ChatRoom` наличием сообщений, способ взаимодействия с этими ресурсами будет сильно различаться. Состав участников в этом случае не определен, их могут создавать только администраторы API, сообщения отправляются лишь в одном направлении и т. д. Такой принцип и можно потенциально «втиснуть» в полиморфный ресурс `ChatRoom` (с особым типом для трансляции). Однако с паттернами доступа будет намного удобнее работать, если создать отдельный ресурс `Broadcast`, который, будучи похожим на `ChatRoom`, в своей основе окажется фундаментально иным и будет рассматриваться независимо.

Теперь, когда у нас есть некоторое представление о том, как выбирать между независимыми и полиморфными ресурсами, изучим построение последних.

16.3.2. Полиморфная структура

Полиморфные ресурсы содержат одно важное отличительное поле: тип. Поскольку сам такой ресурс сродни обобщенному интерфейсу в языке программирования, нам необходим явный способ сообщать, что экземпляр этого обобщенного интерфейса имеет конкретный подтип. Например, в случае интерфейса `Message` нам нужен способ, позволяющий указать, чем является сообщение: видео, аудио или текстом. Но как же должно выглядеть это поле типа?

Определение поля типа

Несмотря на то что здесь может напрашиваться применение перечисления, из главы 5 мы помним, что перечисления могут вызывать множество проблем и предполагают соблюдение ряда специфичных критериев для их использования. Вместо этого можно применить простое строковое поле с проверкой значений, которые могут храниться в нем (листинг 16.3).

Листинг 16.3. Пример полиморфного ресурса с полем `type`

```
interface Message {
  id: string;
  sender: string;
  type: 'text' | 'photo' | 'audio' | 'video';
  // ...
}
```

← Поле `type` — это простая строка, хранящая допустимые подтипы

Поскольку это поле является простой строкой, использовать его можно подобно любому другому. Например, как мы увидим в главе 22, если нам нужно вывести список всех сообщений, но только с фотографиями, то можно вызвать `ListMessages()`, применив фильтр к полю `type`. Это также означает, что создание нового ресурса `Message` может принимать любые релевантные формы с передачей подходящих значений в вызов `CreateMessage()`.

Но несмотря на то что это поле сродни любому другому, здесь все же остается один важный вопрос: должно ли это поле быть постоянным или может изменяться? Иными словами, можно ли трансформировать ресурс из одного типа (например, видеосообщения) в другой (например, текстовое сообщение). И хотя технически ничто в самом ресурсе не мешает реализовать это, обычно так делать не рекомендуется. Причина в том, что по мере усложнения API и возникновения большего количества связей изменение типа полиморфного ресурса может привести к нарушению предположения, сделанного другим ресурсом, который на него ссылается. Когда мы увлекаемся подобным изменением типов и нарушением предположений, это зачастую ведет к запутыванию пользователей и возникновению ошибок в самом API. Так что этого следует по возможности избегать.

Продолжая разбирать идею полиморфных ресурсов, мы приходим к очередному вопросу: как именно сохранять содержимое сообщения? Ведь у каждого типа сообщения оно свое.

Структура данных

Основное преимущество полиморфного ресурса заключается в том, что по определению он может принимать множество форм. И хотя это зачастую упирается в поведение ресурса (например, при вызове одного метода API для разных типов полиморфного ресурса происходят разные действия), лишь в редких случаях такие ресурсы имеют одинаковую структуру. Наоборот, практически всегда разным типам потребуется хранить разную информацию.

В примере с геометрическими фигурами нам нужно хранить радиус для кругов, но длину для квадратов. В случае ресурса `Message` чата текстовые сообщения могут хранить лишь текстовое содержимое. При этом сообщение с фото должно хранить определенное изображение, что явно отличается от хранения текста. Как же быть в этой ситуации?

Самый простой способ (он же является предпочтительным в более строгих языках определения интерфейсов) — заставить ресурсы выступать как надмножества всех полей каждого отдельного типа. То есть ресурс `Message` может содержать поле для хранения строкового содержимого, еще одно — для хранения фото, третье — для видео, и т. д. (листинг 16.4).

Листинг 16.4. Сохранение данных с разными полями для каждого атрибута

```
interface Message {
  id: string;
  sender: string;
  type: 'text' | 'photo' | 'audio' | 'video';
  text?: string;
  photoUri?: string;
  videoUri?: string;
  audioUri?: string;
}
```

← Ресурс выступает как надмножество, определяя все необходимые поля

Иногда такая форма надмножества является необходимым требованием, но во многих языках определения интерфейсов (например, в TypeScript, который мы используем в данной книге) по факту выражать все такие поля можно путем повторного использования одного. Иными словами, вместо того чтобы создавать поля для `text`, `videoUri` и т. д., можно использовать для всего содержания одно поле, принимающее разные значения в зависимости от поля `type` (листинг 16.5).

Листинг 16.5. Представление данных для нескольких полей в одном поле

```
interface Message {
  id: string;
  sender: string;
  type: 'text' | 'photo' | 'audio' | 'video';
  content: string;
}
```

← Сейчас это поле представляет текстовое содержимое или URI различного медиасодержимого, но зависит от типа ресурса Message

В этом примере каждый тип содержимого представляется простой строкой, но что, если наши задачи более обширны? Представьте, что нам нужно отслеживать

тип медиасодержимого (например, формат кодирования видео), а также расположение самого файла. В этом случае можно определить отдельный интерфейс для медиа, продолжив использовать простое строковое значение для текстового содержимого (листинг 16.6).

Листинг 16.6. Поля имеют разные типы в зависимости от типа сообщения

```
interface Message {
  id: string;
  sender: string;
  type: 'text' | 'photo' | 'audio' | 'video';
  content: string | Media;
}
```

В этом случае содержимое может иметь различные типы

```
interface Media {
  contentUri: string;
  contentType: string;
}
```

Можно определить Media как ссылку на URI, где находится фактическое содержимое, а также указать тип содержимого, например video/mp4

Основной принцип всех этих примеров состоит в том, что нам нужно стараться абстрагироваться только до той степени, пока структура сохраняет целесообразность. В случае сообщений у нас нет проблем с абстрагированием между текстом и различным медиа, и по-прежнему сохраняется смысл проверять тип ресурса и на основе этого интерпретировать поле `content`.

А вот в случае с ресурсом `Shape`, где у нас были разные фигуры, несмотря на то, что большая часть полей будут численными, действительно важными окажутся имена этих полей. В результате мы можем использовать поле `dimension`, которое содержит различные размеры, но при этом создать различные типы для хранения соответствующей информации о размерах разных форм (листинг 16.7).

Листинг 16.7. Сохранение разных имен полей для разных фигур

```
interface Shape {
  id: string;
  type: 'square' | 'circle' | 'triangle';
  dimension: SquareDimension | CircleDimension |
    TriangleDimension;
}
```

Поле `dimension` хранит различные способы определения фигур

```
interface SquareDimension {
  length: number;
}
```

```
interface CircleDimension {
  radius: number;
}
```

Размер каждой фигуры имеет свое количество полей, все со своими именами

```
interface TriangleDimension {
  base: number;
  height: number;
}
```

Теперь, когда мы разобрались со структурой, можно уделить внимание изучению взаимодействия с ней. Самое главное, с чем здесь нужно разобраться, — это что произойдет, если мы попытаемся, так сказать, обойти правила?

Проверка данных

Как мы видели в последней теме, разные типы полиморфных ресурсов могут сохранять разную информацию. Более того, если в одних случаях допускается использование одного поля, интерпретируемого в зависимости от типа, то в других необходимо прописывать отдельные поля с разными типами для различающейся информации. Оба этих сценария ведут к дополнительным усложнениям, когда во время создания или обновления полиморфного ресурса передается недействительная информация.

Иначе говоря, аналогично тому, что поля можно интерпретировать в зависимости от поля `type` ресурса, сами правила проверки также должны применяться по-разному. Например, если мы опираемся на простой ресурс `Message` с полем `content: string`, то оно может хранить произвольное текстовое содержимое, если его тип будет `text`. При этом в случае медиа, например с типом `photo`, содержимым уже должен быть URI. В результате если кто-то укажет, что сообщение должно рассматриваться как фото, то критически важно возвращать ошибку, если его поле `content` не будет содержать действительный URI.

Требования к другим структурам несколько более утонченные. Например, в случае нашего примера с различными геометрическими фигурами каждый интерфейс для поля размера содержит свой набор имен полей (несмотря на то что тип у всех один). Что произойдет, если мы предоставим дополнительные поля, выполнив требования, но при этом сами данные окажутся лишены смысла? То есть что, если создать ресурс `Shape`, вызвав `CreateShape({ type: 'square', dimension: { radius: 10, length: 10 } })`? Очевидно, что у квадрата нет радиуса, значит, эта дополнительная информация абсолютно бесполезна.

Здесь может показаться уместным выбрасывать ошибку, как в случае с неверно отформатированным URI, но все же в этом случае будет лучше убеждаться в предоставлении необходимых входных данных, а все лишнее просто отбрасывать, как это делается с неизвестными полями при создании или обновлении любого другого ресурса.

Далее, вооружившись пониманием обработки данных в полиморфном ресурсе, мы займемся изучением поведенческих отличий этих особых ресурсов.

16.3.3. Полиморфное поведение

До сих пор мы говорили о структуре и дизайне полиморфных ресурсов, практически не затрагивая способы взаимодействия с этими ресурсами. Как мы видели в разделе 16.1, преимущества полиморфизма как в языках программирования,

так и в веб-API отчасти состоят в возможности работать с обобщенными интерфейсами, а не конкретной реализацией, которая в таком случае определяет результат операции. Если взять наш пример функции `countSides()` из раздела 16.1, то можно легко перенести ее в веб-API (листинг 16.8).

Листинг 16.8. Методы API для подсчета сторон разных фигур

```
abstract class ShapeApi {
    @post("/{id=shapes/*}:countSides")
    CountShapeSides(req: CountShapeSidesRequest): CountShapeSidesResponse;
}

interface CountShapeSidesRequest {
    id: string;
}

interface CountShapeSidesResponse {
    sides: number;
}
```

В этом примере можно представить варианты ответов: квадрат будет возвращать `{ sides: 4 }`, треугольник — `{ sides: 3 }` и т. д. То есть благодаря реализации ресурса в виде обобщенного интерфейса (например, `Shape` или `Message`) определяемые методы будут работать с ресурсом как обычно. И любое отклонение от этого поведения можно отнести к типичному отклонению, которое мы привыкли ожидать от ресурсов со значениями, различающимися для разных полей. Если коротко, то наши предыдущие решения в плане дизайна привели к очень удобному результату, не слишком отличающемуся от любого другого стандартного или пользовательского метода ресурса.

Но здесь возникает важный вопрос, с которым необходимо разобраться: что делать в случае, когда у нас есть отдельные ресурсы и нужен один метод API, который бы позволил работать с различными типами ресурсов? В следующем подразделе мы подробно это разберем и объясним, почему обычно так делать не стоит.

16.3.4. Почему не стоит использовать полиморфные методы

До этого момента мы рассматривали только ситуацию, когда один ресурс может принимать несколько форм. Это существенно упростило работу с методами API в том, что они работают ровно так, как работали бы для любого другого ресурса. Тем не менее существует совершенно иной тип полиморфизма, который мы не упомянули: полиморфные методы.

Для задач текущего подраздела мы определим полиморфный метод в роли метода API, способного работать с несколькими разными типами ресурсов.

В качестве вымышленного примера мы представим в API обобщенный метод `DeleteResource()`, способный удалять любой ресурс независимо от его типа. Отдельно обратите внимание, что HTTP URL отображается в `"*/**"`, а не в `"chatRooms/*"` (листинг 16.9).

Листинг 16.9. Полиморфный метод может работать с различными типами ресурсов

```
abstract class GenericApi {
  @delete("/{id=**}")
  DeleteResource(req: DeleteResourceRequest): void;
}

interface DeleteResourceRequest {
  id: string;
```

← Это отображение URL будет соответствовать ресурсам любого типа

← Это поле может представлять идентификатор для любого типа ресурса

При виде этого примера вы можете сделать вполне ожидаемый вывод: это же все сильно упрощает. Почему мы просто не делаем так вместо использования всех этих отдельных стандартных методов `Delete` для каждого ресурса? Самое интересное здесь в том, что на техническом уровне это действительно будет работать, поскольку метод сможет обрабатывать удаление, по сути, любого ресурса. Иными словами, так как идентификатор, предоставляемый в интерфейсе `ResourceRequest`, является произвольной строкой, можно передать любой валидный идентификатор ресурса, сославшись на любой из их возможных типов, вследствие чего ресурс будет удален. В некотором смысле это сродни определению в TypeScript функции с сигнатурой метода, выглядящей как `function deleteResource(resource: any): void`. Однако подобные методы могут приносить больше вреда, чем пользы.

Важно помнить, что одно только совпадение у двух методов их имен, целей, форматов запроса или сигнатуры еще не означает, что они идентичны. И когда мы представляем, что они все же одинаковы, и объединяем их, как это происходит здесь, то с большой вероятностью столкнемся с проблемами в дальнейшем.

В случае полиморфного стандартного метода `Delete` мы определенно можем удалить любой ресурс, но ведь не обязательно все ресурсы должны удаляться одинаково. Представьте, что нам нужна поддержка мягкого удаления (глава 25) или проверочные запросы (глава 27) для отдельных ресурсов, а не всех. Причем это выходит за пределы данного конкретного примера и распространяется на любой метод, который может работать с несколькими типами ресурсов.

Использование любого метода, способного работать со множеством типов ресурсов (или всеми ресурсами), подразумевает, что эти типы ресурсов действуют настолько похоже, что заслуживают одного общего метода API. Однако при этом их представления настолько разные, что они заслуживают быть полностью отдельными типами ресурсов.

Еще один важный нюанс в том, что поведение рассматриваемых ресурсов всегда будет оставаться достаточно похожим и любые изменения в желаемом поведении полиморфного метода будут применяться ко всем этим ресурсам. То есть поведение метода всегда будет изменяться для всех типов ресурсов, а не индивидуально.

В обоих этих случаях мы идем на серьезные шаги. В первом мы говорим, что ресурсы идентичны в своем поведении и достойны одного метода API, но при этом полностью различаются в своем представлении, заслуживая независимых определений ресурсов. Во втором же мы предполагаем, что это идентичное поведение является неизменным каркасом, в результате чего все ресурсы будут изменять свое поведение единым образом во всем API.

И хотя определенно может существовать сценарий, в котором описанные предположения окажутся разумными, намного вероятнее, что рано или поздно одно из них будет нарушено. И поскольку мы не можем предсказать будущее API, намного безопаснее будет предположить, что компоненты будут изменяться, причем своим уникальным образом, а не как единая группа. В связи с этим полиморфные методы API, как правило, использовать не рекомендуется.

К сожалению, это означает, что придется набирать больше текста. С другой стороны, API остается гибким и адаптируемым, не превращаясь в хрупкий и неуклюжий.

16.3.5. Итоговое определение API

В листинге 16.10 мы видим, как можно применить полиморфизм к ресурсу `Message`, получив упрощенный набор методов API, а также структурные преимущества.

Листинг 16.10. Итоговое определение API

```
abstract class ChatRoomApi {
  @post("/{parent=chatRooms/*/}/messages")
  CreateMessage(req: CreateMessageRequest): Message;
}

interface CreateMessageRequest {
  parent: string;
  resource: Message;
}

interface Message {
  id: string;
  sender: string;
  type: 'text' | 'image' | 'audio' | 'video';
  content: string | Media
}
```

```
interface Media {
    uri: string;
    contentType: string;
}
```

16.4. КОМПРОМИССЫ

Полиморфизм в API является очень эффективным инструментом, но при этом может вызывать сложности. Несмотря на то что во многих случаях он станет идеальным решением (как в примере с несколькими типами ресурсов `Message`), бывают сценарии, когда ресурсы могут выглядеть похожими, но все равно иметь фундаментальные различия. Хорошим выходом здесь будет оценивать, являются ли ресурсы поистине особыми подтипами одного обобщенного типа (как `PhotoMessage` определенно является подтипом `Message`). Причина в том, что использование данного паттерна ограничивает ресурсы конкретной реализацией, которую в дальнейшем окажется сложно (а то и невозможно) распутать, не сломав.

Помимо всего сказанного, полиморфизм в веб-API по изложенным выше причинам в идеале нужно реализовывать для ресурсов, а не методов. Несмотря на то что возможны случаи, когда полиморфный метод окажется полезен, необходимо тщательно подумать о том, какова вероятность, что в дальнейшем поведение ресурса изменится. Дело в том, что такие методы могут сделать работу более удобной и сократить объем ввода текста сейчас, но ценой тому станет жесткое ограничение гибкости API в будущем.

16.5. УПРАЖНЕНИЯ

1. Представьте, что создаете API, использующий URL-закладки браузера, которые могут быть упорядочены в коллекции или папки. Будет ли разумным поместить оба этих принципа (папку и закладку) в один полиморфный ресурс? Почему?
2. Почему для хранения типов полиморфных ресурсов необходимо задействовать строковое поле, а не что-то сродни перечислению?
3. Почему дополнительные данные нужно игнорировать (например, в случае передачи радиуса для ресурса `Shape` с типом квадрата), а не отвергать с ошибкой?
4. Почему нужно избегать полиморфных методов? Почему будет нежелательно использовать полиморфный набор стандартных методов (например, `UpdateResource()`)?

РЕЗЮМЕ

- Полиморфизм в API позволяет ресурсам выступать в роли различных типов, избавляя разработчика от необходимости дублировать общую функциональность.
- Как правило, полиморфизм разумно использовать, когда пользователям может потребоваться совместный вывод списка, состоящего из нескольких типов ресурсов (например, при выводе списка сообщений в чате, где каждое из них может иметь разный тип).
- Информация типа для полиморфного ресурса должна выражаться строковым полем, но варианты, допустимые для этого поля, следует изменять (добавлять или удалять) осторожно, чтобы не нарушить работу существующего кода.
- При предоставлении недействительных входных данных нужно не выбрасывать ошибку, а с помощью проверки определять наличие необходимых данных, игнорируя все лишние.
- Как правило, использования полиморфных методов желательно избегать, поскольку они исключают возможность дальнейшего изменения поведения.

Часть V

Коллективные операции

До этого момента большинство рассмотренных нами паттернов проектирования фокусировались одновременно на одном или двух ресурсах. В текущей же части мы разберем паттерны, предназначенные для работы с более обширными коллекциями ресурсов.

В главах 17 и 23 мы научимся копировать, перемещать, импортировать и экспортировать ресурсы между коллекциями, а также узнаем обо всех проблемах, которые сопровождают эту функциональность. В главе 18 мы освоим применение ряда стандартных методов для обработки блоков ресурсов, одновременно оперируя с их наборами, а не единичными экземплярами. И хотя можно применить эту идею к стандартному методу `Delete`, в главе 19 мы пойдем дальше, разобрав одновременное удаление множества ресурсов с помощью фильтрации коллекции (о чем будет подробнее рассказано в главе 22), а не по уникальным идентификаторам ресурсов.

В главе 20 мы узнаем, как загружать точки данных для их совокупного использования, а не отдельной адресации, как мы поступали с ресурсами до сих пор. В главе 21 мы рассмотрим обработку большого количества ресурсов управляемым способом с помощью пагинации.

17

Копирование и перемещение

В этой главе

- ✓ Применение пользовательских методов Copy и Move для реорганизации ресурсов в API.
- ✓ Выбор правильного идентификатора (или политики идентификации) для операций копирования и перемещения.
- ✓ Обработка дочерних и прочих ресурсов при копировании или перемещении родительских.
- ✓ Обработка внешних ссылок на данные для копируемых и перемещаемых ресурсов.
- ✓ Уровень атомарности, ожидаемый от этих новых пользовательских методов.

Несмотря на то что неизменяемыми считаются очень немногие ресурсы, зачастую встречаются определенные атрибуты, которые можно уверенно считать неизменными, в частности уникальный идентификатор ресурса. Но что, если мы хотим его переименовать? Как можно сделать это безопасно? А что, если нужно переместить ресурс от одного родителя к другому? А если мы хотим его дублировать? Далее мы познакомимся с безопасным способом выполнения этих операций, охватывающим и копирование (дублирование), и перемещение (изменение уникального идентификатора или смена родителя) ресурсов в API.

17.1. ЗАЧЕМ ЭТО НУЖНО

В идеальном мире иерархические связи между ресурсами спроектированы наилучшим образом и неизменны. Самое же главное, что в этом магическом мире пользователи API никогда не совершают ошибок и не создают ресурсы в неправильных местах. И они определенно никогда не спохватываются, задним числом осознав допущенную оплошность. В этом мире никогда не возникает потребности в переименовании или перемещении ресурса в API, поскольку ни мы как разработчики API, ни наши пользователи как его потребители не допускаем ошибок в схеме расположения ресурсов и их иерархии.

Этот мир мы рассматривали в главе 6 и подробно обсуждали в подразделе 6.3.6. К сожалению, это не тот мир, в котором мы существуем, а значит, нужно учитывать вероятность, что однажды пользователю API потребуется возможность перемещать ресурс под другого родителя в иерархии или изменять его ID.

К тому же все усложняют вероятные сценарии, в которых пользователям требуется дублировать ресурсы, потенциально копируя их в другие участки иерархии. И хотя оба этих случая на первый взгляд кажутся довольно простыми, как и большинство тем в проектировании API, они порождают необходимость ответить на множество различных вопросов. Цель текущего паттерна — обеспечить потребителям API возможность переименовывать и копировать ресурсы по всей их иерархической структуре безопасным, стабильным и (чаще) простым способом.

17.2. ОБЗОР

Для перемещения или переименования ресурса нельзя применять стандартный метод `Update`, поэтому лучше обратиться к пользовательским методам. К счастью, общий принцип работы этих методов копирования и перемещения довольно прост. Вот только дьявол, как обычно, кроется в деталях, таких как копирование ресурса `ChatRoom` и перемещение ресурсов `Message` между разными родительскими ресурсами `ChatRoom` (листинг 17.1).

Листинг 17.1. Примеры копирования и перемещения с помощью пользовательских методов

```

Для обоих пользовательских методов мы применяем
HTTP-глагол POST, указывая на конкретный ресурс,
который нужно переместить либо скопировать
abstract class ChatRoomApi {
    → @post("/{id=chatRooms/*/messages/*}:move")
      MoveMessage(req: MoveMessageRequest): Message;
    → @post("/{id=chatRooms/*}:copy")
      CopyChatRoom(req: CopyChatRoomRequest): ChatRoom;
}

```

В обоих пользовательских методах ответом всегда будет перемещенный либо скопированный ресурс

В этом примере не отражены несколько важных вопросов. Для начала, при копировании/перемещении ресурса выбираете ли вы уникальный идентификатор или же это делает сервис, как при создании нового ресурса? Есть ли разница при осуществлении операций между родителями (где вы можете пожелать оставить прежним идентификатор, просто принадлежащий другому родителю) и их выполнении внутри одного родителя (где вам, предположительно, просто нужно изменить ID)?

Далее, когда вы копируете ресурс `ChatRoom`, копируются ли вместе с ним все принадлежащие ему ресурсы `Message`? Что, если к ним прилагаются объемные вложения? Копируются ли эти дополнительные данные? Причем вопросы на этом не заканчиваются. Нам еще нужно выяснить, как сделать так, чтобы копируемые/перемещаемые ресурсы не находились в работе у пользователей, а также разобраться, что делать со старыми идентификаторами и со всеми наследуемыми метаданными, такими как политика контроля доступа.

Коротко говоря, несмотря на то, что этот паттерн опирается лишь на специфичный пользовательский метод, одного определения данного метода здесь будет недостаточно. В следующем разделе мы углубимся во все эти вопросы, чтобы получить поверхность API, гарантирующую безопасное и стабильное копирование/перемещение ресурсов.

17.3. РЕАЛИЗАЦИЯ

Как мы видели в разделе 17.2, для копирования и перемещения ресурсов по иерархии API можно задействовать пользовательские методы. Однако мы не обратили внимания на некоторые важные нюансы этих методов и то, как они фактически работают. Начнем с очевидного: как нужно определять идентификатор копируемого/перемещаемого ресурса?

17.3.1. Идентификаторы

Как мы узнали в главе 6, обычно лучше оставлять выбор уникальных идентификаторов самому сервису API. Это означает, что при создании новых ресурсов мы можем указывать идентификатор родительского ресурса, но сами новые ресурсы будут получать ID, выходящий за рамки нашего контроля. И на то есть весомая причина: человек склонен выбирать для ресурсов весьма неудачные идентификаторы. То же самое происходит, когда дело касается копирования/перемещения ресурсов. В некотором смысле оба этих действия сродни созданию нового ресурса, выглядящего в точности аналогично существующему, с последующим (в случае перемещения) удалением исходного.

Но даже если созданный ресурс получает идентификатор, присваиваемый сервисом API, то каким он должен быть? В этом случае наиболее удобный вариант будет зависеть от нашего намерения. Если мы хотим переименовать ресурс, чтобы он имел новый ID, но остался на том же месте в иерархии, то более подходящим скорее окажется выбирать идентификатор вручную. Это особенно верно, если API допускает пользовательские ID, поскольку мы наверняка будем менять одно подходящее имя ресурса на другое (например, нечто вроде `databases/database-prod` в `databases/database-prod-old`). С другой стороны, если мы хотим переместить что-то с одной позиции в иерархии в другую, то наверняка окажется лучше, если новый ресурс будет иметь тот же идентификатор, но принадлежать другому родителю (например, при перемещении существующего ID `chatRooms/1234/messages/abcd` в `chatRooms/5678/messages/abcd`; обратите внимание на общую часть `messages/abcd`). И поскольку эти сценарии довольно заметно различаются, мы разберем их по отдельности.

Копирование

Выбор идентификатора для дублирующего ресурса оказывается довольно простой задачей. Независимо от того, копируете вы его в того же или другого родителя, метод `Copy` должен действовать идентично стандартному методу `Create`. То есть если в вашем API выбраны пользовательские идентификаторы, метод `Copy` должен также позволять пользователю указывать целевой ID для нового ресурса. Если же в сервисе поддерживаются лишь генерируемые им самим идентификаторы, то делать исключение и разрешать пользовательские здесь не нужно. (Если разрешить, то возникнет брешь, через которую любой сможет выбирать собственные идентификаторы, просто создавая ресурс, а затем копируя его в нужное место.)

В результате запрос на копирование ресурса будет принимать и целевого родителя (если этот тип ресурса имеет родителя), и, в случае разрешенных пользовательских идентификаторов, целевой ID (листинг 17.2).

Листинг 17.2. Интерфейсы запроса на копирование

```

Всегда необходимо знать
ID копируемого ресурса
interface CopyChatRoomRequest {
    id: string;
    destinationId: string;
}

interface CopyMessageRequest {
    id: string;
    destinationParent: string;
    destinationId: string;
}

Когда у ресурса есть родитель в иерархии, необходимо
указывать, где создаваемый ресурс должен в итоге оказаться

```

Это поле не обязательно должно присутствовать, только если API поддерживает пользовательские идентификаторы

Это может привести к неожиданным результатам. Например, если API не поддерживает пользовательские идентификаторы, то запрос на копирование любого высокоуровневого ресурса требует всего одного параметра: ID самого копируемого ресурса. В противном случае есть вероятность, что мы захотим скопировать ресурс в того же родителя. Тогда поле `destinationParent` будет совпадать с родителем ресурса, на который указывает поле `id`.

Кроме того, даже в случаях, когда API поддерживает пользовательские идентификаторы, при дублировании ресурса нам может потребоваться скопировать автоматически генерируемый ID. Для этого мы просто оставим поле `destinationId` пустым, обратившись таким образом к сервису с просьбой выбрать идентификатор за нас.

Наконец, когда пользовательские идентификаторы поддерживаются, есть вероятность, что целевой ID внутри целевого родителя может оказаться занят. В этом случае может возникнуть соблазн вернуться к идентификатору, генерируемому сервером, чтобы обеспечить успешное дублирование ресурса. Однако этого нужно избегать, поскольку такой подход нарушит пользовательское представление о расположении нового ресурса. Вместо этого сервис должен возвращать эквивалент HTTP-ошибки **409 Conflict** и отменять операцию копирования.

Перемещение

Когда дело доходит до перемещения ресурсов, у нас есть два немного разных сценария. С одной стороны, пользователей может интересовать перемещение ресурса в другого родителя в иерархии, например, перемещение ресурса `Message` из одного ресурса `ChatRoom` в другой. В свою очередь, пользователи могут пожелать переименовать ресурс, технически переместив его в того же самого родителя, но с другим конечным ID. Кроме того, возможно, что нам потребуется проделывать и то и другое одновременно (перемещать и переименовывать). К тому же все усложняется необходимостью помнить, что переименование ресурсов имеет смысл, только если API поддерживает пользовательские идентификаторы.

Для обработки этих сценариев может показаться уместным снова задействовать отдельное поле, как в листинге 17.2, где у нас есть целевой родитель и целевой идентификатор. Однако в этом случае мы всегда знаем, каким именно будет итоговый идентификатор (в сравнении с методом `Copy`, когда мы можем опереться на ID, генерируемый сервером). Этот ID либо совпадает с исходным ресурсом, но имеет другого родителя, либо оказывается совершенно новым, выбранным нами. В результате мы можем использовать одно поле `destinationID` и ввести некоторые ограничения для значения этого поля, в зависимости от того, поддерживаются ли пользовательские идентификаторы. Если да, то допустимым будет любой полноценный идентификатор. Если же нет, то валидными *новыми* идентификаторами будут считаться лишь те, которые меняют только свою родительскую часть, остальную оставляя прежней.

Таким образом, мы получаем структуру запроса `Move`, который принимает только два поля: идентификатор перемещаемого ресурса и предполагаемое место его итогового размещения после завершения операции (листинг 17.3).

Листинг 17.3. Интерфейсы запроса на перемещение

```

interface MoveChatRoomRequest {
    id: string;
    destinationId: string;
}

interface MoveMessageRequest {
    id: string;
    destinationId: string;
}

```

Всегда необходимо знать ID копируемого ресурса

Поскольку ресурсы `ChatRoom` не имеют иерархических родителей, то этот метод окажется уместен только при наличии поддержки в API пользовательских идентификаторов

У ресурсов `Message` есть родители, поэтому можно переместить этот ресурс в другого родителя, изменив целевой ID

Как видите, для ресурсов верхнего уровня (не имеющих родителей) метод перемещения имеет смысл, только если API поддерживает пользовательские идентификаторы. Эта ситуация весьма отличается от метода копирования, который здесь по-прежнему окажется уместен. Проблема состоит в том, что перемещение решает две задачи: перенос ресурса в другого родителя и его переименование. Последнее имеет смысл только при наличии поддержки пользовательских ID. Первое же окажется уместным, только если у ресурса есть родитель. В случаях, когда ни одно из этих условий не выполняется, метод `Move` становится полностью бессмысленным и вообще не должен реализовываться.

Теперь, когда у нас есть представление о форме и структуре запросов на копирование и перемещение ресурсов, перейдем к обработке более сложных сценариев.

17.3.2. Дочерние ресурсы

До сих пор мы работали с условием, что каждый ресурс, который мы хотим переместить или скопировать, является полностью самостоятельным. По существу, мы исходили из того, что ресурс не содержит, помимо одной записи, никакой дополнительной информации, которую бы требовалось скопировать. Это допущение значительно упрощало работу, но, к сожалению, не всегда оказывается безопасным. Вместо этого нам нужно предположить, что с ресурсами будут связаны дополнительные данные, которые также потребуются скопировать или переместить. Как это обработать? Нужно ли вообще беспокоиться об этих лишних данных?

Коротким ответом будет «да». Проще говоря, внешние данные и связанные ресурсы (например, дочерние) редко присутствуют бесцельно. В результате копирование/перемещение ресурса без включения этих связанных данных

наверняка приведет к неожиданному результату: новому целевому ресурсу, который будет работать не так, как исходный. А поскольку одной из основных целей хорошего API является предсказуемость, то исключительно важно обеспечить, чтобы копируемый или перемещаемый ресурс оказался максимально идентичен исходному. Как же этого добиться?

Для начала, будь то копирование или перемещение, необходимо включить все дочерние ресурсы. Например, это означает, что в случае копирования/перемещения ресурса `ChatRoom` все дочерние `Message` также должны быть скопированы/перемещены, получив новый родительский ID. Очевидно, что это предполагает намного больший объем работы, чем обновление одной строки. Более того, количество необходимых обновлений зависит от количества дочерних ресурсов. То есть логично ожидать, что копирование ресурса с несколькими дочерними ресурсами займет намного меньше времени, чем аналогичная операция с похожим ресурсом, имеющим множество потомков. В табл. 17.1 показан пример идентификаторов ресурсов до и после операции копирования/перемещения.

Таблица 17.1. Новые и старые идентификаторы после копирования/перемещения ресурса `ChatRoom`

Тип ресурса	Исходный идентификатор	Новый идентификатор
ChatRoom	chatRooms/old	chatRooms/new
Message	chatRooms/old/messages/2	chatRooms/new/messages/2
Message	chatRooms/old/messages/3	chatRooms/new/messages/3
Message	chatRooms/old/messages/4	chatRooms/new/messages/4

И хотя применительно к копированию на этом сложности заканчиваются, это не относится к перемещению ресурсов. В этом случае после завершения обновления всех ID мы по факту создали совершенно новую проблему: все прежние ссылки на эти ресурсы теперь недоступны, хотя сами ресурсы по-прежнему существуют. В следующем подразделе мы узнаем, как обрабатывать такие ссылки, будь то внутренние или внешние.

17.3.3. Связанные ресурсы

Хоть мы и решили, что все дочерние ресурсы должны копироваться/перемещаться вместе с целевым ресурсом, мы ничего не сказали о связанных ресурсах. Например, представим, что наш API чата с помощью ресурса `MessageReviewReport` предоставляет отчет о сообщениях, которые могут оказаться неподходящими. Этот ресурс сродни обращению в поддержку по поводу сообщения, которое было отмечено как неподходящее и должно быть рассмотрено. Он не обязательно

должен быть потомком ресурса `Message` (или даже ресурса `ChatRoom`), но при этом должен ссылаться на соответствующий ресурс `Message` (листинг 17.4).

Листинг 17.4. Ресурс `MessageReviewReport`, ссылающийся на ресурс `Message`

```
interface MessageReviewReport {
  id: string;
  messageId: string;
  reason: string;
}
```

← Это поле содержит ID ресурса `Message`,
выступая в качестве ссылки

Существование этого ресурса (и тот факт, что его поле `messageId` является ссылкой на ресурс `Message`) вызывает два очевидных вопроса. Во-первых, должны ли при дублировании `Message` дублироваться и все ресурсы `MessageReviewReport`, ссылающиеся на исходное сообщение? Во-вторых, если ресурс `Message` перемещается, то должны ли также обновляться все `MessageReviewReport`, ссылающиеся на перемещаемый ресурс? Эти вопросы еще больше усложняются, если вспомнить, что копирование и перемещение ресурсов `Message` может быть вызвано не обязательным запросом конечного пользователя, нацеленного на ресурс `Message`, а являться результатом каскадирования запроса, нацеленного на родительский ресурс `ChatRoom`.

И неудивительно, что нет единого верного ответа о том, как лучше поступить во всех этих сценариях. Но мы начнем с самого простого и разберемся, как должны реагировать на перемещение ресурсов связанные с ними другие ресурсы.

Ссылочная целостность

Во многих реляционных базах данных есть возможность настроить систему так, чтобы в случае изменения определенных строк эти изменения могли каскадироваться и обновлять другие строки в БД. Это очень ценная, хоть и ресурсозатратная возможность, которая позволяет избежать получения в БД эквивалента ошибки сегментации при попытке разыменовать ставший недействительным указатель. Когда дело доходит до перемещения ресурсов внутри API, в идеале нам нужно стремиться к такому же результату.

К сожалению, это одна из особых сложностей, которая возникает при использовании пользовательского метода `Move`. Дело в том, что нам необходимо отслеживать не только все ресурсы, ссылающиеся на перемещаемый, но также все его дочерние ресурсы и родительский, обеспечивая параллельное обновление и их тоже. Нетрудно представить, что это оказывается невероятно сложным и является одной из многих причин, по которым переименовывать или перемещать ресурсы не рекомендуется.

Например, следование этому руководству будет означать, что если мы соберемся переместить ресурс `Move`, на который ссылается `MessageReviewReport`, то нам также потребуется обновить и сам `MessageReviewReport`. Более того,

в случае перемещения ресурса `ChatRoom`, являющегося родителем этого `Message` с `MessageReviewReport`, нам бы пришлось проделать то же самое, потому что `Message`, будучи потомком `ChatRoom`, переместился бы вслед за ним.

Далее мы кратко разберем обработку связанных ресурсов в случае пользовательского метода `Copy`.

Дублирование связанных ресурсов

Перемещение определенно порождает немало проблем в отношении связанных ресурсов, но будет ли это верно и в случае копирования? Оказывается, здесь тоже не обходится без проблем, но в данном случае они носят иной характер. Вместо сложной технической проблемы перед нами встает непростое решение в плане дизайна. И причина тут в том, что необходимость копировать связанный ресурс вместе с целевым определяется обстоятельствами.

Например, наличие ресурса `MessageReviewReport` определенно важно, но при дублировании ресурса `Message` в точности дублировать отчет нам не нужно. Пожалуй, будет разумнее позволить `MessageReviewReport` ссылаться не на один ресурс `Message`, что позволит вместо копирования отчета просто добавить скопированный ресурс `Message` в его ссылочный список.

Другие ресурсы просто не должны дублироваться. Например, при копировании ресурса `ChatRoom` никогда не нужно копировать ресурсы `User`, перечисленные как его участники. В общем, суть в том, что если копирование одних ресурсов вместе с целевым будет иметь смысл, то копирование других — нет. Эта техническая задача уже куда более простая и во многом будет зависеть от предполагаемого поведения API.

Теперь, разобравшись с подробностями сохранения ссылочной целостности при работе с новыми пользовательскими методами, мы перейдем к рассмотрению случаев, в которых речь идет о ссылках, выходящих из-под нашего контроля.

Внешние ссылки

Несмотря на то что большинство ссылок на ресурсы API будут находиться внутри других ресурсов того же API, так будет не всегда. Возможно немало сценариев, в которых на ресурсы будут указывать ссылки из разных мест интернета, в частности при сохранении файлов или прочих неструктурированных данных. Это представляет довольно очевидную проблему, учитывая, что вся текущая глава посвящена перемещению ресурсов и повсеместному нарушению таких внешних ссылок. Допустим, у нас есть хранилище, отслеживающее файлы. Эти ресурсы `File` могут быть доступны по всему интернету, что подразумевает огромное количество ссылок (листинг 17.5). Что тут можно сделать?

Листинг 17.5. На ресурсы File наверняка будут ссылаться извне API

```

abstract class FileApi {
    @post("/files")
    CreateFile(req: CreateFileRequest): File;

    @get("/{id=files/*}")
    GetFile(req: GetFileRequest): File;
}

interface File {
    id: string;
    content: Uint8Array;
}

```

← Эти URI могут использоваться для ссылки на заданный ресурс File извне API

Важно помнить, что интернет не является идеальным представлением ссылочной целостности. Мы зачастую сталкиваемся с ошибками **404 Not Found**, поэтому будет не совсем честным просто взять внутренние требования API к ссылочной целостности и расширить их на всю мировую сеть. Очень редко при раскрытии ресурса для внешнего мира мы действительно готовы гарантировать пожизненное предоставление этого ресурса в неизменном виде. Как правило, ресурс предоставляется до тех пор, пока это имеет смысл. Суть в том, что раскрытые для интернета ресурсы зачастую предоставляются по мере возможностей и редко с пожизненной гарантией.

В этом случае наш ресурс `File` лучше будет отнести к аналогичному набору принципов и предоставлять до того момента, пока его не потребуется переместить. Это может случиться завтра ввиду срочного запроса от Департамента юстиции США или же в следующем году из-за неоправданного повышения стоимости хранения файла. Дело в том, что нам нужно изначально осознавать не критичность ссылочной целостности в API и фокусироваться на более насущных задачах.

17.3.4. Внешние данные

До сих пор все ресурсы, которые мы копировали/перемещали, являлись такими, которые предполагается хранить в реляционной базе данных или в *плоскости управления*. А как быть со сценариями, когда мы хотим скопировать/переместить ресурс, указывающий на сырой блок байтов, как в нашем ресурсе `File` из листинга 17.5? Нужно ли также копировать эти байты в хранилище?

Эта проблема хорошо изучена в области информатики и всплывает во многих языках программирования в виде выбора между *копированием переменных по значению* или *по ссылке*. Когда переменная копируется по значению, все внутренние данные дублируются и скопированный экземпляр оказывается полностью отделенным от исходного. В случае же копирования по ссылке внутренние

данные остаются на прежнем месте и создается новая переменная, указывающая на эти исходные данные. В таком случае изменение данных одной переменной влечет за собой обновление и другой.

В листинге 17.6 показан пример псевдокода, который делает две копии (одну по значению, другую по ссылке) и затем обновляет получающиеся данные. Итоговые значения показаны на рис. 17.1. Как видите, несмотря на наличие трех переменных, у нас всего два списка, а изменения в переменной `copy_by_reference` также видимы в исходном списке.

Листинг 17.6. Псевдокод для копирования по ссылке и по значению

```
let original = [1, 2, 3];
let copy_by_reference = copyByReference(original);
let copy_by_value = copyByValue(original);
```

Сначала мы дважды копируем исходный список, один раз по значению и один раз по ссылке

```
copy_by_reference[1] = 'ref';
copy_by_value[1] = 'val';
```

Затем обновляем каждую копию (результаты показаны на рис. 17.1)

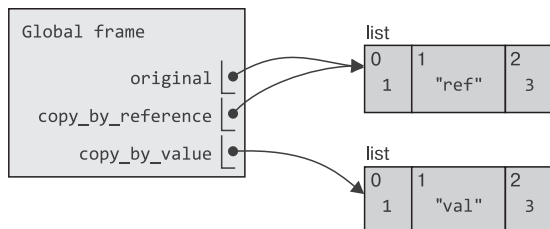


Рис. 17.1. Визуальное представление копирования по ссылке и по значению

При перемещении ресурсов, указывающих на подобные внешние данные, ответ будет прост: оставить внутренние данные как есть. То есть несмотря на то что мы можем переместить саму запись ресурса, данные должны оставаться нетронутыми. Например, переименование ресурса `File` должно вызвать изменение адреса этого ресурса, но внутренние байты перемещаться никуда не должны.

В случае копирования ресурсов ответ очевиден, но более сложен. Как правило, лучшим решением для этого типа внешних данных будет начать с копирования только по ссылке. После этого, если в будущем данные изменятся, нужно будет скопировать все внутренние данные и применить изменения, поскольку копирование всех байтов, когда достаточно скопировать ссылку, может оказаться излишним. Тем не менее здесь мы не получим отображения изменений скопированного ресурса в исходный, поэтому в случае обновления необходимо делать полную копию.

Эта стратегия, называемая *копирование при записи*, довольно распространена в системах хранения данных, и многие из них внутренне будут выполнять всю

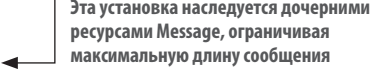
основную работу за вас. Иными словами, вы можете обойтись вызовом функции `copy()`, которая должным образом обработает всю семантику для создания полноценной копии значения только при дальнейшем изменении данных.

17.3.5. Наследуемые метаданные

Во многих API есть набор политик, или метаданных, которые дочерние ресурсы наследуют от родительского. Представим сценарий, в котором хотим контролировать длину сообщений в разных комнатах чата. Этот лимит длины для разных комнат может различаться, значит, он будет атрибутом ресурса `ChatRoom`, в конечном итоге применяемым к дочерним ресурсам `Message` (листинг 17.7).

Листинг 17.7. Ресурсы `ChatRoom` с условиями, наследуемыми их потомками

```
interface ChatRoom {
  id: string;
  // ...
  messageLengthLimit: number;
}
```



Эта установка наследуется дочерними ресурсами `Message`, ограничивая максимальную длину сообщения

Все это хорошо, но возникает путаница, когда ресурсы `Message` копируются из одного ресурса `ChatRoom` в другой. Наиболее распространенная проблема может возникнуть, когда эти разные наследованные ограничения начинают конфликтовать. Например, если `Message`, отвечающее требованиям своего текущего ресурса `ChatRoom`, копируется в другой ресурс `ChatRoom`, чьи требования уже не соответствует. Иными словами, что происходит, если мы хотим скопировать ресурс `Message`, содержащий 140 символов, в `ChatRoom`, где допускаются сообщения длиной лишь в 100 символов?

Один из вариантов — просто разрешить ресурсам нарушать правила и существовать внутри `ChatRoom`, несмотря на выход за установленные им рамки. И хотя технически это сработает, попутно возникнут другие сложности, поскольку стандартные методы `Update` в этом ресурсе будут давать сбой до тех пор, пока ресурс не впишется в правила своего родителя. Зачастую это может делать целевой ресурс, по сути, неизменяемым, внося путаницу и вызывая негодование у тех, кто заинтересован в корректировке прочих аспектов этого ресурса без изменения длины его содержимого.

Еще один вариант — обрезать или иным образом изменять входящий ресурс, чтобы он подчинялся правилам целевого родителя. И хотя технически это приемлемо, такое поведение может оказаться неожиданным для пользователей, которые не знают о требованиях целевого ресурса. В частности, в подобных случаях безвозвратного уничтожения данных такое решение с «принудительным вписыванием в требования» нарушает практические рекомендации

по построению хорошего API, внося непредсказуемость. Еще одной причиной избегать подобных решений является их необратимый характер.

Более разумным вариантом будет просто отвергать входящие ресурсы и отменять операции копирования/перемещения ввиду нарушения установленных правил. Это даст пользователям возможность решать, какие действия предпринять, чтобы операция была успешной: менять ограничение длины, указанное в ресурсе `ChatRoom`, либо обрезать/удалять ресурсы `Message`, нарушающие это ограничение, перед копированием/перемещением данных.

Это касается и случаев, когда копирование активируется ввиду того, что ресурс является потомком ресурса, связанного с тем, над которым проводится операция. В подобных сценариях любой сбой проверки или проблема, вызванная наследуемыми от целевого родителя метаданными, должны приводить к провалу всей операции, четко обозначая причину неудачи. Тогда у пользователя будет возможность изучить результаты и решить, отменить поставленную задачу или же повторить операцию после того, как обозначенная проблема будет исправлена.

17.3.6. Атомарность

На протяжении главы мы неоднократно убедились, что методы `Copy` и `Move` могут быть намного более сложными и ресурсозатратными, чем кажутся. Если в некоторых случаях они оказываются довольно безобидными (например, копирование ресурса без связанных или дочерних ресурсов), то в других могут подразумевать копирование и обновление сотен и даже тысяч других ресурсов в API. Это создает довольно серьезную проблему, поскольку в подобных ситуациях редко бывает так, чтобы операции копирования/перемещения происходили при отсутствии других пользователей API, изменяющих внутренние ресурсы. Как же обеспечить, чтобы эти операции завершались успешно при всей волатильности набора данных? Более того, если в ходе выполнения возникает ошибка, важно иметь возможность отменить работу, проделанную к этому моменту. Если коротко, то нам нужно обеспечить, чтобы и операция `Move`, и операция `Copy` происходили в контексте транзакции.

Интересно здесь то, что на уровне слоя хранения данных операции копирования и перемещения работают по-разному. При копировании мы в основном совершаем запросы для считывания данных, после чего создаем в хранилище на основе этих записей новые записи. В случае же перемещения мы в основном обновляем существующие записи в хранилище, изменяем идентификаторы для перемещения ресурсов с одного места в другое и обновляем существующие ресурсы, которые могут ссылаться на только что измененные. И хотя идеальное решение в обоих случаях одно, проблемы, возникающие в процессе, несколько различаются, в результате чего разумнее будет решать эти сценарии по-разному.

Копирование

Когда мы говорим об атомарности операции копирования ресурса (а также его дочерних и связанных ресурсов), то подразумеваем согласованность данных. То есть нам нужно обеспечить точное соответствие итоговых данных в новом месте исходным, существовавшим на момент начала операции, состояние которых может быть зафиксировано так называемым *моментальным снимком*. Если ваш API построен на основе хранилища, поддерживающего моментальные снимки или подобные транзакции, то эта задача значительно упрощается. При считывании данных можно просто указать временную метку снимка или идентификатор ревизии, после чего уже копировать эти данные в новое расположение или выполнять всю операцию внутри одной транзакции базы данных. Тогда, даже если к тому времени что-то изменится, это не вызовет никаких проблем.

Если же вам такая роскошь недоступна, есть два других варианта. Во-первых, можно просто признать, что это невозможно, и любые копируемые данные будут представлять их слепок за некий отрезок времени, а не последовательный одномоментный снимок. Это определенно неудобно и потенциально может запутывать, особенно в случае изменчивых наборов данных. Тем не менее это может оказаться единственным доступным вариантом, если учесть технологические ограничения и требования к времени работы.

Во-вторых, можно заблокировать данные для записи либо на уровне API (отключив все вызовы API, изменяющие данные), либо на уровне БД (запретив все обновления данных). И хотя такое решение не всегда окажется действенным, это своеобразный жесткий метод, который обеспечит согласованность во время операций, поскольку гарантирует сохранение данных ровно в том виде, в каком они были на момент копирования. Делается это как раз за счет блокировки и запрета изменений на время операции.

Как правило, такой вариант не рекомендуется, поскольку он, по сути, раскрывает простой способ для атаки сервиса API, при котором достаточно отправлять множество операций копирования. В итоге, если ваша система хранения не поддерживает моментальные снимки или транзакционную семантику, это станет еще одной причиной отказаться от поддержки копирования ресурсов в API.

Перемещение

В отличие от копирования, перемещение данных намного больше зависит от обновления ресурсов, в связи с чем перед нами возникают другие задачи. На первый взгляд может показаться, что у нас нет серьезных трудностей, но по факту получается, что согласованность данных все еще остается проблемой. Чтобы понять почему, представьте, что у нас есть `MessageReviewReport`, указывающий на только что перемещенное `Message`. В этом случае нужно обновить

`MessageReviewReport`, чтобы он указывал на новое расположение ресурса `Message`. Но что, если кто-то тем временем изменил `MessageReviewReport` и теперь он указывает на *другое* сообщение? Обычно нам нужно быть уверенными, что связанные ресурсы не менялись с того момента, как мы решили, нужно ли их изменить, перенаправив на перемещенный ресурс.

Для этого у нас есть способ, аналогичный случаю с операцией копирования. Первым и лучшим вариантом здесь будет использовать последовательный снимок или транзакцию БД, чтобы гарантировать выполнение работы относительно согласованного представления данных. Если так сделать не получается, то можно заблокировать базу данных или API для записи, обеспечив согласованность данных на время операции перемещения. Как уже говорилось, это опасная тактика, но иногда она может стать необходимым злом.

Наконец, можно просто проигнорировать проблему и уповать на лучшее. В отличие от операции копирования, игнорирование проблемы при перемещении приведет к намного худшим последствиям, чем просто размазывание данных по времени. Если мы не попытаемся получить последовательное представление данных во время перемещения, то можем столкнуться с риском отмены изменений, внесенных предыдущими обновлениями. Например, если `MessageReviewReport` отмечен как требующий обновления ввиду перемещения и кто-то тем временем изменит цель этого ресурса, то очень вероятно, что операция перемещения перезапишет это обновление, как если бы оно и не происходило. И хотя такой вариант может не вызвать катастрофы в каждом API, он определенно окажется неудачной практикой и его следует по возможности избегать.

17.3.7. Итоговое определение API

Как мы видели, само определение API далеко не такое сложное, как его поведение, особенно когда дело доходит до обработки наследуемых метаданных, дочерних и связанных ресурсов, а также прочих аспектов, относящихся к ссылочной целостности. Пора обобщить все рассмотренное на данный момент. В листинге 17.8 приводится заключительное определение API, который допускает копирование ресурсов `ChatRoom` (с поддержкой пользовательских идентификаторов) и перемещение ресурсов `Message` между родителями.

Листинг 17.8. Итоговое определение API

```
abstract class ChatRoomApi {
    @post("/{id=chatRooms/*}:copy")
    CopyChatRoom(req: CopyChatRoomRequest): ChatRoom;

    @post("/{id=chatRooms/*/messages/*}:move")
    MoveMessage(req: MoveMessageRequest): Message;
}
```



```

interface ChatRoom {
    id: string;
    title: string;
    // ...
}

interface Message {
    id: string;
    content: string;
    // ...
}

interface CopyChatRoomRequest {
    id: string;
    destinationParent: string;
}

interface MoveMessage {
    id: string;
    destinationId: string;
}

```

17.4. КОМПРОМИССЫ

Надеюсь, после того как вы увидели всю сложность поддержки операций копирования и перемещения, у вас возникнет устойчивое желание избегать их везде, где это возможно. И хотя поначалу они кажутся простыми, выясняется, что правильная реализация поведенческих требований и ограничений может оказаться исключительно сложной. К тому же все ухудшается возможными печальными последствиями, приводящими к потере или повреждению данных.

Тем не менее копирование и перемещение различаются как по сложности реализации, так и по своей значимости в API. Во многих случаях копирование ресурсов может являться критическим элементом функциональности. Перемещение же, наоборот, зачастую оказывается необходимым только в результате ошибки или неудачного расположения ресурсов. В итоге, несмотря на то, что поддержка метода `Copy` может оказаться весьма полезной даже для очень хороших API, при желании реализовать метод `Move` сначала желательно еще раз переоценить расположение ресурсов. Зачастую оказывается, что необходимость перемещать ресурсы между родителями (или переименовывать их) вызвана неудачным выбором идентификаторов ресурсов или использованием связи «родитель — потомок» там, где должна быть ссылочная связь.

Наконец, важно отметить: несмотря на то что реализация операций копирования и перемещения может быть затруднительной, путь наименьшего сопротивления в данном случае с гораздо большей вероятностью приведет к печальным последствиям в будущем.

17.5. УПРАЖНЕНИЯ

1. Должны ли при копировании ресурса копироваться и все его дочерние ресурсы? Касается ли это ресурсов, ссылающихся на него?
2. Как сохранить ссылочную целостность за пределами нашего API? Должны ли мы ее гарантировать?
3. Как в процессе копирования/перемещения данных сделать так, чтобы итоговые данные оказались истинной копией, а не размазанными по времени данными ввиду их параллельного изменения другими пользователями API?
4. Представьте, что перемещаете ресурс от одного родителя к другому, но у них установлены разные политики безопасности и контроля доступа. Какая из этих политик должна применяться к перемещаемому ресурсу: старая или новая?

РЕЗЮМЕ

- Как бы нам ни хотелось потребовать постоянства ресурсов, очень вероятно, что пользователям понадобится копировать или перемещать ресурсы в API.
- Вместо того чтобы реализовывать копирование/перемещение с помощью стандартных методов (**Create** и **Update**), для этого следует применять пользовательские методы **Copy** и **Move**.
- Операции копирования и перемещения также должны включать выполнение той же операции для дочерних ресурсов. Тем не менее это поведение необходимо рассматривать индивидуально, а ссылки на перемещаемые ресурсы поддерживать в актуальном виде.
- Когда ресурсы обращаются к внешним данным, методы API должны прояснять, является ли дублируемый ресурс копией по ссылке или по значению (либо копируется при записи).
- Пользовательские методы **Copy** и **Move** должны быть максимально атомарными, если учитывать ограничения внутренней системы хранения.

18

Пакетные операции

В этой главе

- ✓ Пакетные операции и их отличие от выполнения серии стандартных методов.
- ✓ Почему пакетные операции должны быть атомарными.
- ✓ Как пакетные методы запроса могут поднимать общие поля для исключения повторения.
- ✓ Изучение каждого пакетного стандартного метода (Get, Delete, Create и Update).

Этот паттерн предоставляет основные руководящие принципы, по которым пользователи API могут манипулировать множеством ресурсов пакетно, не прибегая к отдельным вызовам API. Эти так называемые *пакетные операции* действуют аналогично стандартным методам (рассмотренным в главе 7), но исключают необходимость во множестве двухсторонних взаимодействий, осуществляемых большим количеством вызовов API. Конечным результатом оказывается набор методов, позволяющих извлекать, обновлять или удалять коллекцию ресурсов с помощью одного вызова API.

18.1. ЗАЧЕМ ЭТО НУЖНО

До этого момента большинство изученных нами паттернов проектирования и руководств фокусировались на взаимодействии с отдельными ресурсами. В главе 8 мы даже пошли глубже, изучив адресацию отдельных полей в отдельном

ресурсе. И хотя до сих пор такой способ показывал свою эффективность, все еще остается пробел в противоположном конце диапазона. Что, если нам нужно оперировать более обширно, охватывая множество ресурсов одновременно?

В типичной базе данных это реализуется с помощью транзакций. То есть если нам нужно одновременно работать с несколькими разными строками БД, мы просто начинаем транзакцию, работаем как обычно (но внутри контекста транзакции) и фиксируем эту транзакцию. В зависимости от используемой в базе данных функциональности блокировки и уровня изменчивости внутренних данных транзакция может закончиться успехом или отказом, но основная суть здесь в атомарности. Операции, содержащиеся в транзакции, все либо завершатся неудачей, либо будут успешными. Здесь невозможен сценарий частичного успеха, когда одни операции выполняются успешно, а другие нет.

К сожалению (или нет, смотря кого спрашивать), большинство API не предоставляют эту обобщенную функциональность транзакций — если только это не API для сервиса хранения транзакций. И причина тому в исключительной сложности реализации такой возможности. Однако это не лишает веб-API потребности в аналогичной, схожей с транзакциями семантике. Один из типичных сценариев — когда пользователь API хочет обновить два отдельных ресурса, но при этом ему нужно, чтобы два этих независимых запроса к API оба либо выполнились успешно, либо завершились неудачей. К примеру, у нас есть ресурс настройки логирования `ChatRoom`, который можно активировать и отключать. Мы не хотим его активировать до начала использования, но при этом не хотим и обращаться к нему как к предустановленной конфигурации логирования до его активации. Такая своеобразная уловка-22 является одной из основных причин существования транзакционных семантик, поскольку без них нашим единственным вариантом будет попытаться выполнить эти две независимые операции (активировать конфигурацию и присвоить ее как предустановленную) максимально близко друг к другу.

Очевидно, что этого недостаточно, но тут возникает серьезный вопрос: что же можно сделать? Как нам добиться видимости атомарности или транзакционной семантики среди множества запросов API, не выстраивая целую систему транзакций, как это происходит в большинстве реляционных баз данных?

18.2. ОБЗОР

Как бы ни было интересно разработать полномасштабный обобщенный паттерн транзакций, это определенно будет излишним. Как же извлечь максимальную пользу из потраченного времени? Иными словами, как сократить необходимый объем работы, сохранив возможность работать со множеством ресурсов атомарно?

Благодаря этому паттерну проектирования мы научимся предоставлять ограниченную версию этой транзакционной семантики, определяя несколько

пользовательских методов, аналогичных стандартным методам из главы 7, которые позволят атомарно оперировать произвольными группами ресурсов, называемыми *пакетами*. Названия этих методов будут схожи с названиями стандартных (например, `BatchDeleteMessages`), но при этом их реализации будут несколько иными в зависимости от того, какой из стандартных методов будет выполняться для пакета (листинг 18.1).

Листинг 18.1. Пример пакетного метода Delete

```
abstract class ChatRoomApi {
    @post("/{parent=chatRooms/*/}/messages:batchDelete")
    BatchDeleteMessages(req: BatchDeleteMessagesRequest): void;
}
```

Как и с большинством подобных идей, сначала нам нужно ответить на ряд вопросов. Например, какой HTTP-глагол использовать для каждого из этих методов? Должен ли это всегда быть `POST`, как того требуют пользовательские методы (см. главу 9)? Или же выбранный HTTP-глагол должен соответствовать каждому отдельному стандартному методу (например, `GET` для `BatchGetMessages` и `DELETE` для `BatchDeleteMessages`)? Или же это зависит от самого метода?

И насколько серьезны мы будем в своих требованиях к атомарности? Если я попытаюсь извлечь пакет ресурсов `Message`, используя `BatchGetMessages`, а за это время один из ресурсов будет удален, должна ли вся операция завершаться отказом? Или же можно пропустить его и вернуть те, что до сих пор существуют? Что насчет работы с ресурсами в нескольких родителях? Другими словами, должна ли существовать возможность удалять ресурсы `Message` из нескольких `ChatRoom`?

В текущем паттерне мы узнаем, как определять эти пакетные методы и как они работают; кроме того, мы разберем все эти любопытные пограничные случаи, с которыми неизбежно будем сталкиваться при использовании этой функциональности в API.

18.3. РЕАЛИЗАЦИЯ

Как только что было сказано в разделе 18.2, для поддержки пакетных операций будут использоваться методы со специальными именами. Они представляют просто пакетные версии стандартных методов, за одним исключением: у стандартного метода `List` пакетная версия отсутствует. В результате мы получаем следующий список:

- `BatchGet<Resources>()`;
- `BatchCreate<Resources>()`;
- `BatchUpdate<Resources>()`;
- `BatchDelete<Resources>()`.

И хотя далее мы будем подробно разбирать каждый из них, определенно стоит пройтись по некоторым общим для всех них аспектам, начав с наиболее важного: атомарности.

18.3.1. Атомарность

Говоря, что набор операций является *атомарным*, мы имеем в виду, что они неотделимы друг от друга. До сих пор все рассмотренные нами операции сами по себе считались атомарными. Все же невозможно представить себе стандартный метод `Create`, который завершается сбоем частично. Ресурс либо создается, либо нет, тут нет среднего варианта. К сожалению, способ, с помощью которого мы проектировали стандартные методы как атомарные, не гарантирует этого в коллективном случае. Цель этих пакетных операций — расширить тот же принцип атомарности на методы, выполняемые для множества ресурсов, а не для одного.

Достижение этой цели означает, что действие должно быть атомарным, даже когда это оказывается не очень удобно. Представьте, к примеру, что мы пытаемся извлечь пакет ресурсов `Message` с помощью метода `BatchGetMessages`. Если извлечение даже одного из этих ресурсов приведет к ошибке, то весь запрос тоже должен вернуть ошибку. И это должно работать, о каком бы количестве сбоев ни шла речь: об одном из пяти или об одном из десяти тысяч.

Причина тому двояка. Для начала во многих случаях (как при обновлении пакетов ресурсов) весь смысл операции заключается в ее атомарности. Нам нужно быть уверенными, что применятся либо все изменения, либо никакие. И хотя это не соответствует *в точности* сценарию пакетного извлечения (нас может устроить получение значения `null` для несуществующих элементов), отсюда вытекает вторая причина. Если нам нужна поддержка частичного успеха, то интерфейс для управления результатами может стать довольно запутанным и сложным. Вместо этого, гарантируя либо полный успех, либо единую ошибку, этот интерфейс всегда ориентирован на абсолютный успех и просто возвращает список ресурсов, включенных в пакет.

В результате все пакетные методы, которые мы изучим в рамках этого паттерна, всегда будут являться полностью атомарными. Сообщения ответа будут спроектированы так, чтобы отражать либо полный успех, либо полный сбой; промежуточные варианты исключены.

18.3.2. Операции над коллекцией

Как мы узнали в подразделе 9.3.2, когда речь идет о пользовательских методах, взаимодействующих с коллекциями, есть два способа выразить их URL. Можно либо установить целью сам родительский ресурс и включить в имя метода

название ресурса, с которым он взаимодействует, либо оставить отражающую пользовательское действие часть в виде глагола и оперировать самой коллекцией. В общем виде эти варианты представлены в табл. 18.1.

Таблица 18.1. Варианты целей и соответствующие URL для пакетных методов

Цель	URL
Родительский ресурс	/chatRooms/*:batchUpdateMessages
Коллекция	/chatRooms/*/messages:batchUpdate

То, на что указывалось в главе 9, остается актуальным и здесь. Когда мы работаем с коллекцией, содержащей множество однотипных ресурсов, практически всегда будет лучше работать с самой коллекцией. Это будет верным для всех пакетных методов, которые мы разберем: они все нацеливаются на коллекцию, в связи с чем их URL оканчиваются на :batch<Method>.

18.3.3. Упорядочивание результатов

Когда мы работаем с произвольной группой ресурсов, их нужно отправлять в своеобразном запросе. В некоторых случаях это может быть просто список уникальных идентификаторов (например, при извлечении или удалении ресурсов), но в других отправлять нужно будет сами ресурсы (например, при их создании или удалении). Хотя во всех этих случаях более важным становится сохранение порядка ресурсов.

Наиболее наглядно эту важность демонстрирует пример, когда мы создаем пакет новых ресурсов и при этом не выбираем идентификаторы сами, а оставляем это действие сервису. Поскольку у нас нет предопределенного способа идентификации этих новых ресурсов, проще всего будет использовать их индекс в повторяющемся поле переданных в запросе ресурсов. Если же порядок не сохранить, то нам придется проводить углубленное сравнение между переданными ресурсами и возвращенными после создания пакета, чтобы сопоставить полученные элементы с их присвоенными сервером идентификаторами (листинг 18.2).

Листинг 18.2. Пример кода для сопоставления запрошенных ресурсов

```
let chatRoom1 = ChatRoom({ title: "Chat 1", description: "Chat 1" });
let chatRoom2 = ChatRoom({ title: "Chat 2", description: "Chat 2" });

let results = BatchCreateChatRooms({ resources: [chatRoom1, chatRoom2] });

chatRoom1 = results.resources[0];
chatRoom2 = results.resources[1];
```

← Когда порядок известен, можно легко связать результаты с запрошенным действием

```
for (let resource of results.resources) {
  if (deepCompare(chatRoom1, resource))
    chatRoom1 = resource;
  else if (deepCompare(chatRoom2, resource))
    chatRoom2 = resource;
}
```

Если же порядок неизвестен, то приходится выполнять полноценное сравнение на предмет совпадения каждого поля ресурса (кроме поля ID), чтобы убедиться в их правильности

В результате очень важно, чтобы ресурсы, возвращаемые пакетным методом, находились в том же порядке, в каком передавались.

18.3.4. Общие поля

Когда дело доходит до пакетных операций, у нас есть две стратегии. Первая, более простая, — рассматривать пакетную версию запроса просто как список обычных запросов, каждый из которых нацелен на свой ресурс. Второй вариант несколько сложнее, но зато может уменьшить повторение кода за счет поднятия релевантных запросов полей и перечисления только их (листинг 18.3).

Листинг 18.3. Стратегии поднятия полей и использования повторяющихся запросов

```
interface GetMessageRequest {
  id: string;
}
```

```
interface BatchGetMessagesRequest {
  requests: GetMessageRequest[];
}
```

Здесь у нас просто список интерфейсов GetMessageRequest

```
interface BatchGetMessagesRequest {
  ids: string[];
}
```

А здесь мы поднимаем релевантное поле (id) и вкладываем его в виде списка ID для пакета

На деле же нам придется задействовать обе эти стратегии, в зависимости от конкретного метода. Дело в том, что упрощенный вариант (предоставление списка запросов) намного лучше подходит для случаев, в которых требуется пользовательская настройка (например, когда нужно создать ресурсы, которые могут иметь родителей), а подход с поднятием полей из запроса намного уместнее в более простых взаимодействиях, таких как извлечение или удаление ресурсов, поскольку не требует никакой дополнительной информации.

К тому же сами эти стратегии не являются взаимоисключающими. В ряде случаев мы будем поднимать некоторые значения, попутно задействуя списки запросов для передачи важной информации. Хороший пример подобного сценария — это обновление пакета ресурсов, когда мы опираемся на повторяющийся список запросов обновления, но также поднимаем поле родителя и потенциально маску этого поля, чтобы обработать частичные обновления пакета.

Такое совмещение стратегий выглядит привлекательным, однако вызывает важный вопрос: что происходит, когда поднимаемое поле отличается от поля, установленного в любом из отдельных запросов? Иными словами, что, если мы поднимаем поле `parent`, которое установлено на `ChatRoom 1`, а в одном из создаваемых ресурсов поле `parent` установлено на `ChatRoom 2`? Краткий ответ следует принципу быстрого и неделимого отказа: API должен просто выбросить ошибку и отклонить запрос. И хотя здесь можно посчитать умышленным перепределение ресурсом такого поднимаемого поля, весьма вероятно, что в коде просто допущена семантическая ошибка, а оперирование с группами ресурсов определенно не подходит для угадывания намерений пользователя. Вместо этого, если пользователь планирует изменять поднятое поле в определенных ресурсах, он должен оставить его пустым или установить для него значение подстановочного символа, о чем мы поговорим ниже.

18.3.5. Оперирование с разными родителями

Одна из наиболее типичных причин использования списка отдельных запросов вместо поднятия полей вверх в пакетном запросе заключается в потребности оперировать множеством ресурсов, которые также могут принадлежать ко множеству разных родителей. В качестве примера в листинге 18.4 показан один из вариантов определения метода `BatchCreateMessages`, который поддерживает создание ресурсов в разных родителях, но делает это необычным (и неверным) способом.

Листинг 18.4. Неверный способ создания ресурсов в нескольких родителях с помощью пакетного метода

```
interface Message {
  id: string;
  title: string;
  description: string;
}

interface CreateMessageRequest {
  parent: string;
  resource: Message;
}

interface BatchCreateMessageRequest {
  parents: string[];
  resources: Message[];
}
```

Нам нужен список родителей, которым будут принадлежать эти ресурсы Message

Здесь мы подняли ресурсы Message из интерфейса CreateMessageRequest

Как видите, код опирается на список создаваемых ресурсов, но нам также нужно знать родителя каждого из них. А поскольку это поле в самом ресурсе не существует, теперь нам нужен способ, позволяющий отслеживать родителя каждого ресурса. Здесь эта потребность реализована с помощью второго списка родителей. И хотя технически такой вариант справляется с задачей, он может оказаться

очень неудобным в работе ввиду строгой зависимости от сохранения порядка в этих двух списках. Как же обработать эту пакетную операцию среди родителей?

Проще всего задействовать для родителя подстановочное значение и позволить самому родителю изменяться в списке запросов. В данном случае мы стандартизируем в качестве символа подстановки дефис (-), который будет означать «среди нескольких родителей» (листинг 18.5).

Листинг 18.5. HTTP-запрос для создания двух ресурсов с разными родителями

```
POST /chatRooms/-/messages:batchCreate HTTP/1.1
Content-Type: application/json
```

Множественность возможных
родителей, которые будут
определяться в запросе,
мы обозначаем дефисом

```
{
  "requests": [
    {
      "parent": "chatRooms/1",
      "resource": { ... }
    },
    {
      "parent": "chatRooms/2",
      "resource": { ... }
    }
  ]
}
```

Фактического родителя мы
определяем в списке запросов
на создание ресурсов

Этот же механизм нужно использовать для всех пакетных методов, которым может потребоваться работать с ресурсами, принадлежащими разным родителям. Однако, как и в случае с поднятием полей, здесь возникает вопрос о конфликтующих значениях: что, если в URL указан один родитель, но при этом некоторые его поля конфликтуют на уровне запроса? Что делать в таком случае?

В подобной ситуации API должен действовать в точности так, как описано в подразделе 18.3.4: если возникает конфликт, то отклонять запрос как недействительный и ожидать от пользователя прояснения в виде другого вызова API. И причины тому аналогичны указанным ранее: намерения пользователя неясны и работа с (потенциально большой) группой ресурсов определенно не тот случай, когда стоит пытаться их угадать.

Теперь можно наконец перейти и к позитивной ноте: определению самих методов и изучению принципа действия каждого из них. Начнем с одного из простейших — пакетного `Get`.

18.3.6. Пакетный метод `Get`

Назначение пакетного метода `Get` — извлекать из API набор ресурсов на основе предоставления группы уникальных идентификаторов. Поскольку этот принцип аналогичен стандартному методу `Get` и является идемпотентным, пакетный `Get` будет задействовать HTTP-глагол `GET` (листинг 18.6). Однако это создает

проблему в том плане, что глаголы GET обычно не имеют тела запроса. В результате список идентификаторов предоставляется в строке запроса, которую мы видели при рассмотрении масок полей в главе 8.

Листинг 18.6. Пример пакетного метода Get

```
abstract class ChatRoomApi {
    @get("/chatrooms:batchGet")
    BatchGetChatRooms(req: BatchGetChatRoomsRequest):
        BatchGetChatRoomsResponse;

    @get("/{parent=chatRooms/*/}/messages:batchGet")
    BatchGetMessages(req: BatchGetMessagesRequest):
        BatchGetMessagesResponse;
}

interface BatchGetChatRoomsRequest {
    ids: string[];
}

interface BatchGetChatRoomsResponse {
    resources: ChatRoom[];
}

interface BatchGetMessagesRequest {
    parent: string;
    ids: string[];
}

interface BatchGetMessagesResponse {
    resources: Message[];
}
```

Поскольку ресурсы ChatRoom не имеют родителя, то в пакетном запросе Get нет места для его указания

Поскольку у ресурсов Message есть родители, то в запросе для их указания (или предоставления символа подстановки) предусмотрено место

Ответ всегда содержит список ресурсов в том же порядке, в каком запрашивались ID

Все пакетные запросы Get указывают список ID для извлечения

Как видите, в сообщении запроса необходимо указывать *что-либо* в качестве родителя, только если группа ресурсов не относится к верхнему уровню, в связи с чем родителя не имеет. Если мы хотим ограничить извлечение до лишь одного родителя, то перед нами очевидный ответ: просто использовать его идентификатор. Если же мы хотим обеспечить возможность извлечения ресурсов от нескольких разных родителей, то должны задействовать в качестве подстановочного символа дефис, как это было показано в подразделе 18.3.5.

И хотя может возникнуть соблазн всегда разрешать извлечение из множества родителей, необходимо хорошо подумать, насколько это имеет смысл в конкретном случае. Все данные могут находиться в одной БД сейчас, но в дальнейшем система хранения может расшириться и распределить эти данные по многим местам. Поскольку очевидным выбором в качестве ключа распределения является родительский ресурс, а обращение к распределенному хранилищу за пределами области одного такого ключа становится дорогостоящим или сложным, то извлечение ресурсов из множества родителей может стать исключительно

дорогостоящим или даже невозможным. В таких случаях единственным вариантом становится запрет использования символа подстановки для родителя, что практически всегда станет критическим изменением (см. главу 24). И запомните: как мы видели в подразделе 18.3.5, если ID, предоставленный в списке, будет конфликтовать с указанным явно родителем (то есть любым значением родителя, не являющимся символом подстановки), то запрос должен отклоняться как недействительный.

Далее, как сообщалось в подразделе 18.3.1, критически важно, чтобы этот метод был абсолютно атомарным, даже если это неудобно. То есть даже когда всего 1 из 100 ID по любой причине оказывается недействительным (например, не существует или запрашивающий пользователь не имеет к нему доступа), запрос должен потерпеть полный крах. Когда вас интересуют менее строгие условия, будет гораздо лучше использовать стандартный метод `List` с фильтром, применяемым для сопоставления с одним из возможных идентификаторов (подробнее в главе 22).

Кроме того, как мы узнали в подразделе 18.3.4, если нам нужна поддержка частичного извлечения ресурсов, как в случае запроса только одного поля из каждого заданного ID, то можно поднять маску поля точно так же, как мы делали с полем родителя (листинг 18.7). Причем ко всем извлекаемым ресурсам должна применяться только одна такая маска. Это не должен быть список масок для применения одной из них к каждому из указанных ресурсов.

Листинг 18.7. Добавление в пакетный запрос `Get` поддержки частичного извлечения

```
interface BatchGetMessagesRequest {
    parent: string;
    ids: string[];
    fieldMask: FieldMask;
}
```

← Чтобы включить частичное извлечение, пакетный запрос должен содержать одну маску полей, применяемую ко всем извлекаемым ресурсам

В завершение важно сказать, что хоть ответ на запрос и может стать довольно большим, пакетные методы не должны реализовывать пагинацию (см. главу 21). Вместо этого в них нужно определять и документировать верхний предел допустимого для извлечения количества ресурсов. Этот лимит может отличаться в зависимости от размера каждого отдельного ресурса, но обычно его желательно минимизировать, чтобы снизить вероятность чрезмерно обширных ответов, способных навредить быстрдействию сервера API или создать трудности для обработки клиентами API.

18.3.7. Пакетный метод `Delete`

По аналогии с пакетной операцией `Get` пакетное удаление оперирует списком идентификаторов, но уже ради совсем другой цели. Его задача — не извлекать все эти ресурсы, а удалить каждый из них (листинг 18.8). Эта операция будет

опираться на HTTP-глагол `POST`, подчиняясь принципам действия пользовательских методов, которые мы изучили в главе 9. А поскольку запрос должен быть атомарным, удаляющим все перечисленные ресурсы либо возвращающим ошибку, возвращаемым в итоге типом будет `void`, который представляет пустой результат.

Листинг 18.8. Пример пакетного метода Delete

```
abstract class ChatRoomApi {
    @post("/chatrooms:batchDelete")
    BatchDeleteChatRooms(req: BatchDeleteChatRoomsRequest): void;

    @post("/{parent=chatRooms/*/}/messages:batchDelete")
    BatchDeleteMessages(req: BatchDeleteMessagesRequest): void;
}
// Запросы ресурсов верхнего уровня
// не содержат поля для указания родителя
interface BatchDeleteChatRoomsRequest {
    ids: string[];
}

interface BatchDeleteMessagesRequest {
    parent: string;
    ids: string[];
}
// Поскольку ресурсы Message относятся
// не к верхнему уровню, то требуется
// поле для хранения значения родителя

Аналогично пакетному Get пакетный Delete получает
список идентификаторов, а не сами ресурсы
```

В большинстве своих аспектов этот метод похож на пакетный `Get`, который мы только что разобрали в подразделе 18.3.6. К примеру, ресурсы верхнего уровня могут не иметь родительского поля при том, что у других оно будет. И когда это поле указано, оно должно либо соответствовать родителю ресурсов, чьи ID предоставляются, либо возвращать ошибку. Более того, как и в случае с пакетными операциями `Get`, удаление по нескольким родителям можно выполнять, используя в родительском поле символ подстановки. Однако его применение нужно тщательно обдумывать, чтобы не столкнуться в дальнейшем с проблемами при работе с распределенными базами данных.

Отдельно здесь стоит еще раз заострить внимание на атомарности: пакетные операции удаления должны либо удалять все перечисленные ресурсы, либо завершаться полным отказом. То есть если один ресурс из ста по любой причине удалить невозможно, то завершаться неудачей должна вся операция. Сюда относятся и случаи, когда ресурс уже удален и не существует, что и являлось изначальной целью операции. Причина этому была подробно раскрыта в главе 7 и состоит в том, что нам нужно рассматривать операцию удаления как императивную, а не декларативную. То есть у нас должна быть возможность выразить не только то, что ресурс, который мы хотели удалить, больше не существует, но и то, что не существует он ввиду выполнения этой конкретной операции, а не той, которая была выполнена какое-то время назад.

Теперь, когда мы разобрали операции на основе ID, пора переходить к более сложным сценариям, начав с пакетного `Create`.

18.3.8. Пакетный метод `Create`

Как и в случае со всеми другими пакетными действиями, цель пакетной операции `Create` — атомарно создать несколько ранее не существовавших ресурсов. В отличие от уже пройденных нами более простых методов (пакетные `Get` и `Delete`), пакетный `Create` будет задействовать более сложную стратегию получения списка стандартных запросов `Create` вместе с поднятым полем `parent` (листинг 18.9).

Листинг 18.9. Пример пакетного метода `Create`

```
abstract class ChatRoomApi {
    @post("/chatrooms:batchCreate")
    BatchCreateChatRooms(req: BatchCreateChatRoomsRequest):
        BatchCreateChatRoomsResponse;

    @post("/{parent=chatRooms/*/}/messages:batchCreate")
    BatchCreateMessages(req: BatchCreateMessagesRequest):
        BatchCreateMessagesResponse;
}

interface CreateChatRoomRequest {
    resource: ChatRoom;
}

interface CreateMessageRequest {
    parent: string;
    resource: Message;
}

interface BatchCreateChatRoomsRequest {
    requests: CreateChatRoomRequest[];
}
interface BatchCreateMessagesRequest {
    parent: string;
    requests: CreateMessageRequest[];
}

interface BatchCreateChatRoomsResponse {
    resources: ChatRoom[];
}

interface BatchCreateMessagesResponse {
    resources: Message[];
}
```

Если ресурс не верхнего уровня, то мы также включаем поле `parent` (в котором может использоваться символ подстановки)

Вместо списка ID ресурсов мы используем список стандартных запросов `Create`

Аналогично остальным пакетным методам поля `parent` имеют смысл только для ресурсов не верхнего уровня (в данном случае ресурса `ChatRoom`), и ограничения остаются теми же (если предоставляется не подстановочный родитель, то он должен согласовываться со всеми создаваемыми ресурсами). Основное различие состоит в форме входящих данных. В этом случае вместо того, чтобы вытаскивать поля из стандартного запроса `Create` прямо в пакетный `Create`, мы включаем стандартные запросы `Create` непосредственно.

Это может показаться необычным, но на то есть довольно важная причина, связанная с родителем. Если коротко, то нам нужно обеспечить возможность атомарного создания нескольких ресурсов, которые могут иметь разных родителей. К сожалению, родитель ресурса обычно не хранится в качестве поля в самом ресурсе при его создании. Вместо этого он зачастую предоставляется в стандартном запросе `Create`, формируя ID, включающий родительский ресурс в виде корня (например, `chatRooms/1/messages/2`). Это ограничение означает, что если мы хотим сделать возможным создание ресурсов в разных родителях, не требуя генерируемых клиентом идентификаторов, то нужно предоставлять сам ресурс и его предполагаемого родителя, что как раз и делает интерфейс стандартного запроса `Create`.

Еще одно сходство с другими пакетными методами заключается в необходимости сохранять порядок. Точно так же, как и прочие методы, создаваемые ресурсы должны располагаться в том же порядке, в каком предоставлялись с запросом. И в данном случае это даже более важно, чем в других пакетных методах, поскольку здесь идентификаторы создаваемых ресурсов могут еще не существовать. То есть могут возникнуть сложности с поиском новых ID для созданных ресурсов, если они вернутся не в том порядке, в каком были переданы.

Ну и последним мы рассмотрим пакетный метод `Update`, который очень похож на пакетный метод `Create`.

18.3.9. Пакетный метод `Update`

Цель пакетного метода `Update` — изменить группу ресурсов в рамках одной атомарной операции. И аналогично пакетному `Create` пакетный `Update` будет опираться на список стандартных запросов `Update`. Разница в данном случае в том, что смысл использования списка запросов вместо списка самих ресурсов довольно очевиден и заключается в частичном обновлении.

При обновлении ресурса нам может потребоваться контроль над обновлением конкретных полей, и довольно часто бывает так, что в разных ресурсах может требоваться обновление разных полей. В результате можно рассматривать маску полей точно так же, как мы рассматривали поле `parent` при разработке пакетного метода `Create`. То есть нам нужна возможность применять к различным обновляемым ресурсам различные маски полей. При этом они, естественно,

не должны конфликтовать, и если в пакетном запросе установлена маска, то маски в отдельных его запросах должны либо совпадать, либо оставаться пустыми. Пример пакетного метода `Update` приведен в листинге 18.10.

Листинг 18.10. Пример пакетного метода `Update`

```
abstract class ChatRoomApi {
    @post("/chatrooms:batchUpdate")
    BatchUpdateChatRooms(req: BatchUpdateChatRoomsRequest):
        BatchUpdateChatRoomsResponse;

    @post("/{parent=chatRooms/*/}messages:batchUpdate")
    BatchUpdateMessages(req: BatchUpdateMessagesRequest):
        BatchUpdateMessagesResponse;
}
```

← Несмотря на то что стандартный метод `Update` использует HTTP-глагол `PATCH`, мы задействуем HTTP-глагол `POST`

```
interface UpdateChatRoomRequest {
    resource: ChatRoom;
    fieldMask: FieldMask;
}
```

```
interface UpdateMessageRequest {
    resource: Message;
    fieldMask: FieldMask;
}
```

```
interface BatchUpdateChatRoomsRequest {
    requests: UpdateChatRoomRequest[];
    fieldMask: FieldMask;
}
```

```
interface BatchUpdateMessagesRequest {
    parent: string;
    requests: UpdateMessageRequest[];
    fieldMask: FieldMask;
}
```

```
interface BatchUpdateChatRoomsResponse {
    resources: ChatRoom[];
}
```

```
interface BatchUpdateMessagesResponse {
    resources: Message[];
}
```

→ Для частичных обновлений можно поднять маску полей в пакетный запрос, если она будет одинакова для всех ресурсов

← Как и в случае с пакетным методом `Create`, мы используем список запросов, а не ресурсов

Кроме того, обратите внимание, что хоть в стандартном методе `Update` и используется HTTP-глагол `PATCH`, во избежание любых потенциальных конфликтов (и нарушения каких-либо стандартов) пакетная версия `Update`, аналогично всем пользовательским методам, использует HTTP-глагол `POST`.

18.3.10. Итоговое определение API

На этом можно подвести итог, взглянув на итоговое определение API для поддержки пакетных методов в нашем интерфейсе чата. В листинге 18.11 показано, как можно объединить все пакетные методы, чтобы поддерживать широкую функциональность, оперируя потенциально огромными группами ресурсов с помощью единичных вызовов API.

Листинг 18.11. Итоговое определение API

```
abstract class ChatRoomApi {
    @post("/chatrooms:batchCreate")
    BatchCreateChatRooms(req: BatchCreateChatRoomsRequest):
        BatchCreateChatRoomsResponse;

    @post("/{parent=chatRooms/*/}/messages:batchCreate")
    BatchCreateMessages(req: BatchCreateMessagesRequest):
        BatchCreateMessagesResponse;

    @get("/chatrooms:batchGet")
    BatchGetChatRooms(req: BatchGetChatRoomsRequest):
        BatchGetChatRoomsResponse;

    @get("/{parent=chatRooms/*/}/messages:batchGet")
    BatchGetMessages(req: BatchGetMessagesRequest):
        BatchGetMessagesResponse;

    @post("/chatrooms:batchUpdate")
    BatchUpdateChatRooms(req: BatchUpdateChatRoomsRequest):
        BatchUpdateChatRoomsResponse;

    @post("/{parent=chatRooms/*/}/messages:batchUpdate")
    BatchUpdateMessages(req: BatchUpdateMessagesRequest):
        BatchUpdateMessagesResponse;

    @post("/chatrooms:batchDelete")
    BatchDeleteChatRooms(req: BatchDeleteChatRoomsRequest): void;

    @post("/{parent=chatRooms/*/}/messages:batchDelete")
    BatchDeleteMessages(req: BatchDeleteMessagesRequest): void;
}

interface CreateChatRoomRequest {
    resource: ChatRoom;
}

interface CreateMessageRequest {
    parent: string;
    resource: Message;
}
```

330 Часть V. Коллективные операции

```
interface BatchCreateChatRoomsRequest {  
  requests: CreateChatRoomRequest[];  
}
```

```
interface BatchCreateMessagesRequest {  
  parent: string;  
  requests: CreateMessageRequest[];  
}
```

```
interface BatchCreateChatRoomsResponse {  
  resources: ChatRoom[];  
}
```

```
interface BatchCreateMessagesResponse {  
  resources: Message[];  
}
```

```
interface BatchGetChatRoomsRequest {  
  ids: string[];  
}
```

```
interface BatchGetChatRoomsResponse {  
  resources: ChatRoom[];  
}
```

```
interface BatchGetMessagesRequest {  
  parent: string;  
  ids: string[];  
}
```

```
interface BatchGetMessagesResponse {  
  resources: Message[];  
}
```

```
interface UpdateChatRoomRequest {  
  resource: ChatRoom;  
  fieldMask: FieldMask;  
}
```

```
interface UpdateMessageRequest {  
  resource: Message;  
  fieldMask: FieldMask;  
}
```

```
interface BatchUpdateChatRoomsRequest {  
  parent: string;  
  requests: UpdateChatRoomRequest[];  
  fieldMask: FieldMask;  
}
```

```

interface BatchUpdateMessagesRequest {
    requests: UpdateMessageRequest[];
    fieldMask: FieldMask;
}

interface BatchUpdateChatRoomsResponse {
    resources: ChatRoom[];
}

interface BatchUpdateMessagesResponse {
    resources: Message[];
}

interface BatchDeleteChatRoomsRequest {
    ids: string[];
}

interface BatchDeleteMessagesRequest {
    parent: string;
    ids: string[];
}

```

18.4. КОМПРОМИССЫ

Для многих из этих пакетных методов мы приняли довольно специфичные решения, которые оказали важное влияние на итоговое поведение API. Начнем с того, что все эти методы ставят во главу угла атомарность, даже когда это может вызывать неудобства. К примеру, если мы хотим удалить несколько ресурсов, один из которых уже удален, то терпит неудачу весь пакетный метод `Delete`. Это важный компромисс, который дает возможность избежать трудностей работы с API, которые поддерживают возвращение частично успешных результатов, позволяя вместо этого придерживаться поведения транзакционной семантики, характерной для большинства современных баз данных. И хотя такой подход может привести к неприятным моментам в типичных сценариях, он гарантирует, что методы API будут максимально согласованны и просты, сохраняя при этом важную функциональность.

Далее, несмотря на возможную несогласованность в формате выходных данных (иногда это будут сырые ID, а иногда интерфейсы стандартных запросов), дизайн этих методов ставит простоту над согласованностью. Например, можно было использовать повторяющийся список стандартных запросов для всех пакетных методов. Однако некоторые из этих методов (в частности, пакетные `Get` и `Delete`) в случае предоставления идентификаторов содержали бы ненужный промежуточный уровень. Подняв эти значения в пакетный запрос, мы упростили работу, заплатив за это разумным объемом несогласованности.

18.5. УПРАЖНЕНИЯ

1. Почему важно, чтобы результаты пакетных методов возвращались в конкретном порядке? Чем грозит нарушение этого порядка?
2. Каким должен быть ответ на пакетный запрос `Update`, если его поле `parent` в одном из ресурсов не соответствует полю `parent` в одном из обновляемых ресурсов?
3. Почему для пакетных запросов важна атомарность? Как должно выглядеть определение API, чтобы одни запросы могли завершаться успехом, а другие — отказом?
4. Почему пакетный метод `Delete` опирается на HTTP-глагол `POST`, а не `DELETE`?

РЕЗЮМЕ

- Пакетные методы должны именоваться согласно формату `Batch<Method><Resources>()` и быть полностью атомарными, выполняя либо все операции в пакете, либо ни одну из них.
- Пакетные методы, оперирующие множеством ресурсов одного типа, обычно должны нацеливаться на коллекцию, а не на родительский ресурс (например, `POST /chatRooms/1234/messages:batchUpdate`, а не `POST /chatRooms/1234:batchUpdateMessages`).
- Результаты пакетных операций должны возвращаться в том же порядке, в каком находились ресурсы или запросы при отправке.
- Используйте подстановочный символ дефиса для обозначения множественности возможных родителей ресурса, определяемых в отдельных запросах.

19

Удаление на основе критерия

В этой главе

- ✓ Как с помощью пользовательского метода `Purge` удалять ресурсы, соответствующие критерию.
- ✓ Чем опасен пользовательский метод `Purge` и почему его следует по возможности избегать.
- ✓ Меры предосторожности, позволяющие избежать удаления данных.
- ✓ Как решать проблемы согласованности в наборе соответствующих критерию результатов.

Изученные нами в главе 18 пакетные операции дают возможность удалять несколько ресурсов с помощью одного вызова API. Но для этого обязательно нужно знать уникальный идентификатор удаляемого ресурса. Тем не менее существует множество сценариев, когда мы больше заинтересованы в удалении не конкретного списка ресурсов, а всех ресурсов, соответствующих конкретному набору критериев. Паттерн проектирования, описанный в этой главе, предоставляет механизм, с помощью которого можно безопасно и атомарно удалять все ресурсы, соответствующие заданным критериям, а не списку идентификаторов.

19.1. ЗАЧЕМ ЭТО НУЖНО

Как стало очевидно из главы 18, нередки случаи, когда есть потребность оперировать множеством ресурсов одновременно. Говоря точнее, у нас может возникнуть желание избавиться от конкретного набора ресурсов. Однако, если сравнивать с другими пакетными операциями, удаление пока являлось самой простой из них, не требуя для своего выполнения какой-либо дополнительной информации и удаляя ресурс лишь по его ID.

Это чудесный элемент функциональности, но он требует точного знания идентификаторов тех ресурсов, которые предполагается удалить. То есть если мы не обладаем данной информацией, то сначала нужно ее собрать. Представьте, к примеру, что нам нужно удалить все ресурсы `ChatRoom`, обозначенные как архивные. Необходимо выяснить, для каких ресурсов установлен такой статус, после чего использовать их ID для удаления с помощью пакетного метода `Delete` (листинг 19.1).

Листинг 19.1. Удаление на основе критерия с помощью стандартных методов `List` и `Delete`

```
function deleteArchivedChatRooms(): void {
  const archivedRooms = ListChatRooms({
    filter: "archived: true"
  });
  return BatchDeleteChatRooms({
    ids: archivedRooms.map( (room) => room.id )
  });
}
```

Сначала необходимо найти все ресурсы, отправленные в архив

Заполучив их идентификаторы, можно удалить все эти ресурсы

К сожалению, в этом дизайне есть ряд проблем. Первая и самая очевидная заключается в том, что для реализации задачи требуется не менее двух вызовов API. К тому же, как мы узнаем в главе 21, вывод списка ресурсов скорее потребует выполнения не одного, а целого списка отдельных запросов для поиска всех соответствующих ресурсов. Вторая и более важная проблема состоит в том, что два этих последовательных метода приведут к получению неатомарного результата. Иными словами, есть шанс, что к моменту сбора ID всех архивных ресурсов некоторые из них будут извлечены из архива. Получается, при удалении этих ресурсов мы можем удалить и те, которые уже не относятся к архивным.

Ввиду этих серьезных проблем важно найти альтернативу, которая предоставит единый метод, позволяющий удалять ресурсы на основе заданного набора критериев, а не исключительно по списку их идентификаторов.

19.2. ОБЗОР

Данный паттерн вводит представление о новом пользовательском методе: **Purge**. Его цель — получить простой фильтр, на основе применения которого можно будет удалить все соответствующие его критерию результаты. По сути, это комбинация стандартного метода **List** и пакетного метода **Delete**. Однако вместо передачи вывода одного метода на вход другого (как показано в листинге 19.1) можно добиться поставленной цели с помощью одного вызова API.

Но несмотря на простоту этого метода, здесь нужно учесть и один важный нюанс, а именно его опасность. Как мы увидим в главе 25, пользователи не застрахованы от ошибок, и нас зачастую волнует возможное удаление данных пользователями, о чем они впоследствии пожалеют. А в этот раз вместо того чтобы предоставлять инструмент для удаления одного ресурса (стандартный **Delete**) или даже их множества (пакетный **Delete**), мы вручим им самый большой инструмент из всех. Используя метод **Purge**, пользователи могут ненароком произвести удаление, не осознав в полной мере его объем. Речь о том, что при определенных условиях (например, когда фильтр соответствует всем ресурсам) мы можем полностью стереть все хранящиеся в системе данные.

Чтобы избежать такого потенциально катастрофического результата, мы предоставим два специальных защитных механизма. Во-первых, нам потребуется явный логический флаг, который будет устанавливаться в запросе (**force**) до фактического удаления чего-либо. Во-вторых, когда этот флаг не будет установлен (а значит, запрос не будет выполнен), мы будем предоставлять метод для предпросмотра того, что могло произойти при выполнении запроса. Сюда будет входить подсчет количества элементов, которые бы удалились (**purgeCount**), а также превью элементов из списка совпавших результатов (**purgeSample**). В некотором смысле это похоже на использование в запросе поля **validateOnly** (см. главу 27), но с противоположной предустановкой: запрос делается всегда только для проверки, если явно не запрошено иное.

19.3. РЕАЛИЗАЦИЯ

Чтобы разобраться в принципе работы этого метода, мы начнем с рассмотрения типичного потока его выполнения. Как показано на рис. 19.1, процесс начинается с запроса, предоставляющего фильтр, но оставляющего флаг **force** установленным как **false** (или не установленным). Поскольку это равнозначно запросу на проверку, то никакие ресурсы по факту удалены не будут и вместо этого поле

`purgeSample` будет заполнено списком идентификаторов ресурсов, которые *были бы* удалены. Кроме того, поле `purgeCount` будет сообщать приблизительное количество ресурсов, соответствующих критерию фильтра, а значит, предназначенных для удаления.

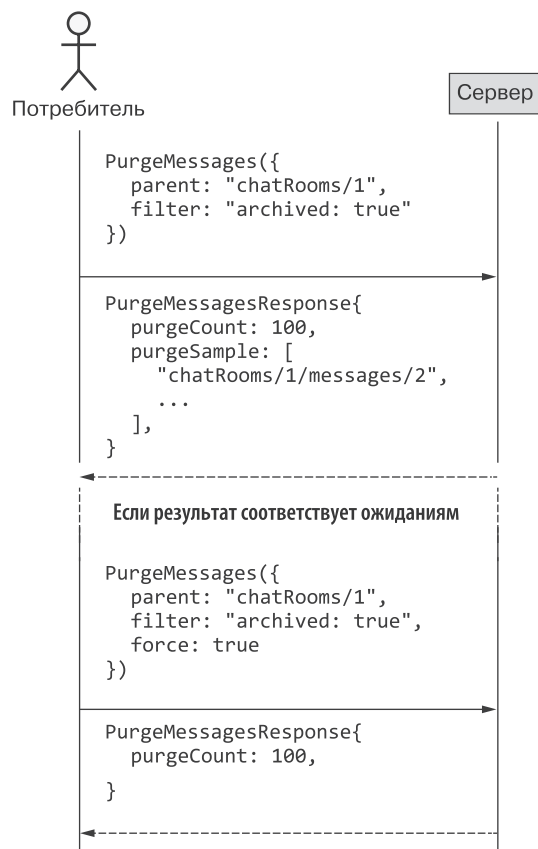


Рис. 19.1. Паттерн взаимодействия с методом `Purge`

Получив эту новую информацию от проверочного запроса, можно перепроверить, действительно ли возвращенные в поле `purgeSample` ресурсы соответствуют замыслу, выраженному в фильтре. И после этого, если мы решим перейти к фактическому удалению всех включенных в ответ ресурсов, можно отправить этот запрос повторно, но уже с флагом `force`, установленным как `true`. В ответе должно заполниться только поле `purgeCount`, где будет указано количество ресурсов, удаленных после выполнения запроса.

Получив общее представление о процессе, перейдем к особенностям каждого из этих полей и изучим их совместную работу, начав с поля `filter`.

19.3.1. Фильтрация результатов

Вполне ожидаемо, что поле `filter`, как мы увидим в главе 22, должно работать в точности аналогично стандартному методу `List`. Весь смысл метода `Purge` в том, что он предоставляет практически идентичную методу `List` функциональность, совмещенную с пакетным методом `Delete`. Это означает, что фильтр, указанный и выполненный методом `Purge`, должен действовать идентично такому же фильтру, предоставленному в стандартном методе `List`.

Необычным и пугающим следствием этого является то, что аналогично тому, как пустой или неустановленный фильтр метода `List` возвращает все хранящиеся в API ресурсы, будет действовать и метод `Purge`. То есть здесь возникает непростая ситуация: если пользователь забывает задать фильтр (или допускает ошибку в коде, приводящую к установке фильтра на `undefined` либо пустую строку), то метод сопоставит с этим фильтром все существующие ресурсы и, в случае активации флага `force`, удалит их (листинг 19.2). Именно поэтому данный дизайн и предусматривает все эти предохранители.

Листинг 19.2. Незначительные опечатки, приводящие к чудовищным последствиям

```
function deleteMatchingMessages(filter: string): number {
  const result = PurgeMessages({
    parent: "chatRooms/1",
    filter: fliter,
    force: true,
  });
  return result.purgeCount;
}
```

← Этот метод получает строку фильтра и удаляет все ресурсы Message, соответствующие ему

← К сожалению, опечатка в этом месте (`fliter` вместо `filter`) приведет к установке фильтра как `undefined`, который в результате будет успешно сопоставляться со всеми ресурсами

И хотя было бы безопаснее запретить подобные запросы, суровая реальность такова, что у пользователей есть потребность в выполнении этой операции, к тому же здесь необходима согласованность со стандартным методом `List` — в противном случае пользователи могут начать думать, что фильтры для разных методов работают по-разному. В результате мы не можем просто отклонять запросы, например, с отсутствующим фильтром. Однако подобные сценарии являются основной причиной добиться того, чтобы по умолчанию метод `Purge` действовал так, будто мы просим лишь превью. В следующем подразделе мы изучим все это более подробно.

19.3.2. По умолчанию только проверка

Как мы узнаем в главе 27, можно задействовать особый флаг `validateOnly`, чтобы метод API только проверял входящие запросы, фактически не выполняя их. Имя для этого поля мы выбираем целенаправленно, чтобы склонить метод к пред-установленному поведению, которому он будет следовать, если его не попросят об ином (более подробная информация по этой теме доступна в разделе 5.2).

Этот предустановленный вариант вполне подходит для подобных случаев, но, как мы только что узнали в подразделе 19.3.1, станет чрезвычайно опасным для метода **Purge**, поскольку позволит мельчайшей ошибке привести к удалению потенциально огромного объема данных в API. Если мы будем опираться исключительно на проверку запроса, то достаточно забыть указать, что запрос предназначен только для проверки, — и вместо ожидаемого превью мы получим удаление данных (листинг 19.3).

Листинг 19.3. Упущение при неправильном предустановленном значении ведет к печальным последствиям

```
PurgeMessages({
  parent: "chatRooms/1",
  filter: "...",
  // validateOnly: true
});
```

Если мы положимся на флаг `validateOnly` и случайно его опустим, то это может привести к удалению большого количества данных

Это один из нечастых сценариев, когда нас интересует метод, который неполноценен по умолчанию, а не наоборот. То есть если поле забыто (возможно, пользователь не прочитал как следует документацию), предустановленное поведение должно быть безопасным и не вести к катастрофическим последствиям.

Для этого мы задействуем дополнительное поле **force**, которое делает то же, что и поле **validateOnly**, но названо так, чтобы вести к другому предустановленному поведению. В результате, если теперь забыть установить это поле (или оставить его как **false**), мы получим полностью безопасный результат: никакие данные не будут удалены. Помимо сохранения данных, мы получим и полезное превью результатов потенциального выполнения запроса. Оно состоит из двух ключевых элементов информации: приблизительного количества соответствующих фильтру ресурсов и их выборки. Следующий подраздел мы начнем с изучения того, как работает этот подсчет результатов.

19.3.3. Подсчет результатов

Независимо от того, выполняется запрос **Purge** полностью или же происходит только проверка, важно иметь представление о количестве ресурсов, соответствующих заданному фильтру. Для этого ответы **Purge** должны включать содержащее эту информацию поле **purgeCount**.

Здесь есть подвох: если в случае полноценного запроса (**force: true**) значение этого поля должно представлять точное количество подлежащих удалению элементов, то при выполнении одной только проверки оно может отразить лишь приблизительное их количество. Это связано с тем, что вычислительная

стоимость поиска и подсчета всех возможных соответствий может оказаться излишне высокой. А поскольку в данном случае мы не собираемся удалять эти элементы и заинтересованы в экономии вычислительных ресурсов, то вполне допустимо использовать приблизительное количество вместо точного. Однако наша цель — добиться максимальной реалистичности, а значит, важно, чтобы это приблизительное значение все-таки отражало действительность.

При использовании для данного поля оценочного значения нужно учитывать, что неверная оценка может вести к неправильным выводам и ее следует по возможности избегать. Чтобы понять почему, рассмотрим сценарий, в котором ответ указывает, что фильтру соответствует примерно 100 ресурсов. Это может сформировать у пользователя ложную уверенность, что метод **Purge** не удалит их больше этого числа. Если же в действительности количество совпадающих ресурсов окажется ближе к 1000, то пользователь будет и шокирован, увидев реальный результат после фактического удаления, и разозлен, понимая, что в случае более точного оценочного значения (скажем, 750) он бы однозначно пересмотрел формулировку фильтра.

По этой причине и ряду других просмотр количества соответствующих критерию ресурсов определенно окажется полезным (как в случае полноценных запросов, так и в случае проверочных). Но в случае именно проверочных есть еще более важная информация, а именно фактическая выборка из соответствующих фильтру ресурсов. Далее мы рассмотрим самый лучший способ ее обработки.

19.3.4. Выборка результатов

Как мы видели, превыю количества соответствующих фильтру ресурсов — полезное, но не идеальное решение. В действительности, даже когда приведенное количество оказывается точным, а не приблизительным, нам все равно нужно понимать, что это всего лишь одна метрика, которая просто подсчитывает соответствующие ресурсы, и во многих случаях подобный вид агрегирования может ввести в заблуждение. Рассмотрим пример, когда фильтру соответствует 50 из 100 элементов. Как мы узнаем, что собираемся удалить именно *задуманные* 50? Возможно, мы планировали удалить 50 архивных ресурсов, но по факту собираемся сделать это для 50 *не архивных*. Подсчет отвечающих критерию результатов часто позволяет обнаружить явные проблемы, но мало помогает в случае более тонких.

Чтобы это исправить, помимо подсчета можно задействовать в ответе дополнительное поле **purgeSample**, которое будет предоставлять выборку элементов, подлежащих удалению. Это поле должно содержать список идентификаторов

совпавших с критерием ресурсов, которые можно будет перепроверить. Например, можно проверить несколько возвращенных ресурсов и убедиться, что они действительно являются архивными, а не наоборот.

И хотя для этого потребуются проделать некую работу, такой прием все равно окажется весьма полезным. Например, в пользовательском интерфейсе можно извлечь несколько из этих элементов с помощью пакетного метода `Get` и показать их пользователю, чтобы он мог их оценить и быть уверенным в правильности ресурсов, отобранных для удаления. Если пользователь решит, что все представленные ресурсы соответствуют его замыслу, то сможет продолжить выполнение запроса, отправив его повторно, но уже с флагом `force`, установленным как `true`.

Однако здесь возникает вопрос: сколько элементов должно отображаться в превью? В целом, поскольку наша задача — обнаружить возможные ошибки в выражении фильтра, важно сделать размер выборки достаточно большим, чтобы пользователь мог заметить в ней неуместные элементы (например, когда в списке появляется ресурс, которого там быть не должно). В связи с этим хорошим правилом будет предоставлять не менее 100 элементов для больших наборов данных и полные списки для более мелких, запрос которых не несет большой вычислительной нагрузки.

19.3.5. Согласованность

Последний нюанс, представляющий некоторый повод для беспокойства, является более изощренным и связан с согласованностью. Что произойдет, если после отправки проверочной формы запроса `Purge` к моменту последующей передачи фактического запроса на удаление данные изменятся так, что фильтру будет соответствовать намного больше ресурсов? Иными словами, есть ли способ, позволяющий гарантировать, что возвращенные во время проверки данные будут соответствовать тем, которые будут удаляться впоследствии? К сожалению, нет.

Даже при возможности выполнять запросы для снимков данных в конкретный момент времени выполнение запроса `Purge` для данных в том виде, в каком они представлялись в прошлом, не приведет к желаемому результату. На самом деле, если такое поведение действительно является желаемым, то уже поддерживается комбинацией стандартного метода `List` и пакетного `Delete`.

Конечно, технически можно сделать так, чтобы метод `Purge` завершался неудачей в случае изменения данных в период между проверочным запросом и запросом на удаление, но тогда в отношении достаточно обширного, конкурентного, волатильного набора данных он станет практически бесполезным. А в мире API такие наборы данных далеко не редкость.

19.3.6. Итоговое определение API

Теперь, когда мы имеем полное представление о работе метода `Purge`, пора взглянуть на краткий пример кода для удаления ресурсов `Message` на основе указанного в строке фильтра с критериями (листинг 19.4).

Листинг 19.4. Итоговое определение API

```
abstract class ChatRoomApi {
    @post("/{parent=chatRooms/*/}/messages:purge")
    PurgeMessages(req: PurgeMessagesRequest): PurgeMessagesResponse;
}

interface PurgeMessagesRequest {
    parent: string;
    filter: string;
    force?: boolean;
}

interface PurgeMessagesResponse {
    purgeCount: number;
    purgeSample: string[];
}
```

19.4. КОМПРОМИССЫ

На случай, если вы не заметили, отдельно напомню, что это опасный метод. Можно сравнить его с выдачей пользователям API «базуки», дающей возможность легко и просто уничтожать большие объемы данных. И хотя в представленном дизайне этот метод содержит максимум защитных механизмов, он все равно оставляет пользователям возможность случайно удалить огромное количество данных, если не все. Как следствие, если эта функциональность не является абсолютно необходимой, то ее поддержку в API обычно лучше исключить.

19.5. УПРАЖНЕНИЯ

1. Почему применение пользовательского метода `Purge` желательно ограничить только теми случаями, в которых он поистине необходим?
2. Почему метод `Purge` по умолчанию выполняет только проверку?
3. Что произойдет, если критерий фильтра окажется пуст?
4. Какое поведение ожидать от метода `Purge`, если ресурс поддерживает мягкое удаление? Должен ли он удалить эти ресурсы мягко или окончательно?
5. Какова цель возвращения количества предполагаемых к удалению ресурсов? И зачем нужна выборка ресурсов, соответствующих критерию фильтра?

РЕЗЮМЕ

- Пользовательский метод **Purge** служит для удаления множества ресурсов, соответствующих заданному набору критериев, но поддерживаться он должен лишь при крайней необходимости.
- По умолчанию запросы **Purge** должны выполнять исключительно проверку, но не фактическое удаление ресурсов.
- Все ответы **Purge** должны содержать количество ресурсов, которые предполагается удалить (и выборку из них), хотя в случае проверочного запроса это количество может быть приблизительным.
- Метод **Purge** должен подчиняться тем же принципам согласованности, что и стандартный метод **List**.

20

Анонимная запись

В этой главе

- ✓ Что подразумевается под анонимными данными.
- ✓ Как внедрять данные в API, не опираясь на создание ресурсов.
- ✓ Как устранить проблемы согласованности в случае внедрения данных.

До сих пор запись новых данных в API подразумевала создание ресурсов, имеющих собственные уникальные идентификаторы и схемы. К сожалению, как бы нам ни хотелось, чтобы это был единственный способ создания данных, иногда он не вписывается в реалии мира API. Действительно, есть сценарии, в которых данные должны быть записаны без последующей уникальной идентификации. Это могут быть записи в файл журнала или точки временных данных, агрегируемые в статистику. В текущем паттерне мы разберем эту идею записи данных вместо создания ресурсов и узнаем, как этот принцип может сочетаться с нашими существующими принципами ресурсно-ориентированного дизайна.

20.1. ЗАЧЕМ ЭТО НУЖНО

Во всех прежних процессах, записывая данные в API (добавляя новые или обновляя существующие), мы рассуждали и действовали в контексте ресурсов. Даже в случаях, когда мы можем несколько переиначивать правила о том, что

именно представляет собой ресурс, как это было в главе 12, в основе все равно сохранялся именно этот принцип. Эти ресурсы всегда действовали как единые, адресуемые, уникально идентифицируемые фрагменты данных, которые можно создавать, обрабатывать, считывать, а при потере их актуальности — уничтожать. К этому моменту они хорошо нам послужили. К сожалению, их не всегда достаточно для охвата всех возможных аспектов и сценариев, с которыми мы сталкиваемся в мире проектирования API.

Рассмотрим, к примеру, такие случаи, как сбор статистики за период времени. Очевидным способом сохранения подобной статистики будет использование ресурса `DataPoint` и стандартного метода `Create` для записи входящих точек данных. Но это окажется сродни отдельному заворачиванию и подписыванию каждого рисового зернышка, а не покупке целого мешка весом 5 кг. Как правило, в случае подобных систем пользователей больше интересует агрегация данных, а не их отдельные точки. Иными словами, намного вероятнее, что мы запросим среднее значение по точкам данных за месяц, а не отдельную точку данных по ее уникальному идентификатору. Это настолько типичный случай, что многие системы аналитики данных, БД для временных рядов или информационные хранилища (например, `BigQuery` (<https://cloud.google.com/bigquery>) или `InfluxDB` (<https://www.influxdata.com/products/influxdb/>)) даже не поддерживают для хранения точек данных уникальные идентификаторы.

Из этого вытекает очевидный вопрос: если для хранения своих аналитических данных мы используем одну из таких систем, то как именно нужно вносить эти данные в API? Нужно ли представить, что у нас есть уникальный идентификатор (выделив в БД отдельное поле), и просто обычным образом задействовать стандартный метод `Create`? А может, нужно изменить метод `Create`, чтобы он вставлял данные в БД, но не возвращал в качестве результата адресуемый ресурс, возможно, оставляя поле `id` пустым? Или же есть совсем другой способ? Текущий паттерн предоставляет альтернативу, которая предназначена для стандартизации наиболее грамотного способа обработки таких нересурсно-ориентированных данных в наших, как правило, ресурсно-ориентированных API.

20.2. ОБЗОР

Работа этого паттерна основана на специальном пользовательском методе, называемом `Write`. По аналогии со стандартным методом `Create` его цель состоит во внедрении новых данных в API. Тем не менее делает он это так, что получающаяся запись данных оказывается анонимной (то есть данные не имеют идентификатора). Как правило, причина в том, что итоговые данные не адресуются и не могут извлекаться, обновляться или удаляться.

Вместо извлечения отдельных записей подобный тип данных должен рассматриваться только в контексте агрегирования. Это означает, что у пользователей вместо возможности запрашивать отдельную точку данных, как это бывает в случае типичного ресурса, будет возможность запрашивать на основе этих данных агрегат, например общее количество записей или их среднее значение. Несложно представить, что подобное взаимодействие важно для информационных API, предоставляющих различную статистику о системе.

Поскольку эти точки данных имеют столь отличную от привычных нам ресурсов схему доступа, практически всегда для описания записываемых данных будет лучше использовать иную терминологию. В данном случае вместо того, чтобы обращаться к ресурсу, мы будем называть внедряемые в API точки данных термином *запись*.

Объединим все вышесказанное. На рис. 20.1 представлена диаграмма последовательности, отражающая пример внесения записей и считывания агрегированных данных (например, результата их подсчета).

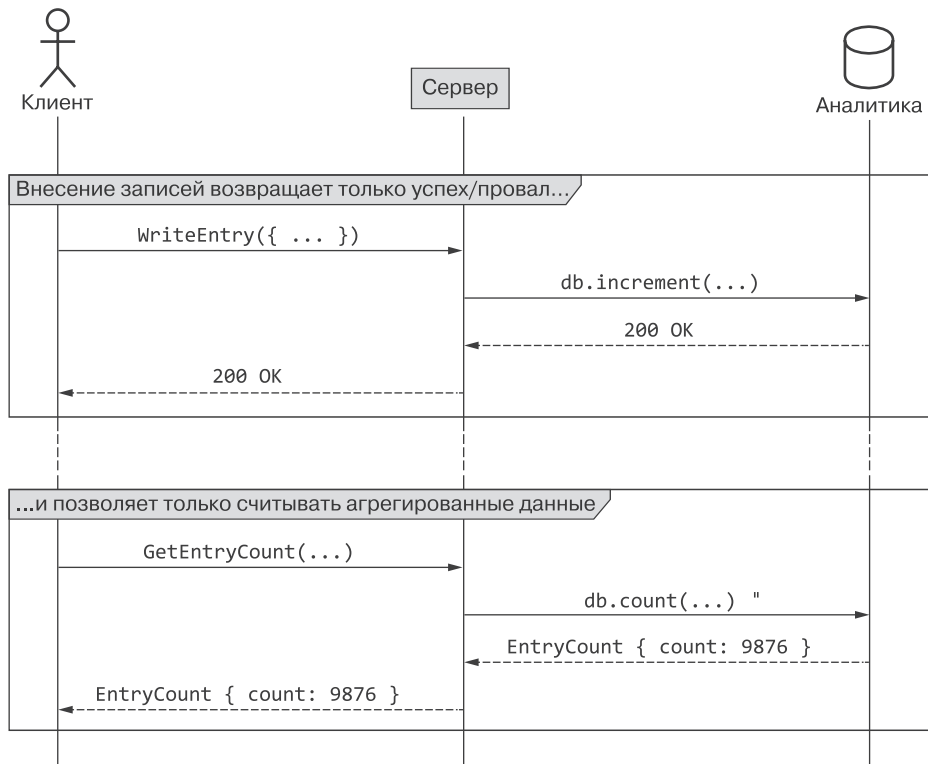


Рис. 20.1. Последовательность событий метода `Write`

В следующем разделе мы познакомимся с нюансами метода `Write` и разберем его реализацию.

20.3. РЕАЛИЗАЦИЯ

Несмотря на то что метод `Write` по своей природе довольно похож на метод `Create`, между ними есть ряд важных различий. Для начала взглянем на возвращаемый тем и другим тип. Если метод `Create` возвращает созданный ресурс, то у метода `Write` такого ресурса для возвращения нет. В действительности метод `Write` подразумевает добавление данных в коллекцию (или вступление в группу в качестве очередного анонимного участника) либо потоковое использование с целью увеличить счетчик или обновить агрегированное значение (например, скользящего среднего). Именно это поведение и делает данный метод особенным, но оно также означает, что мы не можем вернуть из него ничего полезного, кроме как успешный результат или ошибку. В связи с этим метод `Write` должен либо ничего не возвращать, либо возвращать `void`.

Далее нужно разобраться, как именно передавать полезную нагрузку данных. Стандартный метод `Create` опирается на одно поле (`resource`), которое получает значение создаваемого ресурса. Опять же, в отличие от метода `Create`, метод `Write` не работает с ресурсами, используя вместо этого принцип записей. К счастью, это подводит нас к очевидному и простому заключению: рассматривать запрос `Write` просто как запрос `Create`, но переименовать поле из `resource` в `entry` (листинг 20.1).

Листинг 20.1. Пример запроса `Write`

```
interface WriteChatRoomStatEntryRequest {
    parent: string;
    entry: ChatRoomStatEntry;
}
```

← Здесь вместо поля `resource`
мы используем поле `entry`

В завершение нужно выяснить, какой должна быть привязка HTTP URL для метода `Write`. Представьте, что нам нужно записать статистику о ресурсе `ChatRoom`. Как мы узнали в главе 9, в частности в подразделе 9.3.2, обычно лучше нацеливаться на коллекцию, а не на родительский ресурс. Это означает, что метод `Write` должен избегать URL наподобие `/chatRooms/1:writeStatEntry` и склоняться к их альтернативе вида `/chatRooms/1/statEntries:write`. И хотя такой выбор может быть спорным (поскольку `statEntries` в действительности не является коллекцией ресурсов), суть в том, что в дальнейшем у нас будет возможность взаимодействовать с коллекцией записей с помощью других способов. Например, мы сможем использовать пакетный метод `Write` (глава 18) для одновременного добавления множества записей или же метод `Purge` (глава 19) для удаления всех записей.

Теперь, освоив основы, мы перейдем к более сложной теме: согласованности.

20.3.1. Согласованность

Как правило, при внедрении данных в API (например, с помощью стандартного метода `Create`) нам нужна возможность сразу же видеть результат этой операции. То есть если API сообщает об успешном выполнении нашего запроса на создание данных, то у нас должна быть возможность сразу же считать их из API. В действительности это настолько важно, что при возникновении ситуаций, когда для завершения операции требуется время, мы прибегаем к использованию длительных операций (глава 10), чтобы иметь представление о том, когда данные окажутся доступны в системе.

Поскольку метод `Write` действует отчасти аналогично методу `Create`, то возникает важный вопрос: должен ли он в той же степени быть согласованным, делая данные доступными сразу же после завершения операции? Если нет, то нужно ли нам использовать для отслеживания прогресса длительные операции? Если снова нет, то что тогда вообще делать?

Чтобы ответить на эти вопросы, сначала нужно вспомнить, что задача метода `Write` состоит в том, чтобы позволить добавлять в систему неадресуемые данные. То есть способ считывания этих данных будет сильно отличаться от того, как они считываются в случае добавления стандартным методом `Create`. В действительности, поскольку мы практически всегда считываем подобные данные через агрегаты, будет сложно — если вообще возможно — узнать, считываем ли мы данные, которые добавили сами, или же те, которые добавил кто-то другой с помощью собственного вызова `Write`. Это означает, что нет смысла придерживаться стандарта, согласно которому все данные должны быть доступны к моменту завершения операции их записи. Иными словами, вполне допустимо, чтобы метод `Write` возвращал результат сразу (как показано на рис. 20.2), даже до того, как данные станут видимы тому, кто использует API.

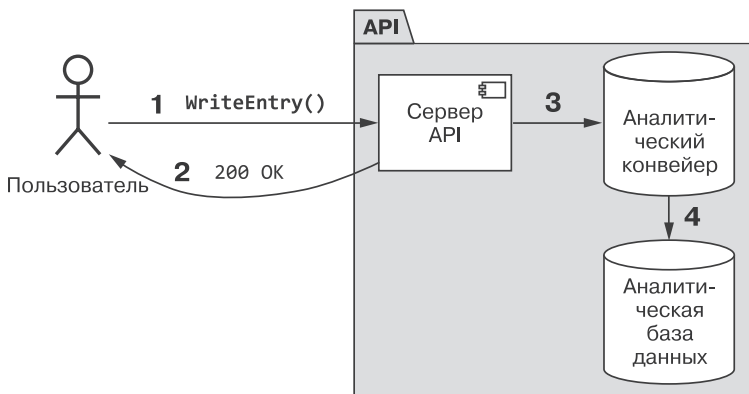


Рис. 20.2. Операция записи данных получает мгновенный ответ в то время, как данные передаются в аналитический конвейер

Это делается для того, чтобы можно было удачно вписаться в большинство случаев использования, где задействуются обширные системы аналитики, поскольку они обычно опираются на конечную согласованность и имеют потенциально длительные задержки в конвейерах обработки данных. Если бы для возвращения результата нам приходилось ждать завершения всех вычислений, то это бы вызвало серьезные проблемы с задержкой обработки запроса, в корне снижая эффективность самого метода.

Тем не менее возникает интересный вариант: если мы хотим возвращать ответ до того, как данные станут видимыми, но не имеем удобной возможности делать это синхронно, то, возможно, стоит применить LRO (см. главу 10) для моментального возвращения и оставить ожидание фактического отображения результатов пользователям? К сожалению, эта идея чревата двумя проблемами. Во-первых, зачастую невозможно знать, когда отдельный элемент данных окончательно сохранится как часть агрегированной статистической информации. Основная причина в том, что отдельные записи не содержат уникальные идентификаторы, в связи с чем нет способа, позволяющего отслеживать единичные точки данных, проходящие через конвейер аналитики.

Во-вторых, даже если бы мы могли отслеживать прохождение точек данных по системе, одна из основных причин использования метода `Write` вместо `Create` заключается в том, что нам не нужно сохранять все записи по отдельности. Наоборот, нам нужно сэкономить пространство и вычислительное время с помощью агрегирования данных. Если бы мы далее решили, что каждый вызов метода `Write` должен создавать новый ресурс `Operation`, то просто передвинули бы таким образом проблему с одного места в другое, не сэкономив ни пространство, ни вычислительные ресурсы.

Исходя из этого, практически всегда будет плохой идеей возвращать LRO в качестве результата метода `Write`. Важно сообщить пользователям, что их данные были приняты в конвейер и станут видимыми лишь спустя какое-то время, что окажется хорошей альтернативой простому возвращению HTTP-кода `202 Accepted` вместо типичного `200 OK`. Более того, если есть какие-то опасения относительно дублирования предоставляемых методу `Write` записей, отличным решением станет паттерн дедубликации, рассматриваемый в главе 26.

Ну а теперь пора взглянуть на то, как можно применить паттерн текущей главы для определения API, поддерживающего метод `Write`.

20.3.2. Итоговое определение API

Представим, что хотим запустить сбор произвольной статистики о комнатах чата. В нее может входить такая информация, как частота посещения пользователями конкретной комнаты, но при этом нам нужно на будущее сохранить гибкость в работе с добавляемыми точками данных. Для этого можно создать интерфейс

`ChatRoomStatEntry`, который будет просто представлять пару «ключ — значение», состоящую из строки и скалярного значения, которые можно будет в дальнейшем агрегировать и затем проанализировать.

Более того, нам может потребоваться поддержка одновременной записи множества ресурсов `ChatRoomStatEntry`. Для этого подойдет пакетная версия метода `Write` (подробнее о пакетных методах в главе 18) (листинг 20.2).

Листинг 20.2. Итоговое определение API

Вместо возвращения ресурса
мы не возвращаем ничего

```

abstract class ChatRoomApi {
    @post("/{parent=chatRooms/*/}/statEntries:write")
    WriteChatRoomStatEntry(req:
        WriteChatRoomStatEntryRequest): void;

    @post("/{parent=chatRooms/*/}/statEntries:batchWrite")
    BatchWriteChatRoomStatEntry(req:
        BatchWriteChatRoomStatEntryRequest): void;
}

interface ChatRoomStatEntry {
    name: string;
    value: number | string | boolean | null;
}

interface WriteChatRoomStatEntryRequest {
    parent: string;
    entry: ChatRoomStatEntry;
}

interface BatchWriteChatRoomStatEntryRequest {
    parent: string;
    requests: WriteChatRoomStatEntryRequest[];
}

```

Как было сказано в подразделе 20.3.1, эти пользовательские методы нацеливаются на коллекцию, а не на родительский ресурс

По тому же принципу даже пакетные методы ничего в качестве результата не возвращают

20.4. КОМПРОМИССЫ

От многих рассмотренных до этого момента паттернов проектирования текущий отличается уникальным случаем использования. В реальности чаще всего это будут сценарии аналитики данных при том, что почти все рассмотренные нами данные являлись по своей природе транзакционными. В результате у нас не так много альтернатив для безопасного добавления этого типа аналитических данных в API.

Мы узнали, что определенно можно задействовать стандартный метод `Create` и рассматривать отдельные точки данных как полноценные ресурсы, каждый

со своим уникальным идентификатором. Тем не менее подобный дизайн с большинством аналитических хранилищ и систем обработки данных будет работать не так успешно. Более того, объем сохраняемых данных будет наверняка очень быстро расти, становясь все менее управляемым. В результате анонимная запись оказывается наиболее эффективной в случаях, когда API требуется поддержка внедрения аналитических данных наряду с более традиционным ресурсно-ориентированным дизайном.

20.5. УПРАЖНЕНИЯ

1. Если при использовании метода `write` нас будет беспокоить возможное дублирование внедряемых данных, то какой способ наиболее эффективно позволяет его избежать?
2. Почему метод `write` не должен возвращать тело ответа? Почему для него нежелательно возвращать ресурс длительной операции?
3. Если мы хотим сообщить, что данные были получены, но еще не обработаны, то какие для этого есть варианты? Какой из них окажется оптимальным для большинства API?

РЕЗЮМЕ

- Когда в систему требуется внести информацию (например, записи аналитических данных), нужно задействовать пользовательский метод `write`, а не создавать ресурсы, к которым никогда не будут обращаться по отдельности.
- Загрузка данных в API с помощью метода `write` происходит в одностороннем и безвозвратном порядке. То есть впоследствии эти данные нельзя будет удалить.
- Методы `write` (и их пакетные версии) не должны возвращать какой-либо ответ, кроме итоговых кодов состояний. При этом они не должны возвращать и какой-либо ресурс (даже ресурс длительной операции), за исключением особых обстоятельств.
- Вместо ресурсов метод `write` работает с записями, которые похожи на ресурсы, но при этом не предполагают адресацию и во многих случаях являются временными.

21

Пагинация

В этой главе

- ✓ Как пагинация позволяет пользователям потреблять огромные наборы данных удобными для восприятия частями.
- ✓ Выбор размера для возвращаемых страниц данных.
- ✓ Как сервис API должен сообщать о завершении пагинации.
- ✓ Определение формата маркеров страниц.
- ✓ Реализация пагинации по огромным фрагментам данных в одном ресурсе.

Этот паттерн поможет нам научиться потреблять данные, когда количество ресурсов или размер отдельного ресурса оказывается слишком большим для одного ответа API. Вместо того чтобы ожидать все данные в одном интерфейсе ответа, мы будем входить в последовательность переходов «вперед-назад», получив возможность запрашивать по одной части данных за раз и перебирать их до тех пор, пока они не закончатся. Такая функциональность нередко встречается во многих веб-интерфейсах, где пользователь может переключаться между страницами результатов, и для API является не менее значимой.

21.1. ЗАЧЕМ ЭТО НУЖНО

В типичном API потребителям может потребоваться извлекать и просматривать ресурсы. А поскольку эти ресурсы растут как в размере, так и в количестве, становится все менее логичным предполагать, что потребители будут считывать свои данные одним большим фрагментом. Например, если API предлагает доступ к миллиарду записей данных, занимающих в хранилище 100 Гбайт, то попытка предоставить потребителям возможность извлекать эти данные с помощью одного запроса и ответа может существенно сказаться на скорости или даже стать технически невозможной. Как же лучше всего представить интерфейс, позволяющий взаимодействовать с этими данными? Иными словами, как можно исключить для потребителей необходимость брать на себя невыполнимую задачу? Очевидно, что можно разделить данные на удобные для работы части и позволить потребителям взаимодействовать с получившимися подмножествами. Но тогда возникает еще один вопрос: как и в каких местах нужно делить данные?

Разделительную черту мы можем провести между любыми двумя записями, но не всегда этого будет достаточно, особенно в случаях, когда сами записи могут стать излишне громоздкими (например, один объект размером 100 Мбайт). В этом случае вопрос разделения данных на подходящие фрагменты становится еще более каверзным, особенно если этот фрагмент окажется структурированными данными (например, строкой базы данных размером 10 Мбайт). Где тогда нужно провести разделение внутри одной записи? Какой уровень контроля мы должны предоставить потребителям над этими точками разделения?

С другой стороны, возможны сценарии, когда потребителям действительно нужно получить все записи базы данных с помощью одного запроса-ответа (например, при резервном копировании или экспорте данных). Однако эти два сценария не обязательно станут взаимоисключающими, так что это не отменяет тот факт, что нам все равно нужен способ взаимодействия с данными в виде удобоваримых частей. Для этого типичного случая и существует паттерн, который стал довольно распространенным на множестве сайтов и прекрасно переносится в API: пагинация.

21.2. ОБЗОР

Термин «*пагинация*» родился как отражение необходимости постранично перебирать данные¹, подобно тому, как мы перелистываем страницы в книге. Такой способ позволяет потребителю запрашивать по одному фрагменту данных за раз, на что API выдает в ответе эти запрошенные фрагменты вместе с указателем для извлечения очередного. На рис. 21.1 показана схема последовательности опера-

¹ В английском языке страница — page, а термин звучит как pagination. — *Примеч. пер.*

ций, демонстрирующая, как пользователь может запрашивать от API страницы ресурсов, пока они не закончатся.

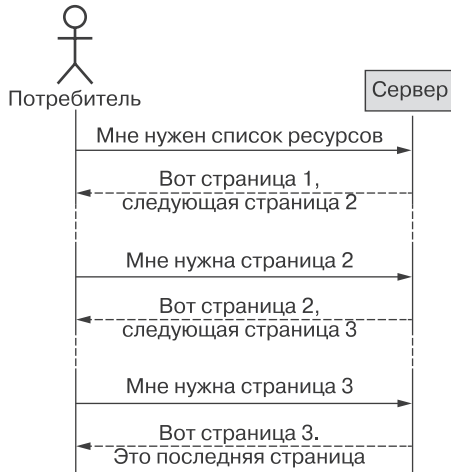


Рис. 21.1. Поток пагинации

Чтобы это реализовать, паттерн пагинации будет опираться на принцип указателя, используя непрозрачные маркеры страниц для выражения эквивалента их нумерации. Учитывая эту непрозрачность, будет важно, чтобы API отправлял в ответе и фрагмент результатов, и очередной маркер. Когда дело доходит до обработки крупных одиночных ресурсов, тот же паттерн можно применить к каждому ресурсу, разбив его содержимое на множество страниц данных.

21.3. РЕАЛИЗАЦИЯ

С позиции верхнего уровня нам потребуется способ, позволяющий возвращаться к месту, на котором мы остановились, просматривая конкретное окно доступных данных. Это свое намерение мы выражаем с помощью трех разных полей:

- 1) **pageToken**, представляющего непрозрачный идентификатор, нужный только для сервера API, чтобы продолжить ранее начатый перебор результатов;
- 2) **maxPageSize**, позволяющего потребителю указать максимальное количество результатов, которое он хочет получить в запросе;
- 3) **nextPageToken**, который сервер API использует для того, чтобы показать, как потребитель должен запрашивать дополнительные результаты с помощью очередного запроса.

Чтобы увидеть это более ясно, можно заменить этими полями часть текста на рис. 21.2.



Рис. 21.2. Пагинация с использованием специальных полей

Если взяться переводить этот паттерн в определение API, то мы получим что-то очень похожее на стандартный метод `List`, где в запрос добавлены `pageToken` и `maxPageSize`, а в ответ — `nextPageToken` (листинг 21.1).

Листинг 21.1. Спецификация API для пагинации в стандартном методе `List`

```

abstract class ChatRoomApi {
    @get("/chatRooms")
    ListChatRooms(req: ListChatRoomsRequest): ListChatRoomsResponse;
}

interface ListChatRoomsRequest {
    pageToken: string;
    maxPageSize: number;
}

interface ListChatRoomsResponse {
    results: ChatRoom[];
    nextPageToken: string;
}
  
```

Добавление трех новых полей в запросы и ответы метода `List` для поддержки пагинации

Само это определение не столь интересно, но выясняется, что каждое из этих полей имеет свои секреты. Начнем с кажущегося безобидным поля `maxPageSize`.

21.3.1. Размер страницы

На первый взгляд установка нужного размера страницы (то есть количества возвращаемых результатов) выглядит довольно простой, но здесь есть свои нюансы. Начнем с рассмотрения имени поля: `maxPageSize`.

Максимум или точность?

Почему мы используем максимум, а не просто устанавливаем точный размер? То есть почему мы возвращаем до десяти результатов, а не ровно десять? В действительности в большинстве сценариев у сервера API всегда будет возможность возвращать точное количество результатов. Однако во многих крупномасштабных системах это потребует существенных накладных расходов.

Представьте систему, хранящую 10 миллиардов строк данных в таблице, которая ввиду ограничений размера не имеет вторичных индексов (что упростило бы запрос этих данных). Далее представьте, что в ней есть ровно 11 совпадающих с запросом строк: пять находятся в самом начале, за ними идут 5 миллиардов неактуальных, а уже за ними следуют остальные шесть запрашиваемых. Схематично этот случай представлен на рис. 21.3. В довершение к этому предположим, что нам нужно вернуть большую часть ответов (скажем, 99 %) в течение 200 миллисекунд.

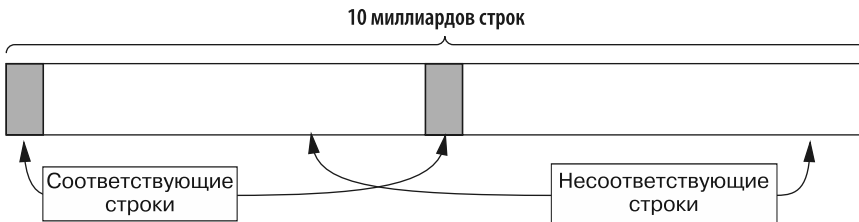


Рис. 21.3. Соответствующие строки могут быть сильно разбросаны

В этом сценарии, когда пользователь запрашивает страницу из десяти результатов, у нас есть два варианта. С одной стороны, можно вернуть ровно десять результатов (поскольку их хватает для заполнения страницы), но вряд ли такой запрос окажется достаточно быстр, учитывая необходимость просканировать для этого около 5 миллиардов строк. С другой стороны, можно также вернуть пять соответствующих строк, обнаруженных до момента истечения нашего временного лимита (для этого может потребоваться 180 мс), после чего при последующем запросе дополнительных данных вернуться к месту, на котором поиск остановился.

Учитывая эти ограничения, стремление всегда заполнить всю страницу создаст серьезную сложность при попытке отличить нарушение критерия задержки запроса от простых изменений в сложности запроса. Иначе говоря, причиной медленного запроса (более 200 мс) могут быть как фактические неполадки, так и необходимость просмотра большого объема данных — и мы не сможем отличить одно от другого, не приложив дополнительных усилий. Если же заполнять страницу до максимально возможного предела в рамках установленного временного ограничения, то нам зачастую удастся возвращать ровно десять результатов (возможно, потому, что они расположены рядом), но при этом мы уже не окажемся

в затруднительном положении, когда набор данных станет невероятно большим. В результате разумнее устанавливать в контракте интерфейса, что нужно возвращать до `maxPageSize` результатов, а не точное их количество.

Значение по умолчанию

Важно грамотно подобрать предустановленные значения для всех необязательных полей, чтобы потребители не удивлялись последствиям, вызванным применением пустых полей. В этом случае выбор предустановленного значения для максимального размера страницы будет зависеть от формы и размера возвращаемых ресурсов. Например, если каждый элемент будет крохотным (скажем, измеряться в байтах, а не килобайтах), то может оказаться разумным использовать в качестве предустановленного количества 100 результатов на страницу. Если же каждый элемент будет весить по нескольку килобайт, то более подходящей предустановленной величиной станет от 10 до 25 результатов. Как правило, рекомендуемый размер страницы по умолчанию составляет десять результатов. Наконец, также будет приемлемым попросить потребителей указывать максимальный размер страницы. Однако этот вариант должен быть зарезервирован для ситуаций, когда сервер API не может предположить разумное предустановленное значение.

Несмотря на всю важность выбора предустановленного максимального размера страницы, еще важнее документировать это значение и сохранять его согласованным во всех частях API, где это возможно. Например, нежелательно использовать для этого поля различные предустановленные значения, не имея на то веской причины. Если потребитель узнаёт, что API по умолчанию выводит десять результатов на страницу, то для него станет неприятной неожиданностью, когда в каком-то другом месте без существенной на то причины по умолчанию будет выводиться уже не 10, а 50 результатов.

Верхний и нижний предел

Очевидно, что для сервера API отрицательный размер страницы не имеет смысла, в связи с чем должен отвергаться с ошибкой, указывающей на недействительность запроса. Но что делать, если в запросе для максимального размера страницы установлено значение 3 миллиарда? К счастью, наша предыдущая структура, опирающаяся на максимальный размер страницы вместо точного, означает, что такие значения можно без проблем принимать, возвращать разумное количество результатов, которое наш сервер API может произвести в рамках установленного времени, и сохранять при этом согласованность с определением API.

Иными словами, хоть и можно отклонять размер страниц, превышающий некую границу (например, больше 1000 результатов), вполне допустимо считать их валидными, поскольку всегда можно вернуть меньшее количество результатов, не нарушая нашего соглашения с потребителем. На этом с простыми деталями покончено и пора переходить к более сложным. Начнем с маркеров страниц.

21.3.2. Маркеры страниц

Мы уже говорили о том, что маркеры страниц должны служить в качестве указателя, но при этом не проясняли само их поведение. Начнем с разбора простого, но одновременно путающего нюанса: как можно узнать, что перебор результатов закончен.

Критерий завершения

Поскольку мы не можем принудить потребителя продолжать выполнять очередные запросы до завершения всех страниц, то технически он может закончить пагинацию в любой момент, когда решит, что дополнительные результаты ему не нужны. Но как сервер должен указывать на то, что результатов больше нет? Во многих системах мы обычно считаем, что на завершение списка указывает возвращение неполной страницы результатов. К сожалению, это предположение в случае нашего определения размера страницы не работает, так как возвращается максимальное количество результатов, а не точное. В итоге ориентироваться на частично заполненную страницу уже не получится. Вместо этого мы будем сообщать о завершении списка с помощью пустого маркера страницы.

Потребителей это может запутать, особенно если учесть, что вполне допустимо возвращение пустого списка результатов вместе с непустым маркером страницы. Действительно, покажется странным, если мы не получаем результатов, но при этом нам говорят, что результаты еще есть. Как мы узнали в подразделе о максимальном размере страницы, такой ответ возможен, когда запрос достигает временного лимита, не находя ни одного результата, но при этом не может быть уверен, что не найдет их в дальнейшем. Иными словами, ответ `{results: [], nextPageToken: 'cGFnZTE='}` является для API способом сообщить: «Я тщательно искал в течение 200 мс, но ничего не нашел. Можете продолжить с того места, где я закончил, используя вот этот маркер».

Прозрачность

До сих пор мы разбирали, как с помощью маркера страницы сообщить потребителю о том, есть ли у него возможность продолжить перелистывание страниц ресурсов. Но при этом мы ничего не говорили о содержимом самого маркера. Что конкретно нужно в нем разместить?

Простыми словами, содержимым токена должно быть все, что требуется серверу API для возвращения к месту, где он закончил перебор списка результатов. Это может быть нечто абстрактное наподобие сериализованного объекта из кода или же что-то конкретное вроде границы и смещения для передачи в реляционную базу данных. Например, можно просто использовать сериализованный объект JSON наподобие `eyJvZmZzZXQiOiAxMn0=`, что в кодировке Base64 означает `{"offset": 10}`. Что еще более важно, так это то, что данная деталь реализации

должна оставаться полностью скрытой от потребителей. Это может показаться необязательным, но в действительности имеет большое значение.

Независимо от того, что вы помещаете в маркер, его структура, формат или смысл должны быть полностью скрыты от потребителя. Это означает, что содержимое в Base64 вместо кодирования нужно шифровать так, чтобы для потребителя оно оказывалось абсолютно бессмысленным. И причина тому довольно проста: если потребитель может распознать структуру или значение маркера, значит, это часть поверхности API, которая должна рассматриваться как часть самого API. Весь смысл использования маркеров страниц состоит в получении возможности внутренне изменять реализацию, не изменяя используемую потребителями поверхность API, и раскрытие этой информации лишает нас данного преимущества. По сути, если мы позволим потребителям заглядывать в маркеры страниц, то допустим утечку деталей реализации и можем оказаться в ситуации, когда не сможем изменить реализацию, не навредив потребителям.

Формат

Теперь, когда мы разобрались, что должно размещаться в маркере, и осознали необходимость сокрытия этого содержимого от потребителей, перед нами возникает вопрос: почему для представления маркера используется строковый тип? Почему не число или сырые байты?

Учитывая потребность в сохранении зашифрованных данных, становится очевидным, что использование численного значения просто может не сработать. С другой стороны, сырые байты — отличный способ, позволяющий хранить зашифрованное значение, но строки мы используем в первую очередь из соображений удобства. Дело в том, что маркеры зачастую будут включаться в URL (в качестве параметров HTTP-запроса GET) и в объекты JSON, где в обоих случаях сырые байты просто не получится обработать должным образом. В итоге наиболее распространенным форматом является зашифрованное значение в кодировке Base64, передаваемое в виде сериализованной строки UTF-8.

Согласованность

Все это время, говоря о пагинации, мы предполагали, что сами данные статичны. К сожалению, так бывает редко. Поэтому не стоит ожидать, что данные окажутся предназначенными только для чтения, и нужно найти способ перебирать их, несмотря на изменения, добавляя новые результаты и удаляя старые. Ситуация усложняется и тем, что разные системы хранения предоставляют разную функциональность запросов, то есть поведение нашего API может быть тесно связано с системой, используемой для хранения данных (например, MySQL).

Вполне ожидаемо, что это зачастую будет вести к озадачивающим сценариям. Представьте, к примеру, что перебираете данные по два результата за раз и в это время кто-то вносит в начало списка два новых ресурса (рис. 21.4). Если вы используете подсчет по *смещению* (когда очередной маркер страницы представляет

смещение, начинающееся с элемента 3), то добавление в начало двух ресурсов приведет к тому, что вы увидите страницу 1 дважды. Причина в том, что смещение нынешней страницы 2 совпадает с началом старой страницы 1. Более того, данный сценарий повторяется, когда к запросу применяется фильтр, и результаты изменяются так, что либо добавляются в группу соответствий, либо покидают ее. В этом случае потребитель может потерять уверенность в том, что ваш API предоставит все соответствующие результаты и что они вообще окажутся именно соответствующими. Как же нам поступить?

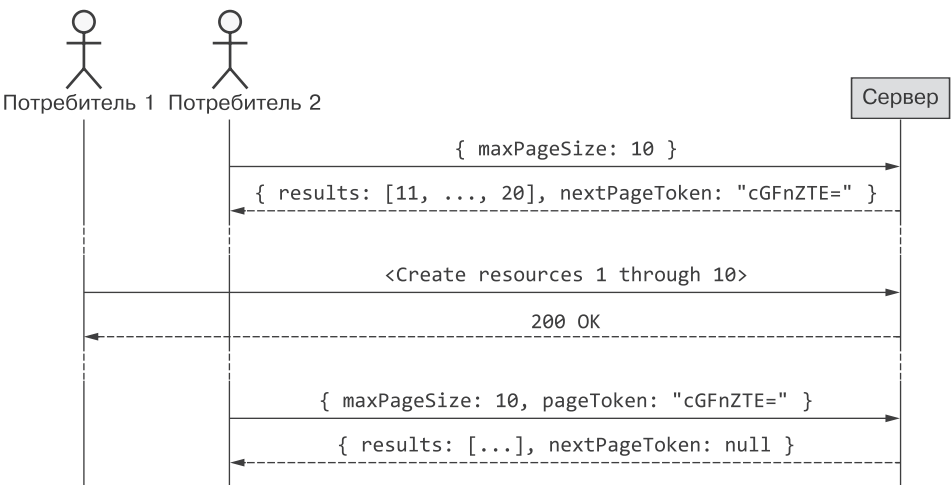


Рис. 21.4. Потребитель может увидеть дублирующие результаты, если в процессе пагинации будут добавлены новые данные

К сожалению, простого ответа здесь нет. Если ваша БД поддерживает моментальные снимки данных (как Google Cloud Spanner или CockroachDB), то может оказаться разумным кодировать эту информацию в маркеры страниц, чтобы можно было гарантировать строго последовательную пагинацию. Но зачастую базы данных не допускают строгую последовательность результатов. И тогда единственным разумным вариантом будет отметить в документации API, что из-за параллельного добавления, удаления или изменения внутренних данных информация может при отображении «съезжать» (рис. 21.5). Как уже говорилось, рекомендуется использовать в качестве указателя не численное смещение, а последний просмотренный результат, чтобы маркер очередной страницы действительно начинался с места последней остановки.



Рис. 21.5. Нумерация страниц может со временем изменяться ввиду изменения внутренних данных

Продолжительность жизни

Рассматривая получение большого списка результатов, мы склонны предполагать, что запросы происходят в относительно короткое временное окно, обычно измеряемое в секундах или минутах. Но что, если они будут происходить намного медленнее (как показано на рис. 21.6)? Естественно, речь не о том, что нам может потребоваться запрашивать очередную страницу через несколько лет, но как насчет 24 часов или даже 60 минут? К сожалению, мы редко документируем продолжительность жизни маркеров страниц, что может приводить к неудобным ситуациям.

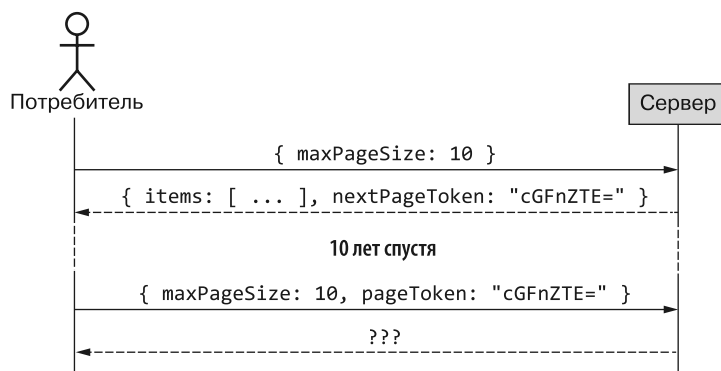


Рис. 21.6. Использование маркеров страниц десятилетней давности вряд ли окажется актуальным

Как правило, в API сроки действия маркеров не определяют. Поскольку пагинация данных должна являться идемпотентной операцией, режим сбоя ввиду просроченного маркера лишь потребует ее повтора. То есть сбой здесь просто вызывает неудобство.

Тем не менее нам нужно прояснить, как долго маркеры должны оставаться действительными, чтобы сориентировать ожидания потребителя относительно операций пагинации. В классическом случае определяется достаточно короткий промежуток жизни, исходя из конкретного сценария. В типичных API речь может идти о минутах или максимум часах. При этом достаточным сроком большинство потребителей считают 60 минут.

21.3.3. Общее количество

Напоследок разберем распространенный вопрос о том, стоит ли при пагинации включать общий подсчет результатов, который обычно используется для показа элементов пользовательского интерфейса наподобие «Результаты 10–20 из 184». Когда речь идет об относительно скромном общем количестве (например, 184),

показ такой статистики не вызывает сложностей. Однако когда результатов оказывается очень много (например, 4 932 493 534), простая операция подсчета может потребовать столько вычислений, что быстро вернуть ее точный итог окажется невозможным.

Здесь у нас встает выбор между медленным предоставлением точного количества и относительно быстрым выводом приблизительного. Ни один из этих вариантов нельзя назвать хорошим, поэтому в API обычно не рекомендуется предоставлять общий подсчет результатов, если только у потребителей нет в этом явной необходимости (то есть без этого они не смогут выполнять свою работу) или же само их количество заведомо никогда не станет настолько большим, чтобы создать сложности. Если же по какой-то причине вам обязательно нужно обеспечить эту функциональность, то для ее реализации в ответ следует добавить целочисленное поле с именем `totalResults` (листинг 21.2).

Листинг 21.2. Интерфейс ответа, включающий общий подсчет результатов

```
interface ListChatRoomsResponse {
    results: ChatRoom[];
    nextPageToken: string;
    totalResults: number;
}
```

← Используйте поле `totalResults` для отражения общего количества результатов в случае завершения пагинации

Теперь, когда мы знаем о маркерах все, пора рассмотреть относительно уникальный случай, в котором ресурс может оказываться достаточно большим, чтобы использовать пагинацию *внутри* него одного.

21.3.4. Пагинация внутри ресурсов

В некоторых сценариях один ресурс может сильно разрастись, ввиду чего пользователям придется перелистывать его содержимое, собирая из него полноценный ресурс с помощью нескольких запросов. В подобных сценариях можно опереться на те же принципы, что и в случае с коллекцией ресурсов, разбив один большой ресурс на фрагменты произвольного размера и используя маркеры продолжения как указатели для навигации по этим фрагментам (листинг 21.3).

Листинг 21.3. Определение API для пагинации по одному большому ресурсу

```
abstract class ChatRoomApi {
    @get("/{id=chatRooms/*/messages/*/attachments/*}:read")
    ReadAttachment(req: ReadAttachmentRequest):
        ReadAttachmentResponse;
}
```

← Пользовательский метод `Read` позволяет получать один ресурс небольшими частями

```
interface ReadAttachmentRequest {
    id: string;
    pageToken: string;
}
```

← Маркер продолжения из предыдущего ответа. Пустой указывает на запрос к первому фрагменту ресурса

```

maxBytes: number;
}

interface ReadAttachmentResponse {
  chunk: Attachment;
  fieldMask: FieldMask;
  nextPageToken: string;
}

```

Максимальное количество байтов для возвращения в заданном фрагменте. Если поле пустое, то используется предустановленное значение, например 1024 байта

Один фрагмент принадлежащих ресурсу данных

Список полей с данными из фрагмента для присоединения к ресурсу

Маркер для использования при запросе очередного фрагмента ресурса. Пустой означает, что это последний фрагмент

В этой схеме мы предполагаем, что считывание одного ресурса может быть строго последовательным. То есть если ресурс в процессе перелистывания его данных будет изменен, то запрос должен быть отменен. Здесь также заложена идея сборки ресурса воедино с помощью нескольких запросов, каждый из которых содержит определенное подмножество данных этого ресурса, присоединяемых к собранным на данный момент. Наглядно это показано на рис. 21.7. Кроме того, мы задействуем маску полей, чтобы явно выразить, какие поля интерфейса `ChatRoom` содержат данные, предназначенные для потребления в заданном фрагменте.

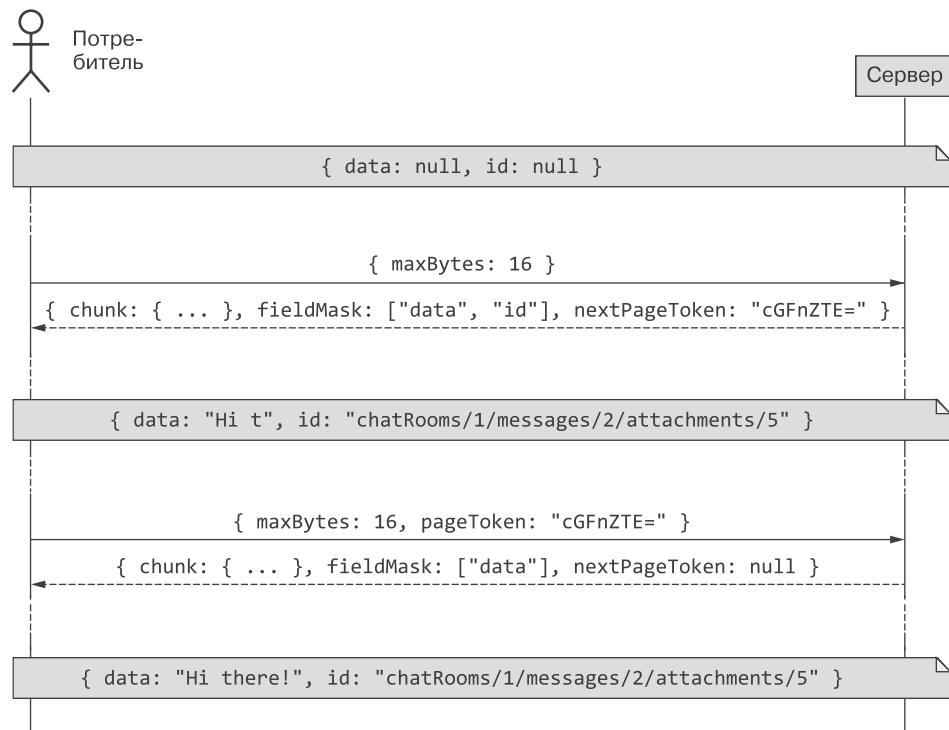


Рис. 21.7. Пример потока пагинации по одному большому ресурсу

21.3.5. Итоговое определение API

Разобравшись со всеми нюансами этого паттерна, можно взглянуть на заключительное определение API, охватывающее пагинацию как по коллекции ресурсов, так и по одному (листинг 21.4).

Листинг 21.4. Итоговое определение API

```
abstract class ChatRoomApi {
    @get("/chatRooms")
    ListChatRooms(req: ListChatRoomsRequest): ListChatRoomsResponse;

    @get("/{id=chatRooms/*/messages/*/attachments/*}:read")
    ReadAttachment(req: ReadAttachmentRequest): ReadAttachmentResponse;
}

interface ListChatRoomsRequest {
    pageToken: string;
    maxPageSize: number;
}

interface ListChatRoomsResponse {
    results: ChatRoom[];
    nextPageToken: string;
}

interface ReadAttachmentRequest {
    id: string;
    pageToken: string;
    maxBytes: number;
}

interface ReadAttachmentResponse {
    chunk: Attachment;
    fieldMask: FieldMask;
    nextPageToken: string;
}
```

21.4. КОМПРОМИССЫ

Теперь, когда мы ознакомились с получившимся API, пора поговорить о том, что мы теряем в случае применения текущего паттерна. Начнем с направления перелистывания страниц.

21.4.1. Двухнаправленная пагинация

Одной из очевидных вещей, которые невозможны в случае данного паттерна, является работа в обоих направлениях. Иначе говоря, этот паттерн не позволяет возвращаться к предыдущей странице результатов. И хотя это может быть

неудобно для пользовательских интерфейсов, которые позволяют просматривать результаты, для программного взаимодействия такой функционал вряд ли можно считать необходимым.

Если пользовательскому интерфейсу действительно нужна эта возможность, то неплохим вариантом будет применять API для формирования кэша результатов, по которому он впоследствии сможет свободно перемещаться. Дополнительным бонусом в этом случае станет исключение проблем с последовательностью пагинации.

21.4.2. Произвольный выбор окна

Аналогичным образом этот паттерн не предоставляет возможности переходить к определенной позиции внутри списка ресурсов. То есть никак нельзя попросить показать именно страницу 5, на что есть простая причина — отсутствие самого понятия нумерации страниц. Вместо того чтобы использовать их номера, потребители для перехода к конкретному соответствующему набору ресурсов должны использовать фильтры и упорядочивание. Опять же, этот функционал больше актуален для пользовательских интерфейсов, которые позволяют просматривать результаты, и не является необходимым для программного взаимодействия.

21.5. АНТИПАТТЕРН: СМЕЩЕНИЯ И ГРАНИЦЫ

Для полноты картины коротко рассмотрим простую (и довольно распространенную) реализацию аналога этого паттерна, которого обычно желательно избегать. Учитывая, что большинство баз данных поддерживают ключевые слова `OFFSET` и `LIMIT`, зачастую возникает соблазн перенести этот принцип в API как способ выражения окна для выделения части списка ресурсов. То есть вместо того, чтобы запрашивать некий фрагмент, мы выбираем конкретный фрагмент, запрашивая данные, начинающиеся с конкретного смещения, и ограничивая размер результата определенным числом, как это показано на рис. 21.8.



Рис. 21.8. Определение окна с помощью смещений и границ

На поверхности API это приводит к добавлению в запрос двух дополнительных целочисленных полей, указывающих начало смещения и границу запроса. Очередное смещение можно легко вычислить, прибавив количество результатов к ранее запрошенному смещению (например, `nextOffset = offset + results.length`). Более того, можно использовать в качестве критерия завершения неполную страницу (например, `hasMoreResults = (results.length != limit)`) (листинг 21.5).

Листинг 21.5. Поверхность API при использовании `offset` и `limit`

```
abstract class ChatRoomApi {
    @get("/chatRooms")
    ListChatRooms(req: ListChatRoomsRequest): ListChatRoomsResponse;
}

interface ListChatRoomsRequest {
    offset: number;
    limit: number;
}

interface ListChatRoomsResponse {
    results: ChatRoom[];
}
```

Чтобы поддержать пагинацию по ресурсам, мы добавляем параметры `offset` и `limit`

При переносе всего этого в базу данных делать, по сути, ничего не нужно: мы просто берем переданные параметры смещения и границы (например, выполняя запрос GET к <https://example.org/chatRooms/5/messages?offset=30&limit=10>) и передаем их вместе с запросом (листинг 21.6).

Листинг 21.6. Пагинация с использованием SQL LIMIT и OFFSET

```
SELECT * FROM messages
WHERE chatRoomId = 5
OFFSET 30
LIMIT 10
```

Модификатор `OFFSET` устанавливает номер страницы (в данном случае смещение 30 при выводе десяти результатов на страницу указывает на страницу 4)

Модификатор `LIMIT` устанавливает размер страницы (в данном случае десять результатов)

Фундаментальная проблема этого паттерна в том, что он допускает утечку деталей реализации в API, в связи с чем этот API должен продолжать поддержку смещений и границ независимо от внутренней системы хранения. Сейчас это может не вызывать опасений, но по мере усложнения системы хранения те реализации, которые используют смещения и границы, работать будут не всегда. Например, если ваши данные хранятся в распределенной системе с конечной согласованностью, то поиск начальной точки смещения по мере увеличения его значения может становиться все более затратным.

У этого паттерна есть проблемы и с последовательностью. В приведенном примере при добавлении новых данных это может привести к возвращению результатов, которые уже отображались на предыдущей странице.

21.6. УПРАЖНЕНИЯ

1. Почему важно использовать максимальный размер страницы, а не точный?
2. Что должно произойти, если поле с размером страницы оставить в запросе пустым? А если оставить отрицательным? А нулевым? А если указать в нем огромное число?
3. Как пользователь узнаёт, что пагинация завершена?
4. Имеет ли смысл использовать для маркеров страниц в некоторых типах ресурсов сроки действия, которые отличаются от используемых в других типах ресурсов?
5. Почему важно, чтобы маркеры страниц были непрозрачными для пользователей? Какой механизм позволяет обеспечить это наилучшим образом?

РЕЗЮМЕ

- Пагинация позволяет потреблять огромные коллекции результатов (или крупные одиночные ресурсы) в виде серии удобных для восприятия фрагментов; для этого используется три специальных поля: `maxPageSize`, `pageToken` и `nextPageToken`.
- Если впереди еще остались страницы, то ответ будет включать значение `nextPageToken`, которое можно передать в поле `pageToken` следующего запроса, чтобы получить очередную страницу.
- Мы используем максимальный размер страниц, а не точный, поскольку не знаем, сколько времени потребуется для заполнения страницы, и должны сохранить возможность возвращать ответ даже до того, как страница результатов будет заполнена целиком.
- Пагинация завершается, когда ответ не содержит значения для очередного маркера страницы (а не когда поле результатов окажется пустым).

22

Фильтрация

В этой главе

- ✓ Почему стандартные методы `List` поддерживают фильтрацию ресурсов.
- ✓ Как передавать выражение фильтра в составе стандартного запроса `List`.
- ✓ Правильная реализация поведения при фильтрации ресурсов.
- ✓ Какая функциональность должна и не должна поддерживаться в условиях фильтра.

Мы уже умеем использовать метод `List` для перебора всего набора ресурсов в API, но до сих пор у нас не было реального способа, позволяющего показать наш интерес только к подмножеству этих ресурсов. Текущий паттерн продемонстрирует использование в запросе `List` особого поля, которое будет выступать в качестве фильтра для всего набора данных в целях получения только соответствующих ему результатов. Мы также подробно разберем структуризацию входящих значений для этого поля.

22.1. ЗАЧЕМ ЭТО НУЖНО

До сих пор в качестве типичного способа извлечения всех ресурсов мы просто использовали стандартный метод `List`. Аналогичным образом, если мы ищем конкретный ресурс, идентификатор которого нам известен, то у нас есть такой же

простой инструмент: стандартный метод `Get`. Но что, если нас интересует некий промежуточный вариант? Что, если мы хотим просмотреть ресурсы, соответствующие конкретному набору критериев? То есть нас интересует не один и не все ресурсы, но при этом мы понимаем, какие именно. Как обработать такой запрос?

К сожалению, до этого момента у нас не было удачного способа выразить такой промежуточный вариант запроса. В качестве лучшего решения можно было разве что оттолкнуться от метода `List` и применить набор критериев к списку возвращаемых ресурсов (листинг 22.1).

Листинг 22.1. Пример фильтрации ресурсов на клиентской стороне

```
async function ListMatchingChatRooms(title?: string):
    Promise<ChatRoom[]> {
    let results: ChatRoom[] = [];
    let response: ListChatRoomsResponse;
    while (response === undefined ||
        response.nextPageToken) {
        response = await ListChatRooms({
            pageToken: response.nextPageToken
        });
        for (let chatRoom of response.resources) {
            if (title === undefined ||
                chatRoom.title === title) {
                results.push(chatRoom);
            }
        }
    }
    return results;
}
```

Повторение первой итерации до тех пор, пока остается очередная страница ресурсов для просмотра

Получение от API страницы ресурсов

Перебор каждого ресурса и проверка соответствия его поля title

Вполне очевидно, что эта структура далека от идеала. Самая заметная проблема здесь в том, что клиенту необходимо получить, перебрать и сопоставить с критериями фильтра весь набор хранящихся в API ресурсов. И это особенно расстраивает в пограничных случаях, когда совпадений нет, — ведь приходится отфильтровать все ресурсы, только чтобы в итоге узнать об отсутствии среди них нужных. Но даже по одному только объему потребления данных во время передачи становится ясно, что это чрезвычайно затратный способ. Как же решить проблему?

22.2. ОБЗОР

К счастью, решение в этой, казалось бы, непростой ситуации оказывается довольно очевидным. Вместо того чтобы извлекать все доступные ресурсы и отфильтровывать ненужные, мы развернем ситуацию и переложим эту ответственность на сам сервер API. Иными словами, нам лишь нужно объявить для сервера критерии, по которым он должен выполнять сопоставление, в результате чего итоговый набор будет содержать только те ресурсы, которые нас интересуют (листинг 22.2).

Листинг 22.2. Дополнительное поле `filter` в стандартном запросе `List`

```
interface ListChatRoomsRequest {
  filter: any;
  maxPageSize: number;
  pageToken: string;
}
```

Сейчас оно указано как `any`, поскольку мы еще не решили, как именно его реализовывать

Эти поля относятся к паттерну пагинации из главы 21

И хотя этот принцип дизайна может показаться очевидным, сама его реализация окажется несколько более сложной. Легко сказать, что нужно передать некий набор критериев серверу при перечислении ресурсов, но как конкретно должен выглядеть этот фильтр? Должен ли он быть просто строкой, напоминающей SQL-запрос? А может быть, более сложной структурой для сопоставления полей, как при фильтрации запросов документов в MongoDB?

Выбрав приемлемый формат для передачи запроса, необходимо подумать о том, какая функциональность должна поддерживаться в нем. Например, нужно ли допустить совпадения только в полях ресурса (скажем, совпадение с конкретным заголовком ресурса `ChatRoom`)? Или же стоит расширить эту операцию и на подстановочные символы (например, совпадение с ключевым словом в любой части заголовка)? Или даже допустить произвольный поиск текста, как в случае поискового запроса (например, нахождение ключевого слова в любом поле)?

А как быть с запросами элементов, которые не хранятся конкретно в ресурсе? Например, нужно ли разрешить запросы на основе метаданных полей, таких как количество участников `ChatRoom`? И создаст ли это возможность для появления запросов, расширяющихся на связанные ресурсы? К примеру, должен ли фильтр предоставлять способ, позволяющий ограничивать результаты поиска `ChatRoom` только теми, в которых состоит участник по имени Джо? Или теми, в которых есть участник с именем Джо, являющийся участником другой `ChatRoom`, включающей участника Лука? Если да, то насколько далеко должна распространяться эта возможность?

Думаю, эти примеры наглядно дают понять, что здесь может легко возникнуть громадное количество вариантов со множеством нюансов, не все из которых нужно поддерживать в спецификации фильтра. Цель текущего паттерна в том, чтобы решить эти ключевые вопросы, в частности показать, как определять фильтр и какая конкретно функциональность достойна поддержки.

22.3. РЕАЛИЗАЦИЯ

Первое, что нам нужно сделать, — это решить, как лучше всего передать фильтр, выражающий для сервера API намерение пользователя. Иными словами, есть много способов, позволяющих представить это намерение, но нам важно выбрать один и придерживаться его согласованно среди всех стандартных методов `List`, поддерживающих фильтрацию. Какой же формат окажется лучшим?

22.3.1. Структура

Аналогично ситуации с выбором из нескольких доступных в API форматов сериализации (например, JSON (<https://www.json.org/json-en.html>), или Protocol Buffers (<https://developers.google.com/protocol-buffers>), или YAML (<https://yaml.org/>)) здесь у нас также есть ряд возможностей для выражения фильтра при выводе списка ресурсов. Все эти варианты по своей сути разделяются на две категории: структурированные и неструктурированные.

В случае неструктурированных нам доступны такие инструменты, как SQL-запросы, в которых фильтр представляется в виде строкового значения, соответствующего очень специфичному синтаксису (листинг 22.3). Затем сервер API парсит это строковое значение и проверяет на предмет возвращения пользователю именно соответствующих результатов.

Листинг 22.3. Образец SQL-запроса для фильтрации ресурсов ChatRoom

```
SELECT * FROM ChatRooms WHERE title = "New Chat!";
```

Что же касается структурированных вариантов, то в них мы часть ответственности перекладываем с сервера API обратно на клиента, по сути, требуя от него предварительно парсить запрос в специальную схему, после чего отправлять этот комплексный интерфейс непосредственно серверу API для оценки (с небольшим парсингом контента или без него). И хотя такой способ не так популярен, как повсеместные строковые SQL-запросы, на структурированные интерфейсы для фильтрации опираются многие инструменты, например MongoDB (листинг 22.4).

Листинг 22.4. Образец структурированного запроса MongoDB для фильтрации ресурсов ChatRoom

```
db.chatRooms.find( { title: "New Chat!" } );
```

Но это подводит нас к определяющему вопросу: какой же способ лучше? Прежде чем углубиться в рассмотрение преимуществ и недостатков каждого из них, важно осознать, что этот выбор не играет большой роли. То есть, несмотря на то что конечный результат будет выглядеть в этих случаях по-разному, оба подхода вполне способны решать одни и те же задачи. В действительности преобразование одного формата в другой должно быть простым и понятным, в некотором смысле похожим на преобразование ресурса JSON в ресурс YAML. Однако в этом случае преобразование опирается на *сериализацию* при конвертировании структурированного представления в строку и на *парсинг* в обратном случае, подобно `JSON.stringify()` и `JSON.parse()` в Node.js. Пример показан на рис. 22.1.

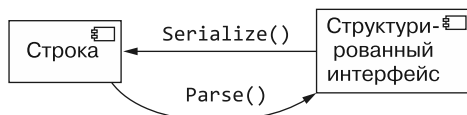


Рис. 22.1. Связь между строковыми и структурированными запросами

Два этих варианта функционально равнозначны, поэтому выбор между ними становится вопросом удобства и будущей гибкости. Иными словами, так как оба они нацелены на удовлетворение одних и тех же функциональных требований, нужно спросить себя: какой из них клиентам будет проще использовать и какой окажется более надежным в перспективе?

Как разработчики API, в этом сценарии мы первым делом практически всегда должны стремиться представить фильтры с помощью повторно используемого, гибкого и простого дизайна. Это стремление весьма похвально, однако, к сожалению, даже лучшие дизайны могут страдать от недочетов ввиду того, что входные данные должны соответствовать структурированной схеме.

Основная проблема заключается в том, что в конечном итоге эти фильтры ничем не отличаются от любого другого кода, который мы пишем на любом языке программирования. Сам фильтр определенно более ограничен, чем некий произвольный код, но если внимательно взглянуть на то, что мы здесь делаем, то на фундаментальном уровне мы просто заполняем специфичное определение функции пользовательским кодом (листинг 22.5). Этот код затем может применять функция `filter()` для массива ресурсов, чтобы определить нужные и исключить лишние.

Листинг 22.5. Пример оценки кода выражения фильтра

```
function resourceMatchesFilter(resource: Object): boolean {
  if (/* пользовательский код! */) {
    return true;
  } else {
    return false;
  }
}
```

Чтобы понять, почему нет смысла пытаться определить структурированное представление для фильтрации, представьте, что вместо получения небольшого выражения фильтра, которое может выступать в качестве условия инструкции `if` в листинге 22.5, мы получим для оценки произвольную функцию. В этом случае, когда мы оцениваем пользовательский код, будет ли реально хорошей идеей определить схему для этого произвольного ввода? Или же разумнее принять в качестве ввода определенные байты и поступить, как поступает большинство компиляторов: применить для содержимого правила синтаксиса, чтобы убедиться в его валидности? Есть причина, по которой мы все еще определяем функции, представляя действия нашей программы с помощью текстовых файлов, а не некой структурированной схемы, — и фильтры ничем в этом не отличаются. Но и это еще не все. Строковые фильтры рекомендуется применять и по другим причинам.

Во-первых, при использовании строкового значения синтаксические правила применяются и проверяются исключительно сервером API. Это означает, что

любые изменения или доработки (например, функциональность) регулируются самим сервером, а значит, не требуют внесения корректировок клиентом. Как мы узнаем в главе 24, возможность вносить доработки, не требуя каких-либо изменений или действий со стороны клиента, может оказаться очень полезной.

Во-вторых, строковые запросы обычно оказываются хорошо известны и понятны для пользователей API, в частности, если им доводилось использовать реляционную базу данных, оперирующую на языке SQL. То есть пользователю будет намного проще понять строковый запрос, так как он опирается на уже знакомые ему принципы.

Наконец, это может показаться удивительным, но даже лучшие по дизайну структуры склонны вызывать трудности в понимании по мере усложнения критериев фильтрации. В табл. 22.1 наглядно показано, как различные условия фильтра могут быть представлены в виде строк в формате SQL по сравнению со структурированным объектом фильтра в MongoDB.

Таблица 22.1. Строковые SQL-запросы в сравнении со структурами фильтров MongoDB

SQL-запрос	Фильтр MongoDB
<code>title = "New Chat!"</code>	<code>{ title: "New Chat!" }</code>
<code>userLimit >= 5</code>	<code>{ userLimit: { \$gt: 5 } }</code>
<code>title = "New Chat!"</code> <code>AND</code> <code>admin = "1234"</code>	<code>{ title: "New Chat!", admin: "1234" }</code>
<code>title IN ("New Chat!", "Old Chat!")</code>	<code>{ title: { \$in: ["New Chat!", "Old Chat!"] } }</code>
<code>title = "New Chat!"</code> <code>OR</code> <code>title = "Old Chat!"</code>	<code>{ \$or: [{ title: "New Chat!" }, { title: "Old Chat!" }] }</code>
<code>(title = "new chat!"</code> <code>AND userLimit >= 5)</code> <code>OR</code> <code>title = "old chat!"</code>	<code>{ \$or: [{ title: "New Chat!", userLimit: { \$gt: 5 } }, { title: "Old Chat!" }] }</code>

Как видите, хоть оба варианта и могут выразить одинаковые логические условия, строки в стиле SQL для многих из нас окажутся более понятными. Причина, вероятно, в том, что они во многом похожи на логические выражения, которые

мы пишем в типичных языках программирования. Иначе говоря, так мы все больше и больше опираемся на то, что уже знакомо пользователям, а не требуем от них осваивать нечто новое специально для нашего API.

Таким образом, хоть структурированный интерфейс для выражения критериев фильтра и сработает, он куда вероятнее приведет к более хрупкому определению API, которое с течением времени будет только усложняться. Кроме того, пользовательские структурированные интерфейсы практически всегда требуют усердного изучения, не давая при этом особых преимуществ в сравнении со строковым представлением. В результате, несмотря на соблазн попробовать спроектировать необычную структуру для представления этих фильтров, почти всегда безопаснее будет положиться на строковое поле, которое сможет развиваться параллельно с вашим API по мере роста самого сервиса.

Теперь, когда мы выяснили, почему представлять фильтр лучше с помощью неструктурированного значения (например, строки), пора углубиться, собственно, в сам синтаксис, который следует использовать для этой строки.

22.3.2. Синтаксис фильтра и поведение

То, что мы вызываем строку неструктурированной, еще не значит, что эти строковые значения вообще лишены каких-либо правил. Наоборот, неструктурированная природа строки относится к типу данных и схеме, а не формату самого значения, которое для того, чтобы считаться допустимым, должно соответствовать конкретному набору правил. Но что это за правила? Каков правильный синтаксис для строки фильтра?

Какими бы интересными ни казались разработка и определение полноценного языка для фильтрации ресурсов, это может выйти далеко за рамки темы книги. В действительности для этого уже существует немало языковых спецификаций, например, Common Expression Language (CEL; <https://github.com/google/cel-spec/blob/master/doc/langdef.md>), JSONQuery (<https://www.sitepen.com/blog/jsonquery-data-querying-beyond-jsonpath>), RQL (<https://github.com/persvr/rql>) или спецификация Google Filtering (<https://google.aip.dev/160>). И в некоторых случаях даже может быть разумно попытаться опереться на внутренние возможности системы хранилища по составлению запросов, например, принудительно обеспечивая соответствие синтаксиса запроса конкретному подмножеству эквивалента SQL-оператора `WHERE` с довольно большим количеством ограничений в допустимой функциональности.

Вместо того чтобы потенциально снова изобретать колесо или даже обуславливать определенный «правильный» синтаксис, мы разберем некоторые характеристики и ограничения, которые, скорее всего, дадут наилучшие результаты, независимо от конкретной реализации и синтаксиса.

Время выполнения

Несмотря на то что выражения фильтров должны состоять лишь из простых сравнений (например, `title = "New Chat!"`), оказывается, что эти сравнения могут довольно сильно усложниться. Если взять книгу по синтаксису SQL и функциональности различных баз данных, то можно удивиться, насколько сложными порой становятся эти виды простых инструкций. Например, что, если мы изменим правую сторону сравнения с литерала (скажем, конкретной строки наподобие `"New Chat!"`) на другую переменную (скажем, `title = description`)? Или что, если втянуть в процесс другой ресурс (например, `administrator.employer.name = "Apple"`)? Где нужно проводить черту между простым условием фильтра и чем-то излишне сложным для нашего языка фильтрации?

В первую очередь при выборе поддерживаемой функциональности мы больше учитываем практический аспект: время выполнения функции оценки. Это означает, что нам нужно вернуться к основам и разбору тем нотации «О большое» и определению худших сценариев для выполнения функций. В этом случае нас интересует, существуют ли строки фильтра, которые могут сделать функцию чрезвычайно медленной или вычислительно затратной. В качестве абсурдного примера представьте, что мы реализовали способ фильтрации данных, получаемых с сервера на Луне, или таких, которые требуют сканирования всех строк в очень большой базе данных. Очевидно, что это приведет к невероятно медленной фильтрации, существенно ухудшив работу пользователей. К счастью, есть довольно простая стратегия, позволяющая гарантировать, что время выполнения функции оценки останется в разумных пределах: ограничить данные, доступные в качестве входных.

Общее правило здесь состоит в том, что фильтры для успешной оценки должны требовать контекст только одного ресурса. То есть если рассматривать функцию оценки, то она будет принимать в качестве входных параметров всего два элемента: саму строку фильтра и ресурс, который может соответствовать ее критериям или нет (листинг 22.6). Это означает, что не получится включить какие-либо дополнительные данные, не являющиеся частью ресурса (например, данные, полученные с Луны).

Листинг 22.6. Оценочная функция получает только фильтр и сопоставляемый ресурс

```
function evaluate(filter: string, resource: ChatRoom): boolean {
    // ...
}
```

Если вместо этого мы разрешим получать для оценки несколько ресурсов (листинг 22.7), то перед нами возникнет еще несколько вопросов. Например, нужно ли допустить неограниченное количество дополнительных ресурсов для сравнения или же следует ограничить их определенным количеством (скажем, пятью)? Как загрузить эту информацию в контекст? Должны ли мы разрешить

ресурсы разных типов или же надо ограничить их типом того, который оценивается? Все это также подразумевает дополнительные сложности с производительностью, такие как необходимость предварительного извлечения или кэширования информации ресурса, которая может использоваться для оценки, и ряд других проблем, связанных со свежестью ресурсов и согласованностью данных.

Листинг 22.7. Оценочная функция с поддержкой дополнительного контекста

```
function evaluate(filter: string, resource: ChatRoom, context: Object[]):
    boolean {
    // ...
}
```

Все эти проблемы решаемы, однако вряд ли усилия будут стоить того. То есть расширение спектра доступных для сравнения переменных может время от времени оказаться полезным. Однако этот функционал далек от необходимого, если придерживаться истинной задачи фильтрации, которая состоит в ограничении результатов на основе простых условий, касающихся самих целевых ресурсов. В итоге практически всегда желательно избегать необходимости отвечать на все эти вопросы. Вместо этого можно просто потребовать, чтобы условия фильтра получали на входе всего два элемента информации: само выражение фильтра и один ресурс, который может соответствовать ему или нет.

Это будет хорошим началом, но оно, к сожалению, не решит все возможные проблемы. Мы поняли, что синтаксис фильтра должен привлекать к оценке всего один ресурс, однако ничего не сказали о том, что именно может делать выражение фильтра во время этой операции. Например, что помешает фильтру обратиться к другому ресурсу, используя связь из начального ресурса? Иными словами, должны ли мы разрешить фильтру обращаться к другим ресурсам и извлекать их откуда-либо (например, из базы данных) для оценки выражения?

Ответом будет твердое «нет». И мы разовьем эту мысль: выражение фильтра обычно не должно иметь никакой возможности связываться с внешними системами. Наоборот, оно должно оставаться *герметичным* и изолированным от внешнего мира. Это означает, что фильтр не должен иметь доступа к связанным с ресурсом данным, если только они уже не вложены в сам этот ресурс. Например, если мы хотим выполнить фильтрацию на основе имени администратора ресурса `ChatRoom`, то это должно быть возможно, только если поле `administrator` непосредственно включает эту информацию. Если же поле `administrator` является строковым идентификатором, ссылающимся на другой ресурс, то фильтры, которые ожидают от системы, что она разыменует это поле, должны давать сбой (например, `administrator.name = "Luca"`).

Если бы оценочные функции фильтра не были герметичны и могли сообщаться с внешним миром (например, запрашивать новые данные ресурса из БД), это бы вскрыло множество дополнительных вопросов, аналогичных тем, о которых мы

говорили. Например, насколько далеко можно в таком случае уйти по связанным ресурсам? Что, если хранящиеся в одном из полей данные при извлечении окажутся в системе с плохим аптаймом или высокой задержкой (например, сервер на Луне)? В конечном итоге изолирование функции и предоставление ей возможности опираться только на данные, переданные в виде параметров, снижает вероятность ее невероятно длительного или вычислительно затратного выполнения. В результате чаще всего желательно реализовывать эту функцию герметичной и изолированной от внешних сервисов, чтобы все необходимые данные уже присутствовали в самом оцениваемом ресурсе.

Адресация массивов

При фильтрации ресурсов большую часть времени эти условия фильтра основываются на конкретных полях (например, `title = "New ChatRoom!"`), и определяется такое поле довольно легко. Тем не менее существуют случаи, в которых указать, какое поле должно использоваться для сравнения, бывает не так просто. Очевидный пример — специфичные ключи в полях карты. Они способны вызывать сложности в первую очередь из-за того, что могут содержать зарезервированные символы, требуя от любого синтаксиса фильтра поддержки экранирования, которое должно прояснять, в каких случаях зарезервированный символ нужно рассматривать как рабочий, а в каких — воспринимать буквально. Но есть и другая важная проблема, которая связана не столько с тем, как мы указываем значение, сколько с последствиями этого. Речь идет о значениях в массиве.

Когда поле хранит не одно значение, а их массив и мы хотим выполнить фильтрацию на основе этих отдельных значений, нам необходимо решить, как именно адресовать их в выражении фильтра. Иными словами, как нужно фильтровать ресурсы на основе первого элемента в поле массива, соответствующем конкретному значению?

Несмотря на то что это вполне можно выразить во многих языках программирования (например, `tags[0] = "new"`), такой подход редко окажется удачным, поскольку подразумевается требование, что элементы массива должны быть статичными. Адресация отдельных элементов неизбежно требует, чтобы они сохраняли строгий порядок. Это кажется простым, но зачастую влечет за собой серьезную путаницу при работе с такими значениями, поскольку изменение порядка элементов в значении массива, по сути, ведет к совершенно новому значению, а не просто его несколько измененной форме. К примеру, если мы внедрим строгие правила упорядочивания, будет ли это означать, что новые значения всегда должны добавляться в конец массива? Эти вопросы могут казаться простыми, но все они накапливаются и быстро выходят из-под контроля.

Вместо этого куда более вероятно, что пользователи захотят проверить, присутствует ли вообще некое значение в массиве, а не то, какое значение находится по

определенному индексу. В итоге будет намного удобнее предоставить механизм для проверки того, содержит ли поле проверяемое значение (например, `tags: "new"` или `"new" in tags`), а не для проверки элементов в конкретной позиции массива. Иными словами, зачастую будет лучше рассматривать поля массива как неупорядоченные или имеющие неопределенный порядок, при котором элементы никогда не имеют конкретной позиции, а просто присутствуют в массиве либо нет. А в случаях, когда потребуются упорядоченные списки элементов, опираться на специальное поле, указывающее приоритет или позицию нужного элемента, и выполнять фильтрацию на основе этого (например, `item.position = 1 and item.title = "new"`).

Здесь может показаться уместным заявить, что фильтры только считывают данные, а значит, обработать все это не так уж сложно — в конце концов, в случае неудачи мы просто получим иные результаты фильтрации; то есть не можем случайно стереть все данные в API. Это определенно так, но одной из ключевых целей хорошего API является его согласованность и прогнозируемость. А уточненные требования к упорядочиванию, ведущие к ложным отрицательным результатам (например, когда мы ожидаем, что ресурс совпадет с фильтром, но из-за порядка элементов этого не происходит), могут вызвать куда больше недоговаривания, чем кажется. В итоге лучше все же противиться желанию реализовать эту возможность и вместо этого сосредоточиться только на поддержке проверки присутствия элементов в массиве.

Строгость

Как и в случае любого определяемого нами синтаксиса, здесь возникает логичный вопрос о степени строгости его правил. Например, одни языки программирования довольно гибкие (скажем, в JavaScript с его иногда необязательными символами точки с запятой), а в других наблюдается крайняя жесткость (как во многих случаях в C++). В конечном итоге все сводится к тому, сколько попыток угадывания интерпретатор готов сделать при обработке входных данных. Тогда возникает следующий очевидный вопрос: насколько строгими должны быть выражения фильтра? Должны ли они быть гибкими и перебирать множество догадок об истинном намерении пользователя, когда оно неоднозначно? Или же эти выражения обязательно должны являться жесткими, не делая никаких предположений о том, что пользователь мог иметь в виду? К примеру, при указании маски полей мы требуем использовать экранирование и обратные кавычки для строк с особыми символами, такими как пробелы или точки, но не при использовании стандартных букв от A до Z.

Зачастую возникает соблазн сделать выражения фильтра достаточно гибкими, так как их основная задача — считывать данные. То есть поскольку неудача в случае неверной интерпретации выражения фильтра не ведет к чему-то катастрофическому, то почему бы не добавить в этот процесс немного догадок? Довод верный, но он не обязательно делает такое решение правильным.

Начнем с того, что у нас есть очевидный случай пользовательского метода `Purge`, который получает список элементов, сопоставляя их с определенными критериями фильтра (глава 19). В этом случае сценарий сбоя окажется действительно катастрофическим и может привести как к удалению тех ресурсов, которые не надо удалять (ложные положительные совпадения), так и к сохранению таких, которые нужно удалить (ложные отрицательные несовпадения). Такой паттерн, хоть обычно и не рекомендуется к использованию, определяется строгостью синтаксиса выражения фильтра, и неверная интерпретация намерений пользователя ввиду недостатка конкретности может вызвать много проблем.

Стоит также отметить, что несоответствия (ложные положительные или ложные отрицательные) могут не быть катастрофическими в случаях только считывания, но это не делает их менее раздражающими. Представьте, что мы пытаемся отфильтровать ресурс на основе значения поля (например, отбираем ресурсы `ChatRoom` с `title = "New"`), но случайно допускаем ошибку при наборе имени поля (например, пишем `ttitle` вместо `title`). Очевидно, что выражение фильтра не сможет догадаться, какое поле в действительности подразумевалось, но нам нужно задаться вопросом: должен ли такой фильтр выдавать ошибку?

С другой стороны, можно поступить так, как мы действовали с масками полей, то есть постараться извлечь значение этого странного поля (`ttitle`) и вернуть отсутствующее значение (например, `undefined` в JavaScript). Оно бы точно не совпало с проверяемым значением (`New`), и этот результат привел бы к отфильтровыванию всех возможных ресурсов. Иными словами, значение этого извлечения всегда будет разрешаться в `undefined` и никогда не окажется равным `New`, значит, мы, по сути, просто отфильтровали все возможные результаты.

Еще один вариант довольно прост: если ссылка на поле недействительна, то выбрасывать ошибку. В таком случае вместо досадной необходимости выискивать опечатку в выражении фильтра мы можем узнать о ее нахождении от самого API (например, `Error: Invalid field specification at position 0 ("ttitle")`). И хотя мы все ненавидим подобные ошибки, будет куда печальнее получать их едва уловимые формы, вызывающие проблемы в дальнейшем.

Аналогичная стратегия применима и к другим деликатным сценариям, таким как приведение значений из одного типа в другой для должного сравнения (например, преобразование строкового `1234` в численное `1234` или наоборот). Во время как вольная интерпретация может просто привести к возвращению несоответствующих результатов для заданного условия, строгость и возврат ошибки в случае недостатка ясности окажется намного полезнее для тех, кто пытается отфильтровать ресурсы. К примеру, если условие сравнивает численное поле со строковым значением (скажем, `userCount= "string"`), то оно точно никогда не будет оценено как `true`. Тем не менее, вместо того чтобы просто возвращать `false`, система фильтрации должна возвращать ошибку для явно ошибочного условия.

Пользовательские функции

Наконец, поскольку API всегда имеют собственный набор уникальных требований, с большой вероятностью будут возникать ситуации, когда пользователям потребуется обходить эти преграды ради случаев, которые не поддерживаются базовым синтаксисом фильтрации нативно. К примеру, хоть обычно мы и не хотим по умолчанию закладывать в выражения фильтра возможность выполнения ресурсно-затратных вычислений, есть один уникальный случай, когда пользователю она реально потребуется. И хотя мы не хотим разрешать подобные действия в полной мере, нам все же нужен способ, позволяющий выполнять их в исключительных случаях. Как это можно реализовать?

Типичное решение для того, чтобы позволить пользователям огибать или нарушать правила в уникальных обстоятельствах, состоит в том, чтобы задействовать в выражениях особые вызовы функций. То есть мы не хотим позволить выполнение произвольного кода, связывающегося с внешними ресурсами, или замысловатые действия с данными, предоставив для этого нативный синтаксис, поэтому можем предложить вспомогательные функции для совершения подобных действий в контролируемом русле.

Чтобы понять, как это может работать, рассмотрим пример выполнения более продвинутых сопоставлений строк, таких как сопоставление на основе префиксов или суффиксов, а не полное сравнение. Чтобы справиться с этим, можно расширить синтаксис фильтра, возможно, добавив поддержку символов подстановки (например, `title = "*(new)"`), но это чревато множеством новых проблем (например, теперь нам потребуется экранирование для сопоставления с конкретно символом звездочки). Вместо этого можно использовать особый вызов функции `endsWith`, который будет выполнять аналогичные действия (например, `endsWith(title, "(new)")`). Можно даже предоставить более простую вспомогательную функцию для управления строками, например, такую, которая будет извлекать из строки подстроку. В итоге фильтрация префиксов может выглядеть так: `substring(title, 0, 5) = "(new)"`.

И эти особые функции не должны ограничиваться просто манипулированием строками. Если в API есть способ выполнения продвинутого анализа данных с помощью машинного обучения, то его можно выразить как особую функцию. Например, если ресурсы `User` содержат поле `profilePhoto` и у нас есть API для распознавания элементов на изображении (скажем, лиц, достопримечательностей или известных объектов), то можно предоставить функцию, которая позволит проверять их присутствие (например, `imageContains(profilePhoto, "dog") = true`).

И здесь нужно учитывать не потенциальные возможности этих функций, а то, что они просто являются способом обойти базовые правила синтаксиса. Таким образом, синтаксис для фильтрации ресурсов остается простым и понятным, но при этом у пользователей API все равно сохраняется возможность выполнять более продвинутые операции, если они окажутся действительно необходимыми для конкретной ситуации.

22.3.3. Итоговое определение API

В этом случае определение API для фильтрации ресурсов довольно простое и включает лишь расширение стандартного запроса `List` с помощью поля фильтра (листинг 22.8). Тем не менее решение о принятии этим полем строкового типа является очень важным.

Листинг 22.8. Итоговое определение API

```
abstract class ChatRoomApi {
    @get("/chatRooms")
    ListChatRooms(req: ListChatRoomsRequest): ListChatRoomsResponse;
}

interface ListChatRoomsRequest {
    filter: string;
    maxPageSize: number;
    pageToken: string;
}

interface ListChatRoomsResponse {
    results: ChatRoom[];
    nextPageToken: string;
}
```

22.4. КОМПРОМИССЫ

Самый очевидный выбор состоит в том, поддерживать вообще фильтрацию или нет. Несмотря на то что предоставление возможности фильтровать ресурсы в стандартном запросе `List` зачастую оказывается ресурсно-затратным, возникающие за счет этого преимущества для пользователей определенно перевешивают любые дополнительные краткосрочные нагрузки. Более того, если пользователи будут вынуждены реализовывать собственные механизмы фильтрации, запрашивая все ресурсы, то будут тратить не только свое время, но и вычислительные ресурсы API, чтобы извлечь и передать огромный объем информации, которую можно было отбросить заранее. В итоге, за исключением случаев, когда фильтрация не является критически необходимой, стандартный метод `List` для большинства ресурсов должен поддерживать эту функциональность, особенно если количество извлекаемых ресурсов может значительно разрастись (например, измеряться в нескольких сотнях или даже тысячах).

Кроме того, вероятно, самым большим компромиссом, рассмотренным в подразделе 22.3.1, является выбор между структурированным фильтром и неструктурированной строкой. Оба варианта технически выражают одно и то же, но использование строки позволит упростить обработку дальнейших изменений формата структуры, не прибегая к необходимости доносить до пользователей новые технические спецификации API (например, документы спецификации OpenAPI).

22.5. УПРАЖНЕНИЯ

1. Каков основной недостаток использования структурированного интерфейса для условия фильтра?
2. Почему идея позволить выполнять фильтрацию на основе позиции элемента в поле массива будет неудачной?
3. Как обрабатывать ключи карт, которые могут существовать или нет? Нужно ли действовать строго и выбрасывать ошибки при выполнении сравнения с несуществующими ключами? Или же следует рассматривать их как отсутствующие?
4. Представьте, что пользователь хочет отфильтровать ресурсы на основе суффикса поля (например, найти всех людей с именами, оканчивающимися на man). Как это можно выразить в строке фильтра?

РЕЗЮМЕ

- Предоставление фильтрации в стандартных методах `List` избавляет пользователей от необходимости получать все данные для того, чтобы найти только те, которые им интересны.
- Спецификации фильтра, как правило, должны являться строками, следующими определенному синтаксису (подобно запросам SQL), а не структурированным интерфейсом.
- Чтобы избежать безудержного увеличения времени выполнения, выражения фильтра должны требовать в качестве контекста оценки только один ресурс.
- Фильтры не должны предоставлять способ сравнения элементов ресурса на основе их позиции, или индекса, в повторяющемся поле (например, массиве).
- Ошибки, обнаруженные во время фильтрации, должны выявляться сразу же, а не игнорироваться или скрываться.
- Если для выражения намерения пользователя недостаточно базовых сравнений, то фильтры должны предоставлять набор задокументированных функций, которые можно будет интерпретировать и выполнять в процессе фильтрации.

23

Импорт и экспорт

В этой главе

- ✓ Специфичные определения для *импорта* и *экспорта*.
- ✓ Непосредственное взаимодействие с удаленными хранилищами.
- ✓ Конвертация сериализованных байтов в ресурсы API.
- ✓ Обработка согласованности для экспорта волатильных наборов данных.
- ✓ Обработка уникальных идентификаторов при импорте и экспорте.
- ✓ Как поступать со связанными импортируемыми или экспортируемыми ресурсами.
- ✓ Обработка сбоев во время операций импорта или экспорта.
- ✓ Чем импорт и экспорт отличаются от резервного копирования и восстановления.
- ✓ Фильтрация входящих и исходящих данных.

Этот паттерн поможет нам узнать, как безопасно и гибко перемещать ресурсы в API и из него с помощью пользовательских методов `Import` и `Export`. Что еще более важно — он основан на взаимодействии API напрямую с внешними хранилищами, без использования промежуточного клиента. Все это мы реализуем с помощью определения нескольких слабо связанных структур конфигурации,

чтобы охватить различные аспекты этих ресурсов на пути между API и внутренними системами хранения данных, максимизировав как гибкость, так и возможность повторного использования.

23.1. ЗАЧЕМ ЭТО НУЖНО

В любом API, который управляет передаваемыми пользователем данными (то есть практически в каждом), мы уже определили несколько способов, позволяющих добавлять ресурсы в систему и извлекать их из нее. Одни из этих способов работают с отдельными ресурсами (например, весь комплект стандартных методов), а другие ориентированы на их группы (например, группа пакетных методов). Но у всех них есть одна важная общая особенность: данные всегда передаются из сервиса API обратно клиенту напрямую.

Ниже мы рассмотрим случай, в котором у нас есть некие сериализованные данные, которые нужно загрузить в API, а именно набор ресурсов `Message`, создаваемых в ресурсе `ChatRoom`. Один из способов, позволяющих реализовать это, — добавление прикладного сервера, который будет извлекать данные из хранилища, парсить сериализованные байты в фактические ресурсы и затем создавать в API эти новые ресурсы с помощью традиционных методов. И хотя это может вызывать неудобства, такой подход определенно справится с задачей.

Есть один ключевой аспект, который может вызвать проблему: маршрут, который необходимо проделать данным. В этом случае им предстоит пройти долгий путь, начиная от системы хранилища, через прикладной код, выполняющий загрузку, и затем уже в API, как показано на рис. 23.1.



Рис. 23.1. Импорт данных требует промежуточного приложения загрузки данных

Если данные окажутся на одной машине с прикладным сервером (или даже просто неподалеку), то эта задача станет не слишком пугающей. Но что, если инфраструктура будет организована не столь дружелюбно? К примеру, вдруг данные и API находятся рядом, но прикладному серверу не разрешено находиться рядом? Это означает, что пользовательская функция для загрузки данных будет передавать весь их набор сначала до прикладного сервера, а затем обратно к серверу API. Это все равно что ходить в магазин всякий раз, когда вам нужен стакан молока, вместо того чтобы купить упаковку, которую можно хранить в холодильнике: в обоих случаях потребуется затратить энергию, но во втором варианте это будет намного эффективнее.

Эта проблема касается и данных, передаваемых в обратном направлении. Если вместо того, чтобы загружать новые ресурсы в систему, мы хотим взять имеющиеся ресурсы из системы, то применение для этой цели пользовательского кода требует передачи данных через промежуточный прикладной сервер. А по мере увеличения объема данных, характерного для современных систем, эта проблема будет лишь усугубляться.

В связи с этим существует довольно весомый аргумент в пользу того, чтобы данные передавались напрямую между API и удаленным хранилищем. Это не только минимизирует объем кода, который придется писать пользователям, но и снизит затраты пропускной способности и количество вычислительных ресурсов, применяемых пользователем для перемещения данных из одного места в другое. В следующем разделе мы приведем решение, позволяющее изменить этот неэффективный процесс.

23.2. ОБЗОР

Изучая этот паттерн, мы будем опираться на два особых пользовательских метода: `Import` и `Export`, представленные на рис. 23.2. Наша основная цель — позволить API взаимодействовать напрямую с внешней системой хранения, так что каждый из этих методов будет отвечать за передачу всех данных между сервисом API и этим хранилищем. Более того, поскольку способ хранения данных в подобных системах вряд ли всегда будет совпадать со способом их хранения

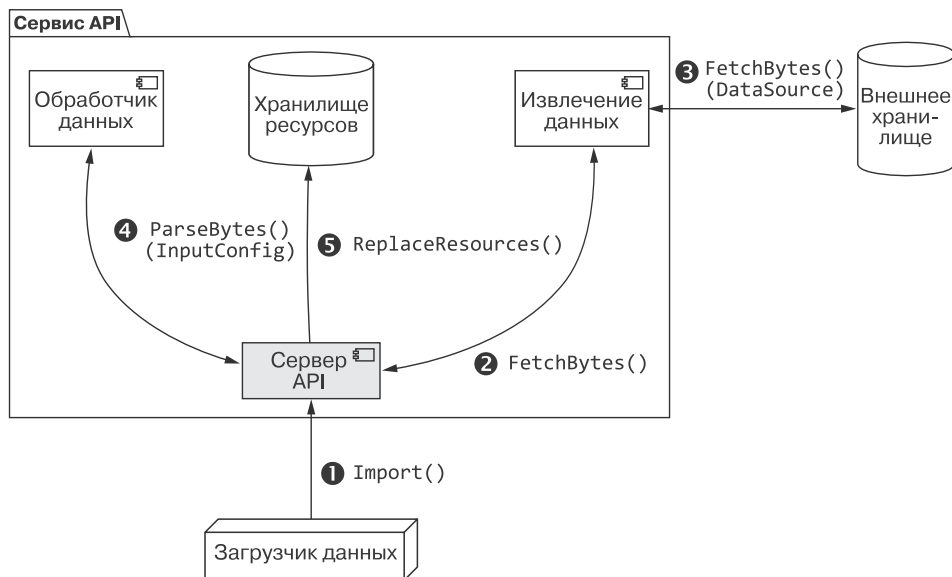


Рис. 23.2. Операция `import` совмещает извлечение и передачу данных в сервис API

в сервисе API, эти методы также будут отвечать за преобразование ресурсов API в сырые байты (и наоборот), которые хранилище сможет распознать.

Обе эти конфигурации чреваты проблемами. Существует невероятное множество систем хранения данных, поэтому крайне важно структурировать сообщения запросов для этих пользовательских методов так, чтобы можно было с легкостью вносить новые по мере их востребованности пользователями API. Кроме того, существует и огромное количество форматов сериализации, с помощью которых можно преобразовывать ресурсы API в сырые байты. Поэтому важно, чтобы структура запроса была достаточно гибкой и могла поддерживать любые из них, но при этом могла развиваться и впоследствии включать новые.

И если это еще не показалось вам достаточно сложным, то после сериализации данных в кучу байтов может возникнуть потребность в выполнении дальнейших преобразований. Например, данные перед сохранением на диск обычно подвергаются сжатию или шифрованию. А поскольку эти пользовательские методы должны взаимодействовать напрямую с внешним хранилищем, они также отвечают за предоставление пользователям API возможности настраивать эти дополнительные параметры.

Чтобы это все заработало, мы будем использовать в качестве основного ввода в методы импорта и экспорта комплексный интерфейс конфигурации. Эти интерфейсы (`InputConfig` для импорта и `OutputConfig` для экспорта) будут отвечать за сохранение всей актуальной информации как для преобразования данных между ресурсами, так и для передачи их байтов во внешнее хранилище и из него.

Поскольку количество систем хранения огромно, вариативно и постоянно меняется, мы задействуем структуры, которые позволят нам хранить конфигурацию передачи данных отдельно и независимо от их преобразования. Иными словами, мы отделим настройку доступа к внешнему хранилищу от настройки сериализации ресурсов API или десериализации байтов. Для этого мы используем обобщенные интерфейсы `DataSource` и `DataDestination`, чтобы каждый расширяющий интерфейс включал всю конфигурацию, необходимую для передачи байтов к целям или от источников.

Как только мы разберем структурные моменты и решим, как выстроить все детали конфигурации, останутся непростые вопросы, связанные с поведением каждого из этих пользовательских методов. Например, как нужно обрабатывать сбой и определять, безопасно ли повторение запроса? А как обстоит дело с внутренней согласованностью данных? Как поступать во время экспорта: делать полный снимок данных и передавать его или же передать смазанное представление? Аналогичным образом, как при выполнении импорта новые ресурсы должны взаимодействовать с существующими? И как обрабатывать коллизии идентификаторов? Нужно ли их удалять перед экспортом данных?

В следующем разделе мы подробно изучим все эти вопросы, связанные со структурой и поведением.

23.3. РЕАЛИЗАЦИЯ

Теперь, когда мы обрисовали общие проблемы, пора переходить к разбору деталей. Начнем с рассмотрения самих пользовательских методов `Import` и `Export`, после чего изучим структуру соответствующих им интерфейсов запроса.

23.3.1. Методы `Import` и `Export`

Поскольку наша цель состоит в импорте данных во внешнее хранилище и экспорте из него, первым делом нужно определить соответствующие методы, которые будут обрабатывать всю сложность этих операций. И хотя мы уже в общих чертах обговорили суть таких методов, важно отметить, что они будут закреплены за конкретным ресурсом API, подобно тем, которые мы видели в главе 9. То есть каждый пользовательский метод `Import` или `Export` будет отвечать за один тип ресурса (например, `ImportMessages` или `ExportMessages`).

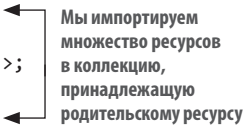
Следующим достойным внимания свойством этих методов является возвращаемый тип. Как мы узнали в главе 10, когда методы API выполняются не мгновенно и могут занимать некоторое время, лучше реализовывать их возврат в виде своеобразного промиса, обещающего завершить работу позднее. Несмотря на то что эти методы могут завершаться относительно быстро, риск их длительного выполнения все же сохраняется, в результате чего они определенно подпадают под инструкцию вместо непосредственного результата возвращать длительную операцию (LRO) и ее конечный результат.

Объединяя эти два нюанса и следуя прочим руководствам из главы 9, мы можем легко написать два образца методов для импорта ресурсов `Messages` в заданный ресурс `ChatRoom` (листинг 23.1).

Листинг 23.1. Пример пользовательских методов `Import` и `Export`

```
abstract class ChatRoomApi {
    @post("/{parent=chatRooms/*/}/messages:export")
    ExportMessages(req: ExportMessagesRequest):
        Operation<ExportMessagesResponse, ExportMessagesMetadata>;

    @post("/{parent=chatRooms/*/}/messages:import")
    ImportMessages(req: ImportMessagesRequest):
        MOperation<ImportMessagesResponse, ImportMessagesMetadata>;
}
```

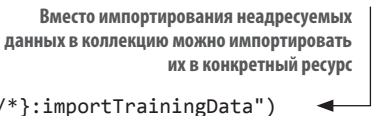


Мы импортируем множество ресурсов в коллекцию, принадлежащую родительскому ресурсу

Кроме того, важно отметить, что два этих пользовательских метода ориентированы на импорт множества ресурсов в коллекцию, но это не единственный доступный вариант. Мы также можем захотеть импортировать данные, которые не представлены набором ресурсов API, а являются просто множеством точек данных, которые после импорта окажутся индивидуально адресуемыми.

Например, если наш API чата будет включать виртуального помощника, обученного на изощренном алгоритме МО, то у нас может появиться возможность импортировать точки обучающих данных в один ресурс `VirtualAssistant`, получив пользовательский метод (листинг 23.2).

Листинг 23.2. Пример импорта неадресуемых точек данных в ресурс



```
abstract class ChatRoomApi {
    @post("/{id=virtualAssistants/*}:importTrainingData")
    ImportTrainingData(req: ImportTrainingDataRequest):
        Operation<ImportTrainingDataResponse, ImportTrainingDataMetadata>;
}
```

Как видите, с виду это довольно обыденные пользовательские методы, возвращающие LRO, которая в конечном итоге разрешится в стандартные ответы. Сложной же частью, как мы узнаем в следующем подразделе, станет структурирование методов запроса.

23.3.2. Взаимодействие с системами хранения данных

Поскольку подключение к внешним системам хранения — значимый элемент нашего проекта для импорта и экспорта данных, вполне логично, что нам потребуется некое место, в котором мы разместим все возможные параметры конфигурации для каждой из этих систем. Зачастую может возникать соблазн просто совместить все возможные параметры, чтобы получить единую схему конфигурации, подходящую для всех хранилищ. В дальнейшем, когда нам потребуется добавить поддержку для очередной системы, можно будет просто вносить в конфигурацию новые поля и на этом считать работу завершённой.

Но помимо громоздкости, такой дизайн будет чреват проблемами ещё и потому, что может вносить массу путаницы. Во-первых, когда у нас есть единый интерфейс с огромным количеством параметров, расположенных рядом друг с другом, становится трудно решить, какие и для каких обстоятельств нужны. Это ведет к излишне затянутому диалогу с сервером, который отвергает различные запросы на импорт или экспорт из-за отсутствующих настроек для источника данных или места их назначения.

Во-вторых, когда мы продолжаем добавлять все больше и больше параметров конфигурации в плоскую структуру, очередным желанием будет повторно использовать существующие поля в интерфейсе так, чтобы они могли применяться ко множеству различных систем хранения. Иначе говоря, вместо того чтобы создавать поля `sambaPassword` наряду с полем `s3secretKey`, мы можем создать просто одно поле `password` и пытаться использовать его как для секретного

ключа S3 в Amazon Web Service, так и для пароля конкретного пользователя в сетевом ресурсе Samba. Поначалу это кажется удачной идеей, но в итоге она вызовет еще большую путаницу, когда одно поле начинает служить нескольким целям только потому, что его имя оказывается созвучным.

Более простым решением будет использовать одно полиморфное поле, содержащее конкретные интерфейсы для каждой поддерживаемой системы хранения. Иными словами, у нас может быть `S3DataSource` для загрузки данных из Amazon Simple Storage Service, а также `SambaDataSource`, способный взаимодействовать с Samba (листинг 23.3).

Листинг 23.3. Пример использования S3 в качестве источника данных

```
interface DataSource {
    type: string;
}

interface S3DataSource extends DataSource {
    type: 's3';
    bucket: string;
    glob: string;
}
```

type — это статическая строка для итоговых интерфейсов, содержащая идентификатор хранилища

Здесь хранится выражение glob (например, archive-*.gzip) соответствующих объектов, предназначенное для использования в качестве источника данных

Конкретные интерфейсы DataSource расширяют основной

Этот же паттерн применим и к сервисам хранилищ, выступающим целевыми точками для извлекаемых из API данных, что ведет к модульному, повторно используемому принципу API, при котором подключения к внешним хранилищам можно повторно применять при импорте или экспорте различных ресурсов. Но здесь возникает интересный вопрос: если эти интерфейсы только определяют, как подключаться к хранилищу, то почему при считывании и записи данных нам нужны отдельные интерфейсы?

Несмотря на тесную связь этих принципов (в конце концов, оба относятся к взаимодействию с внешними хранилищами и передаче данных), они не одинаковы. К примеру, при использовании в качестве источника данных Amazon S3 нам нужно применять паттерн (в данном случае выражение glob (<https://en.wikipedia.org/wiki/Glob>)) для выбора извлекаемых объектов (аналогично файлам на диске). С другой стороны, при экспорте данных у нас скорее будет либо один выходной файл, либо потенциально несколько разных файлов, но все они будут помещены в один каталог (который в S3 выражается с помощью приставки к имени файла, как в `exports/2020/`) (листинг 23.4).

Иными словами, несмотря на крайнюю схожесть этих понятий, они не являются одинаковыми и не представляют одно и то же. Здесь может быть множество пересекающихся полей, но именно вследствие дизайна два этих интерфейса изменяются независимо. И подобный вид слабой связи представляет собой дизайн, который сохраняет гибкость даже в случаях сильной волатильности.

Листинг 23.4. Пример использования S3 в качестве точки для отправки данных

```
interface DataDestination {
    type: string;
}

interface S3DataDestination extends DataDestination {
    type: 's3';
    bucket: string;
    prefix: string;
}
```

В этом случае вместо выражения `glob` именование записываемых объектов будет определяться с помощью префикса

Теперь, когда мы можем настраивать способ, с помощью которого API должен извлекать байты (или отправлять их) из внешних хранилищ, можно переходить к следующему этапу: выполнению преобразования между ресурсами API и сырыми байтами.

23.3.3. Преобразование между ресурсами и байтами

Интересно отметить, что механизм превращения внутренних ресурсов в сериализованные наборы байтов заложен в самой природе построения веб-API. Как правило, этот процесс опирается на форматы наподобие JSON. Тем не менее каждому API нужен способ отправки данных по интернету, а значит, он должен уметь сериализовывать данные для пользователей. И хотя уже заложенный в нем механизм окажется полезным, сериализация для экспорта (или десериализация для импорта) множества ресурсов будет несколько иной. К примеру, мы можем использовать JSON для сериализации одного ресурса, но при экспорте данных опереться либо на измененный формат (скажем, JSON с разбивкой на новые строки, а не массив ресурсов JSON), либо на полностью другой (например, CSV или TSV со строкой заголовка). Здесь мы не станем разбирать все подробности этой темы, но в листинге 23.5 представлен пример сериализации ресурсов `Message` в CSV. (Если вам интересно, почему отсутствует идентификатор, то загляните в подраздел 23.3.5.)

Листинг 23.5. Образец сериализации ресурсов `Message` в формат CSV

```
sender, type, content
"users/1", "text", "hi"
"users/2", "text", "hi back"
```

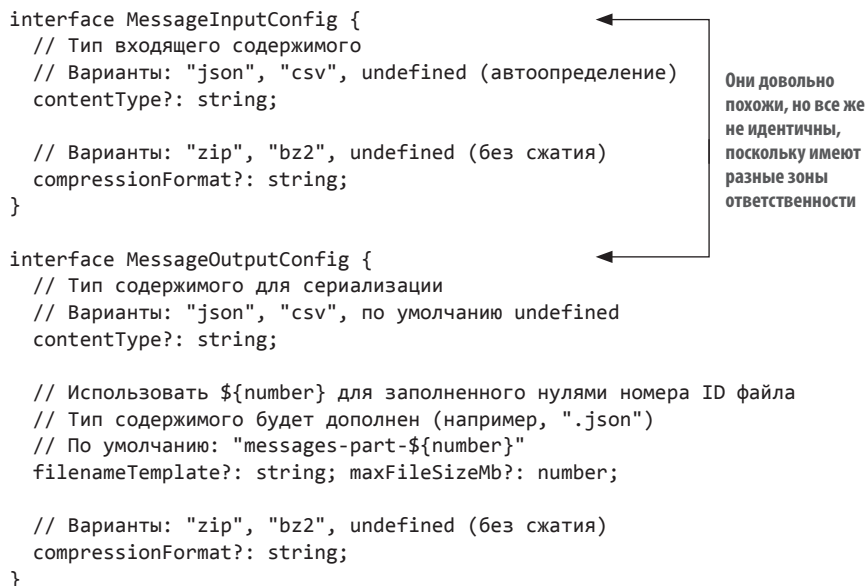
В данных, экспортируемых как CSV, первая строка обычно является заголовком, определяющим столбцы

В каждой строке хранятся поля, подлежащие импорту

Более того, даже после сериализации данных нам может потребоваться выполнить дальнейшие преобразования, прежде чем сохранять их для экспорта (то же касается действия в обратном направлении). Это может быть что-то простое наподобие сжатия или шифрования данных с помощью предоставленного пользователем секретного ключа либо нечто более сложное, например разбивка данных на несколько отдельных файлов для сохранения во внешнем хранилище.

Что бы в итоге мы ни решили делать с данными, нам определенно нужно место, где мы могли бы разместить всю эту конфигурацию, и для этого мы будем использовать интерфейс `inputConfig` (либо `OutputConfig` для экспорта). И аналогично пользовательским методам `Import/Export` эти интерфейсы будут работать для отдельных ресурсов, в результате чего в нашем случае будут выглядеть как `MessageInputConfig` (листинг 23.6).

Листинг 23.6. Пример входящей и исходящей конфигурации для ресурсов `Message`



```

interface MessageInputConfig {
    // Тип входящего содержимого
    // Варианты: "json", "csv", undefined (автоопределение)
    contentType?: string;

    // Варианты: "zip", "bz2", undefined (без сжатия)
    compressionFormat?: string;
}

interface MessageOutputConfig {
    // Тип содержимого для сериализации
    // Варианты: "json", "csv", по умолчанию undefined
    contentType?: string;

    // Использовать ${number} для заполненного нулями номера ID файла
    // Тип содержимого будет дополнен (например, ".json")
    // По умолчанию: "messages-part-${number}"
    filenameTemplate?: string; maxFileSizeMb?: number;

    // Варианты: "zip", "bz2", undefined (без сжатия)
    compressionFormat?: string;
}

```

Они довольно похожи, но все же не идентичны, поскольку имеют разные зоны ответственности

Надеюсь, что все это кажется вам скучным и безобидным. Реальная ценность кроется не в конкретных параметрах для импорта и экспорта данных, а в разделении задач по извлечению сырых данных и их обработке. Такая слабая связанность обязанностей гарантирует, что мы можем повторно использовать источники данных и точки их назначения для импорта и экспорта различных типов ресурсов в API.

Теперь, когда мы разобрались с особенностями структуры, пора переходить к поведенческим нюансам этого паттерна, начав с реализации внутренней согласованности ресурсов в API.

23.3.4. Согласованность

Когда дело касается экспорта данных, нам, несомненно, нужно считывать всю коллекцию ресурсов, которые мы собираемся экспортировать. И как бы нам ни хотелось иметь возможность считывать любое количество ресурсов мгновенно, обычно так бывает, только когда их немного. Если же речь идет о крупных

коллекциях, считывание всех данных однозначно будет занимать какое-то время. Само по себе это еще не является проблемой, но нет никакой гарантии, что такая операция экспорта будет иметь эксклюзивный доступ к системе. И тут возникает важный вопрос: что нужно делать с ресурсами, которые могут в процессе экспорта изменяться?

Здесь есть два простых решения. Первое — использовать снимок данных или транзакционные возможности, предоставляемые внутренней системой хранения. К примеру, можно считать все ресурсы в конкретный момент времени, обеспечив наличие согласованного представления экспортируемой коллекции. Если поступить так, то любые изменения, произошедшие во время экспорта, не будут отражены в отправленных данных.

К сожалению, не все хранилища поддерживают подобную функциональность снимков состояния. Кроме того, системы, способные обрабатывать исключительно большие наборы данных, содержат ее еще реже. Что же тут можно предпринять?

Другое решение, вполне приемлемое для операции экспорта, — признать лучшим вариантом в случае подобных хранилищ предоставление данных, смазанных в процессе передачи. Это означает, что экспортируемые ресурсы могут оказаться не точными представлениями данных в определенный момент времени, а максимально приближенным отражением их содержимого за период выполнения операции. Конечно, это не идеальный способ, но во многих случаях он окажется единственным доступным, а значит, и лучшим.

Если эта идея смазанности данных не до конца вам ясна, то мы разберем пример. В табл. 23.1 первые два столбца показывают действия операции экспорта и пользователя, одновременно манипулирующего данными в системе. Два последних столбца отражают текущее состояние данных (какие ресурсы существуют) и экспортированные данные.

Таблица 23.1. Сценарий экспорта со смазыванием данных

Действия пользователя	Действие экспорта	Сохраненные данные	Экспортированные данные
Create A, B, C		A, B, C	
	Export A	A, B, C	A
Delete A		B, C	A
	Export B	B, C	A, B
Create D		B, C, D	A, B
Delete C		B, D	A, B
	Export D	B, D	A, B, D

Ввиду уникальной поочередности действий обеих сторон (пользователя и операции экспорта) мы в итоге получаем набор ресурсов, который выглядит несколько странно. Ресурсы этой конкретной группы (A, B и D) никогда не существовали вместе (в столбце «Сохраненные данные» видно, что одновременно все эти ресурсы никогда не присутствовали). Подобная смазанность определенно необычна, но оказывается необходимым злом, когда у нас нет возможности работать с согласованным снимком данных.

Если хранилище API не поддерживает снимки для считывания данных, а подобный несогласованный экспорт совершенно неприемлем, то можно рассмотреть вариант отключения изменения ресурсов API на время экспорта. И хотя это определенно вызовет неудобство (и будет подразумевать общесистемный простой на этот период), иногда такой способ оказывается единственным безопасным из доступных (в действительности Google Cloud Datastore рекомендует потребителям экспортировать согласованные представления данных, хранящихся в их системе).

Если делать из текущего подраздела единственный ключевой вывод, то это должна быть идея о том, что экспорт данных *не равнозначен* их резервному копированию. Если резервные копии подразумевают снимок данных, включая всевозможную уникальную идентифицирующую информацию, то экспорт относится к извлечению данных традиционными средствами, перенаправляющими эти данные непосредственно во внешнее хранилище. В следующем подразделе мы разберем, что конкретно нужно делать с уникальной идентифицирующей информацией в экспортируемых ресурсах, поскольку этот нюанс несколько сложнее, чем может показаться.

23.3.5. Идентификаторы и коллизии

До этого момента мы работали на основе предположения, что сериализованная форма ресурса должна включать все его поля, в том числе идентификатор. И хоть эта деталь может показаться незначительной, на деле оказывается, что вокруг одного этого поля перед нами возникает целый ряд интересных вопросов. Например, что нужно делать при импорте ресурсов в коллекцию, если в источнике данных встретится ресурс, ID которого уже используется в API?

Еще один интересный случай возникает, когда мы экспортируем ресурсы с конкретными идентификаторами, а затем пытаемся импортировать их же обратно в API, но уже в другой родительский ресурс. То есть что, если мы экспортируем ресурс `Message` с ID `chatRooms/1234/messages/2` и затем решим импортировать его в `ChatRoom` #5678? Нужно ли создать его как `chatRooms/5678/messages/2` или же необходимо присвоить ему полностью новое имя?

Наконец, нам нужно прояснить случай, в котором API не разрешает применять в ресурсах пользовательские идентификаторы. Как поступать с ID ресурсов, которые мы хотим импортировать? Игнорировать их? Как мы увидим в подразделе 23.3.7, это становится довольно важным, когда мы начинаем изучать, что происходит при попытке повторить импорт, который завершился неудачей.

К счастью, чтобы получить ответ на этот вопрос, можно вернуться к оригинальному определению импорта и экспорта. Задача пользовательских методов `Import` и `Export` — предоставить при переносе данных способ восполнения разрыва между API и внешними системами хранилищ, исключая опосредующий эту связь прикладной код. Это означает, что данная функциональность должна быть эквивалентна получению данных из внешнего хранилища и использованию пакетного метода `Create` в целях создания кучи новых ресурсов на основе предоставленных данных. Все, что находится за пределами этой простой (и наивной) функциональности, оказывается, откровенно говоря, вне области.

Результат этого определения довольно эффективен: если пользовательские ID не поддерживаются (как обычно и должно быть, о чем мы узнали в главе 6), то пользовательский метод `Import` при создании ресурсов в API должен игнорировать любые передаваемые идентификаторы (например, поле `id`). Принимая во внимание все вышесказанное, будет разумным сохранять ID при экспорте в сценариях, когда нам в дальнейшем потребуется определить источник данных.

Но не значит ли это, что у нас будет возможность создавать множество повторяющихся ресурсов в случае повторения импорта одних и тех же данных? Именно так, но это заложено самим дизайном.

Методы импорта и экспорта совершенно не предназначены для использования в качестве механизма для резервного копирования и восстановления данных в API. Эта особая функциональность требует намного более строгих гарантий относительно данных, таких как согласованность снимков состояния при их резервном копировании (как было показано в подразделе 23.3.4), а также полной замены коллекции при импорте данных. И эти пользовательские методы не дают подобных гарантий.

23.3.6. Обработка связанных ресурсов

До сих пор пользовательские методы `Import` и `Export` были сосредоточены исключительно на одном ресурсе одновременно, но это чрезмерное упрощение. В конце концов, если вспомнить дискуссию из главы 4 на тему схемы размещения ресурсов и их иерархии, то вполне можно ожидать ситуации, в которой нам потребуется импортировать или экспортировать ресурсы, имеющие дочерние

или связанные с ними другие ресурсы. Возьмем очень простой пример. Что, если нам нужно экспортировать коллекцию ресурсов `ChatRoom`? Должна ли она также включать все дочерние ресурсы `Message`? Очевидно, что сама информация чата важна, но при экспорте или импорте его комнат нас вряд ли интересует пустая `ChatRoom`.

Мы можем попытаться изменить этот паттерн, чтобы сделать возможным выполнение операций над множеством различных типов ресурсов в одном вызове API, но это никак не согласуется с его сутью. Как мы узнали в разделе 23.1, цель данного паттерна — соединить сервис API и внешнее хранилище, избегая посредника, передающего между ними данные. Он никак не предназначен для введения новой продвинутой функциональности, связанной с извлечением данных или их загрузкой в сам API. В связи с этим пользовательские методы `Import` и `Export` нужно точно так же ограничить одним типом ресурсов, как мы делали это с другими стандартными методами API (например, стандартный метод `List` или пакетный `Create`).

Это означает, что импорт и экспорт данных практически всегда будут ограничиваться до ресурсов, не имеющих потомков, и будут полезны без дополнительных связанных ресурсов. Например, несмотря на предполагаемую бессмысленность импорта или экспорта ресурсов `ChatRoom`, будет довольно актуально делать это с ресурсами `Message`, содержащимися в заданной `ChatRoom`.

Иначе говоря, при импорте/экспорте ресурсов `Message` нас интересует перемещение полезных, полунезависимых данных в/из API. Когда мы проделываем то же самое с ресурсом `ChatRoom`, наш фокус смещается с простой перестановки и перемещения данных в область резервного копирования и восстановления, где нам нужно сохранить определенное состояние данных и потенциально восстановить систему к этому состоянию позднее. И хотя эта идея сохранения/восстановления определенно важна и имеет много общего с идеей импорта/экспорта, они совершенно не являются одинаковыми.

Пора сменить тему и рассмотреть неожиданные случаи, которые неизбежно возникают в любом API.

23.3.7. Сбои и повторы

В любой системе время от времени возникают проблемы. Поэтому когда в процессе участвуют две системы, передающие потенциально огромные объемы данных, с большой вероятностью сбои возникнут и в них. Кроме того, поскольку поведение, подразумеваемое пользовательскими методами `Import` и `Export`, связано с записью данных в то или иное место, у нас возникает более сложный сценарий: если сбой случится в процессе чтения или записи данных (ресурсов

API или байтов во внешнем хранилище), то после него будет гораздо сложнее восстановиться.

Сбои в любом из этих методов могут возникать в двух формах, связанных с тем, в каком направлении передаются данные. При импорте сбой может возникнуть, когда хранилище не может предоставить сырые байты сервису API либо если этот сервис не может создать новые ресурсы. При экспорте данных происходят аналогичные сбои, но уже в обратном направлении, возникая вследствие невозможности считать ресурсы API или передать и записать сырые байты во внешнее хранилище. Как же решить подобные проблемы? А самое главное — как узнать, безопасно ли повторить операцию в случае ее сбоя? Начнем с самого простого.

Сбои экспорта

Любые сбои доставляют неудобство, но применительно к сбою экспорта хорошая новость в том, что его можно безопасно повторить, поскольку очередная попытка экспорта данных из API будет полностью независима от предыдущей (потенциально неудачной).

Но это не означает, что подобный вид сбоя не имеет осложнений. Здесь предстоит разобраться с несколькими возможными проблемами.

Первая самая очевидная: данные одной попытки экспорта не обязательно будут идентичны последующей попытке. В частности, в волатильных наборах данных при наличии множества одновременных пользователей системы очень маловероятно, что две последовательные попытки дадут одинаковые результаты. К счастью, цель метода `Export` состоит в передаче данных из API, и это уже никак не гарантирует их согласованность. С другой стороны, как мы узнали в подразделе 23.3.4, эти данные, скорее всего, окажутся смазанными, а не единым согласованным снимком, поэтому повтор экспорта не должен привести к каким-то значительным проблемам с итоговыми данными.

Далее нам нужно выяснить, что делать с данными из предыдущей, неудачной, попытки экспорта. Нужно ли постараться удалить их или же оставить для ручной очистки, которая будет выполняться позднее? Каким бы затратным это ни казалось, лучше всего обычно оставить данные, независимо от того, в какой момент процесса произошел сбой.

И причина тому двоякая. Во-первых, нет гарантии, что у API вообще будет разрешение на удаление данных из внешней системы хранения (и в данном случае рекомендуется отказаться от этого разрешения). Во-вторых, нам неизвестно, являются ли экспортируемые данные реально ценными или же малозначительными для пользователя, который запросил их экспорт.

Это может показаться нелогичным, но суть в том, что мы находимся не в том положении, чтобы решать, сколько экспортируемых данных будет достаточно

для целей пользователя. Очевидно, что 100 % окажется полезным количеством, а 0 % — нет, но любой промежуточный результат нельзя оценить однозначно. Представьте, к примеру, что мы экспортируем очень большой набор данных объемом 10 Тбайт. Сколько данных нужно передать, чтобы можно было заявить о пересечении черты, отделяющей бесполезность от ценности? Если операция экспорта завершится неудачно на самом последнем ресурсе, то стоит ли удалять все полностью? Возможно, в общей картине этот один ресурс не был настолько важен. Суть в том, что удаление экспортируемых данных находится вне области самого их экспорта, и такое решение нужно оставлять владельцу хранилища.

Исходя из этого, возникает новая проблема: что, если система хранения исчерпает свободное место? К сожалению, если учесть предыдущие рассуждения о том, что принятие решения нужно оставлять пользователю, то эта проблема выходит за рамки нашей ответственности. Хуже того, это может привести к каскадированию сбоев, когда повторяющаяся операция экспорта будет завершаться неудачно ввиду недостатка места в хранилище. Тем не менее, учитывая установленные нами к текущему моменту ограничения и принципы, это тот случай, с которым просто придется смириться, поскольку альтернатива окажется куда более печальной для пользователей API.

Разобравшись со сбоями во время экспорта данных, можно переходить к проблемам, возникающим при их импорте.

Сбои импорта

В отличие от сбоев экспорта, при сбоях во время импорта нельзя просто так повторить процесс в надежде, что все произойдет должным образом. Есть вероятность, что сбой возник ввиду ошибки валидации, а не был вызван временными неполадками сети или самого процесса передачи. Если так и есть, то никакое количество повторов не решит проблему и нам нужно сообщить в результате LRO о записях, которые не прошли проверку.

Даже если сбой произойдет по причине временного явления, нам необходимо осознавать тот факт, что в ходе неудачной попытки все равно были созданы какие-то ресурсы. И это большая проблема: если только импорт не завершился неудачей сразу же, дальнейшие попытки повторить его практически однозначно приведут к созданию повторяющихся ресурсов. Что тут можно сделать?

Одно из очевидных решений — выполнить операцию импорта внутри транзакции, откатывая ее назад при любом признаке сбоя. И хотя это окажется быстрым и простым решением проблемы, к сожалению, не каждая система хранения будет поддерживать полноценную транзакционную семантику. И даже те, которые будут, могут иметь сложности в случае очень больших объемов данных, ведущих к созданию в транзакции невероятного количества ресурсов.

Там, где мы не можем просто положиться на ценный элемент функциональности базы данных, нам явно нужно иное решение. Для этого можно обратиться к странице из главы 26 и защитить наши API от риска дублирующихся запросов API.

Тему дублирования запросов нужно изучать и разбирать, но общая суть в том, что для каждой записи, которую мы будем импортировать, можно присвоить один глобально уникальный ID, называемый идентификатором запроса импорта. Этот `importRequestId` будет присвоен каждому отдельному ресурсу, создаваемому в процессе импорта, и сервис API сможет кэшировать эти идентификаторы при их первичном создании, исключая любые последующие попытки повторно импортировать одну и ту же запись (листинг 23.7).

Листинг 23.7. Образец ресурсов JSON с ID запросов импорта в целях исключения повторений

```
{ importRequestId: "abc", sender: "users/1", content: "hi" }
{ importRequestId: "def", sender: "users/2", content: "hi back" }
```

ID запроса импорта, с помощью которого
API исключает двусмысленность и избегает
создания повторяющихся ресурсов

Чтобы сделать это более понятным для тех, кто пожелает повторно импортировать данные после их экспорта, можно даже рассмотреть добавление в интерфейс `MessageOutputConfig` параметра конфигурации, который будет отвечать за внедрение этих значений `importRequestId` в сериализованные данные во время экспорта. Независимо от того, сделаем мы это или нет, суть в том, что, если мы хотим добиться отсутствия импорта дубликатов ресурсов во время повтора неудачных операций импорта, у нас должен быть способ, позволяющий отличить уже пройденную нами запись ресурса от новой. А поскольку ID ресурсов уже игнорируется по весомым причинам, о которых мы говорили в подразделе 23.3.5, мы вынуждены добавлять нечто новое именно ради этой цели. То есть если это особое поле во входных данных будет отсутствовать, повтор неудачного запроса импорта в системе, не имеющей поддержки транзакций во внутреннее хранилище, в конечном итоге приведет к созданию этой повторной операцией дубликатов ресурсов.

23.3.8. Фильтрация и маски полей

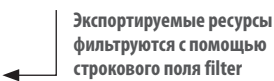
Как мы видели в главе 22, возможность отфильтровывать коллекцию ресурсов — ценный элемент функциональности, позволяющий более эффективно взаимодействовать с данными, хранящимися в API. Тогда речь шла о стандартном методе `List`, но применима ли подобная функциональность еще и к пользовательским методам `Import/Export`? Ответом в данном случае будет как «да», так и «нет».

Фильтрация вывода

Напомню, когда дело доходит до экспорта данных, единственной нашей целью является предоставление способа для связи хранилища напрямую с API. То есть действия метода `Export` ничем не должны отличаться от действий метода `List`. Следовательно, можно заключить, что пользовательский `Export` определенно должен поддерживать фильтрацию элементов до их передачи. При этом вполне ожидаемо, что определение API здесь усложнится лишь добавлением поля `filter` в интерфейс запроса экспорта (листинг 23.8).

Листинг 23.8. Пример добавления фильтра в запрос экспорта

```
interface ExportMessagesRequest {
    parent: string;
    outputConfig: MessageOutputConfig;
    dataDestination: DataDestination;
    filter: string;
}
```



Экспортируемые ресурсы
фильтруются с помощью
строкового поля `filter`

Вы можете поинтересоваться, почему фильтры размещаются рядом с интерфейсом `OutputConfig`, а не внутри него. Дело в том, что конфигурация вывода сосредоточена исключительно на способе, с помощью которого мы берем набор ресурсов и превращаем их в набор байтов. Фильтрация же происходит до этой стадии и сосредоточена на выборе ресурсов, которые необходимо сериализовать, сжать и т. д. И несмотря на возможный соблазн поместить эту конфигурацию в `OutputConfig`, в данном случае очень важно разделить обязанности, чтобы каждый элемент занимался своей единственной задачей.

Фильтрация входных данных

В отличие от экспорта, где мы применяем фильтры к выбираемым API данным, их фильтрация на входе более специфична. Этот процесс подразумевает передачу байтов из внешней системы хранения в сервис API, после чего API применяет к сериализованным байтам действия, прописанные в `InputConfig` (например, декомпрессию, расшифровку и десериализацию), и лишь после этого применяется фильтр, определяющий, нужно отбросить ресурс или же создать его в API.

Кроме того, есть вероятность, что в импортируемых ресурсах будут некие производимые данные, которые окажутся доступны для фильтрации только после загрузки в API. Например, как можно отфильтровать входящие данные на основе поля только для вывода, скажем, `createTime` или любого другого свойства, вычисляемого сервисом? Чтобы проделать это эффективно, потребуется прогнать такие данные через бизнес-логику самого API, что приведет к усложнению и потенциально напрасным усилиям.

Очевидно, что хоть у нас и есть возможность реализовать поддержку фильтрации входящих данных во время импорта, это одна из тех вещей, которые обычно

не делаются в API. Даже если API не столкнется с описанными здесь проблемами, нет гарантии, что так будет всегда. И если мы когда-либо внесем изменения, ведущие к такому сценарию, то нам придется искать способ, позволяющий продолжать поддерживать фильтрацию импорта в ряде потенциально каверзных ситуаций. Вместо этого любые преобразования данных (включая фильтрацию) должен производить пользователь еще до того, как запускать операцию импорта.

23.3.9. Итоговое определение API

В листинге 23.9 показан пример импорта и экспорта ресурсов `Message` в нашем API чата. Здесь присутствует два образца ресурсов и мест назначения, относящихся к хранилищу Samba (<https://ru.wikipedia.org/wiki/Samba>) и Amazon S3 (<https://aws.amazon.com/s3/>).

Листинг 23.9. Итоговое определение API

```
abstract class ChatRoomApi {
    @post("/{parent=chatRooms/*/}/messages:export")
    ExportMessages(req: ExportMessagesRequest):
        Operation<ExportMessagesResponse, ExportMessagesMetadata>;

    @post("/{parent=chatRooms/*/}/messages:import")
    ImportMessages(req: ImportMessagesRequest):
        Operation<ImportMessagesResponse, ImportMessagesMetadata>;
}

interface ExportMessagesRequest {
    parent: string;
    outputConfig: MessageOutputConfig;
    dataDestination: DataDestination;
    filter: string;
}

interface ExportMessagesResponse {
    chatRoom: string;
    messagesExported: number;
    // ...
}

interface ExportMessagesMetadata {
    chatRoom: string;
    messagesExported: number;
    // ...
}

interface MessageOutputConfig {
    // Тип содержимого для сериализации
    // Варианты: "json", "csv", по умолчанию undefined
    contentType?: string;
}
```

400 Часть V. Коллективные операции

```
// Использовать ${number} для номера ID файла, заполненного нулями
// Тип содержимого будет дополнен расширением файла (например, ".json")
// По умолчанию: "messages-part-${number}"
filenameTemplate?: string;

// Undefined для максимального размера файла
maxFileSizeMb?: number;

// Варианты: "zip", "bz2", undefined (без сжатия)
compressionFormat?: string;
}

interface DataDestination {
    // Уникальный идентификатор типа места назначения (например, "s3" или "samba")
    type: string;
}

interface SambaDestination extends DataDestination {
    // Расположение ресурса Samba (например, "smb://1.1.1.1:1234/path")
    type: 'samba';
    uri: string;
}

interface S3Destination extends DataDestination {
    type: 's3';
    bucketId: string; objectPrefix?: string;
}

interface ImportMessagesRequest {
    parent: string;
    inputConfig: MessageInputConfig;
    dataSource: DataSource;
}

interface ImportMessagesResponse {
    chatRoom: string;
    messagesImported: number;
}

interface ImportMessagesMetadata {
    chatRoom: string;
    messagesImported: number;
}

interface MessageInputConfig {
    // Тип содержимого входных данных
    // Варианты: "json", "csv", undefined (автоопределение)
    contentType?: string;

    // Варианты: "zip", "bz2", undefined (без сжатия)
    compressionFormat?: string;
}
```



```
interface DataSource {
    type: string;
}

interface SambaSource extends DataSource {
    type: 'samba';
    uri: string;
}

interface S3Source extends DataSource {
    type: 's3';
    bucketId: string;
    // Одна или несколько масок в формате glob
    // (например, "folder/messages/*.csv")
    mask: string | string[];
}
```

23.4. КОМПРОМИССЫ

Несмотря на то что на первый взгляд этот паттерн вроде бы охватывает все связанное с перемещением данных в API и из него, больше всего запутывает то, что он имеет очень специфичный и узконаправленный дизайн, представляя отдельный способ перемещения данных и стремясь достичь простой цели: заполнить разрыв между сервисом API и внешним хранилищем. В результате этой специфичности и узконаправленности мы получаем два главных недостатка.

Начнем с того, что этот паттерн не предназначен для обработки нескольких типов ресурсов. То есть он не поможет вам с экспортом больших объемов связанных ресурсов, родительских и дочерних ресурсов или с чем-то, что приводит к формированию связей между множеством ресурсов. Этот паттерн предназначен просто для того, чтобы взять множество самостоятельных точек данных и переместить их между сервисом API и внешним хранилищем, не привлекая прикладной сервер в качестве посредника. Если же вам действительно нужна возможность соединения множества ресурсов и управления ими, то этот посредник уже станет не просто посредником, а важной частью бизнес-логики. Пользовательские методы `Import` и `Export` не предназначены для того, чтобы содержать или реализовывать какую-то особую бизнес-логику; их задача — просто перемещать данные.

И здесь вырисовывается второй весомый недостаток: можно легко спутать импорт и экспорт с функциональностью резервного копирования и восстановления (в конце концов, в обоих случаях задействуется перемещение данных между API и хранилищем). Как мы видели в подразделе 23.3.4, эти пользовательские методы совсем не предназначены для реализации задач атомарного сохранения данных или полного их восстановления из ранее сделанного снимка. Вместо этого они работают без гарантий согласованности и в случае недостатка понимания могут привести к сбивающим с толку результатам.

23.5. УПРАЖНЕНИЯ

1. Почему важно использовать два отдельных интерфейса для настройки методов импорта или экспорта? Почему не стоит совмещать их в одном интерфейсе?
2. Какие варианты позволяют избежать импорта (экспорта) смазанных данных? Какой из них лучше?
3. Нужно ли сохранять идентификатор вместе с данными при экспорте ресурса? А в случае импорта тех же данных? Должны ли новые ресурсы иметь те же ID?
4. Если операция экспорта завершится сбоем, должны ли удаляться данные, которые были переданы на тот момент? Почему?
5. Если сервису требуется поддержка импорта и экспорта, включая дочерние и прочие связанные ресурсы, то как можно ее реализовать? Как должны обрабатываться идентификаторы в этом случае?

РЕЗЮМЕ

- Пользовательские операции импорта и экспорта позволяют перемещать данные напрямую между сервисом API и внешней системой хранения.
- Данные методы не предназначены для резервного копирования и восстановления. Использование их для этой задачи приведет к ряду неприятных последствий.
- Определения API строятся на основе двух разделенных интерфейсов конфигурации: один служит для перемещения байтов во внешнее хранилище и из него, а второй — для преобразования этих байтов в представления ресурсов API и обратно.
- Если система не поддерживает загрузку данных в конкретный момент времени, то велика вероятность, что пользовательские методы `Import` и `Export` приведут к сохранению в системе смазанного представления данных.
- Импорт и экспорт данных обычно следует ограничивать одним типом ресурсов за раз, используя функции резервного копирования и восстановления для обработки дочерних и прочих связанных ресурсов.

Часть VI

Безопасность

Легко забыть, что пользователи при использовании API порой будут совершать ошибки. А поскольку эти ошибки, к сожалению, неизбежны, важно спроектировать такой API, который позволит пользователям минимизировать причиненный ими ущерб. В следующих нескольких главах мы разберем паттерны, нацеленные именно на это.

В главе 24 мы изучим высокоуровневые принципы версионирования и совместимости стратегий, доступных для версионирования API. В главе 25 познакомимся с концепцией своеобразной корзины для API, предотвращающей случайное удаление. Главы с 26-й по 28-ю будут посвящены различным стратегиям, которые позволяют избежать повторения работы, тестировать запросы до выполнения и сохранять историю изменений в ресурсах на случай, если пользователи захотят откатить их.

В главе 29 мы узнаем, когда и как безопасно повторять запросы в случае сбоев сети и прочих проблем. Наконец, глава 30 будет посвящена безопасной аутентификации запросов в API.

24

Версионирование и совместимость

В этой главе

- ✓ Что такое версионирование.
- ✓ Что значит совместимость.
- ✓ Как определять обратную совместимость.
- ✓ Компромиссы при выборе стратегии версионирования.
- ✓ Обзор нескольких популярных стратегий версионирования для веб-API.

Создаваемое нами ПО редко оказывается статичным. Наоборот, оно склонно к постепенному развитию и изменениям по мере появления новых возможностей и расширения функциональности. Это, в свою очередь, вызывает проблему: как вносить улучшения, не доставляя существенных неудобств пользователям, которые зависят от прежнего, привычного им представления сервиса? В этой главе мы изучим самый распространенный механизм, позволяющий решить эту задачу: версионирование.

24.1. ЗАЧЕМ ЭТО НУЖНО

Разработка ПО редко бывает не только статичной, но и непрерывной. Даже при постоянной интеграции и развертывании у нас зачастую есть *контрольные точки*, или *запуски*, когда некая новая функциональность обретает жизнь

и становится доступной для пользователей, что является особенно актуальным для веб-API. Эта необходимость дополнительно усиливается тем фактом, что API являются одновременно и жесткими, и общедоступными, в связи с чем безопасное внесение в них изменений оказывается затруднительным. И тут возникает очевидный вопрос: как вносить изменения в веб-API, не причиняя неудобств его пользователям?

Вариант наподобие «пользователи как-нибудь справятся» здесь однозначно не подходит. Да и альтернативный подход «просто никогда не менять веб-API» на практике невозможен, даже когда мы не планируем добавлять новую функциональность. Так, если возникает проблема с безопасностью или юридический нюанс (например, юрист уведомляет вас, что некий вызов API нарушает закон), то внесения изменений не избежать. В качестве решения подобных ситуаций текущий паттерн познакомит вас с концепцией внедрения в API принципа версионирования наряду с различными стратегиями, подходящими под широкий спектр возможных требований к веб-API.

24.2. ОБЗОР

Чтобы исключить влияние последующих изменений на пользователей API, можно ввести контрольные точки, или версии, которые будут соответствовать различным развертываниям. Это означает, что новые изменения, которые обычно могли бы вызвать проблемы у текущих пользователей API, просто развертываются в виде новой версии, а не в качестве изменений текущей. Такой подход, по сути, скрывает эти изменения от той категории пользователей, которая к ним не готова.

К сожалению, сам этот процесс подразумевает намного больше, нежели просто разметку разных развертываний веб-API как отдельных версий. Нам потребуется уделить внимание различным уровням, на которых может быть реализовано версионирование (например, в клиентских библиотеках или проводном протоколе), а также подумать о выборе политики версионирования. И пожалуй, самым сложным аспектом является то, что единственного правильного ответа здесь нет. Выбор реализации версионирования в API будет варьироваться в зависимости от ожиданий и предпочтений его создателей и пользователей.

Хотя один нюанс останется неизменным — основная цель версионирования. Она состоит в том, чтобы предоставить пользователям API максимум возможной функциональности при минимуме неудобств. Ориентируясь на эту цель, мы начнем с изучения возможных способов, которые позволяют минимизировать неудобства с помощью оценки изменений на предмет их совместимости.

24.2.1. Что такое совместимость

Совместимость — это характеристика того, могут ли два компонента успешно взаимодействовать друг с другом. В случае с веб-API это обычно относится к связи между клиентом и сервером.

С виду этот принцип может казаться элементарным, но его сложность возрастает, когда мы начинаем рассматривать временные аспекты совместимости. При запуске веб-API любой клиентский код, очевидно, будет с ним совместим, но, к сожалению, ни API, ни клиенты не склонны с течением времени сохранять неизменность. По мере постепенных изменений мы начинаем видеть все больше комбинаций клиентов и серверов, и нам приходится учитывать возможность их успешного взаимодействия, что оказывается сложнее, чем может показаться. Например, если у вас есть три выпуска клиента API и доступны три версии сервера, то в общей сложности нужно учитывать девять путей связи, не все из которых будут работать.

Поскольку разработчики API не имеют возможности контролировать клиентский код, написанный пользователями API, нам нужно сфокусироваться на том, являются ли изменения, вносимые в серверную часть, совместимыми с кодом клиента. Иными словами, мы назовем две версии сервера API совместимыми друг с другом, если при перемене их мест любой существующий клиентский код не заметит разницы (или, по крайней мере, продолжит работать).

Представьте, что пользователь веб-API написал код, взаимодействующий с API (непосредственно либо через клиентскую библиотеку). А теперь представьте, что у нас есть новая версия этого сервиса API (в данном случае v1.1), показанная на рис. 24.1. Мы назовем эти две версии совместимыми, если клиентский код продолжит работать после того, как трафик будет перенаправлен с v1 на v1.1.

Почему же нас волнует совместимость? В начале раздела мы узнали, что версионирование — средство для достижения главной цели: предоставления пользователям максимума функциональности при минимуме неудобств (в идеале вообще без них). Один из способов добиться этого — развертывать новые версии наряду с существующими так, чтобы пользователи могли использовать новую функциональность, не вызывая сбоев в ранее написанном коде у других. Это работает, поскольку новую версию мы создаем как полностью автономную, но действительно ли это лучшее, что можно сделать? Возможно, есть еще более удачный способ, позволяющий предоставить текущим пользователям дополнительную функциональность без причинения неудобств?

Оказывается, можно поступить эффективнее, внедрив новую функциональность в существующую версию так, чтобы сам API оказался достаточно близок к своему прежнему облику. Измененная версия может оказаться даже настолько похожей, что клиент не заметит разницу между API до и после добавления

функциональности. Это значит, если вносить в API лишь совместимые изменения, то весь существующий код продолжит работать, не подозревая о них.

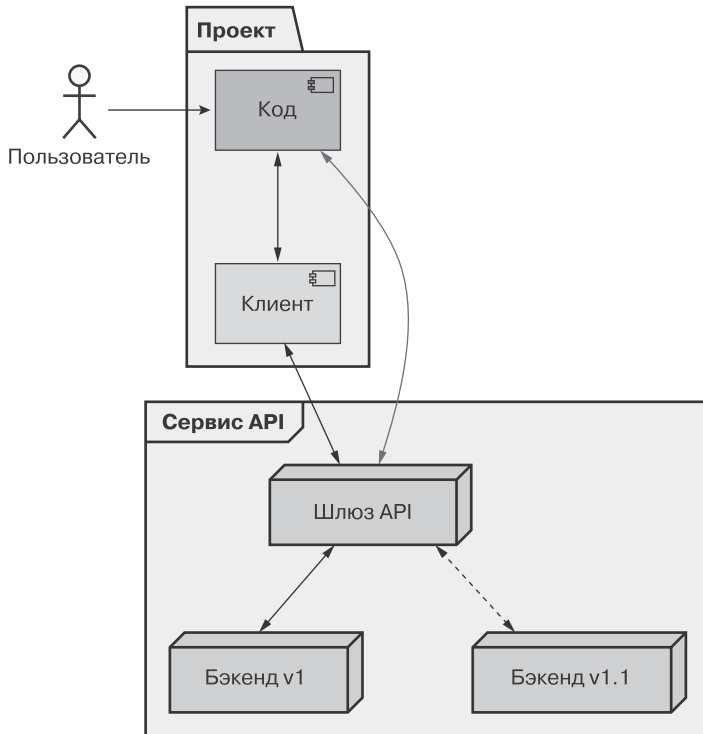


Рис. 24.1. Эксперимент для определения совместимости двух версий

Но тогда возникает довольно сложная и запутанная проблема: как нам решать, является ли некое изменение обратно совместимым? Иначе говоря, как нам решать, окажется ли оно заметным и не приведет ли к сбою существующего клиентского кода?

24.2.2. Определение обратной совместимости

Как мы только что видели, возможность вносить в API только обратно совместимые изменения подразумевает встречную возможность предоставлять пользователям новую функциональность, не требуя выполнения дополнительной работы. Поэтому теперь остается лишь решить, что именно включает в себя обратно совместимое изменение. Как правило, это простое решение. Достаточно задать себе простой вопрос: повредит ли это изменение существующий код? Если да, значит, оно не является обратно совместимым.

К сожалению, в действительности все не так просто, и это один из тех сценариев, в котором нет простого ответа. Вместо этого перед нами возникает множество вариантов, которые все в том или ином смысле верны. Более того, степень верности зависит от пользователя API и его ожиданий. Иными словами, в отличие от многих других паттернов, показанных в книге, это тот случай, когда нет единственного четкого способа применения.

Определяя политику разделения новой функциональности на обратно совместимую и несовместимую, разработчики API должны учитывать несколько аспектов. И хотя этот подраздел не даст гарантированно правильных рекомендаций, он поставит перед вами несколько вопросов, на которые нужно будет ответить самостоятельно. Ни одно решение не окажется исключительно верным или ошибочным; тем не менее разработчики должны выбирать лучший вариант на основе ожиданий пользователей. Например, как мы вскоре увидим, хранилище больших данных при использовании API будет иметь совсем другие ожидания, нежели парк мелких IoT-устройств.

Так что приступим и начнем с рассмотрения наиболее существенных вопросов.

Добавление функциональности

Сначала нужно будет понять, рассматриваете ли вы вообще как способ предоставления новой функциональности возможность расширения существующих версий. То есть следует выяснить, не являются ли пользователи вашего API настолько привязанными к требованиям стабильности, что предпочтут неизменность каждой отдельной версии. Если основными пользователями выступают банки, то это станет вполне разумным предположением. В таком случае их будет интересовать возможность добровольного внедрения новых версий, чтобы им не приходилось менять свой код для использования любых новых возможностей. И это абсолютно нормально. С другой стороны, если ваши пользователи API — группа небольших технологических стартапов, то они могут быть больше заинтересованы в получении новой функциональности, нежели в стабильности отдельной версии API.

Если вы решите, что внедрение новой функциональности должно допускаться в существующей версии, то вам предстоит более тщательно продумать, как она будет представлена пользователям. К примеру, она может выражаться в виде новых полей в существующих ресурсах, полностью новых ресурсов или новых допустимых действий. При этом каждый из перечисленных вариантов повлияет на пользователей по-разному.

Если взять новые поля в существующих ресурсах, то они значительно повлияют на всех пользователей, имеющих строгие требования к ресурсам. Представьте в качестве примера API, используемый IoT-устройством с очень ограниченным объемом памяти. Если в существующий ресурс добавить новое поле, то

работавший до этого HTTP-запрос GET может привести к тому, что у устройства просто не хватит памяти на его обработку, и процесс завершится сбоем. Это может показаться натянутым примером, когда речь идет об извлечении одного ресурса, но представьте добавление поля, которое может содержать большой объем информации, и IoT-устройство, которое выводит список всех ресурсов. При этом риск, что количество данных в этом ресурсе выйдет за ожидаемые пределы, дополнительно увеличивается пропорционально количеству ресурсов, отображаемых в ответе.

В случае же добавления новых ресурсов или новых методов API маловероятно (если вообще возможно), что существующий код узнает об этих новых аспектах API. Тем не менее это не означает, что здесь не будет никаких проблем. Представьте, что клиентский код создает резервную копию всех ресурсов API. Если при создании нового ресурса код не будет о нем знать, то и в резервную копию такой ресурс не попадет. И хотя зачастую лучше стараться не брать на себя ответственность за подобные проблемы, существуют сценарии, когда будет уместным запретить добавление новых ресурсов для таких случаев.

Ситуация несколько усложняется, когда добавленный ресурс вводит некую зависимость либо связь с существующими ресурсами или полями. Представьте, что API вводит новый подресурс `MessagePolicy`, принадлежащий ресурсу `ChatRoom` (и по умолчанию включающий примитивную политику). Если мы сможем удалить ресурс `ChatRoom` только после удаления `MessagePolicy`, то таким образом вынудим пользователей узнать об этом изменении вместо того, чтобы оставить их в спокойном неведении. Такой вариант может оказаться довольно неудачным.

Как мы уже говорили, все это не жесткие правила, которым необходимо следовать. Наоборот, это лишь потенциальные сценарии, которые любой разработчик API должен учитывать при запуске своего программного интерфейса. В конечном итоге выбор относительно безопасности добавления новой функциональности в существующий API является больше творческим, нежели научным. Поэтому как минимум здесь стоит ориентироваться на согласованные и ясно выраженные политики.

Даже если вы решите, что добавление функциональности в любой форме не обязательно должно быть в безопасной степени обратно совместимым, есть еще более сложный вопрос: исправление ошибок.

Исправление ошибок

При написании ПО мы редко получаем идеальный вариант с первого раза. Чаще мы все же допускаем ошибки, на которые нам впоследствии указывает кто-то другой и которые мы в итоге исправляем. Но, как вы можете видеть сейчас, не является ли такое изменение просто другой формой улучшения функциональности? Если да, то нужно ли исправлять ошибки в отдельной версии?

Или же можно безопасно внедрить это изменение в существующую и назвать его обратно совместимым? Несложно догадаться, что ответ на эти вопросы будет зависеть от обстоятельств.

Некоторые ошибки будут проявляться довольно явно. Когда клиент совершает определенный запрос API, сервис выдает ошибку и возвращает крайне неприятный HTTP-код **500 Internal Server Error**. В таких случаях исправление ошибки обычно окажется обратно совместимым, поскольку не может быть такого клиентского кода, который бы предполагал, что данный запрос будет приводить к внутренней ошибке сервера API, и начинал давать сбой, если бы тот переставал так делать.

Но как быть с более деликатными проблемами? Что, если определенный запрос API завершается успешно, возвращая бессмысленный результат в то время, когда должен возвращать ошибку (например, **400 Bad Request**)? Очень вероятно, что кто-то окажется в зависимости от такого ошибочного поведения, скрывающегося под личиной успешного. И если мы «исправим ошибку», то клиентский код, который ранее возвращал успешный результат, теперь начнет выдавать ошибочный.

Развивая тему, представьте случай, в котором эта ошибка скрыта глубоко внутри вычислительного кода и ее исправление подразумевает изменение получающихся чисел. Однажды, когда мы совершим вызов API наподобие `Add(0.1, 0.2)`, в ответ может вернуться `0.30000000000000004`; затем на следующий день мы исправим ошибку арифметики с плавающей запятой, и в ответ будет возвращаться уже `0.3`. Будем ли мы считать это исправление обратно совместимым? Соответствующие примеры представлены в листингах 24.1 и 24.2.

Листинг 24.1. Пример реализации API с проблемами арифметики с плавающей запятой

```
class AdditionApi {
  Add(a: number, b: number): number {
    return a + b;
  }
}
```

Здесь просто происходит сложение двух чисел с плавающей запятой, которое вызывает соответствующую ошибку

Листинг 24.2. Пример, в котором мы исправляем ошибку с плавающей запятой, используя вычисления с фиксированной запятой

```
const Decimal = require('decimal.js');

class AdditionApi {
  Add(a: number, b: number): number {
    return Number(Decimal(a).plus(Decimal(b)));
  }
}
```

Опираясь на конструкцию с фиксированной запятой, мы избегаем ошибки с плавающей. Но является ли такое изменение обратно совместимым?

К сожалению, и здесь нет единственного верного ответа. Как правило, исправление API, выбрасывающих ошибки вызовов, для большинства пользователей

оказывается безопасным. Если же в результате вызова API внезапно начнет выбрасывать ошибки, то это уже не очень удачная идея, особенно если пользователи обеспокоены стабильностью и предпочитают, чтобы их код не ломался по какой-то неочевидной причине.

В случае еще более деликатных исправлений все будет зависеть от образа типичного пользователя и влияния бага. Например, если ошибка заключается просто в сложении двух чисел, приводящем к получению результата, который близок к точному, но имеет проблему с плавающей запятой, то исправление может оказаться не столь важным. Если же эти вычисления используются для расчета траекторий запуска ракет, когда точность крайне важна, то будет уместным следовать политике исправления ошибки, даже если это ведет к изменению результатов день ото дня.

И здесь возникает схожий достойный обсуждения вопрос, который касается обязательных изменений.

Обработка обязательных изменений

Практически всегда решение о добавлении новой функциональности или внесении изменений в веб-API является нашим собственным. Иными словами, обычно именно мы, создатели этого API, решаем внести изменение и начинаем действовать соответствующим образом. Иногда, тем не менее, какая-то работа нам может быть навязана другими. Например, когда Европейский союз (ЕС) принял закон о защите данных (General Data Protection Regulation; GDPR; <https://gdpr-info.eu/>), понадобилось внести множество изменений, которых требовало обслуживание пользователей из ЕС. А поскольку многие из этих требований были ориентированы на сохранение данных и получение согласия на их использование в сети, вполне логично, что для соответствия этим новым законам в API нужно было вносить изменения. Все это ведет к очевидному вопросу: должны ли подобные изменения внедряться в существующую версию или же их нужно развертывать как новую? То есть должны ли мы считать изменения, вносимые для соответствия правилам, установленным в GDPR, обратно совместимыми?

В конечном итоге это решение зависит от конкретных изменений и требований закона, но при этом остается важным и то, кто использует наш API. В случае GDPR вы можете следить за сроками и, развернув новую версию с необходимыми изменениями, сохранить существующий API доступным до даты вступления закона в силу. После этого можно будет начать блокировать все запросы, поступающие от клиентов, зарегистрированных в ЕС (или с IP-адресов, расположенных в ЕС), требуя от этих потребителей использовать новую версию API, соответствующую требованиям GDPR. Или же может оказаться более разумным просто отключить несоответствующую версию API, учитывая потенциальные риски и штрафы за нарушение этого закона. При этом должно быть очевидно, что патчи безопасности и прочие изменения из этой категории зачастую являются обязательными, хоть и не с позиции закона.

И несмотря на то что бывают случаи, когда просто невозможно полностью соответствовать всем законам (например, говорят, что в блокчейне биткойна хранятся метаданные, запрещенные в большинстве стран, и сама технология блокчейна не позволяет изменять прошлые данные), практически всегда можно внести изменения, позволяющие выполнять требования законов. В действительности вопрос в том, как лучше всего реализовать эти изменения, не вызывая серьезного негодования со стороны пользователей. Иными словами, когда возникает необходимость подобных изменений, оградить потребителей API от их влияния практически невозможно. Поэтому вопрос заключается в том, как минимизировать это влияние, по возможности предупредив существующих пользователей заранее и сократив объем работы, которая требуется от них.

Внутренние изменения

Углубляясь в эту тему, мы можем увидеть и прочие тонкие виды изменений, которые будут считаться как обратно совместимыми, так и нет. Сюда обычно относятся такие примеры, как оптимизация производительности или прочие деликатные доработки функциональности, которые не являются ни исправлением ошибок, ни новыми возможностями, но все же представляют собой изменения внутренней системы, отвечающей на запросы API, и приводят к тому или иному изменению возвращаемых результатов.

Одним из примеров этого является оптимизация производительности (или ее снижение), которая приводит к тому, что вызов API возвращает результат быстрее (либо медленнее), чем раньше. Подобное изменение будет означать, что ответ API остается полностью идентичным, но отображается на сервере спустя иное количество вычислительного времени. Такое изменение зачастую считается обратно совместимым; однако на это можно посмотреть и с другой стороны.

Например, если вызов API на данный момент занимает около 100 мс, то будет сложно заметить изменение, которое вызовет сокращение выполнения этого вызова до 50 мс (если быстродействие улучшается) или увеличение до 150 мс (в случае ухудшения). С другой стороны, если вызов API внезапно займет 10 секунд (10 000 мс), то такой разницы во времени будет достаточно, чтобы оправдать изменения в парадигме программирования. Иными словами, при настолько медленном вызове API вы можете предпочесть использовать асинхронный стиль программирования, чтобы ваш процесс во время ожидания ответа от API мог заниматься другими вещами. В подобных случаях можно рассмотреть это как обратно несовместимое изменение или даже как внесенную ошибку, ведущую к столь серьезному падению быстродействия.

Еще один пример глубокого изменения может возникнуть, если вы применяете для генерации результатов вызова API модель машинного обучения. Это могут быть такие задачи, как перевод текста, распознавание лиц на изображении, разметка изменений сцены на видео и прочие. В таких случаях изменение

используемой модели может привести к сильно различающимся результатам подобных вызовов API. К примеру, в один день API может ответить вам, что на представленном изображении находится собака, а в другой распознать в нем маффин (это гораздо более распространенный случай, чем можно подумать; поищите по запросу «Чихуахуа или маффин»).

Если учесть разницу в результатах, такие изменения могут принести улучшения для одного пользователя API, но ухудшения для другого. При этом в зависимости от образа пользователей API здесь может фигурировать потребность в стабильности результатов, получаемых от модели машинного обучения. Иначе говоря, некоторые пользователи могут реально опираться на изображение, дающее согласующиеся результаты, пока сами не попросят произвести своего рода обновление. В подобных случаях такие изменения могут считаться обратно несовместимыми.

Изменение семантики

Последняя категория, которая наверняка может считаться наиболее обширной и деликатной, касается общих изменений семантики. Речь идет об общем изменении поведения или значений различных элементов API (например, ресурсов или полей).

Такие изменения охватывают широкий спектр возможностей. К примеру, это может быть что-то масштабное и очевидное (как поведенческие изменения в результате введения в API новой модели разрешений) или же что-то намного более тонкое и малозаметное (например, порядок элементов, возвращаемых при выводе списка ресурсов в поле массива). Нетрудно догадаться, что для обработки подобных изменений нет однозначной рекомендуемой практики. Лучшее, что можно сделать, — это изучить эффект, который изменение окажет на текущих пользователей и их код, проанализировать, насколько спокойно они смогут это принять, и затем решить, выдержит ли текущая версия его внедрение или же требуется выделение в новую.

Чтобы лучше понять смысл, представьте API чата с ресурсами `ChatRoom` и `User`, где пользователи могут создавать сообщения в комнатах чата. Что, если мы решим ввести новую политику сообщений, которая на основе ряда факторов будет определять, могут ли пользователи создавать сообщения? Такими факторами может выступать частота их отправки (например, запрет на размещение сообщения чаще одного раза в секунду) или даже само содержимое (например, алгоритм машинного обучения будет обнаруживать и блокировать сообщения с оскорблениями). Окажется ли изменение, вводящее такой ресурс, обратно совместимым? Думаю, не удивлю, если скажу, что и здесь ответом будет «возможно».

Начнем с того, что решение может зависеть от предустановленной семантики существующих ресурсов `ChatRoom` (например, применится ли новое поведение

к ним автоматически или же оно будет действовать только для новых). В любом случае нам необходимо оценить влияние на имеющихся пользователей. Не вызовет ли новая политика обработки сообщений сбой в работающем коде? И это тоже будет зависеть от разных факторов. Если у кого-то есть сценарий, который пытается отправлять два сообщения быстро одно за другим, то такой код может перестать корректно работать ввиду ограничения на отправку не более одного сообщения в секунду (листинг 24.3).

Листинг 24.3. Код, который может быть поврежден после изменения семантики

```
function testMessageSending() {
  const chatRoom = ChatRoom.get(1234);
  chatRoom.sendMessage("Hello!");
  chatRoom.sendMessage("How is everyone?");
}
```

Этот вызов `sendMessage()` будет успешен, как обычно

Этот вызов `sendMessage()` может завершиться сбоем, если будет установлено новое ограничение частоты отправки

Но как конкретно разворачивается этот сценарий? Будет ли пользователь получать ответ с ошибкой вместо ранее успешных ответов? Или же сообщение просто будет добавляться в очередь и отображаться секундой позднее? Ни один из этих вариантов не является однозначно правильным, и при этом они сильно различаются. В первом случае пользователь увидит ошибку сразу, и она определенно наведет его на мысль, что сбой кода вызван внесенным изменением. Во втором же случае ошибка может быть обнаружена позднее (например, если пользователь быстро отправит два сообщения, а затем проверит их получение, то этап проверки завершится неудачно, поскольку одно из них будет задержано на целую секунду) (листинг 24.4).

Листинг 24.4. Код, который после изменений может быть поврежден в другом месте

```
function testMessageSending() {
  const chatRoom = ChatRoom.get(1234);
  chatRoom.sendMessage("Hello!");
  chatRoom.sendMessage("How is everyone?");
  const messages = chatRoom.listMessages();
  if (messages.length !== 2) {
    throw new Error("Test failed!");
  }
}
```

Эти два вызова `sendMessage()` завершатся успешно, как обычно

Эта проверка публикации сообщений завершится неудачно, если между их отправкой и проверкой результата пройдет менее одной секунды

Как мы узнали, окончательное решение о том, является ли данное изменение обратно совместимым, будет зависеть от ожиданий текущих пользователей. Практически всегда среди них найдется такой, чей код под воздействием изменений будет повреждаться. Здесь уже вам решать, должен ли пользователь обоснованно ожидать, что конкретный фрагмент кода будет поддерживаться в течение всего срока жизни API или в его конкретной версии. Исходя из этого, можно определить, считать ли изменение семантики обратно совместимым.

Теперь, когда мы немного разобрались с различными проблемами, которые необходимо учитывать при выборе политики обратной совместимости, пора переходить к рассмотрению вопросов, касающихся выбора стратегии версионирования. Для этого нам нужно будет осознать различные компромиссы, подразумеваемые этим решением.

24.3. РЕАЛИЗАЦИЯ

Даже если предположить, что вы сделали ряд выборов относительно того, какие изменения считать обратно совместимыми, впереди у вас еще добрая половина остальных решений, имеющих отношение к реализации версионирования. Единственным случаем, не требующим дополнительных выборов, стал бы такой, в котором вы бы просто решили, что обратно совместимым будет абсолютно все, поскольку у API всегда будет одна версия. Во всех же остальных сценариях нам необходимо изучить и понять множество доступных параметров по обработке добавления новых версий по мере их необходимости. Иными словами, нужно разобраться, как именно происходит процесс добавления новой версии для внесения изменения, которое считается обратно несовместимым. Например, что нужно использовать в качестве имени новых версий? В идеале должен присутствовать некий паттерн, упрощающий для пользователей понимание схемы версионирования. Каким должен быть срок действия версии до момента ее устаревания? Как вариант, он может быть бесконечным, но во всех других случаях придется продумывать соответствующую политику, чтобы уведомить пользователей о сроках выхода устаревшей версии из обращения.

Принимая во внимание все эти нюансы, мы изучим несколько популярных стратегий версионирования, разобрав принцип их работы, плюсы и минусы, а также сравнительные преимущества на основе компромиссов, о которых было сказано в разделе 24.2. Не забывайте, что все эти стратегии могут намекать либо подразумевать набор политик для определения обратной совместимости и не всегда будут предписывать для этого единственную верную. Иными словами, каждая стратегия должна иметь некую гибкость, когда дело доходит до выбора определенных политик, на основе которых изменения относятся к обратно совместимым.

24.3.1. Вечная стабильность

Одна из наиболее популярных стратегий версионирования одновременно является и одной из тех, которые мы используем непреднамеренно. Куда чаще, чем многие это признают, при создании API вообще не планируется какая-либо схема версионирования. И лишь когда возникает необходимость внести большое, пугающее и, что самое главное, обратно несовместимое изменение, мы вспоминаем

об этом. В таком сценарии очередным шагом будет принять решение о том, что все, существующее на сегодня, станет v1, а API с новыми изменениями будет называться v2. В этот момент самое время задуматься о том, что еще вписать в эту v2. Например, то самое поле, которое мы уже давно хотели изменить, можно исправить в этой версии.

Эта стратегия зачастую называется *вечной стабильностью* в первую очередь из-за того, что каждая версия здесь обычно всегда остается стабильной, а потенциально обратно несовместимые изменения вносятся в очередную. И хотя такой подход склонен возникать сам собой, ничто не мешает нам использовать его намеренно. Как он работает в этом случае?

В общих чертах данная стратегия реализуется следующим образом.

1. Вся существующая функциональность обозначается как Version N (например, первая версия будет Version 1).
2. Все изменения из разряда обратно совместимых добавляются непосредственно в Version N.
3. Любая новая функциональность, выходящая за рамки определения совместимости, создается уже как часть Version N+1 (например, все обратно несовместимое с Version 1 сохраняется для Version 2).
4. Как только у нас набирается достаточно функциональности для новой версии, мы выпускаем Version N+1.
5. По желанию в будущем можно будет прекратить поддержку Version N, чтобы избежать излишних вычислительных и временных затрат или просто потому, что ее больше никто не использует.
6. На данном этапе мы возвращаемся к шагу 1, и цикл повторяется.

Этот процесс отлично работает в ряде сценариев и имеет компромиссы, которые отражены на рис. 24.2. Когда вы не вносите множество обратно совместимых изменений, большую их часть можно включить в существующую версию, не создавая особых проблем для пользователей, в результате чего процесс чаще всего будет стопориться на шаге 2. При условии довольно низкой планки определения обратной совместимости мы наверняка получим очень стабильный API, но пожертвуем быстрым развертыванием большого количества новых возможностей.



Рис. 24.2. Компромиссы при использовании стратегии вечной стабильности

Кроме того, при условии низкого порога признания изменения обратно совместимым эта стратегия максимизирует количество людей, которые могут предсказуемо использовать API на ее основе. Она может быть неидеальной, но большинство пользователей окажутся довольны.

Не очень хорошо эта стратегия покажет себя в случаях, когда пользователям требуется высокая детализация. Например, IoT-устройство в качестве клиента, использующего эту стратегию версионирования, наверняка столкнется с проблемами, поскольку она подразумевает внесение множества изменений в текущую версию, в то время как IoT-устройствам обычно нужна возможность замораживать API в определенной версии во избежание пограничных случаев, таких как превышение ресурсов памяти.

В случаях же, когда порог определения обратной совместимости исключительно высок (то есть все изменения считаются несовместимыми), вы можете оказаться в ситуации очень большого количества версий (v1, v2... v100 и т. д.). Очевидно, что это может стать нежелательным как для сервиса, управляющего всеми этими версиями, так и для клиентов, решающих, какая же версия подходит им лучше всего.

Независимо от этих проблем многие популярные API опираются именно на такой механизм версионирования и за долгие годы неплохо его оптимизировали. Например, многие API Google Cloud Platform используют данную стратегию по разным причинам, и, несмотря на все свои недостатки, она неплохо работает для многих потребителей.

24.3.2. Подвижная нестабильность

Еще одна популярная стратегия, иногда называемая *подвижной нестабильностью*, опирается на скользящее окно активных версий в целях минимизации усилий, затрачиваемых разработчиками API на обслуживание, но при этом сохраняет регулярный выпуск новой функциональности. И несмотря на свою неидеальность, эта стратегия может оказаться весьма эффективной при работе с активными пользователями, вовлеченными в развитие продукта до такой степени, чтобы смириться с частым отказом от предыдущих версий в угоду получения новой функциональности.

Принцип этого подхода состоит в прохождении каждой версией стандартного жизненного цикла, начиная с рождения (превью), когда вся функциональность еще может измениться, и до самой смерти (удаление), когда версия перестает быть доступной. Обзор различных состояний версии на этом пути показан в табл. 24.1, но, чтобы лучше понимать принцип работы, мы разберем пример сценария с несколькими версиями API.

Таблица 24.1. Обзор различных статусов версий

Статус	Описание	Политика изменений
Превью	Нестабильное и еще дорабатываемое представление функциональности, которую пользователи могут ожидать в следующей текущей версии	Все изменения разрешены
Текущая	Очень стабильный набор функциональности, который не должен изменяться и будет продолжать существовать в течение предопределенного периода времени	Только необходимые изменения
Устаревшая	Бывшая текущая версия, которую планируется удалить согласно политике депрекации	Только необходимые изменения
Удаленная	Бывшая устаревшая версия, которая была удалена или по иным причинам стала недоступной для пользователей	N/A

В самом начале своей жизни API определенно недостаточно стабилен, чтобы его можно было показывать другим. Самое же главное, что на данном этапе разработчикам нужна свобода изменения такого сырого представления. В текущем сценарии проще всего будет поместить всю работу в версию 1. В таком случае мы назовем эту версию «Превью», поскольку для реальных пользователей она еще не готова. Те же из них, кто принимает тот факт, что на этой стадии нет никаких гарантий стабильности, вполне могут ее использовать. Но при этом они должны четко понимать всю суровую истину: написанный ими сегодня код вполне может перестать работать завтра.

Как только версия 1 станет более зрелой, мы можем предложить реальным пользователям начать строить вокруг нее свои решения. В данном случае мы повышаем ее до статуса «Текущая». Находясь в этой стадии, нам нужно обеспечить непрерывную работоспособность клиентского кода, поэтому единственными изменениями должны быть обязательные требования (например, патчи безопасности) и потенциально критические исправления ошибок. Если коротко, то повышение версии 1 до текущей означает, что мы замораживаем ее в настоящем виде и трогаем только в случае крайней необходимости.

Естественно, разработка функционала не должна на этом прекращаться. Так куда же мы деваем результаты всего последующего труда? Все просто: любая новая функциональность или прочие некритические изменения, которые мы бы хотели внести в версию 1, переносятся в предстоящую версию 2. При этом сама версия 2 также изначально выходит в статусе «Превью» и подвергается действию тех же правил, которые применялись к версии 1, когда та была на аналогичной стадии.

В определенный момент версия 2 также становится достаточно зрелой, чтобы перейти на стадию «Текущая». Когда это происходит, у нас возникает очередная проблема: что делать с существующей версией? Нам не следует продолжать поддерживать две текущие версии, но мы также не хотим удалять прежнюю версию

полностью просто потому, что теперь есть нечто более свежее и удобное. В данной стратегии решением будет обозначить прежнюю версию как «устаревшую». Пока она находится в этом статусе, к ней применяются те же правила относительно изменений, что и в ее бытность «текущей». Тем не менее здесь мы запускаем обратный отсчет времени до момента, когда эта версия будет в конечном итоге исключена из обращения — в конце концов, мы не хотим бесконечно тратить ресурсы на поддержку всех когда-либо вышедших версий.

Как только обратный отсчет заканчивается, мы удаляем устаревшую версию, и она переходит в категорию удаленных. Конкретный промежуток этого периода до удаления зависит от ожиданий пользователей API. Однако самое важное здесь — объявить об этом заранее и придерживаться заявленного срока. В общем виде прохождение версиями различных циклов показано в табл. 24.2, где выпуск новых производится с периодичностью в один месяц, а окончательный отказ от устаревших — раз в два месяца.

Таблица 24.2. Пример жизненного цикла версий и их статусов

Дата	Версия 1	Версия 2	Версия 3	Версия 4	Версия 5
Январь	Превью				
Февраль	Текущая	Превью			
Март	Устаревшая	Текущая	Превью		
Апрель	Устаревшая	Устаревшая	Текущая	Превью	
Май	Удалена	Устаревшая	Устаревшая	Текущая	Превью

В этой стратегии есть ряд интересных моментов, и на рис. 24.3 показаны сопутствующие ей компромиссы. Для начала обратите внимание, что, несмотря на множество различных устаревших (и удаленных) версий, всегда присутствует лишь одна текущая и одна превью. Это важнейший аспект стратегии, определяющий ее стремление к непрерывному улучшению при сохранении минимального количества активных версий. Иными словами, в этой стратегии мы рассматриваем новые версии как улучшения поверх текущей, а не в качестве альтернативных равноценных представлений. Отказываясь от устаревающих версий в пользу более качественных, мы обеспечиваем быстрое предоставление новой функциональности пользователям.



Рис. 24.3. Компромиссы стратегии подвижной нестабильности

Обратите внимание еще и на то, что данное преимущество быстрой смены версий может иметь и недостаток: любой код, написанный под текущую версию, практически однозначно в итоге перестанет работать. По меньшей мере просто не будет никакой гарантии продолжения его работоспособности, и если она сохранится, то лишь по счастливой случайности. Это значит, все пользователи, которые ожидают, что смогут написать код и забыть о нем на пару лет, скорее всего, сочтут такую стратегию неприемлемой. С другой стороны, все, кто активно обновляет свой код и особенно ценит новую функциональность, найдут подобный подход к версионированию удачным.

В целом эта стратегия может отлично подойти в сценариях с активными пользователями, вовлеченными в процесс разработки и нуждающимися лишь в небольших окнах стабильности, сопровождаемых быстрыми обновлениями до новых версий с новой функциональностью. Те же пользователи, которым требуется долгосрочная стабильность в обмен на более редкое получение новой функциональности, определенно сочтут такую модель сложной в применении. И хотя случайно может получиться так, что код продолжит работать без сбоев, отсутствие гарантии этого станет сильным отталкивающим фактором для всех, кто нуждается в подтверждении долгосрочной стабильности.

24.3.3. Семантическое версионирование

Пожалуй, самой популярной формой версионирования сегодня является *семантическое* (или SemVer; <https://semver.org>) — очень эффективный инструмент, позволяющий одной метке версии включать в себя несколько значений, выраженных тремя простыми числами. Самое же важное заключается в практичности такого подхода: он может сообщить пользователю разницу между двумя версиями API и то, будет ли код, написанный для одной из них, работать с другой. Далее мы подробно разберем, как все это реализуется.

Семантическая строка версии (рис. 24.4) составляется из трех чисел, разделенных точками (например, 1.0.0 или 12.5.2), где по мере внесения в API изменений каждое число увеличивается, передавая соответствующий этому изменению смысл. Первое число строки, называемое *старшей версией*, увеличивается в сценариях, когда изменение считается обратно несовместимым, о чем мы говорили в подразделе 24.2.2. Например, изменение имени поля практически всегда считается обратно несовместимым, поэтому переименование поля приведет к тому, что в строке версии увеличится ее старшее значение (например, с 1.0.0 до 2.0.0). Иными словами, вы можете предположить, что любой код, написанный для старшей версии N, практически однозначно перестанет работать ожидаемым образом в старшей версии M. Если же он все же заработает, то лишь по чистой случайности, а не ввиду гарантий самого дизайнера.



1. 4. 2
Старшая Младшая Патч

Рис. 24.4. Составляющие номера версии в семантической строке

Следующим числом в строке обозначается *младшая версия*. Оно увеличивается, когда в соответствии с правилами, определенными в политике совместимости, вносится обратно совместимое изменение, то есть добавляется какая-то новая функциональность или изменяется поведение. Например, если мы решим добавить в API новое поле и в политике совместимости определим это действие как обратно совместимое, то в строке версии нужно будет увеличить младшее значение (скажем, с 1.0.0 на 1.1.0). Эта возможность весьма полезна, поскольку позволяет уверенно предположить, что любой код, написанный для определенной младшей версии, гарантированно не будет поврежден при работе с последующими младшими релизами, так как в каждом из них будут производиться только обратно совместимые изменения (например, код, написанный для версии 1.0.0, продолжит работать и с 1.1.0, и с 1.2.0). И хотя этот более старый код не сможет воспользоваться новой функциональностью, представленной в новых младших релизах, он все равно продолжит работать в них ожидаемым образом.

Третье число в семантической строке означает версию патча. Оно увеличивается при внесении изменения, которое обратно совместимо и, как правило, несет исправление ошибки, а не новую функциональность или коррекцию поведения. В идеале это число не должно подразумевать каких-либо очевидных последствий для совместимости. То есть если две версии отличаются только версиями патчей, то код, написанный для одной из них, вполне должен работать и с другой.

При использовании в веб-API семантического версионирования мы просто следуем описанным правилам. Вносите изменение, которое будет обратно несовместимым? Значит, оно должно выходить как очередная старшая версия. Добавляете обратно совместимую функциональность? Оставляйте старшую версию прежней и повышайте младшую. Вносите обратно совместимое исправление ошибки? Оставляйте старшую и младшую версии прежними, повышая лишь версию патча. Как мы узнали в подразделе 24.2.2, категория изменений будет определяться вашей политикой совместимости. Если вы будете следовать ей, то семантическое версионирование будет работать исправно.

Однако у этой стратегии есть и слабое место, а именно огромное количество доступных пользователю версий, которыми к тому же нужно управлять как отдельными сервисами. Иными словами, семантическое версионирование представляет на выбор настолько много версий, каждую со своим функционалом, что пользователь в итоге может запутаться.

Кроме того, пользователи хотят иметь возможность придерживаться одной конкретной и детализированной версии как можно дольше, и это означает, что веб-API должен поддерживать множество разных версий, что чревато большими проблемами. К счастью, поскольку версии после выпуска практически замораживаются, можно приостановиться и продолжить просто использовать

исполняемый файл, но вычислительная нагрузка все равно останется весьма значительной. Принимая во внимание все это, можно прибегнуть к помощи современных систем оркестрации и управления (например, Kubernetes; <https://kubernetes.io/>), а также специальным парадигмам разработки.

Тем не менее версии не обязательно должны существовать бесконечно. Наоборот, нет ничего плохого в том, чтобы определить политику депрекации и с самого момента выпуска версии заявлять о том, как долго она будет поддерживаться. Впоследствии по истечении этого срока нужно будет просто удалять устаревшую версию из сервиса.

Помимо этой политики депрекации, одним из лучших решений при использовании семантического версионирования будет соблюдение баланса между стабильностью и новой функциональностью, что позволит получить обоюдное преимущество. В определенном смысле пользователи смогут придерживаться конкретных версий (вплоть до конкретного исправления ошибки) и в то же время при необходимости будут иметь доступ к новой функциональности. Здесь есть очевидная проблема, связанная с возможной сложностью выбора определенных функций (например, кто-то может захотеть использовать версию 1.0.0, но получить доступ к особой функциональности, добавленной лишь в версии 2.4.0), но эта проблема касается любой стратегии версионирования, основанной на хронологии.

Наконец, этот подход находит баланс между удовлетворением всех и лишь небольшого количества людей (схематично показано на рис. 24.5). Если коротко, то ввиду богатства выбора и потенциально рациональной политики депрекации большинство клиентов, начиная с мелких стартапов, заинтересованных в быстром получении новой функциональности и заканчивая теми, кто ищет стабильности крупных корпораций, будут довольны такой схемой версионирования. Самое же важное, что это не будет вызывать у пользователей раздражение. Такой подход может оказаться слегка перегруженным в количестве доступных версий, но в целом сама стратегия позволит пользователям получать от API то, что им нужно, если только это присутствует в одном из его релизов.

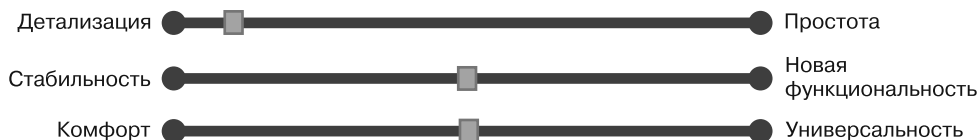


Рис. 24.5. Компромиссы стратегии семантического версионирования

Обратите внимание, что это лишь несколько популярных стратегий версионирования в веб-API, а не полный список. Как мы видим, у каждой есть собственные плюсы и минусы, поэтому ни одна из них не будет идеальной для всех сценариев.

В результате важно понять все доступные параметры и основательно подумать о том, какие из них окажутся более подходящими для вашего веб-API, учитывая ограничения, требования и предпочтения вашей уникальной категории пользователей.

24.4. КОМПРОМИССЫ

Как видите, выбрать политику для определения того, какие изменения считать обратно совместимыми, непросто. Но самым важным является то, что сделанные нами в этом отношении выборы помогут достичь баланса между двумя концами спектра. И несмотря на то что здесь обычно нет идеального варианта, всегда нужно понимать принимаемые компромиссы и стремиться делать эти выборы сознательно. Ниже мы отдельно разберем различные соотношения, которые необходимо учитывать при выборе политики совместимости для каждой уникальной ситуации.

24.4.1. Детализация или простота?

Первый и очень широкий спектр, от которого мы зачастую будем отталкиваться, касается богатства выбора. В своей книге *The Paradox of Choice* («Парадокс выбора») Барри Шварц рассуждает о том, что более богатый выбор потребительских товаров не всегда ведет к повышению удовлетворенности покупателей. Наоборот, зашкаливающее количество предложений может вызвать повышенную раздраженность. При разработке API мы, конечно, не покупаем продукт в гипермаркете, но все же это утверждение в определенной степени работает и здесь. Когда дело доходит до выбора политики версионирования, один из компромиссов заключается в достижении баланса между разнообразием выбора и простым универсальным решением.

Чтобы понять, как это работает, представьте случай, в котором мы решаем, что наши потребители настолько заинтересованы в стабильности, что любое изменение должно рассматриваться в качестве обратно несовместимого. В результате каждое изменение будет разворачиваться как полностью новая версия, и пользователи API будут иметь возможность выбирать из потенциально огромного количества доступных версий. Естественно, такая обширная коллекция гарантирует, что отдельный пользователь сможет подобрать под свои задачи очень специфичную версию, но при этом также возникает ситуация, в которой этот пользователь будет ошарашен или запутается в огромном количестве вариантов.

Кроме того, есть вероятность, что подобные схемы версионирования будут склонны к частому изменению во времени. То есть пользователь не сможет подобрать

конкретные аспекты API, а получит лишь своеобразную машину времени, где можно выбирать некую версию, представляющую API в определенный момент его истории. Если этот пользователь захочет применить поведение двухлетней давности совместно с неким современным функционалом, то обычно ему придется принять все изменения в поведении, произошедшие с тех пор. Коротко говоря, использование любой версии предполагает содержание в ней всех изменений, сделанных до момента ее выпуска.

А теперь рассмотрим другой конец спектра: каждое изменение всегда считается обратно совместимым, независимо от влияния на текущих пользователей. В этом сценарии итоговый API всегда будет иметь только одну версию, то есть выбора у пользователей не будет совсем. И хотя такой вариант сложно обвинить в недостатке выбора, он, вероятно, слишком смещается в сторону чрезмерного упрощения. Иными словами, один вариант проще всего понять, но он редко будет уместен, учитывая типичную аудиторию API, особенно тех, кто заинтересован в особых требованиях к стабильности.

Очевидно, что правильного ответа здесь нет, но лучший вариант находится где-то посередине между этими двумя полюсами спектра. Это может означать подбор такого набора политик обратной совместимости, которые будут приводить к разумному количеству версий (значение «разумного» здесь явно будет индивидуальным) либо же поиску политики депрекации, в которой старые версии с течением времени удаляются. В любом случае итоговый компромисс будет зависеть от пользователей и их жажды выбора (а также вашего желания поддерживать все это разнообразие).

24.4.2. Стабильность или новая функциональность?

Очередной компромисс, пожалуй, является самым очевидным и связан с вопросом о том, сколько же функциональности пользователям действительно нужно. Иными словами, если вы просто будете вносить в API меньше изменений, то вам будет сложнее решить, является ли изменение вообще обратно совместимым. Этот компромисс может подразумевать отказ от политики совместимости, что для пользователей обернется ограничением доступа к новой функциональности, исправлениям ошибок и изменениям в поведении.

Чтобы лучше понять все вышесказанное, представьте, что на одном конце спектра у нас идеальная стабильность: никаких изменений, никогда. Этот сценарий означает, что после запуска API он больше не изменится. Конечно, такой вариант выглядит простым, но в действительности оказывается гораздо более сложным. Чтобы сохранять идеальную стабильность, вам нужно гарантировать отсутствие патчей безопасности или обновления внутренней операционной системы серверов, на которых работает ваш API. Причем это касается

и ваших поставщиков. Если хоть один из этих факторов изменится, то в поле зрения пользователей сможет проникнуть нечто, пусть и едва заметное, но уже технически являющееся изменением. И здесь вам нужно будет определиться, достойно ли оно новой версии.

Будучи на другом конце спектра, вы можете решить, что любую функциональность критически необходимо запустить, а все ошибки непременно нужно исправить. При этом операционная система, обновления библиотек, патчи безопасности и прочие внутренние изменения должны внедряться в API как можно быстрее. В таком случае выборы, касающиеся политики обратной совместимости, обретут невероятную значимость.

Как обычно, здесь нет единого лучшего варианта. Однако API, которые вместо того, чтобы придерживаться определенного компромисса на основе интереса пользователей, делают этот выбор по ходу дела, склонны раздражать намного больше. При этом и здесь лучший вариант тоже находится где-то посередине. Для менее подвижных пользователей (например, крупных правительственных организаций) новые функции могут быть не так важны, как стабильность. В случае же стартапов верным чаще будет противоположное. Но важно определить ожидания потенциальных пользователей API и решить, какая точка спектра окажется наиболее подходящей для них.

24.4.3. Удовлетворенность или универсальность?

Заключительный и, пожалуй, наиболее важный компромисс связан со степенью принятия вашей политики различными пользователями API. До этого момента мы рассматривали пользователей как единую группу, которая либо довольна вашими политиками совместимости, либо нет. К сожалению, такая однородность среди потребителей встречается относительно редко. Иными словами, маловероятно, что ваши политики будут радушно приняты каждым пользователем, по той простой причине, что все пользователи разные и имеют свое мнение относительно обсуждаемых нами вопросов. И хотя некоторые API могут быть ориентированы на однородные группы пользователей (например, интерфейсы для центральных банков или муниципальных госструктур могут в гораздо большей степени требовать стабильности, нежели обновления функциональности), многие API привлекают разнообразные группы потребителей, которых мы уже не можем рассматривать как одну категорию. Что же делать?

Очевидно, что здесь возникает очередной компромисс. И чтобы его понять, нужно образно делить пользователей на четыре категории в соответствии со степенью удовлетворенности нашей политикой. В этот спектр будут входить те, кто откажется от использования API ввиду обстоятельств (категория «Не могут использовать»), те, кто может им пользоваться, но не слишком этому рад

(«Недовольны»), те, кто доволен установленной политикой, но не в восторге («Довольны»), и те, кто доволен ей абсолютно («Счастливы»). И хотя обычно большинство пользователей попадает в категорию «Довольны», некоторые политики могут разделять большую группу потенциальных пользователей. Например, если в API будет действовать политика принятия всех изменений как обратно совместимых, то она может разделить многих людей и подтолкнуть их к поиску решения в других местах (скажем, многие окажутся в категории «Не могут использовать»). Так что рассмотрим наш компромисс, учитывая подобное распределение.

На одном конце спектра у нас максимально довольные пользователи, когда все решения нацелены на увеличение категории «Счастливы». Естественно, невозможно сделать счастливыми всех пользователей, но мы всегда можем стремиться к этому. Однако нужно помнить, что в этом случае мы выбираем политику без учета других категорий. Это может привести к ситуации, показанной на рис. 24.6, когда мы максимизируем количество счастливых пользователей, но из-за недостатка внимания по отношению к другим категориям многие оказываются в категории «Не могут использовать».



Рис. 24.6. Можно максимально увеличить количество счастливых пользователей ценой получения большого количества тех, кто вообще не может использовать API

На противоположном конце спектра у нас максимальная пригодность к использованию для всех пользователей, когда наше решение нацелено на увеличение количества тех, кто точно сможет использовать API. Иначе говоря, этот конец спектра стремится минимизировать категорию «Не могут использовать», в связи с чем вызывает схожую проблему, то есть не учитывает, в какой категории оказываются те, кто не попадает в эту. На рис. 24.7 мы видим потенциальный разброс пользователей, где налицо их минимальная концентрация в категории «Не могут использовать». Но здесь довольно очевидно, что это вряд ли окажется удачной ситуацией, поскольку подавляющее большинство пользователей, хоть и могут работать с API, определенно остаются недовольны этим опытом.



Рис. 24.7. Можно минимизировать количество тех, кто не может использовать API, но ценой увеличения количества недовольных

Как всегда, оптимальное решение находится где-то посередине и зависит от образа наших пользователей. Оно может стремиться к поиску баланса между максимизацией количества счастливых пользователей и минимизацией количества тех, кто вообще не может работать с API. В идеале наше решение также может минимизировать количество недовольных пользователей и искать возможность увеличить объем категорий «Довольны» и «Счастливы». В результате у нас может получаться распределение, показанное на рис. 24.8. Уверенным здесь можно быть лишь в том, что нам вряд ли удастся найти политику, делающую счастливыми всех и всегда. При этом также будет полезным определиться, ориентируемся ли мы на более широкую доступность API за счет уменьшения количества тех, кто не может его использовать, или же нас больше интересует повышение именно удовлетворенности пользователей.

Не могут использовать	Недовольны	Довольны	Счастливы
--------------------------	------------	----------	-----------

Рис. 24.8. Сбалансированное распределение минимизирует категорию тех, кто не может использовать API, но при этом сохраняет внушительное количество довольных пользователей

24.5. УПРАЖНЕНИЯ

1. Будет ли изменение предустановленного значения поля считаться обратно совместимым?
2. Представьте, что у вас есть API, опосредующий финансовые услуги крупного коммерческого банка, являющегося важным клиентом. Многие более мелкие потребители хотят чаще получать обновления и новые возможности, в то время как банк в первую очередь интересуется стабильностью. Как можно сбалансировать эти потребности?
3. Сразу же после запуска API вы замечаете глупую и смущающую опечатку в имени поля: вместо `port_number` вы случайно ввели `porn_number`. Вам хочется исправить эту ошибку как можно быстрее. О чем нужно подумать, прежде чем принять решение о внесении исправлений без изменения номера версии?
4. Представьте сценарий, в котором ваш API при отсутствии обязательного поля случайно возвращает пустой ответ (200 OK в HTTP), а не ошибку (400 Bad Request), указывая на отсутствие этого поля. Можно ли безопасно исправить такую ошибку в текущей версии или же это нужно делать уже в следующей? Если в следующей, то каким изменением это будет считаться согласно принципу семантического версионирования (semver.org): старшим, младшим или патчем?

РЕЗЮМЕ

- Версионирование — инструмент, который позволяет разработчикам постепенно изменять и развивать свои API, создавая для пользователей минимум неудобств и предоставляя им максимум улучшений.
- Совместимость — свойство, характеризующее возможность кода, написанного для одной версии API, работать с другой.
- Новая версия считается обратно совместимой, если код, написанный для предыдущей версии, продолжит работать в ней ожидаемым образом. При этом остается вопрос, как разработчик API определяет ожидания пользователей.
- Зачастую, казалось бы, безобидные вещи (например, исправление ошибок) могут вести к обратно несовместимым изменениям. Определение того, является ли изменение обратно совместимым, зависит от пользователей API.
- Есть несколько различных популярных стратегий версионирования, каждая со своими плюсами и минусами. Большинство из них сводится к компромиссам между детализацией, простотой, стабильностью, новой функциональностью, удовлетворенностью отдельных пользователей и общей доступностью к использованию.

25

Мягкое удаление

В этой главе

- ✓ Что такое мягкое удаление и когда оно полезно.
- ✓ Способы, позволяющие указать, что ресурс отмечен как удаленный, но на деле таковым не является.
- ✓ Необходимые изменения стандартных методов ресурсов, поддерживающих мягкое удаление.
- ✓ Как снять удаление с мягко удаленных ресурсов.
- ✓ Как окончательно удалять мягко удаленные ресурсы.
- ✓ Управление ссылочной целостностью.

Как мы узнали в главе 7, у стандартного метода `Delete` есть одна цель: удалить ресурс из API. Тем не менее во многих сценариях это окончательное удаление данных из API (иначе называемое *жестким удалением*) оказывается слишком радикальным. В некоторых случаях нам требуется подобие стандартной компьютерной корзины: ресурс в ней отмечается как «удаленный» и впоследствии мы можем восстановить его, если удаление произошло по ошибке. В текущем паттерне мы познакомимся с *мягким удалением*, при котором ресурсы скрываются из представления и во многих отношениях действуют так, будто реально были удалены, но при этом сохраняют возможность полноценного восстановления.

25.1. ЗАЧЕМ ЭТО НУЖНО

Начиная с Windows 95, компания Microsoft представила корзину — временную область хранения файлов, которые по запросу пользователя были как бы удалены, но не окончательно. Причиной этого нововведения стала простая проблема: очень много людей удаляли файлы с компьютеров, после чего понимали, что делать этого не стоило. Это могло происходить случайно (щелчок не на том файле), ввиду заблуждения (пользователь считал, что это правильно, а позже осознал, что ошибался) или даже просто потому, что человек передумал. Как бы то ни было, речь шла об одном и том же: пользователям требовалось обратимое действие удаления, отделенное от окончательного стирания данных. Аналогичная проблема нередко возникает и в API.

Мы можем написать код, удаляющий все ресурсы с заданным набором свойств, но вдруг ошибиться в операции сравнения и в итоге удалить кучу тех, которые не соответствуют заданному критерию. У нас может быть сценарий удаления данных, который мы по случайности запустим не в тот момент или же сначала решим, что некоторые ресурсы можно удалить, а потом выяснится, что они нужны. Ситуация еще больше ухудшается в случае больших наборов данных. Можете представить случайное удаление петабайта пользовательских данных из хранилища? (Именно поэтому нельзя удалить всю корзину в сервисе Google Cloud Storage.)

Учитывая все вышесказанное, было бы хорошо, если бы удаление, пусть даже только определенных типов ресурсов, можно было выполнить частично, чтобы данные не стирались окончательно, а просто скрывались из виду при просмотре (например, с помощью стандартного метода `List`). И для того чтобы сделать такую форму удаления действительно полезной, нам потребуется возможность восстанавливать ресурсы так же легко, как мы их удалили. Наконец, нам потребуется способ выполнения стандартного безвозвратного удаления ресурсов.

25.2. ОБЗОР

К счастью, механизм мягкого удаления ресурсов довольно прост: сохранять статус удаления в самом ресурсе с помощью простого логического флага `deleted` либо, если ресурс использует перечисление состояний, состояния `deleted`. Это поле будет служить маркером на протяжении всей жизни ресурса, указывая, что его нужно рассматривать как отличающийся от остальных ресурсов.

После того как мы реализуем этот новый способ сохранения статуса `deleted` в ресурсах, нужно будет изменить стандартный метод `Delete`, чтобы он изменял этот статус, а не удалял ресурс полностью. Для обработки полного удаления мы задействуем особый метод `Expunge`, который возьмет на себя эту работу. Наконец, нам потребуется способ восстановления мягко удаленных ресурсов, в роли

которого выступит пользовательский метод `Undelete`. Этот новый жизненный цикл ресурса с поддержкой мягкого удаления будет соответствовать изображению, показанному на рис. 25.1.

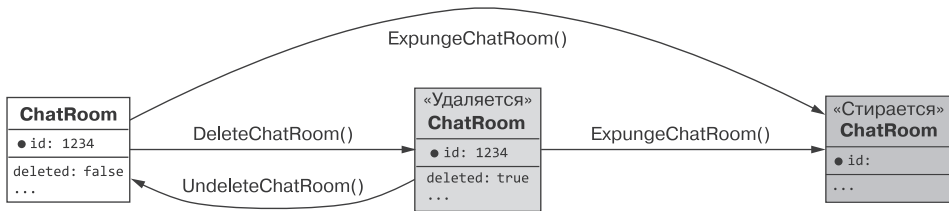


Рис. 25.1. Жизненный цикл ресурса, который мягко удаляется и стирается

25.3. РЕАЛИЗАЦИЯ

Как и в большинстве тем проектирования API, большая часть работы будет заключаться в реализации деталей. В данном случае вся она появляется в результате одного простого решения использовать дополнительное поле для указания того, является ли ресурс удаленным. Когда мы примем решение о том, как должно выглядеть это поле, перед нами останется еще немало задач.

Сначала нужно настроить пару стандартных методов, слегка изменив их поведение, определенное в главе 7. К примеру, вместо того чтобы окончательно удалять ресурс, стандартный метод `Delete` теперь должен действовать больше сродни методу `Update`, изменяя в ресурсе отметку об удалении. Кроме того, стандартный метод `List` должен будет игнорировать ресурсы, удаленные таким образом. В конце концов, если он будет по умолчанию включать и их тоже, то это полностью лишит смысла разметку ресурсов как удаленных. Наконец, нам потребуются пользовательские методы, реализующие каждый вид нового поведения: один для восстановления мягко удаленного ресурса и второй для окончательного удаления ресурса (как это делал стандартный метод `Delete`).

Как обычно, эти новые методы и их поведение вызовут новые непростые вопросы. Например, что происходит, если мы пытаемся мягко удалить ресурс, уже отмеченный как удаленный? И нужно ли нам внедрять какие-либо правила ссылочной целостности, когда другие ресурсы ссылаются на те, которые отмечены как удаленные? А как быть с прочими методами, аналогичными стандартному методу `Delete`, такими как пакетный `Delete`? Должны ли они подстраиваться под это новое поведение или же им следует продолжать действовать как прежде?

Список вопросов выглядит внушительным, так что начнем по порядку, а именно с определения в ресурсах статуса `deleted`.

25.3.1. Обозначение удаления

Чтобы применить этот паттерн к определенному типу ресурсов, нам потребуется способ, позволяющий указать, что система должна считать ресурс удаленным. В данном случае мы можем использовать простое поле с логическим флагом или более сложное поле состояния. Начнем с более простого варианта.

Логический флаг

Обычно обозначать мягкое удаление лучше всего с помощью простого логического поля (листинг 25.1). Очевидно, что по умолчанию оно будет устанавливаться на `false`, поскольку создаваемые ресурсы не должны начинать свой жизненный цикл как удаленные.

Листинг 25.1. Мягко удаленный ресурс с логическим флагом

```
interface ChatRoom {
  id: string;
  // ...
  deleted: boolean;
}
```

← Этот флаг определяет, должен ли ресурс считаться мягко удаленным

Более же интересным аспектом этого поля является тот факт, что его следует рассматривать как только для вывода (листинг 25.2). То есть если кто-то попытается установить его (например, с помощью стандартного метода `Update`), то само значение должно игнорироваться. Вместо этого, как мы увидим позже, единственным способом, позволяющим изменить данное поле, станет использование стандартного метода `Delete` (для изменения с `false` на `true`) или пользовательского `Undelete` (для обратного). Это означает, что код, обновляющий в ресурсе только поле `deleted`, по-прежнему выполнится успешно, но сам ресурс останется прежним.

Листинг 25.2. Пример взаимодействия с полем только для вывода

```
let chatRoom = GetChatRoom({ id: "1234" });
assert chatRoom.deleted == false;
chatRoom.deleted = true;
chatRoom = UpdateChatRoom({
  resource: chatRoom,
  fieldMask: ['deleted']
});
assert chatRoom.deleted == false;
```

← Начинаем с извлечения ресурса с помощью стандартного метода `Get`

← Здесь мы вручную устанавливаем поле `deleted` на `true`

← Обратите внимание: здесь не возвращается ошибочный результат, а просто игнорируются поля только для вывода, в частности `deleted`

← Несмотря на успешный результат, ресурс все равно не удален

С помощью этого простого логического поля «да/нет» можно внести любое другое сложное поведение, о котором будет сказано в подразделе 25.3.2 и далее. Но все же сначала предлагаю рассмотреть второй вариант: поле состояния.

Поле состояния

В некоторых случаях ресурсы могут находиться в одном из нескольких состояний определенного жизненного цикла, которые можно представить с помощью подходящей машины состояний. Представьте мир, в котором новые комнаты чата должен утверждать системный администратор, после чего их все равно можно блокировать, если участники нарушают правила. В таком сценарии у нас может быть схема состояний, включающая ожидание подтверждения, активное состояние и блокировку (рис. 25.2).

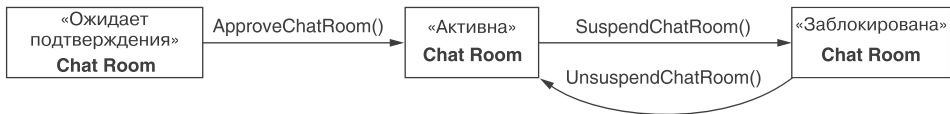


Рис. 25.2. Жизненный цикл ресурса с изменением состояния

Когда у нас уже есть поле, отслеживающее состояние заданного ресурса, вместо добавления логического флага можно задействовать это же поле, дополнив его новым состоянием `deleted`. В итоге мы получим вот такую схему состояний (рис. 25.3).

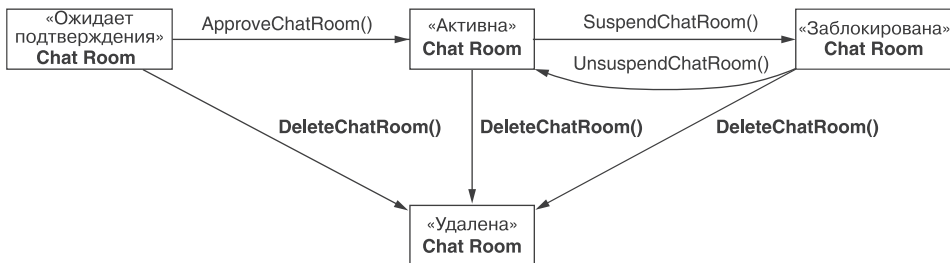


Рис. 25.3. Добавление состояния `deleted`

И хотя на первый взгляд такой подход может казаться вполне рабочим, в действительности он чреват большой проблемой: как узнать состояние ресурса, когда потребуется его восстановить? В этом случае можно предположить, что восстанавливаемый ресурс `ChatRoom` нужно всегда возвращать к стадии подтверждения, рассматривая его аналогично только что созданному (рис. 25.4). Однако такое допущение сработает не во всех сценариях. Иными словами, будет абсолютно приемлемым отслеживать состояние ресурса с помощью поля состояния, а этот отдельный аспект удаления обозначать с помощью логического поля.

Освоив разные способы обозначения ресурса как удаленного, можно разобрать влияние этого нового индикатора на существующие методы. Начнем с рассмотрения коллекции стандартных методов, которые нам нужно будет настроить.

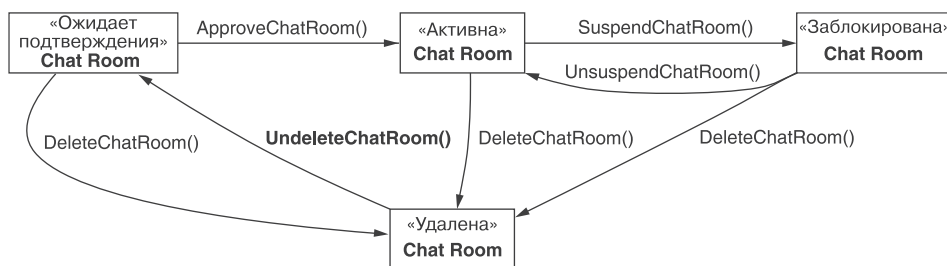


Рис. 25.4. Добавление поддержки пользовательского метода Undelete

25.3.2. Изменение стандартных методов

Теперь, когда у нас есть новый метод для отслеживания удаления ресурса (логическое поле или, возможно, поле состояния), нам нужно каким-то образом выразить это обозначение. В частности, нам потребуется сделать так, чтобы ресурсы, отмеченные как удаленные, таковыми и выглядели, но только до определенной степени. В некоторых случаях методам совсем не нужно будет изменяться. Например, поскольку индикатор `deleted` работает только для вывода и не допускает ручной установки, стандартные методы `Create` и `Update` не потребуют никаких значительных изменений. Так что начнем с рассмотрения тех, которым они все же нужны.

Метод Get

Когда дело доходит до мягкого удаления ресурсов, стандартный метод `Get` должен действовать так же, как и всегда, то есть возвращать запрошенный ресурс. Это означает, что извлечение ресурса сразу после удаления будет возвращать не ошибку `404 Not Found`, а сам этот ресурс, как если бы он не удалялся. Дальше мы увидим, что в прочих случаях есть смысл требовать дополнительных действий для доступа к мягко удаленным ресурсам. Но когда нам известен идентификатор ресурса, его извлечение по этому ID явно означает, что мы хотим узнать, был ли он мягко удален, а не то, что мы знаем о его удалении и все равно хотим просмотреть содержащиеся в нем данные.

Метод List

Стандартный метод `List` относится к тем, которые нужно изменить, чтобы они оставались полезными в контексте мягко удаленных ресурсов. В этом случае необходимо предоставить не только механизм для просмотра удаленных ресурсов, но и возможность включать в результаты мягко удаленные ресурсы, если пользователь того потребует.

Для этого нужно будет сделать две вещи. Во-первых, стандартный метод `List` должен автоматически исключать из набора результатов все мягко удаленные

ресурсы. Во-вторых, нам нужно расширить интерфейс ответа, добавив в него новое поле, позволяющее пользователю затребовать получение мягко удаленных ресурсов (листинг 25.3).

Листинг 25.3. Включение удаленных ресурсов в стандартные методы List

```
abstract class ChatRoomApi {
    @get("/chatRooms")
    ListChatRooms(req: ListChatRoomsRequest): ListChatRoomsResponse;
}

interface ListChatRoomsRequest {
    pageToken?: string;
    maxPageSize?: number;
    includeDeleted?: boolean;
    filter?: string;
}

interface ListChatRoomsResponse {
    resources: ChatRoom[];
    nextPageToken: string;
}
```

Если установлено true, то мягко удаленные ресурсы включаются в результат

Такое определение поднимает очевидный вопрос: почему бы не использовать для включения мягко удаленных ресурсов поле **filter**? На то есть несколько причин. Во-первых, нет никаких гарантий, что все стандартные методы **List** будут поддерживать фильтрацию. Когда это будет не так, нам все равно потребуется механизм для включения мягко удаленных ресурсов в результаты, и тогда придется выбирать между логическим флагом, как было показано, и фильтром, который поддерживает лишь *очень* ограниченный синтаксис для выражения заинтересованности в получении только мягко удаленных ресурсов.

Во-вторых, несмотря на тесную связь этих двух принципов, между ними есть кардинальная разница. Поле **filter** отвечает за получение набора возможных результатов и их очистку до тех, которые соответствуют заданному набору критериев. А поле **includeDeleted** отвечает за противоположное действие, а именно дополнение потенциального набора результатов еще до применения фильтра.

Эта комбинация означает, что у нас есть немного запутанный способ, позволяющий показывать не *только* мягко удаленные ресурсы, но также и то, что назначение каждого поля в API остается согласованным (листинг 25.4).

Листинг 25.4. Пример для просмотра всех мягко удаленных ресурсов

```
softDeletedChatRooms = ListChatRooms({
    filter: "deleted: true",
    includeDeleted: true,
});
```

Чтобы отобразить только мягко удаленные ресурсы, нужно очистить результаты с помощью фильтра

Чтобы добиться включения мягко удаленных ресурсов в набор результатов (а затем отфильтровать), сначала нужно обозначить это, включив их явно

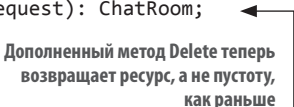
Метод Delete

Как вы уже знаете, стандартный метод **Delete** нужно расширить, чтобы теперь он не просто удалял ресурс, а отмечал его как удаленный с помощью индикатора, рассмотренного в подразделе 25.3.1. При этом он также должен возвращать сам измененный ресурс (листинг 25.5).

Листинг 25.5. Определение измененного стандартного метода Delete

```
abstract class ChatRoomApi {
    @delete("/{id=chatRooms/*}")
    DeleteChatRoom(req: DeleteChatRoomRequest): ChatRoom;
}

interface DeleteChatRoomRequest {
    id: string;
}
```



Дополненный метод Delete теперь возвращает ресурс, а не пустоту, как раньше

Но здесь возникает очередной вопрос: что конкретно должно происходить при попытке мягко удалить ресурс, который уже был мягко удален? То есть если ресурс отмечен как удаленный, должен ли метод **Delete** оставлять его в таком виде? Или же должен выбрасывать ошибку? А может, делать нечто совсем другое?

И ответ будет следовать тем же принципам из главы 7, касающимся повторного удаления: важно иметь возможность различать императивную форму (когда запрос приводит к удалению ресурса) и декларативную (когда ресурс удаляется независимо от того, стало ли это результатом текущего запроса). В связи с этим важно, чтобы новый стандартный метод **Delete** возвращал ошибку (например, 412 **Precondition Failed**), указывая, что ресурс уже был мягко удален, а значит, запрос не может быть выполнен ожидаемым образом.

Теперь, разобравшись со стандартными методами, перейдем к новым пользовательским методам, которые потребуется реализовать, и начнем с **Undelete**.

25.3.3. Метод Undelete

Одно из основных преимуществ поддержки мягкого удаления состоит в том, что метод **Delete** не удаляет данные безвозвратно. Вместо этого, отмечая ресурс как удаленный и оставляя его данные нетронутыми, мы ограждаем себя от возможных ошибочных вызовов API. Однако полноценной эта изоляция станет при условии, что у нас будет способ, позволяющий отменять наши ошибки. Иными словами, если мы удалим ресурс и позднее решим, что этого делать не стоило, то должен быть способ его восстановления. Несложно догадаться, что делать это мы будем с помощью пользовательского метода (листинг 25.6).

Листинг 25.6. Определение пользовательского метода Undelete

```

abstract class ChatRoomApi {
    @post("/{id=chatRooms/*}:undelete")
    UndeleteChatRoom(req: UndeleteChatRoomRequest): ← Подобно стандартному методу
        ChatRoom;                                     Update, мы возвращаем сам ресурс
}

interface UndeleteChatRoomRequest {
    id: string;
}

```

Пользовательский метод **Undelete** довольно прост. Получая идентификатор мягко удаленного ресурса, он снимает с него эту отметку. И здесь возникает еще один вопрос, который касается идемпотентности и повторения: что, если восстанавливаемый ресурс в данный момент не является мягко удаленным? В таком случае по аналогии с изменением стандартного метода **Delete** ответом должна быть ошибка (например, **412 Precondition Failed**). Цель здесь та же: гарантировать, что пользователи могут отличить запрос, вызывающий результат (императивный), от результата, который не обязательно вызван этим запросом (декларативный).

25.3.4. Метод Expunge

Погрузившись в тему новой функциональности мягкого удаления, мы забыли о стандартном полноценном удалении. Но наше желание защититься от собственных ошибок с помощью корзины в API еще не означает, что нам не нужна возможность фактически опустошать ее. К сожалению, мы переопределили стандартный метод **Delete** на выполнение мягкого удаления, как же теперь обрабатывать стандартное?

Восполнить этот пробел призваны два варианта: предоставить дополнительный параметр в стандартном методе **Delete** или же задействовать пользовательский метод. И хотя дополнительный параметр в стандартном запросе **Delete** может привести к меньшему количеству интерфейсов и пользовательских методов, у этого подхода есть и несколько недостатков. Первый заключается в том, что HTTP-метод **Delete** обычно не получает тело. То есть наш новый специальный параметр нужно будет предоставлять в строке запроса. В конечном итоге это означает, что поведение типичного HTTP-запроса на удаление ресурса (**DELETE /chatRooms/1234**) будет сильно отличаться от того же запроса с дополнительным параметром запроса (**DELETE /chatRooms/1234?expunge=true**). Как правило, дополнительные параметры, в частности из строки запроса, не должны кардинально менять поведение и результат запроса. В противном случае зачем вообще создавать разные методы для удаления и создания? Почему бы просто не использовать один запрос с параметром **?action=create**?

Второй недостаток подхода с дополнительным параметром в запросе `Delete` заключается в том, что из-за разрешений и контроля доступа обычно проще управлять доступом к конкретным методам API, чем контролировать один метод с разными наборами параметров. В таком случае, если мы захотим запретить пользователю окончательное удаление ресурсов, оставив возможность мягкого удаления, то нам придется разрешить доступ к стандартному методу `Delete`, но только если специальный флаг в методе будет установлен как `false`. И хотя такое возможно, реализовать это определенно будет сложнее.

По этим причинам обрабатывать полное удаление ресурсов из системы лучше с помощью отдельного метода (листинг 25.7).

Листинг 25.7. Определение пользовательского метода `Expunge`

```
abstract class ChatRoomApi {
    @post("/{id=chatRooms/*}:expunge")
    ExpungeChatRoom(req: ExpungeChatRoomRequest): void;
}

interface ExpungeChatRoomRequest {
    id: string;
}
```



Аналогично стандартному методу `Delete` `Expunge` должен возвращать пустой результат

Поскольку в случае с измененным методом `Delete` и пользовательским `Undelete` нам нужно было учитывать предварительные условия и ошибочные результаты, аналогичный вопрос следует поставить и в случае с методом `Expunge`. И в этом случае, так как результатом является полностью удаленный ресурс, вопрос прозвучит несколько иначе: можно ли вызывать этот метод для любого ресурса, независимо от того, был ли тот мягко удален? Или же нужно сначала удалить ресурс мягко и лишь затем окончательно?

И хотя могут быть причины выполнять промежуточный этап в виде мягкого удаления, в долгосрочной перспективе это только снизит быстродействие API. Иными словами, такой вариант вызовет больше неудобств, по сути, лишь превратив один запрос API в два. В итоге вызов пользовательского метода `Expunge` практически всегда должен быть возможен для любого ресурса, как мягко удаленного, так и нет.

25.3.5. Сроки хранения

Несмотря на то что мягкого и явного окончательного удаления чаще оказывается достаточно, бывают случаи, когда нам нужно, чтобы корзина очищалась регулярно. Например, когда мы удаляем сообщение в Gmail, оно отправляется в категорию «Мусор» (то есть мягко удаляется). Тем не менее спустя 30 дней в этом статусе сообщение удаляется окончательно. Этот паттерн, который мы уже видели в главе 10 с поведением LRO, не является необычным для мягко

удаленных ресурсов. Единственная сложность здесь заключается в выборе лучшего варианта реализации.

Аналогично LRO мягко удаленные ресурсы тоже должны иметь срок хранения. То есть в любом ресурсе, поддерживающем мягкое удаление, помимо описанного в подразделе 25.3.1 поля `deleted`, нам потребуется еще одно поле. Оно должно называться `expireTime` и устанавливаться сразу после удаления ресурса, отсчитывая ожидаемый срок хранения согласно predetermined политике (например, хранится 30 дней) (листинг 25.8). Если с ресурса в течение этого срока будет снята отметка об удалении, то срок хранения должен устанавливаться на явное значение `null`. То есть в случае восстановления ресурса срок его хранения будет сброшен.

Листинг 25.8. Добавление в ресурс поля со сроком хранения

```
interface ChatRoom {
    id: string;
    // ...
    deleted: boolean;
    expireTime: Date;
}
```

← Это поле отслеживает срок, когда ресурс должен быть удален окончательно

Это создает сложности с определением конечного срока хранения, устанавливаемого в методе (в данном случае в стандартном методе `Delete`), но имеет и приятный побочный эффект, в результате которого любые изменения политики по умолчанию влияют на мягко удаляемые ресурсы. К примеру, если мы изменим политику с 30 дней хранения на 45, то все мягко удаленные к этому моменту ресурсы все равно будут окончательно стерты спустя 30 дней с момента удаления, а вот для свежееудаленных уже будет действовать новый срок.

Эта политика гарантирует, что код, обрабатывающий длительность хранения ресурсов, будет прост и понятен. По сути, мы можем опереться на два условия: если ресурс обозначен как удаленный (например, `resource.deleted == true`), а истечение срока хранения установлено как «сейчас» (например, `resource.expireTime >= Date.now()`), то можно просто полностью удалить ресурс.

25.3.6. Ссылочная целостность

Еще одна тема, касающаяся мягкого удаления, связана со ссылочной целостностью. Если коротко, то нам нужно решить, могут ли другие ресурсы продолжать ссылаться на мягко удаленный ресурс. Мы уже достаточно подробно разобрали подобные детали в главе 13, сделав главный вывод о том, что в отношении ссылочной целостности разные API используют разные шаблоны. Некоторые могут ограничивать возможность нарушения ссылки. Другие могут просто каскадировать изменение (например, если связанный ресурс удален, то может удалять за собой и другие причастные). В прочих API ссылку можно оставлять как есть, подобно тикающей бомбе, ожидающей, что ею воспользуется кто-либо в будущем.

В случае мягкого удаления сценарий довольно похож. В результате правила должны оставаться прежними. То есть какой бы паттерн ни применялся стандартным методом **Delete**, изменение, вносящее мягкое удаление, не должно влиять на эту часть поведения. Поэтому если сохранение ссылочной целостности требовало предотвращения удаления, то это должно касаться и мягкого удаления, независимо от того, что технически ссылка не нарушается.

И хотя это создает некоторые сложности, когда дело касается нашей начальной цели иметь возможность отменять ошибочное удаление (поскольку теперь в некоторых случаях нельзя просто так восстановить исходную форму ресурса), дополнительная сложность ввиду изменения поведения только ради того, чтобы оно менялось при истечении срока хранения ресурса или его стирании, не стоит того.

25.3.7. Влияние на другие методы

Даже несмотря на то что мы разобрали всевозможные изменения, необходимые для стандартных методов, мы ничего не сказали о других связанных методах. Например, если мы поддерживаем пакетные методы, то пакетный метод **Delete** вызывает затруднение. Должен ли он продолжать работать в прежнем режиме, окончательно удаляя ресурсы пакетно? Или же должен по умолчанию изменяться и на мягкое удаление? И если второй вариант, то должен ли тогда также присутствовать пакетный метод **Expunge**?

Ответ здесь будет прост: да, он должен изменяться, и пакетный метод **Expunge** тоже должен присутствовать, если подобный тип поведения необходим. Как правило, когда мы изменяем поведение стандартного метода (в данном случае метода **Delete**), нужно учитывать и другие связанные методы практически так, будто они использовали эти измененные методы (листинг 25.9).

Листинг 25.9. Пример реализации пакетного метода **Delete**

```
abstract class ChatRoomApi {
    @post("/chatRooms:batchDelete")
    BatchDeleteMessages(req: BatchDeleteMessagesRequest): void {
        for (let id of req.ids) {
            ChatRoomApi.DeleteMessage({ id: id });
        }
    }
}
```

Поведение мягкого удаления пакетного
метода **Delete** будет наследовано



В итоге пакетный метод **Delete** должен определяться как стандартный **Delete**, но пакетно, в связи с чем будет наследовать новое поведение от измененного метода **Delete**, чтобы поддерживать мягкое удаление.

25.3.8. Добавление мягкого удаления для разных версий

Наконец, поскольку мы очень редко все делаем правильно с первого раза, нам нужно решить, как безопасно добавлять мягкое удаление в API, в котором его раньше не было. В главе 24 говорилось, что критически важно иметь политику определения обратно совместимых изменений, и реализация мягкого удаления в API, который до этого поддерживал лишь жесткое, явно относится к изменениям, требующим обдумывания.

Как и в случае практически всех вопросов, касающихся версионирования, ответ будет зависеть от обстоятельств. Обычно изменение поведения метода не относится к безопасной категории, но во многих случаях такой переход на мягкое удаление может оказаться вполне приемлемым. Например, если мы реализуем мягкое удаление с 24-часовым окном хранения ресурсов, то будет ли действительно важным тот факт, что ресурсы удаляются не сразу, а лишь через 24 часа? Это особенно верно, если удаляемая информация не несет особой значимости (например, не существует законов, регулирующих распоряжение такими данными).

В итоге возможность добавления мягкого удаления в существующий API не нарушающим работу образом определяется контекстом. Это решение практически всегда будет зависеть от конечного результата (например, устанавливается ли попутно срок хранения), но при этом однозначно потребует ясного понимания предназначения данных, а также ожиданий и правил, связанных с их конфиденциальностью и защитой. В целях безопасности обычно лучше предполагать, что в некотором смысле такое изменение стоит считать обратно несовместимым. Так что повышать номер старшей версии (или его эквивалент) при реализации мягкого удаления вполне приемлемо.

25.3.9. Итоговое определение API

Сведем все воедино. В листинге 25.10 показано, как можно определить в нашем API чата интерфейс для поддержки мягкого удаления.

Листинг 25.10. Итоговое определение API

```
abstract class ChatRoomApi {
    @delete("/{id=chatRooms/*}")
    DeleteChatRoom(req: DeleteChatRoomRequest): ChatRoom;

    @get("/chatRooms")
    ListChatRooms(req: ListChatRoomsRequest): ListChatRoomsResponse;
```

```

@post("/{id=chatRooms/*}:expunge")
ExpungeChatRoom(req: ExpungeChatRoomRequest): void;
}

interface ChatRoom {
  id: string;
  // ...
  deleted: boolean;
  expireTime: Date;
}

interface ListChatRoomsRequest {
  pageToken?: string;
  maxPageSize: number;
  filter?: string;
  includeDeleted?: boolean;
}

interface ListChatRoomsResponse {
  resources: ChatRoom[];
  nextPageToken: string;
}

interface ExpungeChatRoomRequest {
  id: string;
}

```

25.4. КОМПРОМИССЫ

Мягкое удаление обычно выступает как компромисс между разрешением пользователям удалять определенную информацию из API окончательно и ограждением их от собственных ошибок. В результате решение о его применении по факту зависит от контекста реализации вашего API.

Например, если сохраняемые данные неким образом регулируются, может быть намного важнее сделать так, чтобы стандартные методы действовали ожидаемым образом, удаляя информацию окончательно. В подобных случаях, чтобы предотвратить случайную потерю данных, может иметь смысл более тщательно продумать стратегию резервного копирования. В итоге пользователи, запрашивающие удаление данных, будут получать желаемое, но данные по-прежнему будут сохранены благодаря политике, которая по любым релевантным правилам наверняка окажется приемлемой.

Но возможны и обратные аргументы. Некоторые требования могут быть ориентированы на то, чтобы данные никогда окончательно не удалялись. В таких случаях нам может потребоваться избегать внедрения сроков хранения или применения пользовательского метода `Expunge`, чтобы не нарушать подобные требования.

25.5. УПРАЖНЕНИЯ

1. Когда имеет смысл обозначать мягкое удаление ресурса с помощью логического флага, а не поля состояния? Будет ли разумным использовать в одном ресурсе оба поля? Почему?
2. Как пользователи должны указывать с помощью стандартного метода `List`, что хотят увидеть *только* мягко удаленные ресурсы?
3. Что должно произойти, если пользовательский метод `Expunge` будет вызван для ресурса, который еще не был мягко удален? А если для него будет вызван пользовательский метод `Undelete`?
4. Каким должен быть разумный срок хранения мягко удаленного ресурса? Как обозначать соответствующий дедлайн?
5. В каких обстоятельствах можно рассматривать добавление поддержки мягкого удаления как обратно совместимое изменение? Когда его следует считать обратно несовместимым?

РЕЗЮМЕ

- Мягкое удаление означает возможность обозначать ресурсы как удаленные, не удаляя их фактически из хранилища API.
- Чаще всего ресурсы обозначаются как мягко удаленные с помощью логического флага `deleted`, но для этого также можно использовать уже существующее поле состояния, дополняемое новым состоянием `deleted`.
- Поведение стандартных методов относительно ресурсов с поддержкой мягкого удаления требует внесения небольших корректировок (например, стандартный метод `Get` при запросе мягко удаленного ресурса не должен возвращать ошибку `404 Not Found`).
- Мягко удаленные ресурсы можно восстанавливать и окончательно удалять с помощью пользовательских методов `Undelete` и `Expunge` соответственно.
- Какие бы рекомендации по ссылочной целостности ни были установлены для стандартного метода `Delete` (например, каскадирование удаления на ссылающиеся ресурсы), они должны применяться и к мягкому удалению ресурсов.

26

Повтор запросов

В этой главе

- ✓ Как предотвращать дублирование запросов с помощью их идентификаторов.
- ✓ Как обрабатывать коллизии ID запросов во избежание путаницы.
- ✓ Баланс между кэшированием запросов/ответов и их согласованностью.

В мире, где мы не можем гарантировать желаемое завершение всех запросов и ответов, неизбежно возникает потребность в их повторении. Это не проблема для идемпотентных методов API, но нам понадобится способ, позволяющий безопасно повторять запросы, избегая повторения работы. Текущий паттерн представляет механизм безопасного повтора запросов в случае неудачи, независимо от идемпотентности метода.

26.1. ЗАЧЕМ ЭТО НУЖНО

Мы живем в очень нестабильном мире, и это особенно касается всего, что связано с удаленными сетями. Учитывая чрезвычайную сложность современных сетей и разнообразие доступных систем передачи данных, можно в некотором смысле счесть чудом, что мы вообще можем устойчиво передавать сообщения.

К сожалению, эта проблема с надежностью распространяется и на веб-API. Более того, поскольку API чаще используются с помощью беспроводных сетей на все более мелких устройствах, отправка запросов и получение ответов на огромных расстояниях заставляет уделять надежности гораздо больше внимания, чем в тех же проводных локальных сетях.

И такая характерная ненадежность неизбежно ведет к сценарию, когда отправленный клиентом запрос не всегда достигает сервера. И даже если достигает, отправленный сервером ответ может не достичь клиента. Когда это происходит с идемпотентными методами (например, теми, которые ничего не меняют, как стандартный метод `Get`), то особой проблемы не возникает. Если мы не получим ответ, то всегда можно просто повторить запрос. Но что, если запрос окажется неидемпотентным? Чтобы ответить на этот вопрос, нужно взглянуть на ситуацию более пристально.

Как правило, если при отправке запроса мы не получаем ответ, то возникают два варианта. В первом, показанном на рис. 26.1, запрос не достигает сервера API, следовательно, нам не возвращается и ответ. Во втором же, представленном на рис. 26.2, запрос достигает сервера, но ответ не достигает клиента.

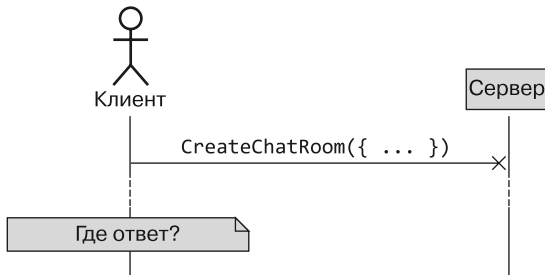


Рис. 26.1. Запрос отправлен, но не получен

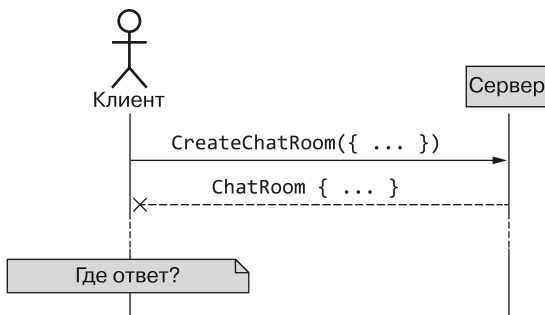


Рис. 26.2. Ответ отправлен, но не получен

И хотя оба сценария неудачны, первый представляет абсолютно поправимую ситуацию. В этом случае можно просто повторить запрос. Поскольку он так и не был получен, то все выглядит так, будто его и не отправляли. А вот второй сценарий намного хуже. Здесь наш запрос был получен сервером, обработан, и на него был отправлен ответ — только он не достиг цели.

Однако самая серьезная проблема заключается в том, что мы не можем различить два этих сценария. Независимо от того, возникла проблема при доставке запроса или возвращении ответа, клиенту известно одно: должен был вернуться ответ, который так и не поступил. В результате нам нужно готовиться к худшему, то есть предполагать, что запрос был доставлен, а ответ нет. Как же быть?

В этом и состоит основная задача текущего паттерна: определить механизм, с помощью которого можно предотвратить повторные запросы в API, в частности для неидемпотентных методов. Иными словами, у нас должна быть возможность безопасно повторять выполнение методов (даже тех, которые запускают ракеты), не зная о том, был ли получен запрос, и не беспокоясь о двойном выполнении метода в результате этого повтора.

26.2. ОБЗОР

Этот паттерн строится на идее предоставления каждому запросу уникального идентификатора, позволяющего исключить его повторное выполнение. С помощью этого идентификатора мы сможем ясно видеть, обрабатывался ли уже входящий запрос сервисом API, и если да, то можно ли избежать повторения данной операции.

Но здесь возникает непростой вопрос: что должно возвращаться в качестве результата при перехвате повторного запроса? Несмотря на то что будет достаточно вернуть ошибку, указывающую, что этот запрос уже обработан, на практике такое решение окажется не слишком полезным. Например, если мы отправили запрос на создание ресурса, но ответ так и не поступил, то при совершении очередного запроса мы будем рассчитывать получить в ответ нечто полезное, в частности уникальный ID созданного ресурса. Ошибка в таком случае предотвратит повтор, но это не совсем тот результат, который нам нужен.

Чтобы обработать подобный случай, можно кэшировать сообщение ответа, сопровождавшее представленный ID запроса, — это позволит при обнаружении повтора вернуть данный ответ так, будто он вернулся при первом выполнении запроса. Пример этого потока действий с использованием ID запросов показан на рис. 26.3.

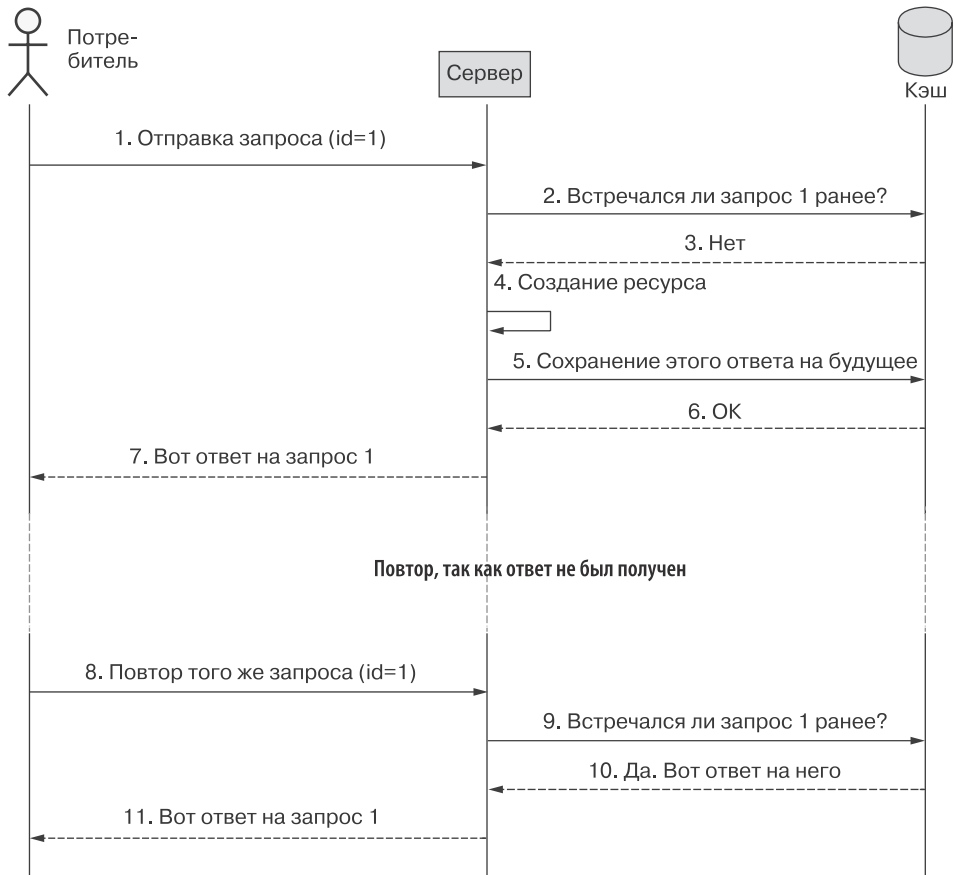


Рис. 26.3. Схема очередности событий по исключению повторных запросов

Теперь, когда мы имеем общее представление о работе этого простого паттерна, пора переходить к деталям его реализации.

26.3. РЕАЛИЗАЦИЯ

Первым делом воплощение этого паттерна потребует внедрения поля идентификатора запроса, чтобы можно было определять получение и обработку запроса в API. Это поле будет представлено во всех интерфейсах запросов, отправляемых клиентами для любых методов, требующих отсутствия повторов (в данном случае это будут просто неидемпотентные методы). Начнем с разбора этого поля и значений, которые в итоге будут в нем храниться.

26.3.1. Идентификатор запроса

Идентификатор запроса — это не что иное, как стандартное поле ID. Тем не менее, вместо того чтобы размещаться в интерфейсе ресурса, это поле помещается в определенных интерфейсах запроса (листинг 26.1). Аналогично тому, как поле ID в ресурсе уникально идентифицирует его по всему API, идентификатор запроса преследует ту же цель. Подвох здесь в том, что, несмотря на неизменность идентификаторов ресурсов (глава 6), идентификаторы запросов больше подобны одноразовым значениям. И поскольку с их помощью клиент идентифицирует свои исходящие запросы, очень важно, чтобы выбирались они именно им.

Листинг 26.1. Определение запроса с полем идентификатора

```
interface CreateChatRoomRequest {
  requestId?: string;
  resource: ChatRoom;
}
```

← Идентификатор запроса определяется
в качестве необязательной строки
в интерфейсе запроса

К сожалению, как мы узнали в главе 6, обычно нежелательно позволять клиентам выбирать идентификаторы, так как этот выбор чаще неудачный. Однако нам это необходимо, и лучшее, что можно сделать, — это установить формат, рекомендуемый случайный выбор идентификаторов, и настоять на следовании этому формату (к примеру, нам нужно, чтобы кто-то отправлял запрос с ID, установленным на 1 или abcd). Идентификаторы запросов должны следовать тем же стандартам, что и идентификаторы ресурсов (рассмотренные в главе 6).

И хотя ID запросов не являются такими же постоянными, это не делает их менее важными. В итоге, если сервер API получит запрос с недействительным ID запроса, то он должен отклонить его и выбросить ошибку (например, **400 Bad Request**). Обратите внимание: это не то же самое, что оставить идентификатор ресурса пустым или отсутствующим. В подобном случае мы должны будем продолжать так, будто запрос никогда не будет повторен, минуя все кэширование, рассмотренное в разделе 26.2, и действуя аналогично любому другому методу API.

Избежание производных идентификаторов

Довольно часто здесь возникает вопрос: почему бы просто не вывести идентификатор из самого ресурса? На первый взгляд это решение кажется элегантным: выполнить определенное хеширование тела запроса и сделать так, чтобы при каждой встрече этого хеша было понятно, что повторно обрабатывать запрос не нужно.

Проблема же в отсутствии гарантии, что нам вдруг не потребуется выполнить один запрос дважды. И вместо явного объявления от клиента («Я точно хочу обеспечить, чтобы этот запрос обрабатывался лишь единожды») мы неявно определяем предполагаемое поведение, не предоставляя четкого способа,

позволяющего отказаться от этой дедупликации. Очевидно, что пользователь может предоставить дополнительные заголовки или бессмысленные параметры строки запроса, чтобы запрос выглядел иначе, но это изменит предустановленное поведение.

По умолчанию запросы не должны дедуплицироваться, поскольку мы не можем знать наверняка намерение пользователя. И безопасным способом позволить пользователю явно выражать смысл своего запроса будет принять выбранный им идентификатор, а не выводить его из содержимого запроса.

Наконец, учитывая информацию из подраздела 26.3.5 (к которому мы перейдем несколько позже), в конечном счете не бывает истинных случаев, когда работа совсем не повторяется. Поскольку значения кэша практически всегда в определенный момент перестают действовать, это решение будет работать скорее как механизм временного ограничения, гарантирующий, что один и тот же запрос не будет повторен в течение какого-то периода.

И пока мы говорим о кэшировании, предлагаю коротко обсудить, что именно будет кэшироваться, и разобрать стоящие за этими решениями доводы.

26.3.2. Кэширование ответа

На рис. 26.4 видно, что для каждого запроса с идентификатором в определенном месте кэша будет храниться результат. Этот процесс начинается с проверки кэша предоставленного ID запроса, когда при наличии его значения возвращается результат, кэшированный при первой фактической обработке запроса сервером. Если же в кэше значение ID не обнаруживается, то мы обрабатываем запрос и, прежде чем отправлять ответ пользователю, обновляем кэш с помощью полученного результата.

Беспокойство в этом паттерне может вызывать тот факт, что в кэше сохраняется все значение ответа. В таком случае требования к объему хранилища могут очень быстро выйти из-под контроля. Ведь технически мы кэшируем результаты каждого неидемпотентного вызова API. А что, если эти ответы сами по себе окажутся довольно большими? Например, представьте пакетный метод `Create`, создающий 100 ресурсов. Если мы получим такой запрос и используем описанный паттерн для его дедупликации, то нам потребуется кэшировать много данных, а ведь это лишь один запрос.

В конечном итоге все грамотные инженеры сходятся в одном: несмотря на возможность спроектировать более сложную систему, которая без кэширования будет повторно вычислять ответ, не повторяя обработку запроса, это все равно окажется не лучшим решением. Дело в том, что временные затраты инженера на отладку сложной системы и управление ею существенно перевешивают

затраты на увеличение ОЗУ или кластера кэширования в облаке. В результате кэширование ответов — просто беспроигрышный вариант в долгосрочной перспективе, поскольку всегда можно довольно быстро добавить дополнительное аппаратное обеспечение, чтобы решить проблему, а вот добавить ментальные ресурсы будет куда сложнее.

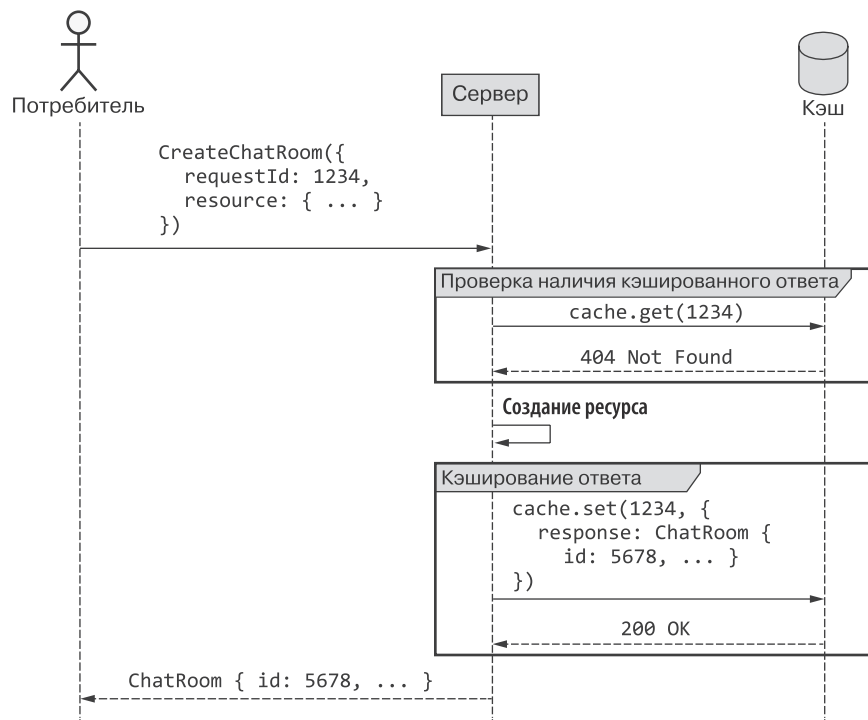


Рис. 26.4. Последовательность кэширования ответов для недублирующихся запросов

26.3.3. Согласованность

В этом требовании кэшировать ответы на обработанные запросы скрывается ключевой вопрос согласованности. Иными словами, кэшированные значения очень легко могут устареть, поскольку данные со временем обновляются и это обновление не отражается в кэше. Отсюда возникает вопрос, насколько это важно в случае дедупликации запросов.

В качестве наглядного примера представим, что два клиента (А и Б) выполняют обновление ресурса, как показано на рис. 26.5. Вдобавок предположим, что в этом случае конфликтов нет. Вместо этого нас беспокоит тот факт, что у клиента А слабое соединение, в связи с чем ему иногда требуется повторять запросы.

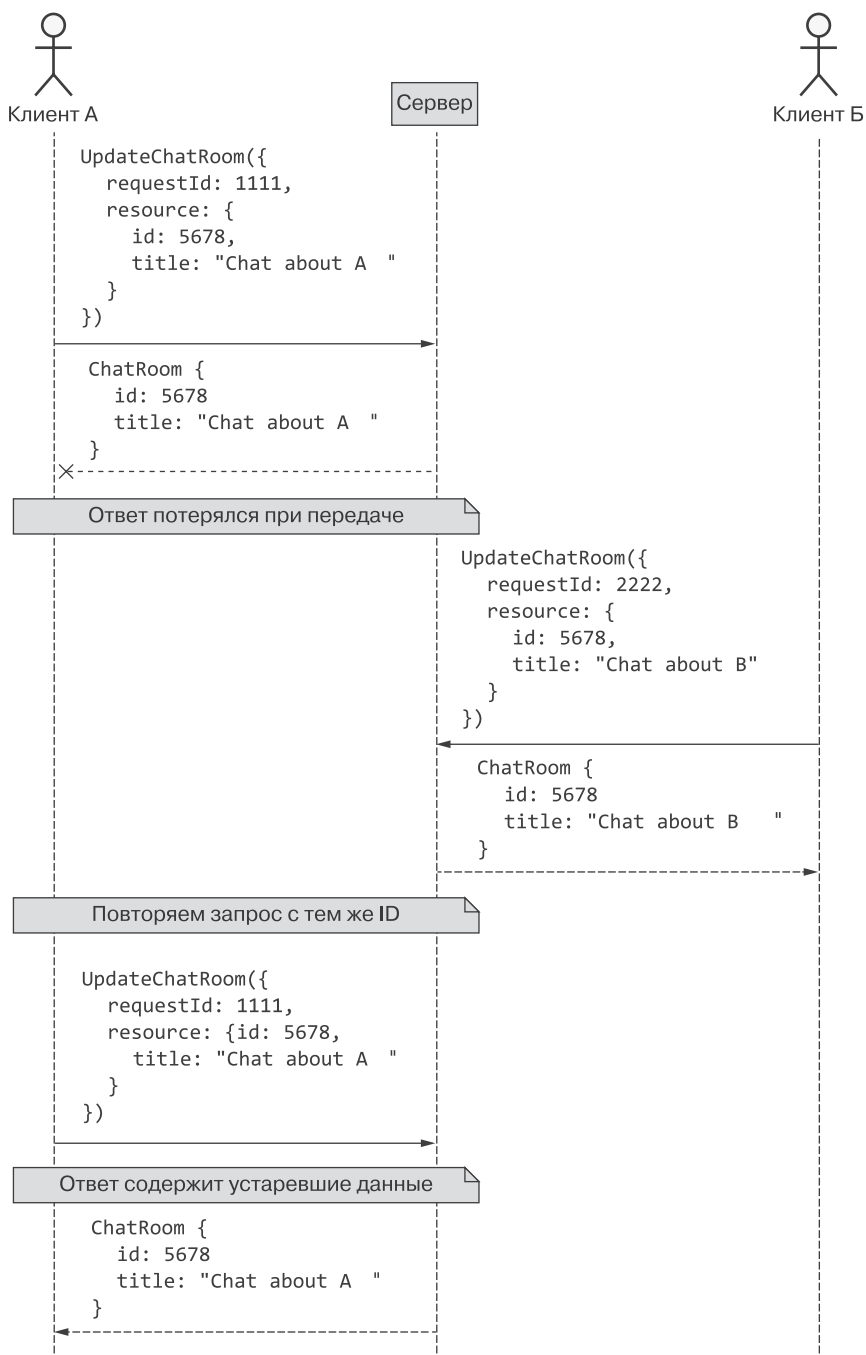


Рис. 26.5. При повторе запросов можно столкнуться с устаревшими данными

В данном случае, несмотря на то что клиенты А и Б обновляют ресурс без каких-либо конфликтов, ответ от обновления А теряется при передаче. В результате А просто решает повторить запрос, используя тот же идентификатор, чтобы вносимые изменения повторились дважды. На эту вторую попытку сервер отвечает кэшированным значением ресурса, показывая А значение `title`, которое устанавливалось в запросе изначально. Но есть одна проблема: сейчас данные выглядят иначе.

Зачастую устаревшие или несогласующиеся данные, как в этом случае, могут стать причиной серьезных проблем, но здесь важно вспомнить основную задачу паттерна: позволить клиентам повторять запросы, не вызывая потенциально опасного повторения работы. И важнейший компонент этой задачи — сделать так, чтобы клиент получал в ответе тот же результат, какой он получил бы в случае успеха при первой попытке. В результате несмотря на то, что эта последовательность событий на рис. 26.5 может выглядеть несколько странной, она полностью разумна и верна. В действительности даже будет более странным и неожиданным вернуть другой результат. То есть кэш совсем не обязан поддерживаться актуальным по мере изменения соответствующих ему данных, поскольку это приведет к еще большей путанице, чем устаревшие данные.

26.3.4. Коллизии ID запросов

Когда выбор идентификаторов поручается самим пользователям, это неизбежно приводит к сложностям. И одним из типичных результатов неудачного выбора ID становится их коллизия, когда идентификатор оказывается вовсе не случайным и уже используется кем-то другим. В нашем дизайне это может привести к каверзным ситуациям.

Например, представьте двух пользователей (А и Б), взаимодействующих с нашим API. Соединение между ними отличное, устойчивое, но они ужасно плохо справляются с выбором ID для запросов. Как видно на рис. 26.6, в итоге каждый выбирает одинаковый идентификатор, что ведет к печальным и очень странным последствиям.

Похоже, что клиент Б каким-то образом, пытаясь создать ресурс `User`, получил вместо него ресурс `ChatRoom`. И причина тому проста: поскольку в ID запроса произошла коллизия, сервер API ее заметил и вернул ранее кэшированный результат. В этом случае оказывается, что ответ полностью лишен смысла и никак не связан с действиями клиента Б. Это весьма неприятный сценарий, но что тут можно сделать?

Верным решением будет убедиться, что сам запрос не менялся с того момента, когда мы последний раз видели этот идентификатор. Иными словами, запрос клиента Б должен иметь то же тело — в противном случае необходимо выбросить ошибку и сообщить об этом клиенту. Чтобы это сработало, нам нужно разобраться с двумя моментами, показанными выше на рис. 26.2.

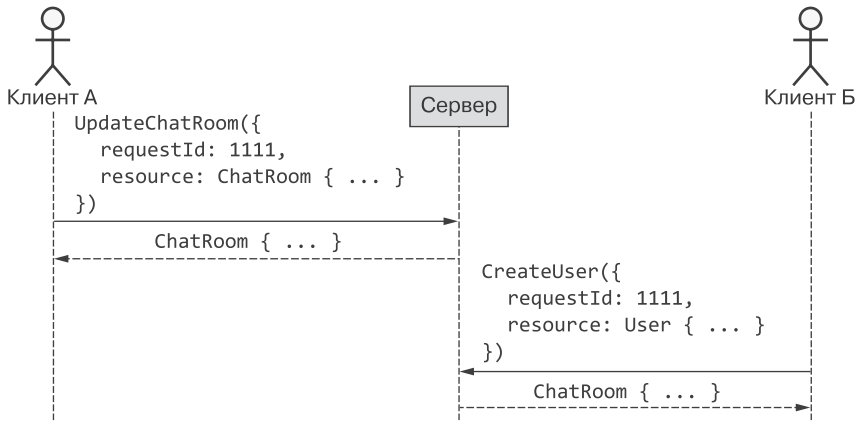


Рис. 26.6. Путаница в результатах из-за коллизии идентификаторов запросов

Во-первых, когда мы после выполнения запроса кэшируем значение ответа, который еще не кэшировался, нам также нужно сохранить уникальный отпечаток в самом теле запроса (например, хеш SHA-256 запроса). Во-вторых, когда мы ищем запрос и находим совпадение, нам нужно убедиться, что тело запроса в этом (потенциально) дубликате совпадает с предыдущим. Если это так, то можно безопасно вернуть кэшированный ответ. Если же нет, то нужно вернуть ошибку, в идеале что-то наподобие `409 Conflict` (листинг 26.2).

Листинг 26.2. Пример метода для обновления ресурса с идентификаторами запросов

```

function UpdateChatRoom(req: UpdateChatRoomRequest): ChatRoom {
  if (req.requestId === undefined) {
    return ChatRoom.update(...);
  }
  const hash = crypto.createHash('sha256')
    .update(JSON.stringify(req))
    .digest('hex');
  const cachedResult = cache.get(req.requestId);
  if (!cachedResult) {
    const response = ChatRoom.update(...);
    cache.set(req.requestId, { response, hash });
    return response;
  }

  if (hash == cachedResult.hash) {
    return cachedResult.response;
  } else {
    throw new Error('409 Conflict');
  }
}
  
```

Если ID запроса не предоставлен, то нужно просто выполнить обновление и вернуть результат

Если кэшированного значения для данного ID запроса нет, то следует обновить этот ресурс

Нужно обновить кэш значением ответа и хеша

Если хеш совпадает, то можно вернуть ответ

Если хеш не совпадает, то следует выбросить ошибку

Используя этот простой алгоритм, при каждой встрече ID уже обработанного запроса можно перепроверить, действительно ли он является дубликатом этого

запроса, а не коллизией. Это позволяет безопасно возвращать ответ в типичных, к сожалению, случаях, когда пользователи безответственно подходят к выбору идентификаторов запросов.

26.3.5. Срок действия кэша

Теперь, когда мы разобрали все самое сложное в этом паттерне, можно переходить к заключительной теме: продолжительности хранения данных в кэше. Очевидно, что в идеальном мире мы могли бы хранить все данные вечно и в случае повтора запроса пользователем, даже спустя несколько лет с момента его первого выполнения, имели бы возможность вернуть кэшированный результат. К сожалению, хранение данных стоит денег, в связи с чем является в API ограниченной роскошью. В конечном итоге это означает, что нам нужно решать, как долго хранить содержимое в кэше запросов.

Чаще всего запросы повторяют в скором времени после сбоя, обычно в течение нескольких секунд после первой попытки. В итоге хорошей отправной точкой для политики хранения кэша будет примерно пять минут, но при каждом повторе запроса этот таймер нужно будет перезапускать. Если сбой повторится в дальнейшем, то нам нужно применять к последующим повторам те же условия хранения, что и к первым.

И у нас есть возможность облегчить это решение. Поскольку кэширование по своей природе со временем приводит к устареванию данных, то что бы мы ни решили, это всегда можно настроить позднее, сопоставив возможности хранилища с объемом трафика. То есть хоть пять минут и будут хорошей стартовой точкой, это решение очень легко переосмыслить и скорректировать на основе пользовательского поведения, доступной памяти и затрат.

26.3.6. Итоговое определение API

Итоговое определение API для данного паттерна будет довольно простым, заключааясь лишь в добавлении поля (листинг 26.3). Полный пример последовательности действий показан на рис. 26.7.

Листинг 26.3. Итоговое определение API

```
abstract class ChatRoomApi {
    @post("/chatRooms")
    CreateChatRoom(req: CreateChatRoomRequest): ChatRoom;
}

interface CreateChatRoomRequest {
    resource: ChatRoom;
    requestId?:string
}
```

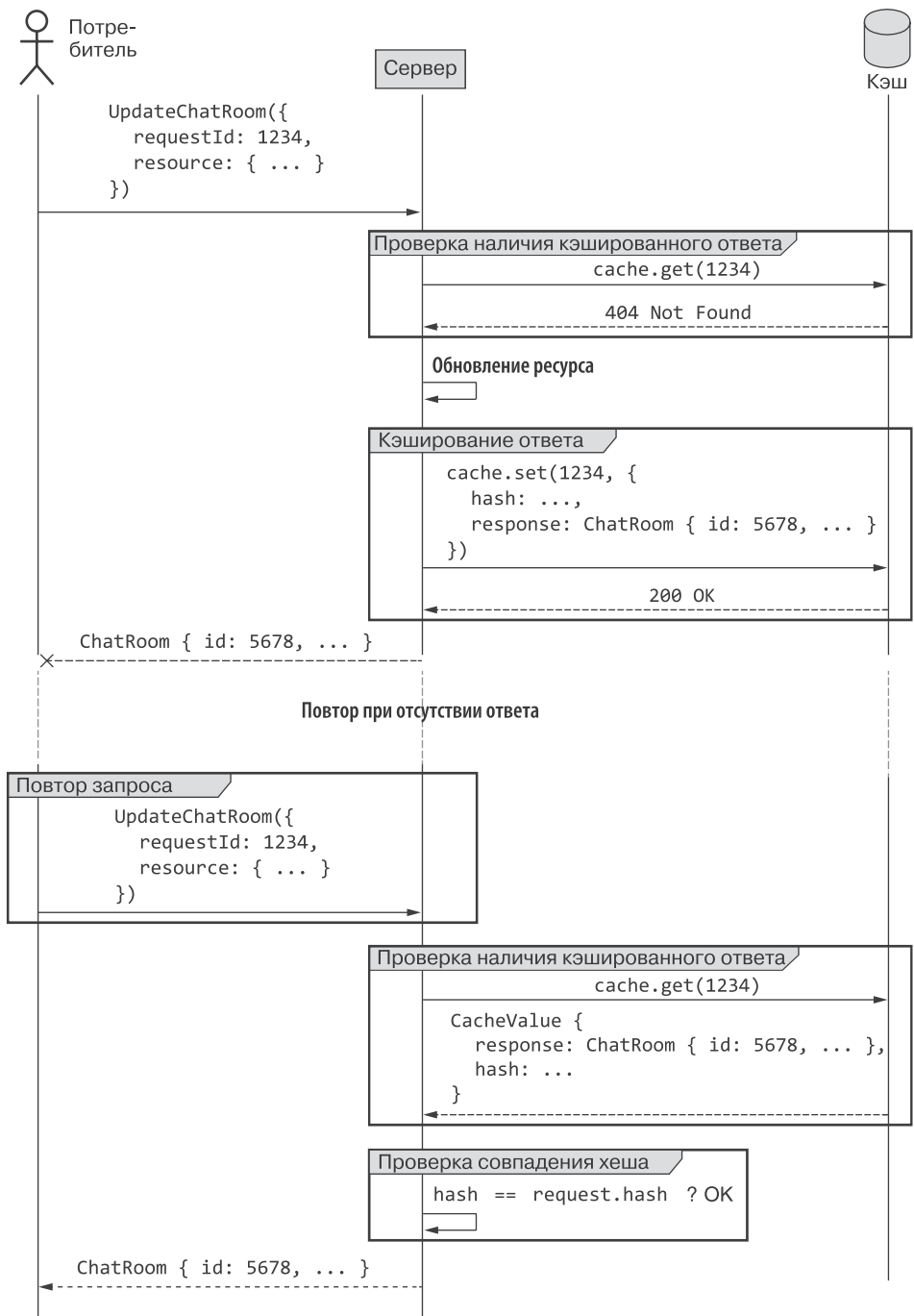


Рис. 26.7. Последовательность событий при дедупликации запроса

26.4. КОМПРОМИССЫ

В данном паттерне мы используем простой уникальный идентификатор, чтобы избежать повторения работы в API. В итоге это приводит к компромиссу между возможностью безопасного повторения неидемпотентных запросов и сложностью методов API (а также дополнительными требованиями к памяти для кэширования). И хотя некоторые API могут вообще не беспокоиться о сценариях дублирования запросов, в прочих случаях этот нюанс может оказаться критически важен — настолько, что идентификаторы запросов станут не факультативными, а обязательными.

В конечном счете решение о том, стоит ли усложнять определенные методы, используя этот механизм, зависит от конкретного сценария. Как правило, разумно добавлять его по мере необходимости, начиная с особо чувствительных методов API и уже со временем охватывая другие.

26.5. УПРАЖНЕНИЯ

1. Почему нежелательно использовать отпечаток (например, хеш) запроса для определения того, не является ли он дубликатом?
2. Почему нежелательно стараться поддерживать кэш запроса-ответа актуальным? Должны ли кэшированные ответы становиться недействительными или им следует обновляться параллельно с изменением данных соответствующего им ресурса?
3. Что произойдет, если проверяющая дубликаты система кэширования выйдет из строя? Каким будет сценарий сбоя? Как от этого нужно защищаться?
4. Почему важно использовать отпечаток запроса, когда его ID уже кэширован? Какой атрибут генерации ID запроса обуславливает это требование?

РЕЗЮМЕ

- Запросы могут в любой момент завершиться сбоем, и если у пользователей не будет подтвержденного ответа от API, то им не удастся убедиться в успешной обработке запроса.
- Идентификаторы запросов генерируются клиентом API и действуют как способ дедупликации отдельных запросов, полученных сервисом API ранее.
- Для большинства запросов можно кэшировать ответ на основе идентификатора запроса, но при этом параметры срока хранения и аннулирования должны быть тщательно подобраны.
- Сервисы API наряду с ID запроса также должны проверять отпечаток содержимого запроса, чтобы избежать странного поведения в случае коллизий ID запросов.

27

Валидация запросов

В этой главе

- ✓ Почему некоторые методы API должны поддерживать предварительную проверку запросов.
- ✓ Как обрабатывать взаимодействие с внешними зависимостями, предназначенное для проверки запросов.
- ✓ Как обрабатывать особые побочные эффекты методов в проверочных запросах.

Иногда API могут оказываться настолько запутанными, что становится неясно, каким будет результат конкретного вызова API. При этом для безопасных методов у нас есть простое решение: просто попробовать. В случае же небезопасных такое решение, естественно, не подойдет. В текущем паттерне мы познакомимся с особым полем `validate only`, которое можно добавлять в интерфейсы запроса и которое будет выступать в качестве механизма, позволяющего предсказать следствие выполнения запроса, фактически не выполняя его.

27.1. ЗАЧЕМ ЭТО НУЖНО

Даже самые простые API могут вводить в заблуждение — в конце концов они представляют собой сложный принцип (иначе бы для их освоения не понадобилась книга). И сколько бы мы ни перечитывали документацию, ни перепроверяли

исходный код, ни изучали предоставленные нам разрешения и ни рассматривали запросы, которые хотим отправить в API, очень редко все удастся сделать правильно с первой попытки. Возможно, это удастся со второй. А может, и с третьей. Суть в том, что попыток будет много, и вполне нормально, если некоторые окажутся неудачными или же увенчаются успехом, но он будет неправильным.

И хотя на первых порах этот метод проб и ошибок работает неплохо, он может вызвать серьезные проблемы, когда дело доходит до промышленных систем. Это можно сравнить с тем, что большинство людей в случае поломки авто предпочитают чинить его в сервисе: они могли бы попробовать и сами, но тогда возникает риск сломать что-то еще. И речь не об игрушечной машинке, а о критически важном инструменте, который доставляет нас на работу, в школу, магазины и обратно.

Поначалу этой опасности можно избегать, экспериментируя только с безопасными методами, которые не меняют никакие данные и не имеют пугающих побочных эффектов. Но как быть с другими? Например, мы наверняка не захотим экспериментировать с методом `DeleteAllDataAndExplode()`. Не то чтобы у нас нет потребности в экспериментах с ним, просто нет удачного механизма.

И это критически важно, когда мы понимаем, насколько часто человеческая ошибка может закрасться в проверку даже чего-то такого простого, как наличие доступа к выполнению определенного метода API. Рассмотрим в качестве примера случай, когда нам нужно проверить, есть ли у нас возможность вызвать метод `DeleteAllDataAndExplode()`. Как показано на рис. 27.1, можно просмотреть необходимые разрешения в документации, затем проверить, какую учетную запись мы используем при отправке вызова API, после чего уточнить в системе, есть ли у этой учетной записи разрешения, указанные в документации. Но что, если в цепочке зависимостей присутствует ошибка?

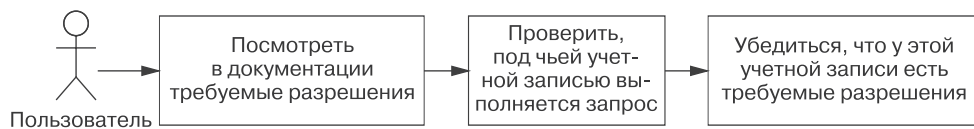


Рис. 27.1. Запросы могут быть подвержены фактору человеческой ошибки в любых проверках системы

В конечном итоге люди совершают ошибки в то время, как компьютеры делают *в точности то*, что указано в инструкции (в том числе когда мы подразумеваем в ней нечто иное). Поэтому даже при 99 % уверенности, что все пройдет гладко, мы не можем знать наверняка, пока не попробуем. Необходим более подходящий способ экспериментирования с методами API — даже с опасными.

27.2. ОБЗОР

Поскольку наша цель — позволить пользователям получать превью ответов на их вызовы API, можно реализовать это с помощью одного поля, указывающего, что запрос нужно рассматривать как «только для валидации» (листинг 27.1). Когда метод API увидит, что запрос нужно проверить, а не выполнить, он сделает столько фактической работы, сколько необходимо, не причиняя вреда внутренней системе. В некотором смысле такие запросы должны вести к выполнению работы внутри самой транзакции, которая в итоге автоматически отменяется и никогда не отправляется.

Листинг 27.1. Пример стандартного метода Create с поддержкой валидации запросов

```
interface CreateChatRoomRequest {
  resource: ChatRoom;
  validateOnly?: boolean;
}
```

← Чтобы указать, что запрос предназначен только для проверки, мы используем простой логический флаг

Причина в том, что пользователи должны получать как можно больше валидации. То есть запросы должны подвергаться проверке на наличие разрешения выполнить соответствующие действия, на конфликты с существующими данными (например, требование к уникальности поля может быть не удовлетворено), на ссылочную целостность (возможно, запрос относится к другому ресурсу, который не существует), а также любой иной форме серверной валидации. И если запрос при фактическом выполнении выбросит ошибку, то он должен выбросить ее и при валидации.

Но несмотря на всю похвальность подобного подхода, он не всегда оказывается возможным. Это может быть следствием множества разных причин (например, связи с внешними сервисами или прочих зависимостей, о чем будет сказано далее в подразделе 27.3.1), но цель остается той же: постараться сделать ответы валидации максимально приближенными к реальным. В следующем разделе мы изучим все детали правильной обработки запросов только для валидации, а также разберем различные каверзные сценарии.

27.3. РЕАЛИЗАЦИЯ

Как мы видели в листинге 27.1, можно разрешить пользователю обозначать запросы как «только для валидации» с помощью установки флага в поле `validateOnly` на явное значение `true`. В противном случае по умолчанию запрос будет рассматриваться как обычный. Такое предустановленное поведение важно: если мы сделаем наоборот (всегда по умолчанию будем только проверять запросы), то просто не позволим всем запросам базово выполнять

свою работу. В этом случае, чтобы получить подобие нормального поведения, нам всегда придется устанавливать флаг, а это окажется большой ошибкой в любом API.

Поскольку наша цель — предоставить реалистичное превью ответа на запрос, в его проверочной форме необходимо стремиться проверить как можно больше аспектов, что может подразумевать связь с удаленными сервисами (только без фактического изменения данных). Например, как показано на рис. 27.2, можно связаться с сервисом контроля доступа для определения того, имеет ли пользователь доступ к выполнению запрошенного действия. Если нас беспокоит ссылочная целостность, то можно проверять, существует ли ресурс. Кроме того, можно проверять параметры, например, убедиться, что строковый SQL-запрос представлен в допустимом формате.

Помимо этого, ответ на запрос должен отражать то же, что отражал бы ответ на стандартный запрос. То есть если запрос будет по какой-то причине вызывать ошибку, нужно возвращать эту ошибку так же, как в любом другом случае.

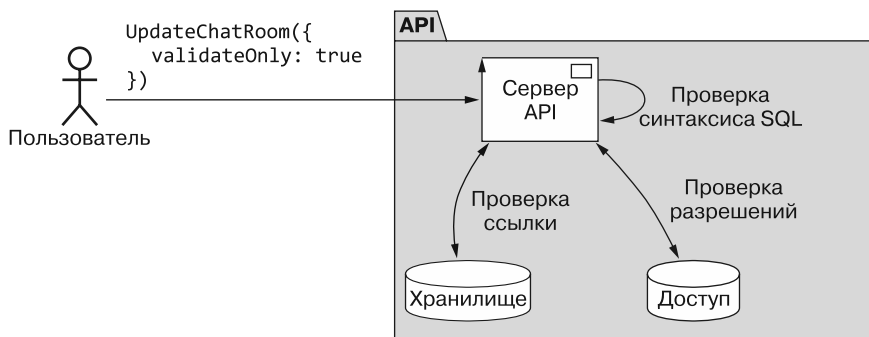


Рис. 27.2. Проверочные запросы могут продолжать связываться с прочими внутренними компонентами

Например, если у нас нет доступа к выполнению запроса, то мы можем получить HTTP-ошибку **403 Forbidden**.

С другой стороны, если запрос окажется успешным, то ответ должен быть максимально близким к реальному. Это означает, что какие бы поля ни могли быть заполнены в реальном запросе, они должны быть заполнены и в проверочном. При этом нужно учитывать, что некоторые поля не смогут заполниться (например, генерируемый сервером идентификатор) и должны оставаться пустыми или заполняться реалистичными (но точно не валидными) значениями. Однако в конечном итоге поля результата в первую очередь должны подтверждать, что запрос выполняется успешно, поэтому не должно быть жестких требований по возврату всей информации. Кроме того, должно быть

понятно, что абсолютно разумно поддерживать проверочные запросы только в определенных методах — в конце концов, в некоторых это просто может оказаться бессмысленным.

Самое же важное во всем этом то, что любой проверочный запрос должен приводить к полностью безопасному и идиempотентному методу. Это значит, никакие данные не должны изменяться, никакие побочные эффекты не должны проявляться, и все запросы, отмеченные как `validateOnly`, должны предполагать возможность их многократного перезапуска с получением одного и того же результата (при условии, что в системе не происходит никаких других изменений).

Это может внести путаницу, когда у нас есть метод API только для чтения, но при этом он не обязательно бесплатный. Представим метод для запроса большого объема данных. Выполнение SQL-запроса огромного набора данных технически их не меняет, но определенно может стоить много денег. И в некоторых ситуациях эта стоимость напрямую связана с количеством соответствующих запросу данных. В этом случае, несмотря на кажущуюся ненужность параметра `validateOnly`, в действительности он окажется очень важным. Иначе как пользователь может узнать наверняка, содержит ли его запрос только валидный синтаксис? Мы не можем проверить все аспекты запроса, но можем хотя бы вернуть ошибку, если он окажется недействительным.

Учитывая все вышесказанное, печальная правда состоит в том, что некоторые аспекты запроса будут вынуждать нас нарушать эти правила, требующие стремления к безопасности и идиempотентности. В связи с этим возникает серьезный вопрос: что делать, когда обеспечение абсолютной точности поведения в результатах вызова приводит к нарушению требования, скажем, идиempотентности метода? В следующем подразделе мы разберем внешние зависимости и узнаем, как работать с ними в контексте валидации запросов.

27.3.1. Внешние зависимости

Одним из очевидных мест, создающих проблемы при валидации запросов, являются зависимости от внешних сервисов или библиотек. И причина тому проста: мы не контролируем эти внешние компоненты, поэтому не можем и объяснить, что связываемся с ними исключительно ради валидации. Например, если наш API отправляет электронные письма при каждом обновлении ресурса `ChatRoom`, то можно проверить, есть ли у нас возможность отправлять их конкретному получателю, послав ему тестовое письмо, как показано на рис. 27.3. Но это окажется существенным побочным эффектом запроса. В результате нам нужно выбирать между исключением проверки определенных аспектов и нарушением собственных правил касательно побочных эффектов, безопасности и идиempотентности.

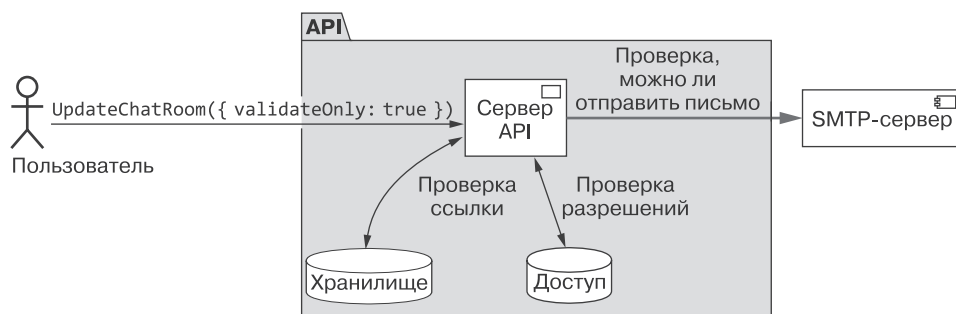


Рис. 27.3. Внешние зависимости усложняют проверку запросов или делают ее невозможной

К счастью, выбор прост: соблюдать правила. Если внешняя зависимость поддерживает подобные проверочные запросы, то можно спокойно положиться на предоставленную ими информацию для проверки вызываемого сервиса. Однако в случае недоступности подобного варианта такую зависимость нужно полностью пропустить. Наличие безопасной, идемпотентной и лишенной побочных эффектов проверки имеет куда большее значение, чем валидация всех мелочей.

Это может означать, что придется дополнительно тратить время на выполнение прочих связанных операций проверки в ситуациях, когда внешний ресурс явно не будет поддерживать необходимый случай использования. Например, несмотря на невозможность протестировать отправку письма с помощью внешнего сервиса, мы наверняка можем выполнить некую проверку формата адреса, по крайней мере исключив очевидно недействительные данные (например, письма без символа @). Определенно, данная альтернатива неидеальна, но все же это лучше, чем полный отказ от проверки.

27.3.2. Особые побочные эффекты

Даже когда мы контролируем всю систему и не имеем внешних зависимостей, все равно остается вероятность, что предоставление полезного преью на проверочный запрос вызовет путаницу. Рассмотрим пример реализации в комнате чата особой функции лотереи с методом, который возвращает победный результат случайно (рис. 27.4). При условии валидности проверочного запроса к этому методу каким должен быть ответ? Или вдруг у нас будет не что-то случайное, а более конкретное, но все равно зависящее от поведения других в комнате чата? Например, вместо случайного выбора победителя это будет своеобразная радиовикторина, где каждый 100-й вызов метода API возвращает особый ответ. Изменит ли это наше представление о возможном результате?

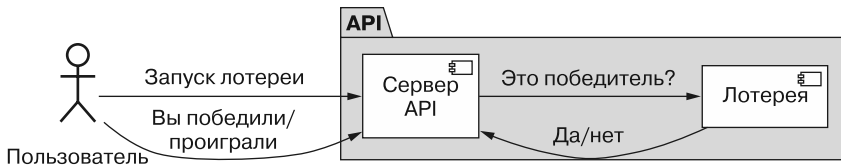


Рис. 27.4. Пример лотерейных запросов с внутренними случайными зависимостями

Поскольку неизбежно будут случаи, когда нельзя опустить информацию и необходимо сделать выбор (например, логическое поле, сообщающее о выигрыше), то важно помнить цели этого способа валидации. Во-первых, нам нужно проверить запрос и вернуть любые потенциальные ошибки, которые могут возникнуть. Во-вторых, при условии валидности запроса нужно вернуть правдоподобный результат, который не обязательно окажется реален, но обязательно реальность отразит.

Если следовать этим принципам, то ответы на ранее поставленные вопросы окажутся намного проще: мы должны спокойно относиться к возвращению в одних случаях победного результата, а в других — проигрышного. Процент этих побед не обязательно должен отражать истинные шансы лотереи при условии, что результат будет таким, какой мог вернуться при реальном запросе. То же касается и более определенного сценария с 100-м вызовом: иногда мы можем победить, а иногда нет, поэтому значение ответа может отражать любой из этих вариантов, ведь они оба представляют реальную возможность. Если коротко, то все, что можно принять за реальный ответ, нужно рассматривать как допустимый результат проверочного запроса.

27.3.3. Итоговое определение API

В данном случае само определение API очень простое: необязательное логическое поле, которое позволяет пользователям указывать, что запрос «только для проверки», тем самым делая его полностью безопасным и идиоматичным (листинг 27.2).

Листинг 27.2. Итоговое определение API

```

abstract class ChatRoomApi {
    @post("/chatRooms")
    CreateChatRoom(req: CreateChatRoomRequest): ChatRoom;
}

interface CreateChatRoomRequest {
    resource: ChatRoom;
    validateOnly?: boolean;
}
  
```

27.4. КОМПРОМИССЫ

Как правило, проверочные запросы представляются простым решением для постоянных проблем. С помощью простого флага пользователи могут получить определенную долю уверенности в свете огромной сложности, возникающей из-за нагромождения всех деталей в API. Тем не менее такие запросы редко оказываются строго необходимыми и в большей степени просто обеспечивают дополнительное удобство, в первую очередь для пользователей.

И хотя основную пользу от них получают пользователи, есть здесь положительные стороны и для самих API. Например, если в сценариях с дорогостоящими методами (вычислительно, денежно или в обоих смыслах) соответствующие затраты не перекладываются на пользователя, то поддержка валидации запросов определенно минимизирует расходы по той простой причине, что пользователи могут выражать желание просто проверить запрос и никогда его не выполнять.

27.5. УПРАЖНЕНИЯ

1. Почему для валидации запросов нужно использовать именно флаг `validateOnly`, а не отдельный проверяющий их метод?
2. Представьте метод API, получающий данные от удаленного сервиса. Должен ли проверочный запрос также связываться с этим сервисом? Почему?
3. Может ли иметь смысл реализация валидации запросов в методах, которые никогда не записывают данные?
4. Почему важно, чтобы предустановленное значение флага `validateOnly` было `false`? Разумно ли изменить его на `true`?

РЕЗЮМЕ

- Некоторые запросы API оказываются опасными, в связи с чем оправдывают предоставление пользователям возможности проверить их перед фактическим выполнением.
- Методы API, поддерживающие эту функциональность, должны предоставлять простое логическое поле (`validateOnly`), указывающее, что запрос предназначен только для проверки и не должен влиять на внутреннюю систему.
- Нередко будут возникать сценарии, в которых методы API, поддерживающие проверочные запросы, будут взаимодействовать с внешними сервисами. В таких случаях они должны проверять эти внешние связи, насколько это возможно, признавая, что не получится проверить безопасность некоторых аспектов.

28

Ревизии ресурсов

В этой главе

- ✓ Как безопасно хранить несколько ревизий одного ресурса по мере его изменения.
- ✓ Как идентифицировать отдельные ревизии.
- ✓ Стратегии для явного и неявного создания ревизий.
- ✓ Как выводить список доступных ревизий и извлекать конкретные.
- ✓ Как работает возвращение к предыдущим ревизиям.
- ✓ Как поступать с дочерними ресурсами тех, что подвергаются ревизии.

Несмотря на то что ресурсы со временем меняются, обычно мы отбрасываем любые изменения, которые могли произойти в прошлом. Иначе говоря, мы всегда сохраняем только текущее представление ресурса и полностью игнорируем прежние. В данном паттерне представлена схема, с помощью которой можно отслеживать множество ревизий одного ресурса во времени. Это позволяет сохранять его историю, а также использовать дополнительную функциональность наподобие отката к предыдущей ревизии.

28.1. ЗАЧЕМ ЭТО НУЖНО

До этого момента все наше взаимодействие с ресурсами в API было сосредоточено исключительно на текущем состоянии ресурса и игнорировало его прежние представления. И хотя мы иногда затрагивали такие темы, как согласованность (например, когда смотрели, что происходит при изменении ресурса в процессе пагинации списка), мы всегда принимали во внимание предположение, что ресурсы существуют только в один момент времени, которым является «сейчас».

Такое допущение все это время исправно работало, но определенные виды ресурсов вполне могут требовать большего, нежели одномоментный снимок во времени. Например, мы зачастую отслеживаем историю в документах MS Word или Google. Аналогичным образом, если наш API включает ресурсы, представляющие контракты, ордера на покупку или даже рекламные кампании, то вполне ожидаемо, что для пользователей может оказаться полезной возможность просматривать историю ресурса во времени. Как итог, будет гораздо проще определить причину возникающей проблемы.

Цель данного паттерна — предоставить схему, которая позволит хранить множество ревизий одного ресурса, а также использовать более сложную функциональность, такую как удаление уже ненужных ревизий или откат к одной из предыдущих, по сути, отменяющий внесенные изменения.

28.2. ОБЗОР

Чтобы решить все эти задачи, паттерн будет опираться на новый принцип, называемый *ревизия ресурса*. Это просто его снимок, обозначенный уникальным идентификатором и временной меткой, обозначающей момент его создания. Эти ревизии, будучи простыми по своей природе, предоставляют много интересной функциональности.

Например, если мы можем просматривать все ревизии заданного ресурса и непосредственно извлекать отдельные из них, то технически у нас есть возможность заглянуть в прошлое и проследить развитие ресурса во времени. Более же важна здесь возможность отменять внесенные изменения, откатываясь к определенной ревизии и возвращая ресурсу ровно тот облик, который он имел в то время.

И как же выглядит ревизия ресурса? Придется ли нам отслеживать два отдельных интерфейса, синхронизированных в потоке жизни API? К счастью, ответ «нет». Ревизия ресурса вовсе не является отдельным интерфейсом. Наоборот, этот принцип привносится простым добавлением в существующий ресурс двух

полей: идентификатора ресурса и временной метки снимка, предназначенной для формирования ревизии (листинг 28.1).

Листинг 28.1. Добавление поддержки ревизий в ресурс Message

```
interface Message {
    id: string;
    content: string;
    // ...

    revisionId: string;
    revisionCreateTime: Date;
}
```

Добавив два поля, мы получили возможность представлять множество ревизий ресурса с помощью одного интерфейса

Если кратко, то в основе лежит сохранение множества записей с одним идентификатором (`id`). Однако каждая из этих записей имеет свой идентификатор ревизии (`revisionId`) и представляет ресурс в том виде, в котором он пребывал в определенный момент времени. И теперь, когда мы извлекаем ресурс по его идентификатору, результатом будет последняя ревизия этого ресурса. В некотором смысле ID ресурса становится псевдонимом для его последней ревизии, и «последняя» определяется как ревизия с самым свежим значением в поле `revisionCreateTime`.

Тем не менее, как обычно, возникает много вопросов о деталях этого процесса. Как решать, когда нужно создавать новую ревизию? Должно ли это происходить автоматически или нужно вмешательство пользователя? Как реализовывать функциональность, позволяющую выводить список ревизий или извлекать определенную ревизию ресурса? И как быть с откатами к прежним ревизиям? В следующем разделе мы все это разберем подробно.

28.3. РЕАЛИЗАЦИЯ

Как работают ревизии ресурсов? Мы уже видели, что внедрение этой возможности требует двух новых полей, но будет ли этого достаточно? Определенно нет. Здесь следует рассмотреть очень много сопутствующих тем, включая создание необходимых пользовательских методов и изменение существующих стандартных в целях соответствия текущему паттерну. Если коротко, то нам потребуется определить новые методы API для каждого нового поведения (например, извлечения отдельных ревизий ресурсов, перечисления ревизий, отката к предыдущим ревизиям и т. д.).

Прежде чем мы сможем сделать что-либо из перечисленного, нам необходимо понять, что конкретно должно включаться в это поле идентификатора ревизии и как он будет работать. В следующем подразделе мы начнем с правильного выбора вида идентификатора и узнаем, чем он может отличаться от типичных идентификаторов ресурсов.

28.3.1. Идентификаторы ревизий

Когда дело доходит до выбора идентификатора ревизии, у нас есть несколько вариантов. Первый — применить порядковый номер, подобно увеличению номера версии (1, 2, 3, 4, ...). Второй — задействовать временной идентификатор, похожий на временную метку (например, 1601562420). И третий — использовать полностью случайный идентификатор, положившись на принципы, изученные нами в главе 6.

Два первых варианта (порядковый номер и временные метки) подразумевают хронологическое упорядочивание ревизий в их идентификаторе отдельно от поля `revisionCreateTime`, что может вызвать проблемы. Если использовать инкрементируемые номера ревизий, то может возникнуть путаница в случае их удаления (см. подраздел 28.3.6 ниже), поскольку тогда будут возникать пробелы (например, 1, 2, 4). Удаление подразумевает полное избавление от чего-либо, поэтому использование инкрементируемых номеров для строки ревизии определенно окажется не лучшим решением, делая очевидным тот факт, что ревизия была удалена.

Временные метки, с другой стороны, не подвержены этой проблеме, поскольку следуют хронологическому порядку и безразличны к пробелам в данных. В случае идентификаторов на основе временных меток нам, наоборот, нужно беспокоиться об их коллизиях. Столкновение с этой проблемой очень маловероятно (обычно оно случается только при крайне высоком уровне параллельного доступа к системе), но это не означает, что данную опасность можно полностью игнорировать. Проще говоря, у нас есть более удачный способ, следующий принципам из главы 6, а именно случайный идентификатор.

В отличие от двух предыдущих вариантов, случайный идентификатор служит исключительно одной цели, являясь непрозрачным набором байтов, уникально идентифицирующих одну ревизию. Но должны ли ID ревизий быть полностью идентичны ID ресурсов? Несмотря на то что нам следует согласованно придерживаться использования идентификаторов в API, здесь есть один важный вопрос, касающийся ревизий: насколько большими должны быть такие идентификаторы?

25-символьный ID в формате Base32 (120 бит) может быть крайне важен при создании миллиардов и более ресурсов, однако важно понять, действительно ли нам нужна поддержка такого же количества ревизий одного ресурса, как и ресурсов отдельного типа. Как правило, у нас оказывается намного меньше ревизий, чем ресурсов, поэтому вряд ли нам потребуется 120-битное пространство ключей. Скорее будет разумным задействовать 13-символьный идентификатор (60 бит) в качестве близкого приближения к тому же размеру пространства ключей, что и у 64-битного целого числа.

Еще один вопрос касается того, имеет ли смысл использовать в идентификаторе символ контрольной суммы, как описывалось в подразделе 6.4.4. Несмотря на то

что он может казаться излишним, напомним, что этот символ предназначен для того, чтобы отличать что-либо отсутствующее (то есть валидный идентификатор без внутренних данных) от того, что вообще не могло присутствовать (то есть от недействительного идентификатора). Разумеется, проводить это различие важно не только для ресурса, но и для его отдельной версии. В результате мы практически однозначно должны задействовать в идентификаторе ревизий символ контрольной суммы.

Можно повторно использовать значительную часть уже известного нам кода из главы 6, сгенерировав с его помощью более короткий идентификатор (листинг 28.2).

Листинг 28.2. Пример кода для генерации случайных идентификаторов ревизий ресурса

```
const crypto = require('crypto');
const base32Decode = require('base32-decode');

function generateRandomId(length: number): string {
  const b32Chars = '012345689ABCDEFGHJKMNPQRSTVWXYZ';
  let id = '';
  for (let i = 0; i < length; i++) {
    let rnd = crypto.randomInt(0, b32Chars.length);
    id += b32Chars[rnd];
  }
  return id + getChecksumCharacter(id);
}

function getChecksumCharacter(value: string): string {
  const bytes = Buffer.from(
    base32Decode(value, 'Crockford'));

  const intValue = BigInt(`0x${bytes.toString('hex')}`);
  const checksumValue = Number(intValue % BigInt(37));

  const alphabet = '0123456789ABCDEFGH' +
    'JKLMNPQRSTUVWXYZ*~$=U';
  return alphabet[Math.abs(checksumValue)];
}
```

Здесь мы генерируем ID, случайно выбирая символы Base32

В заключение мы возвращаем ID вместе с вычисленным символом контрольной суммы

Начинаем с декодирования строки Base32 в буфер байтов

Затем конвертируем буфер байтов в значение BigInt

Вычисляем контрольную сумму, определяя остаток после деления на 37

Здесь мы задействуем алфавит Base32 для вычисления контрольной суммы

Теперь, когда мы разобрались с генерацией уникальных идентификаторов для ревизий, пора взглянуть, как именно рождаются эти ревизии.

28.3.2. Создание ревизий

Очевидно, что возможность идентификации ревизий важна, но сначала нужно разобраться, как вообще создаются эти ревизии. Это можно делать двумя способами: создавать их явно (когда пользователи должны специально просить

новую ревизию) или неявно (когда новые ревизии создаются автоматически без стороннего вмешательства).

И хотя обе стратегии абсолютно приемлемы, важно, чтобы для всех ресурсов с поддержкой ревизий использовалась какая-то одна из них. Если в разных ресурсах использовать разные стратегии, то это может привести к путанице, когда пользователи ожидают одну стратегию, а видят другую. В итоге нужно избегать смешивания и сопоставления и прозрачно документировать случаи, когда применение разных стратегий критически необходимо.

Кроме того, здесь есть один подвод. Независимо от выбранного способа, если ресурс поддерживает ревизии, то при его создании поле `revisionId` в нем обязательно должно быть заполнено. Иначе мы можем получить ресурсы, которые не являются фактическими ревизиями, что вызовет немало проблем при взаимодействии с ними, о котором мы будем говорить далее.

Изучение различных механизмов создания ревизий мы начнем с разбора неявной стратегии.

Неявные ревизии

Чаще всего ревизии ресурсов в любом API, поддерживающем такую функциональность, создаются неявно. Это просто означает, что, вместо того чтобы получать прямые инструкции на создание ревизии от пользователя, API сам решает, когда это нужно сделать. Но это не говорит о том, что пользователи не могут влиять на время создания ревизии. Например, наиболее популярный механизм подразумевает их создание API при каждом изменении хранящихся в ресурсе данных (рис. 28.1). В действительности существует много реальных систем, которые опираются именно на такой механизм, например история ревизий в Google Docs или отслеживание проблем на GitHub. В обоих этих примерах отслеживается вся история ресурса (документа или ишью).

И хотя это очень популярный и определенно наиболее безопасный способ (сохраняющий большую часть истории), есть и много других стратегий неявного отслеживания ревизий. Например, вместо того чтобы создавать новую ревизию для каждого изменения, сервис может делать это по расписанию, когда ревизии создаются в конце каждого дня при условии, что в течение него были внесены изменения. В качестве еще одного примера очередные ревизии могут создаваться после определенного количества изменений (например, трех), а не после каждого.

Более того, ревизии можно основывать на неких специальных ориентирах, а не интервалах. То есть вместо того, чтобы создавать новую ревизию при каждом обновлении ресурса, можно делать это при изменении определенного поля или, возможно, при выполнении конкретного пользовательского метода. К примеру, можно сделать так, что API будет создавать новую ревизию для `BlogPost` каждый раз, когда выполняется пользовательский метод `PublishBlogPost()`.

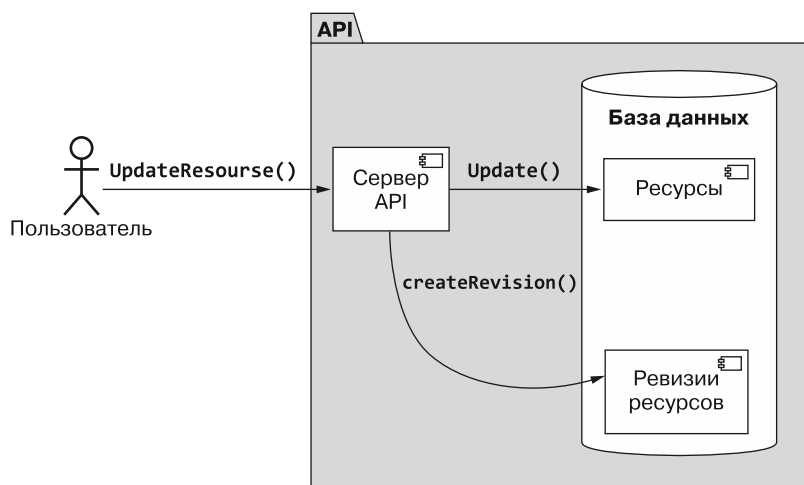


Рис. 28.1. Структурная схема неявного создания ревизий при обновлении ресурса

Все эти варианты полностью приемлемы, и каким бы соблазнительным ни казался подход, предписывающий универсальный рецепт для создания всех ревизий, в данном случае это просто невозможно. Причина в том, что каждый API своеобразен, а разные налагаемые бизнесом или продуктом ограничения определяют разные оптимальные стратегии создания ревизий ресурсов.

Хорошим правилом здесь будет придерживаться большего их количества (техники экономии пространства мы изучим в подразделе 28.3.7), и один из наиболее прогнозируемых подходов — создание ревизий просто при каждом изменении ресурса. Этот подход вполне понятен и удобен, а его использование приносит ощутимую пользу в большинстве случаев, когда пользователям необходима поддержка ревизий ресурсов.

Явные ревизии

Во многих случаях неявное создание ревизий может оказаться совершенно ненужным и излишне затратным, что ведет нас к очевидной альтернативе: позволить пользователям явно заявлять, когда нужно создать новую ревизию, с помощью пользовательского метода (рис. 28.2). Такой явный механизм дает пользователям возможность контролировать время создания ревизий, перекладывая на них ответственность за принятие этого решения.

Для этого можно создать новый пользовательский метод, который во многом будет напоминать стандартный метод `Create`. Вместо того чтобы создавать новый ресурс под существующим родителем, этот пользовательский метод будет отвечать лишь за снятие снимка ресурса в конкретный момент времени

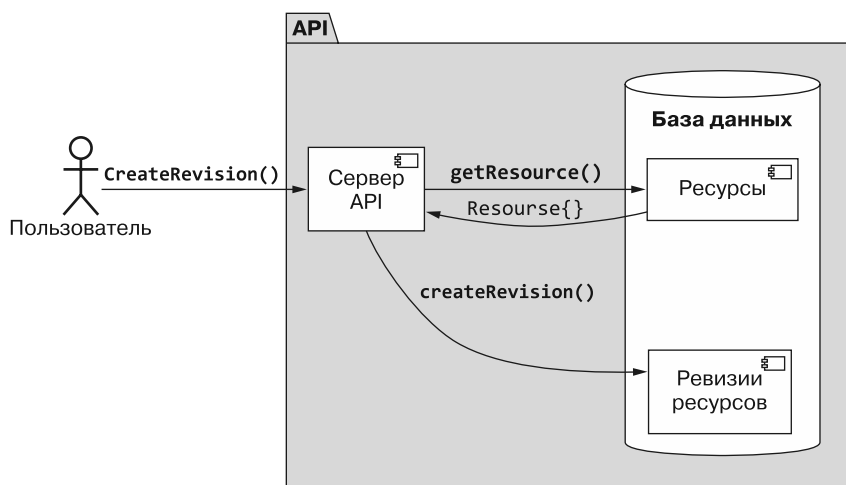


Рис. 28.2. Структурная схема создания ревизий ресурса

и создание новой ревизии для представления этого снимка. Созданная ревизия должна иметь случайно сгенерированный идентификатор (как описано в подразделе 28.3.1), а ее временная метка должна быть установлена на время окончательного сохранения этой ревизии (листинг 28.3).

Листинг 28.3. Определение API для создания ревизии ресурса

```

abstract class ChatRoomApi {
    @post("/{id=chatRooms/*/messages/*}:createRevision")
    CreateMessageRevision(req: CreateMessageRevisionRequest): Message;
}

interface Message {
    id: string;
    content: string;
    // ...
    revisionId: string;
    revisionCreateTime: Date;
}

interface CreateMessageRevisionRequest {
    id: string;
}
  
```

Пользовательский метод создает новую ревизию и возвращает ее (она является просто ресурсом, в котором установлены специальные поля)

Можно добавить в ресурс Message поддержку ревизий с помощью двух новых полей, о которых говорилось в разделе 28.2

Этому методу нужно знать лишь идентификатор ресурса

Освоив способы создания ревизий ресурсов, пора научиться фактически взаимодействовать с ними. Начнем с заглядывания в прошлое путем извлечения конкретных прошлых ревизий.

28.3.3. Извлечение конкретных ревизий

В разделе 28.2 говорилось, что одной из наиболее ценных возможностей, предоставляемых ревизиями ресурсов, является возможность оглядываться назад и считывать данные ресурса в том виде, в котором он выступал в прошлом. Но тут возникает вопрос: как именно нужно запрашивать прошлую ревизию ресурса? Технически идентификатор ревизий (см. подраздел 28.3.1) хранится в поле самого ресурса. Тогда нужно ли нам задействовать стандартный метод `List` с особым фильтром? Чисто технически это сработает, но будет весьма затруднительно и странно использовать метод `List` для извлечения одного ресурса (а не для его основного предназначения, заключающегося в просмотре их множества). К счастью, есть куда более удачный способ, состоящий из двух частей.

Во-первых, поскольку извлечение отдельных элементов определенно входит в ответственность стандартного метода `Get`, этот механизм должен опираться на расширение данного метода, чтобы иметь возможность поддерживать новый случай с ревизиями. Во-вторых, из этого следует, что нам потребуется ясный и простой способ предоставления в запросе к данному методу идентификатора ресурса. Для этого можно задействовать новый разделяющий символ, чтобы разбить идентификатор ресурса на две части: ID ресурса и ID ревизии. В качестве делителя выступит символ `@`, в результате чего полноценные идентификаторы ревизий будут выглядеть как `/chatRooms/1/messages/2@1234`, представляя ревизию 1234 заданного ресурса `Message`.

Преимущество использования символа-делителя состоит в том, что тогда стандартный метод `Get` совсем не придется менять. Поскольку мы просто отправляем чуть более детализированный идентификатор, то можно задействовать в запросе то же самое поле `id` и позволить сервису API трактовать его как ресурс заданной ревизии.

И тут возникает интересный вопрос: когда ресурс отвечает конкретной ревизией, что должно помещаться в его поле ID? Иными словами, если мы запрашиваем ревизию 1234, то будет ли этот ID соответствовать значению, предоставленному в запросе (например, `resources/abcd@1234`), или же нам нужно вернуться к исключительно одному идентификатору ресурса (например, `resources/abcd`) и поместить ревизию в поле `revisionId`?

Ответ будет простым: возвращаемый ресурс должен иметь идентификатор, равнозначный запрошенному (и поле `revisionId` всегда должно быть заполнено идентификатором возвращаемой ревизии), поскольку важно, чтобы мы могли легко определить получение именно запрошенного результата (листинг 28.4).

Листинг 28.4. Инвариант получения того, что было запрошено

```
assert(GetResource({ id: id }).id == id);
```

← Извлечение чего-либо по его ID всегда должно давать результат с этим же ID

Это значит, что при запросе ресурса без указания ID ревизии поле `id` возвращаемого ресурса будет содержать только ID ресурса (без символа `@` и без ID ревизии), но поле `revisionId` все равно будет заполнено. Если затем сделать запрос этой же ревизии ресурса (на основе поля `revisionId`), то результат будет содержать те же данные, но поле `id` будет также включать ID ревизии (листинг 28.5).

Листинг 28.5. Взаимодействие со стороны клиента, показывающее разные идентификаторы для извлеченных ресурсов

<pre> > GetMessage({ id: 'chatRooms/1/messages/2' }); { id: 'chatRooms/1/messages/2', // ..., revisionId: 'abcd' } </pre>	Когда мы запрашиваем ресурс без ID ревизии, то результат содержит идентичный ID (без ID ревизии)
<pre> > GetMessage({ id: 'chatRooms/1/messages/2@abcde' }); { id: 'chatRooms/1/messages/2@abcde', // ..., revisionId: 'abcd' } </pre>	Когда мы запрашиваем тот же ресурс, но также предоставляем ID ревизии, значение ревизии включается в ID результата

Поле `revisionId` всегда заполняется верным значением

Теперь, когда мы научились извлекать отдельную ревизию ресурса, пора разоб-
раться просмотр всех ревизий по аналогии с тем, как мы просматриваем все ресурсы заданного типа.

28.3.4. Перечисление ревизий

К счастью, у нас есть неплохой опыт вывода списков ресурсов на основе стандартного метода `List` (см. подраздел 7.3.4). Однако в данном случае мы будем выводить ревизии, связанные с одним ресурсом, а не дочерние ресурсы определенного родителя. Это значит, несмотря на возможность опереться на большую часть тех же принципов, нельзя просто скопировать метод `List` и заставить работать его для ревизий ресурсов.

Что же тогда нужно изменить в этом методе? Для начала, вместо того чтобы задействовать HTTP-метод `GET` для коллекции ресурсов, мы будем применять нечто более похожее на пользовательский метод, отображающийся в `GET /resources/*:listRevisions`. Кроме того, поскольку целью выступает не сам родитель, а фактический ресурс, в котором хранятся ревизии, мы используем поле `id`, а не `parent`. В остальном метод должен действовать аналогично, даже поддерживать пагинацию, как мы видели в главе 21 (листинг 28.6).

Этот пользовательский метод дает нам возможность просматривать историю ревизий ресурса, опираясь в основном на те же принципы, что и стандартный метод `List`. Теперь, когда мы разобрались с созданием, извлечением и просмотром ревизий, можно заняться чем-то более интерактивным: откатом к предыдущим ревизиям.

Листинг 28.6. Определение API для вывода списка ревизий ресурса

```

abstract class ChatRoomApi {
    @get("/{id=chatRooms/*/messages/*}:listRevisions")
    ListMessageRevisions(req: ListMessageRevisionsRequest):
        ListMessageRevisionsResponse;
}

interface ListMessageRevisionsRequest {
    id: string;
    maxPageSize: number;
    pageToken: number;
}

interface ListMessageRevisionsResponse {
    results: Message[];
    nextPageToken: string;
}

```

Этот пользовательский метод вывода ревизий аналогично методу List поддерживает пагинацию

Обратите внимание: ответ включает сами ресурсы с варьирующимися идентификаторами ревизий

28.3.5. Восстановление прежней ревизии

Несмотря на то что хранение записи изменений ресурса является полезной возможностью, одним из наиболее ценных элементов функциональности, которую можно предоставить на основе истории ревизий, является возможность восстанавливать ресурс до того состояния, в котором он пребывал в прошлом. Для этого мы задействуем пользовательский метод, который будет отвечать за создание *новой* ревизии ресурса, применяя в качестве источника данных одну из имеющихся. Эта новая ревизия будет самой новой (а значит, имеющей самое свежее значение `revisionCreateTime`), так что станет текущим представлением ресурса (листинг 28.7). Это в некотором смысле сродни прохождению по файлу в поиске одной из старых версий документа, снятию фотокопии этого документа и ее помещению поверх стека. В данном случае такой фотокопией становится последняя ревизия, несмотря на то, что она представляет более старые данные.

Листинг 28.7. Определение API для восстановления ревизий ресурса

```

abstract class ChatRoomApi {
    @post("/{id=chatRooms/*/messages/*}:restoreRevision")
    RestoreMessageRevision(req: RestoreMessageRevisionRequest): Message;
}

interface RestoreMessageRevisionRequest {
    id: string;
    revisionId: string;
}

```

Прежняя ревизия восстанавливается с помощью прикрепленного к ресурсу пользовательского метода

Запрос на восстановление ревизии также должен указывать на ревизию, которую нужно восстановить

Важно помнить, что, хоть мы и создаем новый пользовательский метод, эта функциональность строится за счет имеющихся компонентов (не являясь атомарной или такой же удобной) (листинг 28.8).

Листинг 28.8. Реализация восстановления прежней ревизии на стороне клиента

```
function restoreMessageRevision(messageId: string, revisionId: string):
    Message {
        const old = GetMessage({
            id: `${messageId}@${revisionId}`
        });
        UpdateMessage({
            resource: Object.assign(old, { id: messageId })
        });
        return CreateMessageRevision({ id: messageId });
    }
```

Начинаем с извлечения старой версии ресурса

Затем обновляем ресурс с помощью этих старых данных (убеждаясь, что нацеливаемся на само сообщение, а не на старую ревизию)

Если новая ревизия создается не автоматически, то нужно создать ее явно и вернуть результат

Почему бы просто не позволить пользователям самим реализовать эту функциональность? Но тогда, как и в случае многих самодельных методов API, нам придется думать о конкурентности, атомарности и удобстве. Такая реализация функции требует трех полноценных двухсторонних линий коммуникации, любая из которых может как дать сбой, так и быть задержана либо прервана другими пользователями, взаимодействующими с тем же ресурсом. Предоставляя ту же функциональность в виде пользовательского метода, мы даем пользователям атомарный способ, позволяющий выполнять это восстановление до прежних ревизий.

Применительно к этому методу необходимо помнить, что он не удаляет и не изменяет историю ресурса. Он не перемещает старую ревизию на передний план (например, изменяя временную метку создания на последнюю). Вместо этого он создает полностью новую ревизию, скопированную со старой, и помещает этот дубликат в начало, хоть и под другим идентификатором. Данное действие гарантирует, что при просмотре истории мы сможем ясно видеть прогресс изменения данных во времени и не запутаемся среди ревизий, перемещенных в разные места.

Но могут возникать случаи, когда нам действительно нужно переписать историю, каким бы печальным это ни казалось. В следующем подразделе мы узнаем, как изменять хронологические данные путем удаления ревизий.

28.3.6. Удаление ревизий

К сожалению, все мы совершаем ошибки. И хотя ревизии ресурсов являются мощным инструментом, позволяющим заглянуть в историю изменений, они также запечатлевают в ней наши промахи, вынуждая постоянно помнить

о них. Это может казаться незначительным неудобством, но представьте, что мы случайно сохранили какие-то конфиденциальные данные (номера социального страхования, кредитных карт или информацию о здоровье) в ресурсе с поддержкой ревизий. Когда мы в итоге удаляем такие данные, по факту они никогда не стираются, поскольку остаются в предыдущей ревизии. Несложно догадаться, что это может стать большой проблемой.

Чтобы решить данную проблему, во многих системах будет крайне важно реализовать поддержку метода удаления ревизий ресурсов. Этот метод, подобно уже рассмотренным, опирается на те же принципы, что и стандартный метод `Delete`, удаляя ресурс по его уникальному идентификатору, которым в данном случае выступает ID ресурса, объединенный с ID ревизии (листинг 28.9).

Листинг 28.9. API для удаления ревизии ресурса

```
abstract class ChatRoomApi {
    @delete("/{id=chatRooms/*/messages/*}:deleteRevision")
    DeleteMessageRevision(req: DeleteMessageRevisionRequest): void;
}

interface DeleteMessageRevisionRequest {
    id: string;
}
```

По аналогии со стандартным методом `Delete` мы делаем сопоставление с HTTP-глаголом `DELETE`

Нам лишь нужно предоставить полный идентификатор ревизии (включая ID ресурса)

Несмотря на кажущуюся простоту этого метода, здесь есть пара интересных вопросов. И начать стоит с того, почему он вообще должен существовать.

Перегрузка стандартного метода `Delete`

Разве буквально недавно в подразделе 28.3.3 мы не утверждали, что можем повторно использовать имеющийся стандартный метод для выполнения аналогичной работы, если разница будет заключаться лишь в идентификаторе? Почему бы просто не задействовать повторно стандартный `Delete`, дополнив его идентификаторы ревизий?

Ответ будет довольно коротким: их легко спутать. Поскольку удаление данных может причинить серьезный вред, важно провести явное разделение между удалением ресурса и удалением одной ревизии. Если эти методы API не отделить друг от друга, то ошибочная подстановка переменной может обусловить разницу между удалением одной ревизии ресурса и удалением их всех (вместе с самим ресурсом). Будет намного безопаснее положиться на отдельный пользовательский метод, а не перегружать существующий стандартный метод `Delete`.

Удаление текущей ревизии

Что должно произойти, если пользователь захочет удалить текущую (последнюю) ревизию ресурса? Если мы решим допустить такое, то, по сути, предоставим способ удаления ревизии и восстановления другой в одном методе API,

поскольку вторая по давности ревизия в таком случае будет становиться новой «текущей». И хотя подобное допущение определенно выглядит соблазнительно, оно выведет пользовательский метод удаления ревизии за рамки его исходной задачи. Если клиент пытается удалить последнюю ревизию, то запрос должен завершаться неудачно, выдавая HTTP-ошибку **412 Precondition Failed** или равнозначную ей. К счастью, это поведение также гарантирует, что нам не придется иметь дело со сценарием, в котором пользователь пытается удалить единственную оставшуюся (а значит, текущую) ревизию.

Мягкое удаление

В завершение нужно разобраться с тем, как обрабатывать мягкое удаление касательно ревизий (глава 25). Должна ли она подвергаться такой процедуре? И если да, то следует ли реализовывать ее таким же образом? В целом ревизии не стоит рассматривать как допускающие мягкое удаление. В действительности, даже если сам ресурс и можно мягко удалить, важно, чтобы у нас была возможность восстановить прежнюю ревизию, которая тогда сможет перейти из мягко удаленного состояния в неудаленное. В итоге, когда мы удаляем ревизии, это должно происходить окончательно, чтобы они больше не возникали в системе.

28.3.7. Обработка дочерних ресурсов

До сих пор мы следовали довольно обширному допущению, что ревизии ресурсов являются снимками одного ресурса и данных, встроенных в него непосредственно. Но что, если этот ресурс является родителем других ресурсов? Например, вдруг мы захотим поддерживать ревизии для ресурсов **ChatRoom**. Должны ли тогда дочерние ресурсы **Message** храниться вместе как часть ревизии **ChatRoom**? Или же эти ревизии ресурса должны лишь сохранять непосредственные данные о **ChatRoom**, такие как название комнаты?

Это сложная тема. С одной стороны, если мы действительно хотим получить снимок ресурса **ChatRoom**, то определенно будет разумнее сделать так, чтобы восстановление прежней ревизии возвращало нас к тому представлению сообщений, которое существовало на момент создания этой ревизии. С другой же стороны, это будет означать очень много работы, увеличение требований к хранилищу и в конечном итоге возросшую сложность для пользователей, которые захотят восстановить лишь данные, встроенные непосредственно.

К сожалению, это один из тех сценариев, в котором нет единого универсального решения. Как правило, оптимальным вариантом будет фокусировать ревизии исключительно на ресурсе, не включая всю иерархию его потомков. Такой вариант к тому же оказывается и более простым. В прочих, более уникальных

сценариях может возникнуть потребность в отслеживании не только ресурса, но и всей его иерархии.

Если коротко, то при необходимости вполне приемлемо выйти за рамки самого ресурса, но тогда нужно убедиться, что это обязательное требование бизнеса и другие варианты (например, метод `Export`, как показано в главе 23) недоступны. Если же можно обойтись без включения в ревизии иерархии ресурсов, то одна только сложность, которой можно избежать таким образом, станет аргументом, достаточным для принятия подобного решения.

28.3.8. Итоговое определение API

В листинге 28.10 представлен пример всех методов API, нацеленных на обработку ревизий отдельных ресурсов `Message`.

Листинг 28.10. Итоговое определение API

```
abstract class ChatRoomApi {
    @get("/{id=chatRooms/*/messages/*}")
    GetMessage(req: GetMessageRequest): Message;

    @post("/{id=chatRooms/*/messages/*}:createRevision")
    CreateMessageRevision(req: CreateMessageRevisionRequest): Message;

    @post("/{id=chatRooms/*/messages/*}:restoreRevision")
    RestoreMessageRevision(req: RestoreMessageRevisionRequest): Message;

    @delete("/{id=chatRooms/*/messages/*}:deleteRevision")
    DeleteMessageRevision(req: DeleteMessageRevisionRequest): void;

    @get("/{id=chatRooms/*/messages/*}:listRevisions")
    ListMessageRevisions(req: ListMessageRevisionsRequest):
        ListMessageRevisionsResponse;
}

interface Message {
    id: string;
    content: string;
    // ... остальные поля ...
    revisionId: string;
    revisionCreateTime: Date;
}

interface GetMessageRequest {
    id: string;
}

interface CreateMessageRevisionRequest {
    id: string;
}
```

```
interface RestoreMessageRevisionRequest {
    id: string;
    revisionId: string;
}

interface DeleteMessageRevisionRequest {
    id: string;
}

interface ListMessageRevisionsRequest {
    id: string;
    maxPageSize: number;
    pageToken: string;
}

interface ListMessageRevisionsResponse {
    results: Message[];
    nextPageToken: string;
}
```

28.4. КОМПРОМИССЫ

Ревизиям ресурсов присущ большой потенциал, и, тем не менее, они существенно увеличивают сложность как для разработчиков API, так и для его пользователей. Более того, их применение также обуславливает повышенные требования к объему хранилища, поскольку по мере изменения ресурсов и создания ревизий приходится отслеживать все больше данных. В частности, даже если эта функциональность является для API критически важной, ревизии, включающие иерархию (когда одна ревизия включает и ресурс, и всех его потомков), еще больше все усложняют и устанавливают еще более высокие требования к системе хранения.

В результате, если есть возможность избежать применения в API ревизий ресурсов, то так и нужно сделать. Тем не менее в некоторых случаях они являются лучшим решением (например, если API отслеживает юридические документы и нужно сохранить историю ресурса навсегда). В подобных случаях применение ревизий предоставляет безопасный и простой способ поддержки такой функциональности для пользователей API.

28.5. УПРАЖНЕНИЯ

1. Какие виды сценариев лучше всего подходят для неявного создания ревизий, а какие — для явного?
2. Почему для ревизий нужно использовать случайные идентификаторы, а не инкрементируемые номера или временные метки?

3. Почему при восстановлении ревизии создается новая? Почему бы не сослаться на предыдущую?
4. Что должно происходить, когда вы пытаетесь вернуть ресурс к предыдущей ревизии, но он уже ей соответствует?
5. Почему для вывода списка и удаления ресурсов мы используем нотацию пользовательского метода, а не стандартные методы?

РЕЗЮМЕ

- Ревизии ресурсов выступают в качестве снимков этих ресурсов, получаемых по мере их изменения.
- Как и для любого ресурса, для ревизий нужно использовать случайные идентификаторы, даже если кажется, что для этого лучше применить нечто вроде инкрементируемого номера или временной метки.
- Ревизии можно создавать как неявно (например, при каждом изменении ресурса), так и явно (например, по запросу пользователя).
- Извлечение конкретных ревизий ресурсов можно выполнять с помощью символа @, обозначающего ревизию (например, GET /chatRooms/1234@5678). Тем не менее и вывод списка ревизий, и их удаление должны выполняться с помощью пользовательского метода, прикрепленного к рассматриваемому ресурсу.
- Ресурсы можно восстанавливать до состояния прежних ревизий с помощью пользовательского метода, который создает новую ревизию, равнозначную предыдущей, и обозначает ее как текущую.

29

Повтор запросов

В этой главе

- ✓ Как определить, какие запросы к API, завершенные неудачно, можно повторить безопасно.
- ✓ Доработанные стратегии экспоненциальной выдержки, позволяющие определить интервал повторов.
- ✓ Как избежать столпотворения запросов.
- ✓ Способы, которыми API могут определять для клиентов интервал повтора.

Когда в API случаются ошибки, то в одних случаях в них виноваты клиенты, а в других — сторонние проблемы. Зачастую лучшим решением для второй категории оказывается повтор запроса через некоторое время в надежде получить другой результат. В текущем паттерне представлен механизм, с помощью которого клиенты смогут понять, как и когда можно повторять запросы, ранее завершившиеся неудачно из-за ошибок на сервере API.

29.1. ЗАЧЕМ ЭТО НУЖНО

Некоторые запросы API будут неизбежно проваливаться. Многие из таких случаев станут следствием ошибок клиентов, таких как неверные сообщения запросов, другие же могут быть вызваны нарушением предусловий и ограничений, установленных сервером API. Все эти типы ошибок объединяет один общий

атрибут: проблема заключается в самом запросе, нарушающем некие ограничения или идущем против бизнес-логики API. Иначе говоря, если воспроизвести тот же недействительный запрос, то в ответ должна вернуться такая же ошибка. В конце концов, бизнес-логика обычно не изменяется изо дня в день.

Но есть и совершенно иная категория ошибок. Иногда запрос ведет к ошибочному ответу, который является временным. Такой тип ошибки никак не связан с самим запросом и становится результатом внутренней проблемы API. Возможно, сервер API перегружен и не смог обработать входящий запрос, или же система либо необходимый субкомпонент находились в плановом простое, не будучи доступными для обработки запроса. Какой бы ни была проблема, решить ее можно с помощью очень простого и очевидного способа — нужно повторить запрос.

Суть в том, что ошибочный ответ был вызван не конкретным запросом. Напротив, он стал следствием причин, которые могли лишь косвенно относиться к запросу и скорее вообще не были связаны с ним. То есть если просто повторить этот запрос спустя какое-то время, то велика вероятность, что эта попытка завершится успехом (или по меньшей мере приведет к другой ошибке).

Но тогда возникает серьезный вопрос: как именно определять разницу между запросом, который нужно повторить, и тем, который повторять не стоит? Обычно HTTP-ошибки имеют номера, передающие смысл неполадки. И в случае некоторых видов ошибок повтор запроса оказывается вполне рациональным. Например, ошибки 4** (400 Bad Request, 403 Forbidden, 404 Not Found и т. д.), скорее всего, отражают проблемы с конкретным запросом, в то время как ошибки 5** (500 Internal Server Error или 503 Service Unavailable) указывают на проблемы с внутренними сервисами. Из этих двух категорий ошибки 5**, скорее всего, можно будет исправить, но это больше ориентир, нежели твердое правило, поэтому тут определенно требуется более детальное рассмотрение. Кроме того, хоть эти коды ошибок и могут намекать на уместность повторения запроса, они ничего не говорят о том, когда именно клиент должен повторить его, поэтому он вынужден строить догадки.

В текущем паттерне мы узнаём, как в сервисе API можно реализовать политику, которая позволяет определить допускающие повтор запросы, а также алгоритм тайминга, вычисляющий необходимую длительность ожидания повтора. Вдобавок мы разберем, как сервис API может сообщать клиенту, когда именно делать повтор в случае, если сервису известна некая дополнительная информация.

29.2. ОБЗОР

Цель данного паттерна проста: отвечать на максимальное количество запросов, повторяя как можно меньше. Чтобы ее реализовать, нам потребуется решить две задачи. Во-первых, нужно предоставить клиентам алгоритм для минимизации повторов в системе. Во-вторых, если сервису API известно что-то, о чем не знает

клиент, и эта информация ведет к пониманию, когда именно можно повторить запрос успешно, то ему нужен механизм для передачи клиенту соответствующей инструкции. Начнем с рассмотрения алгоритма, позволяющего определить интервал задержки между повторами запросов.

29.2.1. Интервал повторов на стороне клиента

Когда в ходе запроса возникает сбой по вине сервиса, такой запрос практически однозначно следует повторить, исходя из расчета, что на этот раз сервис окажется доступным. Но нужно ли делать это сразу или же стоит подождать? И если нужно ждать, то как долго?

В действительности это очень давняя проблема, корни которой уходят во времена зарождения интернета, когда нам требовался алгоритм, позволяющий понимать, когда стоит повторно отправлять ТСП-пакеты по потенциально перегруженной сети. И эту проблему особенно затрудняет отсутствие понимания, что именно происходит после отправки запроса, в результате чего нам остается лишь гадать, когда он может быть успешно обработан на другой стороне. Исходя из этих ограничений, наиболее эффективным и проверенным решением здесь будет алгоритм *экспоненциальной выдержки*.

Идея этого алгоритма довольно проста: до первой попытки повтора ожидать одну секунду, затем для каждого последующего сбоя ответа мы удваиваем время ожидания. Если запрос завершится неудачно второй раз, то прежде чем его повторить, мы ожидаем две секунды. Если он снова завершится сбоем, то ожидаем уже четыре секунды. Каждый раз время ожидания продолжает удваиваться (до 8, 16, 32, 64 секунд) и т. д., пока либо запрос не завершится успехом, либо мы не сдадимся. В данном же случае мы дополнительно доработаем этот алгоритм, но основной его принцип останется прежним.

Как было сказано, экспоненциальная выдержка отлично подходит, когда система на той стороне представляет собой «черный ящик». То есть если никаких дополнительных сведений об этой системе у нас нет, то этот алгоритм работает хорошо. Однако разве так обстоят дела в случае нашего веб-API? Как раз нет, и в следующем подразделе мы узнаем, что для уникальной подкатегории сценариев существует более удачное решение.

29.2.2. Интервал повторов, определяемый сервером

Несмотря на то что экспоненциальная выдержка по-прежнему останется хорошим вариантом, существуют определенные случаи, содержащие более оптимальное решение. Например, представьте сценарий, когда запрос завершается

сбоем, так как сервис API сообщает, что этот запрос можно выполнять только раз в минуту. В таком случае сервер знает, сколько нужно ожидать, прежде чем повторять запрос (60 секунд).

Как правило, если сервер может предоставить надежное руководство о том, когда следует повторить запрос, то эта информация должна передаваться клиенту. В конце концов, даже частично обоснованная догадка лучше, чем полностью слепая. И хотя подобные сценарии не слишком распространены, они явно недостаточно малочисленны, чтобы их совсем не учитывать.

Мы воспользуемся особым параметром, предназначенным специально для такого сценария. Он будет инструктировать клиента отказаться от алгоритма экспоненциальной выдержки и вместо этого определить время повтора запроса исключительно на основе предоставленного значения. Таким образом, в большинстве случаев нам удастся заменить метод слепого угадывания частично информированным, а в некоторых эпизодах мы сможем быть абсолютно уверенными.

29.3. РЕАЛИЗАЦИЯ

Как стало известно из предыдущего раздела, в большинстве случаев этот паттерн будет опираться на экспоненциальную выдержку, а в некоторых — использовать особое поле, с помощью которого сервер будет инструктировать клиента о том, сколько именно нужно ждать до повтора запроса. Тем не менее, прежде чем перейти к делу, нужно обсудить одну упущенную из виду деталь. Как вообще мы можем определить, что запрос нужно повторить?

29.3.1. Допустимость повтора

Здесь может возникнуть соблазн предположить, что повторить запрос можно после любой ошибки. К сожалению, это опасное допущение. Самое серьезное опасение вызывает тот факт, что оно может привести к непредвиденным последствиям, таким как повтор результатов (например, создание одного ресурса дважды). Как же узнать, какие типы запросов можно повторить, а какие нет?

В целом ответы делятся на три категории: повторить нельзя; повторить можно, но осторожно; и можно смело повторить. Кроме того, возможность повтора также зависит от самого метода API, в частности от того, является ли он идемпотентным и безопасным. В текущем подразделе мы разберем эти категории и обобщим HTTP-коды, которые относятся к каждой из них.

Можно повторить

К сожалению, список кодов ошибок, после которых обычно вполне допустимо повторить запрос, относительно краток и в общем виде представлен в табл. 29.1.

Таблица 29.1. Образец кодов ответов, после которых обычно можно повторить запрос

Код	Название	Описание
408	Request Timeout	Клиент выполнил запрос недостаточно быстро
421	Misdirected Request	Запрос был отправлен на сервер, который не смог его обработать
425	Too Early	Сервер не хочет пытаться обрабатывать запрос, который может быть воспроизведен
429	Too Many Requests	Клиент отправил слишком много запросов за установленный период времени
503	Service Unavailable	Сервер не может обработать запрос, поскольку перегружен

В случае кодов 408, 421, 425, 429 и 503 сервер даже и не начинал обрабатывать полученные запросы. Здесь можно сделать вывод, что при получении подобных ответов запросы практически однозначно можно повторить.

Повторять бессмысленно

Как правило, если сервер получает и обрабатывает запрос, но определяет его фундаментальную недопустимость, такой запрос повторять не стоит. Например, если в ответ на попытку извлечь ресурс мы получим ошибку 403 *Forbidden*, то повтор запроса не приведет к другому результату, если только в API не изменится что-то еще (например, политика предоставления разрешений). Подобные ошибки (некоторые из них показаны в табл. 29.2) довольно легко опознать, поскольку они связаны с самими запросами или ошибками сервера, сообщающего о нарушении установленных правил.

Таблица 29.2. Примеры кодов, говорящих о бессмысленности повтора запроса

Код	Название	Описание
403	Forbidden	Запрос в порядке, но сервер отказался его обрабатывать
405	Method Not Allowed	Указанный метод не разрешен
412	Precondition Failed	Сервер не отвечает требованиям запроса
501	Not Implemented	Сервер не может распознать или обработать запрос

В целом все эти коды ответов указывают на нечто постоянное, подразумевая, что повтор запроса в будущем не приведет к другому ответу (естественно, при условии, что в системе ничего не изменится).

Можно повторить

Самая коварная категория — это ошибки, в случае которых повтор запроса может быть приемлемым, но результат будет зависеть от дополнительных деталей запроса и метода API. Например, ошибка 504 Gateway Timeout указывает, что запрос к API был перенаправлен к вышестоящему серверу, который не ответил на него. В итоге достоверно неизвестно, получил ли этот сервер запрос и начал ли его обработку. Ввиду этой неопределенности можно лишь повторить запрос, если мы уверены, что повтор точно не вызовет проблем. Примеры запросов из этой категории приведены в табл. 29.3.

Таблица 29.3. Примеры кодов, предполагающих допустимость повтора запроса

Код	Название	Описание
500	Internal Server Error	Неожиданный сбой на сервере
502	Bad Gateway	Запрос был передан восходящему серверу, который вернул недействительный ответ
504	Gateway Timeout	Запрос был передан восходящему серверу, который не ответил

Как обычно, лучшим способом убедиться в возможности повтора запросов будет использовать идентификаторы, что позволит серверу дедуплицировать запросы, избавив нас от нежелательного поведения (мы обсуждали это в главе 26).

Теперь, когда мы имеем представление о различных запросах и о возможности их повторения, пора перейти к рассмотрению случаев, в которых было принято решение о приемлемости повторения запроса. В частности, мы изучим способ, позволяющий определить время, которое необходимо выждать, прежде чем выполнять повтор.

29.3.2. Экспоненциальная выдержка

Как мы уже знаем, экспоненциальная выдержка — это алгоритм, по которому интервал между запросами возрастает экспоненциально, удваиваясь каждый раз, когда запрос возвращает ошибку. Пожалуй, понятнее всего будет рассмотреть его на примере кода (листинг 29.1).

Листинг 29.1. Пример, демонстрирующий повтор с использованием экспоненциальной выдержки

```
async function getChatRoomWithRetries(id: string): Promise<ChatRoom> {
  return new Promise<ChatRoom>(async (resolve, reject) => {
    let delayMs = 1000;
    while (true) {
      try {
```

← Определяем начальную задержку в 1 секунду (1000 миллисекунд)

```

    return resolve(GetChatRoom({ id }));
  } catch (e) {
    await new Promise((resolve) => {
      return setTimeout(resolve, delayMs);
    });
    delayMs *= 2;
  }
}
});
}

```

Сначала пытаемся извлечь ресурс и разрешить промис

Если результат завершается сбоем, то ожидаем в течение установленного интервала

В завершение удваиваем последующий интервал ожидания

И хотя эта реализация станет хорошей стартовой точкой, ей не мешает пара доработок. Начнем с рассмотрения максимумов.

Максимальные величина задержки и количество повторов

Как было сказано, этот алгоритм продолжит повторять запросы бесконечно, пока не вернется успешный результат. И несмотря на то что подобной настойчивостью можно восхищаться, обычно это плохая идея, поскольку в таком случае продолжительность запроса до его завершения никак не ограничивается.

Это можно исправить, добавив пару лимитов. Сначала мы установим предел максимального количества повторов. Затем можно ограничить и максимальную продолжительность ожидания между повторами запросов (листинг 29.2).

Листинг 29.2. Добавление поддержки максимальной задержки и количества повторов

```

async function getChatRoomWithRetries(
  id: string, maxDelayMs = 32000, maxRetries = 10): Promise<ChatRoom> {
  return new Promise<ChatRoom>(async (resolve, reject) => {
    let retryCount = 0;
    let delayMs = 1000;
    while (true) {
      try {
        return resolve(GetChatRoom({ id }));
      } catch (e) {
        if (retryCount++ > maxRetries) return reject(e);
        await new Promise((resolve) => {
          return setTimeout(resolve, delayMs);
        });
        delayMs *= 2;
        if (delayMs > maxDelayMs) delayMs = maxDelayMs;
      }
    }
  });
}

```

Отслеживаем количество повторов (итераций цикла), инкрементируя значение и отвергая промис при превышении максимума

Отслеживаем количество повторов (итераций цикла), инкрементируя значение и отвергая промис в случае его превышения

Если очередной интервал превышает максимум, изменяем его на максимум

Установив эти лимиты, мы гарантировали, что функция в конечном счете будет завершаться неудачно, как бы сообщая о том, что после многих попыток не удалось получить успешный ответ. Теперь нам осталось рассмотреть еще одну тему: столпотворение запросов.

Столпотворение запросов

Столпотворение запросов (stampeding herds) подразумевает ситуацию, в которой множество удаленных клиентов одновременно совершают один запрос, перегружая получающую систему. Если коротко, то система вполне может обработать все эти запросы по отдельности, но определенно не одновременно, поскольку перегружается вследствие невероятной конкурентности. Но как это связано с экспоненциальной выдержкой?

Чтобы понять связь, мы рассмотрим случай, в котором 100 клиентов одновременно отправляют запрос, образуя так называемое столпотворение. Когда каждый из этих клиентов увидит, что его запрос вернул ошибку, он тут же запустит механизм повторов в надежде на то, что последующий запрос будет успешным. Но проблема в том, что все клиенты будут делать это согласно одному алгоритму. В результате каждый из этих 100 запросов продолжит поступать на сервер в одно время, просто интервалы между ними будут все увеличиваться. Иными словами, поскольку клиенты оказались в ситуации, когда по случайности синхронизировались, их запросы продолжают мешать друг другу и в итоге ни один из них не завершится успехом. По сути, к этому общему сбою ведет детерминистская природа алгоритма.

Что же с этим можно сделать? Простым решением будет вместо слепого следования алгоритму ввести что-то, что разделит запросы друг от друга. Для этого мы задействуем принцип *случайного джиттера* в тайминге задержки между запросами. То есть нам нужно лишь добавить случайное количество времени к интервалам задержки, которые задает алгоритм (листинг 29.3).

Листинг 29.3. Добавление джиттера во избежание столпотворения запросов

```
async function getChatRoomWithRetries(
  id: string, maxDelayMs = 32000, maxRetries = 10): Promise<ChatRoom> {
  return new Promise<ChatRoom>(async (resolve, reject) => {
    let retryCount = 0;
    let delayMs = 1000;
    while (true) {
      try {
        return resolve(GetChatRoom({ id }));
      } catch (e) {
        if (retryCount++ > maxRetries) return reject(e);
        await new Promise((resolve) => {
          return setTimeout(resolve,
            delayMs + (Math.random() * 1000));
        });
        delayMs *= 2;
        if (delayMs > maxDelayMs) delayMs = maxDelayMs;
      }
    }
  });
}
```

Добавляем до 1000 мс джиттера
во избежание столпотворения запросов

Обратите внимание: случайный джиттер здесь не является аддитивным. То есть он не включается в задержку, которую алгоритм постоянно удваивает.

Теперь, когда мы поняли принцип работы экспоненциальной выдержки, пора перейти к изучению сценариев, в которых сервер API может знать, сколько нужно ждать клиенту.

29.3.3. Retry-After

Экспоненциальная выдержка (с улучшениями, описанными в подразделе 29.3.1) — отличный вариант, позволяющий определить время повтора конкретного запроса, но мы предпочитаем этот алгоритм в целом потому, что имеем предположение об отсутствии дополнительной информации, которая могла бы помочь принять более грамотное решение. Однако в некоторых случаях нам известно, что это не так. Наиболее распространенным примером подобного сценария будет ошибка, вызванная ограничением интервала запросов, когда клиент отправляет запросы слишком часто, и у нас есть возможность определить, спустя какое время очередной из них будет допустимым. Иными словами, поскольку сервер API знает, когда именно будет сброшено установленное сервисом ограничение, он также может сообщить, когда следует повторить запрос, не нарушая это ограничение.

Как же лучше всего сообщить клиенту, что ему нужно пропустить экспоненциальную выдержку и вместо нее действовать согласно инструкциям? Конкретно для этого случая существует стандарт Инженерного совета интернета (Internet Engineering Task Force, IETF), определяющий семантику и спецификацию содержимого HTTP: RFC-7231 (<https://tools.ietf.org/html/rfc7231>). Этот RFC устанавливает HTTP-заголовок `Retry-After` (повторить через) (см. подраздел 7.1.3), который указывает, сколько клиент «должен ждать, прежде чем повторять запрос». Если коротко, то этот HTTP-заголовок — идеальное место, в котором сервер API может сообщить более конкретные инструкции о том, когда следует повторить запрос.

Но здесь возникает очередной вопрос: в каком формате должны быть представлены эти инструкции в HTTP-заголовке?

Формат

В спецификации явно указывается, что заголовок `Retry-After` может содержать либо значение даты (например, `Fri, 31 Dec 1999 23:59:59 GMT`), либо интервал в секундах, спустя который можно будет повторить запрос (например, `120`). Несмотря на тот факт, что эта спецификация (и большинство современных

HTTP-серверов) будет поддерживать оба формата, практически всегда лучше указывать продолжительность интервала, а не временную метку.

И на то есть коварная причина. Если мы положимся на продолжительность, то она будет генерироваться сервером, после чего отправляться по сети обратно клиенту. Это означает, что в целом клиент прождет несколько дольше, чем будет указано в заголовке. Например, если сервер передаст в нем значение **120** и на доставку этого ответа уйдет 100 мс, то общее время ожидания клиента составит 120,1 секунды.

Сравните это с куда более пугающим сценарием, подразумевающим *синхронизацию часов*. Если сервер сообщит клиенту определенное время, после которого тот может повторить запрос, то предполагается, что часы на сервере и клиенте должны быть согласованы. В результате при таком подходе возникает уже не только нюанс с дополнительной задержкой, как при использовании формата продолжительности (**120**), но и потенциальная проблема из-за возможного отсутствия синхронизации часов на клиенте и сервере. Представьте, что на сервере сейчас полдень, а на клиенте — 13:00. И если сервер сообщит клиенту «Не повторять запрос до 13:00», то клиент не станет ждать предполагаемые сервером 60 минут, а повторит запрос сразу.

И хотя все это может казаться преувеличением, в действительности часы неоднократно становились слабым местом многих систем. Учитывая секунды координации, сложность временных зон и проблему с синхронизацией часов, предположение о том, что время в двух системах синхронизировано (и тем более намерение положиться на этот факт, чтобы реализовать нужное поведение), редко будет удачным. Безопаснее будет всегда по возможности использовать формат продолжительности, а не конкретного значения времени.

Теперь, когда мы выяснили, какое значение должно помещаться в заголовок `Retry-After`, можно взглянуть, как легко реализуется необходимая для этого поддержка (листинг 29.4).

Листинг 29.4. Добавление поддержки для передачи информации о продолжительности ожидания

```
async function getChatRoomWithRetries(
  id: string, maxDelayMs = 32000, maxRetries = 10): Promise<ChatRoom> {
  return new Promise<ChatRoom>(async (resolve, reject) => {
    let retryCount = 0;
    let delayMs = 1000;
    while (true) {
      try {
        return resolve(GetChatRoom({ id }));
      } catch (e) {
        if (retryCount++ > maxRetries) return reject(e);
      }
    }
  });
}
```

```

await new Promise((resolve) => {
  let actualDelayMs;
  if ('Retry-After' in e.response.headers) {
    actualDelayMs = Number(
      e.response.headers['Retry-After']) * 1000;
  } else {
    actualDelayMs = delayMs + (Math.random() * 1000);
  }
  return setTimeout(resolve, actualDelayMs);
});
delayMs *= 2;
if (delayMs > maxDelayMs) delayMs = maxDelayMs;
}
}
});
}

```

Если значение Retry-After представлено, то мы используем его в качестве интервала

В противном случае задействуется стандартный алгоритм экспоненциальной выдержки

29.3.4. Итоговое определение API

Сведем все воедино. В листинге 29.5 показан пример оптимального извлечения клиентами ресурса, включая повторы с экспоненциальной выдержкой и принятие к сведению возможного ограничения интервала между запросами.

Листинг 29.5. Итоговое определение API

```

async function getChatRoomWithRetries(
  id: string, maxDelayMs = 32000, maxRetries = 10): Promise<ChatRoom> {
  return new Promise<ChatRoom>(async (resolve, reject) => {
    let retryCount = 0;
    let delayMs = 1000;
    while (true) {
      try {
        return resolve(GetChatRoom({ id }));
      } catch (e) {
        if (retryCount++ > maxRetries) return reject(e);
        await new Promise((resolve) => {
          let actualDelayMs;
          if ('Retry-After' in e.response.headers) {
            actualDelayMs = Number(
              e.response.headers['Retry-After']) * 1000;
          } else {
            actualDelayMs = delayMs + (Math.random() * 1000);
          }
          return setTimeout(resolve, actualDelayMs);
        });
        delayMs *= 2;
        if (delayMs > maxDelayMs) delayMs = maxDelayMs;
      }
    }
  });
}

```

29.4. КОМПРОМИССЫ

Недостатком данного паттерна проектирования является то, что он опирается на клиентов, действующих по инструкции. Это может быть инструкция, содержащая указание использовать экспоненциальную выдержку или повторять запрос определенное количество раз, но сутью остается то, что решение в конечном итоге принимает именно клиент, который находится вне нашего контроля.

Положительная же особенность паттерна в том, что при его использовании мы ничем не жертвуем. Экспоненциальная выдержка — общепринятый стандарт, который уже многие годы весьма успешно применяется по всему миру. При этом конкретные инструкции по ожиданию повторения запроса мы передаем, только когда у сервера есть некая дополнительная информация, следование которой может привести к меньшему количеству повторов. Иными словами, этот паттерн в целом лишен недостатков.

29.5. УПРАЖНЕНИЯ

1. Почему нет простого правила, позволяющего принять решение о том, какие неудачные запросы можно безопасно повторить?
2. Какая внутренняя причина подталкивает нас использовать экспоненциальную выдержку? Какова цель применения случайного джиттера между повторами?
3. Когда имеет смысл задействовать заголовок `Retry-After`?

РЕЗЮМЕ

- В случае ошибок, которые являются краткосрочными или привязаны ко времени (например, `429 Too Many Requests`), запрос, скорее всего, можно повторить. Если же ошибка связана с неким устойчивым состоянием (например, `403 Forbidden`), то в большинстве случаев повтор окажется небезопасным.
- Если код автоматически повторяет запросы, то должен полагаться на некую форму экспоненциальной выдержки с установленными лимитами для количества повторов и задержки между ними. В идеале он также должен вводить определенный джиттер, позволяющий избежать проблемы столпотворения, когда все запросы повторяются согласно одинаковым правилам, в связи с чем всегда прибывают на сервер в одно время.
- Если сервису API известна какая-либо информация о том, когда запрос должен завершиться успешно, то он должен сообщить об этом клиенту с помощью заголовка `Retry-After`.

30

Аутентификация запросов

В этой главе

- ✓ Требования к системе аутентификации запросов.
- ✓ Обзор цифровых подписей.
- ✓ Генерация учетных данных, регистрация и подпись.
- ✓ Создание цифровых отпечатков HTTP-запросов.
- ✓ Передача деталей подписи.
- ✓ Подтверждение подписей и аутентификация HTTP-запросов.

В текущем паттерне мы узнаем, как и почему обмен публичными/закрытыми ключами, а также цифровыми подписями (https://en.wikipedia.org/wiki/Digital_signature) используется для аутентификации всех входящих запросов API. Применение этих техник гарантирует подлинность источника и целостность всех входящих запросов, а также не позволяет отправителю впоследствии от них отказаться. И хотя во многих случаях будут приемлемы альтернативные решения (например, обмен секретными данными и HMAC; <https://ru.wikipedia.org/wiki/HMAC>), они не подойдут для сценариев с привлечением сторонних участников, когда требуется невозможность отказа.

30.1. ЗАЧЕМ ЭТО НУЖНО

До сих пор мы предполагали, что все запросы API гарантированно являются подлинными, откладывая нюансы безопасности на потом. Теперь же пришло время разобрать фундаментальный вопрос: как определить, что входящий запрос принадлежит реально авторизованному пользователю?

В конечном итоге решение о признании конкретного запроса сводится к ответу «да» или «нет», но чтобы его принять, необходимо учесть ряд требований. Начнем с того, что нам нужно знать, поступил ли запрос от того пользователя, от которого он заявлен. То есть если запрос утверждает, что его отправил пользователь 1234, то нам нужна возможность подтвердить, что он действительно поступил от него. Кроме того, нам нужно убедиться, что содержимое запроса не было повреждено или изменено. Наконец, позднее может возникнуть потребность в том, чтобы незаинтересованная третья сторона могла подтвердить, что запрос неопровержимо поступил от конкретного пользователя. Иными словами, если запрос утверждает, что его отправил пользователь 1234 (и это можно проверить), то данный пользователь не должен иметь возможность впоследствии заявить, что запрос был кем-то подделан. Далее мы разберем каждый из этих аспектов, начиная с источника запроса.

30.1.1. Источник

Первая и наиболее важная возможность, которая нас интересует, — определение источника запроса. В данном случае речь идет не о его географическом происхождении, а о пользователе, отправившем этот запрос, что приводит нас к концепции подлинности.

В отличие от понятия подлинности личности из реального мира, нас не интересуют водительские права или свидетельства о рождении. В данном случае, если запрос утверждает, что он поступил от пользователя 1234, то должно иметься некое подтверждение, которое может быть предоставлено только тем, кто зарегистрирован в нашем сервисе как пользователь 1234. Реализовать это можно многими способами, которые будут рассмотрены в разделе 30.2. Пока же перейдем к изучению понятия целостности.

30.1.2. Целостность

Целостность подразумевает уверенность в том, что полученное содержимое запроса в точности соответствует переданному и является критически важным элементом системы безопасности API. Без нее даже при подтверждении

подлинности источника запроса (например, когда запрос, обозначивший своим источником пользователя 1234, содержит учетные данные пользователя 1234) мы не можем быть уверенными в том, что содержимое запроса не изменялось после отправки. А если мы не можем быть уверенными, что запрос представлен в своем изначальном виде, то и подтверждение подлинности его источника не так уж полезно.

Хотя это редкость, но все же запросы могут изменяться во время передачи. И спектр возможных причин простирается от непреднамеренных ошибок (например, передаваемые данные пострадали из-за сбоя в инфраструктуре сети) до умышленных (например, кто-то перехватил запрос и намеренно его изменил), но суть одна: нам необходима возможность убедиться в неизменности содержимого запроса с момента его отправки и отвергать те, которые претерпели изменения при передаче.

Кроме того, хоть мы и можем зачастую положиться на безопасность самой сети (например, TLS), это будет означать, что нам придется беспрекословно принимать входящие запросы, не проверяя их, и предполагать, что запрос был передан правильно. Безусловно, было бы отлично иметь возможность просто предположить, что при обеспечении сетевой безопасности все и всегда все делают правильно, но в действительности ввиду остальных наших требований этот аспект безопасности возникает попутно сам по себе.

30.1.3. Невозможность отказа

Невозможность отказа подразумевает, что после подтверждения источника запроса этот источник не может отрицать, что отправил его именно он. Здесь можно подумать: разве это проблема? Если мы можем верифицировать источник, то это уже подразумевается. На самом деле почти так и происходит, но есть один нюанс.

Реальная проблема в том, что очень легко спроектировать симметричную учетную систему, в которой учетные данные, необходимые для подтверждения подлинности пользователя 1234, будут совпадать с теми, которые требуются для подтверждения того, что запрос поступил от этого пользователя. В некотором смысле это сродни обмену секретными данными между двумя сторонами. Эта идея симметричности учетных данных до сих пор не была проблемой, поскольку действовало допущение, что запросы API никогда не должны поступать от сервера API. Но что происходит, когда подтвердить, что подлинным источником запроса является пользователь 1234, должен не сам этот пользователь и не сервер API, а кто-то другой? Как эта третья сторона может отличить запросы истинного пользователя 1234 от подделанных сервером API, злоупотребившим общими учетными данными, которые предназначались только для проверки пользователя 1234?

Это требование довольно ясно говорит о том, что механизм, с помощью которого пользователь подтверждает подлинность своего запроса, не должен использоваться совместно с сервером API. Иными словами, учетные данные, применяемые для подтверждения и верификации источника запроса API, должны быть асимметричными, то есть разными для каждой стороны. Таким образом мы гарантируем, что пользователь является единственной стороной, имеющей доступ к учетным данным, и третья сторона при рассмотрении запроса от этого пользователя может быть уверена, что запрос поступил именно от него, а не был подделан сервером API.

30.2. ОБЗОР

Реализовать вышесказанное можно несколькими способами, но один из них выделяется: цифровые подписи. Мы подробно разберем принцип их работы в следующем разделе, но важно понять их основную суть и почему они хорошо подходят под описанные выше требования.

В своей основе цифровая подпись — просто набор байтов. В свою очередь, эти байты представляют значение, которое можно получить только путем генерации и проверки с использованием пары учетных данных. Это означает, что подпись, сгенерированную с помощью определенных учетных данных, невозможно воссоздать, не обладая этими данными.

Тем не менее наиболее важное отличительное свойство этих криптографических подписей связано с асимметрией. Она означает, что учетные данные, используемые для генерации подписи, отличаются от тех, которые используются для ее верификации. Иными словами, каждая из этих двух версий учетных данных служит своей цели: одна задействуется при генерации подписи, а вторая — при ее проверке.

Когда дело доходит до аутентификации запросов API, цифровые подписи отлично подходят под все требования. Для начала, их криптографическая природа позволяет легко подтвердить, что подпись была сгенерирована именно тем, кто обладает верными учетными данными. Далее, тот факт, что подпись зависит от подписываемого ей значения, означает, что она окажется недействительной, если сообщение изменится любым образом. Наконец, поскольку генерировать цифровые подписи может только одна сторона, она никак не сможет позднее заявить, что подпись была подделана. Получается, этот механизм полностью соответствует всем трем критериям, перечисленным в разделе 30.1.

Но как именно работают цифровые подписи? И как можно интегрировать их в веб-API? Процесс использования этих подписей для аутентификации запросов состоит из трех стадий. Сначала пользователь создает пару «открытый/закрытый

ключ». Затем мы даем ему возможность зарегистрироваться с помощью сервиса API, устанавливая свою подлинность в системе на основе открытого ключа. С этого момента пользователь может просто сопровождать все запросы своей цифровой подписью, используя закрытый ключ. При этом сервер API сможет верифицировать эти подписи, используя ранее связанный с пользователем открытый ключ.

Если это звучит слишком упрощенно, то не беспокойтесь. В следующем разделе мы разберем описанный процесс более детально.

30.3. РЕАЛИЗАЦИЯ

Как было сказано в разделе 30.2, использование цифровых подписей для аутентификации запросов представляет собой многоэтапный процесс с участием как пользователя, так и сервера API. Изучение этого процесса мы начнем с основы всей системы: генерации подходящих асимметричных учетных данных.

30.3.1. Генерация учетных данных

Прежде чем пользователь сможет что-либо сделать, ему потребуется сгенерировать пару ключей. Генерация представляет особый тип учетных данных, состоящий из двух частей: закрытого ключа и открытого. Крайне важно, чтобы закрытый хранился в совершенном секрете, поскольку, как мы узнали ранее, обладание верными учетными данными обеспечивает подтверждение подлинности. Иными словами, владение верными учетными данными является достаточным доказательством, и на этих данных нет фотографии, чтобы дополнительно сравнить ее с лицом пользователя.

Но почему именно пользователь должен отвечать за генерацию пары ключей? Почему это не может сделать сервер и затем просто передать пользователю закрытый ключ, чтобы тот мог впоследствии применять его? Как мы узнали из требований к невозможности отказа, вся суть использования асимметричных учетных данных (а не общих секретных данных) заключается в том, чтобы не дать пользователю возможность отрицать собственную подпись на запросе. И этот принцип работает, поскольку учетные данные, используемые для подписи запросов, находятся только в распоряжении пользователя и никогда не сообщаются серверу API. Если же поручить их генерацию серверу, то у нас не будет никакой гарантии, кроме его честного слова, что он не сохранил копию секретного ключа, чтобы использовать в собственных злоумышленных интересах. Кроме того, нам нужно учитывать передачу закрытого ключа через интернет, в связи с чем возникает пусть небольшой, но реальный риск его перехвата. В результате, если мы

поистине хотим соблюсти требование по невозможности отказа, то закрытый ключ никогда не должен покидать рук пользователя.

Сгенерировать данный ключ можно с помощью многих инструментов, таких как `ssh-keygen` в большинстве систем на базе Linux, но в приведенном ниже TypeScript-коде (листинг 30.1) показано, как это можно сделать программно.

Листинг 30.1. Генерация учетных данных

```
const crypto = require('crypto');

interface KeyPair {
  publicKey: string;
  privateKey: string;
}

function generateCredentials(size = 2048): Promise<KeyPair> {
  return new Promise<KeyPair>((resolve, reject) => {
    crypto.generateKeyPair('rsa', { modulusLength: size },
      (err, pub, priv) => {
        if (err) { return reject(err); } ← В случае любой ошибки промис просто отвергается
        return resolve({
          publicKey: pub.export({ type: 'pkcs1', format: 'pem' }), ←
          privateKey: priv.export({ type: 'pkcs1', format: 'pem' })
        });
      });
  });
}
```

В качестве значений открытого и закрытого ключей строки используются сериализованные данные

После выполнения такого кода клиент получит открытый ключ, сериализуемый в строку, которую можно отправлять другим. Теперь нужно как-то передать этот открытый ключ серверу API для нашей идентификации.

30.3.2. Регистрация и обмен учетными данными

Как мы узнали в подразделе 30.1.1, подтверждение личности пользователя в целом является важным принципом, но когда дело доходит до аутентификации запросов, его конкретная интерпретация несколько отличается от того, что мы привыкли подразумевать под ней в повседневной жизни. В частности, идентификация пользователя в данном случае не является абсолютной. То есть сервер API никогда не будет верифицировать какие-то врожденные черты пользователя (например, отпечатки пальцев), принимая за его личность любого, кто обладает учетными данными, установившими эту личность изначально. Проще говоря, чтобы доказать, что вы являетесь некой личностью, достаточно подтвердить, что вы обладаете секретными данными, доступными этой личности.

В результате принцип регистрации упрощается. В данном случае можно позволить пользователям регистрироваться с помощью API, предоставляя учетные

данные в виде открытого ключа, на основе которого сервис будет генерировать для каждого уникальный идентификатор. Последовательность этих событий показана на рис. 30.1.

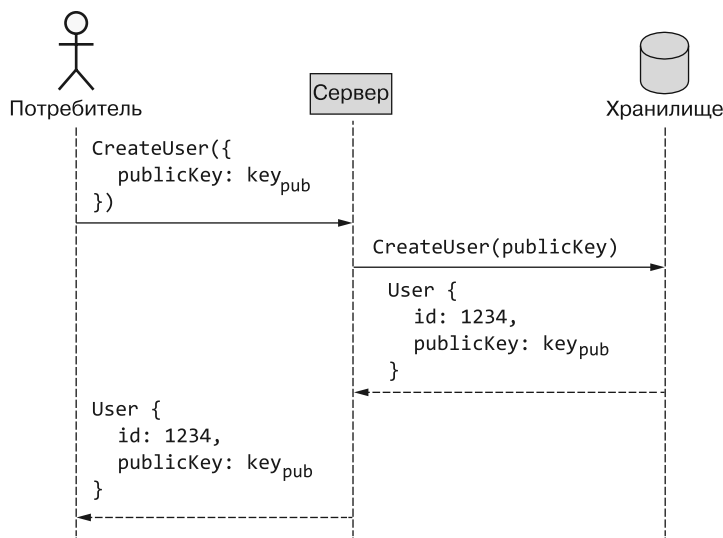


Рис. 30.1. Последовательность регистрации учетной записи с помощью открытого ключа

С помощью этого простого стандартного метода `Create` мы совершили один вызов API, получив уникальный идентификатор для использования в последующих запросах (например, `user 1234`). При этом закрытый ключ можно будет применять для верификации подписей входящих запросов до их принятия. Обратите внимание: это избавляет нас еще и от требования аутентифицироваться в сервисе более традиционным способом с помощью имени пользователя и пароля. Вместо этого данная схема изначально полностью полагается на цифровые подписи.

Теперь, когда мы разобрались с генерацией учетных данных и узнали, как безопасно обмениваться ими с сервером API для определения общего представления о нашей подлинности, нужно выяснить, как именно следует использовать эти ключи.

30.3.3. Генерация и верификация сырых подписей

Если не вдаваться в криптографию и теорию чисел, то цифровая подпись представляет собой просто особое число, которое можно легко вычислить с помощью закрытого ключа и практически невозможно без него. Аналогичным образом

это особое число можно легко и быстро проверить, используя открытый ключ, но сгенерировать то же значение подписи с помощью этого ключа уже не получится. В некотором смысле эта схема сродни закрытому стеклянному ящику: положить что-то внутрь может только обладатель ключа, после чего любой сможет увидеть этот предмет, понимая, что тот был помещен в ящик именно обладателем ключа.

Чтобы вычислить подпись, мы совмещаем тело запроса с закрытым ключом, а всю сопутствующую математическую нагрузку поручаем стандартной библиотеке наподобие предоставляемого Node.js пакета `crypto` (листинг 30.2).

Листинг 30.2. Подписывание произвольной строки

```
const crypto = require('crypto');

function generateSignature(payload: string, privateKey: string): string {
  const signer = crypto.createSign('rsa-sha256');
  signer.update(payload);
  signer.end();
  return signer.sign(privateKey);
}
```

Верификация подписи тоже не вызывает сложностей. Мы берем подпись, полезную нагрузку, подписанную ею, и соответствующий открытый ключ, совмещая все это с помощью функции стандартной библиотеки. Результатом этой функции будет простое логическое значение, сообщающее, прошла ли подпись проверку (листинг 30.3).

Листинг 30.3. Верификация произвольной подписи

```
const crypto = require('crypto');

function verifySignature(
  payload: string, signature: string, publicKey: string): Boolean {
  const verifier = crypto.createVerify('rsa-sha256');
  verifier.update(payload);
  verifier.end();
  return verifier.verify(publicKey, signature);
}
```

Удивительным в обоих этих примерах может показаться тот факт, что параметр `payload` является строкой, а не, например, объектом или иным представлением запроса API. Но в результате перед нами возникает важный вопрос: что именно нужно подписывать?

В действительности он оказывается куда сложнее, чем кажется. В следующем подразделе мы поговорим о том, как определять, что помещать в параметр `payload` при генерации или верификации цифровой подписи.

30.3.4. Цифровой отпечаток для запроса

До сих пор мы говорили лишь о том, что нужно подписать запрос, но фактически никак не конкретизировали этот процесс. Далее мы разберем ряд аспектов, которые необходимо учитывать при принятии решения о том, что нужно подписать до отправки запроса серверу API.

Как бы нам ни хотелось просто взять и подписать тело запроса, даже этот процесс в действительности оказывается более сложным, чем кажется, поскольку тело запроса требует от нас сериализации данных (например, с помощью `JSON.stringify()` в TypeScript). Кроме того, даже при использовании одного формата существует множество способов сериализации. Например, есть различные кодировки символов и формы нормализации, ведущие к получению множества строк, которые могут быть семантически равнозначными, но при этом иметь разные байтовые представления.

Более того, в случае большинства запросов, включая JSON, обычно нет правил о том, в каком порядке свойства должны появляться в различных структурах, подобных картам. К примеру, несмотря на то что `{"a": 1, "b": 2}` семантически равнозначен `{"b": 2, "a": 1}`, байтовое представление этих двух значений будет совершенно разным.

Поскольку в вычислении цифровой подписи задействованы сырые, непрозрачные байты, это означает, что низкоуровневое байтовое представление содержимого весьма значимо. Если, например, верифицирующая подпись сторона использует разное байтовое представление одного и того же семантического сообщения, то это приведет к сбою проверки подписи, когда в действительности содержимое не менялось. Вместо этого байтовое представление просто вычисляется иначе.

Все усложняется необходимостью помнить, что запрос API может включать намного больше, чем просто тело запроса. Например, при удалении ресурса с помощью HTTP и механизма транспорта тело HTTP-запроса оказывается пустым (а не чем-то вроде `{"id": "chatRooms/1"}`). В данном случае предполагаемое действие кодируется в HTTP-глагол (`DELETE`), и цель для удаления представляется в URL (`/chatRooms/1`). Кроме того, в HTTP-заголовках могут присутствовать и другие элементы информации, которые важны для смысла и целостности запроса (например, время его отправки) и должны быть включены в процесс вычисления подписи. Каждый из этих факторов повышает важность присутствия способа вычисления цифрового отпечатка, который сможет уникально представлять конкретный запрос и в конечном итоге выступать в качестве полезной нагрузки при вычислении цифровой подписи с помощью закрытого ключа.

Но как определить этот цифровой отпечаток? Чтобы разобраться в этом, нам потребуется рассмотреть, какие элементы HTTP-запроса должны быть включены

в подпись. К счастью, ответ здесь довольно прост. Сначала нам нужно включить HTTP-метод, URL (хоста и путь), а также тело запроса. Кроме того, если этот параметр присутствует, необходимо указать, в какое время был отправлен запрос. Чаще всего эта информация указывается в HTTP-заголовке Date, чтобы поддержать отклонение слишком старых запросов (в общем виде все эти поля представлены в табл. 30.1).

Таблица 30.1. Компоненты, необходимые для создания цифрового отпечатка HTTP-запроса

Поле	Расположение	Описание
Method	Первая строка	Действие запроса
Path	Первая строка	Целевой ресурс запроса
Host	HTTP-заголовок	Целевой хост запроса
Content	HTTP-тело	Полезная нагрузка запроса
Date	HTTP-заголовок	Время создания запроса

Как видите, данные поступают из разных мест, что несколько усложняет работу. Упростить ее можно двумя способами. Во-первых, можно создать первую строку HTTP-запроса как нечто особенное, зачастую называемое *целью запроса*, включив в нее и HTTP-метод (например, POST), и путь (например, /chatRooms/1). Во-вторых, вместо того чтобы подписывать тело запроса, которое может стать чрезмерно большим, можно вычислить его хеш и сохранить это значение в отдельном HTTP-заголовке (в формате Base64) под названием Digest (листинг 30.4). Задействовав это производное поле, мы сохраним целостность и безопасность отпечатка, одновременно сократив его размер.

Листинг 30.4. Генерация хеша для заголовка Digest

```
const crypto = require('crypto');

function generateDigestHeader(body: string): string {
  return 'SHA-256=' + crypto
    .createHash('sha256')
    .update(body)
    .digest()
    .toString('base64');
}
```

← Перед значением хеша указывается использованный для его вычисления алгоритм (в данном случае SHA-256)

← Хеш должен быть закодирован в Base64

Затем нужно собрать все вместе. Для этого можно использовать упорядоченный список компонентов (например, request target, host, date, digest), должным образом их отформатировать (скажем, преобразовать заголовки в символы нижнего регистра) и поместить каждый в строку, разделив символами новой строки (листинги 30.5, 30.6).

Листинг 30.5. Генерация отпечатка на основе HTTP-запроса

```
const HEADERS = ['(request-target)', 'host', 'date', 'digest'];

function generateRequestFingerprint(
  request: HttpRequest, headers = HEADERS): string {
  return headers.map((header) => {
    let value;
    if (header === '(request-target)') {
      value = `${request.method.toLowerCase()} ${request.path}`;
    } else if (header === 'digest') {
      value = generateDigestHeader(request.body);
    } else {
      value = request.headers[header];
    }
    return `${header.toLowerCase()}: ${value}`;
  }).join('\n');
}
```

← Поле (request-target) должно включать HTTP-метод и путь в нижнем регистре
 ← Поле digest должно генерировать хеш тела запроса
 ← Все остальные значения должны выглядеть так же, как в заголовке запроса
 ← В завершение возвращается набор пар «ключ — значение», разделенных двоеточием (аналогично заголовкам)

Листинг 30.6. Пример отпечатка, используемый для подписывания HTTP-запросов

```
(request-target): patch /chatRooms/1
host: example.org
date: Tue, 27 Oct 2020 20:51:35 GMT
digest: SHA-256=HV9PltG0QPRNs1lFB7ebQA8XPasvPyRg6hhU0QF2l4M=
```

← Особое поле (request-target) рассматривается как любой другой заголовок
 ← Мы также включаем дополнительный заголовок digest

Такого эталонного формата в виде пар «ключ — значение» вполне достаточно в качестве отпечатка запроса, и, что самое главное, его можно будет в конечном итоге использовать как полезную нагрузку для вычисления цифровой подписи. Он содержит всю информацию о целевом ресурсе запроса (`/chat-Rooms/1`), действии (`PATCH`), месте назначения (`example.org`), времени (`Oct.27`) и теле (в форме хеша `SHA-256`), позволяющую проводить уникальную идентификацию запроса в дальнейшем.

Теперь нам нужно найти способ, позволяющий проинформировать сервер API о том, как именно мы вычислили данный отпечаток. В следующем подразделе мы узнаем, как вычислять эти параметры с помощью сервера, чтобы он мог получать для запроса точно такой же отпечаток.

30.3.5. Добавление подписи

Вычислив отпечаток запроса, мы не можем просто его подписать и на этом закончить. Чтобы сервер API мог верифицировать подпись, он должен иметь возможность вычислить такое же значение отпечатка на основе входящего запроса. И для этого необходимо включить ряд элементов метаданных об этом отпечатке и подписи.

Сначала нужно выяснить, какие именно заголовки были включены в отпечаток и (что очень важно) в каком порядке. Наличия одного только списка заголовков

будет недостаточно, так как их представление в другом порядке приведет к иному отпечатку. Далее нам нужно как-то сообщить серверу API, какой открытый ключ должен использоваться для верификации подписи. Этот ID может быть эквивалентен пользовательскому ID, созданному при регистрации, поскольку у каждого пользователя есть ровно один открытый ключ. Наконец, нам также нужно включить алгоритм, использованный для генерации подписи, чтобы сервер API мог верифицировать ее с помощью такого же алгоритма. Все эти поля перечислены в табл. 30.2.

Таблица 30.2. Компоненты, необходимые для определения подписи

Поле	Описание
headers	Упорядоченный список заголовков, использованных для генерации отпечатка запроса
keyId	Уникальный ID, использованный при выборе открытого ключа для верификации
signature	Содержимое цифровой подписи для проверки
algorithm	Алгоритм, использованный для генерации цифровой подписи

Как только у нас будет вся эта информация, можно будет собрать ее и передать сгенерированное значение вместе с запросом в особом заголовке `signature` (листинг 30.7).

Листинг 30.7. Генерация заголовка `signature` на основе отпечатка и прочих компонентов

```
const HEADERS = ['(request-target)', 'host', 'date', 'digest'];

interface SignatureMetadata {
  keyId: string;
  algorithm: string;
  headers: string;
  signature: string;
}

function generateSignatureHeader(
  fingerprint: string, userId: string,
  privateKey: string, headers = HEADERS): string {
  const signatureParts: SignatureMetadata = {
    keyId: userId,
    algorithm: 'rsa-sha256',
    headers: headers.map((h) => h.toLowerCase()).join(' '),
    signature: generateSignature(fingerprint, privateKey)
  };

  return Object.entries(signatureParts).map(([k, v]) => {
    return `${k}="${v}"`;
  }).join(',');
}
```

Указываем список компонентов подписи

Совмещаем все части в пары «ключ — значение», разделенные запятой и обособленные кавычками

На выходе этой функции должна получиться строка разделенных запятой значений каждого из полей, необходимых для правильного определения подписи (листинг 30.8).

Листинг 30.8. Пример значения для заголовка `signature`

```
keyId="1234",algorithm="rsa-sha256",headers="(request-target)
➔ host date digest",signature="mgBAQEsEsoBCgIOBiNfum37y..."
```

Теперь, когда у нас есть все элементы, осталось лишь собрать все вместе. Это означает, что нам нужно взять HTTP-запрос и добавить к нему два новых заголовка: `digest` и `signature`. Всю основную работу можно поручить определенным ранее вспомогательным методам (листинг 30.9).

Листинг 30.9. Функция для подписывания запроса (присваивающая нужные заголовки)

```
const HEADERS = ['(request-target)', 'host', 'date', 'digest'];

function signRequest(
  request: HttpRequest, userId: string,
  privateKey: string, headers = HEADERS): HttpRequest {
  request.headers['digest'] =
    generateDigestHeader(request.body); ← Обновляем заголовок digest
  const fingerprint =
    generateRequestFingerprint(request, headers);
  request.headers['signature'] = generateSignatureHeader(
    fingerprint, userId, privateKey, headers); ← Генерируем необходимый
                                                    для подписи отпечаток
                                                    запроса
  return request; ← Возвращаем
                  дополненный запрос
                  Обновляем заголовок
                  signature
```

На выходе этой функции получится запрос, который должен включать и заголовок `digest`, установленный на значение хеша тела запроса, и заголовок `signature`, включающий не только цифровую подпись, но и информацию, с помощью которой сервер API может верифицировать ее значение (листинг 30.10).

Листинг 30.10. Пример полноценного подписанного HTTP-запроса

```
PATCH /chatRooms/1
Host: example.org
Digest: SHA-256=HV9PltG0QPRNs1lFB7ebQA8XPasvPyRg6hhU0QF2l4M=
Signature: keyId="1234",algorithm="rsa-sha256",headers="
            (request-target) host date digest",
            signature="mgBAQEsEsoBCgIOBiNfum37y..."
Date: Tue, 27 Oct 2020 20:51:35 GMT

{"title":"New title"}
```

Обратите внимание: порядок заголовков в фактическом запросе значения не имеет. Дело в том, что мы указали порядок, в котором заголовки должны

использоваться для генерации отпечатка при верификации подписи. Далее нам нужно понять, как именно сервер может верифицировать подпись и в конечном итоге аутентифицировать запрос. Разберем это в следующем подразделе.

30.3.6. Аутентификация запросов

Как вы могли ожидать, для верификации подписи, ведущей к аутентификации запроса, можно задействовать тот же алгоритм. Но что именно нужно проверить?

Сначала нам следует убедиться, что заголовок **digest** является истинным представлением содержимого тела запроса. Если это не так, значит, тело запроса было изменено с момента вычисления хеша, и такой запрос нужно отклонить. Далее необходимо повторно вычислить отпечаток запроса, используя схему, представленную в заголовке **signature**. После этого нужно будет использовать идентификатор ключа, переданный в этом заголовке в целях поиска открытого ключа, который должен использоваться для верификации подписи. В завершение же мы используем этот открытый ключ и отпечаток, чтобы верифицировать подпись.

Это может показаться чрезмерным, но мы уже написали большую часть кода, которая возьмет на себя всю основную работу (листинг 30.11).

Листинг 30.11. Верификация подписи на основе объекта запроса

```
function parseSignatureHeader(signatureHeader: string): SignatureMetadata {
  const data = Object.fromEntries(
    signatureHeader
      .split(',') // Разделяем пары k="v",k="v"
      .map((pair) => pair.split('=')) // Разделяем пару k="v"
      .map(([k, v]) => [k, v.substring(1, v.length-1)])); // Удаляем кавычки
  data.headers = data.headers.split(' '); // Разделяем только заголовки
  return metadata;
}

async function verifyRequestSignature(request: HttpRequest): Promise<Boolean> {
  if (generateDigestHeader(request.body) !== request.headers['digest']) {
    return false;
  }
  const metadata = parseSignatureHeader(request.headers['signature']);
  const fingerprint = generateRequestFingerprint(request, metadata.headers);
  const publicKey = (await database.getUser(metadata.keyId)).publicKey;
  return verifySignature(fingerprint, metadata.signature, publicKey);
}
```

Сначала нам нужна функция для парсинга заголовка **signature** в отдельные компоненты

Проверяем, соответствует ли тело запроса заголовку **digest**

Парсим заголовок **signature** в несколько элементов

Определяем полезную нагрузку, подписанную на основе заголовков

Определяем открытый ключ для представленного ID (хранящегося в базе данных)

Сверяем подпись с отпечатком, используя открытый ключ

Теперь у нас есть возможность генерировать и регистрировать учетные данные с помощью сервера API, сопровождать исходящий запрос цифровой подписью и аутентифицировать входящий запрос пользователя.

30.3.7. Итоговое определение API

Для данного паттерна конечное определение API будет довольно простым, в него будет включен только метод, позволяющий создавать новых пользователей с помощью связанных открытых ключей и в дальнейшем извлекать этих пользователей по их ID (листинг 30.12). Реализации подписывания запросов и верификации подписей были рассмотрены ранее, поэтому здесь мы не будем повторять их код.

Листинг 30.12. Итоговое определение API

```
abstract class ChatRoomApi {
    @post("/users")
    CreateUser(req: CreateUserRequest): User;

    @get("/{id=users/*}")
    GetUser(req: GetUserRequest): User;
}

interface CreateUserRequest {
    resource: User;
}

interface GetUserRequest {
    id: string;
}

interface User {
    id: string;
    publicKey: string;
    // ...
}
```

30.4. КОМПРОМИССЫ

Поскольку существует много различных методов аутентификации, то при выборе одного из них неизбежно возникают компромиссы. И хотя цифровая подпись для запросов наверняка является самым безопасным способом, она подойдет далеко не для всех сценариев. И самый распространенный недостаток подобного метода аутентификации — то, что для некоторых ситуаций он оказывается чрезмерным.

Например, возможны случаи, когда невозможность отказа больше «приятный бонус», нежели необходимое требование. Этот функционал действительно нужен, когда речь идет о возможном предоставлении подписанных запросов другим для последующего выполнения (включая сторонних участников, которые могут не внушать доверия) или когда сами запросы API являются особо чувствительными (например, обновляющими медицинские или финансовые записи). В подобных случаях невозможность отказа нужна серверу API, чтобы защититься от потенциальных обвинений в некорректности (например, пользователь может заявить, что никогда не делал таких запросов), которые могут возникнуть во время будущего анализа журналов записей.

Однако такие случаи довольно редки. В конце концов, обновляя данные на Facebook или Twitter, мы вряд ли после этого планируем отказываться от запросов к API. И это говорит о том, что нашего внимания определенно заслуживают и прочие системы (например, использующие НМАС с общими секретными данными). Здесь мы не будем подробно говорить о НМАС или прочих стандартных механизмах аутентификации, использующих краткосрочные токены доступа (например, OAuth 2.0; <https://oauth.net/2/>). Они определенно являются допустимыми вариантами для любого API, который будет не столь требователен и сможет положиться на подходящие протоколы безопасности, реализованные на транспортном уровне.

Но почему бы просто не задействовать самый безопасный вариант, когда он доступен? Одна из наиболее актуальных причин состоит в том, что подобный тип криптографии требует больших вычислительных ресурсов и в конечном итоге обрабатывается самим сервером API, а не на нижнем уровне в стеке. На самом деле даже при типичном применении криптографии с открытым ключом, такой как SSL, система применяет асимметричный механизм только для обмена временным симметричным ключом, в последующем полагаясь уже на менее затратные симметричные алгоритмы. В результате системы, для которых важна экономия ресурсов, могут склониться к использованию более эффективной альтернативы.

30.5. УПРАЖНЕНИЯ

1. В чем разница между подтверждением источника запроса и предотвращением возможного будущего отказа? Почему первое не может определить второе?
2. Какое требование не выполняется, если между клиентом и сервером используются общие секретные данные?
3. Почему в отпечаток запроса важно включать атрибуты HTTP-метода, хоста и пути?
4. Уязвимы ли цифровые подписи, описанные в данном паттерне, для атак повторением? Если да, то как можно решить эту проблему?

РЕЗЮМЕ

- Успешная аутентификация запроса подразумевает три требования: подтверждение источника, подтверждение целостности и предотвращение возможного отказа.
- Источник — подтверждение того, что запрос без сомнений поступил от известного отправителя.
- Целостность подразумевает доказательство того, что сам запрос не изменялся с момента его отправки известным источником.
- Невозможность отказа означает, что отправитель запроса не может заявить, что конкретный запрос был отправлен кем-то другим.
- В отличие от проверки подлинности личности в реальном мире, аутентификация определяет ее подлинность по обладанию секретным ключом, который выступает своеобразной облигацией на предъявителя.
- При подписывании запроса фактически должен подписываться отпечаток этого запроса, включающий HTTP-метод, путь, хост и набор содержимого в HTTP-теле.

Джей Джей Гивакс

Паттерны проектирования API

Перевел с английского Д. Брайт

Руководитель дивизиона	<i>Ю. Сергиенко</i>
Руководитель проекта	<i>А. Питуримов</i>
Ведущий редактор	<i>Н. Гринчик</i>
Литературный редактор	<i>Н. Хлебина</i>
Художественный редактор	<i>В. Мостипан</i>
Корректоры	<i>О. Андриевич, Н. Терех</i>
Верстка	<i>Г. Блинов</i>

Изготовлено в России. Изготовитель: ООО «Прогресс книга».
Место нахождения и фактический адрес: 194044, Россия, г. Санкт-Петербург,
Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 03.2023. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 — Книги печатные профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 12.01.23. Формат 70×100/16. Бумага офсетная. Усл. п. л. 41,280. Тираж 1000. Заказ 0000.

Мехди Меджуи, Эрик Уайлд, Ронни Митра, Майк Амундсен

НЕПРЕРЫВНОЕ РАЗВИТИЕ API. ПРАВИЛЬНЫЕ РЕШЕНИЯ В ИЗМЕНЧИВОМ ТЕХНОЛОГИЧЕСКОМ ЛАНДШАФТЕ

2-е издание



Для реализации API необходимо провести большую работу, но эти усилия не всегда окупаются. Чрезмерное планирование может стать пустой тратой сил, а его недостаток приводит к катастрофическим последствиям. Во втором издании представлены решения для отдельных API и систем из нескольких API, которые позволят вам распределить необходимые ресурсы и достичь требуемого уровня эффективности за оптимальное время.

Как соблюсти баланс гибкости и производительности, сохранив надежность и простоту настройки? Четыре эксперта по API объясняют разработчикам, руководителям продуктов и проектов, как максимально увеличить ценность их API, управляя интерфейсами как продуктами с непрерывным жизненным циклом.

КУПИТЬ